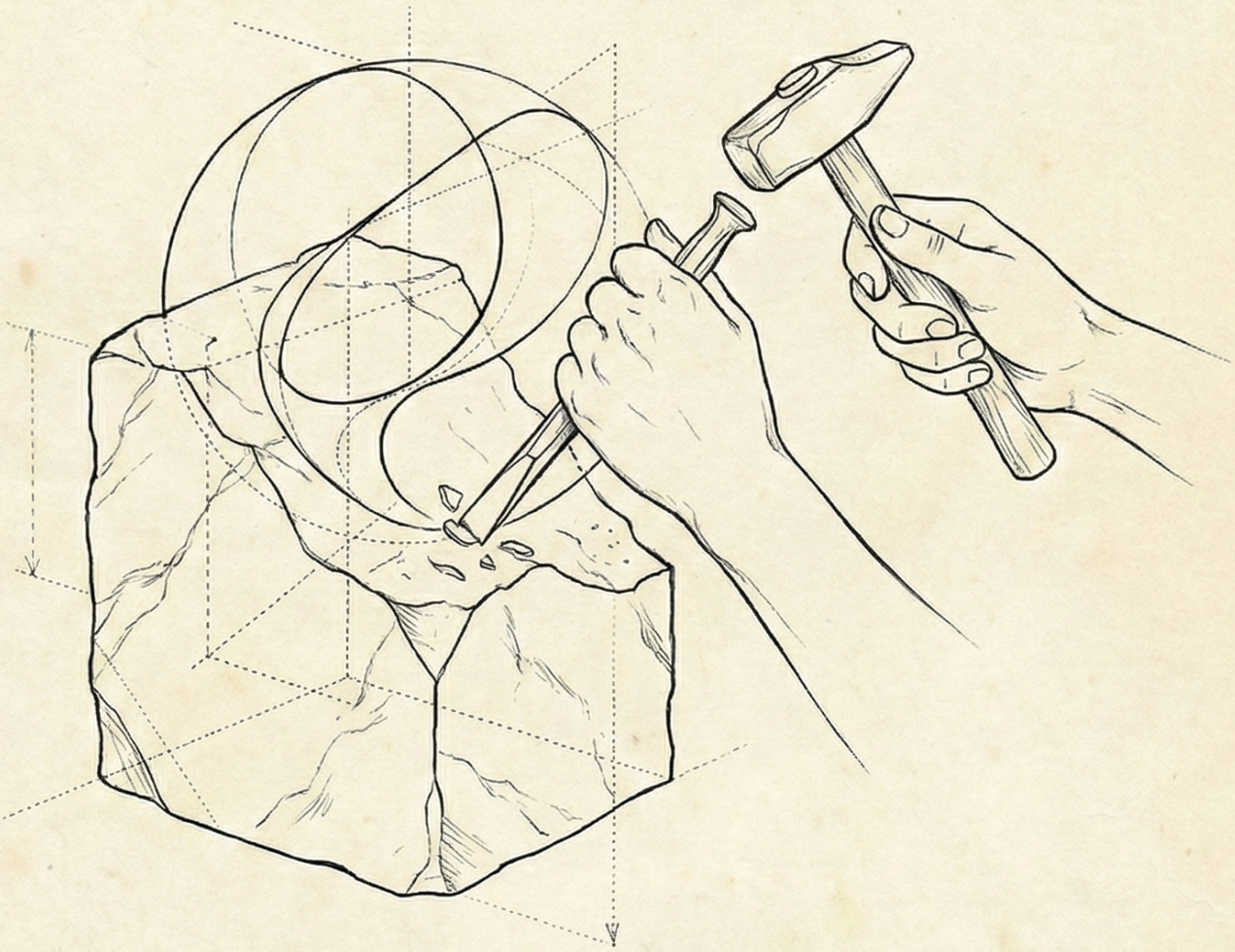
MINIMIZING TRUST

The Architecture of Verifiable Secrets



particle



Minimizing Trust

parti

Contents

Chapter 1: The Trust Problem 12
When Verification Is Easy 13
When Verification Seems As Hard As Computation 14
Interactive Proofs: The Breakthrough 15
Randomness Creates Asymmetry 15
From IP to Succinctness 16
The Proof System Zoo 17
Zero-Knowledge: Proving Without Revealing 23
The Architecture of Modern Proofs 23
Why This Matters 24
The Road Ahead 24
Key takeaways 25
Chapter 2: The Power of Polynomials 26
The Motivating Problem: Beyond NP 26
Why Polynomials? 28
Pillar 1: Representation - From Data to Polynomials 29
Pillar 2: Randomization - The Schwartz-Zippel Lemma 32
Error-Correcting Codes: The Deeper Structure 37
Pillar 3: Compression - From Many Constraints to One 40
Freivald's Algorithm: Polynomials in Disguise 42
Beyond Schwartz-Zippel: Why Polynomials Are Uniquely Suited 45
The Polynomial Paradigm: A Unified View 47
Key takeaways 47
Chapter 3: The Sum-Check Protocol 48
The Problem: Verifying Exponential Sums 49
The Compression Game 50
The Protocol Specification 51
Why Does This Work? 54
Worked Example: Honest Prover and Cheating Prover 55
The Protocol Flow: A Visual Guide 56

Application: Counting Satisfying Assignments (#SAT) . . . 57
Deferred Evaluation . . . 60
Complexity Analysis . . . 60
Why Sum-Check Enables Everything Else . . . 62
The Last Mile: From Oracle Access to Polynomial Commitments . . . 63
Key takeaways . . . 64
Chapter 4: Multilinear Extensions . . . 65
The Boolean Hypercube . . . 65
Why Multilinear? . . . 66
Constructing the Multilinear Extension . . . 67
The Equality Polynomial . . . 69
Worked Example: A 2-Variable Function . . . 70
Efficient Evaluation . . . 71
Tensor Product Structure . . . 75
Multilinear Extensions of Functions on Larger Domains . . . 76
Connection to Sum-Check . . . 76
Evaluations and Coefficients . . . 77
Key takeaways . . . 78
Chapter 5: Univariate Polynomials and Finite Fields . . . 79
Finite Fields: The Algebraic Foundation . . . 80
Roots of Unity . . . 81
The Symmetries . . . 82
Two Representations of Polynomials . . . 83
The Fast Fourier Transform . . . 85
The Vanishing Polynomial . . . 87
The Divisibility Check . . . 88
Lagrange Interpolation over Roots of Unity . . . 88
Cosets: Shifting the Domain . . . 89
The Quotient Argument . . . 90
Univariate vs. Multilinear . . . 91
Key takeaways . . . 91
Chapter 6: Commitment Schemes: Cryptographic Binding . . . 92
The Trust Problem Revisited . . . 93
The Commitment Paradigm . . . 93
Pedersen Commitments: The Discrete Log Approach . . . 94
Worked Example: Pedersen Commitment in $\mathbb{Z}_{23}^*$ . . . 95
The Homomorphic Property . . . 96
Scalar Multiplication . . . 97

From Scalar to Vector Commitments . . . 97
Proving Knowledge of an Opening . . . 98
Worked Example: Proof of Knowledge . . . 99
Beyond Pedersen: A Landscape of Commitment Schemes . . . 100
Why Commitments Matter for ZK Proofs . . . 101
The Hiding-Binding Tradeoff . . . 101
Looking Ahead . . . 102
Key takeaways . . . 103
Chapter 7: The GKR Protocol: Verifying Circuits Layer by Layer . . . 103
From Sum-Check to General Computation . . . 104
Layered Arithmetic Circuits . . . 105
The Wiring Predicates . . . 106
Gate Values as Polynomials . . . 107
The Layer Reduction Lemma . . . 108
The Protocol . . . 109
Worked Example: Verifying $(x_1 + x_2) \cdot x_3$ . . . 111
Why GKR Works . . . 112
Efficiency Analysis . . . 113
The Circuit Model: Power and Limitations . . . 114
The Bigger Picture . . . 114
Key takeaways . . . 115
Chapter 8: From Circuits to Polynomials . . . 116
Arithmetic Circuits . . . 116
Two Problems, Two Paradigms . . . 117
Understanding the Witness . . . 118
The Execution Trace: Witness as Computation History . . . 118
R1CS: The Constraint Language . . . 119
Worked Example: $x^3 + x + 5 = 35$ . . . 122
Two Ways to Prove R1CS . . . 125
PLONKish Arithmetization . . . 127
AIR: Algebraic Intermediate Representation . . . 128
CCS: Unifying the Constraint Formats . . . 130
Handling Non-Arithmetic Operations . . . 133
Lookup Arguments: Breaking the Bit Decomposition Wall . . . 135
The Frontend/Backend Split . . . 136
Key takeaways . . . 137
Chapter 9: Polynomial Commitment Schemes: The Cryptographic Engine . . . 137
The PCS Abstraction . . . 139

KZG: Constant-Size Proofs from Pairings . . . 139
Worked Example: KZG in Action . . . 142
Batch Opening . . . 142
KZG: Properties and Trade-offs . . . 143
Managing Toxic Waste: Powers of Tau Ceremonies . . . 143
IPA/Bulletproofs: No Trusted Setup . . . 144
Comparing KZG and IPA . . . 151
Multilinear Polynomial Commitments . . . 152
The Role of PCS in SNARKs: Replacing Oracle Access . . . 152
The Complete PCS Landscape . . . 154
Key takeaways . . . 154
Chapter 10: Hash-Based Polynomial Commitments and FRI . . . 156
The Merkle Tree: Committing to Evaluations . . . 156
The Core Problem: Low-Degree Testing . . . 158
The Two Phases of FRI . . . 158
The Commit Phase: Split and Fold . . . 159
Commit Phase: Worked Example . . . 160
The Query Phase . . . 161
Query Phase: Worked Example . . . 162
The Folding Paradigm . . . 165
Soundness and DEEP-FRI . . . 166
FRI as a Polynomial Commitment Scheme . . . 166
Practical Considerations . . . 169
Comparing FRI to Algebraic PCS . . . 170
FRI in the Wild: STARKs . . . 171
Key takeaways . . . 171
Chapter 11: The SNARK Recipe: Assembling the Pieces . . . 172
The Three-Layer Architecture . . . 173
Layer 1: Interactive Oracle Proofs . . . 175
Layer 2: Polynomial Commitment Schemes . . . 177
Layer 3: The Fiat-Shamir Transformation . . . 178
Concrete Trace: R1CS to SNARK . . . 181
Zero-Knowledge . . . 182
Modularity in Practice . . . 182
Taxonomy . . . 183
Key takeaways . . . 183
Chapter 12: Groth16: The Pairing-Based Optimal . . . 184
From R1CS to Polynomial Identity . . . 185

The Core Protocol Idea . . . 187
The Trusted Setup . . . 188
Protocol Specification . . . 189
Why the Verification Equation Works . . . 193
Security and the Generic Group Model . . . 195
Trusted Setup: Practical Considerations . . . 197
Comparison with Universal SNARKs . . . 198
Implementation Considerations . . . 199
Key takeaways . . . 200
Chapter 13: PLONK: Universal SNARKs and the Permutation Argument . . . 201
Architecture: Gates and Copy Constraints . . . 202
The Copy Constraint Problem . . . 205
The Permutation Argument . . . 206
Worked Example: The Permutation Argument in Action . . . 210
The Full Protocol . . . 213
Proof Size Analysis . . . 217
Why Roots of Unity? . . . 218
Comparison: PLONK vs. Groth16 . . . 218
Custom Gates and Extensions . . . 219
UltraPLONK . . . 220
Security Considerations . . . 220
Key takeaways . . . 221
Chapter 14: Lookup Arguments . . . 222
The Lookup Problem . . . 222
Plookup . . . 224
Comparison: Custom Gates vs. Lookup Tables . . . 230
Alternative Lookup Protocols . . . 231
Lasso and Jolt . . . 234
Lookups Across Proving Systems . . . 236
Key takeaways . . . 237
Chapter 15: STARKs . . . 238
Why Not Pairings? . . . 238
The Hash Function Gambit . . . 239
The Reed-Solomon Lens . . . 240
Computation as State Evolution . . . 241
Algebraic Intermediate Representation . . . 242
The Complete Protocol . . . 244
A Concrete Example: Fibonacci . . . 246

Adding Zero-Knowledge . . . 248
The Trust and Size Trade-off . . . 249
Circle STARKs and Small-Field Proving . . . 250
Key takeaways . . . 252
Chapter 16: Σ-Protocols: The Simplest Zero-Knowledge Proofs . . . 252
The Discrete Logarithm Problem . . . 254
Schnorr's Protocol . . . 255
Pedersen Commitments and Σ-Protocols . . . 259
Fiat-Shamir: From Interaction to Non-Interaction . . . 261
Composition: AND and OR . . . 262
Key takeaways . . . 263
Chapter 17: The Zero-Knowledge Property . . . 264
The Simulation Argument . . . 265
The Graph Non-Isomorphism Protocol . . . 265
Formal Definition . . . 267
Three Flavors of Zero-Knowledge . . . 267
Honest Verifiers and Malicious Verifiers . . . 268
The Limits of Zero-Knowledge . . . 270
Zero-Knowledge in the Wild: Sum-Check . . . 271
Auxiliary Input . . . 272
Key takeaways . . . 272
Chapter 18: Making Proofs Zero-Knowledge . . . 273
The Leakage Problem . . . 274
Technique 1: Commit-and-Prove . . . 275
Technique 2: Masking Polynomials . . . 277
Comparing the General Techniques . . . 280
Zero-Knowledge in Practice . . . 281
Key takeaways . . . 284
Chapter 19: Fast Sum-Check Proving . . . 285
The Prover's Apparent Problem . . . 286
The Halving Trick . . . 287
Beyond Black-Box Arithmetic . . . 291
High-Degree Products . . . 292
Small-Value Round-Batching . . . 293
Sparse Sums . . . 295
Spartan: Sum-Check for R1CS . . . 300
The PCP Detour and Sum-Check's Return . . . 306
Key takeaways . . . 308

Chapter 20: Fast STARK Proving . . . 309
The prover pipeline . . . 310
AIR design and the degree-blowup tradeoff . . . 312
Small fields and extension field lifting . . . 315
NTT optimization . . . 318
FRI optimization . . . 321
A worked example: proving Poseidon2 hashes . . . 327
Comparison with sum-check optimization . . . 329
Key takeaways . . . 330
Chapter 21: Minimizing Commitment Costs . . . 332
The Two-Stage Paradigm . . . 333
Batch Evaluation Arguments . . . 334
Virtual Polynomials . . . 339
Time-Varying Functions . . . 343
The Padding Problem and Jagged Commitments . . . 347
Small-Value Preservation . . . 352
Key takeaways . . . 353
Chapter 22: The Two Classes of PIOPs . . . 354
The Divide . . . 354
Historical Arc . . . 355
A Common Task: Proving $a \circ b = c$ . . . 356
The Quotienting Path . . . 356
The Sum-Check Path . . . 358
The Comparison . . . 359
Wiring Constraints: The Second Half . . . 360
The PCS Connection . . . 362
Choosing a Paradigm . . . 363
Key takeaways . . . 364
Chapter 23: Composition and Recursion . . . 365
Composition: Proving a Proof Is Valid . . . 366
The Composition Construction . . . 368
Recursion: Composing with Yourself . . . 369
The Curve Cycle Problem . . . 373
Incrementally Verifiable Computation (IVC) . . . 376
Folding Schemes: Cheaper IVC . . . 378
Beyond Nova: HyperNova and ProtoStar . . . 385
BlindFold: Folding for Zero-Knowledge . . . 394
Choosing a strategy . . . 396

Key takeaways . . . 396
Chapter 24: Choosing a SNARK . . . 397
The Five Axes of Trade-off . . . 398
The System Landscape . . . 401
Application-Specific Guidance . . . 404
The Trade-Off Triangle . . . 406
Implementation Realities . . . 407
Quick Reference . . . 408
Key takeaways . . . 409
Chapter 25: MPC and ZK parallel paths . . . 409
The MPC problem . . . 411
Secret-sharing MPC . . . 412
Garbled circuits . . . 415
Oblivious transfer . . . 421
Mixing protocols . . . 422
MPC-in-the-head . . . 423
Threshold cryptography . . . 424
Practical considerations . . . 426
Key takeaways . . . 427
Chapter 26: Frontiers and Open Problems . . . 428
Small fields and Binius . . . 429
Post-quantum SNARKs . . . 431
zkVMs . . . 433
Formal verification . . . 436
Deployment frontiers . . . 437
Theoretical foundations . . . 437
Key takeaways . . . 438
Chapter 27: ZK in the cryptographic landscape . . . 439
Why ZK arrived first . . . 440
Computing on ciphertexts . . . 440
Program obfuscation . . . 443
Convergence . . . 446
The landscape at a glance . . . 446
Key takeaways . . . 447
Appendix A: Cryptographic primitives . . . 448
Mathematical background . . . 448
Secure random sampling . . . 450
Nothing-up-my-sleeve (NUMS) constructions . . . 451

Shamir's secret sharing . . . 451
Feldman's verifiable secret sharing . . . 452
Hash functions in zero-knowledge . . . 453
Modular arithmetic implementation . . . 454
Random beacons . . . 455
Elliptic curves in zero-knowledge . . . 455
Group operations . . . 457
Appendix B: Historical Timeline . . . 458
Theoretical foundations (1985-1992) . . . 458
The ZK winter (1992-2008) . . . 459
Path to practical systems (2008-2016) . . . 459
The scaling era (2017-2020) . . . 460
The modern era (2020-present) . . . 461
Visual timeline . . . 463
Key themes . . . 463
Appendix C: Field equations cheat sheet . . . 464
Schwartz-Zippel lemma . . . 464
Multilinear extensions . . . 464
Sum-check protocol . . . 465
Vanishing polynomials . . . 466
R1CS and QAP . . . 466
KZG polynomial commitments . . . 467
FRI folding . . . 467
AIR (Algebraic Intermediate Representation) . . . 468
PLONK . . . 469
Groth16 . . . 470
Lookup arguments . . . 470
GKR protocol . . . 471
Inner product argument (IPA) . . . 472
Nova folding . . . 472
Fiat-Shamir transform . . . 473
Complexity summary . . . 473
Field sizes (common choices) . . . 473
Quick reference . . . 474
Appendix D: Advanced polynomial commitment schemes . . . 474
Hyrax . . . 475
Dory . . . 483
HyperKZG and Zeromorph: KZG for multilinear polynomials . . . 489

Chapter 1: The Trust Problem

In the summer of 1821, two mathematicians sat in a room in London, exhausted and frustrated. Charles Babbage and John Herschel had been tasked with checking the Nautical Almanac, a book of astronomical tables that sailors used to navigate the globe.

At the time, a "computer" was not a machine. It was a job title. Clerks calculated these tables by hand, other clerks checked their work, and printers typeset the results. Every step was a point of failure. As Babbage and Herschel compared the calculations against the printed proofs, they found error after error. A wrong digit in a logarithm didn't just mean a failed exam; it meant a ship running aground on a reef in the West Indies.

Exasperated, Babbage slammed the table and declared: "I wish to God these calculations had been executed by steam!"

That outburst launched the age of mechanical computation. Babbage spent the rest of his life designing engines to generate mathematical tables automatically, removing the human element from execution. If the machine was built correctly, its outputs could be trusted.

Two centuries later, we have fulfilled Babbage's wish. We have steam, now silicon, executing calculations at speeds he couldn't have imagined. But in solving the speed problem, we reintroduced the trust problem in a new form.

You send your calculation to the cloud. The cloud sends back an answer.

Why should you believe it?

The server might be compromised. The operator might be malicious. The hardware might be faulty. The software might contain bugs. Even if everything works correctly, how would you *know*? The only evidence you have is the answer itself, and the answer, by itself, proves nothing.

Here's the asymmetry: executing a computation takes resources (time, memory, energy). But *checking* whether the computation was done correctly also takes resources. In many cases, the same resources. If you could check the answer cheaply, you wouldn't have outsourced the computation in the first place.

This is the **trust problem in computation**: how do you verify without redoing all the work?

Truth Without a Judge

For millennia, knowledge has traveled through testimony. One person tells another: "I computed this result." The listener judges whether to believe. This judgment rests on reputation, authority, past behavior. All the machinery of social trust.

What if claims could carry their own evidence? Not testimony backed by reputation. Not certificates issued by authorities. Something stranger: an object that proves itself. If the claim is false, the object cannot exist. If the object exists and passes inspection, the claim must be true. No judge required.

This is not metaphor. The technology exists. A computational claim can be accompanied by a mathematical object, a *proof*, that anyone can verify in milliseconds. The proof works not because you trust the prover, but because mathematics makes cheating impossible. Two machines that have never communicated can verify the same proof and reach the same conclusion, not because they negotiated, but because the structure of mathematics forces the same answer from any system capable of arithmetic.

We will call this *arithmetic consensus*: agreement enforced by structure rather than achieved by persuasion. Chapter 2 develops the mechanism (the Schwartz-Zippel lemma) and explores why this represents a genuinely new foundation for intersubjective truth. For now, hold this question: what becomes possible when "I trust you" can be replaced with "I verified the math"?

This book teaches you how to build such proofs.

When Verification Is Easy

Before confronting the hard case, consider situations where verification *is* easy, given the right certificate.

Factorization: Given n and a claim that p, q are its prime factors. Verification: multiply $p \times q$, check it equals n, verify p and q are prime. Finding those factors is believed to require exponential time; checking them takes polynomial time.

Graph coloring: Given a graph and a claimed 3-coloring. Verification: for each edge, check that its endpoints have different colors. Finding such a coloring is NP-hard; verifying one is linear in the number of edges.

Satisfying assignments: Given a Boolean formula and a claimed satisfying assignment. Verification: substitute the values and evaluate each clause. Finding such an assignment is NP-complete; checking one is polynomial.

These are problems in **NP**: the class of problems where, if someone hands you a proposed solution, you can *check* whether it's correct in reasonable time (polynomial in the input size). NP doesn't say

anything about how hard it is to *find* a solution, only how hard it is to *verify* one. The proposed solution serves as a *witness* or *certificate* of correctness.

Note the asymmetry: NP captures "easy to verify," not "hard to find." Some NP problems are easy to solve (every problem in P is also in NP). The interesting cases are those where finding appears hard but verifying is easy. This gap is what proof systems exploit.

When Verification Seems As Hard As Computation

But many problems don't have short certificates.

The obvious verification strategy is to recompute: run the same algorithm on the same inputs and compare results. This works, but it defeats the purpose. You outsourced because you couldn't (or didn't want to) pay the computational cost. Verification that costs as much as the original computation is no verification at all.

For a moment, consider what "cheap verification" would even mean. The computation processes some input of size n, takes T steps, and produces an output. Cheap verification would mean checking correctness in time $o(T)$: strictly less than the original computation. Ideally, much less. Ideally, polylogarithmic in T, or even constant.

But this seems impossible. How can you verify a computation without understanding what it computed? How can you understand what it computed without retracing its steps? The answer is computed from the input through a long chain of operations; surely checking requires following that chain?

The Cost of Blind Trust

On February 25, 1991, during the Gulf War, a Patriot missile battery in Dhahran, Saudi Arabia, failed to intercept an incoming Iraqi Scud. The missile struck an American barracks, killing 28 soldiers.

The cause was a software bug. The Patriot's tracking system measured time in tenths of a second using a 24-bit register, then multiplied by 0.1 to convert to seconds. But 0.1 has no exact binary representation; it's a repeating fraction, like 1/3 in decimal. The system truncated it, introducing a tiny error of about 0.000000095 seconds per tenth.

Tiny, but cumulative. The battery had been running for 100 hours. Over that time, the error accumulated to 0.34 seconds. For a Scud traveling at Mach 5, that's a tracking error of over 600 meters. The missile defense system calculated that the incoming Scud was outside its range gate and didn't fire.

The bug had been discovered two weeks earlier. Israeli defense forces, who had noticed the drift, warned the U.S. Army and recommended rebooting the system regularly to reset the clock. A software

patch was developed. It arrived in Dhahran on February 26, one day after the attack.

Twenty-eight soldiers died because a computation was trusted without verification. The system worked exactly as programmed; the program was wrong. No one checked.

Whether the error comes from a hacker in the server room or a rounding bug in the floating-point unit, the result is the same: a wrong answer accepted as truth. Validity proofs don't care about intent; they care about correctness. They catch malice and accident alike.

The discovery of the 1980s and 1990s was that cheap verification *is* possible.

Interactive Proofs: The Breakthrough

The insight came from complexity theory. It involved a conceptual leap: *interaction* and *randomness* together can create verification power that neither possesses alone.

A computationally unbounded prover claims to have solved a problem. A polynomially bounded verifier wants to check this claim. The verifier cannot solve the problem themselves (that's the whole point), but they can engage in a conversation with the prover.

In an **interactive proof**, the verifier sends random challenges, the prover responds, and after some number of rounds, the verifier decides whether to accept or reject the claim.

Two properties give the conversation its power:

Completeness: If the claim is true, an honest prover can always convince the verifier to accept.

Soundness: If the claim is false, no prover, no matter how clever or powerful, can convince the verifier to accept, except with negligible probability.

The probability in soundness comes from the verifier's randomness. The prover doesn't know in advance what challenges the verifier will send. A cheating prover must prepare for all possible challenges, and this is where they fail. The space of possible challenges is exponentially large; the prover cannot succeed at all of them if the claim is false.

Randomness Creates Asymmetry

Suppose I claim two polynomials $p(x)$ and $q(x)$ are identical. Both polynomials have degree at most d, and their coefficients are elements of a large finite field $\mathbb{F}$ of size $|\mathbb{F}| = 2^{256}$.

Without randomness, verifying this claim requires comparing all $d + 1$ coefficients. If d is large, this is expensive.

With randomness, verification becomes trivial:

1. Pick a random $r \in \mathbb{F}$
2. Evaluate $p(r)$ and $q(r)$
3. Accept if they're equal, reject otherwise

If $p = q$, then $p(r) = q(r)$ for all r. Verification always succeeds.

If $p \neq q$, then $p - q$ is a nonzero polynomial of degree at most d. Such a polynomial has at most d roots. The probability that our random r hits a root is at most $d/|\mathbb{F}|$.

With $d = 10^6$ and $|\mathbb{F}| = 2^{256}$:

$$\Pr[\text{cheating succeeds}] \leq \frac{10^6}{2^{256}} \approx 2^{-236}$$

This is so small it's effectively zero. One random evaluation suffices.

This is the **Schwartz-Zippel lemma** in action. We'll see it again and again throughout this book. It is the central tool in interactive proofs. Random evaluation catches disagreement between polynomials with overwhelming probability.

From IP to Succinctness

The theoretical study of interactive proofs established profound results:

IP = PSPACE: Interactive proofs with polynomial-time verifiers can verify exactly the class **PSPACE**. What is PSPACE? It's the class of problems solvable using a reasonable amount of memory (polynomial in the input size), but with no limit on time. A PSPACE algorithm might run for centuries, but it can only use a bounded scratch pad. This includes problems like determining the winner in generalized chess (with an $n \times n$ board) or evaluating quantified Boolean formulas ("for all x, there exists y, such that…"). These problems are believed to be far harder than NP. The verifier's randomness and the prover's computational power combine to verify claims that seem uncheckable.

But these theoretical protocols had a problem: they weren't *succinct*. The total communication (the number of bits exchanged between prover and verifier) could be polynomial in the computation size. Better than redoing the computation, but not by much.

The goal of **succinct arguments** is more ambitious: proofs that are *polylogarithmic* in the computation size, or even constant. A computation taking billions of steps should yield a proof of hundreds or thousands of bits, not billions.

Achieving this goal required new proof models: extensions and variants of interactive proofs that enabled different trade-offs between interaction, query access, and succinctness.

The Proof System Zoo

The path from interactive proofs to modern SNARKs runs through several distinct proof models. Understanding this taxonomy clarifies where different techniques come from and why modern systems take the forms they do.

This section mentions several *complexity classes* (IP, MIP, PSPACE, NEXP). These are categories that computer scientists use to classify problems by how hard they are to solve or verify. Don't worry if the distinctions feel abstract on first reading. The key intuition is that different proof models have different "verification power," meaning some can verify harder problems than others. The specific class names matter less than the pattern: adding constraints to the prover (like forbidding communication between multiple provers) paradoxically *increases* what the verifier can check.

Interactive Proofs (IP)

The starting point. A prover and verifier exchange messages. The verifier uses randomness to catch cheating. Security is *information-theoretic*: even an all-powerful prover cannot convince the verifier of a false statement (except with negligible probability).

Think of it as courtroom cross-examination. The prover (witness) wants to convince the verifier (judge) of some claim. The judge cannot independently verify the facts; they weren't there, they don't have the evidence. But through clever questioning, the judge can probe for inconsistencies. An honest witness has nothing to hide; their answers will be consistent. A lying witness must maintain a web of fabrications, and random probing questions will eventually find a thread that unravels it.

The class **IP** contains all *languages* with such protocols where the verifier runs in polynomial time. A language here is a set of strings $\mathcal{L} \subseteq \{0,1\}^*$, the formal way complexity theory encodes decision problems: $x \in \mathcal{L}$ means the answer to the problem on input x is "yes." The theorem **IP = PSPACE** (Shamir, 1990) shows this class is remarkably large, far larger than NP. The verifier's random questions, combined with the prover's unbounded computational power, can verify claims that no static certificate could capture.

Multi-Prover Interactive Proofs (MIP)

IP was powerful (it captured all of PSPACE), but verification still required multiple rounds of back-and-forth, and proofs weren't succinct. What if we could constrain the prover more tightly to gain more verification power?

What if the verifier could interrogate *multiple* provers who cannot communicate with each other?

Imagine two suspects in separate rooms: the classic police interrogation. The detective asks each suspect questions, comparing answers for consistency. If the suspects are telling the truth, their stories align effortlessly. If they're lying, they can't coordinate their lies without communicating, and they can't communicate. The detective doesn't need to know the truth themselves; they only need to catch inconsistencies between the two stories.

In a **Multi-Prover Interactive Proof**, two or more provers share the witness but cannot exchange messages during the protocol. The verifier sends different challenges to each prover and cross-checks their responses.

The deep insight here is **non-adaptivity**. In a single-prover IP, the prover sees the verifier's first challenge before answering, then sees the second challenge before answering again. The prover *adapts* to each challenge in sequence. With two non-communicating provers, the verifier can send different questions simultaneously; neither prover knows what the other was asked. This forces both provers to commit to a consistent story *before* seeing the cross-examination.

This apparently simple change unleashes enormous verification power:

MIP = NEXP (Babai, Fortnow, Lund, 1991): Multi-prover proofs can verify problems in **NEXP**, which stands for nondeterministic exponential time. What does this mean? Recall that NP is the class where solutions can be verified quickly. NEXP is the exponentially larger cousin: problems where the solution itself might be exponentially large (so even *writing it down* takes exponential time), but once written, it can be checked in exponential time. These are vastly harder problems than NP or even PSPACE.

The gap from PSPACE to NEXP is vast. The non-communication constraint is what makes it possible: the verifier can probe two points of a story simultaneously, catching inconsistencies that a single adaptive prover could finesse.

This idea of forcing commitment before challenge reappears throughout SNARK design. When we study polynomial commitment schemes, we'll see the same principle: the prover commits to a polynomial, then the verifier challenges. The commitment plays the role of the second prover: it locks in answers before the questions are known.

Probabilistically Checkable Proofs (PCP)

MIP was even more powerful (it captured NEXP), but it required *two separate provers*. In practice, we usually have just one prover. Could we get similar power without needing to literally interrogate two parties in separate rooms?

Here the model shifts from interaction to *query access*. The prover writes down a static proof string π (potentially very long), which is just a sequence of symbols like $\pi = (\pi_1, \pi_2, \pi_3, \dots, \pi_m)$. The verifier

doesn't read the whole string. Instead, they pick a few positions at random and look only at those symbols. For example, the verifier might flip some coins, decide to look at positions 17, 42, and 803, read π_{17}, π_{42}, and π_{803}, and make a decision based only on those three values.

A **PCP** is characterized by two parameters, both functions of the input size n:

- How many random bits the verifier uses (to decide which positions to query)
- How many positions in the proof string the verifier queries

The verifier's decision depends only on the input, their random coin flips, and the few symbols they read from the proof.

The **PCP Theorem** (Arora, Safra; Arora, Lund, Motwani, Sudan, Szegedy; 1992) is one of the landmark results of complexity theory:

$$\textbf{NP} = \textbf{PCP}[O(\log n), O(1)]$$

Notation: **PCP**$[r(n), q(n)]$ is the class of languages decidable by a probabilistic verifier that uses $r(n)$ random bits and queries $q(n)$ positions in a proof string. The theorem says: using only $O(\log n)$ random bits (polynomially many possible random choices) and reading $O(1)$ proof positions (a constant, independent of input size), you can verify any NP statement with constant soundness error.

What does "every NP problem has a PCP" mean? Recall that an NP problem is one where solutions can be verified quickly given a witness (like checking that a proposed graph coloring is valid). The PCP theorem says something stronger: for any such problem, there exists a way to encode the witness into a longer proof string such that the verifier uses only $O(\log n)$ random bits and queries only a *constant* number of proof positions. The proof might be polynomial-size, but verification reads only $O(1)$ bits.

How can this possibly work? The key is **structured redundancy**.

Think of a completed Sudoku puzzle. The puzzle has internal constraints: each row, column, and 3×3 box must contain the digits 1-9 exactly once. Now imagine "corrupting" one cell by changing a 7 to a 3. This single error violates the constraint for its row, its column, and its box. One mistake creates evidence in multiple places. A random spot-check has a decent chance of catching it.

PCPs work the same way, but with vastly more redundancy. The proof is not the raw witness; it's an *encoded* version where local constraints interlock globally. The encoding transforms the witness into a form where any error, any deviation from a valid proof, creates detectable inconsistencies across many positions.

The technology: low-degree polynomial encoding. The witness is interpreted as evaluations of a polynomial, then extended to many more points. Polynomial structure ensures that errors propagate: a

polynomial that's wrong at even one point must disagree with the correct polynomial almost everywhere (Schwartz-Zippel, again). Random queries catch these disagreements with high probability.

Consider what this means. A satisfying assignment to a million-variable formula might require a million bits to write down. But there exists an encoding, a PCP, where checking validity requires reading only, say, 3 bits. The encoding has redundancy; errors anywhere propagate everywhere, detectable by sparse sampling.

The MIP-PCP Connection

There's a deep connection between multi-prover proofs and PCPs. The two non-communicating provers in an MIP can be simulated by a single long proof string: each possible pair of questions to the two provers corresponds to a position in the string, with the answer pair as the value at that position.

The non-communication constraint in MIP becomes a consistency requirement in PCP: the answers at different positions must be consistent with some underlying witness. The verifier's power to cross-check provers becomes the power to query random positions and check consistency.

This connection was key to proving MIP = NEXP and to subsequent PCP constructions.

Interactive Oracle Proofs (IOP)

The PCP theorem was a landmark: it showed any NP statement has a proof checkable with constant queries. But PCPs require enormous proof strings, and they're non-interactive (the prover must anticipate all possible verifier randomness). IP had efficient interaction but no query access. Could we combine the best of both?

Interactive Oracle Proofs do exactly that.

In an IOP, the protocol has multiple rounds. In each round, the prover sends a *proof string* (or, more abstractly, an *oracle*). The verifier can query this oracle at chosen positions, then sends a challenge. The prover responds with another oracle, and so on.

Why combine interaction and oracles? Each compensates for the other's weakness. Pure PCPs require enormous proof strings to achieve low soundness error; the proof must anticipate all possible verifier randomness. Pure IPs require many rounds of back-and-forth; the verifier probes incrementally, each round narrowing the space of consistent lies. IOPs get the best of both: the prover commits to an oracle (like a PCP), then the verifier challenges (like an IP), then another oracle, another challenge. Each oracle only needs to handle the challenges that could follow *given previous commitments*.

This hybrid captures what modern SNARK constructions actually do:

1. Prover commits to a polynomial (an oracle that the verifier can query for evaluations)
2. Verifier sends a random challenge
3. Prover commits to another polynomial
4. Repeat
5. Verifier makes a few queries and decides

The IOP abstraction separates the *protocol logic* from the *implementation of oracles*. The oracle is abstract; the verifier magically gets evaluations at chosen points. Chapter 11 shows how polynomial commitment schemes instantiate these oracles cryptographically.

Linear PCPs

IOPs gave us a clean abstraction, but implementing them required a way to make oracles concrete and binding. If we restrict what *kind* of queries the verifier can make, we can use cryptography to enforce that restriction. This leads to Linear PCPs.

In a standard PCP (as described above), the proof is a string of symbols and the verifier reads a few specific positions: "give me π_{17}, π_{42}, and π_{803}." In a **Linear PCP**, the proof is still a vector of values $\vec{\pi} = (\pi_1, \pi_2, \ldots, \pi_k)$, but the verifier can only ask for linear combinations: "give me $\sum_i q_i \cdot \pi_i$ for my chosen weights q."

Think of it as a library where you can't open the books (that would reveal the witness). You can only ask the librarian to weigh books in specific combinations. "Put 2 copies of book 1 on the scale, plus 3 copies of book 3, plus 1 copy of book 7, and tell me the total weight." The librarian answers with a single number. You ask several such questions. From these weighted sums, you try to verify some property of the books without ever seeing their contents.

The linearity constraint turns out to be exactly what we need, because **certain cryptographic structures only allow weighted-sum operations, nothing else**.

Elliptic curve groups have this property. In an elliptic curve group, you can add points together and multiply points by numbers, but you cannot multiply two points together. Think of it like a calculator that has + and × buttons, but the × only works when one input is a regular number. If the proof values are encoded as elliptic curve points, then anyone holding those points can only compute weighted sums of them.

Concretely: the prover knows the proof values $\pi_1, \pi_2, \ldots$ and a special group point G. The prover creates encoded values $[\pi_i] = \pi_i \cdot G$ (multiply each value by the point G). The verifier receives these encoded points. Given weights $q_1, q_2, \ldots$, the verifier can compute $q_1 \cdot [\pi_1] + q_2 \cdot [\pi_2] + \cdots$, which equals the encoding of $q_1 \cdot \pi_1 + q_2 \cdot \pi_2 + \cdots$. The verifier gets the weighted sum, but cannot extract the

individual π_i values or compute anything beyond weighted sums. The elliptic curve structure itself forces the verifier to play by the Linear PCP rules.

Groth16 (Chapter 12) is built on linear PCPs. The prover's messages are linear combinations of structured reference string elements, which are themselves encodings of powers of a secret. The verifier checks linear relationships via pairings: bilinear maps that allow *one* multiplication in the exponent, just enough to check quadratic constraints.

From Proof Models to SNARKs

All modern SNARKs arise from one of these proof models combined with cryptographic compilation:

Proof Model	+ Cryptography	= SNARK Family
IP	+ Polynomial Commitments	Spartan, HyperPlonk
IOP (polynomial)	+ KZG / FRI	PLONK, Marlin, STARKs
Linear PCP	+ Pairings	Groth16, BCTV14
PCP	+ Merkle trees	Kilian-style arguments

The pattern: start with an information-theoretically secure protocol, then use cryptography to make the prover's messages short and binding.

Polynomial Commitment Schemes (Chapter 9-10) instantiate IOP oracles: the prover commits to a polynomial, the verifier queries evaluations, and a short proof demonstrates correctness of each evaluation.

Fiat-Shamir (Chapter 11) eliminates interaction: derive the verifier's challenges from hashes of the transcript. The prover computes the entire interaction locally and outputs a static proof.

The combination yields **SNARKs**: Succinct Non-interactive Arguments of Knowledge. A SNARK for a computation of size n has:

- Proof size: $O(\log n)$ or even $O(1)$
- Verification time: $O(\log n)$ or $O(1)$
- Prover time: $O(n \log n)$ or similar quasi-linear

The asymmetry is achieved. Verification is exponentially cheaper than computation.

Zero-Knowledge: Proving Without Revealing

There's another dimension to this story. So far, we've focused on *soundness*: preventing false claims from being verified. But what about *privacy*?

Suppose you want to prove you know a password without revealing the password itself. Or that you have sufficient funds for a transaction without revealing your balance. Or that you satisfy some credential requirement without exposing your identity.

Zero-knowledge proofs achieve exactly this. The proof convinces the verifier that the statement is true, but reveals *nothing* beyond this single bit of information. The verifier learns "yes, this is true" and nothing else.

The formal definition involves a *simulator*: an algorithm that produces transcripts indistinguishable from real proof transcripts, without access to the secret witness. If such a simulator exists, the proof is zero-knowledge; the transcript could have been generated by someone who didn't know the secret, so the transcript cannot leak the secret.

Zero-knowledge adds a layer of privacy to succinct verification. Together, they form **zkSNARKs**: Zero-Knowledge Succinct Non-interactive Arguments of Knowledge.

The Architecture of Modern Proofs

This book develops the theory and practice of zkSNARKs. The architecture has emerged from decades of research, but it follows a consistent pattern:

1. Arithmetization (Chapters 4-8): Convert the computational claim into algebraic form. A program becomes a circuit. A circuit becomes a system of polynomial equations. The claim "I computed correctly" becomes "these polynomials satisfy this identity."

2. Information-Theoretic Protocol (Chapters 3, 7): Design an interactive protocol where the prover sends polynomials (or claims about polynomials) and the verifier checks them via random evaluations. This protocol is sound against unbounded provers; no cryptographic assumptions yet.

3. Cryptographic Compilation (Chapters 6, 9-10): Replace the abstract polynomials with cryptographic commitments. The prover commits to polynomials before seeing challenges. Polynomial commitment schemes (KZG, FRI, IPA) provide this binding.

4. Fiat-Shamir Transform (Chapter 11): Eliminate interaction. The verifier's random challenges are derived from a hash of the transcript. The prover computes the entire interaction locally and outputs a static proof.

The result: a proof that anyone can verify, that reveals nothing about the witness, and that is exponentially smaller than the computation it attests to.

Why This Matters

Each application is a trust assumption eliminated.

Verifiable computation removes trust in the cloud. You outsource to untrusted servers, receive a proof with the result, and verify cheaply. The server's incentives, security practices, and internal controls become irrelevant. You don't trust the server; you verify the proof.

Blockchain scalability removes trust in centralized sequencers. Layer 2 solutions process thousands of transactions off-chain, producing a single proof that the main chain verifies. The sequencer cannot lie about execution. Transaction throughput increases by orders of magnitude without introducing new trust assumptions.

Privacy-preserving credentials remove trust in identity intermediaries. Prove you're over 21 without revealing your birthdate. Prove you passed a background check without revealing what was checked. The verifier learns exactly one bit: valid or not. No data broker, no identity provider, no linkable trail.

Computational integrity removes trust in institutions. Scientific simulations, machine learning inference, financial calculations: any computation can be accompanied by a proof of correctness. The question changes from "do I trust this organization?" to "does this proof verify?"

The pattern is consistent: find a trust assumption, replace it with mathematics.

The Road Ahead

The chapters that follow develop this technology piece by piece.

We begin with polynomials (Chapter 2), the universal language of algebraic proof systems. The sum-check protocol (Chapter 3) shows how to verify exponential sums in polynomial time, the foundational technique underlying almost everything that follows.

Multilinear extensions (Chapter 4) and univariate polynomials (Chapter 5) provide two complementary encoding schemes for computational data. Commitment schemes (Chapter 6) bind provers to their claims.

The GKR protocol (Chapter 7) verifies arbitrary circuits using sum-check. Arithmetization (Chapter 8) shows how real computations become circuits.

Polynomial commitment schemes (Chapters 9-10) provide the cryptographic foundation: KZG, IPA, and FRI, each with different trade-offs between proof size, verification time, and trust assumptions.

The SNARK recipe (Chapter 11) explains how these pieces assemble. Groth16 (Chapter 12), PLONK (Chapter 13), lookup arguments (Chapter 14), and STARKs (Chapter 15) are complete systems, each optimizing different aspects.

Σ-protocols (Chapter 16) and zero-knowledge (Chapters 17-18) add privacy.

Chapters 19-21 form Part VI on prover optimization, covering fast sum-check proving, fast STARK proving, and techniques for minimizing commitment costs. These chapters are optional on a first read: they go into engineering depth that matters for implementers but is not required to understand the rest of the book. Chapter 22 then synthesizes by comparing the two PIOP paradigms (quotienting vs. sum-check) and is part of the main thread.

Composition and recursion (Chapter 23) enable proofs about proofs: unlimited computation with constant verification. The book concludes with system selection guidance (Chapter 24), MPC's parallel path (Chapter 25), open frontiers (Chapter 26), and the broader cryptographic landscape (Chapter 27).

By the end, you'll understand what zkSNARKs do and, beneath that, *how* they work: the mathematical structures that make the impossible possible.

Key takeaways

1. **Verification should be cheaper than computation.** If Alice outsources a computation to Bob, she shouldn't have to redo the entire work to check his answer. The goal is *asymmetric verification*: Bob does the hard work once, Alice checks quickly.
2. **Randomness creates verification power.** A deterministic verifier who can't compute the answer can't check it either. But a randomized verifier can probe for inconsistencies. Random questions catch cheaters with high probability.
3. **Schwartz-Zippel is the fundamental tool.** Two different degree-d polynomials agree on at most d points. Evaluating at a random point catches disagreement with probability at least $1 - d/|\mathbb{F}|$. Polynomials are central to proof systems because errors propagate almost everywhere.
4. **Proof models evolved by constraining the prover.** IP captures PSPACE. MIP (multiple non-communicating provers) captures NEXP. PCPs allow constant-query verification. IOPs combine interaction with oracle access. Paradoxically, more constraints on the prover give the verifier more power.
5. **The PCP theorem is foundational.** NP = PCP[$O(\log n)$, $O(1)$]. Any NP statement has a proof where the verifier reads constantly many bits. This requires encoding the witness with structured redundancy so that any error creates detectable inconsistencies.
6. **Polynomial commitments instantiate oracles.** The prover commits to a polynomial; the verifier queries evaluations; a short proof demonstrates each evaluation is correct. Different schemes (KZG, FRI, IPA) offer different trade-offs.

7. **Fiat-Shamir eliminates interaction.** Replace the verifier's random challenges with hashes of the transcript. The prover computes the entire interaction locally and outputs a static proof.
8. **The architecture is modular.** Arithmetization encodes computation as constraints. An IOP proves the constraints are satisfied. Cryptographic compilation (PCS + Fiat-Shamir) makes proofs short and non-interactive. Each layer can be swapped independently.
9. **Zero-knowledge is orthogonal to succinctness.** The proof can reveal nothing beyond the statement's truth. Privacy and compression are independent properties; modern systems achieve both.

Chapter 2: The Power of Polynomials

In 1960, Irving Reed and Gustave Solomon were trying to solve a practical problem: how do you send data through space?

The spacecraft transmitting from millions of miles away couldn't retransmit lost bits. The signal would be corrupted by cosmic radiation, hardware glitches, and the irreducible noise of the universe. Reed and Solomon needed a way to encode information so that even after some of it was destroyed, the original could be perfectly recovered.

Their solution was startlingly simple. Instead of sending raw data, they evaluated a polynomial at many points and transmitted the evaluations. A polynomial of degree d is uniquely determined by $d+1$ points, so if you send many more than $d+1$ evaluations, some can be corrupted or lost, and the receiver can still reconstruct the original polynomial from what remains.

What Reed and Solomon had discovered, without quite realizing it, was one of the most powerful ideas in all of computer science: **polynomials are rigid**. A low-degree polynomial cannot "cheat locally." If you change even a single coefficient, the polynomial's values change at almost every point. This rigidity, this inability to lie in one place without being caught elsewhere, would turn out to be exactly what cryptographers needed, thirty years later, to build systems where cheating is mathematically impossible.

The Motivating Problem: Beyond NP

Before we explore polynomials, let's understand the problem they solve. In Chapter 1, we saw that some problems have the useful property that their solutions are easy to check: multiply the claimed factors to verify factorization, check each edge to verify graph coloring. These are NP problems; the solution serves as its own certificate.

But what about problems that don't have short certificates?

The SAT Problem: The Mother of All NP Problems

The **Boolean Satisfiability Problem (SAT)** asks: given a Boolean formula, is there an assignment of True/False values to its variables that makes the formula evaluate to True?

Consider the formula (where $\vee$ means OR, $\wedge$ means AND, and $\neg$ means NOT):

$$\phi(x_1, x_2, x_3) = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

This is in *Conjunctive Normal Form (CNF)*: an AND of ORs. Each parenthesized group is a *clause*, and each x_i or $\neg x_i$ is a *literal*.

The question: does there exist an assignment $(x_1, x_2, x_3) \in \{\text{True}, \text{False}\}^3$ that satisfies all clauses simultaneously? This is what makes SAT hard: you must determine whether *any* solution exists, not find a specific one. With 3 variables there are $2^3 = 8$ possibilities; with 100 variables there are $2^{100} \approx 10^{30}$. No known algorithm avoids checking exponentially many cases in the worst case.

For this toy example, we can reason through it. Clause 2 ($\neg x_1 \vee x_2 \vee \neg x_3$) needs at least one of: $x_1 = \text{False}$, $x_2 = \text{True}$, or $x_3 = \text{False}$. Clause 3 needs at least one variable true. Setting $x_2 = \text{True}$ helps both. With x_2 fixed, Clause 1 becomes $(x_1 \vee \text{False} \vee x_3)$, requiring x_1 or x_3 true. Try $(x_1, x_2, x_3) = (\text{True}, \text{True}, \text{True})$:

- Clause 1: True $\vee$ False $\vee$ True $=$ True ✓
- Clause 2: False $\vee$ True $\vee$ False $=$ True ✓
- Clause 3: True $\vee$ True $\vee$ True $=$ True ✓

We found a satisfying assignment, so the formula is satisfiable. But notice: finding this solution required insight or luck. If no solution existed, we would have had to check all 2^n possibilities to be certain.

Why SAT matters: The Cook-Levin theorem (1971) proved that SAT is *NP-complete*: every problem in NP can be efficiently reduced to a SAT instance. If you can solve SAT efficiently, you can solve *any* NP problem efficiently. This makes SAT the canonical "hard" problem.

The good news for verification: Once someone *has* a solution, checking it is easy: just plug in the values. The assignment is a certificate that proves satisfiability. The asymmetry is striking: finding a solution may take exponential time, but verifying one takes linear time.

#SAT: When Even Certificates Don't Help

Now consider a harder question: **how many** satisfying assignments does a formula have?

This is the **#SAT problem** (pronounced "sharp SAT" or "number SAT"). It's in a complexity class called **#P**, which is believed to be harder than NP.

Why? Because even if someone tells you "there are exactly 47 satisfying assignments," there's no obvious way to verify this without enumerating possibilities. Having one satisfying assignment doesn't tell you there aren't 46 others. Having 47 assignments doesn't prove there isn't a 48th.

For a formula with n variables, there are 2^n possible assignments. For $n = 100$, that's about 10^{30} assignments (more than the number of atoms in a human body). Even at a trillion checks per second, verifying by enumeration would take longer than the age of the universe.

This is the hopeless case. The output is just a number. There's no obvious certificate that proves the count is correct.

Or so it seems.

The breakthrough insight of interactive proofs is that through *interaction* and *randomness*, we can verify even #SAT efficiently. The prover doesn't give us a certificate; instead, we have a conversation that forces a lying prover to contradict themselves.

Polynomials are the key to making this work. They transform #SAT, this hopelessly unverifiable counting problem, into a series of polynomial identity checks where cheating is detectable with overwhelming probability.

We'll see exactly how in Chapter 3 when we study the sum-check protocol. But first, we need to understand where polynomials get this power.

Why Polynomials?

If you've read any paper on zero-knowledge proofs, you've noticed something striking: polynomials are *everywhere*. Witnesses become polynomial evaluations. Constraints become polynomial identities. Verification reduces to checking polynomial properties. The entire field seems obsessed with these algebraic objects.

This is not an accident. Polynomials possess a trinity of properties that make them uniquely suited for verifiable computation:

1. **Representation**: Any discrete data can be encoded as a polynomial
2. **Compression**: A million local constraints become one global identity
3. **Randomization**: The entire polynomial can be tested from a single random point

The rest of this chapter develops each pillar in turn.

Pillar 1: Representation - From Data to Polynomials

Any finite dataset can be encoded as a polynomial.

But first, we must define the terrain. Where do these polynomials live? Not in the real numbers. Remember the Patriot missile from Chapter 1: a rounding error of 0.000000095 seconds, accumulated over time, killed 28 soldiers. Real number arithmetic is treacherous. Equality is approximate, errors accumulate, and 0.1 has no exact binary representation.

Polynomials in ZK proofs live in *finite fields*, mathematical structures where arithmetic is exact. In a finite field, $1/3$ isn't $0.333...$; it's a precise integer. There's no rounding, no overflow, no approximation. Two values are either exactly equal or they're not. This exactness is what makes polynomial "rigidity" possible: if two polynomials differ, they differ exactly, and we can detect it.

It is a historical irony that this structure was discovered by someone who knew he was about to die. In May 1832, twenty-year-old Évariste Galois spent his final night frantically writing mathematics. He had been challenged to a duel the next morning and expected to lose. In those desperate hours, he outlined a new theory of algebraic symmetry, describing number systems that behaved like familiar arithmetic (you could add, subtract, multiply, and divide) but were finite. They didn't stretch to infinity; they looped back on themselves, like a clock.

The next morning, Galois was shot in the abdomen and died the following day. But his "finite fields" turned out to be the perfect environment for computation. Every SNARK, every polynomial commitment, and every error-correcting code in this book lives inside the structure Galois sketched the night before his death.

Two Ways to Encode Data

Given a vector $a = (a_1, a_2, \ldots, a_n)$ of field elements, we have two natural polynomial representations:

Coefficient Encoding: Treat the values as coefficients:

$$p_a(x) = a_1 + a_2 x + a_3 x^2 + \cdots + a_n x^{n-1}$$

This polynomial has degree at most $n - 1$. Its coefficients *are* the data. Evaluating $p_a(x)$ at any point r gives us a "fingerprint" of the entire vector: a single value that depends on all the data.

Evaluation Encoding: Treat the values as evaluations at fixed points. Find the unique polynomial $q_a(x)$ of degree at most $n - 1$ such that:

$$q_a(0) = a_1, \quad q_a(1) = a_2, \quad \ldots, \quad q_a(n-1) = a_n$$

This polynomial exists and is unique, a fact guaranteed by *Lagrange interpolation*, which we'll explore momentarily. Here, the data becomes "the shape of a curve that passes through specific points."

Both encodings are useful in different contexts. Coefficient encoding is natural for fingerprinting; evaluation encoding is natural when we want to extend a function defined on a small domain to a larger one.

Lagrange Interpolation: The Existence Guarantee

Why does a polynomial passing through n specified points always exist and why is it unique?

Picture a flexible curve that you need to pin down at specific points. With one point, infinitely many curves pass through it. With two points, you've constrained the curve more, but many still fit. The uniqueness guarantee: with n points, there's *exactly one* polynomial of degree at most $n - 1$ that passes through all of them. The points completely determine the curve.

Theorem (Lagrange Interpolation). Given n distinct points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ in a field $\mathbb{F}$, there exists a unique polynomial $p(x)$ of degree at most $n - 1$ such that $p(x_i) = y_i$ for all i.

Construction: Define the *Lagrange basis polynomials*:

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Each L_i has a special property: $L_i(x_i) = 1$ and $L_i(x_j) = 0$ for $j \neq i$. It's a polynomial that "activates" only at point x_i.

The interpolating polynomial is then:

$$p(x) = \sum_{i=1}^{n} y_i \cdot L_i(x)$$

Let's verify: at point x_k, we get $p(x_k) = \sum_i y_i \cdot L_i(x_k) = y_k \cdot 1 + \sum_{i \neq k} y_i \cdot 0 = y_k$.

Worked Example: Find the polynomial through $(0, 2), (1, 5), (2, 10)$.

The Lagrange basis polynomials:

$$L_0(x) = \frac{(x-1)(x-2)}{(0-1)(0-2)} = \frac{(x-1)(x-2)}{2}$$

$$L_1(x) = \frac{(x-0)(x-2)}{(1-0)(1-2)} = \frac{x(x-2)}{-1} = -x(x-2)$$

$$L_2(x) = \frac{(x-0)(x-1)}{(2-0)(2-1)} = \frac{x(x-1)}{2}$$

The interpolating polynomial:

$$p(x) = 2 \cdot \frac{(x-1)(x-2)}{2} + 5 \cdot (-x(x-2)) + 10 \cdot \frac{x(x-1)}{2}$$

Expanding (this is tedious but instructive):

$$p(x) = (x-1)(x-2) - 5x(x-2) + 5x(x-1)$$

$$= (x^2 - 3x + 2) - 5(x^2 - 2x) + 5(x^2 - x)$$

$$= x^2 - 3x + 2 - 5x^2 + 10x + 5x^2 - 5x$$

$$= x^2 + 2x + 2$$

Verification: $p(0) = 2$, $p(1) = 1 + 2 + 2 = 5$, $p(2) = 4 + 4 + 2 = 10$. All match.

Uniqueness: If two degree-$(n-1)$ polynomials p and q agree at n points, their difference $p - q$ is a polynomial of degree at most $n - 1$. But $p - q$ vanishes at each of the n points where p and q agree, meaning it has n roots. A nonzero polynomial of degree $n - 1$ can have at most $n - 1$ roots, so $p - q$ must be the zero polynomial. Therefore $p = q$.

The Rigidity of Polynomials

The key property that makes verification possible is this:

Two different degree-d polynomials can agree on at most d points.

Proof: Let p and q be distinct polynomials of degree at most d. Their difference $p - q$ is non-zero (since $p \neq q$) and has degree at most d. A non-zero polynomial of degree d has at most d roots. Therefore $p(x) = q(x)$ for at most d values of x. □

The statement reads like a routine algebra exercise until you weigh the bound against the size of the field.

If you and I each have a degree-99 polynomial, and they're *different* polynomials, then they can agree on at most 99 input values. Out of, say, 2^{256} possible inputs in a cryptographic field, they *disagree* on all but at most 99 of them.

This is *rigidity*. A polynomial can't "cheat locally." If a prover tries to construct a fake polynomial that agrees with the honest one at a few strategic points, the fake will disagree almost everywhere else.

Compare this to arbitrary functions. Two functions could agree on 99% of inputs and differ on just 1%. But a degree-99 polynomial that differs from another *anywhere* must differ on essentially *all* points. The disagreement isn't localized; it's smeared across the domain.

This rigidity has a striking consequence: you cannot construct a degree-d polynomial that matches another degree-d polynomial at strategically chosen points while differing elsewhere. If two degree-d polynomials differ at all, they differ almost everywhere. A local patch is impossible; any change propagates globally.

A polynomial cannot lie consistently. It must betray itself almost everywhere.

This property alone is purely mathematical. To turn it into a verification tool, we need one more ingredient: randomness.

Pillar 2: Randomization - The Schwartz-Zippel Lemma

In 1976, Gary Miller discovered a fast algorithm to test whether a number is prime. There was one problem: proving it correct required assuming the Riemann Hypothesis, one of the deepest unsolved problems in mathematics. Four years later, Michael Rabin found a way out. He modified Miller's test to use random sampling. The new algorithm couldn't *guarantee* the right answer, but it could make errors arbitrarily unlikely, say, less likely than a cosmic ray flipping a bit in your computer's memory. By embracing randomness, Rabin traded an unproven conjecture for a proven bound on failure probability.

Rabin's move treats randomness as a *resource* for verification. A cheating prover might fool a deterministic check, but fooling a random check requires being lucky, and we can make luck arbitrarily improbable.

The rigidity of polynomials becomes a verification tool through one of the most important theorems in computational complexity:

Schwartz-Zippel Lemma. Let $P(x_1, \dots, x_n)$ be a non-zero polynomial of total degree d over a field $\mathbb{F}$. If we choose $r_1, \dots, r_n$ uniformly at random from a finite subset $S \subseteq \mathbb{F}$, then:

$$\Pr[P(r_1, \dots, r_n) = 0] \leq \frac{d}{|S|}$$

In plain English: A non-zero polynomial almost never evaluates to zero at a random point, provided the field is much larger than the polynomial's degree.

Why This Is Profound

Consider verifying whether two polynomials $p(x)$ and $q(x)$ are equal. The naive approach: compare their coefficients one by one. If each polynomial has degree 1 million, that's a million comparisons.

Schwartz-Zippel offers a shortcut: pick a random r and check if $p(r) = q(r)$.

- If $p = q$: The check always passes.
- If $p \neq q$: The polynomial $p - q$ is non-zero with degree at most 1 million. By Schwartz-Zippel, $\Pr[p(r) = q(r)] = \Pr[(p - q)(r) = 0] \leq \frac{10^6}{|\mathbb{F}|}$.

In a field of size 2^{256}, this probability is about 2^{-236} (far smaller than the odds of guessing a 256-bit private key).

One random evaluation distinguishes degree-d polynomials with probability $1 - d/|\mathbb{F}|$.

A Proof Sketch

For a single variable, the proof is straightforward. A non-zero polynomial of degree d has at most d roots. If S has $|S|$ elements, the probability of hitting a root is at most $d/|S|$.

For multiple variables, the proof proceeds by induction. Write $P(x_1, \ldots, x_n)$ as a polynomial in x_1 with coefficients that are polynomials in $x_2, \ldots, x_n$:

$$P(x_1, x_2, \ldots, x_n) = \sum_{i=0}^{d_1} x_1^i \cdot Q_i(x_2, \ldots, x_n)$$

At least one coefficient polynomial is non-zero (otherwise $P = 0$). Call it Q_j. By induction, a random choice of $r_2, \ldots, r_n$ makes $Q_j(r_2, \ldots, r_n) \neq 0$ with probability at least $1-(d-d_1)/|S|$. Conditioned on this, $P(x_1, r_2, \ldots, r_n)$ is a non-zero univariate polynomial of degree at most d_1, so a random r_1 makes it zero with probability at most $d_1/|S|$. The union bound gives the result.

Application: Polynomial Fingerprinting for File Comparison

Consider a practical problem: Alice and Bob each have a massive file (think terabytes) and want to check if their files are identical. Sending entire files is prohibitively expensive. Can they compare with minimal communication?

The setup: Interpret each file as a vector of n field elements: $a = (a_1, \ldots, a_n)$ for Alice, $b = (b_1, \ldots, b_n)$

for Bob. Encode them as polynomials:

$$p_A(x) = \sum_{i=1}^{n} a_i x^{i-1}, \quad p_B(x) = \sum_{i=1}^{n} b_i x^{i-1}$$

The protocol:

1. Alice picks a random $r \in \mathbb{F}$
2. Alice computes her *fingerprint*: $v_A = p_A(r)$
3. Alice sends (r, v_A) to Bob (just two field elements!)
4. Bob computes $v_B = p_B(r)$ and checks if $v_A = v_B$

Analysis:

- **Completeness:** If $a = b$, the polynomials are identical, so $p_A(r) = p_B(r)$ always. Bob correctly accepts.
- **Soundness:** If $a \neq b$, then $p_A(x) - p_B(x)$ is non-zero with degree at most $n - 1$. By Schwartz-Zippel:
$$\Pr[p_A(r) = p_B(r)] = \Pr[(p_A - p_B)(r) = 0] \leq \frac{n-1}{|\mathbb{F}|}$$

They've compared a terabyte of data by exchanging two field elements. The probability of error is negligible.

A Worked Example with Actual Numbers:

Alice has $a = (2, 1)$, Bob has $b = (3, 5)$. Working in $\mathbb{F}_{11}$ (the field of integers modulo 11).

Their polynomials:

- $p_A(x) = 2 + 1 \cdot x = 2 + x$
- $p_B(x) = 3 + 5 \cdot x$

Alice picks $r = 7$:

- $p_A(7) = 2 + 7 = 9$
- $p_B(7) = 3 + 5 \cdot 7 = 3 + 35 = 38 \equiv 5 \pmod{11}$

Since $9 \neq 5$, Bob correctly concludes the vectors differ.

When would they collide? Only if $p_A(r) = p_B(r)$:

$$2 + r \equiv 3 + 5r \pmod{11}$$

$$-1 \equiv 4r \pmod{11}$$

$$10 \equiv 4r \pmod{11}$$

To solve for r, we need the multiplicative inverse of 4 modulo 11. Since $4 \cdot 3 = 12 \equiv 1 \pmod{11}$, we have $4^{-1} \equiv 3$.

So $r \equiv 3 \cdot 10 = 30 \equiv 8 \pmod{11}$.

The only "bad" random choice is $r = 8$. With 11 possible choices, the collision probability is exactly $1/11 \approx 9\%$. In a cryptographic field with 2^{256} elements, this would be $1/2^{256}$ (essentially zero).

Application: Batch Verification of Signatures

The same principle powers batch verification in signature schemes like Schnorr.

Recall that a Schnorr signature (R, s) on message m under public key P satisfies:

$$s \cdot G = R + e \cdot P$$

where $e = H(R, P, m)$ is the challenge hash. Verifying this requires two scalar multiplications.

Now suppose a node must verify 1000 signatures. Checking each individually costs 2000 scalar multiplications. Can we do better?

The batch verification trick: Take a random linear combination of all verification equations. If each equation $s_i G = R_i + e_i P_i$ holds, then for any coefficients z_i:

$$\left(\sum_i z_i s_i\right) G = \sum_i z_i R_i + \sum_i z_i e_i P_i$$

This is a single multi-scalar multiplication (MSM), dramatically faster than 1000 separate verifications using algorithms like Pippenger's.

Why random coefficients? If we just summed the equations ($z_i = 1$), an attacker could forge two invalid signatures whose errors cancel: one with error $+\Delta$, another with $-\Delta$. The batch check would pass, but individual signatures would fail.

Random z_i prevent this. If any signature is invalid, the batch equation becomes a non-zero polynomial in the z_i variables. By Schwartz-Zippel, random z_i satisfy a non-zero polynomial with negligible probability.

This is polynomial identity testing in disguise. The honest case gives the zero polynomial; any cheating gives a non-zero polynomial that random evaluation catches.

Arithmetic Consensus

Step back and notice something strange about what just happened.

In the fingerprinting protocol, Alice and Bob reached agreement about whether their files match. They didn't trust each other. They didn't consult a third party. They didn't negotiate or compare credentials. They simply evaluated polynomials at the same random point, and mathematics forced them to the same conclusion.

This is a new kind of agreement. Philosophers have long studied how agents come to share beliefs. The epistemology of testimony asks how we gain knowledge from what others tell us, and the answer always involves trust: we believe the speaker because of their reputation, authority, or our assessment of their incentives. Social epistemology studies how groups arrive at consensus, and the mechanisms are social: communication, persuasion, deference to experts.

Schwartz-Zippel enables something different. Two systems that share no trust relationship, that have never communicated, that know nothing about each other's reliability, can independently verify the same polynomial identity and reach the same conclusion. Not because they agreed to agree, but because the structure of low-degree polynomials leaves no room for disagreement. If $p \neq q$, then $p(r) \neq q(r)$ for almost all r. Any system capable of field arithmetic will detect the difference.

Call this **arithmetic consensus**: agreement forced by mathematical structure rather than achieved by social process. The boundaries of this regime are precise. Any statement reducible to polynomial identity testing can be verified this way. Any claim expressible as "these two low-degree polynomials are equal" becomes a claim that any arithmetic system can check, with the same answer guaranteed.

This relates to a tradition in the philosophy of mathematics. Intuitionism, developed by Brouwer in the early 20th century, held that a mathematical statement is meaningful only if we can construct a proof of it. For the intuitionist, "there exists an x" means "we can exhibit an x." Truth is inseparable from proof.

Arithmetic consensus takes a different but related position: for statements about polynomial identities, truth is inseparable from *verification*. The proof object (a random evaluation point and its result) doesn't require trust in whoever produced it. Any verifier running the same arithmetic reaches the same conclusion. This is intersubjective truth without intersubjectivity. The agreement happens not between minds but between any systems capable of arithmetic.

The applications in cryptography (verifiable computation, zero-knowledge proofs, blockchain consensus) are engineering achievements. But underneath them lies a philosophical shift: for a certain class of claims, we can replace "I trust the speaker" with "I checked the math." This is not a small thing. It's a new foundation for agreement in a world of untrusted parties.

Error-Correcting Codes: The Deeper Structure

The polynomial fingerprinting protocol is an instance of a deeper mathematical structure that appears throughout ZK proofs: **error-correcting codes**.

What Is an Error-Correcting Code?

Imagine sending a message through a noisy channel (think: radio transmission through interference, reading data from a scratched DVD, or communicating with a spacecraft). Some bits might get flipped. How do you ensure the receiver can still recover the original message?

The naive approach: send the message three times and take a majority vote. If you send "1" as "111" and one bit flips to "011," the receiver sees two 1s and one 0, guesses "1," and succeeds.

But this is inefficient; you've tripled your transmission length to correct one error.

Error-correcting codes provide a systematic way to add redundancy that can detect and correct errors far more efficiently.

The Key Definitions

An **(n, k, d) code** over alphabet Σ consists of:

- A set of **messages** of length k
- An **encoding function** that maps each message to a **codeword** of length $n > k$
- A **minimum distance** d: any two distinct codewords differ in at least d positions

The minimum distance determines the code's power:

- **Error detection:** Can detect up to $d - 1$ errors (if we see something that's not a valid codeword, we know an error occurred)
- **Error correction:** Can correct up to $\lfloor (d-1)/2 \rfloor$ errors (the corrupted codeword is still closest to the original)

Example: Repetition Code. Encode message bit b as bbb (repeat it 3 times). This is a (3, 1, 3) code: codewords are "000" and "111," which differ in all 3 positions. It can detect 2 errors and correct 1 error.

Reed-Solomon Codes: Polynomials as Codewords

The most important family of error-correcting codes for our purposes is the **Reed-Solomon code**, discovered by Irving Reed and Gustave Solomon in 1960.

Construction: Work over a field $\mathbb{F}$ with at least n elements. Choose n distinct evaluation points $\alpha_1, \ldots, \alpha_n \in \mathbb{F}$.

- **Messages:** Polynomials of degree at most $k-1$ (equivalently, vectors of k coefficients)
- **Encoding:** Evaluate the polynomial at all n points:

$$\text{Encode}(p) = (p(\alpha_1), p(\alpha_2), \ldots, p(\alpha_n))$$

- **Codewords:** Vectors of n field elements

The minimum distance: If $p \neq q$ are distinct polynomials of degree at most $k-1$, then $p-q$ is a non-zero polynomial of degree at most $k-1$, which has at most $k-1$ roots. Therefore p and q can agree on at most $k-1$ of the n evaluation points, meaning they *differ* on at least $n-(k-1) = n-k+1$ positions.

This gives an $(n, k, n-k+1)$ code: an optimal relationship between redundancy and distance, known as a *Maximum Distance Separable (MDS)* code.

Worked Example: Consider a Reed-Solomon code over $\mathbb{F}_{11}$ with $n = 7$ evaluation points $\{0, 1, 2, 3, 4, 5, 6\}$ and message polynomials of degree at most $k-1 = 2$ (so $k = 3$).

Message: the polynomial $p(x) = 2 + 3x + x^2$ (coefficients $[2, 3, 1]$).

Codeword: evaluate at each point:

- $p(0) = 2$
- $p(1) = 2 + 3 + 1 = 6$
- $p(2) = 2 + 6 + 4 = 12 \equiv 1 \pmod{11}$
- $p(3) = 2 + 9 + 9 = 20 \equiv 9 \pmod{11}$
- $p(4) = 2 + 12 + 16 = 30 \equiv 8 \pmod{11}$
- $p(5) = 2 + 15 + 25 = 42 \equiv 9 \pmod{11}$
- $p(6) = 2 + 18 + 36 = 56 \equiv 1 \pmod{11}$

Codeword: $(2, 6, 1, 9, 8, 9, 1)$.

The minimum distance is $n - k + 1 = 7 - 3 + 1 = 5$. Any two codewords differ in at least 5 positions. This code can correct up to $\lfloor 4/2 \rfloor = 2$ errors.

Why Reed-Solomon Codes Power ZK Proofs

The connection to zero-knowledge proofs is now clear:

	Error-Correcting Codes	ZK Proof Systems
Message		Witness (prover's secret)
Encoding		Polynomial evaluation over large domain
Codeword		Prover's committed values
Distance property		Cheating changes most of the codeword
Random sampling		Verifier's random challenges

In ZK:

1. The prover's witness w is encoded as a polynomial p_w.
2. The polynomial is "committed" by evaluating it at many points (or via a polynomial commitment scheme).
3. The verifier samples random points and checks consistency.
4. If the prover cheated (wrong witness), the polynomial won't satisfy required properties, and this corruption spreads across almost all evaluation points due to the Reed-Solomon distance property.

Reed-Solomon encoding is *distance-amplifying*. A small, localized lie (wrong witness value) becomes a large, detectable corruption (wrong polynomial evaluations everywhere).

Real-World Applications of Reed-Solomon

Before we move on, it's worth appreciating how ubiquitous Reed-Solomon codes are:

- **QR codes:** The chunky squares on product labels use Reed-Solomon to remain readable even when partially obscured or damaged.
- **CDs, DVDs, Blu-rays:** Scratches that destroy data are corrected by Reed-Solomon coding.
- **Deep-space communication:** Voyager, Cassini, and other spacecraft use Reed-Solomon codes to send data across billions of miles despite noise and signal degradation.
- **RAID storage:** Disk arrays use Reed-Solomon to survive drive failures.

- **Digital television (DVB):** Broadcast signals use Reed-Solomon to handle transmission errors.

The same mathematical structure that lets your scratched DVD still play a movie is what lets ZK proofs detect a lying prover from a single random query.

Pillar 3: Compression - From Many Constraints to One

We've seen how polynomials encode data and how random sampling detects differences. The third pillar explains how polynomials let us *aggregate* many checks into one.

The Compression Problem

A computation consists of many local constraints. Consider a circuit with a million gates. Each multiplication gate with inputs a and b and output c imposes a constraint: $a \cdot b = c$.

Checking all million constraints individually takes a million operations. Can we do better?

We can aggregate all of them into a single polynomial identity.

The Vanishing Polynomial Technique

Suppose we have n constraints that should all equal zero:

$$C_1 = 0, \quad C_2 = 0, \quad \dots, \quad C_n = 0$$

Step 1: Encode as a polynomial. Find a polynomial $C(x)$ such that:

- $C(1) = C_1$
- $C(2) = C_2$
- ...
- $C(n) = C_n$

Step 2: The equivalence. The statement "all constraints are satisfied" is equivalent to:

$$C(x) = 0 \text{ for all } x \in \{1, 2, \dots, n\}$$

Step 3: Use the Factor Theorem. The polynomial $C(x)$ equals zero at points $\{1, 2, \dots, n\}$ if and only if $C(x)$ is divisible by the *vanishing polynomial*:

$$Z(x) = (x - 1)(x - 2) \cdots (x - n)$$

Think of $Z(x)$ as a stencil with holes at $x = 1, 2, \ldots, n$. If $C(x)$ truly equals zero at those points, it passes through the holes perfectly: the division $C(x)/Z(x)$ comes out clean with no remainder. If $C(x)$ misses even one hole (nonzero at some constraint point), it hits the stencil, and the division leaves a remainder. The polynomial doesn't fit.

Step 4: The divisibility test. The statement "all constraints are satisfied" becomes: there exists a *quotient polynomial* $H(x)$ such that:

$$C(x) = H(x) \cdot Z(x)$$

Step 5: Random verification. By Schwartz-Zippel, if this identity holds everywhere, it holds at a random point r with high probability. Conversely, if it fails anywhere, it fails at r with probability $1 - d/|\mathbb{F}|$.

So the verifier only needs to check:

$$C(r) \stackrel{?}{=} H(r) \cdot Z(r)$$

A million local checks become one divisibility test, verified at a single random point.

A Worked Example: Three Constraints

Let's see this concretely. Suppose we have three constraints that should be zero:

- $C_1 = 0$ (at $x = 1$)
- $C_2 = 0$ (at $x = 2$)
- $C_3 = 0$ (at $x = 3$)

Working in $\mathbb{F}_{17}$, suppose an honest prover has $C_1 = C_2 = C_3 = 0$, so $C(x)$ is the zero polynomial on $\{1, 2, 3\}$.

The vanishing polynomial:

$$Z(x) = (x-1)(x-2)(x-3) = x^3 - 6x^2 + 11x - 6$$

If all constraints are satisfied, $C(x) = H(x) \cdot Z(x)$ for some $H(x)$.

Now suppose a cheating prover has $C_1 = 0$, $C_2 = 5$ (wrong!), $C_3 = 0$. The polynomial $C(x)$ passes through $(1, 0), (2, 5), (3, 0)$.

Using Lagrange interpolation:

$$C(x) = 0 \cdot L_1(x) + 5 \cdot L_2(x) + 0 \cdot L_3(x) = 5 \cdot L_2(x)$$

where $L_2(x) = \frac{(x-1)(x-3)}{(2-1)(2-3)} = \frac{(x-1)(x-3)}{-1} = -(x-1)(x-3) = -x^2 + 4x - 3$.

So $C(x) = 5(-x^2 + 4x - 3) = -5x^2 + 20x - 15 \equiv 12x^2 + 3x + 2 \pmod{17}$.

Is this divisible by $Z(x) = (x-1)(x-2)(x-3)$? Let's check: $C(2) = 12 \cdot 4 + 3 \cdot 2 + 2 = 48 + 6 + 2 = 56 \equiv 5 \pmod{17}$.

Since $C(2) = 5 \neq 0$, but $Z(2) = 0$, the division $C(x)/Z(x)$ would have a pole at $x = 2$. The divisibility fails, and no valid quotient $H(x)$ exists.

The verifier, picking a random r, will find that $C(r) \neq H(r) \cdot Z(r)$ for any claimed H with overwhelming probability.

Freivald's Algorithm: Polynomials in Disguise

Let's examine a beautiful algorithm that shows the polynomial paradigm in a surprising context: verifying matrix multiplication.

The Problem

Given three $n \times n$ matrices A, B, and C, determine whether $C = A \cdot B$.

The naive approach: Compute $A \cdot B$ directly and compare with C. Using the standard algorithm, this takes $O(n^3)$ multiplications. Even with the fastest known algorithm (Strassen's descendants), it's $O(n^{2.37\ldots})$ (still much worse than $O(n^2)$).

If we're trying to *verify* that someone else computed the product correctly, do we really need to redo all their work?

Freivald's Insight (1977)

Rüdiger Freivald proposed a remarkably simple test:

1. Pick a random vector $\vec{x} \in \mathbb{F}^n$
2. Compute $\vec{y} = B\vec{x}$ (one matrix-vector product: $O(n^2)$)
3. Compute $\vec{z} = A\vec{y} = A(B\vec{x})$ (another matrix-vector product: $O(n^2)$)
4. Compute $\vec{w} = C\vec{x}$ (another matrix-vector product: $O(n^2)$)
5. Check if $\vec{z} = \vec{w}$

Total work: Three matrix-vector products, so $O(n^2)$ (a full factor of n faster than matrix multiplication!).

Why It Works

If $C = AB$: Then $C\vec{x} = AB\vec{x} = A(B\vec{x})$, so $\vec{w} = \vec{z}$ always. The test passes.

If $C \neq AB$: Let $D = C - AB \neq 0$. The test passes only if $D\vec{x} = 0$.

Since $D \neq 0$, at least one row of D is non-zero. Call it row i, with entries $(d_{i,1}, d_{i,2}, \ldots, d_{i,n})$ not all zero.

The i-th component of $D\vec{x}$ is:

$$(D\vec{x})_i = d_{i,1}x_1 + d_{i,2}x_2 + \cdots + d_{i,n}x_n$$

This is a linear polynomial in the variables $x_1, \ldots, x_n$. For this polynomial to equal zero, we need:

$$d_{i,1}x_1 + d_{i,2}x_2 + \cdots + d_{i,n}x_n = 0$$

If we pick each x_j uniformly at random from $\mathbb{F}$, what's the probability this equation holds?

Claim: For a non-zero linear polynomial over $\mathbb{F}$, a random input is a root with probability exactly $1/|\mathbb{F}|$.

Proof: Suppose $d_{i,k} \neq 0$ for some k. We can rewrite:

$$x_k = -\frac{1}{d_{i,k}}\left(d_{i,1}x_1 + \cdots + d_{i,k-1}x_{k-1} + d_{i,k+1}x_{k+1} + \cdots + d_{i,n}x_n\right)$$

For any fixed choice of $x_1, \ldots, x_{k-1}, x_{k+1}, \ldots, x_n$, there's exactly one value of x_k that makes the sum zero. Since x_k is chosen uniformly from $|\mathbb{F}|$ possibilities, the probability of hitting that one value is $1/|\mathbb{F}|$. □

So with a single random vector, Freivald's algorithm detects incorrect matrix multiplication with probability at least $1 - 1/|\mathbb{F}|$.

Amplifying Confidence

If $1/|\mathbb{F}|$ isn't small enough, we can repeat with independent random vectors:

1. Pick k independent random vectors $\vec{x}^{(1)}, \ldots, \vec{x}^{(k)}$
2. For each, check if $A(B\vec{x}^{(i)}) = C\vec{x}^{(i)}$
3. Accept if all checks pass

If $C \neq AB$, each check passes with probability at most $1/|\mathbb{F}|$, and the checks are independent. So:

$$\Pr[\text{all } k \text{ checks pass} \mid C \neq AB] \leq (1/|\mathbb{F}|)^k = 1/|\mathbb{F}|^k$$

With $|\mathbb{F}| = 2^{64}$ and $k = 4$ repetitions, the false acceptance probability is 2^{-256} (cryptographically negligible).

Freivald's Algorithm as Polynomial Identity Testing

Here's the connection to polynomials that might not be immediately obvious.

Consider the matrices A, B, C as defining a polynomial identity. The claim $C = AB$ is equivalent to the matrix identity:

$$C - AB = 0$$

We can view each entry $(C - AB)_{ij}$ as a polynomial in the entries of the matrices. The test $D\vec{x} = 0$ is checking that a related set of linear polynomials (one for each row of D) all vanish at the random point $\vec{x}$.

More directly: the expression $\vec{x}^T D\vec{y}$ for random vectors $\vec{x}, \vec{y}$ defines a bilinear polynomial in the entries of D. This polynomial is non-zero if and only if $D \neq 0$. By a bilinear version of Schwartz-Zippel, random inputs make a non-zero bilinear form non-zero with high probability.

Freivald's test is polynomial identity testing in disguise.

This is a recurring theme: many efficient verification algorithms, when analyzed carefully, turn out to be checking polynomial identities via random evaluation.

A Complete Worked Example

Let's verify a matrix multiplication over $\mathbb{F}_7$.

$$A = \begin{pmatrix} 2 & 1 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 \\ 5 & 3 \end{pmatrix}$$

First, the honest computation:

$$AB = \begin{pmatrix} 2 \cdot 1 + 1 \cdot 5 & 2 \cdot 2 + 1 \cdot 3 \\ 3 \cdot 1 + 4 \cdot 5 & 3 \cdot 2 + 4 \cdot 3 \end{pmatrix} = \begin{pmatrix} 7 & 7 \\ 23 & 18 \end{pmatrix} \equiv \begin{pmatrix} 0 & 0 \\ 2 & 4 \end{pmatrix} \pmod 7$$

Suppose the prover claims $C = \begin{pmatrix} 0 & 0 \\ 2 & 4 \end{pmatrix}$ (correct).

Pick random $\vec{x} = (3, 5)^T$.

Compute $B\vec{x}$:

$$B\vec{x} = \begin{pmatrix} 1 & 2 \\ 5 & 3 \end{pmatrix} \begin{pmatrix} 3 \\ 5 \end{pmatrix} = \begin{pmatrix} 3 + 10 \\ 15 + 15 \end{pmatrix} = \begin{pmatrix} 13 \\ 30 \end{pmatrix} \equiv \begin{pmatrix} 6 \\ 2 \end{pmatrix} \pmod 7$$

Compute $A(B\vec{x})$:

$$A(B\vec{x}) = \begin{pmatrix} 2 & 1 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 6 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 + 2 \\ 18 + 8 \end{pmatrix} = \begin{pmatrix} 14 \\ 26 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 5 \end{pmatrix} \pmod 7$$

Compute $C\vec{x}$:

$$C\vec{x} = \begin{pmatrix} 0 & 0 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 5 \end{pmatrix} = \begin{pmatrix} 0 \\ 6 + 20 \end{pmatrix} = \begin{pmatrix} 0 \\ 26 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 5 \end{pmatrix} \pmod 7$$

Since $A(B\vec{x}) = (0, 5)^T = C\vec{x}$, the test passes.

Now suppose a cheating prover claims $C' = \begin{pmatrix} 0 & 1 \\ 2 & 4 \end{pmatrix}$ (wrong in position (1,2)).

With the same $\vec{x} = (3, 5)^T$:

Compute $C'\vec{x}$:

$$C'\vec{x} = \begin{pmatrix} 0 & 1 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 5 \end{pmatrix} = \begin{pmatrix} 5 \\ 26 \end{pmatrix} \equiv \begin{pmatrix} 5 \\ 5 \end{pmatrix} \pmod 7$$

We have $A(B\vec{x}) = (0, 5)^T \neq (5, 5)^T = C'\vec{x}$.

The test fails, catching the cheater.

Beyond Schwartz-Zippel: Why Polynomials Are Uniquely Suited

You might wonder: could we use other functions besides polynomials? What makes them special?

1. Low-Degree Extension

Pillar 1 established that any finite dataset can be encoded as a polynomial via Lagrange interpolation. The cryptographic payoff is the *low-degree extension*: given a function f defined on a small domain like $\{0,1\}^n$ (just 2^n points), we can extend it to a unique low-degree polynomial over the entire field (potentially 2^{256} points). The extension is determined: there's exactly one degree-(2^n-1) polynomial that agrees with f on the Boolean hypercube. This is the foundation of the sum-check protocol and the GKR protocol. Compare this to a hash function $H : \mathbb{F} \to \mathbb{F}$, which can take any value at any input. Knowing H at a million points tells you nothing about H at the next point. There's no interpolation, no structure to exploit.

2. Efficient Evaluation

Given a polynomial's coefficients, we can compute its value at any point in $O(d)$ time using Horner's method:

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{d-1} + x \cdot a_d)\cdots))$$

This is d multiplications and d additions (optimal).

3. Homomorphic Structure

Polynomials form a *ring*: we can add and multiply them, and these operations correspond to coefficient-wise operations. This algebraic structure is what makes polynomial commitment schemes like KZG possible. They allow us to verify polynomial relationships "in the exponent" without revealing the polynomials themselves. If we commit to $p(x)$ and $q(x)$, we can check $p(x) + q(x) = r(x)$ without learning any coefficients.

4. FFT Speedups

Over special domains, specifically the n-th roots of unity in a field, polynomial evaluation and interpolation can be performed in $O(n \log n)$ time via the *Fast Fourier Transform*.

Without FFT, evaluating a degree-n polynomial at n points takes $O(n^2)$ operations. With FFT over roots of unity, it's $O(n \log n)$.

This speedup is necessary for practical ZK systems. Prover complexity in many SNARKs is dominated by FFT operations.

5. Composability

Polynomials compose predictably:

- If p has degree d_p and q has degree d_q, then $p(q(x))$ has degree $d_p \cdot d_q$
- Products $p \cdot q$ have degree $d_p + d_q$
- Sums $p + q$ have degree $\max(d_p, d_q)$

This predictability enables rigorous protocol analysis. When the verifier asks for $p(r) \cdot q(r)$, they know the result should come from a polynomial of degree $d_p + d_q$, and can set the soundness parameters accordingly.

The Polynomial Paradigm: A Unified View

We can now state the polynomial paradigm that underlies nearly all modern ZK proofs:

1. **Represent** the computation as polynomials: witness values, constraint evaluations, everything becomes polynomial data
2. **Compress** many constraints into a single polynomial identity, typically a divisibility condition or a summation equality
3. **Randomize** to check the identity: evaluate at random points, relying on Schwartz-Zippel to catch any cheating

This paradigm appears in every major ZK system:

- **Groth16:** R1CS constraints become a QAP divisibility check: $L(x) \cdot R(x) - O(x) = H(x) \cdot Z(x)$
- **PLONK:** Gate constraints and wiring constraints become polynomial identities checked via random challenges
- **STARKs:** AIR constraints become low-degree polynomial conditions verified by the FRI protocol
- **Sum-check:** Summation claims over exponentially many terms reduce to a single polynomial evaluation

Key takeaways

1. **The counting problem (#SAT) motivates why polynomials matter:** Some computations have no obvious short certificate, but polynomial encodings enable efficient verification through interaction and randomness.

2. **Polynomials encode data:** Any finite dataset becomes a polynomial through coefficient encoding (data = coefficients) or evaluation encoding (data = values at fixed points). Lagrange interpolation guarantees this encoding exists and is unique.

3. **Polynomials are rigid:** Two different degree-d polynomials agree on at most d points. Local differences become global differences; you can't cheat in one place without affecting almost everywhere.

4. **Schwartz-Zippel enables efficient testing:** A non-zero polynomial evaluates to zero at a random point with probability at most $d/|\mathbb{F}|$. For cryptographic fields, this is negligible.

5. **This is an error-correcting code:** The polynomial paradigm is the Reed-Solomon code applied to computation verification. A small lie in the witness becomes corruption across essentially all evaluation points.

6. **Freivald's algorithm is polynomial identity testing:** Matrix multiplication verification in $O(n^2)$ time (instead of $O(n^3)$) works because it's checking linear polynomial identities via random evaluation.

7. **Constraints compress to identities:** Many local constraints become a single polynomial divisibility condition: $C(x) = H(x) \cdot Z(x)$ where Z is the vanishing polynomial.

8. **The structure is unique:** Polynomials combine efficient evaluation, unique interpolation, homomorphic properties, FFT speedups, and composability in ways no other mathematical object does.

9. **The paradigm is universal:** Every major ZK system (Groth16, PLONK, STARKs, sum-check) uses the same three-step approach: represent as polynomials, compress constraints to identities, verify via random evaluation.

10. **Commitment + evaluation = proof architecture:** Committing to a polynomial locks the prover to a single function; random evaluation checks that function is correct. This commit-then-evaluate pattern is the skeleton of every modern SNARK.

Chapter 3: The Sum-Check Protocol

In late 1989, the field of complexity theory was stuck.

Researchers believed that Interactive Proofs were a relatively weak tool, capable of verifying only a handful of graph problems. The consensus was clear: interaction helped, but not by much.

Then came the email.

Noam Nisan, a master's student at Hebrew University, sent a draft to Lance Fortnow at the University of Chicago. It contained a protocol that used polynomials to verify something thought impossible: the permanent of a matrix. Fortnow showed it to his colleagues Howard Karloff and Carsten Lund. They realized the technique didn't just apply to matrices. It applied to *everything* in the polynomial hierarchy.

When the paper was released, it didn't just solve a problem. It caused a crisis. The result implied that "proofs" were far more powerful than anyone had imagined. Within weeks, Adi Shamir (the "S" in RSA) used the same technique to prove IP = PSPACE: interactive proofs could verify any problem solvable with polynomial memory, even if finding the solution took eons.

The engine powering this revolution was a single elegant idea: the **sum-check protocol**.

The sum-check protocol takes a claim that seems expensive to verify, the sum of a polynomial over all points of a high-dimensional domain, and reduces it to something trivial: a single evaluation at a random point. The verifier's work scales *linearly* with the number of variables, not exponentially with the size of the domain.

This chapter develops the sum-check protocol from first principles. We'll see exactly how the protocol works, why it's sound, and how any lie propagates through the protocol until it becomes a simple falsehood the verifier can catch. Along the way, we'll trace through complete worked examples with actual field values, because this protocol is too important to understand only abstractly.

The protocol requires only basic polynomial facts from Chapter 2 (Schwartz-Zippel, evaluation, degree). The next two chapters develop the polynomial representations used in practice: multilinear extensions (Chapter 4) enable linear-time provers and scale to domains of size 2^{128}, while univariate techniques (Chapter 5) offer smaller proofs via FFT-friendly structure. Sum-check itself is agnostic to representation; it works on any multivariate polynomial.

The Problem: Verifying Exponential Sums

Suppose a prover claims to know the value of the following sum:

$$H = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \cdots \sum_{b_\nu \in \{0,1\}} g(b_1, b_2, \ldots, b_\nu)$$

Here g is a ν-variate polynomial over a finite field $\mathbb{F}$, and the sum ranges over all 2^ν points of the **boolean hypercube** $\{0,1\}^\nu$: the set of all binary strings of length ν. Think of it as the corners of a ν-dimensional cube, where each coordinate is either 0 or 1. For $\nu = 3$, these are the eight vertices $(0,0,0), (0,0,1), \ldots, (1,1,1)$. The prover says the answer is H. Do you believe it?

A naive verifier would evaluate g at every point of the hypercube and add up the results. But this requires 2^ν evaluations, exponential in the number of variables. For $\nu = 100$, this is hopelessly infeasible.

The sum-check protocol solves this problem. It allows a verifier to check the claimed value of H with high probability, in time that is only linear in ν and the time it takes to evaluate g at a *single* random point. This represents an exponential speedup.

But how can you verify a sum without computing it? The answer lies in a beautiful idea: **claim reduction via deferred evaluation**. Instead of computing the sum directly, the verifier engages in a multi-round dialogue with the prover. In each round, the prover makes a smaller, more specific claim, and the verifier uses randomness to drill down on a single point. An initial lie, no matter how cleverly constructed, gets amplified at each step until it becomes a simple falsehood about a single evaluation, which the verifier catches at the end.

The Compression Game

Think of the sum-check protocol as a game of progressive compression, or better yet, as a police interrogation.

The suspect (prover) claims to have an alibi for every minute of a 24-hour day (2^ν moments). The detective (verifier) cannot review surveillance footage for the entire day. Instead, the detective asks for a summary: "Tell me the sum of your activities."

The suspect provides a summary polynomial.

The detective picks one random second (r_1) and asks: "Explain this specific moment in detail." To answer, the suspect must provide a new summary for that specific timeframe. If the suspect lied about the total day, they must now lie about that specific second to make the math add up. The detective drills down again: "Okay, explain this millisecond."

The lie has to move. It has to hide in smaller and smaller gaps. Eventually, the detective asks about a single instant that can be fact-checked directly. If the suspect's story at that final instant doesn't match the evidence, the whole alibi crumbles.

More precisely: the prover holds an enormous object, a table of 2^ν values. The verifier wants to know their sum but cannot afford to examine the table. In round 1, the prover compresses the table into a univariate polynomial. The verifier probes it at a random point r_1, and that answer becomes the new target: a compressed representation of a table half the size.

Each round, the table shrinks by half while the verifier accumulates random coordinates. After ν rounds, the "table" has size 1: a single value. The verifier can compute that value herself.

Honest compression is consistent, but lies leave fingerprints. If the prover's initial polynomial doesn't represent the true sum, it must differ from the honest polynomial somewhere. The random probes find these differences with overwhelming probability. A cheating prover would need to predict all ν random challenges in advance; against cryptographic randomness, that's impossible.

The Protocol Specification

Let's make this precise. The sum-check protocol verifies a claim of the form:

$$H = \sum_{(b_1,\ldots,b_\nu)\in\{0,1\}^\nu} g(b_1, \ldots, b_\nu)$$

where g is a ν-variate polynomial of degree at most d in each variable. (More generally, each variable X_j can have its own degree bound d_j; we use uniform d for simplicity.) The verifier must know these degree bounds before the protocol begins. They're part of the problem specification, not something the prover provides. If the prover could claim arbitrary degree bounds, soundness would collapse: a high-degree polynomial can pass through any finite set of points while matching an honest polynomial elsewhere. The sum ranges over boolean points, but g is a polynomial over the field $\mathbb{F}$ and can have degree greater than 1. For example, $g(X_1, X_2) = X_1^2 + X_2^2 + X_1X_2$ has $d = 2$; when we sum over $\{0, 1\}^2$, we get $g(0, 0) + g(0, 1) + g(1, 0) + g(1, 1) = 0 + 1 + 1 + 3 = 5$. The degree bound d matters because it determines how many coefficients the prover must send in each round: a degree-d univariate polynomial requires $d + 1$ coefficients. The protocol proceeds in ν rounds.

Round 1

The prover computes and sends a univariate polynomial $g_1(X_1)$, claimed to equal:

$$g_1(X_1) = \sum_{(b_2,\ldots,b_\nu)\in\{0,1\}^{\nu-1}} g(X_1, b_2, \ldots, b_\nu)$$

In words: g_1 is the polynomial obtained by summing g over all boolean values of the last $\nu-1$ variables, leaving X_1 as a formal variable.

The verifier performs two checks:

1. **Consistency check**: Verify that $g_1(0) + g_1(1) = H$. This ensures the prover's polynomial is consistent with the claimed total sum.
2. **Degree check**: Verify that g_1 has degree at most d in X_1. This is necessary for soundness; without it, the protocol breaks completely. (We'll see why shortly.)

If either check fails, the verifier rejects. Otherwise, she samples a random field element $r_1 \leftarrow \mathbb{F}$ and sends it to the prover.

The verifier now evaluates the prover's polynomial at this random point, computing $V_1 = g_1(r_1)$. This value represents what the prover is *implicitly* asserting about a reduced sum. The verifier doesn't compute this sum herself; she simply records what the prover's polynomial claims it to be. This V_1 becomes the target for round 2: the prover must now justify that the sum over $2^{\nu-1}$ points, with the first variable fixed to r_1, actually equals V_1.

The verifier has now *reduced* the original claim about a sum over 2^ν points to a new claim about a sum over $2^{\nu-1}$ points. Specifically, the prover is now implicitly claiming that:

$$g_1(r_1) = \sum_{(b_2,\ldots,b_\nu)\in\{0,1\}^{\nu-1}} g(r_1, b_2, \ldots, b_\nu)$$

Why the degree check matters: The soundness argument relies on Schwartz-Zippel: two *distinct* degree-d polynomials agree on at most d points, so a random evaluation catches the difference with probability $\geq 1 - d/|\mathbb{F}|$. But what if the prover sends a high-degree polynomial instead?

Suppose the true sum is $H^* = 6$ but the prover claims $H = 100$. The honest polynomial is $s_1(X) = 2X + 2$, with $s_1(0) + s_1(1) = 6$. The prover needs a polynomial passing through $(0, a)$ and $(1, b)$ where $a + b = 100$.

Without a degree bound, the cheating prover can construct a degree-$(|\mathbb{F}|-1)$ polynomial g_1 with two properties:

1. $g_1(0) + g_1(1) = 100$ (passes the consistency check with the false claim $H = 100$)
2. $g_1(r) = s_1(r)$ for every $r \notin \{0, 1\}$ (agrees with the honest polynomial everywhere the verifier might query)

A degree-$(|\mathbb{F}|-1)$ polynomial has $|\mathbb{F}|$ coefficients, enough freedom to prescribe its value at every point in $\mathbb{F}$ independently. So the prover sets $g_1(0) = a$, $g_1(1) = b$ (with $a + b = 100$), and $g_1(r) = s_1(r)$ for all other r.

Here is why this is devastating. The verifier samples some $r_1 \notin \{0, 1\}$ and computes the reduced claim $V_1 = g_1(r_1)$. Since $g_1(r_1) = s_1(r_1)$, this reduced claim is *correct*: it equals the true partial sum $\sum_{b_2,\ldots,b_\nu} g(r_1, b_2, \ldots, b_\nu)$. From this point on, the prover can follow the honest protocol for rounds 2 through ν, and the final oracle check will pass. The false sum $H = 100$ was injected in round 1, but the cheat left no trace in any subsequent round.

The degree bound is the handcuffs. It forces the polynomial to be *stiff*. If it must pass through the wrong sum, its stiffness forces it to miss the honest polynomial almost everywhere else.

Formal argument: Suppose the prover sends $g_1 \neq s_1$ with $\deg(g_1) \leq d$. The difference $g_1 - s_1$ is a non-zero polynomial of degree at most d, so it has at most d roots. Therefore g_1 and s_1 agree on at most d points. When the verifier samples r_1 uniformly from $\mathbb{F}$, the probability that $g_1(r_1) = s_1(r_1)$ is at most $d/|\mathbb{F}|$. If $g_1(r_1) \neq s_1(r_1)$, the cheating prover is now committed to a false claim that propagates through subsequent rounds.

Round j (for $j = 2, \dots, \nu$)

At the start of round j, the verifier holds a value $V_{j-1} = g_{j-1}(r_{j-1})$ from the previous round. This represents the prover's implicit claim about a sum over $2^{\nu-j+1}$ points.

The prover sends the next univariate polynomial $g_j(X_j)$, claimed to equal:

$$g_j(X_j) = \sum_{(b_{j+1},\dots,b_\nu)\in\{0,1\}^{\nu-j}} g(r_1, \dots, r_{j-1}, X_j, b_{j+1}, \dots, b_\nu)$$

The verifier checks:

1. **Consistency check**: $g_j(0) + g_j(1) = V_{j-1}$
2. **Degree check**: $\deg(g_j) \leq d$

If checks pass, she samples $r_j \leftarrow \mathbb{F}$ and computes $V_j = g_j(r_j)$.

Final Check (After Round ν)

After ν rounds, the verifier has received $g_\nu(X_\nu)$ and chosen r_ν. The prover's final claim is that $g_\nu(r_\nu) = g(r_1, \dots, r_\nu)$.

The verifier now evaluates g at the single point $(r_1, \dots, r_\nu)$, using her "oracle access" to g, and checks whether this equals $g_\nu(r_\nu)$.

If the values match, she accepts. Otherwise, she rejects.

A Note on Oracle Access

In complexity theory, we say the verifier has "oracle access" to g. Sometimes this is trivial: if g encodes a multiplication gate, the verifier knows that $g(a, b) = a \cdot b$ and just plugs in the random values $r_1, \dots, r_\nu$. No magic needed.

But in many SNARK constructions, g depends on the prover's *private data*. The verifier cannot evaluate g on her own, because she doesn't know the inputs that define it. Sum-check has done its job,

reducing an exponential sum to one evaluation, but the verifier is stuck at the last step. How this gap is closed (using polynomial commitment schemes) is a central question we return to in Chapter 9.

Why Does This Work?

Completeness

If the prover is honest, all checks pass trivially. The polynomials g_j are computed exactly as specified, so the consistency checks hold by construction. The verifier accepts.

Soundness

The soundness argument is more subtle and relies on the **polynomial rigidity** we developed in Chapter 2.

Suppose the prover's initial claim is false: the true sum is $H^* \neq H$. For the first consistency check to pass, the prover must send some polynomial $g_1(X_1)$ such that $g_1(0) + g_1(1) = H$.

Let $s_1(X_1)$ be the *true* polynomial: the one computed by honestly summing g over the hypercube. By assumption, $s_1(0) + s_1(1) = H^* \neq H$. So the prover's polynomial g_1 must be different from s_1.

This is exactly where rigidity traps the cheater. The prover wants to send a polynomial that passes through the lie (H) but behaves like the truth (H^*) everywhere else. Rigidity makes this impossible. The polynomial is too stiff: if $g_1 \neq s_1$, they can agree on at most d points.

By the Schwartz-Zippel lemma, when the verifier samples a random r_1 from $\mathbb{F}$, the probability that $g_1(r_1) = s_1(r_1)$ is at most $d/|\mathbb{F}|$.

With overwhelming probability, $g_1(r_1) \neq s_1(r_1)$. The prover has "gotten lucky" only if the random challenge happened to land on one of the few points where the two polynomials agree.

But what does $g_1(r_1) \neq s_1(r_1)$ mean? It means the prover is now committed to defending a *false* claim in round 2: he must convince the verifier that the sum $\sum_{b_2,\ldots} g(r_1, b_2, \ldots)$ equals $g_1(r_1)$, when in fact it equals $s_1(r_1)$.

The same logic cascades through all ν rounds. In each round, either the prover gets lucky (probability $\leq d/|\mathbb{F}|$) or he's forced to defend a new false claim. By the final round, the prover must convince the verifier that $g_\nu(r_\nu) = g(r_1, \ldots, r_\nu)$, but the verifier checks this directly.

By a union bound, the total probability that a cheating prover succeeds is at most:

$$\delta_s \leq \frac{\nu \cdot d}{|\mathbb{F}|}$$

In cryptographic applications, $|\mathbb{F}|$ is enormous (e.g., 2^{256}), so this probability is negligible.

Worked Example: Honest Prover and Cheating Prover

Let's trace through the entire protocol with actual values: first with an honest prover, then with a cheater. Seeing both cases with the same polynomial makes the soundness argument concrete.

Setup: Consider the polynomial $g(x_1, x_2) = x_1 + 2x_2$ over a large field $\mathbb{F}$. We have $\nu = 2$ variables.

Goal: The prover wants to convince the verifier of the sum over $\{0,1\}^2$:

$$H = g(0,0) + g(0,1) + g(1,0) + g(1,1) = 0 + 2 + 1 + 3 = 6$$

The Honest Case

Round 1: The prover claims $H = 6$ and sends:

$$g_1(X_1) = g(X_1, 0) + g(X_1, 1) = (X_1 + 0) + (X_1 + 2) = 2X_1 + 2$$

The verifier checks: $g_1(0) + g_1(1) = 2 + 4 = 6 = H$. ✓

She samples $r_1 = 5$ and computes $V_1 = g_1(5) = 12$.

Round 2: The prover sends $g_2(X_2) = g(5, X_2) = 5 + 2X_2$.

The verifier checks: $g_2(0) + g_2(1) = 5 + 7 = 12 = V_1$. ✓

She samples $r_2 = 10$.

Final check: The verifier queries her oracle for $g(5, 10) = 25$ and compares to $g_2(10) = 25$. They match. **Accept.**

The Cheating Case

Now suppose the prover lies: he claims $H = 7$ instead of the true sum $H^* = 6$.

Round 1: To pass the consistency check, the prover must send some $g_1(X_1)$ with $g_1(0) + g_1(1) = 7$. The true polynomial $s_1(X_1) = 2X_1 + 2$ sums to 6, so he can't use it.

He sends a lie: $g_1(X_1) = X_1 + 3$. Check: $g_1(0) + g_1(1) = 3 + 4 = 7$. ✓

The verifier samples $r_1 = 5$.

- Prover's value: $g_1(5) = 8$
- True value: $s_1(5) = 12$

The prover is now committed to defending a false claim: $\sum_{x_2} g(5, x_2) = 8$. But the true sum is $g(5, 0) + g(5, 1) = 5 + 7 = 12$.

Round 2: The prover needs $g_2(0) + g_2(1) = 8$. He sends $g_2(X_2) = 3 + 2X_2$.

The verifier samples $r_2 = 10$.

Final check:

- Prover claims: $g_2(10) = 3 + 20 = 23$
- Verifier queries oracle: $g(5, 10) = 25$

$23 \neq 25$. **Reject.**

The Moral

The initial lie forced the prover to send polynomials different from the true ones. By Schwartz-Zippel, the random challenges almost certainly landed on points where these polynomials disagreed. The lie didn't just persist; it *amplified* through the rounds until it became a simple, detectable falsehood.

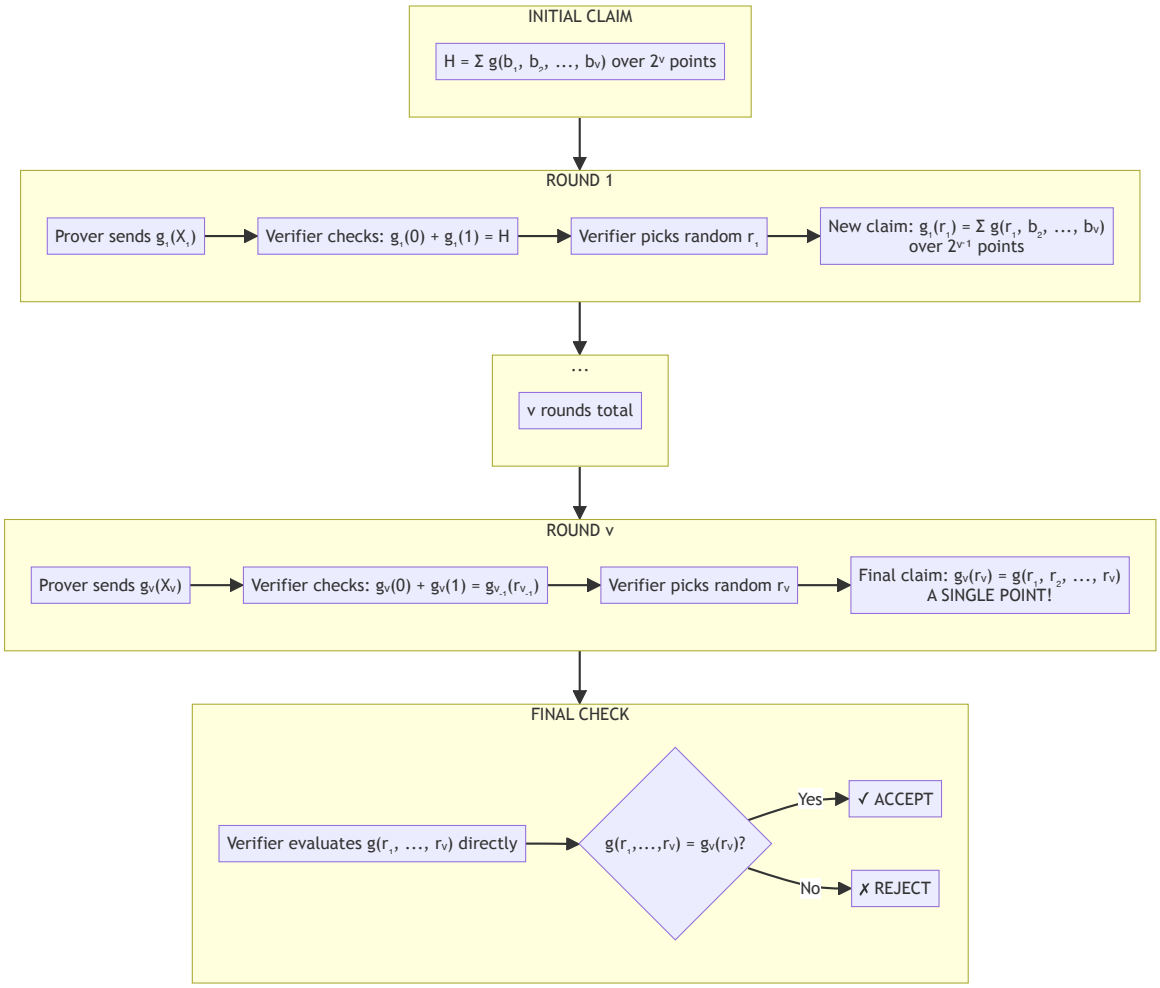
Notice what happened to the cheating prover. After sending the first dishonest polynomial, they weren't free. The verifier's random challenge $r_1 = 5$ created a new constraint: the prover must now justify that $\sum_{x_2} g(5, x_2) = 8$. But they didn't choose 5; the verifier did, unpredictably. The prover is forced to fabricate an answer for a question they couldn't anticipate.

Each round tightens the trap. The second lie must be consistent with the first. The third with the second. Each fabrication constrains the next, and the prover never controls which constraints they'll face. By the final round, the accumulated lies have painted the cheater into a corner: they must claim that $g(5, 10) = 23$ when any honest evaluation reveals 25. The system of fabrications collapses under its own weight.

The prover's only hope is that every random challenge happens to land on a point where the cheating polynomial agrees with the true one. For degree-d polynomials over a field of size $|\mathbb{F}|$, this probability is at most $d/|\mathbb{F}|$ per round, negligible in cryptographic settings.

The Protocol Flow: A Visual Guide

The following diagram traces the claim reduction through each round:



The reduction is exponential: $2^\nu \to 2^{\nu-1} \to 2^{\nu-2} \to \ldots \to 2^0 = 1$.

Application: Counting Satisfying Assignments (#SAT)

The sum-check protocol becomes truly powerful when combined with **arithmetization**: the process of translating discrete, combinatorial problems into the language of polynomials over finite fields. We touched on #SAT in Chapter 2 as motivation for why polynomials matter in complexity theory. Now we see exactly how the translation works and why it enables efficient verification. The full theory of arithmetization will occupy later chapters; for now, we need just enough to see sum-check in action.

The #SAT problem: Given a boolean formula ϕ with ν variables, count how many of the 2^ν possible assignments make ϕ true.

This is a canonical #P-complete problem, even harder than NP. Verifying the count naively requires checking all 2^ν assignments. But with sum-check, a prover can convince a verifier of the correct count

in polynomial time.

Arithmetization of Boolean Formulas

The reduction transforms the boolean formula into a polynomial that equals 1 on satisfying assignments and 0 otherwise.

Step 1: Arithmetize literals

- The variable x_i stays as x_i
- The negation $\neg x_i$ becomes $1 - x_i$

Over $\{0, 1\}$, these give the right values: if $x_i = 1$, then $\neg x_i = 0$, and $1 - x_i = 0$. ✓

Step 2: Arithmetize clauses Consider a clause $C = (z_1 \vee z_2 \vee z_3)$ where each z_i is a literal. The clause is false only when all three literals are false. So:

$$g_C(x) = 1 - (1 - z_1)(1 - z_2)(1 - z_3)$$

where each z_i is the polynomial form of the literal.

Example: For the clause $C = (x_1 \vee \neg x_2 \vee x_3)$:

$$g_C(x_1, x_2, x_3) = 1 - (1 - x_1) \cdot x_2 \cdot (1 - x_3)$$

This equals 0 precisely when $x_1 = 0, x_2 = 1, x_3 = 0$: the only assignment that falsifies the clause.

Step 3: Arithmetize the full formula For a CNF formula $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$, the formula is satisfied when *all* clauses are satisfied:

$$g_\phi(x_1, \dots, x_\nu) = \prod_{j=1}^{m} g_{C_j}(x_1, \dots, x_\nu)$$

Over $\{0, 1\}^\nu$, this product equals 1 if all clauses are satisfied and 0 otherwise.

The Protocol

The number of satisfying assignments is:

$$\#SAT(\phi) = \sum_{(b_1, \dots, b_\nu) \in \{0,1\}^\nu} g_\phi(b_1, \dots, b_\nu)$$

This is exactly a sum over the boolean hypercube! The prover can use the sum-check protocol to convince the verifier of this count.

Degree analysis: For a 3-CNF formula, each clause polynomial has degree at most 3. With m clauses, the product g_ϕ has total degree at most $3m$. The degree in any single variable is at most $3m$ as well (though often much smaller due to variable sharing).

Verifier's work: The verifier performs ν rounds of sum-check, checking polynomials of degree at most $3m$. The final check requires evaluating g_ϕ at a random point; this takes $O(m)$ time since g_ϕ is a product of m clause polynomials.

Total verifier time: $O(\nu \cdot m)$, polynomial in the formula size, despite the exponentially large space of assignments.

Worked Example: A Tiny #SAT Instance

Consider the formula $\phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2)$ with $\nu = 2$ variables and $m = 2$ clauses.

Step 1: Arithmetize.

Clause 1: $(x_1 \vee x_2) \to 1 - (1 - x_1)(1 - x_2) = x_1 + x_2 - x_1 x_2$

Clause 2: $(\neg x_1 \vee x_2) \to 1 - x_1(1 - x_2) = 1 - x_1 + x_1 x_2$

Full formula: $g_\phi(x_1, x_2) = (x_1 + x_2 - x_1 x_2)(1 - x_1 + x_1 x_2)$

Step 2: Evaluate on $\{0,1\}^2$.

(x_1, x_2)	Clause 1	Clause 2	g_ϕ	ϕ satisfied?
$(0,0)$	0	1	0	No
$(0,1)$	1	1	1	Yes
$(1,0)$	1	0	0	No
$(1,1)$	1	1	1	Yes

Step 3: Count.

$$\#SAT(\phi) = \sum_{(b_1,b_2)\in\{0,1\}^2} g_\phi(b_1, b_2) = 0 + 1 + 0 + 1 = 2$$

The formula has exactly 2 satisfying assignments: $(0,1)$ and $(1,1)$ (both require $x_2 = 1$).

The prover uses sum-check to convince the verifier of this count. The polynomial g_ϕ has degree 2 in each variable (degree 4 total), so each round polynomial has degree at most 2, requiring 3 field elements per round.

Deferred Evaluation

The sum-check protocol rests on the principle that **you don't need to compute a sum to verify it**.

Consider what the verifier actually does:

1. She receives polynomials $g_1, g_2, \ldots, g_\nu$ from the prover.
2. She checks consistency: does $g_j(0) + g_j(1)$ equal the previous round's value?
3. She checks degree bounds.
4. At the very end, she evaluates g at a single random point.

The verifier never computes any intermediate sums. She never evaluates g at any point of the boolean hypercube. All the hard work, computing the actual sums, is done by the prover. The verifier merely checks that the prover's story is internally consistent.

This is claim reduction in action. Each round, the claim shrinks:

- Round 0: "The sum over 2^ν points is H"
- Round 1: "The sum over $2^{\nu-1}$ points (at a random slice) is V_1"
- Round 2: "The sum over $2^{\nu-2}$ points is V_2"
- …
- Round ν: "The value at one specific point is V_ν"

By the end, we've reduced an exponential claim to a trivial one. And the random challenges ensure that any cheating at an earlier stage propagates into a detectable error at the final stage.

Complexity Analysis

Let's be precise about the efficiency gains.

Prover complexity: In round j, the prover must compute a univariate polynomial of degree at most d. To specify this polynomial, the prover evaluates it at $d + 1$ points (say, $0, 1, 2, \ldots, d$). For each such point α, the prover computes:

$$g_j(\alpha) = \sum_{(b_{j+1},\ldots,b_\nu)\in\{0,1\}^{\nu-j}} g(r_1, \ldots, r_{j-1}, \alpha, b_{j+1}, \ldots, b_\nu)$$

This requires summing over $2^{\nu-j}$ terms. Across all rounds, the prover's total work is:

$$O\left(\sum_{j=1}^{\nu}(d+1)\cdot 2^{\nu-j}\right) = O(d\cdot 2^\nu)$$

The prover does work proportional to the size of the hypercube, but this is what the prover would need to do anyway to compute the sum. The sum-check protocol doesn't add significant overhead to the prover. Note that achieving $O(2^{\nu-j})$ per round (rather than recomputing from scratch each time) requires an algorithmic trick: maintaining and folding intermediate arrays so that each round reuses the previous round's work. Chapter 19 develops this technique in detail.

Verifier complexity: In each round, the verifier:

- Receives a degree-d polynomial (specified by $d+1$ coefficients)
- Checks that $g_j(0) + g_j(1)$ equals the previous value
- Samples a random field element
- Evaluates g_j at the random point

This is $O(d)$ work per round, or $O(\nu d)$ total.

At the end, the verifier evaluates g at a single point $(r_1, \ldots, r_\nu)$. Let T be the time to evaluate g at one point. The verifier's total work is:

$$O(\nu d + T)$$

The speedup: The verifier avoids evaluating g at 2^ν points, an exponential savings. If g arises from a "structured" computation (like a circuit or formula), then T is polynomial in the description of that structure, making the whole protocol efficient.

Communication complexity: The prover sends ν univariate polynomials, each of degree at most d. Naively, this requires $d+1$ field elements per polynomial (to specify the coefficients), for a total of $\nu(d+1)$ field elements. But there's a trick.

The one-coefficient trick: At each round, the verifier checks $s_i(0) + s_i(1) = V_{i-1}$. This is one linear equation in the polynomial's coefficients, so the polynomial has only d degrees of freedom, not $d+1$.

Write $s_i(X) = c_0 + c_1 X + c_2 X^2 + \cdots + c_d X^d$. Then:

$$s_i(0) + s_i(1) = c_0 + (c_0 + c_1 + c_2 + \cdots + c_d) = 2c_0 + c_1 + c_2 + \cdots + c_d = V_{i-1}$$

So: $c_1 = V_{i-1} - 2c_0 - c_2 - c_3 - \cdots - c_d$.

The prover sends only $(c_0, c_2, c_3, \ldots, c_d)$, and the verifier recovers c_1 from the constraint. This saves one field element per round: νd field elements total instead of $\nu(d+1)$.

For the common case of multilinear polynomials ($d = 1$), this halves communication: one field element per round instead of two.

Soundness error: As computed earlier, the probability that a cheating prover succeeds is at most $\nu d/|\mathbb{F}|$. For a 256-bit field and reasonable values of ν and d, this is negligible.

Why Sum-Check Enables Everything Else

The sum-check protocol is the foundation upon which much of modern verifiable computation is built.

The celebrated IP = PSPACE theorem, which shows that every problem solvable in polynomial space has an efficient interactive proof, uses sum-check as its core building block. The LFKN protocol arithmetizes quantified boolean formulas and applies sum-check recursively. To verify that an arithmetic circuit was evaluated correctly, the GKR protocol (Chapter 7) expresses the relationship between adjacent circuit layers as a sum over a hypercube, then uses sum-check to reduce a claim about one layer to a claim about the next, peeling back the circuit layer by layer until we reach the inputs.

Many of today's practical succinct arguments (Spartan, HyperPlonk, and the entire family of "sum-check based" SNARKs) use sum-check as their information-theoretic core. The protocol's structure, where a prover commits to polynomials and a verifier checks random evaluations, maps cleanly onto polynomial commitment schemes. As we'll see in the next chapter, multilinear polynomials (those with degree at most 1 in each variable) have a natural correspondence with functions on the boolean hypercube. Sum-check works especially elegantly with multilinear polynomials, and this paradigm has become one of the two major approaches to building modern proof systems.

For years after the initial theoretical breakthroughs, practical SNARK systems moved away from sum-check toward other approaches (PCPs, linear PCPs, univariate techniques). But recently, sum-check has made a dramatic comeback. Systems like Lasso and Jolt use sum-check at their core, achieving remarkable prover efficiency. It turns out that sum-check provers can run in *linear time* for structured polynomials, and the protocol meshes beautifully with modern polynomial commitment schemes. We'll explore this renaissance in depth in Chapter 19.

The sum-check protocol is where the abstract power of polynomials (their rigidity, their compression of constraints, their amenability to random testing) first crystallizes into a concrete verification procedure. Every protocol we study from here forward either uses sum-check directly or is in dialogue with the principles it established.

The Last Mile: From Oracle Access to Polynomial Commitments

We noted earlier that the final check of sum-check requires evaluating $g(r_1, \ldots, r_\nu)$, and that sometimes the verifier cannot do this herself because g depends on the prover's private data. This deserves a closer look, because the mechanism that closes this gap is what turns sum-check from a complexity-theoretic curiosity into a practical proof system.

Consider two representative scenarios.

When the verifier can compute g directly. If g is built entirely from public information, the final evaluation is straightforward. For instance, in the #SAT application above, g_ϕ is a product of clause polynomials derived from the public formula ϕ. The verifier knows ϕ, so she can evaluate $g_\phi(r_1, \ldots, r_\nu)$ in $O(m)$ time. Similarly, in the GKR protocol (Chapter 7), the polynomial g at each layer encodes the circuit's wiring pattern, which is public. No additional machinery is needed.

When g depends on the prover's private witness. This is the harder and more common case in SNARK constructions. In systems like Spartan (Chapter 19), the polynomial g involves the multilinear extension of the prover's private witness vector w. (We develop multilinear extensions in Chapter 4.) The verifier can compute the public parts of g at the random point, but the witness-dependent part requires a value she does not know.

Sum-check has reduced the exponential sum to a single evaluation, but the verifier cannot complete the check alone. The protocol needs one more ingredient: a way for the prover to credibly reveal a single evaluation of a polynomial she committed to earlier, without revealing the polynomial itself.

This ingredient is a **polynomial commitment scheme (PCS)**, developed in Chapter 9. The mechanism is simple in outline:

1. Before sum-check begins, the prover sends a short, binding commitment C to the witness polynomial. "Binding" means the prover cannot change the polynomial after sending C.
2. During sum-check, the random challenges $r_1, \ldots, r_\nu$ are determined. The commitment was sent before any challenges were chosen, so the prover cannot adapt the polynomial to the challenge point.
3. After sum-check, the prover opens the commitment at $(r_1, \ldots, r_\nu)$: she provides the evaluation v together with a proof π that v is consistent with C. The verifier checks π and uses v to complete the final sum-check check.

This is where sum-check transitions from an information-theoretic protocol (what the literature calls an "Interactive Oracle Proof") to a cryptographic argument. The PCS replaces oracle access with commit-then-open. The prover is bound to a specific polynomial before seeing the evaluation point, and the succinctness of the commitment means the verifier never needs to see the full polynomial.

Every sum-check-based SNARK makes this transition at the final step. The oracle is not a black box. It is a commitment scheme.

Key takeaways

1. **The sum-check protocol verifies exponential sums efficiently**: A prover can convince a verifier that $\sum_{b \in \{0,1\}^\nu} g(b) = H$ with the verifier doing only $O(\nu)$ work, plus one evaluation of g. The verifier never computes any sum herself.

2. **Claim reduction is the key mechanism**: Each round reduces a claim about 2^k points to a claim about 2^{k-1} points. After ν rounds, the exponential sum becomes a single evaluation.

3. **Lies propagate and amplify**: A false initial claim forces the prover to send dishonest polynomials. Random challenges catch the discrepancy with probability $1 - d/|\mathbb{F}|$ per round. The lie can't hide; it gets cornered.

4. **The degree bound is essential**: Without it, a cheating prover could craft high-degree polynomials that pass consistency checks at 0 and 1 while matching the honest polynomial elsewhere. The degree bound forces rigidity.

5. **Arithmetization connects sum-check to computation**: Problems like #SAT encode as sums over the boolean hypercube. The prover does $O(2^\nu)$ work; the verifier does $O(\nu)$. This asymmetry is what makes verification useful.

6. **The final evaluation is the bridge to cryptography**: Sum-check reduces an exponential sum to one evaluation. When that evaluation depends on the prover's private data, a polynomial commitment scheme (Chapter 9) closes the gap: the prover commits before seeing the challenge point, then opens at the end. This is the "last mile" that turns the information-theoretic protocol into a SNARK.

7. **Sum-check is foundational**: IP = PSPACE, GKR, Spartan, Lasso, and most multilinear SNARKs build on sum-check. The protocol's comeback in practical systems (Chapter 19) shows that its elegance survives contact with real provers.

Chapter 4: Multilinear Extensions

In 1971, the Mariner 9 probe became the first spacecraft to orbit another planet. Its mission: map the surface of Mars. But transmitting high-resolution images across 100 million miles of static-filled space was a nightmare. A single burst of cosmic noise could turn a crater into a glitch.

NASA didn't send raw pixels. They used a code developed years earlier by Irving Reed and David Muller: treat the pixel data as evaluations of a *multivariate polynomial*. The Reed-Muller code could correct up to seven bit errors per 32-bit word. When Mariner 9 arrived to find Mars engulfed in a planet-wide dust storm, mission control reprogrammed the spacecraft from Earth and waited. When the dust cleared, the code delivered 7,329 images, mapping 85% of the Martian surface.

Why not Reed-Solomon? In Chapter 2, we encoded n values as a univariate polynomial of degree $n-1$. That works when n is modest. But Mariner's data was indexed by *bit positions*: a 32-bit word has 2^5 bit combinations, a memory address space has 2^{64} locations, a boolean formula with 100 variables has 2^{100} possible assignments. Encoding 2^{100} values as a univariate polynomial means degree $2^{100} - 1$. Impossible.

The solution: let each bit be its own variable. A 100-bit index becomes 100 coordinates, each 0 or 1. The polynomial has 100 variables instead of degree 2^{100}. Data lives not on a line but on a hypercube. This chapter develops that theory.

In Chapter 2, we turned data into polynomials via Lagrange interpolation: given n values, construct the unique degree-$(n-1)$ univariate polynomial passing through them. That was interpolation over a *line*.

Now we need interpolation over a *hypercube*. The data lives at 2^n vertices, indexed by bit strings. The polynomial must agree with the data at these vertices and extend smoothly to all of $\mathbb{F}^n$. The construction is analogous to univariate Lagrange, but the geometry is different, and the efficiency implications are dramatic.

This chapter develops the theory of **multilinear extensions**: the canonical way to extend functions from the Boolean hypercube $\{0,1\}^n$ to polynomials over $\mathbb{F}^n$. These extensions are the workhorses of sum-check-based proof systems, encoding everything from circuit wire values to constraint satisfaction.

The Boolean Hypercube

Consider the set $\{0,1\}^n$, all n-bit binary strings. This is the **Boolean hypercube**, and it contains exactly 2^n points.

```
n = 2:
    (1,1)
     /  \
 (0,1)  (1,0)
     \  /
    (0,0)

n = 3: A cube with 8 vertices
```

Any function $f : \{0,1\}^n \to \mathbb{F}$ assigns a field element to each vertex of this hypercube. There are 2^n vertices, so f is essentially a table of 2^n values.

For example:

- A vector $(v_1, \ldots, v_{2^n})$ can be viewed as $f(b) = v_{1+\text{bin}(b)}$ where bin(b) converts the bit string to an index
- The output values of a layer of circuit gates
- A database of 2^n records indexed by n-bit keys

Why does the hypercube matter? Because computation is fundamentally boolean. A memory address is a bit string. A circuit's inputs are bits. A satisfying assignment to a boolean formula is a point in $\{0,1\}^n$. When we want to verify a computation, the objects we care about (wire values, memory contents, constraint satisfaction) are naturally indexed by binary strings. The hypercube $\{0,1\}^n$ is where computational problems live.

But polynomials live over fields, not just $\{0,1\}$. We want a polynomial that agrees with f on the hypercube but extends smoothly to all of $\mathbb{F}^n$. This extension is what lets us apply the algebraic machinery (Schwartz-Zippel, sum-check) that makes verification efficient.

Why Multilinear?

In Chapter 2, we used univariate polynomials (Reed-Solomon). Why switch to multivariate now?

The problem with univariate encoding is degree: if you encode $N = 2^{20}$ data points into a single-variable polynomial $p(x)$, that polynomial has degree about one million. Manipulating degree-million polynomials is expensive, requiring heavy FFT operations.

Multilinear polynomials avoid this. If you encode the same 2^{20} points into a 20-variable multilinear polynomial, the degree in each variable is just 1. The total degree is only 20. By increasing the number of variables, we drastically lower the per-variable degree. This tradeoff (more variables, lower degree)

enables the linear-time prover algorithms that power modern systems like HyperPlonk and Lasso, avoiding the expensive FFTs required by univariate approaches.

A polynomial in n variables has terms like $X_1^{a_1} X_2^{a_2} \cdots X_n^{a_n}$ with various exponents. The **degree** in variable X_i is the maximum exponent of X_i across all terms.

A polynomial is **multilinear** if its degree in every variable is at most 1. Every term looks like a product of distinct variables (or subsets thereof). We write $\tilde{f}$ (with a tilde) to denote the multilinear extension of a function f:

$$\tilde{f}(X_1, \ldots, X_n) = \sum_{S \subseteq \{1,\ldots,n\}} c_S \prod_{i \in S} X_i$$

For example, with $n = 2$:

$$\tilde{f}(X_1, X_2) = c_\emptyset + c_{\{1\}} X_1 + c_{\{2\}} X_2 + c_{\{1,2\}} X_1 X_2$$

There are 2^n possible subsets S, hence 2^n coefficients. A multilinear polynomial in n variables is fully specified by 2^n numbers, exactly matching the number of points in the hypercube.

This is not a coincidence. It's the key theorem:

Theorem (Multilinear Extension). For any function $f : \{0,1\}^n \to \mathbb{F}$, there exists a unique multilinear polynomial $\tilde{f} : \mathbb{F}^n \to \mathbb{F}$ such that $\tilde{f}(b) = f(b)$ for all $b \in \{0,1\}^n$.

The function $\tilde{f}$ is called the **multilinear extension (MLE)** of f.

Constructing the Multilinear Extension

The theorem claims uniqueness. How do we actually construct $\tilde{f}$?

The Lagrange Basis

For each point $w \in \{0,1\}^n$, define the **Lagrange basis polynomial**:

$$L_w(X) = \prod_{i=1}^{n} (w_i \cdot X_i + (1 - w_i)(1 - X_i))$$

Here $w = (w_1, \ldots, w_n)$ is a fixed boolean vector, where each $w_i \in \{0,1\}$. You can read w as the binary representation of an index from 0 to $2^n - 1$, addressing one of the 2^n vertices of the hypercube.

Meanwhile $X = (X_1, \ldots, X_n)$ is a vector of formal variables where each X_i ranges over all of $\mathbb{F}$. Geometrically, w lives at a corner of the unit hypercube, while X can be any point in $\mathbb{F}^n$, including points "between" corners. The polynomial L_w is defined over all of $\mathbb{F}^n$, but it has a special property on the hypercube: it equals 1 at w and 0 at every other boolean point.

To see why, consider what happens at point w:

- If $w_i = 1$: the factor is $1 \cdot X_i + 0 \cdot (1 - X_i) = X_i$, which evaluates to 1
- If $w_i = 0$: the factor is $0 \cdot X_i + 1 \cdot (1 - X_i) = 1 - X_i$, which evaluates to 1

Every factor equals 1, so $L_w(w) = 1$.

At any other point $b \neq w$:

- There exists some coordinate i where $b_i \neq w_i$
- If $w_i = 1$ and $b_i = 0$: the factor X_i evaluates to 0
- If $w_i = 0$ and $b_i = 1$: the factor $1 - X_i$ evaluates to 0

One factor is zero, so $L_w(b) = 0$.

The Extension Formula

The multilinear extension is now simply:

$$\tilde{f}(X) = \sum_{w \in \{0,1\}^n} f(w) \cdot L_w(X)$$

At any hypercube point b:

$$\tilde{f}(b) = \sum_{w} f(w) \cdot L_w(b) = f(b) \cdot 1 + \sum_{w \neq b} f(w) \cdot 0 = f(b)$$

The extension agrees with f on the hypercube. Since it's a sum of multilinear terms (each L_w is multilinear), $\tilde{f}$ is multilinear.

Uniqueness

Claim: If a multilinear polynomial p vanishes on all of $\{0,1\}^n$, then $p \equiv 0$.

Proof by induction on n:

Base case ($n = 1$): A multilinear polynomial in one variable has form $p(X) = a + bX$. If $p(0) = 0$ and $p(1) = 0$, then $a = 0$ and $a + b = 0$, so $b = 0$. Thus $p \equiv 0$.

Inductive step: Write $p(X_1, \dots, X_n) = q_0(X_2, \dots, X_n) + X_1 \cdot q_1(X_2, \dots, X_n)$ where q_0, q_1 are multilinear in $n - 1$ variables. Evaluating at $X_1 = 0$: $p(0, X_2, \dots, X_n) = q_0(X_2, \dots, X_n)$. Since p vanishes on all of $\{0,1\}^n$, in particular q_0 vanishes on $\{0,1\}^{n-1}$. By induction, $q_0 \equiv 0$. Similarly, $p(1, X_2, \dots, X_n) = q_1(X_2, \dots, X_n)$ vanishes on $\{0,1\}^{n-1}$, so $q_1 \equiv 0$. Thus $p \equiv 0$. □

Corollary: If two multilinear polynomials agree on $\{0,1\}^n$, their difference vanishes there, hence is identically zero, so they are equal.

The Equality Polynomial

One Lagrange basis polynomial deserves special attention: the **equality polynomial**.

$$\widetilde{\mathsf{eq}}(X, Y) = \prod_{i=1}^{n} (X_i Y_i + (1 - X_i)(1 - Y_i))$$

This is the MLE of the equality function:

$$\mathsf{eq}(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{otherwise} \end{cases}$$

for $a, b \in \{0,1\}^n$.

The Lagrange basis polynomials are just the equality polynomial with one input fixed:

$$L_w(X) = \widetilde{\mathsf{eq}}(w, X)$$

The equality polynomial appears constantly in sum-check-based protocols, through the identity:

$$\sum_{x \in \{0,1\}^n} \widetilde{\mathsf{eq}}(\tau, x) \cdot f(x) = \tilde{f}(\tau)$$

This follows directly from the Lagrange formula: $\tilde{f}(\tau) = \sum_x f(x) \cdot L_x(\tau) = \sum_x f(x) \cdot \widetilde{\mathsf{eq}}(\tau, x)$. Summing f weighted by $\widetilde{\mathsf{eq}}(\tau, \cdot)$ over the hypercube gives the MLE of f evaluated at τ. This means evaluating an MLE at a random challenge τ reduces to a sum-check on $g(x) = \widetilde{\mathsf{eq}}(\tau, x) \cdot f(x)$.

This immediately gives a powerful zero test. Suppose the verifier wants to check that f vanishes on the entire Boolean hypercube. By the identity above, checking that all $f(x)$ values are zero is the same as checking that $\tilde{f}(\tau) = 0$. The verifier picks a random $\tau \in \mathbb{F}^n$ and runs sum-check on:

$$\sum_{x \in \{0,1\}^n} \widetilde{\mathsf{eq}}(\tau, x) \cdot f(x) = 0$$

This is a random linear combination of all $f(x)$ values. If f truly vanishes on the hypercube, then $\tilde{f} \equiv 0$ (by the uniqueness theorem above), so the sum is always 0. If even one value $f(x^*) \neq 0$, then $\tilde{f}$ is a nonzero multilinear polynomial, and Schwartz-Zippel guarantees $\tilde{f}(\tau) \neq 0$ with probability at least $1 - n/|\mathbb{F}|$. Over a 254-bit field, this is negligible. This "zero-on-hypercube" test is the foundation of Spartan and related sum-check-based proof systems.

Worked Example: A 2-Variable Function

Let's trace through a complete example.

Consider $f : \{0,1\}^2 \to \mathbb{F}$ defined by the table:

(X_1, X_2)	$f(X_1, X_2)$
$(0,0)$	3
$(0,1)$	7
$(1,0)$	2
$(1,1)$	5

The Lagrange basis polynomials are:

$$L_{(0,0)}(X) = (1 - X_1)(1 - X_2)$$

$$L_{(0,1)}(X) = (1 - X_1) \cdot X_2$$

$$L_{(1,0)}(X) = X_1 \cdot (1 - X_2)$$

$$L_{(1,1)}(X) = X_1 \cdot X_2$$

The multilinear extension is then:

$$\tilde{f}(X_1, X_2) = 3 \cdot (1 - X_1)(1 - X_2) + 7 \cdot (1 - X_1)X_2 + 2 \cdot X_1(1 - X_2) + 5 \cdot X_1 X_2$$

Expanding:

$$= 3(1 - X_1 - X_2 + X_1X_2) + 7(X_2 - X_1X_2) + 2(X_1 - X_1X_2) + 5X_1X_2$$
$$= 3 - 3X_1 - 3X_2 + 3X_1X_2 + 7X_2 - 7X_1X_2 + 2X_1 - 2X_1X_2 + 5X_1X_2$$
$$= 3 + (-3 + 2)X_1 + (-3 + 7)X_2 + (3 - 7 - 2 + 5)X_1X_2$$
$$= 3 - X_1 + 4X_2 - X_1X_2$$

We can verify this matches the table:

- $\tilde{f}(0, 0) = 3 - 0 + 0 - 0 = 3$ (matches)
- $\tilde{f}(0, 1) = 3 - 0 + 4 - 0 = 7$ (matches)
- $\tilde{f}(1, 0) = 3 - 1 + 0 - 0 = 2$ (matches)
- $\tilde{f}(1, 1) = 3 - 1 + 4 - 1 = 5$ (matches)

What happens at a non-boolean point? Evaluating at $(0.5, 0.3)$:

$$\tilde{f}(0.5, 0.3) = 3 - 0.5 + 4(0.3) - (0.5)(0.3) = 3 - 0.5 + 1.2 - 0.15 = 3.55$$

This value has no "meaning" on the hypercube; $(0.5, 0.3)$ isn't a Boolean point. But this is exactly what we want: the polynomial is defined everywhere, and random evaluation is the key to probabilistic verification.

Efficient Evaluation

Given the table of values $\{f(w) : w \in \{0, 1\}^n\}$ and a query point $r \in \mathbb{F}^n$, how fast can we compute $\tilde{f}(r)$?

The naive approach sums over all 2^n terms:

$$\tilde{f}(r) = \sum_{w \in \{0,1\}^n} f(w) \cdot L_w(r)$$

Each $L_w(r)$ takes $O(n)$ to compute. Total: $O(n \cdot 2^n)$.

We can do better with streaming evaluation. $\tilde{f}(r)$ is computable in $O(2^n)$ time with the following observation.

Define T_k as the "partial extension" using only the first k variables of r:

$$T_k(x_{k+1}, \ldots, x_n) = \sum_{(b_1,\ldots,b_k)\in\{0,1\}^k} f(b_1, \ldots, b_k, x_{k+1}, \ldots, x_n) \cdot \prod_{i=1}^{k} L_{b_i}(r_i)$$

At $k = 0$: $T_0 = f$ (the original table).

At $k = n$: $T_n = \tilde{f}(r)$ (a single value).

The recursion from T_k to T_{k+1}:

$$T_{k+1}(x_{k+2}, \ldots, x_n) = (1 - r_{k+1}) \cdot T_k(0, x_{k+2}, \ldots) + r_{k+1} \cdot T_k(1, x_{k+2}, \ldots)$$

Each step halves the table size. Total work: $2^n + 2^{n-1} + \cdots + 1 = O(2^n)$.

This is linear in the table size, optimal for any algorithm that must touch all values.

Worked Example: Streaming Evaluation

Let's trace through this algorithm with our earlier function $f : \{0,1\}^2 \to \mathbb{F}$:

(b_1, b_2)	$f(b_1, b_2)$
$(0, 0)$	3
$(0, 1)$	7
$(1, 0)$	2
$(1, 1)$	5

We want to compute $\tilde{f}(r_1, r_2)$ at the point $r = (0.4, 0.7)$.

Step 0: Initialize T_0

T_0 is just the original table, a function of both variables:

$$T_0(x_1, x_2) = f(x_1, x_2)$$

Think of it as four values indexed by $(x_1, x_2) \in \{0,1\}^2$:

		$x_2 = 0$	$x_2 = 1$
$T_0 =$	$x_1 = 0$	3	7
	$x_1 = 1$	2	5

Step 1: Compute T_1 by "folding in" $r_1 = 0.4$

The recursion says:

$$T_1(x_2) = (1 - r_1) \cdot T_0(0, x_2) + r_1 \cdot T_0(1, x_2)$$

This is a weighted combination of the two rows, using $1 - r_1 = 0.6$ and $r_1 = 0.4$:

- $T_1(0) = 0.6 \cdot T_0(0,0) + 0.4 \cdot T_0(1,0) = 0.6 \cdot 3 + 0.4 \cdot 2 = 1.8 + 0.8 = 2.6$
- $T_1(1) = 0.6 \cdot T_0(0,1) + 0.4 \cdot T_0(1,1) = 0.6 \cdot 7 + 0.4 \cdot 5 = 4.2 + 2.0 = 6.2$

The table has shrunk from 4 values to 2 values: $T_1 = [2.6, 6.2]$.

Step 2: Compute T_2 by "folding in" $r_2 = 0.7$

$$T_2 = (1 - r_2) \cdot T_1(0) + r_2 \cdot T_1(1) = 0.3 \cdot 2.6 + 0.7 \cdot 6.2 = 0.78 + 4.34 = 5.12$$

The table has shrunk from 2 values to 1 value. This single value is $\tilde{f}(0.4, 0.7) = 5.12$.

We can verify using the explicit formula $\tilde{f}(X_1, X_2) = 3 - X_1 + 4X_2 - X_1X_2$:

$$\tilde{f}(0.4, 0.7) = 3 - 0.4 + 4(0.7) - (0.4)(0.7) = 3 - 0.4 + 2.8 - 0.28 = 5.12 \checkmark$$

This works because the Lagrange basis polynomial factorizes into independent pieces, one per coordinate:

$$L_{(b_1,b_2)}(r_1, r_2) = L_{b_1}(r_1) \cdot L_{b_2}(r_2)$$

where $L_0(r) = 1 - r$ and $L_1(r) = r$ are univariate selectors. This factorization holds because the multilinear Lagrange formula is a *product* over coordinates:

$$L_w(X) = \prod_{i=1}^{n} (w_i \cdot X_i + (1 - w_i)(1 - X_i))$$

Each factor depends only on one coordinate of w and one coordinate of X. So evaluating at (r_1, r_2) gives a product of independent terms.

The algorithm exploits this factorization. The MLE evaluation is:

$$\tilde{f}(r_1, r_2) = \sum_{b_1, b_2 \in \{0,1\}} f(b_1, b_2) \cdot L_{b_1}(r_1) \cdot L_{b_2}(r_2)$$

Rearranging the sum (grouping by b_2):

$$= \sum_{b_2} L_{b_2}(r_2) \cdot \underbrace{\left(\sum_{b_1} f(b_1, b_2) \cdot L_{b_1}(r_1) \right)}_{T_1(b_2)}$$

The inner sum is exactly what Step 1 computes: for each value of b_2, it combines the two b_1 cases using weights $L_0(r_1) = 1 - r_1$ and $L_1(r_1) = r_1$. The result T_1 has half as many entries. Step 2 then folds in the r_2 weights similarly.

An analogy helps here: think of a single-elimination tournament with 2^n players. In each round, pairs compete and half are eliminated. After n rounds, one champion remains. The streaming algorithm works the same way: 2^n table entries enter, each round uses a random weight to combine pairs, and after n rounds a single evaluation emerges. The tournament bracket is the structure of multilinear computation.

This pattern of using a random challenge to collapse pairs of values and halving the problem size will reappear throughout this book. In Chapter 10 (FRI), we'll name it **folding** and see it as one of the central techniques in zero-knowledge proofs.

Code: Streaming MLE Evaluation

The algorithm above translates directly to code. Each coordinate of r folds the table in half.

```
def mle_eval(table, r):
    """
    Evaluate the multilinear extension of `table` at point `r`.

    Args:
        table: List of 2^n field elements (the function values on hypercube)
        r: Tuple of n coordinates (r_1, ..., r_n)

    Returns: The value of the MLE at r
    """
    T = table.copy()

    for r_i in r:
        half = len(T) // 2
        # Fold: T'[j] = (1 - r_i) * T[2j] + r_i * T[2j+1]
        T = [(1 - r_i) * T[2*j] + r_i * T[2*j + 1]
             for j in range(half)]
```

```
    return T[0]  # Single value remains

# Example from the worked example above
table = [3, 7, 2, 5]  # f(0,0)=3, f(0,1)=7, f(1,0)=2, f(1,1)=5
r = (0.4, 0.7)

result = mle_eval(table, r)
print(f"Streaming: MLE({r}) = {result}")

# Verify against explicit formula: f(X1,X2) = 3 - X1 + 4*X2 - X1*X2
explicit = 3 - 0.4 + 4*0.7 - 0.4*0.7
print(f"Explicit:  MLE({r}) = {explicit}")
```

Output:

```
Streaming: MLE((0.4, 0.7)) = 5.12
Explicit:  MLE((0.4, 0.7)) = 5.12
```

The streaming algorithm touches each table entry exactly once. For a table of size $N = 2^n$, total work is $N/2 + N/4 + \cdots + 1 = N - 1 = O(N)$.

Tensor Product Structure

The factorization we used in the streaming algorithm generalizes to any number of variables. For $w = (w_1, \ldots, w_n) \in \{0,1\}^n$:

$$L_w(r_1, \ldots, r_n) = \prod_{i=1}^{n} L_{w_i}(r_i)$$

where $L_0(r_i) = 1 - r_i$ and $L_1(r_i) = r_i$.

This is a **tensor product** structure. To see what this means concretely, consider $n = 2$. Define the vectors:

$$\vec{v}_1 = (L_0(r_1), L_1(r_1)) = (1 - r_1, r_1)$$
$$\vec{v}_2 = (L_0(r_2), L_1(r_2)) = (1 - r_2, r_2)$$

Their tensor product $\vec{v}_1 \otimes \vec{v}_2$ is the 2×2 matrix (or equivalently, length-4 vector) of all pairwise products:

$$\vec{v}_1 \otimes \vec{v}_2 = \begin{pmatrix} (1-r_1)(1-r_2) & (1-r_1)r_2 \\ r_1(1-r_2) & r_1 r_2 \end{pmatrix}$$

Reading off the entries: $L_{(0,0)}(r), L_{(0,1)}(r), L_{(1,0)}(r), L_{(1,1)}(r)$. The tensor product *is* the vector of Lagrange evaluations.

For general n, the vector of all 2^n Lagrange evaluations is:

$$(L_0(r_1), L_1(r_1)) \otimes (L_0(r_2), L_1(r_2)) \otimes \cdots \otimes (L_0(r_n), L_1(r_n))$$

The streaming algorithm exploits this tensor structure. Instead of computing all 2^n Lagrange values (expensive), it processes one coordinate at a time, folding the tensor product incrementally. This is why MLE evaluation costs $O(2^n)$ instead of $O(n \cdot 2^n)$. The same tensor structure enables:

- Efficient prover algorithms for sum-check (Chapter 19)
- Recursive proof constructions
- Memory-efficient streaming over large tables

Multilinear Extensions of Functions on Larger Domains

What if our function isn't defined on $\{0,1\}^n$?

Suppose $f : \{0, 1, \ldots, m-1\} \to \mathbb{F}$ for some $m = 2^n$. We can *interpret* the domain as $\{0,1\}^n$ via binary encoding:

$$\tilde{f}(X_1, \ldots, X_n) = \text{MLE of } (k \mapsto f(k)) \text{ with } k = \sum_i 2^{i-1} X_i$$

Any function on a power-of-two domain has a natural multilinear extension.

For domains not of size 2^n, we can pad with zeros or use more sophisticated encodings. The key insight: as long as the domain is *finite*, we can always encode it in binary and take the MLE.

Connection to Sum-Check

The sum-check protocol (Chapter 3) proves claims of the form:

$$H = \sum_{b \in \{0,1\}^n} g(b)$$

for some polynomial g. When g is the multilinear extension of a function f, this sum equals $\sum_{b\in\{0,1\}^n} f(b)$, the sum of all function values on the hypercube.

As an example, suppose we want to prove that a vector $(v_1, \dots, v_N)$ with $N = 2^n$ sums to a claimed value H.

Let $\tilde{v}$ be the MLE encoding the vector. Then:

$$\sum_{b\in\{0,1\}^n} \tilde{v}(b) = \sum_{i=1}^{N} v_i = H$$

Sum-check verifies this identity without the verifier seeing all of v. The protocol reduces the sum to a single random evaluation $\tilde{v}(r)$, which the prover supplies (with a commitment proof).

This is the bridge from "data" to "proof": encode data as an MLE, verify properties via sum-check, bind via polynomial commitment.

Evaluations and Coefficients

A perspective that clarifies many constructions:

A multilinear polynomial $\tilde{f}$ has 2^n coefficients (the c_S values in the monomial expansion $\sum_S c_S \prod_{i\in S} X_i$). These coefficients live in an abstract "coefficient space."

But $\tilde{f}$ also has 2^n evaluations on the hypercube. These evaluations are just $f(w)$, the original table values you started with.

These are not the same numbers. The table entry $f(0,0) = 3$ in our worked example is not a coefficient of the polynomial. The polynomial $\tilde{f}(X_1, X_2) = 3 - X_1 + 4X_2 - X_1X_2$ has coefficients $\{3, -1, 4, -1\}$, while the table values are $\{3, 7, 2, 5\}$. They're related by the Lagrange interpolation formula.

For multilinear polynomials, the evaluation table *is* a complete description. You can recover coefficients from evaluations and vice versa. They're just two bases for the same 2^n-dimensional vector space.

The transformation between bases is exactly the Lagrange interpolation formula and its inverse. Both can be computed in $O(2^n)$ time.

This means:

- Committing to a multilinear polynomial = committing to its evaluation table
- Evaluating at a random point = a linear combination of table entries

- Sum-check over an MLE = verifying global properties through local queries

The table has 2^n entries. The verifier touches $O(n)$ of them. The polynomial is what bridges the gap: it's a compressed representation that can be probed at random points, and those random probes reveal whether the full table satisfies the claimed property. Extension creates redundancy; redundancy enables compression; compression enables succinctness.

Polynomial Evaluation as Inner Product

There's a beautiful way to see this algebraically: **polynomial evaluation is an inner product**.

For a multilinear polynomial, the evaluation at any point r is:

$$\tilde{f}(r) = \sum_{w \in \{0,1\}^n} f(w) \cdot L_w(r) = \langle \vec{f}, \vec{L}(r) \rangle$$

where $\vec{f} = (f(w))_{w \in \{0,1\}^n}$ is the table of values and $\vec{L}(r) = (L_w(r))_{w \in \{0,1\}^n}$ is the vector of Lagrange basis evaluations at r.

This linear algebra perspective is surprisingly powerful. For decades, sum-check was seen as a beautiful theoretical result with limited practical use. Then came the realization: polynomial evaluation is an inner product, and inner products interact beautifully with commitment schemes. No FFTs, no trusted setups, just vectors and dot products. Systems like Spartan, HyperPlonk, and Lasso all exploit this insight. Chapter 19 tells the full story of this "Sum-Check Renaissance."

The consequences are immediate:

- **Commitment**: Committing to $\tilde{f}$ means committing to the vector $\vec{f}$
- **Evaluation proof**: Proving $\tilde{f}(r) = y$ means proving an inner product claim $\langle \vec{f}, \vec{L}(r) \rangle = y$
- **The verifier knows** $\vec{L}(r)$: Given r, anyone can compute the Lagrange evaluations

This reduces polynomial evaluation proofs to inner product proofs, and inner products interact beautifully with homomorphic commitments. We'll exploit this connection in Chapters 6 and 9.

Key takeaways

1. **The Boolean hypercube** $\{0,1\}^n$ is the natural domain for multilinear polynomials. It has 2^n points.
2. **Multilinear extension (MLE)**: The unique polynomial of degree at most 1 in each variable that agrees with f on the hypercube.

3. **Lagrange basis polynomials** $L_w(X)$ equal 1 at w and 0 elsewhere. The MLE is $\tilde{f}(X) = \sum_w f(w) \cdot L_w(X)$.
4. **The equality polynomial** $\widetilde{\mathsf{eq}}(X, Y)$ is the MLE of the equality indicator. Lagrange bases are $L_w(X) = \widetilde{\mathsf{eq}}(w, X)$.
5. **Tensor product structure**: $L_w(r) = \prod_i L_{w_i}(r_i)$. The basis factorizes, enabling fast algorithms.
6. **Efficient evaluation**: Given the table and a point, compute the MLE in $O(2^n)$ time via streaming.
7. **Sum over the hypercube**: $\sum_b \tilde{f}(b) = \sum_w f(w)$. Sum-check verifies such sums efficiently.
8. **Evaluations = coefficients**: For MLEs, the table of values completely determines the polynomial. They're dual representations.
9. **Binary encoding**: Any function on $\{0, \ldots, 2^n - 1\}$ can be encoded as a function on $\{0, 1\}^n$, then extended multilinearly.
10. **The bridge to proofs**: MLEs encode data; sum-check verifies properties; polynomial commitment binds the prover. This trinity underlies sum-check-based SNARKs.

Chapter 5: Univariate Polynomials and Finite Fields

Gauss discovered the Fast Fourier Transform in 1805. He needed to predict the orbits of asteroids Pallas and Juno, so he wrote an algorithm that computed transforms in $O(n \log n)$ time instead of $O(n^2)$. He wrote it in Latin, in a notebook, and never published it. The algorithm waited 160 years for someone to notice.

Cooley and Tukey rediscovered it in 1965. They gave it a name. It became one of the most important algorithms in computing: MRI machines, audio compression, the entire edifice of digital signal processing. All of it built on mathematics that had been sitting, unread, in the papers of a man who died in 1855.

Why does the same algorithm keep appearing? Because the symmetries of roots of unity make it inevitable. Once you see the structure, the algorithm writes itself. Those symmetries now power zero-knowledge proofs.

This chapter develops the univariate polynomial paradigm: finite fields, roots of unity, and the techniques that make systems like Groth16, PLONK, and STARKs possible. Where Chapter 4 explored multilinear polynomials over the Boolean hypercube, here we explore a single variable of high degree over a very different domain.

Finite Fields: The Algebraic Foundation

Zero-knowledge proofs live in finite fields. Not the real numbers, not the integers; finite fields, where arithmetic wraps around and every division is exact.

A finite field $\mathbb{F}_p$ consists of the integers $\{0, 1, 2, \ldots, p-1\}$ with arithmetic modulo a prime p. Addition and multiplication work as usual, then you take the remainder when dividing by p:

$$3 + 5 = 8 \equiv 1 \pmod{7}$$

$$3 \times 5 = 15 \equiv 1 \pmod{7}$$

Division is where finite fields differ from ordinary modular arithmetic. Every nonzero element has a multiplicative inverse: this is guaranteed because p is prime. (More generally, finite fields exist for any prime power p^k, but prime fields $\mathbb{F}_p$ are the simplest case.) In $\mathbb{F}_7$, we have $3^{-1} = 5$ because $3 \times 5 = 15 \equiv 1$. You can divide by any nonzero element, and the result is exact (no fractions, no approximations).

This is why we call it a *field*. A *ring* (like the integers $\mathbb{Z}$) lets you add, subtract, and multiply. A *field* lets you also divide. The integers are not a field because $1/2$ isn't an integer. But in $\mathbb{F}_7$, division always works: $1/2 = 1 \cdot 2^{-1} = 1 \cdot 4 = 4$, since $2 \cdot 4 = 8 \equiv 1$.

The nonzero elements $\mathbb{F}_p^* = \{1, 2, \ldots, p-1\}$ form a **cyclic group** under multiplication. This is fundamental: there exists a **generator** g such that every nonzero element is some power of g.

Example in $\mathbb{F}_7$: The element 3 generates everything:

Power	$3^k \mod 7$
3^1	3
3^2	2
3^3	6
3^4	4
3^5	5
3^6	1

Every nonzero element appears exactly once. The powers cycle through all of $\mathbb{F}_7^*$ before returning to 1.

For cryptographic applications, we use primes of 256 bits or more. The field is vast, roughly 2^{256} elements, making exhaustive search impossible.

Roots of Unity

Because $\mathbb{F}_p^*$ is cyclic of order $p-1$, it contains subgroups of every order dividing $p-1$. The most useful are the **roots of unity**.

An element $\omega \in \mathbb{F}_p$ is an n**-th root of unity** if $\omega^n = 1$. It's a **primitive** n-th root if additionally $\omega^k \neq 1$ for any $0 < k < n$: the smallest positive power that gives 1 is exactly n.

If ω is a primitive n-th root, the complete set of n-th roots is:

$$H = \{1, \omega, \omega^2, \ldots, \omega^{n-1}\}$$

This is a subgroup of order n. It's the evaluation domain that powers univariate-based SNARKs.

Worked Example: Fourth Roots in $\mathbb{F}_{17}$

Take $p = 17$. The multiplicative group has order $16 = 2^4$. Since 4 divides 16, fourth roots of unity exist.

Is $\omega = 4$ a primitive fourth root?

$$4^1 = 4$$

$$4^2 = 16 \equiv -1 \pmod{17}$$

$$4^3 = 64 \equiv 13 \equiv -4 \pmod{17}$$

$$4^4 = 256 \equiv 1 \pmod{17}$$

Yes. The fourth roots of unity are:

$$H = \{1, 4, 16, 13\} = \{1, 4, -1, -4\}$$

Notice the structure: 4 and $-4 = 13$ are negatives of each other, as are 1 and $-1 = 16$. This is not a coincidence.

The Symmetries

Roots of unity have two key symmetries that enable fast algorithms.

Symmetry 1: Squaring Halves the Group

When n is even:

$$\omega^{n/2} = -1$$

Why is this true? Start with the defining property: $\omega^n = 1$. Taking the square root of both sides: $(\omega^{n/2})^2 = 1$. So $\omega^{n/2}$ is a square root of 1. In any field, the square roots of 1 are exactly 1 and -1. But $\omega^{n/2} \neq 1$ because ω is *primitive*: its first power to equal 1 is ω^n, not $\omega^{n/2}$. Therefore $\omega^{n/2} = -1$.

This has a remarkable consequence. If you square every element of H:

$$(\omega^k)^2 = \omega^{2k}$$

The squares form the $(n/2)$-th roots of unity. And since $(\omega^{k+n/2})^2 = (\omega^k \cdot \omega^{n/2})^2 = (\omega^k)^2 \cdot 1 = (\omega^k)^2$, each square root of unity appears exactly twice.

In $\mathbb{F}_{17}$: Squaring the fourth roots $\{1, 4, 16, 13\}$:

$$1^2 = 1, \quad 4^2 = 16, \quad 16^2 = 1, \quad 13^2 = 16$$

The squares are $\{1, 16\}$: the square roots of unity, each appearing twice.

Symmetry 2: Opposite Elements are Negatives

Elements half a cycle apart are negatives:

$$\omega^{k+n/2} = \omega^k \cdot \omega^{n/2} = -\omega^k$$

In $\mathbb{F}_{17}$:

- $\omega^0 = 1$ and $\omega^2 = 16 = -1$
- $\omega^1 = 4$ and $\omega^3 = 13 = -4$

These two symmetries, squaring halves the group and opposites are negatives, are the engine of the Fast Fourier Transform.

The DFT Is Polynomial Evaluation

Here is one of those facts that seems almost too good to be true.

The Discrete Fourier Transform (DFT) is defined as a matrix-vector multiplication. Given a vector $(c_0, c_1, \ldots, c_{n-1})$, the DFT produces a new vector whose k-th entry is:

$$\sum_{j=0}^{n-1} c_j \cdot \omega^{jk}$$

where ω is a primitive n-th root of unity.

If you've seen the continuous Fourier transform, this is the same idea. The continuous version projects a function onto $e^{i\theta} = \cos\theta + i\sin\theta$ via integration, measuring how much of each frequency is present. Here, the integral becomes a sum, and the exponentials become n-th roots of unity: $\omega^k = e^{2\pi ik/n}$, equally spaced points on the unit circle. The projection interpretation is identical. You're decomposing a signal into frequency components; the discretization just replaces integration with summation.

Now look at polynomial evaluation. Given a polynomial $P(X) = c_0 + c_1 X + \cdots + c_{n-1}X^{n-1}$, evaluate it at ω^k:

$$P(\omega^k) = \sum_{j=0}^{n-1} c_j \cdot (\omega^k)^j = \sum_{j=0}^{n-1} c_j \cdot \omega^{jk}$$

They are identical. The DFT of the coefficient vector *is* the evaluation vector at roots of unity. This is not a useful analogy or a computational trick. It is a mathematical identity.

The FFT, then, is not "like" converting between polynomial representations. It *is* converting between polynomial representations. Coefficient form and evaluation form are the two natural bases for the same vector space, and the DFT matrix is the change-of-basis matrix. The FFT is the fast algorithm for this change of basis, made possible by the recursive structure of roots of unity.

This is why the same algorithm appears in signal processing, image compression, and zero-knowledge proofs. They are the same mathematical operation in different disguises.

Two Representations of Polynomials

A polynomial of degree less than n can be viewed in two ways.

Coefficient form: The polynomial is stored as its coefficients.

$$P(X) = c_0 + c_1 X + c_2 X^2 + \cdots + c_{n-1} X^{n-1}$$

Evaluation form: The polynomial is stored as its values at n distinct points. Using the n-th roots of unity:

$$[P(1), P(\omega), P(\omega^2), \ldots, P(\omega^{n-1})]$$

These two forms carry exactly the same information. A polynomial of degree less than n is uniquely determined by its values at any n points (this is Lagrange interpolation). The coefficient form and evaluation form are just two different coordinate systems for the same object.

Why care about evaluation form? In zero-knowledge proofs, constraints are naturally expressed as evaluations. Gate i must satisfy some relation; this becomes: the constraint polynomial $C(X)$ must equal zero at ω^i. The evaluation form directly represents these constraints.

Polynomial Evaluation as Inner Product

In Chapter 4, we saw that evaluating a multilinear polynomial is an inner product: $\tilde{f}(r) = \langle \vec{f}, \vec{L}(r) \rangle$. The same structure appears for univariate polynomials, in two forms.

In coefficient form:

$$P(z) = c_0 + c_1 z + c_2 z^2 + \cdots + c_{n-1} z^{n-1} = \langle \vec{c}, \vec{z} \rangle$$

where $\vec{c} = (c_0, c_1, \ldots, c_{n-1})$ is the coefficient vector and $\vec{z} = (1, z, z^2, \ldots, z^{n-1})$ is the "powers of z" vector.

In evaluation form, the same polynomial can be written via Lagrange interpolation:

$$P(z) = \sum_{i=0}^{n-1} P(\omega^i) \cdot L_i(z) = \langle \vec{P}, \vec{L}(z) \rangle$$

where $\vec{P} = (P(1), P(\omega), \ldots, P(\omega^{n-1}))$ is the evaluation vector and $\vec{L}(z)$ is the vector of Lagrange basis evaluations. Each $L_i(z) = \prod_{j \neq i} \frac{z - \omega^j}{\omega^i - \omega^j}$ is the unique degree-$(n-1)$ polynomial that equals 1 at ω^i and 0 at all other roots of unity (Chapter 2). We'll see a cleaner closed form for roots of unity later in this chapter.

Either way, polynomial evaluation is an inner product. Committing to a polynomial reduces to committing to a vector; proving an evaluation reduces to proving an inner product claim.

The difference from Chapter 4 is computational. For multilinear polynomials, the Lagrange basis factors beautifully:

$$L_w(r) = \prod_{i=1}^{n} (r_i \cdot w_i + (1 - r_i)(1 - w_i))$$

Each term depends on one coordinate; the product of n terms costs $O(n)$ per basis element. With 2^n basis elements, streaming through all of them takes $O(2^n)$ total.

For univariate polynomials, no such factorization exists. Each $L_i(z) = \prod_{j \neq i} \frac{z - \omega^j}{\omega^i - \omega^j}$ is a product of $n-1$ terms that all depend on the same variable z. Computing one basis element costs $O(n)$; computing all n of them naively costs $O(n^2)$. The FFT is what rescues us.

We'll exploit the inner product connection extensively in Chapter 9.

Two ways to commit: This duality (coefficient form vs evaluation form) manifests directly in polynomial commitment schemes:

- **KZG** (Chapter 9) commits in coefficient form: $C = g^{f(\tau)} = \prod_i (g^{\tau^i})^{c_i}$. The commitment encodes "evaluate the coefficients at a secret point τ."
- **FRI** (Chapter 10) commits in evaluation form: a Merkle tree over $[f(1), f(\omega), \ldots, f(\omega^{n-1})]$. The commitment is a hash of all the evaluations.

The FFT is what makes these equivalent: you can convert between representations in $O(n \log n)$ time. But the choice of representation affects everything: proof size, prover cost, setup requirements, and the algebraic tricks available for verification.

The Fast Fourier Transform

Converting between coefficient and evaluation form naively takes $O(n^2)$ operations: you'd compute each of n evaluations, each requiring $O(n)$ work.

The **Fast Fourier Transform (FFT)** does it in $O(n \log n)$. This speedup is essential; without it, the polynomials in modern proof systems would be computationally intractable.

The FFT exploits the symmetries of roots of unity through divide-and-conquer.

The Core Idea

Split a polynomial into its even and odd terms:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

where:

- $P_{\text{even}}(Y) = c_0 + c_2 Y + c_4 Y^2 + \cdots$ (even-indexed coefficients)
- $P_{\text{odd}}(Y) = c_1 + c_3 Y + c_5 Y^2 + \cdots$ (odd-indexed coefficients)

Both have half the degree of P.

Now, when we square the n-th roots of unity, we get the $(n/2)$-th roots (each appearing twice). So to evaluate P at all of H, we:

1. Recursively evaluate P_{even} and P_{odd} at the $(n/2)$-th roots
2. Combine the results

The combination uses the antisymmetry property:

$$P(\omega^k) = P_{\text{even}}(\omega^{2k}) + \omega^k \cdot P_{\text{odd}}(\omega^{2k})$$

$$P(\omega^{k+n/2}) = P_{\text{even}}(\omega^{2k}) - \omega^k \cdot P_{\text{odd}}(\omega^{2k})$$

Proof of first equation: By definition, $P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$. Substituting $X = \omega^k$: $P(\omega^k) = P_{\text{even}}((\omega^k)^2) + \omega^k \cdot P_{\text{odd}}((\omega^k)^2) = P_{\text{even}}(\omega^{2k}) + \omega^k \cdot P_{\text{odd}}(\omega^{2k})$.

Proof of second equation: Substitute $X = \omega^{k+n/2} = -\omega^k$: $P(-\omega^k) = P_{\text{even}}((-\omega^k)^2) + (-\omega^k) \cdot P_{\text{odd}}((-\omega^k)^2) = P_{\text{even}}(\omega^{2k}) - \omega^k \cdot P_{\text{odd}}(\omega^{2k})$. The even part is unchanged (squaring kills the sign); the odd part flips sign. □

Two evaluations of P from one evaluation each of P_{even} and P_{odd}: the same work computes both, with just an addition versus subtraction.

Worked Example: 4-Point FFT

Evaluate $P(X) = 5 + 3X + X^2 + 2X^3$ at $H = \{1, 4, 16, 13\}$ in $\mathbb{F}_{17}$.

Split:

- $P_{\text{even}}(Y) = 5 + Y$ (coefficients $c_0 = 5, c_2 = 1$)
- $P_{\text{odd}}(Y) = 3 + 2Y$ (coefficients $c_1 = 3, c_3 = 2$)

Evaluate on $\{1, 16\}$ (the square roots of unity):

Y	$P_{\text{even}}(Y) = 5 + Y$	$P_{\text{odd}}(Y) = 3 + 2Y$
1	6	5

Y	$P_{\text{even}}(Y) = 5 + Y$	$P_{\text{odd}}(Y) = 3 + 2Y$
16	$21 \equiv 4$	$35 \equiv 1$

Combine using $\omega^0 = 1, \omega^1 = 4, \omega^2 = 16, \omega^3 = 13$:

$$P(1) = P_{\text{even}}(1) + 1 \cdot P_{\text{odd}}(1) = 6 + 5 = 11$$

$$P(4) = P_{\text{even}}(16) + 4 \cdot P_{\text{odd}}(16) = 4 + 4 = 8$$

$$P(16) = P_{\text{even}}(1) - 1 \cdot P_{\text{odd}}(1) = 6 - 5 = 1$$

$$P(13) = P_{\text{even}}(16) - 4 \cdot P_{\text{odd}}(16) = 4 - 4 = 0$$

Result: $[P(1), P(4), P(16), P(13)] = [11, 8, 1, 0]$.

Verification: $P(4) = 5 + 3(4) + 16 + 2(64) = 5 + 12 + 16 + 128 = 161 \equiv 8 \pmod{17}$. Correct.

The inverse FFT, going from evaluations back to coefficients, uses the same algorithm with ω^{-1} instead of ω and a factor of $1/n$.

A note on terminology. When the FFT operates over a finite field rather than the complex numbers, it is called the **Number Theoretic Transform (NTT)**. The algorithm is identical. The only difference is the domain: complex roots of unity $e^{2\pi ik/n}$ become finite-field roots of unity $\omega^k \in \mathbb{F}_p$. Every FFT computation in this book is technically an NTT. Implementation papers and libraries (gnark, arkworks, Plonky2) use "NTT" exclusively, so recognizing the equivalence matters when moving from theory to code.

The Vanishing Polynomial

Here is the central insight of univariate arithmetization.

The **vanishing polynomial** of a set H is:

$$Z_H(X) = \prod_{h \in H} (X - h)$$

For the n-th roots of unity, this simplifies dramatically:

$$Z_H(X) = X^n - 1$$

Proof: By definition, $h \in H$ means $h^n = 1$, so every element of H is a root of $X^n - 1$. Since $|H| = n$ and $X^n - 1$ has degree n, these are *all* the roots. By the factor theorem, $X^n - 1 = \prod_{h \in H}(X - h) = Z_H(X)$. □

The key theorem: A polynomial $C(X)$ vanishes at every point of H if and only if $Z_H(X)$ divides $C(X)$.

Proof: ($\Leftarrow$) If $C(X) = Q(X) \cdot Z_H(X)$, then for any $h \in H$: $C(h) = Q(h) \cdot Z_H(h) = Q(h) \cdot 0 = 0$.

($\Rightarrow$) If $C(h) = 0$ for all $h \in H$, then each $(X - h)$ divides $C(X)$. Since the $(X - h)$ are coprime (distinct linear factors), their product $Z_H(X)$ divides $C(X)$. □

This is the compression at the heart of univariate SNARKs:

1. Encode n constraints as: "$C(\omega^i) = 0$ for all i"
2. This is equivalent to: "$Z_H(X)$ divides $C(X)$"
3. Which is equivalent to: "There exists $Q(X)$ such that $C(X) = Q(X) \cdot Z_H(X)$"

One polynomial divisibility condition captures n separate constraint checks.

The Divisibility Check

How do we verify divisibility efficiently?

The prover computes the quotient $Q(X) = C(X)/Z_H(X)$ and commits to it. The verifier picks a random challenge $z \in \mathbb{F}$ and checks:

$$C(z) \stackrel{?}{=} Q(z) \cdot Z_H(z)$$

If $C(X) = Q(X) \cdot Z_H(X)$ as polynomials, this equation holds for all z, including the random one.

If $C(X) \neq Q(X) \cdot Z_H(X)$, their difference is a nonzero polynomial. By Schwartz-Zippel, a random z catches this disagreement with probability at least $1 - d/|\mathbb{F}|$, where d is the degree.

A single random check verifies all n constraints.

Lagrange Interpolation over Roots of Unity

We saw earlier that the Lagrange basis $L_i(X) = \prod_{j \neq i} \frac{X - \omega^j}{\omega^i - \omega^j}$ is the polynomial that equals 1 at ω^i and 0 at all other roots. For roots of unity, this product simplifies to a closed form:

$$L_i(X) = \frac{\omega^i}{n} \cdot \frac{X^n - 1}{X - \omega^i}$$

Why does this work? The numerator $X^n - 1$ vanishes at all n-th roots of unity. Dividing by $(X - \omega^i)$ removes the zero at ω^i, leaving a polynomial that vanishes at all roots *except* ω^i. The prefactor $\frac{\omega^i}{n}$ normalizes so that $L_i(\omega^i) = 1$.

Worked example: Let $n = 4$ in $\mathbb{F}_5$. Here $\omega = 2$ is a primitive 4th root of unity: $2^1 = 2, 2^2 = 4, 2^3 = 3$, $2^4 = 1$. The roots are $\{1, 2, 4, 3\}$.

For $L_1(X)$, the polynomial that equals 1 at $\omega^1 = 2$ and 0 at $\{1, 4, 3\}$:

$$L_1(X) = \frac{2}{4} \cdot \frac{X^4 - 1}{X - 2}$$

In $\mathbb{F}_5$, we have $4^{-1} = 4$ (since $4 \cdot 4 = 16 \equiv 1$), so $\frac{2}{4} = 2 \cdot 4 = 8 \equiv 3$.

Factor $X^4 - 1 = (X - 1)(X - 2)(X - 4)(X - 3)$ over $\mathbb{F}_5$. Dividing out $(X - 2)$:

$$L_1(X) = 3 \cdot (X - 1)(X - 4)(X - 3)$$

Check at $X = 2$: $L_1(2) = 3 \cdot (2 - 1)(2 - 4)(2 - 3) = 3 \cdot (1)(-2)(-1) = 3 \cdot 2 = 6 \equiv 1$. ✓

Check at $X = 1$: $L_1(1) = 3 \cdot (0)(-3)(-2) = 0$. ✓

The polynomial passing through points (ω^i, y_i) is then $P(X) = \sum_{i=0}^{n-1} y_i \cdot L_i(X)$.

Cosets: Shifting the Domain

Lagrange interpolation just did something powerful: it extended values defined on H (the roots of unity) to a polynomial defined on all of $\mathbb{F}$. This is the univariate analog of multilinear extension from Chapter 4. There, we extended a function on the Boolean hypercube $\{0, 1\}^n$ to all of $\mathbb{F}^n$. Here, we extend a function on roots of unity to all of $\mathbb{F}$.

But sometimes we need more than just extension. We need structured evaluation points *outside* H. **Cosets** provide exactly this.

If $k \notin H$ is any nonzero field element, then:

$$k \cdot H = \{k, k\omega, k\omega^2, \ldots, k\omega^{n-1}\}$$

is a coset of H. It's a "shifted" copy: n new points, disjoint from H.

Worked example: In $\mathbb{F}_{13}$, let $\omega = 5$ (a primitive 4th root: $5^2 = 12, 5^3 = 8, 5^4 = 1$). The subgroup is $H = \{1, 5, 12, 8\}$.

Take $k = 2$. The coset is $2H = \{2, 10, 11, 3\}$. The two sets are disjoint, giving 8 evaluation points.

The key property: to evaluate $P(X)$ on $2H$, you don't need a new algorithm. If $P(X) = c_0 + c_1X + c_2X^2 + c_3X^3$, then evaluating at $2\omega^i$ is the same as evaluating $P'(X) = c_0 + 2c_1X + 4c_2X^2 + 8c_3X^3$ at ω^i. Scale the coefficients by powers of k, then run the standard FFT on H. Cosets give you new evaluation domains for free.

Why cosets matter in ZK: Several proof systems depend on cosets:

- **PLONK's permutation argument**: Uses multiple cosets to encode wire positions. If you have n gates with 3 wires each (a, b, c), PLONK encodes them on H, kH, and k^2H (three disjoint domains of size n each). This lets the permutation polynomial distinguish "wire a of gate 5" from "wire b of gate 5."
- **FRI's low-degree testing**: The prover evaluates on a domain larger than the polynomial's degree (for "rate" or "blowup"). Using $H \cup kH$ doubles the evaluation domain while maintaining FFT structure.
- **Quotient degree management**: If $C(X)$ has degree $2n$ but we've only committed to evaluations on H (size n), we need more points to pin down the quotient. Using $H \cup kH$ gives $2n$ points (enough to determine a polynomial of degree less than $2n$).

The FFT works on cosets too: just multiply each root of unity by k before running the algorithm.

The Quotient Argument

The divisibility check above verified vanishing on a *set* of points (all of H). The quotient argument is the single-point version: prove that $P(z) = y$ for a committed polynomial P.

The **factor theorem** says: $P(z) = y$ if and only if $(X - z)$ divides $P(X) - y$.

The prover computes:

$$Q(X) = \frac{P(X) - y}{X - z}$$

If $P(z) = y$, this is a polynomial. If not, the division has a remainder; Q isn't a polynomial.

The verifier checks the polynomial identity:

$$P(X) - y = Q(X) \cdot (X - z)$$

at a random point. This is the foundation of KZG opening proofs (Chapter 9).

Univariate vs. Multilinear

We now have two paradigms for polynomial proofs:

Aspect	Multilinear	Univariate
Variables	n variables, degree 1 each	1 variable, degree $N-1$
Domain	Boolean hypercube $\{0,1\}^n$	Roots of unity H
Size	$N = 2^n$ points	N points
Constraint encoding	Sum over hypercube	Divisibility by Z_H
Key algorithm	Recursive halving	FFT
Prover cost	$O(N)$ (linear)	$O(N \log N)$ (quasi-linear)
Verification	Sum-check protocol	Random evaluation
Systems	GKR, Spartan, Lasso	PLONK, Marlin, STARKs

Both achieve the same essential goal: reduce exponentially many constraint checks to a constant number of random evaluations. They're complementary perspectives on the same phenomenon (the rigidity of low-degree polynomials).

A note on Groth16: Groth16 uses univariate polynomials but doesn't require roots of unity; it encodes constraints via QAP (Quadratic Arithmetic Programs) and verifies satisfaction through pairing equations, not divisibility checks at structured domains. Provers *can* use FFT as an optimization for polynomial arithmetic, but it's not fundamental to the protocol. PLONK and STARKs, by contrast, rely structurally on roots of unity: constraints are encoded as "polynomial vanishes on H," checked via the divisibility pattern described above.

Key takeaways

1. **Finite fields** provide exact arithmetic with every nonzero element invertible. The nonzero elements form a cyclic group.
2. **Roots of unity** are elements with $\omega^n = 1$. They form a subgroup of size n when n divides $p-1$.
3. **The key symmetries**: Squaring halves the group; opposite elements are negatives. These enable the FFT.
4. **Two representations**: Polynomials can be stored as coefficients or evaluations. The FFT converts between them in $O(n \log n)$ time.

5. **The vanishing polynomial** $Z_H(X) = X^n - 1$ captures all roots of unity. A polynomial vanishes on H iff Z_H divides it.
6. **Constraint compression**: n constraints "$C(\omega^i) = 0$" become one divisibility "$Z_H | C$", verified by one random check.
7. **Lagrange interpolation** over roots of unity has a clean closed form exploiting the structure of Z_H.
8. **Cosets** extend the domain while preserving FFT-friendliness.
9. **Quotient arguments** prove evaluation claims: to show $P(z) = y$, prove $(X - z)$ divides $P(X) - y$.
10. **The FFT exists because of roots of unity.** The algorithm is a direct consequence of the symmetries $\omega^{n/2} = -1$ and $(\omega^k)^2 = \omega^{2k}$.

Chapter 6: Commitment Schemes: Cryptographic Binding

In 1981, Manuel Blum posed a simple question: can two people play a fair game of coin-flipping over the telephone?

Blum was working on what cryptographers called **Mental Poker**: how can two people play a card game over the phone without a trusted dealer? How do I know you didn't shuffle the Aces to the top of the deck? The coin flip was the atomic unit of this problem. Get that right, and you could build up to full card games.

The problem seems impossible. Alice flips a coin and announces "heads." Bob has no way to verify she actually flipped anything. She might have waited to hear his guess first. Or she might change her answer after hearing his response. Without shared physical reality, without a coin both parties can see, how can either trust the outcome?

Blum's solution introduced one of the most fundamental primitives in cryptography. Alice doesn't announce her flip directly. Instead, she first sends a *commitment*: a cryptographic object that locks in her choice without revealing it. Only after Bob makes his guess does Alice *open* the commitment, proving what she had chosen all along. The commitment is binding (Alice cannot change her answer after sending it) and hiding (Bob learns nothing until the reveal).

This two-phase structure, commit then reveal, turns out to be exactly what our proof systems need. You've designed a protocol where the prover claims a polynomial evaluates to some value, and you want to check this with random queries. But the prover responds *after* seeing your challenge. What stops them from constructing a fake polynomial that happens to pass your spot-checks?

This is the **binding problem**. The verifier's randomness is meant to catch a cheating prover off-guard. But if the prover can adapt their answers after seeing the challenge, they can tailor responses to pass. The polynomial identity testing that underlies our protocols becomes meaningless.

We need a mechanism that forces the prover to fix their polynomial before verification begins.

The Trust Problem Revisited

Consider the sum-check protocol from Chapter 3. The verifier sends random challenges $r_1, r_2, ...$, and the prover responds with univariate polynomials. At the end, the verifier must check that some claimed evaluation matches the actual polynomial. But how does the verifier know the prover didn't just fabricate a polynomial that happens to satisfy the final check?

The issue is subtle. Our soundness proofs assumed the prover is committed to *some* polynomial before the interaction begins. But in a raw interactive protocol, nothing enforces this. A dishonest prover could:

1. Wait to see all the verifier's challenges
2. Work backwards to construct a polynomial that passes
3. Claim they had this polynomial all along

This attack doesn't violate the *information-theoretic* soundness of the protocol; it violates the *execution model*. We assumed a sequential game where the prover moves first; in reality, we need cryptography to enforce this ordering.

The Commitment Paradigm

A **commitment scheme** solves this problem through a two-phase protocol:

Phase 1 (Commit): The prover publishes a *commitment*, a short, seemingly random string that binds them to a value without revealing it.

Phase 2 (Reveal): Later, the prover can open the commitment by revealing the original value. Anyone can verify that the revealed value matches the original commitment.

Formal Properties:

- **Binding**: Once committed, the committer cannot open to a different value. More precisely, no efficient adversary can find two different values that produce the same commitment.
- **Hiding**: The commitment reveals nothing about the committed value. An observer cannot distinguish between commitments to different values.

These properties exist in tension. Perfect binding means each value maps to a unique commitment, but then the commitment might leak information about the value. Perfect hiding means commitments are statistically indistinguishable, but then multiple values might share commitments. Cryptographic schemes typically achieve one property perfectly and the other computationally.

Pedersen Commitments: The Discrete Log Approach

The most elegant commitment scheme comes from a surprising source: the hardness of computing discrete logarithms in cyclic groups.

Setup: Let G be a cyclic group of prime order q (think of an elliptic curve group). Select two generators g and h such that *nobody knows* the discrete logarithm $\log_g h$. The public parameters are (G, q, g, h).

Commit: To commit to a value $m \in \mathbb{Z}_q$, the committer:

1. Chooses a random *blinding factor* $r \leftarrow \mathbb{Z}_q$
2. Computes the commitment $C = g^m \cdot h^r$

Reveal: To open, the committer reveals (m, r). The verifier checks that $g^m \cdot h^r = C$.

The scheme uses multiplicative notation, but on elliptic curves (the dominant implementation), we write $C = m \cdot G + r \cdot H$ using additive notation.

Why Binding Holds

Suppose Alice commits $C = g^m h^r$ and later wants to open it as a *different* value $m' \neq m$. She needs to find r' such that:

$$g^m h^r = g^{m'} h^{r'}$$

Rearranging:

$$g^{m-m'} = h^{r'-r}$$

This means:

$$\log_g h = \frac{m - m'}{r' - r}$$

But computing $\log_g h$ is the discrete logarithm problem! If Alice could find such (m', r'), she could break DLog in G. The binding property holds computationally, as long as discrete log is hard.

Formal reduction: Suppose adversary $\mathcal{A}$ breaks binding with non-negligible probability, outputting (m, r) and (m', r') with $m \neq m'$ and $g^m h^r = g^{m'} h^{r'}$. We construct a DLog solver $\mathcal{B}$: given challenge

h, run $\mathcal{A}$ to get the two openings, then compute $\log_g h = (m-m')/(r'-r) \mod q$. Note $r' \neq r$ since $m \neq m'$ would otherwise give $g^{m-m'} = 1$, implying $m = m' \mod q$. Thus $\mathcal{B}$ solves DLog whenever $\mathcal{A}$ breaks binding.

Why Hiding Holds

The commitment $C = g^m h^r$ is **perfectly hiding**. Since r is uniformly random in $\mathbb{Z}_q$ and h is a generator of G, the term h^r is uniformly distributed over all of G.

For any message m, the commitment $C = g^m \cdot h^r$ is a uniformly random group element. This means:

- Commitment to $m_1 \sim \text{Uniform}(G)$
- Commitment to $m_2 \sim \text{Uniform}(G)$

The two distributions are *statistically* identical, not merely computationally indistinguishable. Even an unbounded adversary cannot determine the committed value from the commitment alone.

The Independence Requirement

There's a critical subtlety: the generators g and h must be *independently chosen* such that nobody knows $\log_g h$.

If Alice knows that $h = g^x$ for some x, she can break binding:

$$C = g^m h^r = g^m (g^x)^r = g^{m+xr}$$

She can open this as (m', r') for any m' by computing $r' = r + (m - m')/x$. The verification passes because:

$$g^{m'} h^{r'} = g^{m'} g^{x(r+(m-m')/x)} = g^{m'+xr+m-m'} = g^{m+xr} = C$$

If Alice knows this relationship $h = g^x$, she holds a **trapdoor**. It allows her to open the commitment to *any* value she wants. This is why trusted setups in SNARKs are so sensitive: if the creator knows the "toxic waste" (the secret exponents used to generate the parameters), they can forge proofs. We prevent this by generating g and h from "nothing-up-my-sleeve" numbers like the digits of π or by hashing different strings to curve points, ensuring nobody knows the discrete log relationship.

Worked Example: Pedersen Commitment in $\mathbb{Z}_{23}^*$

Let's trace through a concrete example using the multiplicative group modulo 23.

Setup: Work in $\mathbb{Z}_{23}^*$, which has order $\phi(23) = 22$. Take generators $g = 5$ and $h = 7$. We assume nobody knows $\log_5 7$.

Commitment to $m = 10$:

- Choose random blinding factor $r = 3$
- Compute $C = g^m \cdot h^r = 5^{10} \cdot 7^3 \pmod{23}$

Computing $5^{10} \pmod{23}$:

- $5^2 = 25 \equiv 2$
- $5^4 \equiv 4$
- $5^8 \equiv 16$
- $5^{10} = 5^8 \cdot 5^2 \equiv 16 \cdot 2 = 32 \equiv 9$

Computing $7^3 \pmod{23}$:

- $7^2 = 49 \equiv 3$
- $7^3 = 7 \cdot 3 = 21$

So $C = 9 \cdot 21 = 189 \equiv 5 \pmod{23}$.

Verification: Given $(m = 10, r = 3)$, the verifier checks:

$$5^{10} \cdot 7^3 \equiv 9 \cdot 21 \equiv 5 \pmod{23} \checkmark$$

The commitment opens correctly.

The Homomorphic Property

Pedersen commitments have a remarkable algebraic property: they are **additively homomorphic**. You can compute on committed values without knowing what they are.

Given two commitments:

$$C_1 = g^{m_1} h^{r_1} \quad \text{and} \quad C_2 = g^{m_2} h^{r_2}$$

Their product is:

$$C_1 \cdot C_2 = g^{m_1} h^{r_1} \cdot g^{m_2} h^{r_2} = g^{m_1 + m_2} h^{r_1 + r_2}$$

This is a valid commitment to $m_1 + m_2$ with blinding factor $r_1 + r_2$!

Worked Example (continuing):

Commit to $m_2 = 4$ with $r_2 = 6$:

$$C_2 = 5^4 \cdot 7^6 \pmod{23}$$

Computing $5^4 \equiv 4$ and $7^6 = (7^3)^2 \equiv 21^2 = 441 \equiv 441 - 19 \cdot 23 = 441 - 437 = 4$.

So $C_2 = 4 \cdot 4 = 16$.

Homomorphic addition:

$$C_3 = C_1 \cdot C_2 = 5 \cdot 16 = 80 \equiv 80 - 3 \cdot 23 = 80 - 69 = 11 \pmod{23}$$

This should be a commitment to $m_1 + m_2 = 14$ with blinding factor $r_1 + r_2 = 9$.

Verification:

$$5^{14} \cdot 7^9 \pmod{23}$$

For $5^{14} = 5^{10} \cdot 5^4 \equiv 9 \cdot 4 = 36 \equiv 13$.

For $7^9 = 7^6 \cdot 7^3 \equiv 4 \cdot 21 = 84 \equiv 84 - 3 \cdot 23 = 84 - 69 = 15$.

So $5^{14} \cdot 7^9 \equiv 13 \cdot 15 = 195 \equiv 195 - 8 \cdot 23 = 195 - 184 = 11 \pmod{23}$.

It matches $C_3 = 11$.

This property is extraordinarily useful. A verifier can combine multiple commitments, add constants, or compute linear combinations, all without learning the committed values. This enables protocols where computations happen "in the encrypted domain."

Scalar Multiplication

The homomorphic property extends to scalar multiplication. For a constant k:

$$(C)^k = (g^m h^r)^k = g^{km} h^{kr}$$

This is a commitment to $k \cdot m$ with blinding factor $k \cdot r$. The verifier can scale committed values without opening them.

From Scalar to Vector Commitments

The Pedersen scheme naturally extends from committing to a single value to committing to an entire vector. Given n independent generators $G_1, \ldots, G_n$ and a blinding generator H, we can commit to a vector $\vec{m} = (m_1, \ldots, m_n)$:

$$C = \sum_{i=1}^{n} m_i \cdot G_i + r \cdot H$$

This **Pedersen vector commitment** is still a single group element, regardless of the vector length. The homomorphic property extends: adding two vector commitments yields a commitment to the component-wise sum.

But here's where things get interesting for our purposes. Recall from Chapters 4 and 5 that a polynomial evaluation is just an inner product:

$$f(z) = \sum_{i=0}^{n-1} c_i z^i = \langle \vec{c}, \vec{z} \rangle$$

where $\vec{c} = (c_0, \ldots, c_{n-1})$ are the coefficients and $\vec{z} = (1, z, z^2, \ldots, z^{n-1})$ is the evaluation vector.

If we commit to the coefficient vector using a Pedersen vector commitment, we've effectively committed to the polynomial itself. The verifier knows the evaluation point z, so they know $\vec{z} = (1, z, z^2, \ldots)$. The prover knows $\vec{c}$. Proving that a claimed value v equals $f(z) = \langle \vec{c}, \vec{z} \rangle$ becomes an inner product argument: the prover convinces the verifier that their committed vector has the right inner product with the public vector $\vec{z}$.

Homomorphism alone doesn't give us inner products. But it's a key ingredient: inner product arguments (like Bulletproofs) use the additive homomorphism to recursively fold commitments, shrinking the problem size logarithmically. The commitment structure enables the protocol; additional machinery makes it work.

This observation, that polynomial evaluation reduces to inner product, is the conceptual bridge from simple commitments to full polynomial commitment schemes. We'll cross that bridge in Chapter 9.

Proving Knowledge of an Opening

A commitment alone proves nothing; the prover must eventually reveal the opening to be useful. But what if we want to prove we *know* a valid opening without revealing it?

This is where Σ-protocols (Chapter 16) enter the picture. A prover who knows the opening for a commitment can convince a verifier they know it without revealing the values. This is a *proof of knowledge*: the prover demonstrates possession of the witness (m, r), not a property of m.

Setup: The prover has committed $C = g^m h^r$, where m is the secret message and r is the blinding factor. Both are hidden. The prover wants to prove they know (m, r) without revealing either.

The protocol follows the classic three-move structure:

Round 1 (Commit to randomness): The prover picks random $d, s \leftarrow \mathbb{Z}_q$ and sends $T = g^d h^s$.

Round 2 (Challenge): The verifier sends a random challenge $e \leftarrow \mathbb{Z}_q$.

Round 3 (Response): The prover computes:

- $z_1 = d + e \cdot m$
- $z_2 = s + e \cdot r$

and sends (z_1, z_2).

Verification: The verifier checks:

$$g^{z_1} h^{z_2} \stackrel{?}{=} T \cdot C^e$$

Why it works: Expanding the right side:

$$T \cdot C^e = (g^d h^s) \cdot (g^m h^r)^e = g^{d+em} h^{s+er} = g^{z_1} h^{z_2}$$

The equation holds if the prover knows (m, r).

Why it's zero-knowledge: The values z_1 and z_2 look random because they're masked by the truly random d and s. The verifier learns nothing about m or r beyond the fact that the prover knows them.

Why it's sound: A prover who doesn't know (m, r) cannot answer two different challenges e and e' consistently. Given two accepting transcripts with the same T but different challenges, one can extract the witness; this is the "special soundness" property.

Worked Example: Proof of Knowledge

Continuing our example with $g = 5$, $h = 7$ in $\mathbb{Z}_{23}^*$, suppose the prover committed $C = 5$ and claims to know the opening.

Prover's commitment:

- Choose random $d = 8$, $s = 2$
- Compute $T = 5^8 \cdot 7^2 \pmod{23}$
- $5^8 \equiv 16$, $7^2 = 49 \equiv 3$
- $T = 16 \cdot 3 = 48 \equiv 2$

Verifier's challenge: $e = 4$

Prover's response (recall $m = 10$, $r = 3$):

- $z_1 = d + e \cdot m = 8 + 4 \cdot 10 = 48 \equiv 48 \pmod{22} = 4$ (arithmetic mod group order)
- $z_2 = s + e \cdot r = 2 + 4 \cdot 3 = 14$

Verification:

- Left side: $5^4 \cdot 7^{14} \pmod{23}$
  - $5^4 \equiv 4$
  - $7^{14} = 7^{11} \cdot 7^3$ (since $14 \equiv 14 \pmod{22}$)
  - $7^{11} = 7^8 \cdot 7^3$. We have $7^2 \equiv 3$, $7^4 \equiv 9$, $7^8 \equiv 81 \equiv 81 - 3 \cdot 23 = 12$
  - $7^{11} = 12 \cdot 21 = 252 \equiv 252 - 10 \cdot 23 = 22 \equiv -1$
  - $7^{14} = (-1) \cdot 21 = -21 \equiv 2$
  - Left side: $4 \cdot 2 = 8$
- Right side: $T \cdot C^e = 2 \cdot 5^4 \pmod{23}$
  - $5^4 \equiv 4$
  - Right side: $2 \cdot 4 = 8$

Both sides equal 8. The proof verifies.

Beyond Pedersen: A Landscape of Commitment Schemes

Pedersen commitments are beautiful but not the only option. Different commitment schemes offer different trade-offs:

Hash-Based Commitments: Commit as $C = H(m\|r)$ where H is a cryptographic hash. Binding follows from collision resistance; hiding follows from the hash acting as a random oracle. These are simple and quantum-resistant, but they lack the homomorphic property.

Polynomial Commitments: The heart of modern SNARKs. Instead of committing to a single value, we commit to an entire polynomial and can later prove evaluations at arbitrary points. Chapter 9 explores KZG (using pairings) and IPA (using discrete log) in depth.

ElGamal-style Commitments: Related to encryption, where the commitment can be "decrypted" with a secret key. Useful in some multi-party protocols.

Each scheme involves trade-offs between:

- **Setup**: Does it require a trusted setup?
- **Assumptions**: Discrete log? Pairings? Hashes?
- **Efficiency**: Commitment size, proof size, computation time
- **Properties**: Homomorphic? Additively? Multiplicatively?
- **Quantum resistance**: Will it survive quantum computers?

Why Commitments Matter for ZK Proofs

We opened this chapter with the binding problem: how do we ensure the prover doesn't cheat by choosing their polynomial after seeing the verifier's challenges?

Commitment schemes provide the answer through the **commit-and-prove** paradigm:

1. **Commit phase**: Before any interaction, the prover commits to their polynomial (or the witness encoding it).
2. **Interaction phase**: The verifier sends challenges, the prover responds. But the prover's polynomial was fixed in step 1.
3. **Opening phase**: At the end, the prover opens relevant parts of their commitment. The verifier checks consistency.

The binding property ensures the prover cannot change their polynomial mid-protocol. The hiding property ensures the commitment itself doesn't leak information about the witness. Every modern SNARK (Groth16, PLONK, STARKs) follows this pattern, varying only in which commitment scheme they use (KZG for Groth16/PLONK, Merkle trees for STARKs).

The Hiding-Binding Tradeoff

There's a fundamental tension in commitment schemes that deserves attention: you cannot have both *perfect* hiding and *perfect* binding simultaneously.

Perfect binding means each commitment corresponds to exactly one value: no two distinct messages ever produce the same commitment. This is an information-theoretic guarantee: even with unlimited computation, opening to a different value is impossible.

Perfect hiding means the commitment reveals nothing about the value: all messages produce statistically indistinguishable commitment distributions. Again, this is information-theoretic: even unbounded adversaries learn nothing.

Why can't we have both? Consider what each requires:

- Perfect binding needs the commitment function to be injective (one-to-one). Every value maps to a unique commitment.
- Perfect hiding needs all commitments to look identical regardless of the input. The commitment must be independent of the value.

These requirements conflict. Perfect hiding means the distributions $\{\text{Commit}(m_0; r)\}$ and $\{\text{Commit}(m_1; r)\}$ are identical for all messages m_0, m_1. But if the distributions are identical,

every commitment value c must be reachable from both m_0 and m_1 (otherwise we could distinguish them). So there exist openings (m_0, r_0) and (m_1, r_1) that both produce c. Binding is broken.

The resolution: Relax one property to *computational* rather than *information-theoretic*.

An **information-theoretic** (or *statistical*, or *perfect*) guarantee holds against adversaries with unlimited computational power. No amount of computation can break it. A **computational** guarantee holds only against efficient (polynomial-time) adversaries. An unbounded adversary could break it, but doing so requires solving a problem believed to be hard (like discrete log or factoring).

The tradeoff:

- **Perfectly hiding, computationally binding**: Pedersen commitments. As we proved earlier, for any message m there exists an r that produces any given commitment, so an unbounded adversary cannot determine which value is inside. But finding two openings requires solving discrete log, so binding holds against efficient adversaries. Even an all-powerful being cannot tell which value is committed (perfect hiding), but a quantum computer could eventually break the lock (computational binding).
- **Perfectly binding, computationally hiding**: Hash-based commitments $C = H(m\|r)$. A hash function is deterministic: each (m, r) pair maps to exactly one commitment, and collision resistance means you cannot find two pairs that collide. The value is locked in tight (perfect binding). But an unbounded adversary could brute-force all possible inputs to find (m, r) (computational hiding).

This tradeoff shapes the design space. For ZK proofs, we typically want hiding (don't reveal the witness) and accept computational binding (secure against poly-time adversaries). Pedersen commitments are the natural choice: the witness stays perfectly hidden, and binding holds as long as discrete log is hard.

Looking Ahead

We've established the cryptographic primitive that makes succinct proofs possible. Commitments transform interactive protocols, where timing and ordering are honor-system, into cryptographically enforced games where cheating is computationally infeasible.

In Chapter 9, we'll see how polynomial commitment schemes (KZG, IPA, and FRI) extend these ideas to commit to polynomials and prove evaluations. These are the engines that power modern SNARKs.

But first, we need to understand *what* we're proving. Chapter 7 introduces the GKR protocol, which uses sum-check to verify layered arithmetic circuits. And Chapter 8 shows how arbitrary computations become circuits, which become polynomials. Together, these chapters complete the story of how a computation becomes a succinct proof.

Key takeaways

1. **The binding problem**: Interactive proofs need cryptographic enforcement to prevent provers from adapting their answers to verifier challenges.
2. **Commitment = seal**: A commitment locks in a value before revealing it. Binding ensures it can't change; hiding ensures it reveals nothing.
3. **Pedersen commitments**: $C = g^m h^r$ achieves perfect hiding and computational binding (from discrete log hardness). The generators g and h must have unknown discrete log relationship, or binding fails.
4. **Homomorphic structure**: Pedersen commitments allow addition in the committed domain ($C_1 \cdot C_2$ commits to $m_1 + m_2$), and extend naturally to vectors. Committing to a coefficient vector effectively commits to a polynomial.
5. **Proof of knowledge**: Sigma protocols let a prover demonstrate they know a commitment's opening without revealing it.
6. **Commit-and-prove paradigm**: The foundation of all modern SNARKs: commit first, then prove properties of the committed values.
7. **Bridge to polynomial commitments**: Polynomial evaluation is an inner product. This connects vector commitments to the polynomial commitment schemes (Chapter 9) that power SNARKs.

Chapter 7: The GKR Protocol: Verifying Circuits Layer by Layer

In 2006, Amazon launched AWS, and the world changed. Companies stopped buying servers and started renting "compute" from invisible data centers. It was efficient, but it created a trust gap. If a bank rents a server to calculate interest rates, how do they know the server isn't buggy, or malicious?

Verifying the computation by re-running it defeats the purpose of outsourcing. You want the cloud to do the heavy lifting, and you want to check the work with the effort of a text message.

In 2008, Shafi Goldwasser, Yael Kalai, and Guy Rothblum published a theoretical solution. They proposed a protocol where a supercomputer could prove a massive calculation to a laptop, and the laptop could verify it in seconds. While it took a decade for hardware and cryptographic engineering to catch up to their math, every modern rollup and scaling solution on Ethereum is spiritually a descendant of that 2008 paper.

The sum-check protocol is versatile. It transforms exponentially large sums (2^n terms) into verification that runs in $O(n)$ time, logarithmic in the sum size. But every application we've seen requires a custom polynomial tailored to that specific problem. Each new computation demands a new arithmetization.

What if we want to verify *any* computation, not just counting problems? The GKR protocol provides a *universal* framework for verifying any computation that can be expressed as an arithmetic circuit (which turns out to be everything). Rather than designing a new protocol for each problem, GKR gives us a machine: feed in a circuit, get out an efficient verification protocol.

From Sum-Check to General Computation

Let's understand the conceptual leap. The sum-check protocol verifies claims of the form:

$$H = \sum_{x \in \{0,1\}^n} g(x)$$

Given a polynomial g, it checks whether the claimed sum H is correct. The polynomial g encodes the problem, and sum-check verifies the encoding.

But computation is more than summation. A real computation involves:

- Input values
- Intermediate calculations (additions, multiplications)
- Data dependencies (the output of one step becomes the input to another)
- A final output

The insight of GKR is that these computations have *layered structure*. A circuit consists of gates organized into layers, where each layer's outputs feed into the next layer's inputs. The relationship between adjacent layers can be expressed as a polynomial identity, one that sum-check can verify.

> **Remark (GKR as a chain of sum-checks).** GKR is a sequence of sum-checks, each reducing a claim about layer i to a claim about layer $i + 1$. This is a special case of a more general pattern: sum-checks composing into directed graphs, where each sum-check is a node and evaluation claims are edges. GKR's graph is a path (linear chain from output to input). More complex protocols like Spartan (Chapter 19) have branching structure: one outer sum-check spawns multiple inner sum-checks. The graph perspective, where depth determines sequential stages and width enables batching, becomes central to prover efficiency in Chapters 19-20.

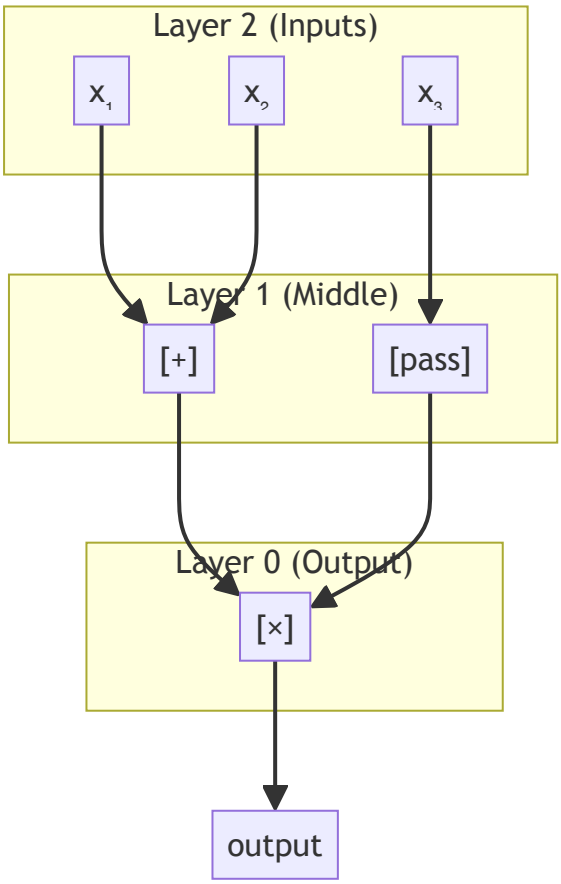
Layered Arithmetic Circuits

GKR operates on **layered arithmetic circuits**: directed acyclic graphs (graphs with edges that have direction, and no cycles; you can never follow edges back to where you started) where:

1. **Layers**: Gates are organized into layers $0, 1, \dots, d$
    - Layer d is the input layer
    - Layer 0 is the output layer
    - Wires only connect adjacent layers (from layer $i+1$ to layer i)
2. **Gate operations**: Each gate performs either addition or multiplication, with exactly two inputs
3. **Indexing**: Gates within each layer are numbered using binary strings
    - If layer i has S_i gates, we use $k_i = \lceil \log_2 S_i \rceil$ bits to index them
    - Gate j in layer i has label $j \in \{0,1\}^{k_i}$

Any circuit can be transformed into this layered form. If a wire spans multiple layers, we insert "pass-through" gates (identity gates that output their input unchanged).

Example Circuit: Let's trace through a simple circuit computing $(x_1 + x_2) \cdot x_3$.



Gate labeling:

- Layer 2 (inputs): $k_2 = 2$ bits needed for 3 gates
  - $x_1 \to (0, 0)$, $x_2 \to (0, 1)$, $x_3 \to (1, 0)$
- Layer 1: $k_1 = 1$ bit for 2 gates
  - Addition gate $\to (0)$, pass-through $\to (1)$
- Layer 0 (output): $k_0 = 1$ bit for 1 gate
  - Multiplication gate $\to (0)$

The Wiring Predicates

The circuit's structure is encoded by **wiring predicates**: functions that describe which gates connect to which.

For layer i, we define:

$$\text{add}_i(a,b,c) = \begin{cases} 1 & \text{if gate } a \text{ in layer } i \text{ is an addition gate with inputs } b,c \text{ from layer } i+1 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{mult}_i(a,b,c) = \begin{cases} 1 & \text{if gate } a \text{ in layer } i \text{ is a multiplication gate with inputs } b,c \text{ from layer } i+1 \\ 0 & \text{otherwise} \end{cases}$$

For our example circuit, look at layer 0. It contains a single multiplication gate, labeled (0). This gate multiplies the outputs of gate (0) (the addition gate computing x_1+x_2) and gate (1) (the pass-through carrying x_3) from layer 1. The wiring predicate encodes exactly this:

$$\text{mult}_0(a,b,c) = \begin{cases} 1 & \text{if } a=0, b=0, c=1 \\ 0 & \text{otherwise} \end{cases}$$

Reading this: "Gate $a=0$ in layer 0 is a multiplication gate whose left input comes from gate $b=0$ in layer 1, and whose right input comes from gate $c=1$ in layer 1." The predicate returns 1 only for this specific triple; all other combinations yield 0.

The layer has no addition gates, so add_0 is identically zero.

These predicates depend only on the *circuit structure*, not on the input values. The verifier, who knows the circuit, can compute these predicates efficiently.

Gate Values as Polynomials

For each layer i, define $W_i : \{0,1\}^{k_i} \to \mathbb{F}$ as the function mapping each gate label in layer i to its output value. There is exactly one such function per layer (not a family): W_0 captures all gate values in the output layer, W_1 captures all gate values in layer 1, and so on. The prover, having evaluated the circuit on specific inputs, knows all of $W_0, W_1, \ldots, W_d$.

We extend these to multilinear polynomials $\tilde{W}_i$ over $\mathbb{F}^{k_i}$. Similarly, we extend the wiring predicates to multilinear polynomials $\widetilde{\text{add}}_i$ and $\widetilde{\text{mult}}_i$.

For our example with inputs $x_1=2, x_2=3, x_3=4$:

Layer 2 values (inputs):

$$W_2(0,0)=2, \quad W_2(0,1)=3, \quad W_2(1,0)=4, \quad W_2(1,1)=0$$

(The fourth entry is padding: we have 3 inputs but need $2^2 = 4$ slots for 2-bit indexing. Unused slots are set to 0.)

The MLE is:

$$\tilde{W}_2(y_1, y_2) = 2(1-y_1)(1-y_2) + 3(1-y_1)y_2 + 4 \cdot y_1(1-y_2)$$

Layer 1 values:

$$W_1(0) = x_1 + x_2 = 5, \quad W_1(1) = x_3 = 4$$

The MLE is:

$$\tilde{W}_1(z) = 5(1-z) + 4z = 5 - z$$

Layer 0 values (output):

$$W_0(0) = (x_1 + x_2) \cdot x_3 = 20$$

The MLE is:

$$\tilde{W}_0(u) = 20(1-u)$$

The Layer Reduction Lemma

The heart of GKR is a beautiful algebraic identity that links adjacent layers:

GKR Lemma: For any point $z \in \mathbb{F}^{k_i}$:

$$\tilde{W}_i(z) = \sum_{b \in \{0,1\}^{k_{i+1}}} \sum_{c \in \{0,1\}^{k_{i+1}}} f_i(z, b, c)$$

Here $k_{i+1} = \lceil \log_2 S_{i+1} \rceil$ is the number of bits indexing gates in layer $i+1$, so the sum ranges over all $2^{k_{i+1}} \times 2^{k_{i+1}}$ possible pairs of gate indices from that layer. The polynomial f_i is defined as:

$$f_i(z, b, c) = \widetilde{\mathsf{add}}_i(z, b, c) \cdot (\tilde{W}_{i+1}(b) + \tilde{W}_{i+1}(c)) + \widetilde{\mathsf{mult}}_i(z, b, c) \cdot (\tilde{W}_{i+1}(b) \cdot \tilde{W}_{i+1}(c))$$

Why does this work? The sum ranges over all possible pairs of input gates (b, c). For most pairs, the wiring predicates are zero: gate z doesn't receive input from those gates. Only the *actual* input pair contributes, and for that pair:

- If z is an addition gate: $\widetilde{\mathsf{add}}_i = 1$, contributing $\tilde{W}_{i+1}(b) + \tilde{W}_{i+1}(c)$
- If z is a multiplication gate: $\widetilde{\mathsf{mult}}_i = 1$, contributing $\tilde{W}_{i+1}(b) \cdot \tilde{W}_{i+1}(c)$

The sum collapses to exactly what gate z should compute.

This identity expresses the output of layer i as a *sum*, which is exactly what sum-check can verify.

The Protocol

The GKR protocol reduces verification of the entire circuit to a single check on the input layer.

Initial Setup:

The verifier knows three things: (1) the circuit structure, meaning the wiring predicates add_i and mult_i for each layer; (2) the inputs to the circuit; and (3) the claimed output. She does *not* know the intermediate gate values. Those are computed by the prover and never directly revealed.

1. The prover evaluates the circuit and sends the claimed output W_0 to the verifier
2. The verifier picks a random point $r_0 \in \mathbb{F}^{k_0}$ and computes $V_0 = \tilde{W}_0(r_0)$
3. The goal: verify that V_0 is correct

Layer-by-Layer Reduction (for $i = 0, 1, \ldots, d-1$):

At the start of round i, the verifier holds a claim: "$\tilde{W}_i(r_i) = V_i$"

1. **Invoke sum-check**: Using the Layer Reduction Lemma, the verifier expresses V_i as a sum:

$$V_i = \sum_{b \in \{0,1\}^{k_{i+1}}} \sum_{c \in \{0,1\}^{k_{i+1}}} f_i(r_i, b, c)$$

   The prover and verifier run sum-check on this polynomial. The number of variables is $2k_{i+1}$.

2. **Sum-check conclusion**: Sum-check runs for $2k_{i+1}$ rounds. In each round, the verifier sends a random field element as a challenge. The first k_{i+1} challenges become $s_b \in \mathbb{F}^{k_{i+1}}$; the next k_{i+1} become $s_c \in \mathbb{F}^{k_{i+1}}$. At the end, the verifier must verify:

$$f_i(r_i, s_b, s_c) = \widetilde{\text{add}}_i(r_i, s_b, s_c) \cdot (\tilde{W}_{i+1}(s_b) + \tilde{W}_{i+1}(s_c)) + \widetilde{\text{mult}}_i(r_i, s_b, s_c) \cdot (\tilde{W}_{i+1}(s_b) \cdot \tilde{W}_{i+1}(s_c))$$

3. **The problem**: The verifier can compute the wiring predicates (she knows the circuit), but she doesn't know $\tilde{W}_{i+1}(s_b)$ and $\tilde{W}_{i+1}(s_c)$; those depend on intermediate gate values only the prover knows.

4. **Reduce two claims to one**: The prover sends the claimed values $\tilde{W}_{i+1}(s_b)$ and $\tilde{W}_{i+1}(s_c)$. But now the verifier has *two* claims to verify in the next round. To maintain efficiency, we reduce them to one:

   - The verifier picks a fresh random challenge $\alpha \in \mathbb{F}$
   - Define $r_{i+1} = s_b + \alpha(s_c - s_b)$ (a random point on the line $\ell(t) = s_b + t(s_c - s_b)$ through s_b and s_c)
   - The prover sends a univariate polynomial $q(t) = \tilde{W}_{i+1}(\ell(t))$ of degree k_{i+1}

- The verifier checks $q(0) = \tilde{W}_{i+1}(s_b)$ and $q(1) = \tilde{W}_{i+1}(s_c)$ against the prover's earlier claims
- Set $V_{i+1} = q(\alpha)$, which equals $\tilde{W}_{i+1}(r_{i+1})$

Restricting a multilinear polynomial to a line yields a low-degree univariate polynomial. The random α serves double duty: (1) it tests consistency; (2) it produces a fresh random point r_{i+1} that combines both claims into one for the next round.

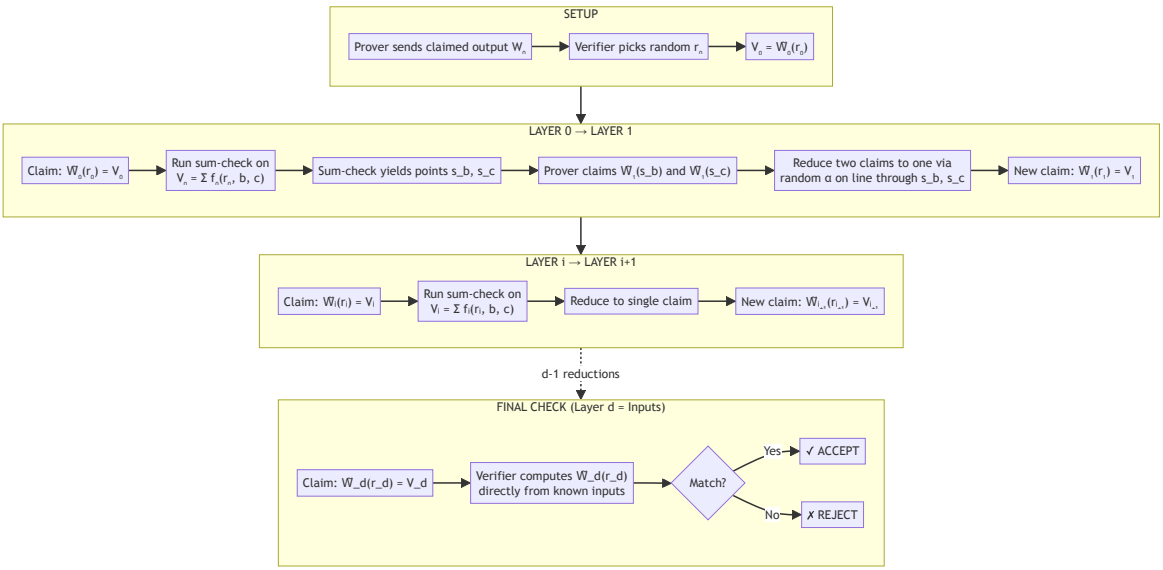
Why does this catch inconsistency? If the prover lied about either $\tilde{W}_{i+1}(s_b)$ or $\tilde{W}_{i+1}(s_c)$, they cannot produce a degree-k_{i+1} polynomial $q(t)$ that passes through both false values while also being the restriction of the true $\tilde{W}_{i+1}$ to the line ℓ. The degree bound is the handcuff: a low-degree polynomial through the wrong points must differ from the true polynomial almost everywhere. By Schwartz-Zippel, the probability that the random α lands on one of the at most k_{i+1} points where a false q happens to agree with the truth is at most $k_{i+1}/|\mathbb{F}|$, which is negligible

Alternative: random linear combination. Some implementations (Church-Forbes-Spooner 2017) instead use $V_{i+1} = \alpha_1 \cdot \tilde{W}_{i+1}(s_b) + \alpha_2 \cdot \tilde{W}_{i+1}(s_c)$ for fresh random α_1, α_2, verifying via a single combined claim. Both approaches achieve the same goal with similar security.

Final Check:

After d reductions, the verifier holds a claim: "$\tilde{W}_d(r_d) = V_d$"

But layer d is the input layer! The verifier knows the inputs. She computes $\tilde{W}_d(r_d)$ herself and checks if it equals V_d.



Worked Example: Verifying $(x_1 + x_2) \cdot x_3$

Let's trace through the protocol with $x_1 = 2, x_2 = 3, x_3 = 4$.

Honest computation:

- Layer 2: $W_2(0,0) = 2, W_2(0,1) = 3, W_2(1,0) = 4$
- Layer 1: $W_1(0) = 5, W_1(1) = 4$
- Layer 0: $W_0(0) = 20$

The prover claims the output is 20.

Round 0: Reducing Layer 0 to Layer 1

The verifier picks $r_0 = 7$ (say). Recall from earlier that $\tilde{W}_0(u) = 20(1-u)$ (the MLE of the single output value 20). She computes:

$$V_0 = \tilde{W}_0(7) = 20(1-7) = -120$$

The sum to verify (by the GKR Lemma):

$$-120 = \sum_{b,c \in \{0,1\}} \widetilde{\mathsf{mult}}_0(7, b, c) \cdot (\tilde{W}_1(b) \cdot \tilde{W}_1(c))$$

(The $\widetilde{\mathsf{add}}_0$ term vanishes since layer 0 has no addition gates.)

The wiring predicate's MLE: Since $\mathsf{mult}_0(0,0,1) = 1$ and is 0 elsewhere:

$$\widetilde{\mathsf{mult}}_0(u, v, w) = (1-u)(1-v)w$$

At $u = 7$:

$$\widetilde{\mathsf{mult}}_0(7, v, w) = (1-7)(1-v)w = -6(1-v)w$$

The sum becomes:

$$\sum_{b,c \in \{0,1\}} -6(1-b)c \cdot (\tilde{W}_1(b) \cdot \tilde{W}_1(c))$$

Sum-check on this polynomial proceeds for 2 rounds (one for b, one for c). The verifier sends random challenges after each round. Suppose these random challenges result in evaluation points $s_b = 3$ and $s_c = 5$; these are where the verifier needs to know $\tilde{W}_1$.

The prover claims:

$$\tilde{W}_1(s_b) = \tilde{W}_1(3) = 5 - 3 = 2, \quad \tilde{W}_1(s_c) = \tilde{W}_1(5) = 5 - 5 = 0$$

Now the verifier has two claims to verify. To reduce to one, she picks random $\alpha = 2$ and considers the line $\ell(t) = s_b + t(s_c - s_b) = 3 + 2t$ passing through s_b (at $t = 0$) and s_c (at $t = 1$). The prover sends the univariate polynomial $q(t) = \tilde{W}_1(\ell(t)) = 5 - (3 + 2t) = 2 - 2t$. The verifier checks:

- $q(0) = 2$ matches the claimed $\tilde{W}_1(s_b) = 2$ ✓
- $q(1) = 0$ matches the claimed $\tilde{W}_1(s_c) = 0$ ✓

The verifier computes $r_1 = \ell(\alpha) = 3 + 2(2) = 7$ and $V_1 = q(\alpha) = 2 - 2(2) = -2$. She now holds a new claim for the next round: $\tilde{W}_1(7) = -2$.

Round 1: Reducing Layer 1 to Layer 2

The verifier now holds the claim: $\tilde{W}_1(7) = -2$.

Using the GKR Lemma for layer 1:

$$\tilde{W}_1(7) = \sum_{b,c \in \{0,1\}^2} \left[\widetilde{\mathsf{add}}_1(7, b, c) \cdot (\tilde{W}_2(b) + \tilde{W}_2(c)) + \widetilde{\mathsf{mult}}_1(7, b, c) \cdot (\tilde{W}_2(b) \cdot \tilde{W}_2(c))\right]$$

Another sum-check reduces this to claims about $\tilde{W}_2$ at random points.

Final Check (Layer 2):

After the sum-check for layer 1, the verifier holds a claim about $\tilde{W}_2$ at some random point $r_2 = (r_{2,1}, r_{2,2}) \in \mathbb{F}^2$. This point emerged from the sum-check challenges and the two-to-one reduction, just as $r_1 = 7$ emerged in the previous round. She computes:

$$\tilde{W}_2(r_2) = 2(1 - r_{2,1})(1 - r_{2,2}) + 3(1 - r_{2,1})r_{2,2} + 4 \cdot r_{2,1}(1 - r_{2,2})$$

using the known inputs $x_1 = 2, x_2 = 3, x_3 = 4$. If this matches the prover's claim, she accepts.

Why GKR Works

Completeness: If the prover is honest, all polynomials they send in sum-check are correct, and all claimed evaluations are accurate. Every check passes.

Soundness: Suppose the prover claims a wrong output. Then $\tilde{W}_0$ is incorrect. By the Layer Reduction Lemma, either:

- The sum-check protocol catches a lie (soundness of sum-check), or
- The prover's claimed values for layer 1 are inconsistent

The lie propagates backward through the layers. By induction, if the original claim is false, either some sum-check fails, or the final claim about the input layer is false (which the verifier catches by direct computation).

The soundness error is bounded by:

$$\epsilon \leq \frac{d \cdot \deg(f)}{|\mathbb{F}|}$$

where d is the circuit depth and $\deg(f)$ is the degree of the sum-check polynomial.

Efficiency Analysis

Verifier's work:

- For each layer, participate in a sum-check with $O(\log S)$ rounds (where S is the layer size)
- Evaluate wiring predicates at random points (depends on circuit structure)
- Final check: compute $\tilde{W}_d(r_d)$ in time $O(n)$ where n is the number of inputs

Total: $O(d \log S + n)$ for a depth-d circuit with layers of size at most S.

For circuits with "regular" wiring (like FFT butterflies or matrix multiplication), evaluating wiring predicates takes $O(\log S)$ time. The verifier achieves **polylogarithmic verification** in the circuit size!

Why structure is the holy grail. If the circuit is random (spaghetti wiring), the verifier has to store the entire wiring diagram ($O(S)$ work), which defeats the purpose of succinctness. But if the circuit is *structured*, like a matrix multiplication where the same wiring pattern repeats thousands of times, the verifier doesn't need to read a massive list of wires. She can write a tiny loop that *generates* the wiring predicates on the fly. This data parallelism is what makes GKR efficient in practice. It is why modern provers like Lasso and Jolt are so fast: they treat computation not as a random circuit, but as a structured, repeating pattern.

Prover's work:

- Must compute the univariate polynomials for each sum-check round
- Requires summing over all gate values in each layer
- Total: $O(S \log S)$ where S is the total number of gates

The prover does work linear in the circuit size: roughly the cost of evaluation itself, with logarithmic overhead.

The Circuit Model: Power and Limitations

GKR works for any layered arithmetic circuit. This is remarkably general: any polynomial-time computation can be expressed as a polynomial-size arithmetic circuit.

Why addition and multiplication suffice: Over a finite field, these two operations generate all polynomial functions. And any Boolean function can be computed by polynomials: represent true as 1, false as 0, then AND becomes multiplication ($a \cdot b$), NOT becomes subtraction from 1 ($1 - a$), and OR follows from De Morgan ($1 - (1 - a)(1 - b)$). Since Boolean circuits are universal for computation (any Turing machine can be simulated), arithmetic circuits inherit this universality. The overhead is polynomial: a computation with T steps and S space becomes a circuit of size $O(T \cdot S)$.

What circuits capture well:

- Numerical computations (matrix operations, polynomial evaluation)
- Field arithmetic (cryptographic operations)
- Regular patterns (FFT, convolutions)

Challenges:

- Data-dependent control flow (if-then-else based on inputs) requires unrolling all branches
- Memory access patterns: Random access memory is expensive to arithmetize
- Bit operations: Non-arithmetic operations require special encoding

Chapter 8 will explore **arithmetization**, the art of expressing computations as circuits, in depth. We'll see how R1CS and QAP provide systematic ways to convert programs into the algebraic form that protocols like GKR can verify.

The Bigger Picture

GKR represents a conceptual leap in verifiable computation. Instead of designing a custom protocol for each problem:

1. Express the computation as a circuit (a general, mechanical process)
2. Apply GKR (a universal verification protocol)
3. Achieve efficient verification (polylogarithmic in circuit size for regular circuits)

This modularity is powerful. The "frontend" (how to express a computation as a circuit) separates from the "backend" (how to verify circuit evaluation). Improvements to either benefit all applications.

But GKR as originally described is an **interactive** protocol. The prover and verifier exchange messages over multiple rounds. For practical applications (blockchain verification, privacy-preserving credentials) we want **non-interactive** proofs that anyone can verify without interaction.

Chapter 11 will show how to compile interactive protocols like GKR into non-interactive SNARKs using polynomial commitment schemes and the Fiat-Shamir transformation. The journey from sum-check to practical zero-knowledge proofs passes through GKR.

Is GKR actually used? For years, GKR was primarily of theoretical interest; the prover overhead and circuit structure requirements made pairing-based SNARKs (Groth16, PLONK) more practical. But GKR is experiencing a resurgence. Modern systems like Lasso and Jolt use GKR-style sum-check reductions as their core verification mechanism, achieving state-of-the-art prover performance for certain computations.

GKR's prover is *native*, working directly with the computation's structure rather than reducing to generic polynomial arithmetic. To see why this matters, consider the alternative. In R1CS-based systems (Groth16, Spartan), every computation, no matter how structured, gets flattened into a uniform constraint system: thousands of equations of the form $a \cdot b = c$. A 256-bit multiplication, a hash function, a simple addition: all become rows in the same homogeneous matrix. The prover then does generic linear algebra over this matrix, blind to the original structure.

GKR is different. The prover traverses the actual circuit layer by layer, computing the sum-check polynomials from the wiring predicates and gate values directly. If your circuit has repeated structure, say 1000 copies of the same subcircuit, the prover can exploit that. If a layer is sparse (few gates), the work is proportionally smaller. The algorithm "sees" the computation's shape.

This becomes dramatic for certain operations. Lookup tables, for instance: proving "this value appears in that table" via R1CS requires encoding the entire table as constraints. GKR-based approaches (like Lasso) can instead prove lookups with work proportional to the number of lookups, not the table size. For memory operations, range checks, and other structured primitives, native provers can be orders of magnitude faster.

GKR is also transparent (no trusted setup) and plausibly post-quantum when instantiated with hash-based commitments. The protocol you've learned here isn't a historical curiosity; it's foundational to an active and growing family of proof systems.

Key takeaways

1. **Backward propagation**: GKR reduces output verification to input verification by propagating claims backward through layers. Each layer reduction is a sum-check.

2. **Wiring predicates as circuit DNA**: The functions add_i and mult_i encode the circuit's structure. The verifier can evaluate these efficiently because she knows the circuit.
3. **Two claims to one**: Without the line-restriction trick, claims would double each layer (exponential blowup). The random α on a line through two points compresses them into one.
4. **Structure is everything**: GKR verification is polylogarithmic only when wiring predicates have efficient descriptions. Random spaghetti circuits defeat the purpose.
5. **Native prover advantage**: Unlike R1CS systems that flatten all structure into uniform constraints, GKR's prover traverses the actual circuit. Repeated patterns, sparse layers, and regular wiring all translate to concrete speedups.
6. **Grounded in inputs**: The reduction chain terminates at the input layer, which the verifier knows. This is what makes the protocol sound: lies cannot hide when the final claim is directly checkable.

Chapter 8: From Circuits to Polynomials

In 1931, Kurt Gödel shattered the foundations of mathematics. He proved that any formal system powerful enough to express arithmetic is "haunted": it contains true statements that cannot be proven. More precisely: if a formal system F is *consistent* (it cannot prove both a statement and its negation) and capable of expressing basic arithmetic, then F is *incomplete* (there exists a statement G such that neither G nor $\neg G$ is provable in F). To establish this, Gödel had to solve a technical nightmare: how do you make math talk about itself?

His solution was Gödel numbering. He assigned a unique integer to every logical symbol ($+$, $=$, $\forall$), turning logical statements into integers and logical proofs into arithmetic relationships between those integers. He turned logic into arithmetic so that arithmetic could reason about logic.

What we do in zero-knowledge proofs is a direct descendant of Gödel's trick. We take the logic of a computer program (loops, conditionals, memory access) and encode it as polynomial equations. This translation is called **arithmetization**, and it's the subject of this chapter.

Arithmetic Circuits

An **arithmetic circuit** over a field $\mathbb{F}$ is a directed acyclic graph where each node is either an input, a constant, or a gate (addition or multiplication). Wires carry field elements from gate outputs to gate inputs. The circuit computes a function $f : \mathbb{F}^n \to \mathbb{F}^m$ by propagating values from inputs through gates to outputs.

Think of it as a recipe: inputs enter at the top, flow through a network of additions and multiplications, and produce outputs at the bottom. The recipe is fixed (the circuit structure), but you can run it on different ingredients (input values).

Why circuits? They're the universal language of computation. Any program, any algorithm, any function computable by a computer can be expressed as a (possibly enormous) arithmetic circuit. This universality is what makes circuit-based proof systems so powerful: prove you can verify circuits, and you can verify anything.

Two Problems, Two Paradigms

Before diving in, we must distinguish two fundamentally different problems:

Circuit Evaluation: Given a circuit C and input x, prove that $C(x) = y$.

The prover claims they computed the circuit correctly. The verifier could recompute it themselves, but the proof system makes verification faster. GKR handles this directly.

Circuit Satisfiability: Given a circuit C, public input x, and output y, prove there exists a secret witness w such that $C(x, w) = y$.

The prover claims they *know* a secret input that makes the circuit output the desired value. They reveal nothing about this secret. This is the paradigm behind most real-world ZK applications, and it's what enables privacy.

Note that GKR (Chapter 7) natively handles circuit *evaluation*, not satisfiability: it proves "$C(x) = y$" for public inputs, with no secrets involved. To handle satisfiability, where the prover has a private witness, you need additional machinery: polynomial commitments that hide the witness values, combined with sum-check to verify the computation. Systems like Jolt use GKR-style sum-check reductions but wrap them with commitment schemes that provide zero-knowledge. The distinction matters: "GKR-based" doesn't mean "evaluation only"; it means the verification logic uses sum-check over layered structure, while commitments handle privacy.

Example: Proving Knowledge of a Hash Preimage

Suppose $y = \text{SHA256}(w)$ for some secret w. The prover wants to demonstrate they know w without revealing it.

- The circuit C implements SHA256
- The public input is (essentially) empty
- The public output is y (the hash)
- The witness is w (the secret preimage)

The prover demonstrates: "I know a value w such that when I run SHA256 on it, I get exactly y." The verifier learns nothing about w except that it exists.

This *satisfiability* paradigm underlies almost all practical ZK applications: proving password knowledge, transaction validity, computation integrity, and more.

Understanding the Witness

The **witness** is central to zero-knowledge proofs. It's what separates a mere computation from a proof of knowledge.

What Exactly Is a Witness?

A witness is an input that, together with the public inputs, satisfies the circuit's constraints. In zero-knowledge proofs, the witness is kept private. In the equation $x^3 + x + 5 = 35$, the witness is $x = 3$. Anyone can verify that $3^3 + 3 + 5 = 35$, but the prover is demonstrating they *know* this solution.

More precisely, for a relation R, a witness w for statement x is a value such that $R(x, w) = 1$. The relation encodes the computational problem:

- **Hash preimage**: $R(y, w) = 1$ iff $\text{Hash}(w) = y$
- **Digital signature**: $R((m, \sigma, \text{pk}), \text{sk}) = 1$ iff $\text{Sign}(\text{sk}, m) = \sigma$
- **Sudoku solution**: $R(\text{puzzle}, \text{solution}) = 1$ iff the solution correctly fills the puzzle

The Sudoku Analogy. Think of a ZK proof as a solved Sudoku puzzle. The *circuit* is the rules of Sudoku: every row, column, and 3×3 square must contain the digits 1 through 9. The *public input* is the pre-filled numbers printed in the newspaper. The *witness* is the numbers you penciled in to solve it. Verifying the solution is easy: check the rows, columns, and squares (the constraints). You don't need to know the order in which the solver filled the numbers, nor the mental logic they used. You just check that the final grid (witness + public input) satisfies the rules.

The Execution Trace: Witness as Computation History

Modern arithmetization uses a clever insight: instead of building a circuit that *performs* the computation, we build a circuit that *verifies* a claimed execution trace.

What Is an Execution Trace?

An **execution trace** is a complete record of a computation's execution: every instruction, every intermediate value, every memory access. Think of it as a detailed log file that captures everything that

happened during the computation.

Checking that a trace is valid is much easier than producing the computation. Validity checking is *local*. To verify a trace, you only need to check that each step follows from the previous one according to the program's rules. The prover does the hard computational work; the circuit does the much easier work of checking consistency.

For simple computations (evaluating a polynomial, computing a hash), the trace is just the sequence of intermediate values at each gate. For more complex computations like CPU execution, the trace includes registers, program counters, and memory operations. The machinery for handling such traces (time consistency, memory consistency via permutation arguments) is developed in Chapter 21 in the context of efficient proving techniques. Here, we focus on the simpler case: a circuit where the witness captures all intermediate gate values.

R1CS: The Constraint Language

How do we express these checks algebraically? The classic approach is **Rank-1 Constraint System (R1CS)**.

An R1CS instance consists of:

- Three matrices A, B, C of dimension $m \times n$
- A witness vector Z of length n

The constraint is: for each row i,

$$(A_i \cdot Z) \times (B_i \cdot Z) = C_i \cdot Z$$

In words: (linear combination) × (linear combination) = (linear combination).

The matrices encode which wires participate in each constraint. Each row enforces one multiplication gate.

Why this particular form? The fundamental reason is that degree-2 polynomial constraints are the simplest non-trivial form that's still universal. Linear constraints (degree 1) can't express multiplication. Degree 2 is the minimal step up, and it turns out to be enough: any computation can be decomposed into steps involving at most one multiplication each. Historically, pairings reinforced this choice. A bilinear map can verify one multiplication "for free," so early SNARKs (Groth16, BCTV14) were designed around degree-2 constraints. But the format isn't pairing-specific: modern systems verify R1CS using FRI or IPA, no pairings required.

At first glance, "one multiplication per constraint" seems limiting. What if you need to compute $a \cdot b \cdot c$? That requires two multiplications, not one. What about x^4? That's three multiplications. How can a format that allows only one multiplication per constraint express arbitrary computations?

The answer: introduce intermediate variables. To compute $a \cdot b \cdot c$, define a helper variable $t = a \cdot b$, then write two constraints:

- Constraint 1: $a \times b = t$
- Constraint 2: $t \times c =$ result

Each constraint has exactly one multiplication. The witness vector grows to include t, but that's fine since the prover computed it anyway. This is the general pattern: any polynomial computation of degree d can be flattened into $O(d)$ R1CS constraints by naming intermediate products.

Addition, by contrast, is free. To constrain $a + b + c = d$, we write $(a + b + c) \times 1 = d$, which costs one constraint but involves no "real" multiplication. More generally, we can pack arbitrary additions into either side of a multiplication: $(a + b + c) \times (d + e) = f + g$ is still a single R1CS row. Why? Because $A \cdot Z$ computes a weighted sum of witness variables. Matrix-vector multiplication is just addition, so combining $a + b + c + ...$ into one linear combination costs nothing. We only "pay" when we multiply the result of $A \cdot Z$ by the result of $B \cdot Z$.

This decomposition is why R1CS can encode arbitrary arithmetic circuits. Every gate becomes one constraint. The "one multiplication" rule isn't a limitation; it's a *normal form* that any computation can be converted into.

Any arithmetic circuit with m multiplication gates and a addition gates can be expressed as an R1CS with exactly m constraints. The witness vector has length at most $m + a +$ inputs $+$ outputs. Addition gates require no constraints; they're absorbed into the linear combinations.

The Witness Vector in R1CS

The witness vector Z in R1CS has a specific structure. It concatenates three parts:

$$Z = \begin{pmatrix} 1 \\ \text{io} \\ W \end{pmatrix}$$

The constant 1: Always the first element. This allows encoding constants and pure additions. To constrain $x = 5$, write $x \times 1 = 5 \times 1$. For addition $a + b = c$, write $(a + b) \times 1 = c$.

The public inputs/outputs (io): Values the verifier knows. For a hash preimage proof, this is the hash value y. For a transaction validity proof, it might include the transaction amount and recipient.

The private witness (W): The secret values only the prover knows, plus all intermediate computation values.

For example, proving $x^3 + x + 5 = 35$ with secret $x = 3$:

	Index	Value	Description
Z_0	1		Constant
Z_1	35		Public output
Z_2	3		Private: x
Z_3	9		Private: x^2
Z_4	27		Private: x^3
Z_5	30		Private: $x^3 + x$
Z_6	35		Private: $x^3 + x + 5$

The witness includes every intermediate value, not only the input x. The constraint system checks that each step was performed correctly.

Basic Gates in R1CS

Multiplication ($a \cdot b = c$):

- Row i of A selects a from Z
- Row i of B selects b from Z
- Row i of C selects c from Z
- Constraint: $a \times b = c$

Addition ($a + b = c$):

- Set B to select the constant 1
- Row i of A selects both a and b (with coefficients 1, 1)
- Row i of C selects c
- Constraint: $(a + b) \times 1 = c$

Constant multiplication ($k \cdot a = c$):

- Row i of A selects a
- Row i of B selects constant k (or encode k in A)
- Row i of C selects c

Worked Example: $x^3 + x + 5 = 35$

Let's arithmetize a complete example. The prover claims to know x such that $x^3 + x + 5 = 35$. (The secret is $x = 3$.)

Step 1: Flatten to Basic Operations

Break the computation into primitive gates:

```
v1 = x * x        (compute x²)
v2 = v1 * x       (compute x³)
v3 = v2 + x       (compute x³ + x)
v4 = v3 + 5       (compute x³ + x + 5)
assert: v4 = 35   (check the result)
```

Step 2: Define the Witness Vector

The witness contains:

- The constant 1 (always included)
- The public output 35
- The secret input x
- All intermediate values

$$Z = (1, 35, x, v_1, v_2, v_3, v_4)$$

With $x = 3$:

$$Z = (1, 35, 3, 9, 27, 30, 35)$$

Step 3: Build the Constraint Matrices

Each gate becomes a row in the matrices:

Gate 1: $v_1 = x \cdot x$

- $A_1 = (0, 0, 1, 0, 0, 0, 0)$: selects x
- $B_1 = (0, 0, 1, 0, 0, 0, 0)$: selects x
- $C_1 = (0, 0, 0, 1, 0, 0, 0)$: selects v_1

Check: $(A_1 \cdot Z) \times (B_1 \cdot Z) = 3 \times 3 = 9 = C_1 \cdot Z$

Gate 2: $v_2 = v_1 \cdot x$

- $A_2 = (0, 0, 0, 1, 0, 0, 0)$: selects v_1
- $B_2 = (0, 0, 1, 0, 0, 0, 0)$: selects x
- $C_2 = (0, 0, 0, 0, 1, 0, 0)$: selects v_2

Check: $9 \times 3 = 27$

Gate 3: $v_3 = v_2 + x$

For addition, we use the trick: $(v_2 + x) \times 1 = v_3$

- $A_3 = (0, 0, 1, 0, 1, 0, 0)$: selects $v_2 + x$
- $B_3 = (1, 0, 0, 0, 0, 0, 0)$: selects constant 1
- $C_3 = (0, 0, 0, 0, 0, 1, 0)$: selects v_3

Check: $(27 + 3) \times 1 = 30$

Gate 4: $v_4 = v_3 + 5$

- $A_4 = (5, 0, 0, 0, 0, 1, 0)$: selects $5 \cdot 1 + v_3$
- $B_4 = (1, 0, 0, 0, 0, 0, 0)$: selects 1
- $C_4 = (0, 0, 0, 0, 0, 0, 1)$: selects v_4

Check: $(5 + 30) \times 1 = 35$

Gate 5: $v_4 = 35$ (the public output constraint)

- $A_5 = (0, 0, 0, 0, 0, 0, 1)$: selects v_4
- $B_5 = (1, 0, 0, 0, 0, 0, 0)$: selects 1
- $C_5 = (0, 35, 0, 0, 0, 0, 0)$: selects $35 \cdot 1$

Check: $35 \times 1 = 35$

All five constraints are satisfied. The R1CS captures the entire computation.

The complete matrices:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$C = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 35 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Each row corresponds to one constraint. The columns are indexed by $Z = (1, \text{out}, x, v_1, v_2, v_3, v_4)^T$. Notice the sparsity: most entries are zero. This is typical of R1CS matrices and is why efficient implementations use sparse representations.

Two Ways to Prove R1CS

Once we have R1CS constraints, how do we prove they're all satisfied? There are two major approaches.

Approach 1: QAP (Quadratic Arithmetic Program)

QAP was introduced by Gennaro, Gentry, Parno, and Rabin in the Pinocchio system (2013), one of the first practical SNARKs. Groth16 (2016) refined and optimized this approach, achieving the smallest proof size known for pairing-based systems. Today, QAP is primarily associated with Groth16. Modern systems have moved to other arithmetizations (PLONKish, AIR, sum-check), but QAP remains important for applications where proof size is paramount.

The key idea: instead of checking m separate constraints, check one polynomial divisibility.

For each column j of the R1CS matrices, define polynomials $A_j(X), B_j(X), C_j(X)$ that interpolate the column values at points $\{1, 2, \ldots, m\}$. (So $A_j(i)$ equals the entry in row i, column j of matrix A.)

Now let $\vec{Z} = (Z_0, Z_1, \ldots, Z_n)$ be the **witness vector**, the full assignment including the constant 1, public inputs, and private witness values. Define:

$$A(X) = \sum_j Z_j \cdot A_j(X), \quad B(X) = \sum_j Z_j \cdot B_j(X), \quad C(X) = \sum_j Z_j \cdot C_j(X)$$

Each Z_j is a scalar (from the witness), while $A_j(X)$ is a polynomial. The sum computes a linear combination, exactly mirroring how R1CS constraints are matrix-vector products.

The R1CS is satisfied iff $A(X) \cdot B(X) - C(X) = 0$ at all constraint points $\{1, 2, \ldots, m\}$.

By the Factor Theorem, this means the vanishing polynomial $Z_H(X) = (X-1)(X-2)\cdots(X-m)$ divides $A(X) \cdot B(X) - C(X)$.

The prover exhibits a quotient polynomial $H(X)$ such that:

$$A(X) \cdot B(X) - C(X) = H(X) \cdot Z_H(X)$$

We develop QAP fully in Chapter 12, where Groth16 uses it to achieve the smallest possible pairing-based proofs.

Approach 2: Sum-Check on Multilinear Extensions (Spartan)

Spartan was introduced by Setty in 2019, reviving ideas from the GKR protocol (2008) and sum-check literature. While Groth16 uses univariate polynomials and FFTs, Spartan showed that multilinear extensions and the sum-check protocol could handle R1CS directly: no Lagrange interpolation, no roots of unity, optimal prover time. This "sum-check renaissance" led to systems like Lasso, Jolt, and HyperNova.

R1CS constraint satisfaction can be expressed as a polynomial sum equaling zero:

$$\sum_{x \in \{0,1\}^k} \tilde{\mathsf{eq}}(x) \cdot \left[\tilde{A}(x) \cdot \tilde{B}(x) - \tilde{C}(x)\right] = 0$$

Here $\tilde{A}(x), \tilde{B}(x), \tilde{C}(x)$ are the MLEs of the matrix-vector products $A \cdot Z, B \cdot Z, C \cdot Z$ respectively, each viewed as a function from row index $x \in \{0,1\}^{\log m}$ to a field element.

This formulation matters for three reasons:

1. **Time-optimal proving**: The prover's work is $O(N)$ where N is the number of constraints, just reading the constraints, no FFTs.
2. **Sparsity-preserving**: Multilinear extensions preserve the structure of sparse matrices. In R1CS, most matrix entries are zero. The MLE directly reflects this sparsity.
3. **Natural fit with sum-check**: The sum-check protocol (Chapter 3) is designed exactly for this type of problem.

Comparing QAP and Spartan:

Property	QAP (Groth16)	Spartan
Polynomial type	Univariate, high-degree	Multilinear
Core technique	Divisibility by $Z_H(X)$	Sum-check
Prover time	$O(N \log N)$	$O(N)$
Setup	Circuit-specific trusted	Transparent

When to use each:

- **When proof size matters most**: Use QAP-based systems (Groth16, BCTV14, Pinocchio). On-chain verification on Ethereum costs gas proportional to proof size, making Groth16's ~200-byte proofs attractive despite the circuit-specific setup. Groth16 is the most optimized of this family and dominates in practice.

- **When prover time matters most**: Use Spartan or other sum-check systems. The $O(N)$ prover (vs $O(N \log N)$ for FFT-based systems) becomes significant at scale. Transparent setup avoids trust assumptions entirely. Natural fit for recursive composition and folding schemes (Nova, HyperNova). The tradeoff: larger proofs and more expensive verification.

PLONKish Arithmetization

R1CS isn't the only way to encode computations. **PLONKish** takes a fundamentally different approach, one that has become widely adopted in production ZK systems.

Historical context: PLONK (Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge) was introduced by Gabizon, Williamson, and Ciobotaru in 2019. It addressed Groth16's main limitation, circuit-specific trusted setup, by providing a **universal** setup: one ceremony works for any circuit up to a given size. PLONK spawned a family of "PLONKish" systems (Halo 2, Plonky2, HyperPlonk) that now power most production ZK applications.

The Universal Gate Equation

PLONK's core innovation is a single standardized gate equation:

$$Q_L \cdot a + Q_R \cdot b + Q_O \cdot c + Q_M \cdot a \cdot b + Q_C = 0$$

The Q values are **selectors**, public constants that "program" each gate.

Addition gate ($a + b = c$): Set $Q_L = 1, Q_R = 1, Q_O = -1$, rest zero.

Multiplication gate ($a \cdot b = c$): Set $Q_M = 1, Q_O = -1$, rest zero.

Public input ($a = k$): Set $Q_L = 1, Q_C = -k$, rest zero.

The same equation handles all gate types!

Copy Constraints: The Permutation Argument

PLONK's gate equation only relates wires *within* a single gate. It doesn't enforce that the output of gate 1 feeds into the input of gate 5.

This is where the **permutation argument** enters. Number all wire positions in the circuit as $1, 2, 3, \ldots, n$. Some positions must hold equal values (because one gate's output connects to another's input). We encode these equalities as a permutation σ: positions that must be equal form cycles under σ. The constraint "all wiring is respected" becomes:

$$f(i) = f(\sigma(i)) \quad \text{for all } i$$

where $f(i)$ is the value at position i. PLONK proves this via a grand product check. With random challenges β, γ:

$$\prod_i \frac{f(i) + \beta \cdot i + \gamma}{f(i) + \beta \cdot \sigma(i) + \gamma} = 1$$

The intuition: each fraction pairs a value with its position. If copy constraints hold, the numerators and denominators rearrange to cancel. If any constraint fails, the random β, γ ensure the product differs from 1 with overwhelming probability. We develop the full permutation argument in Chapter 13.

When to Use PLONKish

PLONKish shines when you need **flexibility without sacrificing succinctness**:

- **Universal setup** (vs Groth16's circuit-specific): One ceremony covers all circuits up to a size bound
- **Custom gates**: Optimize specific operations (hash functions, range checks, elliptic curve arithmetic)

The tradeoff versus Groth16 (which uses R1CS + QAP): slightly larger proofs (~2-3x), but no circuit-specific ceremony.

Note: Sum-check systems like Spartan go further with fully transparent setup (no ceremony at all), but with larger proofs.

AIR: Algebraic Intermediate Representation

A third constraint format takes yet another path, designed specifically for computations with **repetitive structure**: state machines, virtual machines, and iterative algorithms.

An AIR consists of:

- **Execution trace**: A table where each row represents a "state" and columns hold state variables
- **Transition constraints**: Polynomials that relate row i to row $i + 1$ (the local rules)
- **Boundary constraints**: Conditions on specific rows (initial state, final state)

The insight: many computations are naturally described as "apply the same transition rule repeatedly." A CPU executes instructions in a loop. A hash function applies the same round function many times. AIR captures this by encoding the transition rule once and proving it holds for all consecutive row pairs.

Example: A simple counter that increments by 1:

- Transition constraint: $s_{i+1} - s_i - 1 = 0$
- Boundary constraint: $s_0 = 0$ (start at zero)

This single transition constraint, applied to n rows, proves correct execution of n steps.

The algebraic formulation uses a clever trick. Interpolate each trace column as a polynomial $P(X)$ over a domain $H = \{1, \omega, \omega^2, \ldots, \omega^{T-1}\}$ where ω is a T-th root of unity. Now $P(\omega^i)$ gives the value at step i, and $P(\omega \cdot \omega^i) = P(\omega^{i+1})$ gives the value at step $i + 1$. So the "next step" value is $P(\omega X)$.

The transition constraint $s_{i+1} - s_i - 1 = 0$ becomes the polynomial identity:

$$P(\omega X) - P(X) - 1 = 0 \quad \text{for all } X \in H' = \{1, \omega, \ldots, \omega^{T-2}\}$$

If this holds, the constraint polynomial $C(X) = P(\omega X) - P(X) - 1$ vanishes on H', so the quotient $Q(X) = C(X)/Z_{H'}(X)$ is a polynomial (not a rational function with poles). The prover commits to $Q(X)$ and proves it's low-degree via FRI. Boundary constraints work similarly: $P(1) = 0$ becomes $(P(X) - 0)/(X - 1)$ being a polynomial.

AIR is the native format for **STARKs**, which we develop fully in **Chapter 15**. The combination of AIR's repetitive structure with FRI's hash-based commitments yields transparent, plausibly post-quantum proofs.

Comparing the Three Formats

Property	R1CS	PLONKish	AIR
Structure	Sparse matrices	Gates + selectors	Execution trace + transitions
Gate flexibility	One mult/constraint	Custom gates	Transition polynomials

Property	R1CS	PLONKish	AIR
Best for	Simple circuits	Complex, irregular ops	Repetitive state machines
Used by	Groth16, Spartan	PLONK, Halo 2	STARKs, Cairo

In practice:

- **R1CS + Groth16**: When proof size dominates (on-chain verification)
- **PLONKish**: When you need flexibility and universal setup
- **AIR + STARKs**: When transparency and post-quantum security matter

CCS: Unifying the Constraint Formats

We now have three constraint formats (R1CS, PLONKish, AIR) each with distinct strengths. But this proliferation creates fragmentation: tools, optimizations, and folding schemes must be reimplemented for each format.

Why do we need yet another format? The answer is folding (Chapter 23). Newer protocols like Nova and HyperNova work by "folding" two proof instances into one. R1CS folds easily, but PLONKish constraints do not. **Customizable Constraint Systems (CCS)** was invented to give us both: the expressiveness of PLONK's custom gates with the foldability of R1CS's matrix structure. CCS provides a unifying abstraction that captures all three formats without overhead.

The CCS Framework

A CCS instance consists of:

- **Matrices** $M_1, \ldots, M_t$: sparse matrices over $\mathbb{F}$, encoding constraint structure
- **Constraint specifications**: which matrices combine in each constraint, with what operation

Any constraint system can be expressed as:

$$\sum_i c_i \cdot \bigcirc_{j \in S_i} (M_j \cdot z) = 0$$

where:

- z is the witness vector (including public inputs and the constant 1)
- S_i specifies which matrices participate in term i
- $\bigcirc$ is the Hadamard (element-wise) product: $(a_1, a_2, a_3) \circ (b_1, b_2, b_3) = (a_1 b_1, a_2 b_2, a_3 b_3)$
- c_i are scalar coefficients

The notation $\bigcirc_{j \in S_i}$ means: for each matrix index j in the set S_i, compute the vector $M_j \cdot z$, then Hadamard-multiply all those vectors together. If $S_i = \{1, 2\}$, you get $(M_1 \cdot z) \circ (M_2 \cdot z)$. If $S_i = \{3\}$ (a single matrix), you just get $M_3 \cdot z$ with no Hadamard.

Each term i in the sum takes a subset of matrices $\{M_j : j \in S_i\}$, multiplies each by the witness vector z, Hadamard-multiplies the results together, and scales by c_i. The constraint is satisfied when all terms sum to zero.

Every constraint format we've seen boils down to two operations: (1) selecting and summing witness values (matrix-vector products), and (2) multiplying those sums together (Hadamard products). CCS makes these two operations explicit and composable:

- **Linear constraints**: A single matrix-vector product, no Hadamard
- **Quadratic constraints**: Hadamard of two matrix-vector products
- **Higher-degree constraints**: Hadamard of more products
- **Mixed constraints**: Different terms can have different degrees

Recovering Standard Formats

R1CS as CCS:

- Three matrices: $M_1 = A$, $M_2 = B$, $M_3 = C$
- Two terms: $S_1 = \{1, 2\}$ (Hadamard of A and B), $S_2 = \{3\}$ (just C)
- Coefficients: $c_1 = 1, c_2 = -1$

The CCS formula becomes:

$$1 \cdot ((M_1 \cdot z) \circ (M_2 \cdot z)) + (-1) \cdot (M_3 \cdot z) = 0$$

which is exactly $(A \cdot z) \circ (B \cdot z) - C \cdot z = 0$, the R1CS equation.

PLONKish as CCS:

The PLONK gate equation $Q_L \cdot a + Q_R \cdot b + Q_O \cdot c + Q_M \cdot a \cdot b + Q_C = 0$ becomes:

- Matrices: M_a (selects wire a), M_b (selects wire b), M_c (selects wire c), M_{Q_L} (selector), M_{Q_R}, M_{Q_O}, M_{Q_M}, M_{Q_C}
- Terms map to the gate equation:

- $Q_L \cdot a$: Hadamard of selector and wire → $S_1 = \{Q_L, a\}$
- $Q_M \cdot a \cdot b$: Hadamard of three matrices → $S_2 = \{Q_M, a, b\}$
- …and so on for each term

The CCS formula becomes:

$$1 \cdot (M_{Q_L} \cdot z) \circ (M_a \cdot z) + 1 \cdot (M_{Q_R} \cdot z) \circ (M_b \cdot z) + ... = 0$$

Each term in PLONK's gate equation maps to one term in the CCS sum.

AIR as CCS:

Recall from the AIR section that the transition constraint $s_{i+1} - s_i - 1 = 0$ becomes the polynomial identity $P(\omega X) - P(X) - 1 = 0$. CCS captures this same structure with matrices instead of the ωX shift.

A transition constraint like $s_{i+1} = 2 \cdot s_i + 1$ becomes:

- Matrices: M_{curr} (extracts current-row values), M_{next} (extracts next-row values), M_{const} (constant column)
- The constraint $s' - 2s - 1 = 0$ becomes:

$$1 \cdot (M_{\text{next}} \cdot z) + (-2) \cdot (M_{\text{curr}} \cdot z) + (-1) \cdot (M_{\text{const}} \cdot z) = 0$$

The matrix M_{next} plays the role of the ωX shift: it extracts "next step" values from the witness vector, just as $P(\omega X)$ evaluates the polynomial at the next domain point.

Here all terms have $|S_i| = 1$ (no Hadamard products), so the constraint is purely linear in state variables. Quadratic AIR constraints (like $s' = s^2$) would use Hadamard: $(M_{\text{next}} \cdot z) - (M_{\text{curr}} \cdot z) \circ (M_{\text{curr}} \cdot z) = 0$.

Why CCS Matters

CCS enables unified tooling: compilers, analyzers, and optimizers can target CCS once. The specific frontend (Circom, Cairo, Noir) produces CCS; the backend (Spartan, Nova, HyperNova) consumes it. HyperNova folds CCS instances directly, so any constraint format expressible as CCS inherits folding for free. Matrix sparsity, constraint reordering, and parallel proving apply uniformly regardless of the original constraint format. Theoretical results about CCS apply to all formats it subsumes.

CCS in Practice

Modern systems increasingly use CCS as their internal representation:

- **HyperNova**: Folds CCS directly, achieving the benefits of PLONK's flexibility with Nova's efficiency
- **Sonobe**: A folding framework that targets CCS
- **Research prototypes**: Use CCS for cleaner proofs of concept

The constraint format ecosystem is consolidating. R1CS, PLONKish, and AIR remain useful surface-level abstractions, but CCS provides the common substrate beneath.

Handling Non-Arithmetic Operations

Real programs use operations that aren't native to field arithmetic: comparisons, bitwise operations, conditionals, hash functions. These require careful encoding, and this is where constraint counts explode.

Bit Decomposition: The Fundamental Technique

The standard technique: represent an integer a as bits $(a_0, a_1, \ldots, a_{W-1})$.

Enforce "bitness": Each a_i must satisfy $a_i \cdot (a_i - 1) = 0$. This polynomial is zero iff $a_i \in \{0, 1\}$.

Why? If $a_i = 0$: $0 \cdot (0-1) = 0$ (satisfied). If $a_i = 1$: $1 \cdot (1-1) = 0$ (satisfied). If $a_i = 2$: $2 \cdot (2-1) = 2 \neq 0$ (fails).

Reconstruct the value: Verify $a = \sum_{i=0}^{W-1} a_i \cdot 2^i$.

Constraint Costs: A Reality Check

Here's where things get expensive. Let's count constraints for common operations:

Operation	Constraints	Notes
Field addition	0	Free! Just combine wires
Field multiplication	1	Native R1CS operation
64-bit decomposition	64	One per bit (bitness check)
64-bit reconstruction	1	Sum with powers of 2

Operation	Constraints	Notes
64-bit AND	~130	Decompose both, multiply bits, reconstruct
64-bit XOR	~130	Decompose both, compute XOR per bit
64-bit comparison	~200	Decompose, subtract, check sign bit
64-bit range proof	~65	Decompose + bitness checks
SHA256 hash	~20,000	Many bitwise operations
Poseidon hash	~250	Field-native design

Bitwise operations are roughly 100x more expensive than field operations. This is why:

- ZK-friendly hash functions (Poseidon, Rescue) exist: they avoid bit operations
- zkVMs are expensive because they must handle arbitrary CPU instructions
- Custom circuits beat general-purpose approaches for specific computations

Simulating Logic Gates

With bits exposed, we can simulate Boolean logic:

AND ($c = a \wedge b$): For each bit position i:

$$c_i = a_i \cdot b_i$$

Cost: 1 multiplication constraint per bit

OR ($c = a \vee b$): For each bit position i:

$$c_i = a_i + b_i - a_i \cdot b_i$$

Cost: 1 multiplication constraint per bit

XOR ($c = a \oplus b$): For each bit position i:

$$c_i = a_i + b_i - 2 \cdot a_i \cdot b_i$$

Cost: 1 multiplication constraint per bit

NOT ($c = \neg a$): For each bit position i:

$$c_i = 1 - a_i$$

Cost: 0 (just linear combination)

Range Proofs: Proving $a < 2^k$

To prove a value is within a range $[0, 2^k)$:

1. Decompose into k bits
2. Check each bit satisfies $b_i(b_i - 1) = 0$
3. Verify reconstruction: $a = \sum_{i=0}^{k-1} b_i \cdot 2^i$

Cost: k bitness constraints + 1 reconstruction constraint

Comparison: Proving $a < b$

To prove $a < b$ for values in range $[0, 2^k)$:

Approach 1: Subtraction with underflow

1. Compute $d = b - a + 2^k$ (shifted to avoid underflow)
2. Decompose d into $k + 1$ bits
3. Check the most significant bit equals 1 (meaning $b - a \geq 0$, so $b > a$)

Cost: ~$k + 1$ constraints for bit decomposition + bitness checks

Approach 2: Lexicographic comparison

1. Decompose both a and b into bits
2. Starting from the MSB, find the first position where they differ
3. At that position, check $a_i = 0$ and $b_i = 1$

Cost: More complex, often not better for general comparisons

The pattern is clear: anything involving bits is expensive. For years, circuit designers accepted this cost as unavoidable, until lookup arguments changed everything.

Lookup Arguments: Breaking the Bit Decomposition Wall

The constraint costs above create a fundamental problem. A silicon CPU executes a XOR b in one cycle. In R1CS, that same XOR costs ~25 constraints: decompose both operands into bits, check bitness, compute per-bit XOR, reconstruct. For a 64-bit instruction set, every operation requires hundreds of constraints. Building a zkVM this way is like simulating a Ferrari using wooden gears.

Lookup arguments solve this by replacing *computation* with *table membership*. Instead of proving *how* you computed a result, prove *that* the result appears in a precomputed table.

To prove an 8-bit XOR:

- **Bit decomposition**: 16 bitness checks + 8 XOR computations + reconstruction ≈ 25 constraints
- **Lookup**: Precompute all $256 \times 256 = 65,536$ valid XOR triples $(a, b, a \oplus b)$. Prove (a, b, c) is in the table ≈ 3 constraints

The savings compound. A 64-bit XOR via bit decomposition costs ~130 constraints. Via lookups on 8-bit chunks: 8 lookups × 3 constraints = 24 constraints.

This changes what's feasible:

Operation	Without Lookups	With Lookups
16-bit range check	17 constraints	~3 constraints
8-bit XOR	~25 constraints	~3 constraints
64-bit XOR	~130 constraints	~24 constraints
SHA-256 (via chunks)	~20,000 constraints	~2,000 constraints

The "how" of lookup arguments (Plookup's grand products, LogUp's logarithmic derivatives, Lasso's decomposition for huge tables) is developed in **Chapter 14**. What matters for arithmetization is architectural: non-field-native operations that would otherwise dominate constraint counts can be handled via table membership at roughly constant cost per lookup.

This is why modern zkVMs are practical. Cairo, RISC-Zero, SP1, and Jolt prove instruction execution not by encoding CPU semantics in constraints, but by verifying that each instruction's behavior matches a precomputed table. The paradigm shifted from encoding *logic* to referencing *data*.

The Frontend/Backend Split

This chapter describes **frontends**, compilers that transform high-level programs into arithmetic form. The **backend** is the proof system (GKR, Groth16, PLONK, STARKs) that proves the resulting constraints.

CPU-style frontends (Cairo, RISC-Zero, SP1, Jolt):

- Define a virtual machine with a fixed instruction set
- Any program compiles to that instruction set
- The arithmetization verifies instruction execution
- General-purpose but with overhead

ASIC-style frontends (Circom, custom circuits):

- Create a specialized circuit for each specific program
- Maximum efficiency for fixed computations
- Poor for general-purpose or data-dependent control flow

Hybrid approaches:

- Use custom circuits for the common case
- Fall back to general VM for edge cases
- Example: Specialized hash circuit + general VM for the rest

The choice depends on your use case. Verifying a hash? A custom circuit is fastest. Running arbitrary computation? You need a zkVM. Running the same computation millions of times? The circuit development cost is amortized.

Key takeaways

1. **The pipeline**: Program → execution trace (witness) → constraint system → polynomial identity → proof. Arithmetization is the bridge between computation and algebra.
2. **Circuit satisfiability vs. evaluation**: Most applications prove knowledge of a secret witness, not just correct evaluation.
3. **The witness is everything**: It's the complete set of values (public, private, and intermediate) that satisfies the constraints.
4. **Three constraint formats**: R1CS (sparse matrices, $(A \cdot Z) \times (B \cdot Z) = C \cdot Z$), PLONKish (universal gate + permutation), AIR (transition polynomials). CCS unifies them all.
5. **Bit decomposition is expensive**: A 64-bit operation costs ~65-200 constraints via traditional encoding. Lookup arguments (Chapter 14) reduce this to ~3 constraints per table lookup.
6. **Frontend/backend split**: Frontends handle arithmetization; backends handle proving. They can be mixed and matched.
7. **Constraint cost guides design**: Choose field-friendly operations (hashes, curves) over bit-heavy operations.

Chapter 9: Polynomial Commitment Schemes: The Cryptographic Engine

In 2016, six people met in a hotel room to birth the Zcash privacy protocol. Their task: generate a cryptographic secret so dangerous that if even one of them kept a copy, it could forge unlimited fake

coins, undetectable forever. They called it "toxic waste."

The ceremony was a paranoid ballet. Participants were scattered across the globe, connected by encrypted channels. One flew to an undisclosed location, computed on an air-gapped laptop, then incinerated the machine and its hard drive. Another broadcast their participation live so viewers could verify no one was coercing them. The randomness they generated was combined through multi-party computation, ensuring that if *any single participant* destroyed their secret, the final parameters would be safe.

Why such extreme measures? Because polynomial commitment schemes, the cryptographic engine that makes SNARKs work, sometimes require a *structured reference string*: public parameters computed from a secret that must then cease to exist. The Zcash ceremony became legendary in cryptography circles, part security protocol, part performance art. It demonstrated both the power and the peril of pairing-based commitments.

This chapter explores that peril and its alternatives. We'll see two fundamental approaches to polynomial commitments: **KZG**, which achieves constant-size proofs at the cost of trusted setup, and **IPA/Bulletproofs**, which eliminates the toxic waste but pays with linear verification. Each represents a different answer to the same question: how do you prove facts about a polynomial without revealing it? A third major scheme, **FRI**, takes a fundamentally different approach based on hashing rather than algebraic assumptions; we cover it in Chapter 10. (For advanced schemes like **Dory** that achieve logarithmic verification without trusted setup, see Appendix D.)

Everything we've built (sum-check, GKR, arithmetization) reduces complex claims to polynomial identities. A prover claims that polynomial $p(X)$ has certain properties: it equals another polynomial, it vanishes on a domain, it evaluates to a specific value at a point.

But there is a catch. Verifying these claims directly would require the verifier to see the entire polynomial. For a polynomial of degree n, that's $n + 1$ coefficients, exactly as much data as the original computation. We've achieved nothing.

Polynomial Commitment Schemes (PCS) solve this problem. A PCS allows a prover to commit to a polynomial with a short commitment, then later prove claims about the polynomial (its evaluations at specific points) without revealing the polynomial itself. The commitment is binding (the prover can't change the polynomial), and the proofs are succinct (much smaller than the polynomial).

This is where abstract algebra meets cryptography.

The PCS Abstraction

A polynomial commitment scheme consists of three algorithms:

Commit $(f) \to C$: Given a polynomial $f(X)$, produce a short commitment C.

Open $(f, z) \to (v, \pi)$: Given the polynomial f, a point z, compute the evaluation $v = f(z)$ and a proof π that this evaluation is correct.

Verify $(C, z, v, \pi) \to \{\textbf{accept}, \textbf{reject}\}$: Given the commitment, point, claimed value, and proof, check correctness.

Properties:

- **Binding**: A commitment C can only be opened to evaluations consistent with *one* polynomial (computationally)
- **Hiding** (optional): The commitment reveals nothing about the polynomial
- **Succinctness**: Commitments and proofs are much smaller than the polynomial

If the prover is bound to a specific polynomial before seeing the verifier's challenge point, and the commitment is much smaller than the polynomial, then we can verify polynomial identities by checking at random points.

KZG: Constant-Size Proofs from Pairings

The Kate-Zaverucha-Goldberg (KZG) scheme achieves the holy grail: constant-size commitments *and* constant-size evaluation proofs. No matter how large the polynomial, the proof is just one group element.

Pairings

A **bilinear pairing** is a map $e : G_1 \times G_2 \to G_T$ between three groups with the property:

$$e(aP, bQ) = e(P, Q)^{ab}$$

for all scalars a, b and group elements $P \in G_1, Q \in G_2$.

This one equation lets us check multiplicative relationships *in the exponent*. Given commitments g^a and g^b, we cannot compute g^{ab} (that would break CDH). But if someone claims to know $c = ab$, we can *verify* their claim by checking:

$$e(g^a, g^b) = e(g^c, g)$$

One multiplication check "for free" in the hidden exponent world. This is exactly what polynomial evaluation needs.

The Trusted Setup

KZG requires a **structured reference string (SRS)**: a set of public parameters computed from a secret:

1. Choose a random secret $\tau \in \mathbb{F}_p$ (the "toxic waste")
2. Compute the SRS: $(g, g^\tau, g^{\tau^2}, \ldots, g^{\tau^D})$
3. **Destroy** τ

The SRS encodes powers of the secret τ "in the exponent." Anyone can use these elements without knowing τ itself. But if anyone learns τ, they can forge proofs for false statements, so the setup must ensure τ is never known to any party. In practice, this is done via multi-party computation ceremonies where many participants contribute randomness, and security holds as long as *any one* participant is honest.

Commitment

To commit to a polynomial $f(X) = \sum_{i=0}^{d} c_i X^i$:

$$C = g^{f(\tau)} = g^{\sum c_i \tau^i} = \prod_{i=0}^{d} (g^{\tau^i})^{c_i}$$

The prover computes this using the SRS elements, never learning τ. The result is a single group element: the polynomial "evaluated at the secret point τ, hidden in the exponent."

Evaluation Proof

To prove $f(z) = v$ for a public point z:

1. **The polynomial identity**: If $f(z) = v$, then $(X - z)$ divides $f(X) - v$. Define:

$$w(X) = \frac{f(X) - v}{X - z}$$

   This quotient $w(X)$ is a valid polynomial of degree $d - 1$.

2. **The proof**: Commit to the quotient:
$$\pi = g^{w(\tau)}$$

3. **Verification**: The verifier checks:
$$e(\pi, g^{\tau} \cdot g^{-z}) = e(C \cdot g^{-v}, g)$$

Why Verification Works

The verification equation $e(\pi, g^{\tau} \cdot g^{-z}) = e(C \cdot g^{-v}, g)$ is equivalent to the polynomial identity $w(\tau)(\tau - z) = f(\tau) - v$. To see this, substitute the definitions:

- $\pi = g^{w(\tau)}$
- $g^{\tau} \cdot g^{-z} = g^{\tau - z}$
- $C \cdot g^{-v} = g^{f(\tau)} \cdot g^{-v} = g^{f(\tau) - v}$

This gives:

$$e(g^{w(\tau)}, g^{\tau - z}) = e(g^{f(\tau) - v}, g)$$

By bilinearity:
$$e(g, g)^{w(\tau)(\tau - z)} = e(g, g)^{f(\tau) - v}$$

This holds iff $w(\tau)(\tau - z) = f(\tau) - v$, which is exactly the polynomial identity $f(X) - v = w(X)(X - z)$ evaluated at τ.

Why this implies soundness: Suppose the prover lies; they claim $f(z) = v$ when actually $f(z) \neq v$. Then $f(X) - v$ is *not* divisible by $(X - z)$, so no polynomial $w(X)$ satisfies the identity $f(X) - v = w(X)(X - z)$. Without such a $w(X)$, the prover must instead find some $w'(X)$ where the identity fails as polynomials but happens to hold at τ:

$$w'(\tau)(\tau - z) = f(\tau) - v$$

But the prover doesn't know τ; it's hidden in the SRS. From their perspective, τ is a random field element. Two distinct degree-d polynomials agree on at most d points (Schwartz-Zippel), so the probability that a "wrong" w' accidentally satisfies the check at the unknown τ is at most $d/|\mathbb{F}|$ (negligible for large fields).

Formal soundness statement: Let $f(X)$ be the committed polynomial of degree at most d. For any

adversary $\mathcal{A}$ that outputs (z, v, π) with $f(z) \neq v$:

$$\Pr[\text{Verify}(C, z, v, \pi) = \text{accept}] \leq \frac{d}{|\mathbb{F}|}$$

where the probability is over the random choice of τ in the trusted setup. Under the q-Strong Diffie-Hellman assumption (that computing $g^{1/(\tau+a)}$ from the SRS is hard), this bound holds even for adversaries who choose f adaptively.

Worked Example: KZG in Action

Let's trace through a complete example.

Setup: Maximum degree $D = 2$, secret $\tau = 5$.

- SRS: (g, g^5, g^{25})

Commit to $f(X) = X^2 + 2X + 3$:

$$C = g^{f(5)} = g^{25+10+3} = g^{38}$$

Prove $f(1) = 6$:

- Check: $f(1) = 1 + 2 + 3 = 6$
- Quotient: $w(X) = \frac{f(X)-6}{X-1} = \frac{X^2+2X-3}{X-1}$

  Factor: $X^2 + 2X - 3 = (X + 3)(X - 1)$

  So $w(X) = X + 3$
- Proof: $\pi = g^{w(5)} = g^{5+3} = g^8$

Verify:

- LHS: $e(\pi, g^\tau \cdot g^{-z}) = e(g^8, g^5 \cdot g^{-1}) = e(g^8, g^4) = e(g, g)^{32}$
- RHS: $e(C \cdot g^{-v}, g) = e(g^{38} \cdot g^{-6}, g) = e(g^{32}, g) = e(g, g)^{32}$

Both sides equal. The verification passes.

Batch Opening

KZG has a remarkable property: proving evaluations at multiple points is barely more expensive than proving one.

To prove $f(z_1) = v_1, \ldots, f(z_k) = v_k$:

1. Define the vanishing polynomial $Z(X) = \prod_i (X - z_i)$
2. Compute the interpolating polynomial $R(X)$ such that $R(z_i) = v_i$
3. The quotient $w(X) = \frac{f(X) - R(X)}{Z(X)}$ exists iff all evaluations are correct
4. The proof is just $g^{w(\tau)}$ (still one group element!)

This is why KZG scales so well in practice. A SNARK verifier might need to check dozens of polynomial evaluations; with batch opening, these collapse into a single pairing check.

KZG: Properties and Trade-offs

Advantages:

- **Constant commitment size**: One group element, regardless of polynomial degree
- **Constant proof size**: One group element per evaluation
- **Constant verification time**: A few pairings and exponentiations
- **Batch opening**: Multiple evaluations verified with a single proof

Disadvantages:

- **Trusted setup**: The "toxic waste" must be destroyed. If compromised, soundness breaks.
- **Not post-quantum**: Pairing-based cryptography falls to quantum computers
- **Degree-bounded**: The SRS size caps the maximum polynomial degree

Managing Toxic Waste: Powers of Tau Ceremonies

The trusted setup creates a serious practical problem: someone must generate τ, compute the powers, and then *verifiably destroy* τ. How do you convince the world that the toxic waste is truly gone?

The solution is **multi-party computation (MPC) ceremonies**. Instead of trusting a single party, we chain together contributions from many independent participants:

1. **Participant 1** picks secret τ_1, computes $[1]_1, [\tau_1]_1, [\tau_1^2]_1, \ldots$ and destroys τ_1
2. **Participant 2** picks secret τ_2, raises each element to τ_2, getting $[1]_1, [\tau_1\tau_2]_1, [(\tau_1\tau_2)^2]_1, \ldots$ and destroys τ_2
3. Continue for hundreds or thousands of participants…

The final structured reference string encodes powers of $\tau = \tau_1 \cdot \tau_2 \cdot \tau_3 \cdots \tau_n$. **The setup is secure if *any single participant* destroyed their secret**. This is the "1-of-N" trust model; you only need to trust that *one* honest participant existed among potentially thousands.

The Zcash Powers of Tau ceremony (2017-2018) had 87 participants contribute to a universal phase, followed by circuit-specific ceremonies for Sapling. The Ethereum KZG Ceremony (2023) dwarfed this with over 140,000 contributions for EIP-4844 blob commitments.

Some ceremonies produce parameters usable for any circuit up to a size bound (universal), while others are tailored to specific circuits. KZG setups are inherently universal; the same powers of tau work for any polynomial of degree at most d.

The scale of modern ceremonies makes collusion effectively impossible. When 140,000 independent participants contribute, the probability that *all* of them colluded or were compromised approaches zero.

IPA/Bulletproofs: No Trusted Setup

The Inner Product Argument emerged from a different lineage than KZG. Bootle et al. (2016) introduced the core folding technique for efficient inner product proofs. Bünz et al. (2017) refined this into **Bulletproofs**, originally designed for *range proofs*, proving that a committed value lies in a range $[0, 2^n)$ without revealing it. This was motivated by confidential transactions in cryptocurrencies: prove your balance is non-negative without revealing the amount.

The terminology can be confusing:

- **IPA** (Inner Product Argument) is the *technique*: the recursive folding protocol that proves $\langle \vec{a}, \vec{b} \rangle = c$
- **Bulletproofs** is the *system* that used IPA for range proofs and general arithmetic circuits

Today, "IPA" and "Bulletproofs" are often used interchangeably to describe the folding-based polynomial commitment scheme. The scheme achieves transparency (no toxic waste) at the cost of logarithmic proofs and linear verification.

Polynomial Evaluation as Inner Product

As we saw in Chapters 4 and 5, polynomial evaluation is an inner product. For univariate polynomials:

$$f(z) = \sum_{i=0}^{n-1} c_i z^i = \langle \vec{c}, \vec{z} \rangle$$

where $\vec{c} = (c_0, \ldots, c_{n-1})$ are coefficients and $\vec{z} = (1, z, z^2, \ldots, z^{n-1})$ is the powers vector. For multilinear polynomials, the structure differs: $\tilde{f}(r_1, \ldots, r_n) = \langle \vec{f}, \vec{L}(r) \rangle$, where $\vec{f}$ contains evaluations on the

Boolean hypercube and $\vec{L}(r)$ contains Lagrange basis weights. Both cases reduce polynomial evaluation to an inner product claim, but the vectors involved differ. If we can prove inner product claims efficiently, we can prove polynomial evaluations. IPA does exactly this: it reduces the inner product by *folding* both vectors with random challenges, halving the dimension each round. This is the same algebraic trick as sum-check, with different cryptographic wrapping. We'll develop IPA using the univariate representation (coefficients × powers), but the technique applies to any inner product.

From Vector Commitments to Inner Product Claims

We've reduced polynomial evaluation to an inner product, and inner products operate on vectors. So to commit to a polynomial, we commit to a vector (its coefficients). Pedersen commitments provide exactly this: a way to commit to a vector such that we can later prove inner product claims about it.

The basic Pedersen vector commitment uses generators $\vec{G} = (G_0, \dots, G_{n-1})$ (one per coefficient) and H for blinding:

$$C = \langle \vec{c}, \vec{G} \rangle + r \cdot H = \sum_{i=0}^{n-1} c_i \cdot G_i + r \cdot H$$

This commits to the polynomial's coefficient vector $\vec{c} = (c_0, \dots, c_{n-1})$. The commitment C binds us to these coefficients, but to prove an evaluation $\langle \vec{c}, \vec{z} \rangle = v$, we need to bind the claimed value v into the protocol as well. IPA does this by introducing a separate generator U and forming:

$$P = \langle \vec{c}, \vec{G} \rangle + v \cdot U + r \cdot H$$

Think of P as encoding two things simultaneously: the coefficient vector (via $\vec{G}$) and the claimed inner product (via U). The folding protocol will manipulate both parts together, and only if v is the true inner product will everything stay consistent through the recursion.

The Folding Trick

The brilliant idea of IPA is recursive "folding" that shrinks the problem by half each round.

Setup

Prover holds coefficient vector $\vec{c}$ of length n. They've committed to it as $P = \langle \vec{c}, \vec{G} \rangle + v \cdot U$ where $v = \langle \vec{c}, \vec{z} \rangle$ is the claimed evaluation. (We omit the blinding term rH for clarity.)

One round of folding

1. Split $\vec{c} = (\vec{c}_L, \vec{c}_R)$ into two halves

2. Split $\vec{z} = (\vec{z}_L, \vec{z}_R)$ and $\vec{G} = (\vec{G}_L, \vec{G}_R)$ similarly
3. Prover computes and sends cross-term *commitments*:

$$L = \langle \vec{c}_L, \vec{G}_R \rangle + \langle \vec{c}_L, \vec{z}_R \rangle \cdot U$$

$$R = \langle \vec{c}_R, \vec{G}_L \rangle + \langle \vec{c}_R, \vec{z}_L \rangle \cdot U$$

   Note: L commits to the left coefficients using right generators, plus the cross inner product. Similarly for R.

4. Verifier sends random challenge α
5. **Prover** computes the folded coefficient vector (secretly):

$$\vec{c}' = \alpha \cdot \vec{c}_L + \alpha^{-1} \cdot \vec{c}_R$$

6. **Both parties** compute (using public information):
   - Folded evaluation vector: $\vec{z}' = \alpha^{-1} \cdot \vec{z}_L + \alpha \cdot \vec{z}_R$
   - Folded generators: $\vec{G}' = \alpha^{-1} \cdot \vec{G}_L + \alpha \cdot \vec{G}_R$
   - Updated commitment: $P' = L^{\alpha^2} \cdot P \cdot R^{\alpha^{-2}}$

Why this works

We need to show that P' is a valid commitment to $(\vec{c}', v')$ under the folded generators $\vec{G}'$.

First, expand what P' *should* be if the prover is honest:

$$P'_{\text{honest}} = \langle \vec{c}', \vec{G}' \rangle + v' \cdot U$$

where $v' = \langle \vec{c}', \vec{z}' \rangle$ is the new inner product claim.

Now expand $\langle \vec{c}', \vec{G}' \rangle$ using the folding formulas:

$$\langle \vec{c}', \vec{G}' \rangle = \langle \alpha \vec{c}_L + \alpha^{-1} \vec{c}_R,\ \alpha^{-1} \vec{G}_L + \alpha \vec{G}_R \rangle$$

Distributing the inner product (which is bilinear):

$$= \alpha \cdot \alpha^{-1} \langle \vec{c}_L, \vec{G}_L \rangle + \alpha \cdot \alpha \langle \vec{c}_L, \vec{G}_R \rangle + \alpha^{-1} \cdot \alpha^{-1} \langle \vec{c}_R, \vec{G}_L \rangle + \alpha^{-1} \cdot \alpha \langle \vec{c}_R, \vec{G}_R \rangle$$

$$= \langle \vec{c}_L, \vec{G}_L \rangle + \langle \vec{c}_R, \vec{G}_R \rangle + \alpha^2 \langle \vec{c}_L, \vec{G}_R \rangle + \alpha^{-2} \langle \vec{c}_R, \vec{G}_L \rangle$$

Similarly, expanding the new inner product $v' = \langle \vec{c}', \vec{z}' \rangle$:

$$v' = \langle \vec{c}_L, \vec{z}_L \rangle + \langle \vec{c}_R, \vec{z}_R \rangle + \alpha^2 \langle \vec{c}_L, \vec{z}_R \rangle + \alpha^{-2} \langle \vec{c}_R, \vec{z}_L \rangle = v + \alpha^2 L_{\mathsf{ip}} + \alpha^{-2} R_{\mathsf{ip}}$$

Now look at $P' = L^{\alpha^2} \cdot P \cdot R^{\alpha^{-2}}$ and expand each term:

- $P = \langle \vec{c}_L, \vec{G}_L \rangle + \langle \vec{c}_R, \vec{G}_R \rangle + v \cdot U$
- $L = \langle \vec{c}_L, \vec{G}_R \rangle + L_{\mathsf{ip}} \cdot U$
- $R = \langle \vec{c}_R, \vec{G}_L \rangle + R_{\mathsf{ip}} \cdot U$

So:

$$L^{\alpha^2} \cdot P \cdot R^{\alpha^{-2}} = \alpha^2 L + P + \alpha^{-2} R$$

$$= \alpha^2 (\langle \vec{c}_L, \vec{G}_R \rangle + L_{\mathsf{ip}} \cdot U) + (\langle \vec{c}_L, \vec{G}_L \rangle + \langle \vec{c}_R, \vec{G}_R \rangle + v \cdot U) + \alpha^{-2} (\langle \vec{c}_R, \vec{G}_L \rangle + R_{\mathsf{ip}} \cdot U)$$

Collecting terms:

$$= \underbrace{\langle \vec{c}_L, \vec{G}_L \rangle + \langle \vec{c}_R, \vec{G}_R \rangle + \alpha^2 \langle \vec{c}_L, \vec{G}_R \rangle + \alpha^{-2} \langle \vec{c}_R, \vec{G}_L \rangle}_{= \langle \vec{c}', \vec{G}' \rangle} + \underbrace{(v + \alpha^2 L_{\mathsf{ip}} + \alpha^{-2} R_{\mathsf{ip}})}_{= v'} \cdot U$$

This equals $\langle \vec{c}', \vec{G}' \rangle + v' \cdot U = P'_{\mathsf{honest}}$. The update formula produces exactly the right commitment!

The recursion

After $\log_2 n$ rounds, the vectors have length 1. The prover reveals the final scalar, and the verifier checks directly.

Final Verification: The Endgame

After $\log_2 n$ rounds of folding, the vectors have length 1:

- Prover holds a single scalar c' (the folded coefficient)
- The z-vector has folded to z' (known to both parties)
- The commitment has transformed to P_{final} through all the updates

The prover reveals

- The final coefficient $c' \in \mathbb{F}$
- The final blinding factor $r' \in \mathbb{F}$

The verifier must check: does c' actually correspond to the final commitment?

$$P_{\text{final}} \stackrel{?}{=} c' \cdot G'_1 + (c' \cdot z'_1) \cdot U + r' \cdot H$$

where z'_1 is the final folded evaluation point (known to both parties).

But what is G'_1? It's the result of folding all the generators through all $\log n$ rounds:

$$\vec{G}' = \alpha_1^{-1}\vec{G}_L^{(1)} + \alpha_1\vec{G}_R^{(1)} \quad \text{(first fold)}$$
$$\vec{G}'' = \alpha_2^{-1}\vec{G}'_L + \alpha_2\vec{G}'_R \quad \text{(second fold)}$$
$$\vdots$$

Computing this folded generator is the verifier's bottleneck: it requires applying all $\log n$ folding operations to the original n generators, taking $O(n)$ group operations. The verifier needs to know what commitment value a correctly-folded polynomial *should* produce, and there's no shortcut without computing the folded generators. This is IPA's fundamental trade-off: no trusted setup, but linear verification. We'll analyze when this cost is acceptable after the worked example.

Worked Example: IPA Verification

Let's trace through a complete IPA proof for a polynomial with 4 coefficients. This requires 2 rounds of folding. We work in $\mathbb{F}_{101}$, where $2^{-1} = 51$ (since $2 \cdot 51 = 102 \equiv 1$) and $3^{-1} = 34$ (since $3 \cdot 34 = 102 \equiv 1$).

Setup

- Coefficient vector: $\vec{c} = (3, 5, 2, 7)$ (prover's secret)
- Evaluation point: $z = 2$, so $\vec{z} = (1, 2, 4, 8)$ (public)
- Claimed evaluation: $v = \langle \vec{c}, \vec{z} \rangle = 3(1) + 5(2) + 2(4) + 7(8) = 77$
- Generators: G_1, G_2, G_3, G_4 (for coefficients), U (for inner product)
- Initial commitment: $P = (3G_1 + 5G_2 + 2G_3 + 7G_4) + 77U$

The verifier knows: P, $\vec{z}$, $v = 77$, and all generators. The verifier does *not* know $\vec{c}$.

Round 1 (reduce from 4 to 2 elements)

Prover's work (uses secret $\vec{c}$):

Split: $\vec{c}_L = (3, 5)$, $\vec{c}_R = (2, 7)$, $\vec{z}_L = (1, 2)$, $\vec{z}_R = (4, 8)$

Compute cross inner products:

- $\langle \vec{c}_L, \vec{z}_R \rangle = 3(4) + 5(8) = 52$
- $\langle \vec{c}_R, \vec{z}_L \rangle = 2(1) + 7(2) = 16$

Send commitments to verifier:

- $L_1 = (3G_3 + 5G_4) + 52U$
- $R_1 = (2G_1 + 7G_2) + 16U$

Verifier's challenge: $\alpha_1 = 2$, so $\alpha_1^{-1} = 51$

Both parties compute (verifier uses only public information):

Folded generators: $\vec{G}' = \alpha_1^{-1}\vec{G}_L + \alpha_1\vec{G}_R$

- $G'_1 = 51G_1 + 2G_3$
- $G'_2 = 51G_2 + 2G_4$

Folded evaluation vector: $\vec{z}' = \alpha_1^{-1}\vec{z}_L + \alpha_1\vec{z}_R$

- $z'_1 = 51(1) + 2(4) = 59$
- $z'_2 = 51(2) + 2(8) = 102 + 16 = 17 \pmod{101}$

Updated commitment: $P' = \alpha_1^2 L_1 + P + \alpha_1^{-2} R_1 = 4L_1 + P + 76R_1$

(Here $\alpha_1^{-2} = 51^2 = 2601 \equiv 76 \pmod{101}$)

The U-coefficient of P' becomes $v' = 77 + 4(52) + 76(16) = 77 + 208 + 1216 \equiv 87 \pmod{101}$.

Prover also computes (secretly):

$\vec{c}' = \alpha_1\vec{c}_L + \alpha_1^{-1}\vec{c}_R = 2(3,5) + 51(2,7) = (6 + 102, 10 + 357) = (108, 367) \equiv (7, 64) \pmod{101}$

Sanity check: $\langle \vec{c}', \vec{z}' \rangle = 7(59) + 64(17) = 413 + 1088 = 1501 \equiv 87 \pmod{101}$ ✓

Round 2 (reduce from 2 to 1 element)

Prover's work:

Split: $c'_L = 7, c'_R = 64, z'_L = 59, z'_R = 17$

Compute cross inner products:

- $c'_L \cdot z'_R = 7 \cdot 17 = 119 \equiv 18 \pmod{101}$
- $c'_R \cdot z'_L = 64 \cdot 59 = 3776 \equiv 38 \pmod{101}$

Send commitments:

- $L_2 = 7G'_2 + 18U$
- $R_2 = 64G'_1 + 38U$

Verifier's challenge: $\alpha_2 = 3$, so $\alpha_2^{-1} = 34$

Both parties compute:

Folded generator: $G'' = 34G'_1 + 3G'_2$

Folded evaluation point: $z'' = 34(59) + 3(17) = 2006 + 51 \equiv 36 \pmod{101}$

Updated commitment: $P'' = 9L_2 + P' + 45R_2$

(Here $\alpha_2^{-2} = 34^2 = 1156 \equiv 45 \pmod{101}$)

The U-coefficient of P'' becomes $v'' = 87 + 9(18) + 45(38) = 87 + 162 + 1710 \equiv 41 \pmod{101}$.

Prover computes:

$c'' = 3(7) + 34(64) = 21 + 2176 \equiv 75 \pmod{101}$

Sanity check: $c'' \cdot z'' = 75 \cdot 36 = 2700 \equiv 41 \pmod{101}$ ✓

Final verification

Prover reveals: $c'' = 75$

Verifier computes the fully folded generator G'' in terms of original generators:

$$G'' = 34G'_1 + 3G'_2 = 34(51G_1 + 2G_3) + 3(51G_2 + 2G_4)$$

$$= 1734G_1 + 153G_2 + 68G_3 + 6G_4 \equiv 17G_1 + 52G_2 + 68G_3 + 6G_4 \pmod{101}$$

This is the $O(n)$ work: computing a linear combination of all n original generators.

Verifier checks: $P'' \stackrel{?}{=} c'' \cdot G'' + (c'' \cdot z'') \cdot U$

Substituting: $P'' \stackrel{?}{=} 75 \cdot (17G_1 + 52G_2 + 68G_3 + 6G_4) + 41 \cdot U$

Expanding (mod 101): $P'' \stackrel{?}{=} 62G_1 + 61G_2 + 48G_3 + 46G_4 + 41U$

The verifier also computes P'' from the commitment updates: $P'' = 9L_2 + P' + 45R_2$, which traces back through $P' = 4L_1 + P + 76R_1$ to the original commitment $P = 3G_1 + 5G_2 + 2G_3 + 7G_4 + 77U$. Both sides match, so the proof is valid. The verifier is convinced that the prover knows $\vec{c}$ such that $\langle \vec{c}, \vec{z} \rangle = 77$, without ever learning $\vec{c}$.

Efficiency

- **Commitment size**: One group element (same as KZG)
- **Proof size**: $O(\log n)$ group elements (the L_i, R_i cross-terms from each round)
- **Verifier time**: $O(n)$ (must compute folded generators; this is the fundamental limitation)

- **Prover time**: $O(n \log n)$

The verifier's linear work is the main drawback compared to KZG's constant verification. However, IPA requires no trusted setup; the generators can be chosen transparently (e.g., by hashing).

The Linear Verifier Problem

This $O(n)$ verification cost is a serious limitation. For a polynomial with $N = 2^{20}$ coefficients (about 1 million), the verifier must perform over a million group operations, each involving expensive elliptic curve arithmetic. A scalar multiplication on an elliptic curve involves roughly 400 group additions, and each group addition involves 6-12 base field operations. The result: verification can be ~4000× slower than simple field arithmetic.

For interactive proofs where verification happens once, this is acceptable. For applications like blockchains where proofs are verified by thousands of nodes, or for recursive proof composition, linear verification becomes prohibitive.

This limitation motivated the development of schemes like Hyrax and Dory that exploit additional structure to achieve sublinear verification. (See Appendix D for Dory's approach.)

Comparing KZG and IPA

Property	KZG	IPA/Bulletproofs
Trusted setup	Required	None
Commitment size	$O(1)$	$O(1)$
Proof size	$O(1)$	$O(\log n)$
Verification time	$O(1)$	$O(n)$
Prover time	$O(n)$	$O(n \log n)$
Assumption	Pairings (q-SDH)	DLog only
Quantum-safe	No	No
Batch verification	Excellent	Good

- **KZG** is the right choice when verification efficiency is paramount and a trusted setup is acceptable. Most production SNARKs (Groth16, PLONK with KZG) use this approach.
- **IPA** is the right choice when trust minimization is critical, or in systems designed for transparent setups (Halo, Pasta curves).

What if we want both transparency *and* efficient verification? Schemes like **Hyrax** and **Dory** achieve sublinear verification without trusted setup by exploiting additional algebraic structure. The machinery is more complex, so we cover these advanced schemes in Appendix D.

Multilinear Polynomial Commitments

Both KZG and IPA extend naturally to multilinear polynomials, exploiting the tensor structure of Lagrange basis evaluations.

Multilinear KZG uses an SRS encoding Lagrange basis polynomials at a secret point. Opening proofs require ℓ commitments (one witness polynomial per variable), with verification using $\ell + 1$ pairings. Proof size grows linearly with the number of variables, not exponentially with coefficient count.

Multilinear IPA exploits the tensor structure of multilinear extensions. The evaluation vector has product structure that folding can exploit systematically, achieving logarithmic proof size with linear verification time.

The Role of PCS in SNARKs: Replacing Oracle Access

Polynomial commitment schemes are the cryptographic core that transforms interactive protocols into succinct, non-interactive proofs. To understand why, recall the gap we flagged in Chapter 3.

The Oracle Gap

Throughout this book, interactive proof protocols assume the verifier can evaluate certain polynomials at chosen points. Complexity theorists call this "oracle access": the verifier sends a query point, the oracle returns the correct evaluation, and the protocol moves on. Sum-check (Chapter 3) is the concrete example we have already seen: its final step requires the verifier to evaluate $g(r_1, \ldots, r_\nu)$. But the pattern is entirely general. Any interactive oracle proof (IOP) assumes that the verifier can query prover-supplied polynomials at arbitrary points. In practice, no oracle exists.

Sometimes the verifier can evaluate g herself. If g is built entirely from public data (circuit structure, known constants, Fiat-Shamir challenges), the verifier just computes. But in most SNARK applications, g depends on the prover's private witness. To make this concrete, consider Spartan's sum-check for R1CS satisfaction (we will study Spartan in detail in a later chapter):

$$g(x) = \widetilde{\mathsf{eq}}(\tau, x) \cdot \left[\widetilde{Az}(x) \cdot \widetilde{Bz}(x) - \widetilde{Cz}(x)\right]$$

Recall from Chapter 4 that $\widetilde{\mathsf{eq}}(\tau, x) = \prod_i (\tau_i x_i + (1-\tau_i)(1-x_i))$ is the equality polynomial: it "pins" the sum to a random evaluation point τ chosen by the verifier. The terms $\widetilde{Az}$, $\widetilde{Bz}$, $\widetilde{Cz}$ are MLEs of matrix-vector products involving the witness z. The verifier can compute $\widetilde{\mathsf{eq}}(\tau, r)$ on her own (she chose τ and knows r from the sum-check challenges), but she cannot compute $\widetilde{Az}(r)$, $\widetilde{Bz}(r)$, or $\widetilde{Cz}(r)$ without knowing the witness. Sum-check has done its job, reducing an exponential sum to evaluations at a single random point, but the verifier is stuck at the last mile.

How PCS Closes the Gap

The pattern for any IOP is the same. The prover holds a polynomial f that the verifier needs to query. A PCS turns the abstract oracle into a concrete mechanism via three steps:

1. **Before the IOP begins**, the prover commits to the polynomial: $C = \mathsf{Commit}(f)$. This commitment is short (a single group element for KZG, a hash root for FRI) and *binding*: the prover cannot change f after sending C.
2. **During the IOP**, random challenges are determined interactively (or via Fiat-Shamir). The commitment C was sent before any challenges were chosen, so the prover cannot adapt f to the query point.
3. **When the verifier needs an evaluation**, say $f(r)$ at some challenge point r, the prover provides the value v along with an *opening proof* π. The verifier checks $\mathsf{Verify}(C, r, v, \pi)$, confirming that v is the evaluation of the committed polynomial at the challenge point.

To return to our sum-check example: the prover commits to the witness polynomial $\widetilde{w}$ before the protocol starts, the sum-check challenges $r_1, \ldots, r_\nu$ are generated during the interaction, and at the end the prover opens $\widetilde{w}(r_1, \ldots, r_\nu)$ with a proof that the verifier checks against the commitment. But the same three-step structure applies whenever an IOP assumes oracle access, regardless of which protocol produced the query point.

The binding property is what makes this work. Because the prover committed to f before seeing the evaluation point, they cannot cheat: the committed polynomial is fixed, and the opening proof ties them to it. Schwartz-Zippel guarantees that checking at a random point catches any discrepancy with overwhelming probability.

This is the bridge between information theory and cryptography. An IOP achieves soundness assuming the verifier has oracle access to prover-supplied polynomials. The PCS *instantiates* that oracle, compiling the IOP into a cryptographic argument. Sum-check-based SNARKs are the most prominent example, but the compilation is universal: any IOP can be paired with any PCS. Chapter 11 develops this in full generality.

The Complete PCS Landscape

Now that we've seen both commitment schemes in depth, let's compare them systematically (including Dory from Appendix D and FRI from Chapter 10 for reference):

Property	KZG		IPA/Bulletproofs	Dory (App. D)	FRI (Ch. 10)
Trusted setup	Required	None	None	None	
Commitment size	$O(1)$	$O(1)$	$O(1)$	$O(1)$	
Proof size	$O(1)$	$O(\log N)$	$O(\log N)$	$O(\log^2 N)$	
Verification time	$O(1)$	$O(N)$	$O(\log N)$	$O(\log^2 N)$	
Prover time	$O(N)$	$O(N \log N)$	$O(N)$	$O(N \log N)$	
Assumption	q-SDH + Pairings	DLog only	DLog + Pairings	Hash collision	
Post-quantum	No	No	No	**Yes**	
Batching	Excellent	Good	Very good	Good	

KZG dominates when verification cost matters and trust is acceptable, which is why Ethereum L1 and most production SNARKs use it. IPA suits applications where trust minimization outweighs verification speed, like privacy-focused systems. FRI is the only option that survives quantum computers.

Key takeaways

The Core Abstraction

1. **Polynomial commitments bridge theory and practice.** Interactive proofs reduce complex claims to polynomial identities, but verifying those identities directly requires seeing the entire polynomial. A PCS lets the prover commit to a polynomial with a short commitment, then prove evaluations at specific points without revealing anything else.

2. **The interface is simple: Commit, Open, Verify.** Binding ensures the prover can't change the polynomial after committing. Succinctness ensures commitments and proofs are much smaller than the polynomial itself. These two properties are what make succinct proofs possible.

3. **Polynomial evaluation reduces to inner product.** For a polynomial $f(X) = \sum c_i X^i$, the evaluation $f(z) = \langle \vec{c}, (1, z, z^2, ...) \rangle$. This connection underlies IPA, which proves inner products directly via recursive folding.

The Two Paradigms

4. **KZG achieves constant-size proofs via pairings.** Everything rests on the fact that $(X - z)$ divides $f(X) - v$ exactly when $f(z) = v$. The prover commits to the quotient; the verifier checks divisibility at a secret point τ using one pairing equation. No matter the polynomial's size, the proof is one group element.

5. **KZG requires trusted setup.** The structured reference string encodes powers of a secret τ. If anyone learns τ, they can forge proofs. Multi-party ceremonies with thousands of participants ensure security under the "1-of-N" trust model: security holds if any single participant was honest.

6. **IPA eliminates trusted setup via recursive folding.** Each round halves the problem size by combining left and right halves with a random challenge. After $\log n$ rounds, the prover reveals a single scalar. The verifier checks consistency by tracking commitment updates through all rounds.

7. **IPA's bottleneck is linear verification.** The verifier must compute folded generators, requiring $O(n)$ group operations. This is acceptable for single proofs but prohibitive for recursive composition or blockchain verification where proofs are checked thousands of times. Schemes like Dory (Appendix D) address this limitation.

Practical Considerations

8. **Batching amortizes costs across many polynomials.** KZG batches evaluations at multiple points into one proof. For systems with dozens of committed polynomials, batching dominates the cost savings.

9. **The choice of PCS determines SNARK properties.** KZG gives constant verification with trusted setup (Groth16, PLONK). IPA gives transparency with linear verification (Halo). FRI (next chapter) gives post-quantum security. The right choice depends on whether you prioritize verification speed, trust minimization, or quantum resistance.

Chapter 10: Hash-Based Polynomial Commitments and FRI

In 2016, the National Institute of Standards and Technology issued a warning that sent cryptographers scrambling. Quantum computers were coming, and they would break everything built on elliptic curves: RSA, Diffie-Hellman, ECDSA. This included every SNARK that existed. Groth16, the darling of the blockchain world, would become worthless the day a sufficiently powerful quantum computer came online.

The "toxic waste" problem of trusted setups was bad. The "quantum apocalypse" was existential.

This urgency drove the creation of a new kind of proof system. Removing the trusted setup was only half the goal. The other half was to build on cryptographic primitives believed to resist quantum attacks. Hash functions are one such primitive (lattice-based cryptography is another).

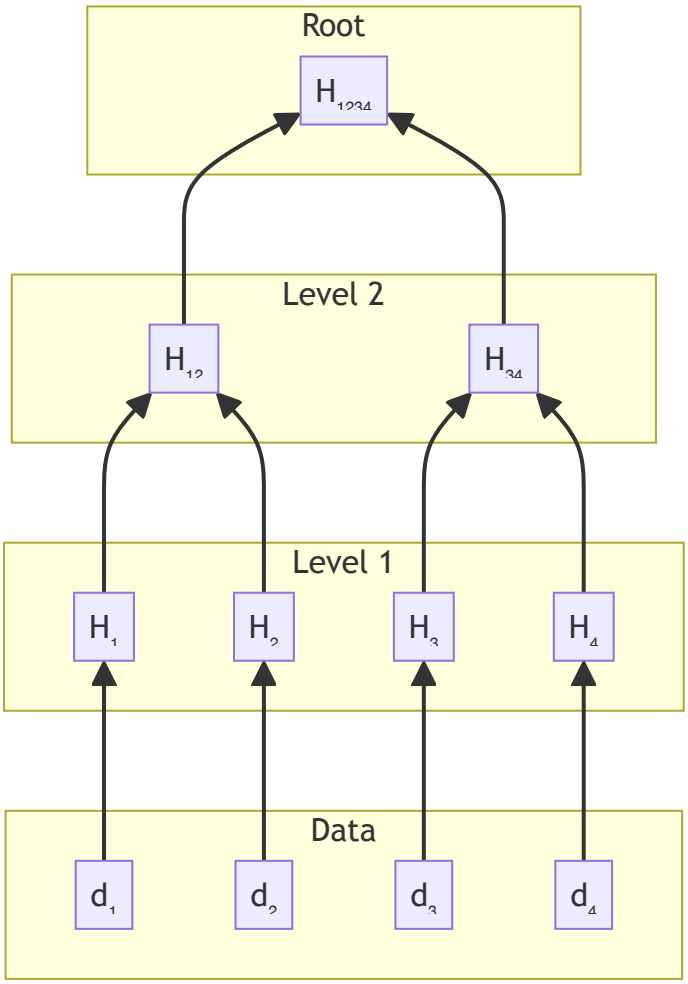
One answer came from Eli Ben-Sasson and collaborators: FRI (2017) and STARKs (2018). These are proof systems built entirely on hash functions, where "transparency" is not marketing but a technical property. No secrets. No ceremonies. No trapdoors that could compromise the system if they leaked, because no trapdoors exist at all.

The Merkle Tree: Committing to Evaluations

The foundation of hash-based commitments is the **Merkle tree**. If you've worked with Git or blockchain systems, you've already used one. The idea is simple: commit to a large dataset with a single hash value, then later prove any element is in the dataset without revealing the rest.

Construction:

1. Start with your data elements at the bottom (these are the "leaves")
2. Hash pairs of adjacent elements together: $H(\text{left} \| \text{right})$
3. Now you have half as many values. Repeat: hash pairs together again
4. Keep going until only one hash remains, the root



The root is your commitment. It's just 32 bytes, regardless of whether you're committing to 8 values or 8 million.

Opening a value: Suppose someone wants to verify that element x is at position i:

1. The prover provides x plus the "authentication path," which consists of the $\log n$ sibling hashes needed to recompute the path from x up to the root
2. The verifier recomputes hashes from leaf to root, checking the result matches the committed root

If any element were different, the root would change (assuming collision-resistant hashes). This makes the commitment *binding*.

Properties:

- Commitment size: One hash (32 bytes typically)

- Opening proof size: $O(\log n)$ hashes
- Binding: Changing any leaf changes the root (collision-resistance of hash)

For polynomial commitments, we commit to the polynomial's evaluations over a domain. The Merkle root becomes the polynomial commitment.

The Core Problem: Low-Degree Testing

Suppose the prover commits to a function $f : D \to \mathbb{F}$ by Merkle-committing its evaluations on a domain D of size n. The prover claims f is a low-degree polynomial (say degree less than d).

A polynomial evaluation vector *is* a Reed-Solomon codeword. In coding theory, a *codeword* is simply the encoded version of some message. If you have a polynomial $f(X)$ of degree $d-1$ and you evaluate it at n points (where $n > d$), the resulting vector $(f(x_1), f(x_2), \ldots, f(x_n))$ is a codeword of the Reed-Solomon code with parameters $[n, d]$. The polynomial's coefficients are the "message"; its evaluations are the "codeword." The extra evaluations beyond the d needed to specify the polynomial are the "redundancy" that lets us detect errors.

How can the verifier check that a Merkle-committed polynomial is actually low-degree without reading all n evaluations? The naive approach of checking random points doesn't help much: a function that agrees with a degree-d polynomial on all but one point would pass most spot-checks but isn't low-degree. The key is that low-degree polynomials are a sparse subset of all possible functions, and a function that's *not* low-degree must differ from every valid codeword in many positions. FRI exploits this structure to catch deviations with high probability.

Strictly speaking, FRI does not prove that f *is* a low-degree polynomial. It proves that f is *close* to one, meaning it differs from some valid codeword in at most a small fraction of positions (say, 10%). This distinction matters because a cheater could take a valid polynomial and change just one evaluation point. FRI might miss that single corrupted point on any given query.

More formally, a function $f : D \to \mathbb{F}$ is δ-close to degree d if there exists a polynomial $p(X)$ of degree $\leq d$ such that f and p agree on at least $(1-\delta)|D|$ points. The distance $\Delta(f, d) = \min_{\deg p \leq d} |\{x : f(x) \neq p(x)\}|/|D|$ measures how far f is from being low-degree. We tune the parameters (rate, number of queries) so that being "close" is good enough for our application, or so that the probability of missing the difference is cryptographically negligible (e.g., 2^{-128}). In practice, the gap between "is low-degree" and "is close to low-degree" vanishes into the security parameter.

The Two Phases of FRI

FRI has two phases. In the **commit phase**, the prover repeatedly folds the polynomial: each round, commit to the current polynomial's evaluations via Merkle tree, receive a random challenge, fold to

a smaller polynomial. This continues until the polynomial becomes a constant. In the **query phase**, the verifier spot-checks that the prover actually followed the folding rules, rather than committing to arbitrary values.

The commit phase is where the prover does the work; the query phase is where the verifier checks it.

The Commit Phase: Split and Fold

FRI transforms the low-degree testing problem through a recursive technique.

Any polynomial $f(X)$ can be decomposed into even and odd parts:

$$f(X) = f_E(X^2) + X \cdot f_O(X^2)$$

where:

- $f_E(Y)$ contains the even-power coefficients: $c_0 + c_2Y + c_4Y^2 + \cdots$
- $f_O(Y)$ contains the odd-power coefficients: $c_1 + c_3Y + c_5Y^2 + \cdots$

If $\deg(f) \leq d$, then $\deg(f_E) \leq d/2$ and $\deg(f_O) \leq d/2$. More precisely, $\deg(f_E) \leq \lfloor d/2 \rfloor$ and $\deg(f_O) \leq \lfloor (d-1)/2 \rfloor$. This degree halving is what makes FRI work.

Given a random challenge α from the verifier, we fold the polynomial:

$$f_1(Y) = f_E(Y) + \alpha \cdot f_O(Y)$$

This new polynomial has degree $\leq d/2$. The claim "f has degree at most d" reduces to "f_1 has degree at most $d/2$," a strictly smaller problem.

Where do Merkle trees fit in? Each round, the prover: 1. Evaluates the current polynomial f_i on domain D_i 2. Builds a Merkle tree over these evaluations (leaves are the $|D_i|$ field elements) 3. Sends the Merkle root to the verifier 4. Receives a random challenge α_i 5. Computes the folded polynomial f_{i+1} and repeats

The Merkle root commits the prover to specific evaluation values *before* seeing the challenge. The ordering matters. If the prover could see α_i first, they could craft fake evaluations that satisfy the folding check. By committing first, cheating becomes detectable.

Let's trace through the algebra with a concrete example.

Commit Phase: Worked Example

Let's trace through folding in $\mathbb{F}_{17}$.

Setup:

- Initial polynomial: $f_0(X) = X^3 + 2X + 5$ (degree 3, so $d = 4$)
- Domain D_0: The subgroup of order 8 generated by $\omega = 9$

  $D_0 = \{1, 9, 13, 15, 16, 8, 4, 2\}$

Round 0: Commit to f_0

The prover evaluates f_0 on D_0 and builds a Merkle tree over the 8 evaluations. The prover sends the Merkle root r_0 to the verifier.

Step 1: Decompose into even and odd parts

Coefficients: $(c_3, c_2, c_1, c_0) = (1, 0, 2, 5)$

- Even part: $f_{0,E}(Y) = c_2 Y + c_0 = 0 \cdot Y + 5 = 5$
- Odd part: $f_{0,O}(Y) = c_3 Y + c_1 = Y + 2$

Verify: $f_0(X) = 5 + X(X^2 + 2)$

Step 2: Receive challenge and fold

Verifier sends challenge $\alpha_0 = 3$ (only after receiving r_0).

$$f_1(Y) = f_{0,E}(Y) + \alpha_0 \cdot f_{0,O}(Y) = 5 + 3(Y + 2) = 3Y + 11$$

Result: We've reduced proving $\deg(f_0) < 4$ to proving $\deg(f_1) < 2$.

Step 3: New domain

The new domain D_1 consists of the squares of elements in D_0:

$D_1 = \{1^2, 9^2, 13^2, 15^2\} = \{1, 13, 16, 4\}$ (size 4)

Round 1: Commit to f_1

The prover evaluates f_1 on D_1:

- $f_1(1) = 3(1) + 11 = 14$
- $f_1(13) = 3(13) + 11 = 50 \equiv 16 \pmod{17}$
- $f_1(16) = 3(16) + 11 = 59 \equiv 8 \pmod{17}$
- $f_1(4) = 3(4) + 11 = 23 \equiv 6 \pmod{17}$

The prover builds a Merkle tree over these 4 evaluations and sends root r_1 to the verifier.

Step 4: Next challenge and fold

Verifier sends challenge $\alpha_1 = 7$ (only after receiving r_1).

$$f_2(Z) = 11 + 7 \cdot 3 = 11 + 21 = 32 \equiv 15 \pmod{17}$$

f_2 is a constant! The recursion terminates. The prover sends the constant 15 in the clear.

After $\log_2(d)$ rounds, the verifier holds $\log_2(d)$ Merkle roots (one per round), the random challenges $\alpha_0, \ldots, \alpha_{\log d - 1}$, and a claimed final constant c. But how does the verifier know the prover didn't just make up a convenient constant? The Merkle commitments bind the prover to specific values, but the verifier hasn't actually checked any of them yet. This is where the query phase comes in.

The Query Phase

The commit phase produced $k = \log_2(d)$ Merkle trees, one for each folding round. The i-th tree commits to the evaluations of f_i on domain D_i, where $|D_i| = |D_0|/2^i$. Each folding halves the domain size, so the trees get progressively smaller: D_0 has n leaves, D_1 has $n/2$, and so on down to D_{k-1} with $n/2^{k-1}$ leaves. A leaf in the i-th tree is a single evaluation $f_i(x)$ for some $x \in D_i$, and an "opening" is a Merkle path proving that leaf belongs to the committed root.

The verifier's goal is to check that these committed codewords are consistent with honest folding. If the prover cheated anywhere, the folding relationships won't hold for most positions. The verifier catches this by spot-checking: pick random positions and verify the folding formula.

The *rate* of a Reed-Solomon code is $\rho = d/n$, where d is the degree bound and n is the domain size. This is the fraction of positions that carry "real information" vs. redundancy. For example, if we commit to a degree-d polynomial by evaluating on a domain of size $n = 4d$, then $\rho = 1/4$.

Why does rate matter? A cheating prover who committed to the wrong polynomial faces this problem: the wrong polynomial differs from the correct one at most positions (they can agree on at most d points). Each random query has probability roughly ρ of hitting one of the "lucky" positions where cheating goes undetected. So each query catches the prover with probability at least $1 - \rho$.

With λ independent queries, the probability that *all* queries miss the cheating is at most ρ^λ. To achieve κ-bit security (soundness error $\leq 2^{-\kappa}$), we need $\rho^\lambda \leq 2^{-\kappa}$, which gives:

$$\lambda \geq \frac{\kappa}{\log_2(1/\rho)}$$

For $\rho = 1/4$ and $\kappa = 128$ bits of security: $\lambda \geq 128/\log_2(4) = 128/2 = 64$ queries. Lower rate means more redundancy and fewer queries needed, but larger proof size during the commit phase.

Each query works as follows. The verifier picks a random point y in the final domain D_k and traces backward through all k Merkle trees. Each folded domain D_{i+1} is the set of squares from D_i, i.e., $D_{i+1} = \{x^2 : x \in D_i\}$. Since $(-x)^2 = x^2$, every point $y \in D_{i+1}$ has exactly two preimages in D_i: some x and its negation $-x$.

To check the folding from f_i to f_{i+1}, the verifier needs three values: $f_i(x)$, $f_i(-x)$, and $f_{i+1}(y)$. The prover opens two leaves in the i-th Merkle tree (at positions x and $-x$) and one leaf in the $(i+1)$-th tree (at position y). Each opening includes a Merkle path proving the leaf belongs to the committed root. The verifier then checks that the folding formula holds:

$$f_{i+1}(y) = \frac{f_i(x) + f_i(-x)}{2} + \alpha_i \cdot \frac{f_i(x) - f_i(-x)}{2x}$$

This is the same $f_E + \alpha \cdot f_O$ folding from before, rewritten to use evaluations. The first term $\frac{f_i(x)+f_i(-x)}{2}$ recovers $f_E(y)$ and the second term $\frac{f_i(x)-f_i(-x)}{2x}$ recovers $f_O(y)$, since $f_i(x) = f_E(x^2) + x \cdot f_O(x^2)$ and $f_i(-x) = f_E(x^2) - x \cdot f_O(x^2)$.

This pairing structure relies on multiplicative subgroups: if ω generates D_0, then $-1 = \omega^{n/2}$, so $-x$ is in the group whenever x is.

The consistency check includes the final round: the verifier computes what $f_k(y)$ *should* be from the last committed codeword f_{k-1}, and checks that it equals the claimed constant c. If the prover lied about c, this check will fail with high probability.

In summary, one query opens $O(\log d)$ Merkle paths (two leaves per round for the sibling pairs, plus the positions in subsequent rounds). The verifier repeats this λ times with independent random positions, achieving soundness error ρ^λ as described above.

Query Phase: Worked Example

Let's continue our earlier example and trace through a complete query. Recall:

- $f_0(X) = X^3 + 2X + 5$ over $\mathbb{F}_{17}$
- Domain $D_0 = \{1, 9, 13, 15, 16, 8, 4, 2\}$ (8 elements)
- Challenge $\alpha_0 = 3$ produced $f_1(Y) = 3Y + 11$
- Domain $D_1 = \{1, 13, 16, 4\}$ (4 elements)
- Challenge $\alpha_1 = 7$ produced $f_2 = 15$ (constant)

The prover has built two Merkle trees: one committing to f_0's evaluations on D_0 (8 leaves), another committing to f_1's evaluations on D_1 (4 leaves). The prover sent both Merkle roots during the commit phase, then sent the final constant 15.

Step 1: Verifier picks a random query point

The verifier chooses a random position in D_1, say $y = 13$.

Step 2: Unfold to find preimages

What points in D_0 square to 13? We need x such that $x^2 \equiv 13 \pmod{17}$.

Checking: $9^2 = 81 \equiv 13$ and $(-9)^2 = 8^2 = 64 \equiv 13$. ✓

So the preimages are $x = 9$ and $-x = 8$.

Step 3: Query the prover

The verifier requests:

- $f_0(9)$ and $f_0(8)$ from the first Merkle tree
- $f_1(13)$ from the second Merkle tree

The prover supplies these values with Merkle authentication paths. Let's compute:

- $f_0(9) = 9^3 + 2(9) + 5 = 729 + 18 + 5 \equiv 15 + 1 + 5 = 21 \equiv 4 \pmod{17}$
- $f_0(8) = 8^3 + 2(8) + 5 = 512 + 16 + 5 \equiv 2 + 16 + 5 = 23 \equiv 6 \pmod{17}$
- $f_1(13) = 3(13) + 11 = 50 \equiv 16 \pmod{17}$

Step 4: Verify consistency (Round 0 → 1)

The verifier checks: does $f_1(13)$ equal the folded value from $f_0(9)$ and $f_0(8)$?

The consistency formula recovers the even and odd parts from evaluations at x and $-x$:

$$f_{0,E}(y) = \frac{f_0(x) + f_0(-x)}{2}, \quad f_{0,O}(y) = \frac{f_0(x) - f_0(-x)}{2x}$$

Why this works: Since $f_0(X) = f_{0,E}(X^2) + X \cdot f_{0,O}(X^2)$, we have $f_0(x) = f_{0,E}(y) + x \cdot f_{0,O}(y)$ and $f_0(-x) = f_{0,E}(y) - x \cdot f_{0,O}(y)$ where $y = x^2$. Adding these gives $2f_{0,E}(y)$; subtracting gives $2x \cdot f_{0,O}(y)$. Solving recovers the even and odd parts.

With $x = 9$, $-x = 8$, $y = x^2 = 13$:

$$f_{0,E}(13) = \frac{4 + 6}{2} = \frac{10}{2} = 5$$

For the odd part, note that $2x = 18 \equiv 1 \pmod{17}$:

$$f_{0,O}(13) = \frac{4-6}{1} = -2 \equiv 15 \pmod{17}$$

Now apply the folding with $\alpha_0 = 3$:

$$f_1(13) \stackrel{?}{=} f_{0,E}(13) + \alpha_0 \cdot f_{0,O}(13) = 5 + 3 \cdot 15 = 5 + 45 = 50 \equiv 16 \pmod{17}$$

✓ The Round 0 → 1 consistency check passes.

Step 5: Verify consistency (Round 1 → 2)

Now check: does the claimed constant $c = 15$ match what we'd get from folding f_1?

For the final round, the "domain" D_2 has collapsed to a single point. The verifier checks:

$$c \stackrel{?}{=} \frac{f_1(y) + f_1(-y)}{2} + \alpha_1 \cdot \frac{f_1(y) - f_1(-y)}{2y}$$

We have $y = 13$, so $-y = -13 \equiv 4 \pmod{17}$.

We need $f_1(4)$. The verifier requests this from the second Merkle tree (the prover opens the leaf at position 4 with a Merkle path). We have $f_1(4) = 3(4) + 11 = 23 \equiv 6 \pmod{17}$.

$$\frac{f_1(13) + f_1(4)}{2} = \frac{16+6}{2} = \frac{22}{2} = 11$$

For the second term, we need $(2 \cdot 13)^{-1} = 26^{-1} = 9^{-1} \equiv 2 \pmod{17}$:

$$\frac{f_1(13) - f_1(4)}{2 \cdot 13} = \frac{16-6}{26} = \frac{10}{9} = 10 \cdot 2 = 20 \equiv 3 \pmod{17}$$

$$c \stackrel{?}{=} 11 + 7 \cdot 3 = 11 + 21 = 32 \equiv 15 \pmod{17}$$

✓ **The query passes.** Both consistency checks hold, confirming that (at this query point) the prover's commitments are consistent with honest folding.

If the prover had lied about the constant, say claimed $c = 10$ instead of 15, this final check would fail: $11 + 21 = 32 \equiv 15 \neq 10$.

The verifier repeats this process at multiple random query points. Each independent query that passes increases confidence that the prover's polynomial truly has low degree.

The Folding Paradigm

FRI's "split and fold" is not an isolated trick; it's an instance of one of the most powerful patterns in zero-knowledge proofs. Now that we've seen both phases concretely, let's step back and recognize where we've encountered folding before.

The core idea: use a random challenge to collapse two objects into one, halving the problem size while preserving the ability to detect cheating. More precisely:

1. You have a claim about a "large" object (size n, degree d, dimension k)
2. Split the object into two "halves"
3. Receive a random challenge α
4. Combine the halves via weighted sum: new $=$ left $+ \alpha \cdot$ right
5. The claim about the original reduces to a claim about the folded object (size $n/2$, degree $d/2$, dimension $k - 1$)
6. Repeat until trivial

Randomness is what makes this work. If the original object was "bad" (not low-degree, not satisfying a constraint), the two halves encode this badness. A cheater would need the errors in left and right to cancel: $\text{error}_L + \alpha \cdot \text{error}_R = 0$. But they committed to both halves *before* seeing α, so this requires $\alpha = -\text{error}_L/\text{error}_R$ (a single value out of the entire field). Probability $\leq d/|\mathbb{F}|$.

We have already seen this pattern multiple times:

- **Sum-check (Chapter 3)**: Each round folds the hypercube in half. A claim "$\sum_{b \in \{0,1\}^n} g(b) = H$" becomes "$\sum_{b \in \{0,1\}^{n-1}} g(r_1, b) = V_1$".
- **MLE streaming evaluation (Chapter 4)**: Fold a table of 2^n values down to one. Each step combines $(T(0, ...), T(1, ...))$ with weights $(1 - r, r)$.
- **IPA/Bulletproofs (Chapter 9)**: Fold the commitment vector. Two group elements become one: $C' = C_L^{\alpha^{-1}} \cdot C_R^{\alpha}$.
- **FRI (this chapter)**: Fold the polynomial's coefficient space. A degree-d polynomial becomes degree-$d/2$ via $f_E + \alpha \cdot f_O$.

The deep insight is that folding is *dimension reduction via randomness*. High-dimensional objects are hard to verify directly; you'd need to check exponentially many conditions. But each random fold projects away one dimension while preserving the distinction between valid and invalid objects (with overwhelming probability). After $\log n$ folds, you're left with a trivial claim.

And yet the structure persists. At each level, the polynomial is smaller but the relationships that matter (the algebraic constraints, the divisibility conditions, the distance from invalidity) all survive the descent. You're looking at a different polynomial in a smaller domain, but it's recognizably the same

kind of object, facing the same *kind* of test. The recursion doesn't change the nature of the problem, only its scale.

Algebraically, this works because the objects being folded have low-degree polynomial structure. Schwartz-Zippel guarantees that distinct low-degree polynomials disagree almost everywhere. A random linear combination of two distinct polynomials is still distinct from the "honest" combination; you can't make errors cancel without predicting the randomness.

Another way to see it: one way to test if a polynomial is zero is to evaluate at a random point. Folding is this idea applied recursively with structure. Each fold is a random evaluation in disguise, and the structure ensures that evaluations compose coherently across rounds.

This paradigm extends beyond what we cover here. *Nova* and *folding schemes* (Chapter 23) fold entire R1CS instances: not polynomials, but constraint systems. The same principle applies: random linear combination of two instances yields a "relaxed" instance that's satisfiable iff both originals were.

Soundness and DEEP-FRI

The original FRI analysis (Ben-Sasson et al. 2018) established soundness but with somewhat pessimistic bounds. Achieving 128-bit security required many queries, increasing proof size.

DEEP-FRI (Ben-Sasson et al. 2019) improves soundness by sampling *outside* the evaluation domain. The idea: after the prover commits to the polynomial f, the verifier picks a random point z outside D and asks the prover to reveal $f(z)$. This "out-of-domain" sample provides additional security because a cheating prover can't anticipate which external point will be queried.

The name stands for **D**omain **E**xtending for **E**liminating **P**retenders. The technique achieves tighter soundness bounds, reducing the number of queries needed for a given security level. More recent work continues to improve these bounds: STIR (2024) achieves query complexity $O(\log d + \lambda \log\log d)$ compared to FRI's $O(\lambda \log d)$, where λ is the security parameter and d is the degree bound. WHIR (2024) further improves verification time to a few hundred microseconds. These protocols maintain FRI's core split-and-fold structure while optimizing the recursion.

FRI as a Polynomial Commitment Scheme

So far we've shown how FRI proves that a function is close to a low-degree polynomial. But a polynomial commitment scheme needs to prove *evaluation claims*: "my committed polynomial f satisfies $f(z) = v$." How do we bridge this gap?

The answer uses the **divisibility trick** from earlier chapters.

Applying the Divisibility Trick

Recall that $f(z) = v$ if and only if $(X - z)$ divides $f(X) - v$. When the claim is true, the quotient $q(X) = \frac{f(X)-v}{X-z}$ is a polynomial of degree $\deg(f) - 1$. When the claim is false, this "quotient" has a pole at z; it's not a polynomial at all.

This transforms evaluation proofs into degree bounds:

If…	Then the quotient $q(X) = \frac{f(X)-v}{X-z}$…
$f(z) = v$ (honest)	is a polynomial of degree $\deg(f) - 1$
$f(z) \neq v$ (cheating)	has a pole at z; not a polynomial at all

To prove $f(z) = v$, the prover constructs $q(X)$ and runs FRI configured with degree bound $d - 1$ to prove that q has degree $< d - 1$. This is not the same as proving q is merely "low-degree" in some vague sense; FRI must target the *specific* bound $d - 1$ matching the claimed degree of f. If a cheating prover submitted a q of degree d (one too high), FRI with bound $d - 1$ would catch it.

But FRI on q alone is not sufficient. It shows q has the right degree; it does not show that q is actually $\frac{f(X)-v}{X-z}$ rather than some unrelated polynomial of the same degree. The verifier must also spot-check the relationship $f(x) - v = (x - z) \cdot q(x)$ at random query points. If both checks pass, the quotient has the right degree *and* is correctly derived from f, which together imply $f(z) = v$.

The Full Protocol

Setup: Fix an evaluation domain D of size n (a multiplicative subgroup), a hash function H for Merkle trees, and a degree bound $d < n$.

Commit (prover): 1. Evaluate f on D to get $(f(x_1), \ldots, f(x_n))$ 2. Build a Merkle tree T_f over these evaluations 3. Send the Merkle root root_f to the verifier

After commit, the verifier holds only root_f. The prover holds f, all evaluations, and the full Merkle tree T_f.

Open (interactive, to prove $f(z) = v$):

Step 1: Construct the quotient - Prover computes $q(x) = \frac{f(x)-v}{x-z}$ for each $x \in D$ - Prover builds Merkle tree T_{q_0} over evaluations of $q_0 := q$, sends root_{q_0}

Step 2: FRI commit phase (folding)

For $i = 0, 1, \ldots, k-1$ where $k = \log_2(d)$: - Verifier sends random challenge α_i - Prover computes folded polynomial $q_{i+1}(Y) = q_{i,E}(Y) + \alpha_i \cdot q_{i,O}(Y)$ - Prover evaluates q_{i+1} on the folded domain D_{i+1} - Prover builds Merkle tree $T_{q_{i+1}}$, sends root$_{q_{i+1}}$

After k rounds, q_k is a constant c. Prover sends c.

Step 3: FRI query phase - Verifier sends λ random query positions - For each query, prover opens: - $f(x)$ from T_f (to check divisibility relation) - $q_0(x)$ from T_{q_0} (to check divisibility relation) - Sibling pairs $q_i(x), q_i(-x)$ from each T_{q_i} (to check folding consistency)

Verify: 1. Check Merkle proofs for all opened values 2. Check divisibility at each query: $f(x) - v \stackrel{?}{=} (x - z) \cdot q_0(x)$ 3. Check folding consistency at each query: for each round i, verify $q_{i+1}(x^2) = \frac{q_i(x)+q_i(-x)}{2} + \alpha_i \cdot \frac{q_i(x)-q_i(-x)}{2x}$ 4. Check final constant: the last folding step yields c

The verifier never sees the full polynomials f or q. They only see λ spot-checked evaluations, verified against the Merkle commitments.

Note that FRI doesn't speed anything up. It *is* the low-degree test. Without FRI, you'd have a Merkle commitment but no way to prove anything about degree: the prover could commit to arbitrary garbage. FRI is what makes this a *polynomial* commitment scheme rather than just a vector commitment.

There is a subtlety in the protocol above that deserves explicit attention. The FRI run proves q has degree $< d - 1$, and the spot-check proves q is consistent with f. Together these imply $f(z) = v$, *but only if f itself has degree $< d$*. What if the prover committed to a high-degree f (or arbitrary non-polynomial values) in the Merkle tree? Then $q = (f(X) - v)/(X - z)$ could pass the degree check by coincidence: a high-degree f minus v, divided by $(X - z)$, might produce a low-degree quotient if the high-degree terms cancel. The spot-check at query points would still pass because it verifies $f(x) - v = (x - z) \cdot q(x)$, which holds by construction regardless of the degree of f.

In the standalone FRI-as-PCS protocol presented here, f needs its own degree proof. The prover must either run a separate FRI instance on f to establish $\deg(f) < d$, or bundle f into a batched FRI alongside q (the next section shows how). In the STARK setting (Chapter 15), this issue is handled differently: FRI runs on the *composition polynomial*, which is a random linear combination of constraint quotients. The composition polynomial's degree bound implicitly constrains the trace polynomials, so no separate degree proof for the trace is needed. But in any context where FRI serves as a general-purpose PCS for opening evaluations, the degree of the committed polynomial must be established independently.

Batching

Multiple polynomials and evaluation points can be combined into a single FRI proof. Suppose we have k opening claims: $f_1(z_1) = v_1, \ldots, f_k(z_k) = v_k$. Each claim produces a quotient polynomial $q_i(X) = \frac{f_i(X) - v_i}{X - z_i}$.

The verifier sends a random batching challenge β. The prover computes the combined quotient:

$$Q(X) = q_1(X) + \beta \cdot q_2(X) + \beta^2 \cdot q_3(X) + \cdots + \beta^{k-1} \cdot q_k(X)$$

Now the prover runs FRI on Q:

1. Build a Merkle tree T_Q over evaluations of Q on domain D
2. For each folding round, build a Merkle tree over the folded polynomial
3. Send all Merkle roots and the final constant

The individual quotients q_i don't need their own FRI proofs since they're combined into Q before FRI runs. The savings come from running one FRI proof instead of k.

However, the verifier still needs to check that each original divisibility relation holds. At each query point x, the verifier: - Opens $f_1(x), \ldots, f_k(x)$ from their respective Merkle trees - Opens $Q(x)$ from T_Q - Computes each $q_i(x) = \frac{f_i(x) - v_i}{x - z_i}$ - Checks that $Q(x) = q_1(x) + \beta \cdot q_2(x) + \cdots + \beta^{k-1} \cdot q_k(x)$

The FRI cost (the expensive part) is amortized across all k claims. The divisibility spot-checks scale linearly with k, but these are just field arithmetic, cheap compared to FRI.

Practical Considerations

The Blow-up Factor

FRI evaluates polynomials on a domain much larger than their degree. If a polynomial has degree d, the evaluation domain has size $n = \rho^{-1} \cdot d$ where $\rho < 1$ is the **rate**.

Typical choices: $\rho = 1/4$ to $1/16$ (blow-up factor 4x to 16x).

Trade-off: Lower rate (more redundancy) means:

- Larger initial commitment (more evaluations)
- But stronger soundness per query (fewer queries needed)
- Net effect often neutral on total proof size

Chapter 20 quantifies this tradeoff for STARK provers, showing how grinding (proof-of-work) and batched FRI interact with the blowup factor to determine the optimal operating point for prover speed versus proof size.

Coset Domains

The examples above used multiplicative subgroups directly: $D_0 = \{1, \omega, \omega^2, ...\}$ where $\omega^n = 1$. In practice, FRI implementations typically use **cosets** instead: sets of the form $D = g \cdot H = \{g, g\omega, g\omega^2, ...\}$ where H is a multiplicative subgroup and $g \notin H$ is a generator offset.

Why the difference? Subgroups always contain 1, and satisfy $x^n = 1$ for all elements. This structure can be exploited in certain attacks. Cosets avoid this: no element satisfies $x^n = 1$ (since $g^n \neq 1$), removing a potential attack surface.

The folding arithmetic works identically. If $D_i = g_i \cdot H_i$, then squaring every element gives $D_{i+1} = g_i^2 \cdot H_{i+1}$ where $H_{i+1} = \{x^2 : x \in H_i\}$. The sibling structure (x and $-x$ mapping to the same $y = x^2$) is preserved. The only change is bookkeeping: the verifier tracks the coset offset g_i alongside the subgroup.

Hash Function Choice

STARKs using FRI rely on collision-resistant hash functions:

- Traditional: SHA-256, Keccak
- SNARK-friendly: Poseidon, Rescue (fewer constraints when verified in-circuit)

The hash function determines concrete security. If the hash has 256-bit output, and we assume collision-resistance, FRI inherits 128-bit security (birthday bound).

Comparing FRI to Algebraic PCS

Property	FRI	KZG	IPA
Trusted setup	None	Required	None
Assumption	Hash collision-resistance	Pairings + DLog	DLog
Post-quantum	Yes	No	No
Commitment size	$O(1)$	$O(1)$	$O(1)$
Proof size	$O(\lambda \log^2 d)$	$O(1)$	$O(\log d)$
Verifier time	$O(\lambda \log^2 d)$	$O(1)$	$O(d)$
Prover time	$O(d \log d)$	$O(d)$	$O(d \log d)$

When to use FRI:

- Trust minimization is critical (no setup ceremony)
- Post-quantum security is required
- Larger proofs are acceptable (still polylogarithmic)

When to avoid FRI:

- Proof size must be constant (KZG better)
- On-chain verification cost is critical (pairing checks cheaper than FRI verification)

FRI in the Wild: STARKs

FRI is the cryptographic backbone of **STARKs** (Scalable Transparent ARguments of Knowledge):

1. **Arithmetization**: Convert computation to polynomial constraints (AIR format)
2. **Low-degree extension**: Encode computation trace as polynomial evaluations
3. **Constraint checking**: Combine with composition polynomial
4. **FRI**: Prove the composed polynomial is low-degree

The "T" in STARK stands for "Transparent": no trusted setup, enabled by FRI. The "S" stands for "Scalable": prover time is quasi-linear, enabled by FFT and the recursive structure of FRI.

Modern systems like **Plonky2** and **Plonky3** combine PLONK's flexible arithmetization with FRI-based commitments, getting the best of both worlds.

Key takeaways

1. **Merkle trees commit to evaluations, not coefficients.** FRI commits to a polynomial by hashing its evaluations over a domain D. The Merkle root is 32 bytes regardless of polynomial size. Opening a single evaluation costs $O(\log |D|)$ hashes.
2. **FRI proves proximity to low-degree, not exact low-degree.** A function passing FRI is δ-close to some degree-d polynomial (agrees on at least $(1-\delta)|D|$ points). For cryptographic applications, "close enough" suffices because the gap vanishes into the security parameter.
3. **Folding halves the degree per round.** The decomposition $f(X) = f_E(X^2) + X \cdot f_O(X^2)$ splits a degree-d polynomial into two degree-$d/2$ parts. A random combination $f_1 = f_E + \alpha \cdot f_O$ preserves errors: if f wasn't low-degree, neither is f_1 (with overwhelming probability).
4. **Commit before challenge, verify after.** Each round the prover Merkle-commits to the current polynomial's evaluations, *then* receives the folding challenge α. This ordering prevents the prover from crafting fake evaluations that happen to satisfy the folding check.

5. **Query cost depends on rate.** With rate $\rho = d/n$, each query catches cheating with probability $\geq 1 - \rho$. For κ-bit security: $\lambda \geq \kappa / \log_2(1/\rho)$ queries. Lower rate means fewer queries but larger commitments.

6. **Divisibility converts evaluation claims to degree bounds.** To prove $f(z) = v$, show that $q(X) = (f(X) - v)/(X - z)$ is a polynomial of degree $d - 1$. If $f(z) \neq v$, then q has a pole at z and isn't a polynomial at all.

7. **FRI is the mechanism, not an optimization.** Without FRI, a Merkle commitment is just a vector commitment with no degree guarantees. FRI is what makes this a *polynomial* commitment scheme.

8. **Transparency comes from hash functions.** The only cryptographic assumption is collision-resistance of the hash. No trusted setup, no toxic waste, no trapdoors. Anyone can verify proofs with the same public parameters.

9. **Post-quantum security.** Hash functions are believed to resist quantum attacks (Grover's algorithm only provides quadratic speedup). FRI-based proofs remain secure when elliptic curve schemes break.

10. **The cost is proof size.** FRI proofs are $O(\lambda \log^2 d)$ compared to KZG's $O(1)$. For applications where on-chain verification cost dominates (Ethereum L1), this matters. For applications prioritizing trust minimization or quantum resistance, FRI wins.

Chapter 11: The SNARK Recipe: Assembling the Pieces

Before compilers, programmers wrote machine code by hand. Each program required intimate knowledge of the target CPU's instruction set. A program for one machine wouldn't run on another. It was slow, error-prone, and expertise barely transferred between architectures.

Then came FORTRAN (1957) and the idea of a compiler: a standardized translation process that takes a high-level program and produces machine code for any target. The programmer writes once; the compiler handles the details. Different programs produce different executables, but the *methodology* is uniform.

For the first 30 years of zero-knowledge (1985–2015), protocols were like hand-written assembly. A cryptographer would craft a protocol for Graph Isomorphism, then start from scratch for Hamiltonian Cycles. Each proof system was a custom creation.

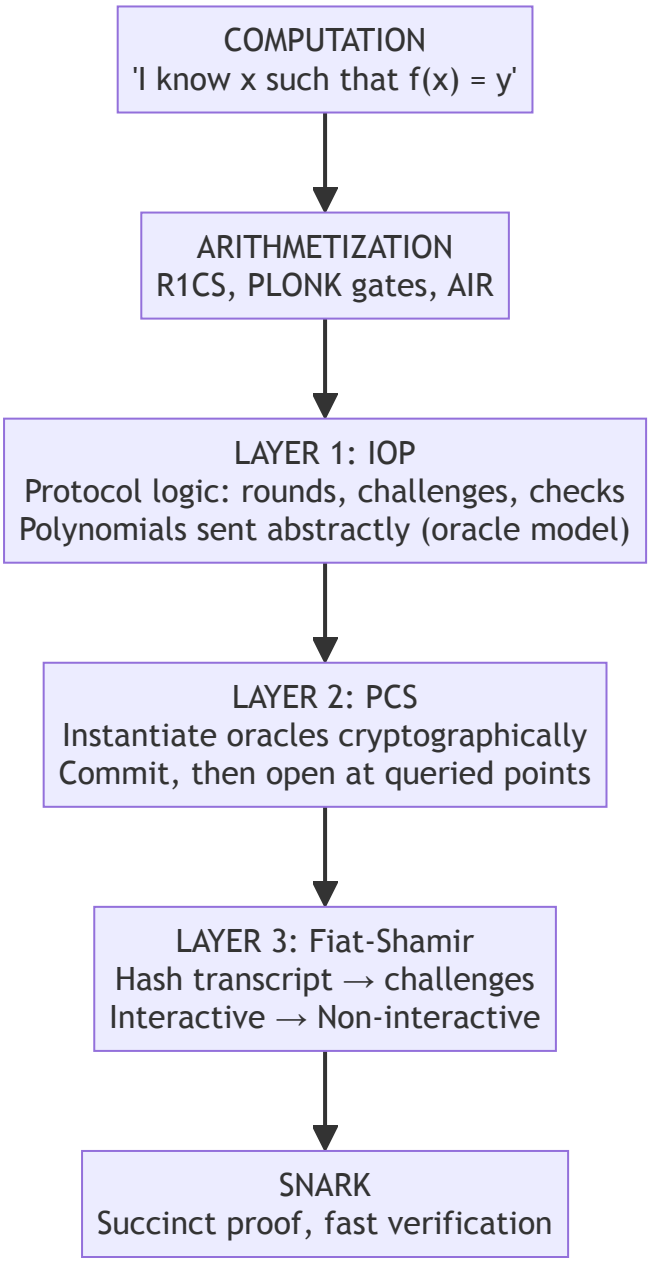
Modern SNARKs are like compilers. You feed in a computation, and out comes a proof. Different computations produce different proofs, but the *recipe* is standardized. This chapter describes that recipe. It powers every modern SNARK from Groth16 to Halo 2 to STARKs.

(The analogy extends further: a zkVM is like compiling an *interpreter* once, then running arbitrary programs through it. One circuit, any computation. If you're unfamiliar with zkVMs, don't worry; the concept will make more sense after seeing how circuits work.)

Modern SNARKs decompose into three layers, each with a distinct role. Understanding this decomposition is more valuable than memorizing any particular system; it provides the conceptual vocabulary to navigate the entire landscape. The key abstraction enabling this modularity is the *Interactive Oracle Proof* (IOP), introduced by Ben-Sasson, Chiesa, and Spooner in 2016. IOPs unified the earlier notions of interactive proofs and probabilistically checkable proofs into a single framework that makes the "IOP + PCS" compilation strategy possible.

The Three-Layer Architecture

Every modern SNARK follows the same structural pattern:



Layer 1 defines the protocol logic: the sequence of rounds, what polynomials the prover "sends," what queries the verifier makes, and what checks determine acceptance. This is where sum-check lives, where PLONK's permutation argument is specified, where GKR's layer-by-layer reduction happens. The prover "sends polynomials" in an abstract sense; the verifier has *oracle access* (can query any evaluation without seeing the full polynomial). Layer 1 specifies *what* to prove and *how* to check it.

Layer 2 instantiates the oracle model cryptographically. Oracle access becomes commitment and opening: the prover commits to a polynomial before seeing queries, then provides evaluation proofs at requested points. The binding property of the commitment scheme ensures the prover cannot retroactively modify their polynomial.

Layer 3 eliminates interaction. The verifier's random challenges are replaced by hash function outputs computed from the transcript. The prover simulates the entire interaction locally and outputs a static proof.

This separation enables genuine modularity: the same IOP can be compiled with different commitment schemes, yielding systems with different trust assumptions, proof sizes, and verification costs. PLONK with KZG gives constant-size proofs requiring trusted setup. PLONK with FRI gives larger proofs but no trusted setup and post-quantum security. The IOP is unchanged; only the cryptographic instantiation differs.

Layer 1: Interactive Oracle Proofs

An **Interactive Oracle Proof (IOP)** is an interactive protocol where the prover sends *polynomials* rather than field elements, and the verifier has oracle access to these polynomials: they can query any evaluation without seeing the full polynomial description. The IOP defines the protocol logic: what polynomials are exchanged, what queries the verifier makes, and what checks determine acceptance.

Example: Sum-Check as an IOP

To make this concrete, consider how the sum-check protocol fits into the IOP framework. (This is just one example; other IOPs like PLONK's permutation argument or GKR have different structures.)

1. Prover sends univariate polynomial $g_1(X_1)$
2. Verifier evaluates $g_1(0)$ and $g_1(1)$, checks $g_1(0) + g_1(1) = H$ (the claimed sum)
3. Verifier sends random challenge r_1
4. Prover sends univariate polynomial $g_2(X_2)$
5. Verifier evaluates $g_2(0)$ and $g_2(1)$, checks $g_1(r_1) = g_2(0) + g_2(1)$
6. Continue for n rounds
7. **Final step**: Verifier queries the original polynomial f at $(r_1, \ldots, r_n)$ and checks $g_n(r_n) = f(r_1, \ldots, r_n)$

The univariate polynomials g_i are low-degree (degree at most d in one variable), so they can be sent explicitly as $O(d)$ coefficients. But the final step requires oracle access to f: the verifier must query $f(r_1, \ldots, r_n)$ to verify that the sum-check reductions were honest. This is where the PCS comes in.

IOP Quality Metrics

Not all IOPs are equivalent. The critical parameters:

Query complexity: The number of evaluation queries the verifier makes. Each query becomes an evaluation proof in the compiled SNARK, directly affecting proof size.

Round complexity: The number of prover-verifier exchanges. Each round becomes a hash computation in Fiat-Shamir. Sum-check has $O(\log n)$ rounds; some IOPs achieve constant rounds.

Prover complexity: The computational cost of generating the prover's messages. This should be quasi-linear in the computation size: $O(n \log n)$ or $O(n \log^2 n)$. Quadratic prover complexity renders the system impractical for large computations.

Soundness error: The probability that a cheating prover convinces the verifier. Typically $O(d/|\mathbb{F}|)$ per round, where d is the maximum polynomial degree.

These parameters trade off against each other. Fewer queries mean smaller proofs but often require more prover work or stronger assumptions. The art of IOP design lies in navigating these trade-offs for specific applications.

From Oracle Model to Cryptography

IOPs assume the verifier can query certain polynomials at points of their choosing, with the polynomial fixed before the query point is revealed. In sum-check, the univariate polynomials g_i are sent explicitly, so the verifier evaluates them directly. But the original polynomial f is too large to send. The verifier needs to query $f(r_1, \ldots, r_n)$ at the final step, and this query must be answered by something other than sending the entire polynomial. This is where oracle access matters.

Why does the ordering matter? Recall the Schwartz-Zippel lemma: a nonzero polynomial of degree d has at most d roots. If the verifier picks a random point r from a field of size $|\mathbb{F}|$, a cheating prover's polynomial (which should be zero but isn't) will fail the check with probability at least $1 - d/|\mathbb{F}|$. With typical parameters ($|\mathbb{F}| = 2^{256}$, $d = 10^6$), a single random query catches cheating with overwhelming probability.

But this analysis assumes the polynomial is fixed before r is chosen. If the prover sees r first, they can construct a polynomial that passes the check at r while being wrong elsewhere. The oracle model captures this constraint abstractly; Layer 2 enforces it cryptographically through commitment schemes.

Layer 2: Polynomial Commitment Schemes

The IOP assumes the verifier can query polynomial evaluations. In reality, there is no oracle: the prover must send something over a communication channel. The **polynomial commitment scheme (PCS)** bridges the gap, turning the abstract oracle into a concrete cryptographic mechanism. Chapters 9 and 10 covered PCS in detail; here's the quick reminder of what matters for compilation.

A PCS provides three operations: Commit (polynomial to short commitment), Open (produce evaluation proof), and Verify (check the proof). The critical property is **binding**: once the prover sends a commitment, they cannot open it to evaluations of a different polynomial. For arguments *of knowledge*, the PCS must also be *extractable*: if a prover can pass verification, there exists an extractor that can reconstruct the polynomial they committed to.

Compilation

The compilation from IOP to **interactive argument**, a protocol where prover and verifier exchange messages with soundness based on cryptographic assumptions rather than information-theoretic guarantees, is mechanical:

- When the IOP specifies "prover sends polynomial f," the compiled protocol has the prover send $C = \text{Commit}(f)$
- When the IOP specifies "verifier queries $f(z)$," the compiled protocol has the verifier announce z, the prover respond with $v = f(z)$ and proof π, and the verifier check $\text{Verify}(C, z, v, \pi)$

Why Compilation Preserves Soundness

The IOP's soundness proof assumes the verifier receives the true evaluation $f(z)$ when they query. After compilation, the verifier instead receives a claimed value v with a proof π.

The binding property ensures the prover can only open to evaluations the committed polynomial actually takes. Since the prover sends C before seeing the query point z, binding cryptographically enforces the ordering that the oracle model assumes. If binding fails, the prover could commit to one polynomial and open to another, collapsing soundness entirely.

PCS Choices

Different commitment schemes offer different trade-offs:

PCS	Setup		Proof Size	Verification	Assumption
KZG	Trusted	$O(1)$	$O(1)$	q-SDH + Pairings	
IPA	Transparent	$O(\log n)$	$O(n)$	DLog	
FRI	Transparent	$O(\log^2 n)$	$O(\log^2 n)$	Collision-resistant hash	

The choice is application-dependent. On-chain verification pays per byte and per operation; KZG's constant-size proofs minimize gas costs. Systems prioritizing trust minimization accept larger proofs for transparent setup. Long-term security considerations may favor FRI's resistance to quantum attacks.

Soundness Composition

Recall that soundness error is the probability a cheating prover convinces the verifier of a false statement, and binding error is the probability a prover can open a commitment to two different values. Both are negligible for secure constructions.

Let the IOP have soundness error ϵ_{IOP} and the PCS have binding error ϵ_{bind}. The resulting SNARK (IOP + PCS) has soundness error at most $\epsilon_{\mathsf{IOP}} + \epsilon_{\mathsf{bind}}$.

Proof sketch: A cheating prover either (1) breaks the IOP soundness by finding a cheating strategy that succeeds with the committed polynomial, or (2) breaks binding by opening to evaluations inconsistent with the commitment. By union bound, cheating succeeds with probability at most $\epsilon_{\mathsf{IOP}} + \epsilon_{\mathsf{bind}}$.

Layer 3: The Fiat-Shamir Transformation

The Fiat-Shamir transformation fits in one line of pseudocode. Virtually every deployed SNARK uses it, and subtle implementation errors have led to real-world vulnerabilities.

Adi Shamir and Amos Fiat introduced the technique in 1986, originally to convert interactive identification schemes into digital signatures. Their insight was that if the verifier's only role is to provide randomness, a hash function can play that role instead. The idea predates SNARKs by decades, but it applies directly: after PCS compilation, we have an *interactive* argument where the verifier's only contribution is random challenges. For many applications (blockchain verification, credential systems, asynchronous protocols) this interaction is unacceptable. We need a static proof that anyone can verify without engaging in a conversation.

The **Fiat-Shamir transformation** achieves this by replacing the verifier's random challenges with hash function outputs.

In the interactive protocol:

```
Prover -> commitment C_1 -> Verifier
Verifier -> random r_1 -> Prover
Prover -> commitment C_2 -> Verifier
Verifier -> random r_2 -> Prover
...
```

After Fiat-Shamir:

```
Prover computes:
  C_1 = Commit(f_1)
  r_1 = Hash(C_1)
  C_2 = Commit(f_2)
  r_2 = Hash(C_1 || r_1 || C_2)
  ...
Prover outputs: (C_1, C_2, ..., evaluations, proofs)
```

The verifier reconstructs challenges from the transcript and performs all checks.

Security Analysis

The interactive protocol's soundness rests on *unpredictability*: the prover commits to C_1 without knowing what challenge r_1 will be. This prevents the prover from crafting commitments that exploit specific challenges.

In an interactive proof, the verifier sends a random challenge *after* the prover commits. The prover cannot change the past. In a non-interactive proof, the prover generates the challenge themselves. What stops them from cheating?

Fiat-Shamir preserves unpredictability under the **random oracle model**: the assumption that the hash function behaves like a truly random function. If the prover cannot predict Hash(C_1) before choosing C_1, they face the same constraint as in the interactive setting.

A cheating prover's only recourse is to try many values of C_1, compute Hash(C_1) for each, and hope to find one yielding a favorable challenge. This is a **grinding attack**. If the underlying protocol has

soundness error ϵ, and the prover can compute T hashes, the effective soundness error becomes roughly $T \cdot \epsilon$.

For a protocol with $\epsilon = 2^{-128}$ and an adversary computing $T = 2^{40}$ hashes, the effective soundness is 2^{-88} (still negligible). Larger fields provide additional margin.

Transcript Construction

A subtle but critical requirement: the hash must include the *entire* transcript up to that point.

The challenge r_i must depend on:

- The public statement being proved
- All previous commitments $C_1, \ldots, C_{i-1}$
- All previous challenges $r_1, \ldots, r_{i-1}$
- All previous evaluation proofs

Omitting the public statement allows the same proof to verify for different statements (a complete soundness failure). Omitting previous challenges may allow the prover to fork the transcript and find favorable paths. These aren't hypothetical concerns: the "Frozen Heart" vulnerability (2022) affected Bulletproofs, PlonK, and multiple production codebases because public inputs weren't included in transcript hashes. The "Last Challenge Attack" (2024) exploited similar issues in KZG batching. A 2023 survey found over 30 weak Fiat-Shamir implementations across 12 different proof systems.

Modern implementations prevent these errors using the **sponge model** for transcript construction. Every time the prover speaks, they "absorb" their message into the sponge state. Every time they need a challenge, they "squeeze" to extract random bits. This ensures each challenge depends on the *entire* history, not just the most recent message. You cannot get fresh randomness out without first putting your commitment in, and once something is absorbed, it permanently affects all future outputs.

The Random Oracle Caveat

Fiat-Shamir security is proven in the random oracle model. Real hash functions are not random oracles; they are deterministic algorithms with internal structure.

No practical attacks are known against carefully instantiated Fiat-Shamir. But there is no proof of security from standard assumptions. The hash function must be collision-resistant, but collision resistance alone does not suffice for Fiat-Shamir security.

This remains one of the gaps between theory and practice in deployed cryptography.

Concrete Trace: R1CS to SNARK

The three layers assume the computation is already expressed as polynomial identities. This prior step, **arithmetization**, converts the statement "I know w such that $C(x, w) = 1$" into constraint systems (R1CS, PLONK gates, AIR) that the IOP can work with.

Consider proving knowledge of a satisfying R1CS witness.

Arithmetization

The R1CS constraint $(A \cdot Z) \circ (B \cdot Z) = C \cdot Z$ must hold for the witness vector $Z = (1, \text{io}, W)$, where io contains the public inputs/outputs and W contains the private values. The full witness is encoded as its multilinear extension $\tilde{Z}$: the unique polynomial of degree at most 1 in each variable satisfying $\tilde{Z}(b) = Z_b$ for all $b \in \{0,1\}^n$. Define $\tilde{g}(X)$ such that $\tilde{g}$ vanishes on all of $\{0,1\}^n$ if and only if the constraints are satisfied.

IOP (Sum-Check)

To prove all constraints are satisfied, the prover proves:

$$\sum_{X \in \{0,1\}^n} \tilde{g}(X) = 0$$

Each sum-check round, the prover sends the univariate polynomial g_i in the clear (it's low-degree, so this is just a few field elements). After n rounds, this reduces to a single evaluation of $\tilde{Z}$ at a random point $(r_1, \ldots, r_n)$.

PCS Compilation (with KZG)

The only polynomial requiring commitment is $\tilde{Z}$ (too large to send explicitly):

- Prover sends $C_Z = \text{KZG.Commit}(\tilde{Z})$ at the start
- Final evaluation $\tilde{Z}(r_1, \ldots, r_n)$ comes with a KZG opening proof

In the **Fiat-Shamir** transform, each challenge r_i is computed as $\text{Hash}(\text{transcript})$. The final proof is the transcript of round polynomials plus the opening proof.

Proof Size Analysis

For a circuit with $n = 20$ variables (approximately one million gates), with KZG:

- Sum-check round polynomials: ~20 rounds × ~3 coefficients × 32 bytes = ~2 KB
- Batched KZG opening proof: ~48 bytes

Total: approximately 2 KB.

The witness contains millions of field elements. The proof is five orders of magnitude smaller. This is succinctness.

With FRI instead of KZG, proof size grows to ~100 KB (larger, but still succinct, and requiring no trusted setup).

Zero-Knowledge

We have focused on succinctness and soundness. The basic construction does not provide zero-knowledge: the sum-check polynomials reveal information about the witness.

A proof system is *zero-knowledge* if there exists a simulator $\mathcal{S}$ that, given only the statement (not the witness), produces transcripts indistinguishable from real proofs. Intuitively: the proof reveals nothing about the witness beyond the truth of the statement. The verifier could have generated the same transcript themselves without seeing the witness.

Adding zero-knowledge requires additional techniques:

- **Hiding commitments**: randomized commitments (Pedersen with blinding factors) so the commitment reveals nothing about the polynomial
- **Masking polynomials**: random low-degree polynomials added to the prover's messages that sum to zero (preserving correctness) but obscure individual evaluations

Chapter 17 develops these techniques in detail. The key point here: zero-knowledge is a property *layered on top* of the basic SNARK construction. The three-layer architecture applies equally to zero-knowledge and non-zero-knowledge systems.

Modularity in Practice

The three-layer decomposition has practical consequences beyond conceptual clarity.

- **Upgradability**: When a better PCS is developed, existing IOPs can adopt it. PLONK was originally specified with KZG. It now has FRI-based variants (Plonky2, Plonky3) that inherit PLONK's arithmetization and IOP while gaining transparency and post-quantum resistance.
- **Specialized optimization**: Each layer can be optimized independently. Improvements to sum-check proving (Chapter 19) benefit all sum-check-based SNARKs regardless of their PCS. Improvements to KZG batch opening benefit all KZG-based systems regardless of their IOP.
- **Analysis decomposition**: Security analysis can proceed layer by layer. The IOP's soundness is analyzed in the oracle model. The PCS's binding property is analyzed under its cryptographic

assumption. Fiat-Shamir security is analyzed in the random oracle model. Each analysis is self-contained.

- **System comprehension**: When encountering a new SNARK, the first questions are: What is the IOP? What is the PCS? This decomposition makes the landscape navigable. New systems become variations on known themes rather than entirely novel constructions.

Taxonomy

With the three-layer model, we can classify the SNARK landscape:

By IOP:

- *Linear PCP-based*: Groth16 (the prover's messages are linear combinations of wire values, enabling constant verification via encrypted linear checks)
- *Polynomial IOP-based*: PLONK, Marlin (the prover sends polynomials, the verifier checks polynomial identities)
- *Sum-check-based*: Spartan, Lasso (verification reduces to sum-check over multilinear polynomials)
- *FRI-based*: STARKs (low-degree testing via the FRI protocol)

By PCS:

- *Pairing-based*: KZG (constant-size proofs, trusted setup)
- *Discrete-log-based*: IPA/Bulletproofs (logarithmic proofs, transparent)
- *Hash-based*: FRI (polylogarithmic proofs, post-quantum)

By setup requirements:

- *Circuit-specific*: Groth16 (new trusted setup per circuit)
- *Universal*: PLONK, Marlin (single trusted setup for all circuits up to a size bound)
- *Transparent*: STARKs, Spartan+IPA (no trusted setup)

No single system dominates all metrics. The choice depends on what constraints bind most tightly in a given application. The coming chapters examine many of these systems in detail: Groth16, PLONK, STARKs, Spartan, and others.

Key takeaways

1. **Three-layer architecture**: IOP defines protocol logic, PCS provides cryptographic binding, Fiat-Shamir eliminates interaction. Each layer is analyzed independently.

2. **Commitment ordering is the key**: The prover commits before the verifier queries. The PCS's binding property cryptographically enforces this ordering, which is what enables random evaluation to catch cheating.

3. **Fiat-Shamir security requires complete transcripts**: Every prover message must enter the hash, including the public statement. Omissions break soundness; grinding attacks bound the effective advantage.

4. **Modularity is structural**: Same IOP, different PCS yields different systems. This is how the field evolves.

5. **Query complexity determines proof size**: Each IOP query becomes a PCS opening proof.

6. **Zero-knowledge is additive**: The basic construction gives succinctness and soundness. Zero-knowledge requires additional masking.

7. **No universal optimum**: KZG minimizes proof size with trusted setup. FRI eliminates setup with larger proofs. IPA trades verification time for transparency. The choice is application-dependent.

Chapter 12: Groth16: The Pairing-Based Optimal

In 2016, when Zcash was preparing to launch, they faced a practical problem. Blockchain transactions are expensive. Every byte costs money. The existing SNARKs (Pinocchio and its descendants) required proofs of nearly 300 bytes. It was workable, but clunky.

Then Jens Groth published a paper that seemed to violate the laws of physics. He shaved the proof down to 128 bytes on BN254. To demonstrate just how small this was, developers realized they could fit an entire zero-knowledge proof, verifying a computation of millions of steps, into a single tweet:

```
[Proof: 0x1a2b3c...] #Zcash
```

This was the theoretical minimum. Groth proved mathematically that for pairing-based systems, you literally cannot get smaller than 3 group elements. He had found the floor.

The paper, "On the Size of Pairing-based Non-interactive Arguments," became the most deployed SNARK in history. When Zcash launched its Sapling upgrade in 2018, it used Groth16. When Tornado Cash and dozens of other privacy applications needed succinct proofs, they used Groth16. The answer to "what's the smallest possible proof?" turned out to be the answer the entire field needed.

The SNARKs we've studied follow a common pattern: construct an IOP, compile it with a polynomial commitment scheme, apply Fiat-Shamir. This modular approach yields flexible systems (swap the PCS, change the trust assumptions) but leaves efficiency on the table.

Groth16 takes a different path. Rather than instantiating a generic framework, it was designed from first principles to minimize proof size. The layers are fused: optimized as a unit rather than composed as modules. Chapter 8 introduced QAP as one approach to arithmetization; here we develop it fully.

This optimality comes with constraints. The trusted setup is circuit-specific: change a single gate and you need a new ceremony. The prover cannot be made faster than $O(n \log n)$ without giving up something else. Zero-knowledge requires careful blinding woven into the protocol's fabric rather than layered on top.

From R1CS to Polynomial Identity

Chapter 8 introduced R1CS: the prover demonstrates knowledge of a witness vector Z satisfying

$$(A \cdot Z) \circ (B \cdot Z) = C \cdot Z$$

where A, B, C are matrices encoding the circuit and $\circ$ denotes the Hadamard (element-wise) product. Each row enforces one constraint of the form $(a \cdot Z)(b \cdot Z) = c \cdot Z$.

Groth16's first move is to transform this system of m constraints into a single polynomial identity.

The QAP Transformation

Fix a set of m distinct evaluation points $\omega_1, \ldots, \omega_m$ in the field $\mathbb{F}$. For each column j of the matrices, define polynomials $A_j(X), B_j(X), C_j(X)$ by Lagrange interpolation:

$$A_j(\omega_i) = A_{ij}, \quad B_j(\omega_i) = B_{ij}, \quad C_j(\omega_i) = C_{ij}$$

These are the **basis polynomials**: one for each wire in the circuit. They encode the circuit's structure: which wires participate in which constraints, with what coefficients.

Given witness $Z = (z_0, z_1, \ldots, z_{n-1})$, form the **witness polynomials**:

$$A(X) = \sum_{j=0}^{n-1} z_j \cdot A_j(X), \quad B(X) = \sum_{j=0}^{n-1} z_j \cdot B_j(X), \quad C(X) = \sum_{j=0}^{n-1} z_j \cdot C_j(X)$$

The construction ensures that at each evaluation point ω_i, the witness polynomial $A(\omega_i)$ equals the dot product $A_i \cdot Z$: exactly the value appearing in the i-th constraint. The polynomial encapsulates all constraints simultaneously.

The R1CS Condition Becomes a Polynomial Vanishing Condition

The R1CS is satisfied if and only if:

$$A(\omega_i) \cdot B(\omega_i) - C(\omega_i) = 0 \quad \text{for all } i \in \{1, \dots, m\}$$

This says the polynomial $P(X) = A(X) \cdot B(X) - C(X)$ vanishes at every ω_i. By the factor theorem, $P(X)$ must be divisible by the **vanishing polynomial**:

$$Z_H(X) = \prod_{i=1}^{m} (X - \omega_i)$$

The R1CS is satisfied if and only if there exists a polynomial $H(X)$, the **quotient** or **cofactor**, such that:

$$A(X) \cdot B(X) - C(X) = H(X) \cdot Z_H(X)$$

This is the **QAP (Quadratic Arithmetic Program) identity**. It compresses m constraint checks into one polynomial divisibility claim.

Worked Example: Continuing $x^3 + x + 5 = 35$

From Chapter 8, we have 5 constraints encoding the circuit: $v_1 = x \cdot x$, $v_2 = v_1 \cdot x$, $v_3 = v_2 + x$, $v_4 = v_3 + 5$, and output $= v_4$. This gives 7 witness positions. Let the evaluation points be $\{1, 2, 3, 4, 5\}$.

The witness is $Z = (1, 35, 3, 9, 27, 30, 35)$ representing $(1, \text{output}, x, x^2, x^3, x^3 + x, x^3 + x + 5)$.

For the second column (corresponding to variable x), the column vector in A is $(1, 0, 1, 0, 0)$, representing that x appears in constraints 1 and 3. The basis polynomial $A_2(X)$ interpolates through points $(1, 1), (2, 0), (3, 1), (4, 0), (5, 0)$:

$$A_2(X) = 1 \cdot L_1(X) + 1 \cdot L_3(X)$$

where $L_i(X)$ is the i-th Lagrange basis polynomial (recall from Chapter 2: $L_i(X) = \prod_{j \neq i} \frac{X - j}{i - j}$, satisfying $L_i(i) = 1$ and $L_i(j) = 0$ for $j \neq i$).

Each basis polynomial $A_j(X)$, $B_j(X)$, $C_j(X)$ has degree at most $m - 1 = 4$. Once we compute all of them, the witness polynomials are:

$$A(X) = \sum_{j=0}^{6} Z_j \cdot A_j(X) = 1 \cdot A_0(X) + 35 \cdot A_1(X) + 3 \cdot A_2(X) + \cdots$$

and similarly for $B(X)$ and $C(X)$. Each witness polynomial has degree at most $m - 1 = 4$.

The polynomial $P(X) = A(X) \cdot B(X) - C(X)$ has degree at most $2(m - 1) = 8$. Since the R1CS is satisfied, $P(X)$ vanishes at all five evaluation points $\{1, 2, 3, 4, 5\}$, so the vanishing polynomial $Z_H(X) = (X-1)(X-2)(X-3)(X-4)(X-5)$ divides $P(X)$. The quotient $H(X) = P(X)/Z_H(X)$ has degree $2(m-1) - m = m - 2 = 3$.

In practice, the prover computes $H(X)$ via polynomial division: evaluate $P(X)$ and $Z_H(X)$ at enough points, divide pointwise, then interpolate. FFT-based methods make this efficient.

The Core Protocol Idea

Verifying the QAP identity directly requires evaluating polynomials of degree $O(m)$, far too expensive for succinctness. The Schwartz-Zippel approach suggests evaluating at a random point τ: if $A(\tau) \cdot B(\tau) - C(\tau) = H(\tau) \cdot Z_H(\tau)$, then the identity holds with overwhelming probability.

But the witness polynomials encode the secret witness. We cannot simply send $A(\tau)$ to the verifier.

Groth16 solves this with three ideas working in concert:

1. **Homomorphic hiding**: Evaluate in the exponent. Send $g^{A(\tau)}$ instead of $A(\tau)$.
2. **Pairing verification**: Check multiplication via bilinear pairing. The equation $e(g^a, g^b) = e(g, g)^{ab}$ lets the verifier check multiplicative relations on hidden values.
3. **Structured randomness**: Embed the check into the trusted setup. The verifier never sees τ; they receive encoded values that enable verification without knowing the secret.

Linear PCPs: The Abstraction

Groth16 is best understood through the lens of **Linear PCPs**, introduced in Chapter 1. Recall: in a standard PCP, the verifier queries specific positions of a proof string. In a Linear PCP, the "proof" is a linear function $\pi : \mathbb{F}^k \to \mathbb{F}$, and the verifier can only ask for linear combinations $\pi(q) = \sum_i q_i \cdot \pi_i$ for chosen query vectors q.

This restriction enables a clever trick: if the queries are encrypted as g^q, the prover can compute $g^{\pi(q)}$ homomorphically, without ever learning q itself.

Groth16's trusted setup embeds carefully chosen query vectors into group elements. The prover computes responses using only scalar multiplication: linear operations on the encrypted queries. The verifier checks a quadratic relation using a single pairing equation.

This is why the proof has exactly three elements. Verification is a single pairing equation of the form $e(A, B) = e(\cdot, \cdot) \cdot e(\cdot, \cdot)$. Pairings take one element from $\mathbb{G}_1$ and one from $\mathbb{G}_2$, so the proof needs elements in both source groups: two in $\mathbb{G}_1$ (conventionally called A and C) and one in $\mathbb{G}_2$ (called B).

The Trusted Setup

Groth16 requires a **Structured Reference String (SRS)** generated by a trusted ceremony. The ceremony has two phases with fundamentally different properties.

Phase 1: Powers of Tau (Universal)

A secret random value $\tau \in \mathbb{F}^*$ is chosen. The ceremony outputs encrypted powers:

$$\{g_1, g_1^{\tau}, g_1^{\tau^2}, \ldots, g_1^{\tau^d}\} \quad \text{and} \quad \{g_2, g_2^{\tau}, g_2^{\tau^2}, \ldots, g_2^{\tau^d}\}$$

where d is large enough to support circuits up to a certain size.

This phase is **universal**: the same Powers of Tau can be used for any circuit within the size bound. Public ceremonies like "Perpetual Powers of Tau" provide reusable parameters. The MPC ceremony structure (1-of-N trust model, chained contributions) was covered in Chapter 9.

Phase 2: Circuit-Specific Secrets

Phase 2 generates additional secrets $\alpha, \beta, \gamma, \delta \in \mathbb{F}^*$ that are specific to the circuit being proven. Their roles will become clear when we see the verification equation; for now, here's the intuition:

α and β (Cross-term cancellation): When the prover constructs their proof elements, the verification equation produces "cross-terms" like $\alpha \cdot B(\tau)$. The α, β blinding ensures these terms cancel correctly without revealing the witness.

γ (Public input binding): Separates public from private inputs in the verification equation. The verifier computes a commitment to the public inputs and checks it against the γ-scaled portion of the SRS.

δ **(Private witness binding)**: Forces the prover to use consistent values across the A, B, and C polynomials. Without δ, the prover could use different witnesses for different polynomials (a completeness attack).

Why Phase 2 Cannot Be Universal

The Phase 2 parameters are not generic encrypted powers; they are **circuit-specific combinations** like:

$$g_1^{\frac{\beta \cdot A_j(\tau) + \alpha \cdot B_j(\tau) + C_j(\tau)}{\delta}}$$

These encode the basis polynomials A_j, B_j, C_j directly. Change the circuit, change the basis polynomials, and these elements no longer make cryptographic sense.

At a deeper level, computing these elements requires knowing $\alpha, \beta, \gamma, \delta$ in the clear. After the ceremony, these secrets are destroyed. They cannot be recovered to compute new circuit-specific values.

This is Groth16's central tradeoff. The circuit-specific encoding enables the minimal proof size. It also mandates a new ceremony for every circuit.

Protocol Specification

With setup complete, we specify the prover and verifier algorithms. We first present the **soundness core** without zero-knowledge, then show how randomization achieves privacy.

Common Reference String

The **Proving Key** pk contains:

- Encrypted powers: $\{g_1^{\tau^i}\}, \{g_2^{\tau^i}\}$
- Blinding elements: $g_1^{\alpha}, g_1^{\beta}, g_2^{\beta}, g_1^{\delta}, g_2^{\delta}$
- Basis polynomial commitments: $\{g_1^{A_j(\tau)}\}, \{g_1^{B_j(\tau)}\}, \{g_2^{B_j(\tau)}\}$
- Consistency check elements for private inputs:

$$\left\{ g_1^{\frac{\beta \cdot A_j(\tau) + \alpha \cdot B_j(\tau) + C_j(\tau)}{\delta}} \right\}_{j \in \text{private}}$$

- Quotient polynomial support: $\{g_1^{\tau^i \cdot Z_H(\tau)/\delta}\}$

The **Verification Key** vk contains:

- Pairing elements: $g_1^{\alpha}, g_2^{\beta}, g_2^{\gamma}, g_2^{\delta}$
- Public input consistency elements:

$$\left\{ g_1^{\frac{\beta \cdot A_j(\tau) + \alpha \cdot B_j(\tau) + C_j(\tau)}{\gamma}} \right\}_{j \in \text{public}}$$

Prover Algorithm (Soundness Core)

Given witness $Z = (1, \text{io}, W)$ where io are public inputs and W is the private witness:

1. **Compute witness polynomials**: Form $A(X), B(X), C(X)$ from the witness.
2. **Compute quotient**: Calculate $H(X) = \frac{A(X) \cdot B(X) - C(X)}{Z_H(X)}$.
3. **Construct proof elements** (without zero-knowledge):

$$\pi_A = g_1^{\alpha + A(\tau)}$$

$$\pi_B = g_2^{\beta + B(\tau)}$$

$$\pi_C = g_1^{\frac{\sum_{j \in \text{priv}} z_j (\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau))}{\delta} + \frac{H(\tau) \cdot Z_H(\tau)}{\delta}}$$

The α, β terms enforce that the prover uses the *same* witness in A, B, and C. Without them, a cheating prover could use inconsistent values.

Adding Zero-Knowledge

The soundness-only version above leaks information: given multiple proofs for related statements, an adversary might learn about the witness. To achieve zero-knowledge, the prover adds randomization.

Sample fresh randomness: $r, s \leftarrow \mathbb{F}$.

Randomized proof elements:

$$\pi_A = g_1^{\alpha + A(\tau) + r\delta}$$

$$\pi_B = g_2^{\beta + B(\tau) + s\delta}$$

$$\pi_C = g_1^{\frac{\sum_{j \in \text{priv}} z_j(\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau))}{\delta} + \frac{H(\tau) \cdot Z_H(\tau)}{\delta} + s(\alpha + A(\tau) + r\delta) + r(\beta + B(\tau) + s\delta) - rs\delta}$$

The formula looks arbitrary, but it follows from a constraint: the verification equation must still hold. We need $e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(\mathsf{vk}_x, g_2^\gamma) \cdot e(\pi_C, g_2^\delta)$.

With blinding, $e(\pi_A, \pi_B)$ expands to (in exponent form):

$$(\alpha + A(\tau) + r\delta)(\beta + B(\tau) + s\delta)$$

This contains new cross-terms: $\alpha s\delta$, $r\beta\delta$, $A(\tau)s\delta$, $rB(\tau)\delta$, and $rs\delta^2$. These don't appear in the soundness-only version.

The term $e(\pi_C, g_2^\delta)$ contributes $\delta \cdot$ (exponent of π_C) to the equation. So π_C must contain terms that, when multiplied by δ, cancel the unwanted cross-terms. Working backwards:

- To cancel $\alpha s\delta$: include $s\alpha$ in π_C's exponent (becomes $s\alpha\delta$ after multiplying by δ)
- To cancel $A(\tau)s\delta$: include $sA(\tau)$
- To cancel $r\beta\delta$: include $r\beta$
- To cancel $rB(\tau)\delta$: include $rB(\tau)$
- To cancel $rs\delta^2$: include $rs\delta$

Grouping: $s(\alpha + A(\tau)) + r(\beta + B(\tau)) + rs\delta$. But π_A's exponent is $\alpha + A(\tau) + r\delta$, so we can write $s(\alpha + A(\tau) + r\delta) + r(\beta + B(\tau) + s\delta) - rs\delta$. The $-rs\delta$ corrects for double-counting.

The formula is not arbitrary. It is the unique solution ensuring the blinding terms cancel while the QAP check remains intact.

The prover outputs $\pi = (\pi_A, \pi_B, \pi_C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$.

Proof Size

On the BN254 curve:

- $\pi_A \in \mathbb{G}_1$: 32 bytes (compressed)
- $\pi_B \in \mathbb{G}_2$: 64 bytes (compressed)

- $\pi_C \in \mathbb{G}_1$: 32 bytes (compressed)

Total: 128 bytes.

This is the smallest proof size achieved by any pairing-based SNARK. The paper proves a lower bound: any SNARK in this model requires at least two group elements. Groth16's three elements are close to optimal.

Verifier Algorithm

The verification equation is identical for both versions. The verifier doesn't know (or care) whether zero-knowledge randomization was used. The r, s terms cancel algebraically.

Given public inputs io $= (z_0, z_1, \ldots, z_\ell)$ where $z_0 = 1$:

1. **Compute public input combination**:

$$\mathsf{vk}_x = \sum_{j=0}^{\ell} z_j \cdot (\mathsf{vk}_{IC})_j$$

   where $(\mathsf{vk}_{IC})_j = g_1^{\frac{\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau)}{\gamma}}$

2. **Check pairing equation**:

$$e(\pi_A, \pi_B) \stackrel{?}{=} e(g_1^\alpha, g_2^\beta) \cdot e(\mathsf{vk}_x, g_2^\gamma) \cdot e(\pi_C, g_2^\delta)$$

The verifier accepts if the equation holds, rejects otherwise. Note that only π_A, π_B, π_C come from the proof; the elements $g_1^\alpha, g_2^\beta, g_2^\gamma, g_2^\delta$ are part of the verification key (fixed per circuit).

Verification Cost

The verification requires:

- One multi-scalar multiplication in $\mathbb{G}_1$ (size proportional to public input count)
- Four pairing computations (or three pairings after rearrangement)

Pairings are expensive: roughly 2-3ms each on modern hardware. But the cost is independent of circuit size. A circuit with a million constraints verifies as fast as one with a hundred.

Why the Verification Equation Works

We first verify the soundness-only version (without r, s), then show how the zero-knowledge terms cancel.

The Core Check (Without Zero-Knowledge)

With the simplified proof elements $\pi_A = g_1^{\alpha + A(\tau)}$, $\pi_B = g_2^{\beta + B(\tau)}$:

$$e(\pi_A, \pi_B) = e(g_1^{\alpha + A(\tau)}, g_2^{\beta + B(\tau)})$$

Using bilinearity, the exponent in $\mathbb{G}_T$ is:

$$(\alpha + A(\tau))(\beta + B(\tau)) = \alpha\beta + \alpha B(\tau) + \beta A(\tau) + A(\tau)B(\tau)$$

On the right-hand side:

Term 1: $e(g_1^{\alpha}, g_2^{\beta})$ contributes exponent $\alpha\beta$.

Term 2: $e(\mathsf{vk}_x, g_2^{\gamma})$ contributes:

$$\sum_{j \in \text{public}} z_j \cdot (\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau))$$

after the γ cancels.

Term 3: $e(\pi_C, g_2^{\delta})$ contributes the private witness consistency check plus the quotient:

$$\sum_{j \in \text{private}} z_j \cdot (\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau)) + H(\tau) \cdot Z_H(\tau)$$

after the δ cancels.

Combining public and private terms:

$$\sum_{\text{all } j} z_j \cdot (\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau)) = \beta A(\tau) + \alpha B(\tau) + C(\tau)$$

The RHS exponent is: $\alpha\beta + \beta A(\tau) + \alpha B(\tau) + C(\tau) + H(\tau)Z_H(\tau)$

Setting LHS = RHS and canceling matching terms:

- $\alpha\beta$ cancels
- $\alpha B(\tau)$ cancels
- $\beta A(\tau)$ cancels

What remains:

$$A(\tau)B(\tau) = C(\tau) + H(\tau)Z_H(\tau)$$

This is exactly the QAP identity.

The Full Check (With Zero-Knowledge)

With the full proof elements (including r, s):

$$e(\pi_A, \pi_B) = e(g_1^{\alpha+A(\tau)+r\delta}, g_2^{\beta+B(\tau)+s\delta})$$

Using bilinearity, the exponent in $\mathbb{G}_T$ is:

$$(\alpha + A(\tau) + r\delta)(\beta + B(\tau) + s\delta)$$

Expanding:

$$= \alpha\beta + \alpha B(\tau) + \alpha s\delta + \beta A(\tau) + A(\tau)B(\tau) + A(\tau)s\delta + r\beta\delta + rB(\tau)\delta + rs\delta^2$$

This contains the desired term $A(\tau)B(\tau)$ mixed with cross-terms involving the randomness r, s.

Term 3 now contributes additional terms: $e(\pi_C, g_2^{\delta})$ includes (after the δ cancels):

$$H(\tau) \cdot Z_H(\tau) + s\alpha\delta + sA(\tau)\delta + r\beta\delta + rB(\tau)\delta + rs\delta^2$$

The RHS exponent becomes:

$$\alpha\beta + \beta A(\tau) + \alpha B(\tau) + C(\tau) + H(\tau)Z_H(\tau) + \alpha s\delta + A(\tau)s\delta + \beta r\delta + B(\tau)r\delta + rs\delta^2$$

Setting LHS = RHS and canceling:

- $\alpha\beta$ cancels
- $\alpha B(\tau)$ cancels

- $\beta A(\tau)$ cancels
- All r, s terms cancel: $\alpha s\delta$, $A(\tau)s\delta$, $r\beta\delta$, $rB(\tau)\delta$, $rs\delta^2$

What remains is unchanged:

$$A(\tau)B(\tau) = C(\tau) + H(\tau)Z_H(\tau)$$

The elaborate construction of π_C provides exactly the terms needed to cancel the zero-knowledge blinding while preserving the soundness check.

Soundness

If the QAP is not satisfied (i.e., $A(X)B(X) - C(X) \neq H(X)Z_H(X)$ as polynomials), then the difference $A(X)B(X) - C(X) - H(X)Z_H(X)$ is a non-zero polynomial. By Schwartz-Zippel, it vanishes at the random point τ with probability at most $\deg / |\mathbb{F}|$. Since τ is hidden in the SRS, a cheating prover cannot target it. Thus false proofs are rejected with overwhelming probability.

Security and the Generic Group Model

Groth16's security proof relies on the **Generic Bilinear Group Model**: an idealization where the adversary can only perform group operations without exploiting the specific structure of the underlying curve.

The Model

In this model, group elements are represented by opaque handles. The adversary can:

- Add/subtract group elements
- Check equality
- Compute pairings

The adversary cannot:

- Look inside a group element to see its discrete log
- Exploit number-theoretic structure of the curve

The SRS contains group elements encoding powers of τ and combinations involving $\alpha, \beta, \gamma, \delta$. The prover never sees these scalars directly, only their encrypted forms. To produce a valid proof, the prover must construct group elements satisfying the verification equation.

The security argument asks: what group elements can a prover actually compute? They can only form linear combinations of SRS elements (scalar multiplication and addition). The proof shows that any linear combination satisfying the verification equation must encode a valid QAP solution. There's no way to "forge" the right algebraic structure without knowing a witness, because the prover can't extract τ from g^τ or construct arbitrary polynomials evaluated at τ.

What the Model Implies

Under this model, Groth16 is **knowledge-sound**: any adversary that produces a valid proof must "know" a valid witness. More precisely, there exists an extractor that, given the adversary's state, can produce a witness.

The model also implies the proof is **zero-knowledge**: the proof reveals nothing about the witness beyond what follows from the public statement.

The Assumption's Strength

The generic group model is non-standard. Real elliptic curves have algebraic structure; real adversaries might exploit it. No attacks are known against Groth16 on standard curves, but the security proof doesn't rule out structure-dependent attacks.

This is the price of efficiency. Schemes provable under weaker assumptions (discrete log, CDH) typically have larger proofs. Groth16 achieves optimal size by assuming more.

Concrete Assumptions

At a technical level, security reduces to the following assumptions:

- **q-Strong Diffie-Hellman (q-SDH)**: Given $\{g^{\tau^i}\}_{i=0}^{q}$, it's hard to produce $(c, g^{1/(\tau+c)})$ for any c.
- **Knowledge of Exponent**: If an adversary outputs (g^a, g^{ab}), they must "know" a.

These are strong but well-studied assumptions on pairing groups.

Proof Malleability

Groth16 proofs are **malleable**: given a valid proof (π_A, π_B, π_C), the tuple $(-\pi_A, -\pi_B, \pi_C)$ is also valid for the same statement. This follows from the verification equation; negating both π_A and π_B preserves the pairing product since $e(-\pi_A, -\pi_B) = e(\pi_A, \pi_B)$.

Malleability is not forgery. This distinction is important. Malleability allows an attacker to change the *appearance* of a valid proof (flipping signs), but not the *content*. They cannot change the public inputs or the witness. It is like taking a valid check and folding it in half: it is still a valid check for the same amount, but the physical object has changed. This matters for transaction IDs (which often hash the proof), but not for the validity of the statement itself.

This matters for applications that use proofs as unique identifiers or assume proof uniqueness (e.g., preventing double-spending by rejecting duplicate proofs). Mitigations include hashing the proof into the transaction identifier, or requiring proof elements to lie in a specific half of the group.

Trusted Setup: Practical Considerations

The circuit-specific setup is Groth16's most significant operational constraint.

What "Toxic Waste" Means

The secrets $\tau, \alpha, \beta, \gamma, \delta$ must be destroyed after the ceremony. If any participant retains them:

- Knowing τ breaks binding: allows computing arbitrary polynomial evaluations
- Knowing α, β, δ allows forging proofs for false statements

The secrets are called "toxic waste" because their existence post-ceremony compromises all proofs using that SRS.

Multi-Party Ceremonies

Production deployments run MPC ceremonies with many participants. Each participant raises the current parameters to a fresh random power, then destroys their secret; the mechanism was covered in Chapter 9. The 1-of-N trust model applies: security holds if *any single participant* destroyed their contribution.

Groth16's Phase 2 requires the same ceremony structure but with circuit-specific parameters. Each circuit needs its own Phase 2, coordinated among willing participants.

Phase 2 Complexity

Phase 1 (Powers of Tau) is performed once per maximum circuit size and reused indefinitely.

Phase 2 requires:

- Computing circuit-specific elements for every wire
- MPC ceremony among willing participants
- Verification that each contribution was correct

For a circuit with n wires, Phase 2 generates $O(n)$ group elements. Large circuits require large ceremonies.

When Circuit-Specific Setup Is Acceptable

Groth16 makes sense when:

1. **The circuit is fixed**: Same computation proved repeatedly (e.g., confidential transactions)
2. **Proof size dominates costs**: On-chain verification where bytes are expensive
3. **Verification speed is critical**: Applications requiring <10ms verification
4. **Trust model is manageable**: Established communities can coordinate ceremonies

It makes less sense when:

1. **Circuits change frequently**: Development, iteration, bug fixes
2. **Many different circuits needed**: General-purpose computation
3. **No trusted community exists**: Public good infrastructure without coordination

Comparison with Universal SNARKs

Since 2016, the field has developed universal SNARKs: systems with a single trusted setup reusable across circuits.

PLONK (Chapter 13)

- **Setup**: Universal, updatable
- **Proof size**: ~400-500 bytes (with KZG)
- **Verification**: ~10ms (several pairings)
- **Prover**: Comparable to Groth16

PLONK trades 3-4x larger proofs for the ability to prove any circuit without new ceremonies.

Marlin/Sonic

These are universal SNARKs that emerged around the same time as PLONK. **Sonic** (2019) pioneered the "universal and updateable" trusted setup: a single ceremony works for any circuit up to a size

bound, and users can add their own randomness to strengthen trust. **Marlin** (2020) keeps R1CS arithmetization (like Groth16) but achieves universality through algebraic holographic proofs. Both have similar proof sizes to PLONK (~500 bytes) but different verification costs and prover trade-offs. In practice, PLONK's flexibility and ecosystem support led to wider adoption.

STARKs (Chapter 15)

- **Setup**: Transparent (no trusted setup)
- **Proof size**: ~100 KB
- **Verification**: ~10-50ms (hash-based)
- **Prover**: Faster than pairing-based systems

STARKs eliminate trust assumptions entirely but with much larger proofs.

The Trade-Off Summary

System	Setup	Proof Size	Verification	Security Model
Groth16	Circuit-specific	128 bytes	3 pairings	Generic Group
PLONK+KZG	Universal	~500 bytes	~10 pairings	q-SDH
PLONK+IPA	Transparent	~10 KB	O(n)	DLog
STARKs	Transparent	~100 KB	$O(\log^2 n)$	Hash collision

Groth16 remains optimal when proof size is the binding constraint and circuit stability justifies the setup cost.

Implementation Considerations

Curve Selection

Groth16 requires pairing-friendly curves. Common choices:

BN254 (alt_bn128):

- 254-bit prime field
- Fast pairing computation
- Ethereum precompiles at addresses 0x06, 0x07, 0x08

- ~100 bits of security (debated; some analyses suggest less)

BLS12-381:

- 381-bit prime field
- Higher security (~120 bits)
- Slower pairings
- Used by Zcash Sapling, Ethereum 2.0 BLS signatures

Prover Complexity

The prover performs:

- $O(n)$ scalar multiplications to form witness polynomials from basis polynomials
- $O(n \log n)$ operations for polynomial multiplication and division (computing $H(X)$)
- Multi-scalar multiplications (MSM) to compute proof elements

The MSM dominates for large circuits. Significant engineering effort goes into MSM optimization: Pippenger's algorithm, parallelization, GPU acceleration.

On-Chain Verification

Ethereum's precompiled contracts enable efficient Groth16 verification:

- `ecAdd` (0x06): Elliptic curve addition in $\mathbb{G}_1$
- `ecMul` (0x07): Scalar multiplication in $\mathbb{G}_1$
- `ecPairing` (0x08): Multi-pairing check

A typical Groth16 verifier contract:

1. Computes vk_x via ecMul and ecAdd for each public input
2. Calls ecPairing with four pairs: $(-\pi_A, \pi_B), (vk_\alpha, vk_\beta), (vk_x, vk_\gamma), (\pi_C, vk_\delta)$
3. Returns true if the pairing product equals 1

Gas cost: ~200,000-300,000 gas depending on public input count.

Key takeaways

1. **Optimal proof size.** Three group elements (128 bytes on BN254). Groth proved this is the theoretical minimum for pairing-based SNARKs.

2. **QAP compresses constraints.** R1CS's m constraint checks become one polynomial divisibility condition: $A(X) \cdot B(X) - C(X) = H(X) \cdot Z_H(X)$. Lagrange interpolation encodes constraint participation into basis polynomials.

3. **Pairings check multiplication on hidden values.** The verification equation $e(\pi_A, \pi_B) = ...$ checks that $A(\tau) \cdot B(\tau) = C(\tau) + H(\tau)Z_H(\tau)$ without revealing τ or the witness polynomials. Bilinearity is the mechanism.

4. **The prover is algebraically constrained.** The SRS contains group elements encoding τ^i, α, β, γ, δ in specific combinations. The prover can only form linear combinations of these. Any proof satisfying the verification equation must encode a valid QAP solution. There is no way to "forge" the algebraic structure.

5. **Circuit-specific setup.** Phase 1 (powers of tau) is universal. Phase 2 embeds the circuit's basis polynomials $A_j(\tau), B_j(\tau), C_j(\tau)$ into the SRS. Change one gate, redo Phase 2.

6. **1-of-N trust.** If any ceremony participant destroys their toxic waste, the setup is secure. This makes the trust assumption practical despite requiring a trusted setup.

7. **Zero-knowledge by algebraic design.** The blinding terms in π_C are not arbitrary. They are the unique values ensuring the $r\delta$, $s\delta$ masks cancel in the verification equation. The protocol's ZK property is woven into its algebraic structure.

8. **Generic group model.** Security relies on assuming adversaries cannot exploit the curve's number-theoretic structure. Stronger than standard assumptions, but no practical attacks are known.

Chapter 13: PLONK: Universal SNARKs and the Permutation Argument

By 2018, Groth16 had proven SNARKs worked in production. Zcash was live, proofs were 128 bytes, verification was fast. But every protocol upgrade required a new trusted setup ceremony, a multi-party computation specific to that circuit. For a project planning rapid iteration, this was a bottleneck. The cryptographic world wanted a setup you could perform once and reuse for any circuit.

Ariel Gabizon, Zachary Williamson, and Oana Ciobotaru found the path. Their insight was *permutations*: instead of encoding circuit structure directly into the setup, separate two concerns: what each gate computes (local) and how gates connect (global). The wiring could be encoded as a permutation, checked with a polynomial argument that worked identically for any circuit.

The result was PLONK (2019): **P**ermutations over **L**agrange-bases for **O**ecumenical **N**oninteractive arguments of **K**nowledge. "Oecumenical" signals universality: one ceremony suffices for all circuits up to a maximum size. Since PLONK needs only powers of tau (no circuit-specific Phase 2), the entire

setup is **updatable**: anyone can strengthen security by adding a contribution, without coordinating with previous participants.

PLONK's modularity extends to the commitment scheme. The core is a **Polynomial IOP**: an interactive protocol where the prover sends polynomials and the verifier queries evaluations. Compile it with KZG for constant-size proofs with trusted setup. Compile with FRI for larger proofs without trust assumptions. The IOP is unchanged; only the cryptographic layer differs.

The cost of universality is larger proofs (~400-500 bytes versus 128) and more verification work (~10 pairings versus 3). Whether this trade-off makes sense depends on deployment constraints: Groth16 remains preferred when proof size or verification cost is the priority; PLONK variants dominate when development velocity or custom gates matter more.

Architecture: Gates and Copy Constraints

Chapter 8 introduced PLONKish arithmetization: the universal gate equation $Q_L \cdot a + Q_R \cdot b + Q_O \cdot c + Q_M \cdot ab + Q_C = 0$ and the permutation argument for copy constraints. Here we develop the full protocol.

The key architectural distinction from R1CS: PLONK separates gate constraints (each gate satisfies a polynomial equation relating its wires) from copy constraints (wires at different positions carry equal values when the circuit's topology demands it).

This separation has consequences for extensibility. Gate logic becomes uniform: one equation for all gates. Wiring becomes explicit: a permutation argument proves all copy constraints simultaneously. Because gate definitions and wiring are independent, adding custom gates or lookup arguments doesn't require rethinking the copy constraint mechanism.

The Gate Equation

Recall from Chapter 8: every gate has three wires (a_i, b_i, c_i) and the universal gate equation

$$Q_L \cdot a + Q_R \cdot b + Q_O \cdot c + Q_M \cdot ab + Q_C = 0$$

where selectors Q_L, Q_R, Q_O, Q_M, Q_C are public constants that program each gate's operation. Addition sets $Q_L = Q_R = 1, Q_O = -1$; multiplication sets $Q_M = 1, Q_O = -1$; constant assignment sets $Q_L = 1, Q_C = -k$. Modern variants extend to more wires (5+ instead of 3) and higher-degree terms (a^5 for Poseidon S-boxes).

From Discrete Checks to Polynomial Identity

The circuit has n gates. We want to verify all n gate equations simultaneously.

Define a domain $H = \{1, \omega, \omega^2, \ldots, \omega^{n-1}\}$ where ω is a primitive n-th root of unity. The i-th gate corresponds to domain point ω^i.

Each selector has one value per gate. For Q_L, we have a vector $(Q_{L,0}, Q_{L,1}, \ldots, Q_{L,n-1})$ where $Q_{L,i}$ is the left-wire selector at gate i. Interpolation finds the unique polynomial $Q_L(X)$ of degree $< n$ passing through the points $(\omega^0, Q_{L,0}), (\omega^1, Q_{L,1}), \ldots, (\omega^{n-1}, Q_{L,n-1})$. The result: $Q_L(\omega^i) = Q_{L,i}$ for all i. We do the same for Q_R, Q_O, Q_M, Q_C, and for the witness polynomials $a(X), b(X), c(X)$ (where $a(\omega^i) = a_i$, the left input at gate i).

The witness structure differs from R1CS. In R1CS (Chapter 8), the witness is a single flattened vector $Z = (1, \text{public inputs}, \text{private inputs}, \text{intermediate values})$. Each wire has exactly one index in Z. When two constraints reference the same wire, they use the same index; wiring is implicit in the indexing scheme.

PLONK structures the witness differently: three separate vectors (a, b, c), each of length n (the number of gates). Entry a_i is gate i's left input; b_i is its right input; c_i is its output. When the same value appears in multiple positions (say, a variable feeding two different gates) it occupies multiple slots in these vectors. This has a direct consequence: PLONK needs explicit "copy constraints" to enforce that slots holding the same logical wire actually contain the same value. We'll see how this works shortly.

To make this concrete, consider $y = (x + z) \cdot z$ with $x = 3$, $z = 2$, so $y = 10$.

R1CS representation (2 constraints, 5 wires):

Witness vector: $Z = (1, x, z, v_1, y) = (1, 3, 2, 5, 10)$ where $v_1 = x + z$.

$$A = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(Columns correspond to x, z, v_1, y; we omit the constant column for brevity.)

Row 1:

$(1 \cdot x + 1 \cdot z) \times (1) = v_1$ checks $x + z = v_1$.

Row 2:

$(1 \cdot z) \times (1 \cdot v_1) = y$ checks $z \cdot v_1 = y$.

The matrices encode *which wires participate in which constraints*. Wire z (column 2) appears in both rows; the matrix structure encodes this sharing.

PLONK representation (2 gates):

Gate	a	b	c	Q_L	Q_R	Q_O	Q_M	Q_C
1	3	2	5	1	1	-1	0	0
2	5	2	10	0	0	-1	1	0

Witness vectors: $a = (3, 5), b = (2, 2), c = (5, 10)$.

Gate 1:

$1 \cdot 3 + 1 \cdot 2 + (-1) \cdot 5 + 0 + 0 = 0$ ✓ (addition)

Gate 2:

$0 + 0 + (-1) \cdot 10 + 1 \cdot 5 \cdot 2 + 0 = 0$ ✓ (multiplication)

Notice: $z = 2$ appears twice (b_1 and b_2), and $v_1 = 5$ appears twice (c_1 and a_2). The gate equations don't enforce $b_1 = b_2$ or $c_1 = a_2$; a cheating prover could use different values. Copy constraints will enforce these equalities.

The structural difference: R1CS matrices *select* from a shared witness vector (same wire, same column, automatic equality). PLONK has vectors where each gate slot is independent (same value, different slots, explicit copy constraints needed).

How does this compare to QAP (Chapter 12)? In QAP, each wire j gets basis polynomials $A_j(X), B_j(X), C_j(X)$ encoding how that wire participates across all constraints. The witness appears as coefficients weighting these basis polynomials: $A(X) = \sum_j z_j A_j(X)$. The basis polynomials encode the circuit structure.

PLONK separates these concerns differently:

- **Selector polynomials** (Q_L, Q_R, Q_O, Q_M, Q_C): Define the circuit. Fixed once the circuit is designed. Different circuits have different selectors.
- **Witness polynomials** (a, b, c): Computed fresh by the prover for each proof. Different inputs produce different witness values, interpolated into different polynomials.

Circuit structure lives in the selector polynomials, which are ordinary polynomials, not special objects requiring circuit-specific setup. This separation is what enables universality: the same trusted setup works for any circuit, because it doesn't need to "know" about selectors in advance.

With all these polynomials defined, the per-gate equation $Q_L \cdot a + Q_R \cdot b + Q_O \cdot c + Q_M \cdot ab + Q_C = 0$ becomes a polynomial identity:

$$Q_L(X) \cdot a(X) + Q_R(X) \cdot b(X) + Q_O(X) \cdot c(X) + Q_M(X) \cdot a(X) \cdot b(X) + Q_C(X) = 0$$

for all $X \in H$.

If this holds on H, the vanishing polynomial $Z_H(X) = X^n - 1$ divides the left side. There exists quotient $t(X)$ with:

$$Q_L(X)a(X) + Q_R(X)b(X) + Q_O(X)c(X) + Q_M(X)a(X)b(X) + Q_C(X) = Z_H(X) \cdot t(X)$$

The prover demonstrates this divisibility: a single polynomial identity encoding all gate constraints.

The Copy Constraint Problem

Gate equations ensure internal consistency: the output of each gate equals the specified function of its inputs. They say nothing about how gates connect.

Consider a circuit computing $y = (x + z) \cdot z$:

- Gate 1: Addition, output $c_1 = a_1 + b_1$
- Gate 2: Multiplication, output $c_2 = a_2 \cdot b_2$

The wiring requires $c_1 = a_2$ (Gate 1's output feeds Gate 2's left input) and $b_1 = b_2$ (variable z feeds both gates).

Because PLONK's witness consists of three separate vectors (a, b, c), nothing in the gate equation relates c_1 to a_2; they're independent entries. A cheating prover could satisfy all gate equations with disconnected, inconsistent values. The circuit would "verify" despite computing garbage.

Copy constraints are the explicit assertions: wire i equals wire j. The challenge is proving all copy constraints efficiently (potentially thousands of equality assertions) without enumerating them individually.

The name "copy constraint" is slightly misleading. We aren't copying data from one location to another. We are enforcing *equality*: two wire slots that represent the same logical variable must contain identical values. The permutation argument detects whether slots that should hold the same value actually do.

The Permutation Argument

PLONK's central innovation is reducing all copy constraints to a single polynomial identity via a **permutation argument**, building on techniques from Bayer and Groth (Eurocrypt 2012).

From Gates to Cycles

Before diving into the mechanism, understand the key mental shift. So far, we've thought of circuits as *gates*: local computational units that take inputs and produce outputs. Copy constraints seem like *connections between gates*: wire c_1 connects to wire a_2.

The permutation argument reframes this. Instead of "connections," think of *equivalence classes*. All wires that should hold the same value belong to the same class. Within each class, the wires form a *cycle* under a permutation: $c_1 \to a_2 \to c_1$ (a 2-cycle), or longer chains like $a_1 \to b_3 \to c_5 \to a_1$ (a 3-cycle). Wires with no copy constraints form trivial 1-cycles (fixed points).

If we traverse each cycle, do all the values match? This shift from "gates and wires" to "values and cycles" is what makes efficient verification possible. We're not checking connections one by one, but verifying that the entire wiring topology is consistent in one algebraic test.

Representing Wiring as a Permutation

The circuit's wiring defines a permutation σ on wire slots. If two wires must hold the same value, σ maps one to the other (and vice versa, forming a cycle). Unconnected wires map to themselves: $\sigma(w) = w$.

All copy constraints hold if and only if every wire's value equals the value at the position σ maps it to:

$$\text{value}(w) = \text{value}(\sigma(w)) \quad \forall w$$

Example: For our circuit $y = (x + z) \cdot z$ with 2 gates, label the 6 wire slots as $a_1, b_1, c_1, a_2, b_2, c_2$. The copy constraints are $c_1 = a_2$ (output of gate 1 feeds gate 2) and $b_1 = b_2$ (variable z used twice). The permutation σ encodes this: $\sigma(c_1) = a_2$, $\sigma(a_2) = c_1$ (a 2-cycle), and $\sigma(b_1) = b_2$, $\sigma(b_2) = b_1$ (another 2-cycle). Wires a_1 and c_2 aren't copied anywhere, so $\sigma(a_1) = a_1$ and $\sigma(c_2) = c_2$ (fixed points).

The Grand Product Check

How do we verify this equality-under-permutation efficiently?

For a circuit with n gates, there are $3n$ wire slots (each gate has wires a, b, c). Consider two multisets: the wire values $\{v_1, v_2, \ldots, v_{3n}\}$ and the same values permuted according to σ. If copy constraints hold, these multisets are identical; they contain the same elements, just in different order.

A naive approach checks whether the products match:

$$\prod_{i=1}^{3n} v_i \stackrel{?}{=} \prod_{i=1}^{3n} v_{\sigma(i)}$$

This fails: $\{1, 6\}$ and $\{2, 3\}$ have equal products but differ. Adding a random challenge γ fixes this:

$$\prod_{i=1}^{3n} (v_i + \gamma) = \prod_{i=1}^{3n} (v_{\sigma(i)} + \gamma)$$

Why is this sound? If the multisets differ (some value appears with different multiplicities), then the polynomials $\prod_{i=1}^{3n}(X + v_i)$ and $\prod_{i=1}^{3n}(X + v_{\sigma(i)})$ are distinct. By Schwartz-Zippel, distinct degree-$3n$ polynomials agree on at most $3n$ points, so a random γ satisfies the equality with probability at most $3n/|\mathbb{F}|$ (negligible for cryptographic fields).

Binding Values to Locations

The multiset check has a flaw. A cheating prover could satisfy copy constraints on some wires by *violating* them on others, as long as they swap equal amounts. The overall multiset remains unchanged even though specific equalities fail.

Example: Circuit requires $c_1 = a_2$. Honest values: $c_1 = 5$, $a_2 = 5$. Cheating prover sets $c_1 = 5$, $a_2 = 99$, but compensates by swapping some other wire that should be 99 to 5. The multiset of all values is preserved.

The fix: bind each value to its **location** using a second challenge β:

$$\text{randomized value} = v_i + \beta \cdot \text{id}_i + \gamma$$

Each wire slot gets a unique identity id:

- Gate i's left wire: $\text{id}(a_i) = \omega^i$
- Gate i's right wire: $\text{id}(b_i) = k_1\omega^i$
- Gate i's output wire: $\text{id}(c_i) = k_2\omega^i$

where k_1, k_2 are distinct constants separating the three wire columns.

The grand product check becomes:

$$\prod_{w \in \text{wires}} (\text{value}(w) + \beta \cdot \text{id}(w) + \gamma) = \prod_{w \in \text{wires}} (\text{value}(w) + \beta \cdot \sigma(\text{id}(w)) + \gamma)$$

The left side combines each wire's value with its own identity. The right side combines each wire's value with its *permuted* identity.

To see why this works, consider two wires that should be equal: c_1 (output of gate 1, identity $k_2\omega^1$) and a_2 (left input of gate 2, identity ω^2), both holding value v. The permutation swaps their identities: $\sigma(k_2\omega^1) = \omega^2$, $\sigma(\omega^2) = k_2\omega^1$.

Left side:

$$(v + \beta \cdot k_2\omega^1 + \gamma)(v + \beta \cdot \omega^2 + \gamma)$$

Right side (using $\sigma(k_2\omega^1) = \omega^2$ and $\sigma(\omega^2) = k_2\omega^1$):

$$(v + \beta \cdot \sigma(k_2\omega^1) + \gamma)(v + \beta \cdot \sigma(\omega^2) + \gamma) = (v + \beta \cdot \omega^2 + \gamma)(v + \beta \cdot k_2\omega^1 + \gamma)$$

Same factors, just reordered, so the products match.

Now suppose a cheating prover violates the copy constraint by putting value v at c_1 but value $v' \neq v$ at a_2. The left side becomes:

$$(v + \beta \cdot k_2\omega^1 + \gamma)(v' + \beta \cdot \omega^2 + \gamma)$$

The right side becomes:

$$(v + \beta \cdot \omega^2 + \gamma)(v' + \beta \cdot k_2\omega^1 + \gamma)$$

These are different factors, so the products don't match. The β term tags each value with its location, so the check detects when two positions that should hold equal values actually don't.

If $c_1 = a_2$ (copy constraint holds), the term for c_1 on the right equals the term for a_2 on the left; they cancel in the product. If $c_1 \neq a_2$, no cancellation occurs; the products differ.

The Accumulator Polynomial

Computing a product over $3n$ terms naively requires $O(n)$ work per verification query, which is not succinct. PLONK encodes the product as a polynomial.

The **accumulator polynomial** $Z(X)$ computes a running product across all gates. It starts at 1, and at each gate multiplies in a ratio: numerator terms use the wire's own identity, denominator terms use the permuted identity. If all copy constraints hold, numerators and denominators cancel across the full circuit, and the accumulator returns to 1.

Define $Z(X)$ recursively:

Initialization: $Z(\omega) = 1$

Recursion: For domain points ω^i:

$$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(a_i + \beta\omega^i + \gamma)(b_i + \beta k_1\omega^i + \gamma)(c_i + \beta k_2\omega^i + \gamma)}{(a_i + \beta S_{\sigma_1}(\omega^i) + \gamma)(b_i + \beta S_{\sigma_2}(\omega^i) + \gamma)(c_i + \beta S_{\sigma_3}(\omega^i) + \gamma)}$$

The **permutation polynomials** $S_{\sigma_1}, S_{\sigma_2}, S_{\sigma_3}$ encode where σ maps each wire's identity. For each gate i:

- $S_{\sigma_1}(\omega^i) = \sigma(\omega^i)$: where the left wire of gate i maps to
- $S_{\sigma_2}(\omega^i) = \sigma(k_1\omega^i)$: where the right wire of gate i maps to
- $S_{\sigma_3}(\omega^i) = \sigma(k_2\omega^i)$: where the output wire of gate i maps to

If wire c_1 (identity $k_2\omega^1$) connects to wire a_2 (identity ω^2), then $S_{\sigma_3}(\omega^1) = \omega^2$. Unconnected wires map to themselves: if a_1 has no copy constraint, $S_{\sigma_1}(\omega^1) = \omega^1$.

The permutation constraints:

1. **Initialization**: $Z(\omega) = 1$

   We need this constraint to hold only at the first domain point, not everywhere. Recall from Chapter 5 that $L_1(X)$ is the Lagrange basis polynomial that equals 1 at ω and 0 at all other roots of unity. Multiplying by $L_1(X)$ "activates" the constraint only where we want it:

   $$(Z(X) - 1) \cdot L_1(X) = 0$$

   At $X = \omega$: $(Z(\omega) - 1) \cdot 1 = 0$, so $Z(\omega) = 1$ is enforced. At other $X = \omega^i$: $(Z(\omega^i) - 1) \cdot 0 = 0$, satisfied regardless of $Z(\omega^i)$.

2. **Recursion**: The step-by-step product relation holds across the domain.

   At each gate i, the accumulator must satisfy:

   $$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(a_i + \beta\omega^i + \gamma)(b_i + \beta k_1\omega^i + \gamma)(c_i + \beta k_2\omega^i + \gamma)}{(a_i + \beta S_{\sigma_1}(\omega^i) + \gamma)(b_i + \beta S_{\sigma_2}(\omega^i) + \gamma)(c_i + \beta S_{\sigma_3}(\omega^i) + \gamma)}$$

   As a polynomial identity, this becomes:

$$Z(X\omega) \cdot (\text{denominator terms}) = Z(X) \cdot (\text{numerator terms})$$

Evaluating at $X = \omega^i$ gives the recurrence: $Z(X\omega)$ evaluated at ω^i equals $Z(\omega^{i+1})$.

Both constraints, like the gate constraint, reduce to divisibility by $Z_H(X)$.

Worked Example: The Permutation Argument in Action

The abstraction clarifies; the concrete convinces. Let's trace through the permutation argument on a minimal circuit: proving $z = (x + y) \cdot y$ for inputs $x = 2, y = 3$.

The Circuit

Gate 1 (addition): $c_1 = a_1 + b_1$ **Gate 2** (multiplication): $c_2 = a_2 \cdot b_2$

Witness assignment (for $x = 2, y = 3, z = 15$):

- Gate 1: $a_1 = 2, b_1 = 3, c_1 = 5$
- Gate 2: $a_2 = 5, b_2 = 3, c_2 = 15$

Copy constraints:

- $c_1 = a_2$ (the intermediate value 5 feeds from Gate 1's output to Gate 2's left input)
- $b_1 = b_2$ (the input $y = 3$ is used in both gates)

Wire Identities

With domain $H = \{1, \omega\}$ (two gates) and constants k_1, k_2:

Wire	Identity	Value
a_1	1	2
b_1	k_1	3
c_1	k_2	5
a_2	ω	5
b_2	$k_1\omega$	3
c_2	$k_2\omega$	15

The Permutation σ

The wiring groups wire identities into cycles:

Cycle 1 (the y input): $b_1 \leftrightarrow b_2$

$$\sigma(k_1) = k_1\omega, \quad \sigma(k_1\omega) = k_1$$

Cycle 2 (the intermediate value): $c_1 \leftrightarrow a_2$

$$\sigma(k_2) = \omega, \quad \sigma(\omega) = k_2$$

Fixed points (unconnected wires):

$$\sigma(1) = 1, \quad \sigma(k_2\omega) = k_2\omega$$

Permutation Polynomials

The polynomials $S_{\sigma_1}(X)$, $S_{\sigma_2}(X)$, $S_{\sigma_3}(X)$ encode σ for each wire column.

$S_{\sigma_1}(X)$ (the a wires):

- $S_{\sigma_1}(1) = \sigma(1) = 1$ (wire a_1 is a fixed point)
- $S_{\sigma_1}(\omega) = \sigma(\omega) = k_2$ (wire a_2 connects to c_1)

$S_{\sigma_2}(X)$ (the b wires):

- $S_{\sigma_2}(1) = \sigma(k_1) = k_1\omega$ (wire b_1 connects to b_2)
- $S_{\sigma_2}(\omega) = \sigma(k_1\omega) = k_1$ (wire b_2 connects to b_1)

$S_{\sigma_3}(X)$ (the c wires):

- $S_{\sigma_3}(1) = \sigma(k_2) = \omega$ (wire c_1 connects to a_2)
- $S_{\sigma_3}(\omega) = \sigma(k_2\omega) = k_2\omega$ (wire c_2 is a fixed point)

These evaluations uniquely determine the permutation polynomials (degree at most 1 over a domain of size 2).

The Accumulator Trace

Let random challenges be β and γ. The accumulator $Z(X)$ computes a running product.

Initialization: $Z(1) = 1$

Step at $X = 1$ (processing Gate 1):

$$Z(\omega) = Z(1) \cdot \frac{(a_1 + \beta \cdot 1 + \gamma)(b_1 + \beta \cdot k_1 + \gamma)(c_1 + \beta \cdot k_2 + \gamma)}{(a_1 + \beta \cdot S_{\sigma_1}(1) + \gamma)(b_1 + \beta \cdot S_{\sigma_2}(1) + \gamma)(c_1 + \beta \cdot S_{\sigma_3}(1) + \gamma)}$$

Substituting values:

Numerator = $(2 + \beta + \gamma)(3 + \beta k_1 + \gamma)(5 + \beta k_2 + \gamma)$

Denominator = $(2 + \beta \cdot 1 + \gamma)(3 + \beta \cdot k_1\omega + \gamma)(5 + \beta \cdot \omega + \gamma)$

The a_1 term $(2 + \beta + \gamma)$ appears in both numerator and denominator; it cancels (wire a_1 is a fixed point).

The b_1 numerator term is $(3 + \beta k_1 + \gamma)$; the denominator has $(3 + \beta k_1\omega + \gamma)$.

The c_1 numerator term is $(5 + \beta k_2 + \gamma)$; the denominator has $(5 + \beta\omega + \gamma)$.

Step at $X = \omega$ (processing Gate 2):

$$Z(\omega^2) = Z(\omega) \cdot \frac{(a_2 + \beta\omega + \gamma)(b_2 + \beta k_1\omega + \gamma)(c_2 + \beta k_2\omega + \gamma)}{(a_2 + \beta \cdot S_{\sigma_1}(\omega) + \gamma)(b_2 + \beta \cdot S_{\sigma_2}(\omega) + \gamma)(c_2 + \beta \cdot S_{\sigma_3}(\omega) + \gamma)}$$

Substituting:

Numerator = $(5 + \beta\omega + \gamma)(3 + \beta k_1\omega + \gamma)(15 + \beta k_2\omega + \gamma)$

Denominator = $(5 + \beta k_2 + \gamma)(3 + \beta k_1 + \gamma)(15 + \beta k_2\omega + \gamma)$

The $(15 + \beta k_2\omega + \gamma)$ term appears in both numerator and denominator of step 2, so it cancels immediately (wire c_2 is a fixed point).

The interesting cancellations happen across steps. Consider wire c_1 (value 5, identity k_2):

- Step 1 numerator: $(5 + \beta k_2 + \gamma)$: the value plus its own identity
- Step 2 denominator: $(5 + \beta \cdot S_{\sigma_1}(\omega) + \gamma) = (5 + \beta k_2 + \gamma)$

Why does step 2's denominator have k_2? Because $S_{\sigma_1}(\omega)$ asks "where does wire a_2 map under σ?" Since $c_1 = a_2$ is a copy constraint, σ maps a_2's identity (ω) to c_1's identity (k_2). So $S_{\sigma_1}(\omega) = k_2$.

Similarly for wire $b_1 = b_2$ (value 3):

- Step 1 numerator: $(3 + \beta k_1 + \gamma)$
- Step 2 denominator: $(3 + \beta \cdot S_{\sigma_2}(\omega) + \gamma) = (3 + \beta k_1 + \gamma)$

Here $S_{\sigma_2}(\omega) = k_1$ because σ maps b_2's identity ($k_1\omega$) to b_1's identity (k_1).

The converse cancellations work the same way: step 1's denominator terms match step 2's numerator terms because the permutation is symmetric (if σ maps $a \to b$, it also maps $b \to a$).

Every term cancels. The result: $Z(\omega^2) = 1$.

Since $\omega^2 = 1$ for $n = 2$, we have $Z(1) = 1$ as required. The accumulator returns to its starting value, confirming all copy constraints hold.

What If a Constraint Were Violated?

Suppose the prover cheats: sets $a_2 = 7$ instead of 5 (breaking $c_1 = a_2$).

The term $(5+\beta k_2+\gamma)$ from c_1 no longer matches $(7+\beta k_2+\gamma)$ from the fraudulent a_2. No cancellation occurs. The accumulator ends at a value $\neq 1$, and the constraint $(Z(X) - 1) \cdot L_n(X) = 0$ fails.

The random challenges β, γ ensure this failure is detectable with overwhelming probability.

The Full Protocol

The core ideas are now in place: the gate equation checks local correctness, the permutation argument enforces wiring via a grand product, and the accumulator polynomial encodes this product for efficient verification. This section specifies the complete protocol with KZG commitments. It can be skipped on first reading without losing the conceptual thread.

Preprocessed Data (Circuit-Specific)

Fixed at circuit compilation:

- Selector polynomial commitments: $[Q_L]_1, [Q_R]_1, [Q_O]_1, [Q_M]_1, [Q_C]_1$
- Permutation polynomial commitments: $[S_{\sigma_1}]_1, [S_{\sigma_2}]_1, [S_{\sigma_3}]_1$

Common Reference String (Universal)

The SRS, shared across all circuits up to size n:

- $\{[1]_1, [\tau]_1, [\tau^2]_1, \ldots, [\tau^{n+5}]_1\}$
- $[\tau]_2$

The prover needs the full $\mathbb{G}_1$ sequence. The verifier needs only $[\tau]_2$, an asymmetry that enables efficient verification.

Round 1: Commit to Witness

The prover:

1. Computes witness polynomials $a(X), b(X), c(X)$ by interpolating wire values
2. **Blinds** each polynomial for zero-knowledge: $a(X) \leftarrow a(X) + (b_1 X + b_2) Z_H(X)$, where b_1, b_2 are random field elements
3. Commits: sends $[a]_1, [b]_1, [c]_1$

Why does blinding work? The term $(b_1 X + b_2) Z_H(X)$ is zero on H (since $Z_H(\omega^i) = 0$ for all $\omega^i \in H$), so adding it doesn't change the polynomial's values at gate positions; correctness is preserved. But outside H, this random term "scrambles" the polynomial, hiding information about the original witness values. The verifier will later query the polynomial at a random point $\zeta \notin H$; without blinding, these evaluations could leak witness information.

Round 2: Commit to Accumulator

The prover:

1. Derives challenges β, γ via Fiat-Shamir (hash of transcript including Round 1 commitments)
2. Computes accumulator polynomial $Z(X)$ from the recursive definition
3. Blinds with higher-degree term (three random scalars, since Z is checked at two points: z and $z\omega$)
4. Commits: sends $[Z]_1$

Round 3: Compute Quotient

The prover:

1. Derives challenge α via Fiat-Shamir
2. Forms the combined constraint polynomial using α for random linear combination:

$$P(X) = \text{(gate constraint)} + \alpha \cdot \text{(permutation recursion)} + \alpha^2 \cdot \text{(permutation initialization)}$$

The **gate constraint** is $Q_L(X)a(X) + Q_R(X)b(X) + Q_O(X)c(X) + Q_M(X)a(X)b(X) + Q_C(X)$, the polynomial identity from earlier that encodes all gate equations. The **permutation recursion** forces the accumulator to update correctly at each step: the polynomial form of "$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{\text{numerator}}{\text{denominator}}$" from the grand product. The **permutation initialization** is the boundary condition: the accumulator must start at 1, encoded as $(Z(X) - 1) \cdot L_1(X)$ where L_1 is the Lagrange polynomial that equals 1 at ω and 0 elsewhere.

3. Computes quotient: $t(X) = P(X)/Z_H(X)$
4. Splits $t(X)$ into lower-degree pieces for commitment (since $\deg(t) > n$)
5. Commits to quotient pieces

Round 4: Evaluate and Open

The prover:

1. Derives evaluation point ζ via Fiat-Shamir
2. Evaluates all relevant polynomials at ζ:
    - Witness: $a(\zeta), b(\zeta), c(\zeta)$
    - Accumulator: $Z(\zeta)$, and $Z(\zeta\omega)$ (the shifted evaluation)
    - Permutation: $S_{\sigma_1}(\zeta), S_{\sigma_2}(\zeta)$
3. Sends evaluations to verifier
4. Computes batched opening proofs (we explain the linearization trick in the verification section below)

Round 5: Batched Opening Proofs

The prover:

1. Derives batching challenge v via Fiat-Shamir
2. Constructs opening proof for all evaluations at ζ (batched)
3. Constructs opening proof for evaluation at $\zeta\omega$ (the shifted point)
4. Sends two KZG proofs

Verification

The verifier performs the following steps:

1. Reconstruct Challenges

From the transcript (all prover commitments), derive:

- β, γ from Round 1 commitments (for permutation argument)
- α from Round 2 commitments (for constraint aggregation)
- ζ from Round 3 commitments (evaluation point)
- v from Round 4 evaluations (batching challenge)

All challenges are deterministic functions of the transcript via Fiat-Shamir.

2. Compute the Linearization Polynomial Commitment

The combined constraint polynomial $P(X)$ contains products like $Q_M(X) \cdot a(X) \cdot b(X)$. The verifier has commitments $[Q_M]_1$, $[a]_1$, $[b]_1$ but cannot compute $[Q_M \cdot a \cdot b]_1$ from these. There is no way to multiply group elements to get a commitment to a product of polynomials.

The linearization trick solves this. Once the prover sends evaluations $a(\zeta), b(\zeta)$ as field elements, these become scalars. The verifier can compute:

$$[Q_M]_1 \cdot a(\zeta) \cdot b(\zeta)$$

This scalar multiplication is possible and gives the right contribution at point ζ. The verifier constructs the linearized commitment $[r]_1$:

- **Gate constraint**: $[Q_L]_1 \cdot a(\zeta) + [Q_R]_1 \cdot b(\zeta) + [Q_O]_1 \cdot c(\zeta) + [Q_M]_1 \cdot a(\zeta)b(\zeta) + [Q_C]_1$
- **Permutation recursion** (scaled by α): Terms involving $[Z]_1$, the permutation polynomials, and the evaluated witness values
- **Permutation initialization** (scaled by α^2): $(Z(\zeta) - 1) \cdot L_1(\zeta)$

3. Compute the Expected Evaluation

The verifier computes what $r(\zeta)$ *should* equal if the prover is honest. This involves:

- The quotient polynomial contribution: $t(\zeta) \cdot Z_H(\zeta)$
- Witness polynomial contributions at ζ

4. Batched Opening Verification

The verifier checks two batched KZG opening proofs:

Opening at ζ: All polynomials evaluated at ζ are batched using challenge v:

$$[F]_1 = [r]_1 + v[a]_1 + v^2[b]_1 + v^3[c]_1 + v^4[S_{\sigma_1}]_1 + v^5[S_{\sigma_2}]_1$$

The verifier checks that $[F]_1$ opens to the batched evaluation:

$$F(\zeta) = r(\zeta) + v \cdot a(\zeta) + v^2 \cdot b(\zeta) + ...$$

Opening at $\zeta\omega$: The accumulator's shifted evaluation:

$$e([Z]_1 - [Z(\zeta\omega)]_1, [\tau]_2) \stackrel{?}{=} e([W_{\zeta\omega}]_1, [\tau - \zeta\omega]_2)$$

where $[W_{\zeta\omega}]_1$ is the KZG opening proof for evaluation at $\zeta\omega$.

5. Pairing Check

The final verification reduces to two pairing equations (often combined into one via random linear combination):

$$e([W_\zeta]_1 + u \cdot [W_{\zeta\omega}]_1, [\tau]_2) = e(\zeta \cdot [W_\zeta]_1 + u\zeta\omega \cdot [W_{\zeta\omega}]_1 + [F]_1 - [E]_1, [1]_2)$$

where u is a random challenge for batching the two opening proofs, and $[E]_1$ is the commitment to the expected evaluations.

Verification Cost

Operation	Count
Scalar multiplications in $\mathbb{G}_1$	~15-20
Field multiplications	~30-50
Pairing computations	2

Total verification time: ~5-10ms on commodity hardware, independent of circuit size.

Proof Size Analysis

With KZG over BN254:

Element	Size	Count	Total
$\mathbb{G}_1$ commitments	32 bytes	~10	320 bytes
$\mathbb{G}_1$ opening proofs	32 bytes	2	64 bytes
Field element evaluations	32 bytes	~7	224 bytes

Total: ~600 bytes (varies with optimizations)

This is 4-5× larger than Groth16's 128 bytes. The cost buys universality: one setup ceremony, any circuit.

Why Roots of Unity?

PLONK's use of roots of unity (multiplicative subgroup of order 2^k) is not arbitrary. Three properties make them necessary:

- Polynomial operations (interpolation, multiplication, division) run in $O(n \log n)$ via FFT. Without roots of unity, these cost $O(n^2)$.
- The vanishing polynomial has a simple form: $Z_H(X) = X^n - 1$. Compact representation, efficient evaluation.
- The accumulator's recursive relation compares $Z(X)$ and $Z(X\omega)$. Multiplication by ω shifts through the domain cyclically, which encodes the step-by-step product check.

Groth16 uses an arithmetic progression $\{1, 2, \ldots, m\}$ because its prover doesn't interpolate; it computes linear combinations of precomputed basis polynomials. The FFT advantage doesn't apply.

Comparison: PLONK vs. Groth16

The preceding sections developed these architectural differences in detail. Here's a side-by-side summary:

Aspect	Groth16	PLONK
Witness role	Coefficients weighting basis polynomials	Evaluations interpolated into polynomials
Copy constraints	Implicit (R1CS matrix reuses indices)	Explicit (permutation argument)
Setup	Circuit-specific (basis polynomials in SRS)	Universal (only powers of τ)
Constraint form	$(a \cdot w)(b \cdot w) = c \cdot w$	$Q_L a + Q_R b + Q_O c + Q_M ab + Q_C = 0$
Proof size	128 bytes	~500 bytes
Verification	3 pairings	2 pairings + ~15 scalar muls
Prover work	MSM-dominated	FFT + MSM
Extensibility	Fixed	Custom gates, lookups

Custom Gates and Extensions

PLONK's gate equation generalizes naturally. Custom gates aren't exclusive to PLONKish systems. Spartan's CCS (Customizable Constraint Systems) also supports arbitrary polynomial constraints, generalizing both R1CS and PLONKish arithmetization. But PLONK variants were the first to deploy custom gates widely in production.

More Wires

Modern systems (Halo2, UltraPLONK) use 5+ wires per gate:

$$\sum_{i=1}^{5} Q_i \cdot w_i + Q_{M_{12}} w_1 w_2 + Q_{M_{34}} w_3 w_4 + \cdots = 0$$

More wires mean fewer gates for complex operations.

Higher-Degree Terms

The Poseidon hash uses x^5 in its S-box. A custom gate term $Q_{\text{pow5}} \cdot a^5$ computes this in one gate rather than five multiplications.

Non-Native Arithmetic

A major driver for custom gates is *non-native arithmetic*: computing over a field different from the proof system's native field. PLONK (with BN254) operates over a ~254-bit prime field. But many applications require arithmetic over other fields: Bitcoin uses secp256k1's scalar field, Ethereum signatures use different curve parameters, and recursive proof verification requires operating over the "inner" proof's field.

Without custom gates, non-native field multiplication requires decomposing elements into limbs, performing schoolbook multiplication with carries, and range-checking intermediate results. A single non-native multiplication can cost 50+ native gates. Custom gates can batch these operations, reducing the cost by 5-10×. This is why efficient ECDSA verification (for Ethereum account abstraction or Bitcoin bridge verification) demands sophisticated custom gate design.

Boolean Constraints

Enforcing $x \in \{0, 1\}$ requires $x(x - 1) = 0$, equivalently $x^2 - x = 0$. With selector Q_{bool}:

$$Q_{\text{bool}} \cdot (a^2 - a) = 0$$

One gate, one constraint.

Lookup Arguments

The most powerful extension. Rather than computing a function in gates, prove that (input, output) pairs appear in a precomputed table.

Example: Range check. Proving $x \in [0, 2^{16})$ via bit decomposition costs 16 gates. A lookup into a table of $\{0, 1, \ldots, 2^{16} - 1\}$ costs ~3 constraints.

Chapter 14 develops lookup arguments in detail.

UltraPLONK

"UltraPLONK" denotes PLONK variants combining custom gates and lookup arguments. These systems achieve dramatic efficiency gains for real-world circuits: composite gates encode multiple operations simultaneously (e.g., $a + b = c$ and $d \cdot e = f$ in one gate), the permutation argument extends to prove set membership in lookup tables, and Poseidon-specific gates reduce hash computation by 10-20× compared to vanilla PLONK. The architecture remains a polynomial IOP compiled with KZG (or alternatives). The IOP grows more sophisticated, but the verification structure persists.

Aztec Labs, co-founded by Zac Williamson (one of PLONK's creators), developed UltraPLONK in their Barretenberg library. Their system has since evolved to Honk, which replaces the univariate polynomial IOP with sum-check over multilinear polynomials (similar to Spartan's approach). Honk retains PLONKish arithmetization but gains the memory efficiency of sum-check (Chapter 19 explains why: sum-check's linear memory access pattern is cache-friendly, unlike FFT's butterfly shuffles). For on-chain verification, Aztec compresses Honk proofs into UltraPLONK proofs; UltraPLONK's simpler verifier (fewer selector polynomials, no multilinear machinery) reduces gas costs. Their Goblin PLONK technique further optimizes recursive proof composition by deferring expensive elliptic curve operations rather than computing them at each recursion layer.

Security Considerations

Trusted Setup

PLONK's universality doesn't eliminate trust; it redistributes it.

The SRS still encodes secret τ. If known, proofs can be forged. The advantage is logistical: one ceremony covers all circuits. Updates strengthen security without coordination.

Production deployments (Aztec, zkSync, Scroll) run multi-party ceremonies with hundreds of participants. The 1-of-N trust model, where security holds if any participant is honest, provides strong guarantees.

Soundness Assumptions

PLONK's security depends on the polynomial commitment scheme used:

- **With KZG**: Security relies on pairing-based assumptions (q-SDH, discrete log). These are well-studied but would break under quantum computers.
- **With FRI**: Security relies only on collision-resistant hashing. Fewer assumptions, and potentially quantum-resistant, but larger proofs.

Key takeaways

1. **Universal setup**: One ceremony works for all circuits up to a size bound. This comes from treating witness values as polynomial evaluations (interpolated at proving time) rather than coefficients (baked into setup).

2. **Separation of concerns**: Gate constraints check local correctness (each gate's equation holds). Copy constraints check global wiring (connected wires hold equal values). Each has its own polynomial mechanism.

3. **The permutation argument**: All copy constraints reduce to one polynomial identity. The accumulator polynomial computes a running product; if all constraints hold, it returns to 1.

4. **Roots of unity**: FFT enables $O(n \log n)$ polynomial operations. The shift structure ($Z(X)$ vs $Z(X\omega)$) encodes the accumulator's step-by-step recursion.

5. **The linearization trick**: The verifier can't compute commitments to polynomial products. Linearization uses the prover's evaluation values to turn polynomial multiplications into scalar multiplications of commitments.

6. **Proof size vs setup trade-off**: ~500 bytes (vs Groth16's 128 bytes) buys universality. Whether this trade-off makes sense depends on deployment constraints.

Chapter 14: Lookup Arguments

In 2019, ZK engineers hit a wall.

They wanted to verify standard computer programs, things like SHA-256 or ECDSA signatures, but the circuits were exploding in size. The culprit was *bit decomposition*. Operations that are trivial in silicon (bitwise XOR, range checks, comparisons) require decomposing values into individual bits, processing each bit, and reassembling. A single XOR takes roughly 30 constraints. A range check proving $x < 2^{32}$ costs 32 boolean constraints. Verifying a 64-bit CPU instruction set was like simulating a Ferrari using only wooden gears.

Ariel Gabizon and Zachary Williamson realized they didn't need to simulate the gears. They just needed to check the answer key. This realization, that you can replace *computation* with *table lookups*, broke the bottleneck. Instead of decomposing values into bits, just look up the answer in a precomputed table.

The insight built on earlier work (Bootle et al.'s 2018 "Arya" paper had explored lookup-style arguments), but Plookup made it practical by repurposing PLONK's permutation machinery. Range checks become a lookup into a table of valid values. Bitwise operations become a lookup into a table of valid input-output triples. Membership in these tables costs a few constraints, regardless of what the table encodes. The architecture shifted, and complexity moved from constraint logic to precomputed data.

The field accelerated. Haböck's **LogUp** (2022) replaced grand products with sums of logarithmic derivatives, eliminating sorting overhead and enabling cleaner multi-table arguments. Setty, Thaler, and Wahby's **Lasso** (2023) achieved prover costs scaling with lookups performed rather than table size, enabling tables of size 2^{128}, large enough to hold the evaluation table of any 64-bit instruction. The "lookup singularity" emerged: a vision of circuits that do nothing but look things up in precomputed tables.

Today, every major zkVM relies on lookups. Cairo, RISC-Zero, SP1, and Jolt prove instruction execution not by encoding CPU semantics in constraints, but by verifying that each instruction's behavior matches its entry in a precomputed table. Complexity moves from constraint logic to precomputed data.

The Lookup Problem

Chapter 13 introduced the **grand product argument** for copy constraints in PLONK. The idea: to prove that wire values at positions related by permutation σ are equal, compute $\prod_i \frac{a_i + \beta \cdot i + \gamma}{a_i + \beta \cdot \sigma(i) + \gamma}$. If the permu-

tation constraint is satisfied (values at linked positions match), this product telescopes to 1. Lookup arguments generalize this technique from equality to containment, proving not that two multisets are the same, but that one is contained in another.

The formal problem:

Given a multiset $f = \{f_1, \ldots, f_n\}$ of witness values (the "lookups") and a public multiset $t = \{t_1, \ldots, t_d\}$ (the "table"), prove $f \subseteq t$.

The name "lookup" comes from how these proofs work in practice. Imagine you're proving a circuit that computes XOR. The table t contains all valid XOR triples: $(0, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0)$. Your circuit claims $a \oplus b = c$ for some witness values. Rather than encoding XOR algebraically, you "look up" the triple (a, b, c) in the table. If it's there, the XOR is correct. The multiset f collects all the triples your circuit needs to verify; the subset claim $f \subseteq t$ says every lookup found a valid entry.

A dictionary example makes this concrete. Imagine you want to prove you spelled "Cryptography" correctly. The *arithmetic approach* would be to write down the rules of English grammar and phonetics, then derive the spelling from first principles. Slow, complex, error-prone. The *lookup approach* would be to open the Oxford English Dictionary to page 412, point to the word "Cryptography," and say "there." The lookup argument is proving that your tuple (the word you claim) exists in the set (all valid English words). You don't need to understand *why* it's valid; you just need to show it's in the book.

The Naive Approach: Product of Roots

A natural idea: two multisets are equal iff the polynomials having those elements as roots are equal. If every lookup f_i appears in the table t, we can write:

$$\prod_{i=1}^{n}(X - f_i) = \prod_{j=1}^{d}(X - t_j)^{m_j}$$

where m_j counts how many times table entry t_j appears among the lookups.

Example: Lookups $f = \{2, 2, 5\}$ into table $t = \{1, 2, 3, 4, 5\}$.

- Left side: $(X - 2)(X - 2)(X - 5) = (X - 2)^2(X - 5)$
- Right side: $(X - 1)^0(X - 2)^2(X - 3)^0(X - 4)^0(X - 5)^1 = (X - 2)^2(X - 5)$

The polynomials match because the multisets match: f contains two 2s and one 5, which is exactly what the multiplicities $m_2 = 2$, $m_5 = 1$ encode.

This identity is mathematically valid, but expensive to verify in a circuit. Computing $(X - t_j)^{m_j}$ requires the binary decomposition of each multiplicity m_j. If lookups can repeat up to n times, each multiplicity needs $\log n$ bits, blowing up the circuit inputs.

Different lookup protocols avoid this cost in different ways. **Plookup** sidesteps multiplicities entirely by using a sorted merge. **LogUp** transforms the product into a sum where multiplicities become simple coefficients rather than exponents.

Plookup

Plookup's insight is to transform the subset claim into a permutation claim. The construction involves three objects:

- f: the lookup values (what you're looking up, your witness data)
- t: the table (all valid values, public and precomputed)
- s: the sorted merge of f and t (auxiliary, constructed by prover)

The key is that s encodes *how* f fits into t. If every f_i is in t, then s is just t with duplicates inserted at the right places.

Plookup's Sorted Vector s

Define $s = \text{sort}(f \cup t)$, the concatenation of lookup values and table values, sorted.

If $f \subseteq t$, then every element of f appears somewhere in t. In the sorted vector s, elements from f "slot in" next to their matching elements from t.

For every adjacent pair (s_i, s_{i+1}) in s, either:

1. $s_i = s_{i+1}$ (a repeated value, meaning some f_j was inserted next to its matching t_k), or
2. (s_i, s_{i+1}) is also an adjacent pair in the sorted table t

If some $f_j \notin t$, then s contains a transition that doesn't exist in t, and the check fails.

Example (3-bit range check):

- Lookups: $f = \{2, 5\}$ (prover claims both are in $[0, 7]$)
- Table: $t = \{0, 1, 2, 3, 4, 5, 6, 7\}$
- Sorted: $s = \{0, 1, 2, 2, 3, 4, 5, 5, 6, 7\}$

Adjacent pairs in s: $(0,1),(1,2),(2,2),(2,3),(3,4),(4,5),(5,5),(5,6),(6,7)$

The pairs $(2,2)$ and $(5,5)$ are repeats; these correspond to the lookups. All other pairs appear as adjacent pairs in t. The subset claim holds.

If instead $f = \{2,9\}$:

- Sorted: $s = \{0,1,2,2,3,4,5,6,7,9\}$
- The pair $(7,9)$ is neither a repeat nor an adjacent pair in t
- The subset claim fails

Plookup's Grand Product Check

The adjacent-pair property translates to a polynomial identity via a grand product. The construction is clever, so let's build it step by step.

The core idea is to encode each adjacent pair (s_i, s_{i+1}) as a single field element $\gamma(1+\beta) + s_i + \beta s_{i+1}$. The term β acts as a "separator": different pairs map to different field elements (with high probability over random β). Multiplying all these pair-encodings together gives a fingerprint of the multiset of adjacent pairs.

$G(\beta,\gamma)$, the fingerprint of s's adjacent pairs:

$$G(\beta,\gamma) = \prod_{i=1}^{n+d-1} (\gamma(1+\beta) + s_i + \beta s_{i+1})$$

This is just the product of all adjacent-pair encodings in the sorted vector s.

$F(\beta,\gamma)$, the fingerprint we *expect* if $f \subseteq t$:

$$F(\beta,\gamma) = (1+\beta)^n \cdot \prod_{i=1}^{n}(\gamma + f_i) \cdot \prod_{i=1}^{d-1}(\gamma(1+\beta) + t_i + \beta t_{i+1})$$

Where does this come from? Think about what s looks like when $f \subseteq t$. The sorted merge contains the table t as a "backbone," with lookup values from f inserted as duplicates next to their matches. So the adjacent pairs in s fall into two categories:

1. **Pairs from** t: The $d-1$ consecutive pairs (t_i, t_{i+1}) from the original table. These appear in s regardless of what f contains; they're the skeleton that f gets merged into. In F, these correspond to the last product $\prod_{i=1}^{d-1}(\gamma(1+\beta) + t_i + \beta t_{i+1})$, which doesn't factorize.

2. **Repeated pairs from inserting** f: When a lookup value f_j slots into s next to its matching table entry, we get a repeated pair (f_j, f_j). The encoding of (v, v) is $\gamma(1+\beta)+v+\beta v = (\gamma+v)(1+\beta)$. This *does* factorize. So the n repeated pairs contribute $(1+\beta)^n \cdot \prod(\gamma + f_i)$ to F.

F is the fingerprint of *exactly these pairs*, the table backbone plus n valid duplicate insertions. If G (the actual fingerprint of s) equals F, then s has the right structure: no "bad" transitions like $(7, 9)$ that would appear if some $f_j \notin t$.

Let's use a 3-element table to see the algebra concretely.

- Table: $t = \{0, 1, 2\}$ (so $d = 3$)
- Lookups: $f = \{1\}$ (so $n = 1$)
- Sorted merge: $s = \{0, 1, 1, 2\}$

Computing G (fingerprint of s's adjacent pairs):

The pairs in s are: $(0, 1), (1, 1), (1, 2)$. Encode each:

$$G = (\gamma(1+\beta) + 0 + \beta \cdot 1) \cdot (\gamma(1+\beta) + 1 + \beta \cdot 1) \cdot (\gamma(1+\beta) + 1 + \beta \cdot 2)$$
$$= (\gamma(1+\beta) + \beta) \cdot (\gamma(1+\beta) + 1 + \beta) \cdot (\gamma(1+\beta) + 1 + 2\beta)$$

Computing F (expected fingerprint):

- Table pairs (t_i, t_{i+1}): $(0, 1)$ and $(1, 2)$
- Lookup duplicate: $f_1 = 1$ contributes $(\gamma + 1)(1 + \beta)$

$$F = (1+\beta)^1 \cdot (\gamma + 1) \cdot (\gamma(1+\beta) + 0 + \beta \cdot 1) \cdot (\gamma(1+\beta) + 1 + \beta \cdot 2)$$
$$= (1+\beta)(\gamma + 1) \cdot (\gamma(1+\beta) + \beta) \cdot (\gamma(1+\beta) + 1 + 2\beta)$$

Why $F = G$**?** Notice that the pair $(1, 1)$ in G encodes as $\gamma(1 + \beta) + 1 + \beta = (\gamma + 1)(1 + \beta)$. This factors! So G's middle term equals F's $(1 + \beta)(\gamma + 1)$ term. The other two terms match directly. The products are identical.

Claim (Plookup): $F(\beta, \gamma) = G(\beta, \gamma)$ if and only if $f \subseteq t$ and s is correctly formed.

Completeness: If $f \subseteq t$, then s consists of t's pairs plus repeated pairs (f_j, f_j) for each lookup. Each repeated pair encodes as $(\gamma + f_j)(1 + \beta)$, which exactly matches F's structure.

Soundness: If some $f_j \notin t$, then when sorted into s, f_j creates an adjacent pair (a, f_j) or (f_j, b) where neither a nor b equals f_j. This "bad transition" doesn't appear in F's table backbone, and can't factor as $(1 + \beta)(\gamma + f_j)$ either. For random β, γ, the probability that $F = G$ despite this mismatch is at most $2(n + d)/|\mathbb{F}|$ by Schwartz-Zippel (the products have total degree at most $2(n + d)$ in (β, γ)).

The following implementation computes F and G for the 3-bit range check example above:

```
def encode_pair(a, b, beta, gamma):
    """Encode adjacent pair (a, b) as a field element."""
    return gamma * (1 + beta) + a + beta * b

def plookup_check(lookups, table, beta=2, gamma=5):
    """Verify lookups subset of table via Plookup grand product."""
    s = sorted(lookups + table)

    # G: fingerprint of s's adjacent pairs
    G = 1
    for i in range(len(s) - 1):
        G *= encode_pair(s[i], s[i+1], beta, gamma)

    # F: expected fingerprint = (1+beta)^n * prod(gamma + f_i) * prod(table
    ↪  pairs)
    F = (1 + beta) ** len(lookups)
    for f in lookups:
        F *= (gamma + f)
    for i in range(len(table) - 1):
        F *= encode_pair(table[i], table[i+1], beta, gamma)

    return F, G, (F == G)

# 3-bit range check: {2, 5} in [0, 7]
plookup_check([2, 5], list(range(8)))  # (563374005, 563374005, True)

# Invalid: 9 not in table
plookup_check([2, 9], list(range(8)))  # F != G, returns False
```

Integrating with PLONK

The grand product check $F = G$ is the mathematical core of Plookup (Gabizon-Williamson 2020). But to use it in a SNARK, we need to encode the check as polynomial constraints that PLONK can verify. This means:

- The table t becomes a polynomial committed during setup
- The sorted vector s becomes polynomials the prover commits to
- The $F = G$ check becomes an accumulator that the verifier checks via a single polynomial identity

Setup The table is public and fixed before any proof. Encode it as a polynomial $t(X)$ where $t(\omega^i) = t_i$ for each table entry. This polynomial is committed once and reused across all proofs; the verifier never touches the full table during verification.

The prover holds witness values $\{f_1, \ldots, f_n\}$ to look up. These are private.

Prover Computation The prover's job is to construct the sorted vector s and prove $F = G$ without revealing the witness values.

1. **Construct** s: Merge f and t, then sort. This is the (f, t)-sorted vector from the theory above.
2. **Split** s **into** h_1, h_2: The sorted vector has length $n + d$ (lookups plus table), but PLONK's evaluation domain has size matching the circuit. To fit s into the constraint system, split it into two polynomials h_1 and h_2. The constraints will check adjacent pairs *within* each half and *across* the boundary.
3. **Commit to sorted polynomials**: Send $[h_1]_1, [h_2]_1$ to the verifier.
4. **Receive challenges**: After Fiat-Shamir, obtain β, γ. These randomize the fingerprint encoding, making it infeasible for a cheating prover to forge a valid $F = G$.
5. **Build accumulator**: Construct $Z(X)$, the polynomial that computes the running F/G ratio. It starts at 1, accumulates one ratio term per domain point, and returns to 1 if the lookup is valid.
6. **Commit to accumulator**: Send $[Z]_1$.

Constraints Recall the goal: prove $F(\beta, \gamma) = G(\beta, \gamma)$, where F is the expected fingerprint and G is the actual fingerprint of s's adjacent pairs. In PLONK, we encode this as polynomial identities checked via the quotient polynomial.

The accumulator $Z(X)$ computes a running ratio of F and G terms. If $F = G$, the ratio telescopes to 1 over the full domain.

Initialization: Z starts at 1.

$$(Z(X) - 1) \cdot L_1(X) = 0$$

Recursion: At each domain point, Z accumulates one step of the F/G ratio. The left side encodes adjacent pairs from s (split across h_1, h_2); the right side encodes the expected F terms (table pairs and lookup duplicates):

$$Z(X\omega) \cdot \underbrace{\prod_{j \in \{1,2\}} (\gamma(1+\beta) + h_j(X) + \beta h_j(X\omega))}_{G \text{ terms: actual pairs in } s}$$

$$= Z(X) \cdot \underbrace{(1+\beta)^m \cdot (\gamma + f(X))}_{\text{repeated pairs}} \cdot \underbrace{(\gamma(1+\beta) + t(X) + \beta t(X\omega))}_{\text{table pairs}}$$

The parameter m is the number of lookups per gate (typically 1 or 2).

If $F = G$, then Z returns to 1 at the end of the domain as the product telescopes. We don't add an explicit finalization constraint for this. Instead, the recursion constraint forces $Z(\omega^n) = Z(\omega^0) \cdot \prod(\text{ratio terms})$. Since $Z(\omega^0) = 1$ by initialization, and we're working over a cyclic domain, the constraint system implicitly checks that the final value is 1.

The accumulator alone isn't sufficient. It verifies that adjacent pairs in s are valid, but what if the prover constructs a fake s that doesn't actually contain the lookup values f? The grand product equality handles this: the left side of the recursion constraint multiplies over pairs from h_1, h_2, while the right side multiplies over f and t. For the products to match, the multisets must be equal. This is the same principle as the permutation argument in Chapter 13, but here it's embedded directly in the accumulator constraint rather than as a separate check.

The constraint assumes s is sorted, since that's what makes duplicates land next to their matches. Plookup enforces this implicitly rather than with an explicit sorting check. The adjacent-pair encoding $(s_i + \beta s_{i+1})$ captures ordering information: since s must be "sorted by t" (elements appear in the same order as in t), each adjacent pair in t must appear exactly once as an adjacent pair in s. If the prover reorders s, the adjacent pairs change, and the grand product fails. The randomness β prevents the prover from constructing a fake s that happens to produce the same product despite having different pairs.

Both properties are enforced by the single recursion constraint: 1. The grand product equality ensures s contains exactly $(f \cup t)$, with no values conjured from thin air. 2. The adjacent-pair encoding ensures every consecutive pair is valid (either a repeat or a table step). 3. The same encoding implicitly enforces sorting: reordering s changes its adjacent pairs, breaking the grand product.

If all hold, every element in f found a matching entry in t. A cheating prover cannot slip in a value outside the table since it would create an invalid pair that breaks the accumulator.

Verification The verifier checks the polynomial identities (initialization, recursion) via the standard PLONK batched evaluation. The verifier never touches the table directly. The table polynomial $t(X)$ was committed during setup, and the verifier only checks openings at random evaluation points. Verification cost is independent of table size d: a lookup into a 256-entry table costs the same as a lookup into a million-entry table.

Comparison: Custom Gates vs. Lookup Tables

Both custom gates and lookup tables extend PLONK beyond vanilla arithmetic, but they solve different problems.

Custom gates add terms to the universal gate equation. For example, adding a selector Q_{pow5} enables a^5 computation in a single constraint:

$$Q_L a + Q_R b + Q_O c + Q_M ab + Q_{\text{pow5}} a^5 + Q_C = 0$$

This works well for Poseidon S-boxes, which need fifth powers. The constraint is low-degree, requires no precomputation, and adds no extra commitments. But custom gates hit a wall when the relation isn't algebraically compact. A boolean check is easy: $x^2 - x = 0$ has degree 2. A 16-bit range check would need $x(x-1)(x-2)\cdots(x-65535) = 0$, a degree-65536 polynomial that no proof system can handle efficiently.

Lookup tables solve this by shifting complexity from constraint degree to table size. Instead of encoding "x is in $[0, 65535]$" as a high-degree polynomial, we precompute a table of valid values and prove membership via the grand product. As we saw in the Verification section, the verifier never touches the table directly, so verification cost scales with the number of lookups, not the table size.

The tradeoff is that lookups add overhead. Each lookup requires entries in the sorted vector s, contributions to the accumulator polynomial, and additional commitment openings. For a simple boolean check, this machinery is overkill. For a 64-bit range check or an 8-bit XOR operation, lookups are necessary.

Problem	Custom Gate	Lookup Table
Boolean check ($x \in \{0, 1\}$)	Ideal	Overkill
8-bit range check	Possible	Efficient
64-bit range check	Impractical	Essential
XOR/AND/OR operations	Complex	Clean
Poseidon x^5	One gate	Unnecessary
Valid opcode check	Complex	Direct

Modern systems like UltraPLONK use both: custom gates for algebraic primitives, lookup tables for everything else.

Alternative Lookup Protocols

Plookup was seminal but not unique. Several alternatives offer different trade-offs.

LogUp: The Logarithmic Derivative Approach

Recall the naive product identity from the beginning of this chapter:

$$\prod_{i=1}^{n}(X - f_i) = \prod_{j=1}^{d}(X - t_j)^{m_j}$$

Plookup avoided the multiplicity problem by using the sorted merge s. LogUp takes a different route: transform the product into a sum where multiplicities become coefficients rather than exponents. Taking the logarithmic derivative (i.e., $\frac{d}{dX}\log(\cdot)$) of both sides, and using $\frac{d}{dX}\log(X - a) = \frac{1}{X-a}$ and $\frac{d}{dX}\log((X - a)^m) = \frac{m}{X-a}$:

$$\sum_{i=1}^{n}\frac{1}{X - f_i} = \sum_{j=1}^{d}\frac{m_j}{X - t_j}$$

The exponentiation $(X - t_j)^{m_j}$ that required binary decomposition becomes simple scalar multiplication $m_j \cdot \frac{1}{X-t_j}$. Over finite fields, we don't actually compute logs or derivatives; the identity is purely algebraic. If the multisets match, the rational functions are equal. Evaluating at a random challenge $\gamma \in \mathbb{F}$ gives Schwartz-Zippel soundness.

This matters for several reasons:

1. **No sorting required.** Plookup requires constructing and committing to the sorted vector s. LogUp skips this entirely: no sorted polynomial, no sorting constraints.
2. **Additive structure.** Products become sums of fractions. This enables:
    - Simpler multi-table handling (just add the sums)
    - Natural integration with sum-check protocols
    - Easier batching of multiple lookup arguments
3. **Better cross-table lookups.** When a circuit uses multiple tables (range, XOR, opcodes), LogUp handles them in a unified sum rather than separate grand products.

Worked Example: Lookups $f = \{2, 2, 5\}$ into table $t = \{1, 2, 3, 4, 5\}$ over $\mathbb{F}_{97}$.

The multiplicities are $m_1 = 0, m_2 = 2, m_3 = 0, m_4 = 0, m_5 = 1$. The verifier sends random challenge $\gamma = 10$. Both sides evaluate at $X = \gamma$:

Left side (lookups):

$$\frac{1}{10-2}+\frac{1}{10-2}+\frac{1}{10-5}=\frac{1}{8}+\frac{1}{8}+\frac{1}{5}$$

Over $\mathbb{F}_{97}$: $8^{-1}\equiv 85$ and $5^{-1}\equiv 39$ (since $8\times 85=680=7\times 97+1$ and $5\times 39=195=2\times 97+1$). So the left side is $85+85+39=209\equiv 15 \pmod{97}$.

Right side (table with multiplicities):

$$\frac{0}{10-1}+\frac{2}{10-2}+\frac{0}{10-3}+\frac{0}{10-4}+\frac{1}{10-5}=\frac{2}{8}+\frac{1}{5}=2\cdot 85+39=170+39=209\equiv 15 \pmod{97}$$

Both sides equal 15. The identity holds.

Verification: $15=15$ ✓

If we had tried to look up $f=\{2,2,9\}$ (with $9\notin t$), the left side would include $\frac{1}{10-9}=\frac{1}{1}=1$. The left sum becomes $85+85+1=171\equiv 74 \pmod{97}$. No assignment of multiplicities to the table entries can make the right side equal 74, so the check fails with overwhelming probability over the choice of γ.

The LogUp bus

LogUp's additive structure enables a pattern that has become the standard architecture in STARK-based zkVMs: the **bus argument**. When a system has multiple specialized components (an ALU chip, a memory chip, a program counter chip), each component produces or consumes values that must be consistent across components. A CPU chip "sends" an addition operation (a,b,c) to the ALU chip, which "receives" it and checks $a+b=c$.

The bus formalizes this as a global sum constraint. Each sender contributes $+\frac{1}{\gamma-v}$ for a value v it sends. Each receiver contributes $-\frac{1}{\gamma-v}$ for a value v it receives. If every sent value is received exactly once, the global sum is zero:

$$\sum_{\text{sends}}\frac{1}{\gamma-v_i}-\sum_{\text{receives}}\frac{1}{\gamma-v_j}=0$$

This is just the LogUp identity rewritten: senders play the role of lookups f, receivers play the role of table t. The zero-sum condition replaces the multiset equality check. Each component adds one auxiliary "running sum" column to its trace, accumulating its contribution row by row. The boundary constraint asserts that the global sum of all components' final running-sum values is zero.

The bus scales linearly with the number of components ($O(k)$ for k tables) rather than quadratically ($O(k^2)$) as pairwise permutation arguments would require. Every major STARK-based zkVM (SP1, RISC Zero, Stwo, OpenVM) now uses LogUp bus arguments for inter-component consistency. Chapter 20 discusses how interaction columns implementing LogUp fit into the STARK prover pipeline.

LogUp-GKR combines the bus with the GKR protocol (Chapter 7) for even greater efficiency. Instead of committing to a helper column for the reciprocals $\frac{1}{\gamma - v}$, the prover uses a GKR interactive proof to verify the fractional sums directly. This eliminates helper columns entirely, adding only $O(\log n)$ interaction rounds. StarkWare's Stwo prover uses LogUp-GKR over Mersenne31.

cq (Cached Quotients)

A refinement of the logarithmic derivative approach optimized for repeated lookups.

cq pre-computes quotient polynomials for the table, amortizing table processing across multiple lookup batches. The trade-off is setup overhead; benefits emerge with many lookups against the same table.

Caulk and Caulk+

Caulk (2022) asked a different question: what if the table is *huge* but you only perform a few lookups? Plookup's prover work scales linearly with table size, making it impractical for tables of size 2^{30} or larger.

The core idea: encode the set (or table) $\{t_1, \ldots, t_d\}$ as a polynomial $t(X) = \prod_{j=1}^{d}(X - t_j)$, whose roots are exactly the set elements. To prove that a value v is in the set, observe that $(X - v)$ divides $t(X)$ iff v is a root. KZG lets you prove this divisibility via a quotient polynomial $q(X) = t(X)/(X - v)$, without revealing which root v is. The quotient commitment can be computed from the table commitment using properties of KZG, and this computation is sublinear in d.

Prover work is $O(m^2 + m \log d)$ for m lookups into a table of size d, sublinear in d when $m \ll d$. The trade-off: Caulk requires trusted setup (KZG), and the quadratic term in m limits scalability for many lookups.

Caulk is actually a general *membership proof* protocol: given a KZG commitment to a set, prove that certain values belong to that set without revealing which positions they occupy. This makes it useful beyond lookup tables, e.g., as an alternative to Merkle proofs for set membership. Plookup and LogUp can't serve this role because they require the prover to process the entire table during proving, which defeats the purpose of a compact membership proof. Caulk's sublinear prover cost is what enables the generalization.

Caulk+ refined this to $O(m^2)$ prover complexity, removing the $\log d$ term entirely.

Halo2 Lookups

Halo2, developed by the Electric Coin Company (Zcash), integrates lookups natively with a "permutation argument" variant rather than Plookup's grand product.

The core idea: to prove $A \subseteq S$ (lookups A are contained in table S), the prover constructs permuted columns A' and S' such that A' is a permutation of A, S' is a permutation of S, and in each row either $A'_{i+1} = A'_i$ (a repeat) or $A'_{i+1} = S'_i$ (a table match). This forces every element in A' to equal some element in S'. The permutation constraints are enforced via a grand product argument similar to PLONK's copy constraints. Unlike Plookup, there is no explicit sorted merge; the "sorting" happens implicitly through the permutation.

Halo2's lookup API lets developers define tables declaratively. The proving system handles the constraint generation automatically. This made Halo2 popular for application circuits: you specify *what* to look up, not *how* the lookup argument works. Scroll, Taiko, and other L2s built on Halo2 rely on its lookup system for zkEVM implementation.

Lasso and Jolt

All the protocols above (Plookup, LogUp, Caulk, Halo2) share a limitation: the prover must commit to polynomials whose degree scales with table size.

For Plookup, the sorted vector s has length $n+d$ (lookups plus table). For LogUp, the multiplicity polynomial has degree d. For Caulk, the table polynomial $t(X)$ must be committed during setup. In every case, a table of size 2^{20} means million-coefficient polynomials. A table of size 2^{64} means polynomials with more coefficients than atoms in a grain of sand.

This is a hard wall, not a soft cost. The evaluation table of a 64-bit ADD instruction has 2^{128} entries. No computer can store that polynomial, let alone commit to it.

Early zkVMs worked around this by using small tables (8-bit or 16-bit operations) and paying the cost in constraint complexity for larger operations. A 64-bit addition became a cascade of 8-bit additions with carry propagation. It worked, but it was slow.

Lasso (2023, Setty-Thaler-Wahby) breaks through this wall: prover costs scale with *lookups performed* rather than *table size*.

Static vs. Dynamic Tables

Before diving into Lasso's mechanism, distinguish two types of lookups:

Static tables (read-only): Fixed functions like XOR, range checks, or AES S-boxes. The table never changes during execution. Plookup, LogUp, and Lasso excel here.

Dynamic tables (read-write): Simulating RAM (random access memory). The table starts empty and fills up as the program runs. This requires different techniques (like memory-checking arguments or timestamp-based permutation checks) because the table itself is witness-dependent.

Lasso focuses on static tables, but its decomposition insight is what makes truly large tables tractable.

Decomposable Tables

Lasso exploits *decomposable tables*. Many tables have structure: their MLE (multilinear extension) can be written as a weighted sum of smaller subtables:

$$\tilde{T}(y) = \sum_{j=1}^{\alpha} c_j \cdot \tilde{T}_j(y_{S_j})$$

Each subtable $\tilde{T}_j$ looks at only a small chunk of the total input y. This "Structure of Sums" (SoS) property enables dramatic efficiency gains. (This is a cousin of the tensor product structure for Lagrange bases in Chapter 4, since both exploit how multilinear functions over product domains inherit structure from their factors.)

Consider 64-bit AND. The conceptual table has 2^{128} entries (all pairs of 64-bit inputs). But bitwise AND decomposes perfectly: split inputs into sixteen 4-bit chunks, perform 16 lookups into a tiny 256-entry AND_4 table, concatenate results. The prover never touches the 2^{128}-entry table.

Why Prover Costs Scale with Lookups

Lasso represents the sparse access pattern (which indices were hit, how many times) using commitment schemes optimized for sparse polynomials, then proves correctness via sum-check. The prover commits only to the accessed entries and their multiplicities, never to the full table. For structured tables, the verifier can evaluate $\tilde{T}(r)$ at a random challenge point in $O(\log N)$ time using the table's algebraic formula, without ever seeing the table itself.

Jolt: A zkVM Built on Lasso

Jolt applies Lasso to build a complete zkVM for RISC-V. The philosophy: replace arithmetization of instruction semantics with lookups.

The entire RISC-V instruction set can be viewed as one giant table mapping (opcode, operand1, operand2) to results. This table is far too large to materialize, but it's *decomposable*: most instructions break into independent operations on small chunks. A 64-bit XOR decomposes into 16 lookups into a 256-entry XOR_4 table. The subtables are tiny, pre-computed once, and reused across all instructions.

Jolt combines Lasso (for instruction semantics) with R1CS constraints (for wiring: program counter updates, register consistency, data flow). Why this hybrid? Arithmetizing a 64-bit XOR in R1CS requires 64+ constraints for bit decomposition; Jolt proves it with 16 cheap lookups. But simple wiring constraints are trivial in R1CS. Use each tool where it excels.

Limitations

Lasso and Jolt require decomposable table structure. Tables without chunk-independent structure don't benefit. But for CPU instruction sets, the structure is natural: most operations are bitwise or arithmetic with clean chunk decompositions.

The field continues evolving. The core insight (reducing set membership to polynomial identity) admits many instantiations, each optimizing for different table sizes, structures, and use cases.

Lookups Across Proving Systems

The lookup techniques above: Plookup, LogUp, Lasso, adapt to different proving backends. Plookup and Halo2 integrate naturally with PLONK's polynomial commitment model. Lasso and Jolt use sumcheck and R1CS (via Spartan). STARK-based systems take a different path.

In STARKs, computation is represented as an execution trace: a matrix where each row is a state and columns hold registers, memory, and auxiliary values. Lookup arguments integrate by adding columns to this trace:

- The lookup table becomes one or more *public columns* (known to the verifier)
- Values to be looked up appear in *witness columns*
- A *running product column* accumulates the grand product (Plookup-style) or running sum (LogUp-style)
- *Transition constraints* enforce the recursive accumulator relation row-by-row

The FRI-based polynomial commitment then proves that these trace columns satisfy all constraints. The lookup argument's algebraic core is unchanged; only the commitment mechanism differs.

STARK-based zkVMs (Cairo, RISC0, SP1) rely heavily on this integration. Their execution traces naturally represent VM state transitions, and lookups handle instruction semantics, memory consistency,

and range checks. The trace-based model makes it easy to add new lookup tables: just add columns and constraints.

Key takeaways

General principles (apply to all lookup arguments):

1. **Lookup arguments shift complexity from logic to data**: Precompute valid tuples; prove membership rather than computation. This is the core insight shared by Plookup, LogUp, Lasso, and all variants.
2. **The formal problem**: Given lookups f and table t, prove $f \subseteq t$. Different protocols reduce this multiset inclusion to different polynomial identities.
3. **Cost structure**: Lookup-based proofs achieve roughly constant cost per lookup, independent of the logical complexity of what the table encodes. A 16-bit range check or an 8-bit XOR costs the same as a simple membership test.
4. **Complements custom gates**: Lookups handle non-algebraic constraints; custom gates handle algebraic primitives. Modern systems (UltraPLONK, Halo2) use both.
5. **zkVM foundation**: Without lookup arguments, verifying arbitrary computation at scale would be infeasible. Every major zkVM relies on lookups for instruction semantics.

Plookup-specific mechanics (the sorted-merge approach from Section 2):

6. **Sorted vector reduction**: Plookup transforms $f \subseteq t$ into a claim about the sorted merge $s = \text{sort}(f \cup t)$.
7. **Adjacent pair property**: In Plookup, every consecutive pair in s is either a repeat (from f slotting in) or exists as adjacent in t.
8. **Grand product identity**: The polynomial identity $F \equiv G$ encodes Plookup's adjacent-pair check. The accumulator $Z(X)$ enforces this recursively, integrating with PLONK's permutation machinery.

Alternative approaches (different trade-offs):

9. **LogUp** replaces products with sums of logarithmic derivatives: no sorting, cleaner multi-table handling, natural sum-check integration.
10. **Caulk** achieves sublinear prover work in table size via KZG-based subset arguments, useful when few lookups access a huge table.

11. **Halo2** uses permutation arguments rather than sorted merges, with lookups integrated into the constraint system declaratively.
12. **Lasso** exploits *decomposable tables* (SoS structure) to achieve prover costs scaling with lookups performed, not table size. Combined with sparse polynomial commitments, this enables effective tables of size 2^{128}. Jolt applies this to build a complete zkVM.
13. **STARK integration**: Lookup arguments adapt to trace-based proving via running product/sum columns and transition constraints, used by Cairo, RISC0, and SP1.

Chapter 15: STARKs

While Gabizon and Williamson were building PLONK, a parallel revolution was underway.

Eli Ben-Sasson had been working on probabilistically checkable proofs (PCPs) since the early 2000s: the discovery that any proof can be encoded so a verifier need only spot-check a few random bits to detect errors. PCPs transformed complexity theory but remained practically useless. The constructions were galactic.

All the pairing-based SNARKs we've seen (Groth16, PLONK) require trusted setup. Ben-Sasson asked a different question: could you build proof systems using *nothing but hash functions*?

In 2018, Ben-Sasson and colleagues (Bentov, Horesh, Riabzev) published the **STARK** (Scalable Transparent ARgument of Knowledge) construction: transparent (no trusted setup), post-quantum (no pairings), with security based only on collision-resistant hashing. The theoretical ingredients, Interactive Oracle Proofs (IOPs), the FRI protocol (see Chapter 10), the ALI protocol (Algebraic Linking IOP), had been developed over the preceding years, often by the same researchers. The 2018 paper synthesized them into a complete, practical system.

STARKs have since become one of the two dominant proof system families, with independent implementations by StarkWare, Polygon, and RISC Zero. This chapter develops the STARK paradigm: how FRI combines with a state-machine model of computation to yield transparent, scalable, quantum-resistant proofs, at the cost of larger proof sizes than their pairing-based cousins.

Why Not Pairings?

The most efficient SNARKs in Chapters 12-13 rely on pairing-based polynomial commitments. Groth16 builds pairings directly into its verification equation. PLONK is a polynomial IOP, agnostic to the com-

mitment scheme, but achieves its smallest proofs when compiled with KZG, which requires pairings. The bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is what enables constant-size proofs and $O(1)$ verification.

This foundation is remarkably productive. But it carries costs that grow heavier with scrutiny.

The first cost is trust. A KZG commitment scheme requires a structured reference string: powers of a secret τ encoded in the group. Someone generated that τ. If they kept it, they can forge proofs. The elaborate ceremonies of Chapter 12 (the multi-party computations, the public randomness beacons, the trusted participants) exist to distribute this trust. But distributed trust is still trust. The ceremony could fail. Participants could collude. The procedures could contain subtle flaws discovered years later.

The second cost is quantum vulnerability. Shor's algorithm solves discrete logarithms in polynomial time on a quantum computer. The security of KZG, Groth16, and IPA all rest on the hardness of discrete log in elliptic curve groups. Pairings add structure on top of this assumption but don't change the underlying vulnerability. When a sufficiently large quantum computer exists, all these schemes break. When that day comes is uncertain. That it will come seems increasingly likely. A proof verified today may need to remain trusted for decades.

The third cost is field rigidity. Only a small family of elliptic curves support efficient pairings while remaining cryptographically secure, and each curve dictates a specific large prime field (e.g., the 254-bit field of BN254, the 381-bit field of BLS12-381). Pairing-based proof systems are locked into these fields, ruling out optimizations over smaller or differently structured fields where arithmetic is dramatically cheaper.

STARKs abandon elliptic curves entirely. They ask a more primitive question: *what can we prove using only hash functions?*

The Hash Function Gambit

A collision-resistant hash function is perhaps the most conservative cryptographic assumption we have. SHA-256, Blake3, Keccak: these primitives are analyzed relentlessly, deployed universally, and trusted implicitly. They offer no algebraic structure, no homomorphisms, no elegant equations. Just a box that takes input and produces output, where finding two inputs with the same output is computationally infeasible.

The quantum story here differs from discrete log in kind. Grover's algorithm provides a quadratic speedup for unstructured search, reducing the security of a 256-bit hash from 2^{256} to 2^{128} operations. This is manageable: use a larger hash output and security is restored. Contrast this with Shor's exponential speedup against discrete log, which breaks the problem entirely rather than merely weakening it.

This seems like a step backward. Algebraic structure is what made polynomial commitments possible. KZG works because $g^{p(\tau)}$ preserves polynomial relationships, because the commitment scheme respects the algebra of the underlying object. A hash function respects nothing. $H(a+b) \neq H(a) + H(b)$. The hash of a polynomial evaluation tells you nothing about the polynomial.

Yet hash functions offer something pairings cannot: a Merkle tree. Chapter 10 developed this machinery in detail; here we summarize the key ideas before showing how STARKs compose them into a complete proof system.

Commit to a sequence of values by hashing them into a binary tree. The root is the commitment. To open any leaf, provide the authentication path, the $O(\log n)$ hashes connecting that leaf to the root. The binding property is information-theoretic within the random oracle model: changing any leaf changes the root. No trapdoors, no toxic waste, no ceremonies.

The problem is that a Merkle commitment is simultaneously too strong and too weak. It's too strong in that opening a single position already costs $O(\log n)$ hash values, compared to $O(1)$ for KZG. And it's too weak in that there's no way to prove anything *about* the committed values without opening them. A KZG commitment to a polynomial p lets you prove $p(z) = v$ at any point with a single group element. A Merkle commitment to evaluations of p on a domain lets you prove $p(z) = v$ only if z happens to be in the domain, and only by opening that leaf explicitly.

The insight behind STARKs is that these limitations can be overcome by a shift in perspective. Instead of proving polynomial identities directly, we prove that a committed function is *close to* a low-degree polynomial. This is the domain of coding theory, not algebra. And coding theory has powerful tools for detecting errors through random sampling.

The Reed-Solomon Lens

Every proof system we've seen reduces computation to polynomial constraints. The prover commits to polynomials; the verifier checks that these polynomials satisfy certain identities. In pairing-based systems (Groth16, PLONK), the commitment scheme itself enforces polynomial structure: KZG commitments can only represent polynomials, and pairing checks verify evaluations algebraically. Low-degree-ness is built into the commitment.

With Merkle trees, this is no longer free. A Merkle tree commits to arbitrary sequences of field elements, with no structural guarantee. The prover *claims* the committed values are evaluations of a low-degree polynomial, but nothing about the commitment prevents them from committing garbage.

The Reed-Solomon encoding (Chapter 2) solves this. The prover's polynomial has degree at most $k-1$, determined by k evaluations. But the prover evaluates it on a much larger domain of n points, with $n \gg k$, and commits to all n values. This serves two purposes. First, it creates something to check:

any k field elements are consistent with *some* degree-$(k-1)$ polynomial (by Lagrange interpolation), so a commitment to just k values can never be invalid. But most sequences of n values are *not* consistent with any low-degree polynomial, so cheating becomes detectable. Second, the verifier queries random points in the extended domain rather than the trace domain, so the actual computation data is never revealed.

The Reed-Solomon distance property quantifies the first point. If the committed values don't correspond to a degree-$(k-1)$ polynomial, they disagree with *every* such polynomial on at least $n-k+1$ of the n positions. A random query hits a disagreement with probability at least $\delta = 1-(k-1)/n$, and q independent queries miss all disagreements with probability at most $(1-\delta)^q$. For a blowup factor $\rho = n/k = 8$ and $q = 45$ queries: $(1/8)^{45} < 2^{-135}$. A random sample suffices.

The **FRI protocol** (Chapter 10) turns this sampling argument into a complete interactive low-degree test, replacing the structural guarantee that KZG provides for free.

So STARKs have a way to commit to polynomials (Merkle trees) and a way to verify they're low-degree (FRI). But FRI only proves a degree bound: the committed function is close to *some* low-degree polynomial. We still need to prove it's the *right* polynomial, one that encodes a valid computation. That requires a way to express computation as polynomial constraints.

Computation as State Evolution

How should we encode computation into polynomials for this framework? The proof systems of previous chapters use circuits: directed acyclic graphs where wires carry values and gates impose constraints. This works, but it handles iteration awkwardly. A loop executing n times becomes n copies of the loop body, each a separate subcircuit. The repetition that made the loop simple to write is obscured in the flattened graph.

STARKs adopt a different model: the **state machine**.

A computation is a sequence of states $S_0, S_1, \ldots, S_{T-1}$ evolving over discrete time. Each state is a tuple of register values. A transition function f maps S_i to S_{i+1}, and f is the same at every timestep. Only the register values change.

This uniformity is what makes the model efficient. A hash function running for n rounds, a CPU executing n instructions: both are n applications of the same transition function. In a circuit, each iteration contributes its own gates and constraints, scaling linearly with n. In a state machine, the transition constraints describe a single step and apply identically at every timestep. The description has fixed size, independent of n.

Suppose we want to prove we computed $3^8 = 6561$. The state machine has two registers: a counter c and an accumulator a. The transition rule: $c' = c + 1$ and $a' = a \cdot 3$. The trace:

Step	c	a
0	0	1
1	1	3
2	2	9
3	3	27
4	4	81
5	5	243
6	6	729
7	7	2187
8	8	6561

This table is a *trace*: a matrix with $w = 2$ registers and $T = 9$ rows. The "Step" column is just a label; the actual data is the c and a columns. Each row captures the complete state at one moment; each column tracks one register's evolution through time.

The transition constraint ("next accumulator equals current accumulator times 3") is the same at every row. We don't need 8 separate multiplication gates; we need one constraint that holds 8 times. The prover commits to the entire trace, then proves the constraint holds everywhere. For $3^{1000000}$, the constraint is still just one equation; only the trace grows longer.

Algebraic Intermediate Representation

An **AIR** (Algebraic Intermediate Representation) encodes the trace and its transition constraints in polynomial form.

The trace is a matrix with w columns (registers) and T rows (timesteps). Each column, viewed as a sequence of T field elements, becomes a polynomial via interpolation. Choose a domain $H = \{1, \omega, \omega^2, \ldots, \omega^{T-1}\}$ where ω is a primitive T-th root of unity. The column polynomial $P_j(X)$ is the unique polynomial of degree less than T satisfying $P_j(\omega^i) = \text{trace}[i][j]$.

In the 3^8 trace, we have two registers: c (the counter) and a (the accumulator). These become two column polynomials:

- $P_c(X)$: the unique degree-8 polynomial passing through $(1, 0), (\omega, 1), (\omega^2, 2), \ldots, (\omega^8, 8)$
- $P_a(X)$: the unique degree-8 polynomial passing through $(1, 1), (\omega, 3), (\omega^2, 9), \ldots, (\omega^8, 6561)$

Since $P_j(\omega^i)$ is the value of register j at step i, replacing X with ωX shifts forward by one step: $P_j(\omega \cdot \omega^i) = P_j(\omega^{i+1})$, which is step $i+1$. This lets us express "next row" algebraically. The transition constraint "next accumulator = current accumulator × 3" becomes $P_a(\omega X) = 3 \cdot P_a(X)$. At $X = \omega^2$, this says $P_a(\omega^3) = 3 \cdot P_a(\omega^2)$, i.e., $27 = 3 \cdot 9$. The single polynomial identity encodes all 8 transition checks at once.

Another example: if a different transition function requires that register r_0 at step $i+1$ equals $r_0^3 + r_1$ at step i, this becomes:

$$P_0(\omega X) = P_0(X)^3 + P_1(X)$$

This identity must hold for $X \in \{1, \omega, \ldots, \omega^{T-2}\}$, covering all $T-1$ transitions. Define the constraint polynomial:

$$C(X) = P_0(\omega X) - P_0(X)^3 - P_1(X)$$

If the trace is valid, $C(X)$ vanishes on $H' = \{1, \omega, \ldots, \omega^{T-2}\}$. By the factor theorem, $C(X)$ is divisible by the vanishing polynomial $Z_{H'}(X) = \prod_{h \in H'}(X - h)$. The quotient:

$$Q(X) = \frac{C(X)}{Z_{H'}(X)}$$

is a polynomial of known degree. If $C(X)$ doesn't vanish on H' (if the trace violates the transition constraint somewhere) then $Q(X)$ isn't a polynomial. It's a rational function with poles at the violation points.

> **Why Constraint Degree Matters**
>
> The degree of the constraint polynomial $C(X)$ directly impacts prover cost. If a transition constraint involves $P_0(X)^3$, that term has degree $3(T-1)$ (since P_0 has degree $T-1$). The composition polynomial inherits this: $\deg(\text{Comp}) \approx \deg(\text{constraint}) \times T$. The prover must commit to this polynomial over the LDE domain, and FRI must prove its degree bound.
>
> This creates a trade-off. Higher-degree constraints let you express more complex transitions in a single step, but they blow up the prover's work. A degree-8 constraint over a million-step trace produces a composition polynomial of degree ~8 million, requiring proportionally more commitment and FRI work. Most practical AIR systems keep constraint degree between 2 and 4, accepting more trace columns (more registers) to avoid high-degree terms. The art of AIR design is balancing expressiveness against this degree bottleneck. Chapter 20 quantifies this tradeoff and develops the AIR design patterns (auxiliary columns, periodic columns, wide versus tall traces)

> that production systems use to minimize prover cost.

Transition constraints enforce the rules at every step, but they say nothing about *which* computation we're proving. We also need **boundary constraints** to pin down the inputs and outputs. In our 3^8 example:

- **Input**: $P_a(1) = 1$ (accumulator starts at 1)
- **Output**: $P_a(\omega^8) = 6561$ (accumulator ends at 3^8)

Each becomes a divisibility check. If the input requires register 0 to equal 5 at step 0, the constraint $P_0(1) = 5$ becomes $P_0(X) - 5$ vanishing at $X = 1$, quotient $(P_0(X) - 5)/(X - 1)$.

We now have multiple constraint quotients: Q_{trans} for the transition, Q_{in} and Q_{out} for boundaries, possibly more. Rather than prove each separately, we batch them into a single polynomial using random challenges $\alpha_1, \alpha_2, ...$ (derived via Fiat-Shamir):

$$\text{Comp}(X) = \alpha_1 Q_{\text{trans}}(X) + \alpha_2 Q_{\text{in}}(X) + \alpha_3 Q_{\text{out}}(X) + ...$$

Why does this work? If all quotients are polynomials, their linear combination is a polynomial. If any quotient has a pole (from a violated constraint), the random combination almost certainly preserves that pole: the α_i values would need to be precisely chosen to cancel it, which happens with negligible probability over a large field.

Putting it together for our 3^8 example, the three quotients are:

1. **Transition**: $Q_{\text{trans}}(X) = C(X)/Z_{H'}(X)$ is a polynomial (each step follows the rules)
2. **Input boundary**: $(P_a(1) - 1)/(X - 1)$ is a polynomial (accumulator starts at 1)
3. **Output boundary**: $(P_a(\omega^8) - 6561)/(X - \omega^8)$ is a polynomial (accumulator ends at 6561)

If any constraint fails, the corresponding quotient has a pole, the composition polynomial inherits it, and FRI rejects it as non-low-degree.

To make this concrete: the trace polynomials $P_j(X)$ have degree at most $T-1$, since the trace domain $H = \{1, \omega, ..., \omega^{T-1}\}$ has T points (9 in our 3^8 example). The prover evaluates them not on H alone, but on a larger domain $D \supset H$, typically 4 to 16 times larger. This is the **low-degree extension (LDE)**. As we saw in the Reed-Solomon section, this redundancy is what makes cheating detectable: FRI's random queries in D catch the non-low-degree composition polynomial. The prover commits to these LDE evaluations via Merkle tree, with the root as the commitment.

The Complete Protocol

Prover's Algorithm:

1. Execute the computation, producing the execution trace.
2. Interpolate each trace column to obtain polynomials $P_1(X), \ldots, P_w(X)$ over domain H.
3. Evaluate all w polynomials on the LDE domain D, forming a $|D| \times w$ matrix. Commit this matrix in a single Merkle tree: each leaf is the hash of one row $(P_1(x), \ldots, P_w(x))$ for a domain point $x \in D$. Send the trace root to the verifier.
4. Derive random challenges $\alpha_1, \alpha_2, \ldots$ by hashing the transcript (Fiat-Shamir).
5. Compute constraint polynomials, form quotients, and batch them into the composition polynomial using the challenges from step 4.
6. Evaluate the composition polynomial on D. Commit via a second Merkle tree and send the composition root to the verifier.
7. Run FRI on the composition polynomial, proving it has degree less than the known bound.
8. Derive query points $x_1, \ldots, x_k$ by hashing the transcript (Fiat-Shamir). For each x_i: open the trace polynomials and composition polynomial, providing Merkle authentication paths.

Each query catches a cheater with probability roughly $1 - 1/\rho$, where ρ is the blowup factor ($|D|/|H|$). With k queries, soundness error is roughly $(1/\rho)^k$. For 128-bit security with blowup factor 8, around 45 queries suffice.

Verifier's Algorithm:

1. Receive the Merkle roots (trace and composition), FRI commitments, and query responses.
2. Derive all Fiat-Shamir challenges from the transcript.
3. Verify FRI: check that the committed function is close to a low-degree polynomial.
4. For each query point x:
   - The prover opens the trace Merkle tree at x, providing the row $(P_1(x), \ldots, P_w(x))$ and an authentication path. The verifier hashes the row and checks it against the trace root.
   - The prover also opens the composition Merkle tree at x, providing Comp(x) and its authentication path. The verifier checks it against the composition root (which FRI proved corresponds to a low-degree polynomial).
   - The verifier plugs the trace values into the constraint equations, forms the quotients, applies the batching coefficients α_i, and locally recomputes what Comp(x) should be. If this doesn't match the opened composition value, reject.
5. Accept if all checks pass.

This last sub-step is the **AIR-FRI link**: it connects FRI (which only proves low-degree-ness, knowing nothing about constraints or computations) to the actual claim being verified. Without it, a cheating prover could commit to Comp$(X) = 0$, pass FRI trivially, and hope the verifier is satisfied.

Why is this sound? The prover committed to the trace *before* learning the query points (Fiat-Shamir). If the trace violates any constraint, the composition polynomial has poles and isn't low-degree; FRI catches this. If the trace is valid but the prover committed to a *different* composition polynomial, the opened value and the locally recomputed value disagree at most points (Schwartz-Zippel); the random queries catch this.

There is a subtle gap in standard FRI: the verifier only queries points in the LDE domain D, so a cheating prover could commit to a function that's low-degree on D but encodes wrong trace values. **DEEP-FRI** (introduced in Chapter 10) closes this gap. The verifier samples a random point z *outside* D and requires the prover to open the trace polynomials there. Since honest trace polynomials are globally low-degree, they can be evaluated anywhere; a cheater who faked values only on D cannot consistently answer at z. In the STARK context, this means the AIR-FRI link is checked at a point the prover could not have anticipated when constructing the trace commitment, which is why most STARK implementations use DEEP-FRI rather than standard FRI. Chapter 20 develops the full DEEP-ALI optimization, showing how it also eliminates the separate composition polynomial commitment.

A Concrete Example: Fibonacci

Let's trace the protocol on a minimal computation: proving knowledge of the 7th Fibonacci number.

The claim: starting from $F_0 = 1, F_1 = 1$, the sequence satisfies $F_6 = 13$. The trace has two registers (a, b) representing consecutive Fibonacci numbers, with 6 rows (steps 0-5):

Step	a	b
0	1	1
1	1	2
2	2	3
3	3	5
4	5	8
5	8	13

The transition constraints enforce, at each step $i \in \{0, \ldots, 4\}$:

- $a_{i+1} = b_i$ (the next a is the current b)
- $b_{i+1} = a_i + b_i$ (the next b is the sum)

The boundary constraints pin down the endpoints:

- $a_0 = 1$ (initial condition)
- $b_0 = 1$ (initial condition)
- $b_5 = 13$ (the claimed output F_6)

Let ω be a primitive 6th root of unity. Interpolating the columns gives $A(X)$ with $A(\omega^i) = a_i$ and $B(X)$ with $B(\omega^i) = b_i$. Using the ω-shift from the AIR section, the constraint polynomials are:

- $C_1(X) = A(\omega X) - B(X)$: next a equals current b
- $C_2(X) = B(\omega X) - A(X) - B(X)$: next b equals current $a + b$
- $C_{B1}(X) = A(X) - 1$, vanishing at $X = 1$
- $C_{B2}(X) = B(X) - 1$, vanishing at $X = 1$
- $C_{B3}(X) = B(X) - 13$, vanishing at $X = \omega^5$

Each constraint polynomial is divided by the appropriate vanishing polynomial. The transition constraints must hold at steps 0-4, so they're divided by $Z_5(X) = (X^6 - 1)/(X - \omega^5)$. Batching with random challenges $\alpha_1, \ldots, \alpha_5$:

$$\text{Comp}(X) = \alpha_1 \frac{C_1(X)}{Z_5(X)} + \alpha_2 \frac{C_2(X)}{Z_5(X)} + \alpha_3 \frac{C_{B1}(X)}{X - 1} + \ldots$$

If the trace is valid (and it is) this composition is a polynomial of degree roughly $\deg(A) + \deg(B) - 5 \approx 5$.

Now the commitment step. The prover evaluates $A(X)$ and $B(X)$ on a larger LDE domain D (say 48 points, with blowup factor 8). Each leaf of the trace Merkle tree holds the pair $(A(x), B(x))$ for one $x \in D$. The prover sends the trace root. After deriving the Fiat-Shamir challenges $\alpha_1, \ldots, \alpha_5$, the prover evaluates $\text{Comp}(X)$ on D, commits it in a second Merkle tree, and sends the composition root.

At query time, one detail this example reveals: to check $C_1(x) = A(\omega x) - B(x)$, the verifier needs trace values at both x and ωx. So queries come in *pairs*: the prover opens the trace Merkle tree at x and ωx together, giving the verifier both the "current row" and "next row" values. The prover also opens the composition tree at x. The verifier recomputes $\text{Comp}(x)$ from the trace values and checks it matches the opened composition value.

FRI then proves the composition polynomial is low-degree via the folding protocol from Chapter 10. For our degree-5 polynomial over a 48-point LDE domain (blowup factor 8), three folding rounds reduce it to a constant. At each round, the verifier spot-checks that the folded layer is consistent with

the previous one. The same query points serve both the AIR consistency check (opening trace values) and FRI verification (opening composition values at y and $-y$ for folding), so one set of openings handles both.

Adding Zero-Knowledge

The protocol as described so far is a transparent argument of knowledge, but it is not zero-knowledge. When the verifier queries a point $x \in D$ and the prover opens the trace Merkle tree, the verifier learns the actual values $P_1(x), \ldots, P_w(x)$. These are evaluations of the trace polynomials, and they leak information about the witness (the execution trace).

Chapter 18 covers the general theory of making proof systems zero-knowledge. Two broad techniques apply: commit-and-prove (hiding values behind homomorphic commitments) and polynomial masking (adding randomness that is invisible on the constraint domain but randomizes the verifier's queries). Here we focus on the approach specific to STARKs: **trace randomization**.

The idea is to extend the execution trace with random data before committing. The prover appends k random rows to the trace (typically $k = 2$ to 4), filled with random field elements, extending it from T to $T + k$ rows. The trace polynomials are then interpolated over a domain of size $T + k$ rather than T.

Why does this help? The trace polynomials now encode both the real computation (on the first T rows) and random noise (on the last k rows). A low-degree polynomial is globally determined by its values, so the random rows "contaminate" evaluations everywhere outside the original domain H. More precisely, each trace polynomial has degree $T+k-1$, determined by T real values and k random values. The k random degrees of freedom make the polynomial's evaluations at any k points outside H statistically independent of the real trace. Since the verifier's queries land in $D \setminus H$, the opened values reveal nothing about the witness.

The constraint system requires only minor adjustments. The random rows do not satisfy the transition constraints, but they don't need to: $Z_{H'}(X)$ already vanishes only at $\{\omega^0, \ldots, \omega^{T-2}\}$, so the quotient $C(X)/Z_{H'}(X)$ remains a polynomial even though $C(X)$ is nonzero at the random row positions. Boundary constraints are unaffected since they pin specific rows within the original trace (e.g., $P_a(\omega^0) = 1$). The composition polynomial is formed as before but over the larger domain, and FRI proves the slightly larger degree bound $T + k - 1$.

Verification works directly on the blinded polynomials. The verifier never needs to see the actual trace values on H. At a query point $x \in D \setminus H$, the prover opens the blinded evaluations $P_1(x), \ldots, P_w(x)$, and the verifier recomputes $C(x)/Z_{H'}(x)$ from them, checking consistency with FRI. The quotient check confirms that some low-degree polynomial satisfies the constraints on H, which is all the verifier needs. The boundary constraints are verified through their own quotient terms in the composition

polynomial.

A simulator that knows only the public inputs and outputs can produce identically distributed transcripts: it picks random trace polynomials consistent with the boundary constraints and simulates the protocol. The random rows provide enough freedom to match any set of query responses the real prover would produce. This technique is specific to the STARK setting because it exploits the separation between the trace domain H and the query domain $D \setminus H$. Pairing-based systems use different masking strategies suited to their algebraic structure (see Chapter 18).

The Trust and Size Trade-off

STARKs achieve transparency at a cost: proof size.

Property	Groth16	PLONK (KZG)	STARKs
Trusted setup	Per-circuit	Universal	**None**
Proof size	**128 bytes**	~500 bytes	**20-100 KB**
Verification	**O(1)**	O(1)	O(polylog n)
Post-quantum	No	No	**Yes**
Assumptions	Pairing-based	q-SDH	**Hash function**

The gap is stark: two orders of magnitude in proof size, from hundreds of bytes to tens of kilobytes. For on-chain verification, where every byte costs gas, this matters enormously. A Groth16 proof costs perhaps 200K gas to verify on Ethereum. A raw STARK proof would cost millions.

But the size gap has motivated clever engineering. Proof wrapping is a general composition technique where one proof system verifies the output of another, and any system can in principle be wrapped. STARKs benefit from this the most because their large proofs are precisely the problem wrapping solves. Concretely, a STARK proves the bulk of the computation (transparently, with the state machine model's natural fit for VMs), then a Groth16 proof attests "I verified a valid STARK proof." The Groth16 verification circuit is fixed-size and small. The on-chain cost is the cost of verifying Groth16, regardless of the original computation's size.

This hybrid architecture is deployed in production systems like StarkNet, zkSync, and Polygon zkEVM. The STARK itself remains fully transparent, relying only on hash functions. Pairings enter only through the Groth16 wrapper, which verifies a fixed, auditable circuit. Part of why STARKs dominate in these systems is AIR's natural fit for virtual machines: the transition constraints encode the VM's instruction set once, and the trace varies with the program while the constraints stay fixed. The circuit model

would require a different circuit for each program, or "unrolling" the VM for a fixed number of steps. AIR handles arbitrary-length execution with fixed constraint complexity.

Circle STARKs and Small-Field Proving

Throughout this chapter, we interpolated trace columns over a domain $H = \{1, \omega, \omega^2, \ldots, \omega^{T-1}\}$ of roots of unity. This choice wasn't arbitrary: roots of unity enable the FFT, which is what makes interpolation and evaluation over H efficient ($O(n \log n)$ rather than $O(n^2)$). But FFT requires a multiplicative subgroup of size 2^k, which constrains the field: we need primes p where $p - 1$ is divisible by a large power of 2. Fields like Goldilocks ($2^{64} - 2^{32} + 1$) and BabyBear ($2^{31} - 2^{27} + 1$) are carefully constructed to meet this requirement.

Circle STARKs remove this constraint by working over a different algebraic structure: the circle group.

The Circle Group

Consider a prime p and the set of points (x, y) satisfying $x^2 + y^2 = 1$ over $\mathbb{F}_p$. This is an algebraic curve, specifically a "circle" over a finite field.

For Mersenne primes like $p = 2^{31} - 1$, the circle group has particularly nice structure:

- The group has order $p + 1 = 2^{31}$, a perfect power of 2
- This enables FFT-like algorithms directly, without the $(p - 1)$ divisibility constraint
- Mersenne primes have fast modular arithmetic (reduction is just addition and shift)

The group operation on the circle is defined via the "complex multiplication" formula:

$$(x_1, y_1) \cdot (x_2, y_2) = (x_1 x_2 - y_1 y_2, x_1 y_2 + x_2 y_1)$$

This is the standard multiplication formula for complex numbers $z = x + iy$ restricted to the unit circle. Over $\mathbb{F}_p$, it's well-defined and creates a cyclic group.

The M31 Advantage

The Mersenne prime $M_{31} = 2^{31} - 1$ deserves special attention. Two properties converge to make it exceptionally efficient for STARKs.

The first is cheap arithmetic, a property of Mersenne primes themselves. For any product $a \cdot b < 2^{62}$, split the result into low and high 31-bit parts, $ab = \text{lo} + \text{hi} \cdot 2^{31}$. Since $2^{31} \equiv 1 \pmod{M_{31}}$,

reduction is just lo $+$ hi plus a conditional subtraction. No division, no extended multiplication. Since elements range from 0 to $2^{31} - 2$, each fits in a single 32-bit word, so CPUs handle them natively and SIMD instructions process 4-8 elements per cycle. Compare this to 64-bit Goldilocks (needs 64-bit multiplies, harder to vectorize) or 254-bit BN254 (requires multi-precision arithmetic, roughly 10x slower per operation). This fast arithmetic is a property of the prime, not the circle group. STARKs can exploit it because their security comes from hash functions, not from discrete log hardness over the field, so 31-bit elements provide enough room. Pairing-based systems like Groth16 and PLONK (with KZG) cannot: the pairing-friendly curve fixes the scalar field at ~254 bits, and no pairing-friendly curve exists over a 31-bit field. Sum-check based systems occupy a middle ground: sum-check itself is field-agnostic, but the PCS dictates the field. With KZG commitments, they inherit the same ~254-bit constraint. With hash-based commitments (Brakedown, Binius), they too can use small fields.

The second property is the circle group's order. Over M31, the multiplicative group has order $p - 1 = 2(2^{30} - 1)$, which is not a large power of 2, so traditional FFT-based STARKs cannot use M31 directly. But the circle group has order $p + 1 = 2^{31}$, a perfect power of 2, enabling FFT-like algorithms over the circle. Trace lengths of 2^{20} or 2^{25} divide evenly with no wasted bits.

These advantages compound. Implementations using M31 Circle STARKs, such as StarkWare's Stwo and Polygon's Plonky3, report order-of-magnitude speedups over provers using larger fields. The security model is unchanged: the circle structure is used for FFTs, not for cryptographic assumptions.

The Trade-off

Circle STARKs require adapting the polynomial machinery:

- Polynomials are defined over the circle group, not a multiplicative subgroup
- FRI folding uses the circle structure
- Some constraint types require reformulation

The implementation complexity is higher. But for systems targeting maximum prover speed, particularly zkVMs where prover time dominates, Circle STARKs offer a path to concrete performance improvements. Chapter 20 develops Circle FRI's folding mechanism in detail, traces the full prover pipeline over M31, and shows how Stwo achieves over 500,000 Poseidon2 hashes per second.

The Broader Lesson

Circle STARKs exemplify a general principle: *match the algebraic structure to hardware capabilities*. Traditional STARKs chose fields for mathematical convenience (large primes with smooth multiplicative order). Circle STARKs choose fields for computational efficiency (Mersenne primes with fast reduction), then build the necessary mathematical structure (the circle group) around that choice. Binius

(Chapter 26) pushes this further by working over binary tower fields, where addition is XOR and field elements match the computer's native data types. As proof systems mature, field choice increasingly reflects hardware realities rather than purely mathematical aesthetics.

Key takeaways

1. **STARKs eliminate trusted setup** by building on hash functions rather than pairings. Merkle trees provide binding commitments; FRI proves low-degree properties.
2. **Computation becomes a trace.** The state machine model represents computation as a matrix of register values over timesteps. Each column interpolates to a polynomial over a root-of-unity domain H, and uniform transition constraints relate consecutive rows via the ωX shift.
3. **The algebraic pipeline reduces all constraints to a single degree check.** Constraint satisfaction becomes polynomial divisibility (quotients), quotients batch into a composition polynomial via random weights, and FRI verifies the degree bound. Low-degree extension over $D \supset H$ ensures any violation spreads across most of D.
4. **The AIR-FRI link.** The verifier opens trace values at query points, locally recomputes the composition, and checks it matches the committed value. The same queries feed into FRI consistency checks: one query, two purposes.
5. **Trace randomization adds zero-knowledge.** Appending random rows before committing contaminates evaluations outside H, so queries in $D \setminus H$ reveal nothing about the witness. The existing constraint structure accommodates this with no changes to the vanishing polynomial.
6. **Circle STARKs unlock small-field proving.** By replacing multiplicative subgroups with the circle group, STARKs can use Mersenne primes like M_{31}, where 31-bit arithmetic and SIMD vectorization yield order-of-magnitude speedups. This is possible because STARK security depends on hash functions, not on field size.
7. **The STARK trade-off**: post-quantum security and transparency at the cost of larger proofs (tens of kilobytes versus hundreds of bytes). Hybrid architectures wrap STARKs in pairing-based proofs for on-chain verification.

Chapter 16: Σ-Protocols: The Simplest Zero-Knowledge Proofs

In 1989, a Belgian cryptographer named Jean-Jacques Quisquater faced an unusual challenge: explaining zero-knowledge proofs to his children.

The mathematics was forbidding. Goldwasser, Micali, and Rackoff had formalized the concept four years earlier, but their definitions involved Turing machines, polynomial-time simulators, and computational indistinguishability. Quisquater wanted something a six-year-old could grasp.

So he invented a cave.

> **The Children's Story**
>
> In Quisquater's tale, Peggy (the Prover) wants to prove to Victor (the Verifier) that she knows the magic word to open a door deep inside a cave. The cave splits into two paths (Left and Right) that reconnect at the magic door.
>
> Peggy enters the cave and takes a random path while Victor waits outside. Victor then walks to the fork and shouts: "Come out the Left path!"
>
> If Peggy knows the magic word, she can always comply. If she originally went Left, she walks out. If she went Right, she opens the door with the magic word and exits through the Left. Either way, Victor sees her emerge from the Left.
>
> If Peggy *doesn't* know the word, she's trapped. Half the time, Victor shouts for the path she's already on (she succeeds). Half the time, he shouts for the other side (she fails, stuck behind a locked door).
>
> They repeat this 20 times. A faker has a $(1/2)^{20} \approx$ one-in-a-million chance of consistently appearing from the correct side. But someone who knows the word succeeds every time.
>
> This story, published as "How to Explain Zero-Knowledge Protocols to Your Children," captures the essence of what we now call a Σ-protocol: **Commitment** (entering the cave), **Challenge** (Victor shouting), **Response** (appearing from the correct side). Almost all modern cryptography, from your credit card chip to your blockchain wallet, is a mathematical version of this cave.

The paper became a classic, despite the fact that most children would probably stop listening after "takes a random path" to ask what "random" means. The cave analogy appears in nearly every introductory cryptography course regardless. What makes it so powerful is that it captures the *structure* of zero-knowledge: the prover commits to a position before knowing the challenge, then demonstrates knowledge by responding correctly.

This chapter develops the mathematics behind the cave. A prover commits to something random. A verifier challenges with something random. The prover responds with something that combines both randomnesses with their secret. The verifier checks a simple algebraic equation. If it holds, accept; if not, reject.

This is a Σ-protocol. The name comes from the shape of the message flow: three arrows forming the Greek letter Σ when drawn between prover and verifier. The structure is so pervasive that it appears

everywhere cryptography touches authentication: digital signatures, identification schemes, credential systems, and as building blocks within the complex SNARKs we've studied.

Why study something so simple after the machinery of Groth16 and STARKs?

Because Σ-protocols crystallize the *core* ideas of zero-knowledge. The simulator that we'll construct, picking the response first then computing what the commitment "must have been," is the archetype of all simulation arguments. The special soundness property (that two accepting transcripts with different challenges allow witness extraction) is the template for proofs of knowledge everywhere. And the Fiat-Shamir transform, which converts interaction into non-interaction, was developed precisely for Σ-protocols.

Understand Σ-protocols, and the zero-knowledge property itself becomes clear. This chapter prepares the ground for Chapter 17, where we formalize what "zero-knowledge" means. Here, we see it in its simplest form.

The Discrete Logarithm Problem

We return to familiar ground. Chapter 6 introduced the discrete logarithm problem as the foundation for Pedersen commitments. Now it serves a different purpose: enabling proofs of knowledge.

The setting is a cyclic group $\mathbb{G}$ of prime order q with generator g. Every element $h \in \mathbb{G}$ can be written as $h = g^w$ for some $w \in \mathbb{Z}_q$. This w is the *discrete logarithm* of h with respect to g. Computing w from h is hard; computing h from w is easy. This asymmetry, the one-wayness that made Pedersen commitments binding, now enables something new.

We use multiplicative notation throughout this chapter. In practice, most implementations use elliptic curves, where the group operation is written additively: g^w becomes $w \cdot G$, $g^r \cdot g^s$ becomes $r \cdot G + s \cdot G$, and the Schnorr verification equation $g^z = a \cdot h^e$ becomes $z \cdot G = A + e \cdot H$. The mathematics is identical; only the symbols change.

The prover *knows* w. The verifier sees h but cannot compute w directly. The prover wants to convince the verifier that they know w without revealing what w is.

The naive approach fails immediately. If the prover just sends w, the verifier can check $g^w = h$, but the secret is exposed. If the prover sends nothing, the verifier has no basis for belief. There seems to be no middle ground.

Interactive proofs create that middle ground.

Schnorr's Protocol

Claus Schnorr discovered the canonical solution in 1989. The protocol is three messages, two exponentiations for the prover, two exponentiations for the verifier.

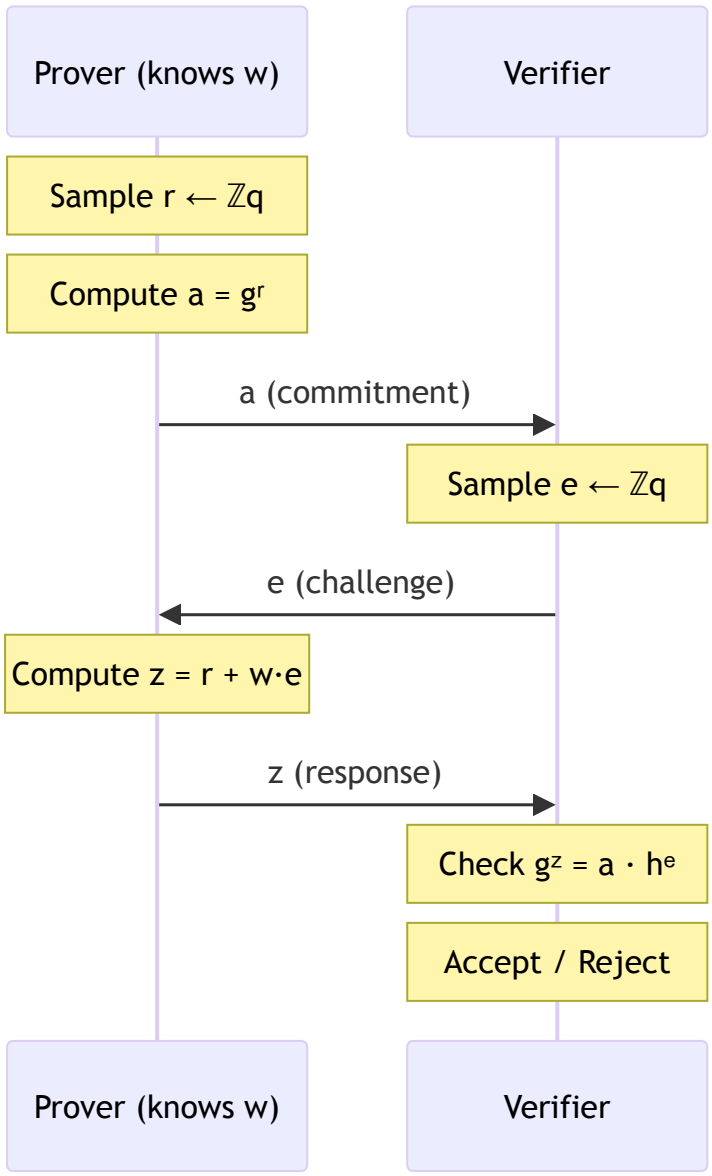
Both parties know a group $\mathbb{G}$, a generator g, and the public value $h = g^w$. The prover alone knows the witness w. The protocol proceeds in three moves:

1. **Commitment.** The prover samples a random $r \leftarrow \mathbb{Z}_q$ and computes $a = g^r$. The prover sends a to the verifier.

2. **Challenge.** The verifier samples a random $e \leftarrow \mathbb{Z}_q$ and sends e to the prover.

3. **Response.** The prover computes $z = r + w \cdot e \mod q$ and sends z to the verifier.

4. **Verification.** The verifier checks whether $g^z = a \cdot h^e$. Accept if yes, reject otherwise.



That's the entire protocol. The diagram above makes the Σ shape visible: three arrows zigzagging between prover and verifier. Let's understand why it works.

Completeness. An honest prover with the correct w always passes verification:

$$g^z = g^{r+we} = g^r \cdot g^{we} = g^r \cdot (g^w)^e = a \cdot h^e$$

The algebra is straightforward. The commitment $a = g^r$ hides r; the response $z = r + we$ reveals a linear combination of r and w; but one equation in two unknowns doesn't determine either.

Soundness. A prover who doesn't know w can cheat only by guessing the challenge e before committing. Once they send a, they're locked in. For a random e, there's exactly one z that satisfies the verification equation (namely $z = r + we$). A cheating prover who doesn't know w cannot compute this z.

More precisely: suppose a cheater could answer two different challenges e_1 and e_2 for the same commitment a. Then we'd have:

$$g^{z_1} = a \cdot h^{e_1} \quad \text{and} \quad g^{z_2} = a \cdot h^{e_2}$$

Dividing these equations:

$$g^{z_1 - z_2} = h^{e_1 - e_2}$$

The extractor computes w from the known exponents:

$$w = \frac{z_1 - z_2}{e_1 - e_2} \mod q$$

This is well-defined since $e_1 \neq e_2$ and q is prime. To verify: $g^w = (g^{z_1}/g^{z_2})^{1/(e_1 - e_2)} = (ah^{e_1}/ah^{e_2})^{1/(e_1 - e_2)} = h^{(e_1 - e_2)/(e_1 - e_2)} = h$. $\square$

This extraction is a proof technique, not something the verifier can actually do. In a real execution, the prover answers only one challenge, so w stays hidden. But the argument shows that anyone who *could* answer two challenges for the same commitment must already know w. Contrapositively, someone who doesn't know w cannot reliably answer even a single random challenge. This property is called **special soundness**: two accepting transcripts with different challenges allow extracting the witness. It is why Σ-protocols prove you *know* something, not merely that something exists.

There is a clean geometric way to see this. Schnorr's protocol is secretly proving you know the equation of a line. In $z = r + w \cdot e$, think of w as the slope and r as the y-intercept. The prover commits to the intercept (r, hidden as $a = g^r$). The verifier picks an x-coordinate (e). The prover reveals the y-coordinate (z). In a single execution, the verifier learns one point (e, z) on the line, which is consistent with infinitely many slopes, so w stays hidden. But if an extractor could rewind and obtain a second point (e_2, z_2) on the same line (same intercept r, since a was fixed), two points would determine the slope.

Honest-verifier zero-knowledge (HVZK). Here is where things become subtle. What follows is a restricted form of zero-knowledge that assumes the verifier behaves honestly (samples e uniformly at random). Chapter 17 formalizes the full definition, which must handle malicious verifiers. For now, consider a simulator that doesn't know w but wants to produce a valid-looking transcript (a, e, z). The simulator proceeds *backwards*:

1. Sample $e \leftarrow \mathbb{Z}_q$ (the challenge first!)

2. Sample $z \leftarrow \mathbb{Z}_q$ (the response, uniform and independent)
3. Compute $a = g^z \cdot h^{-e}$ (the commitment that *makes* the equation hold)

Check: $g^z = a \cdot h^e = g^z h^{-e} \cdot h^e = g^z$.

The transcript (a, e, z) is valid. And its distribution is identical to a real transcript:

- In a real transcript: e is uniform (verifier's randomness), $z = r + we$ is uniform (because r is uniform), and $a = g^r$ is determined.
- In a simulated transcript: e is uniform (simulator's choice), z is uniform (simulator's choice), and $a = g^z h^{-e}$ is determined.

Both distributions have e and z uniform and independent, with a determined by the verification equation. They are identical.

More formally, let $\mathcal{T}_{\text{real}}$ denote the distribution of real transcripts and $\mathcal{T}_{\text{sim}}$ the simulator's output. Both are distributions over $\mathbb{G} \times \mathbb{Z}_q \times \mathbb{Z}_q$. In $\mathcal{T}_{\text{real}}$: $(a, e, z) = (g^r, e, r + we)$ where $r, e \xleftarrow{\$} \mathbb{Z}_q$. In $\mathcal{T}_{\text{sim}}$: $(a, e, z) = (g^z h^{-e}, e, z)$ where $e, z \xleftarrow{\$} \mathbb{Z}_q$. In both cases, e and z are uniform and independent (in the real case, $z = r + we$ is uniform because r is uniform and independent of e). The value a is then uniquely determined by the verification equation $g^z = ah^e$. Since both distributions have identical marginals on (e, z) and a is a deterministic function of (e, z), we have $\mathcal{T}_{\text{real}} \equiv \mathcal{T}_{\text{sim}}$ (perfect equality, not just computational indistinguishability).

The transcript reveals nothing about w that the verifier couldn't have generated alone.

In real execution, events unfold forward: Commitment → Challenge → Response. The simulator reverses this. It picks the answer first (z), invents a question that fits (e), then back-calculates what the commitment "must have been" ($a = g^z h^{-e}$). This temporal reversal is invisible in the final transcript. Anyone looking at (a, e, z) cannot tell whether it was produced forward (by someone who knows w) or backward (by someone who doesn't). If a transcript can be faked without the secret, then having the secret cannot be what makes the transcript convincing. The transcript itself carries no information about w.

A Concrete Computation

Let's trace through Schnorr's protocol with actual numbers, then see how a simulator fakes a transcript.

Work in $\mathbb{Z}_{11}^*$ (order 10) with generator $g = 2$. The prover knows $w = 6$, and the public value is $h = 2^6 \equiv 9 \pmod{11}$.

Real transcript: The prover samples $r = 4$, computes $a = 2^4 \equiv 5$, and sends it. The verifier sends challenge $e = 7$. The prover computes $z = r + we = 4 + 42 = 46 \equiv 6 \pmod{10}$ (note: we reduce

modulo the group order 10, not the prime 11). Verification: $g^z = 2^6 \equiv 9$ and $a \cdot h^e = 5 \cdot 9^7 \equiv 5 \cdot 4 \equiv 9 \pmod{11}$. Both sides match.

The transcript is $(a, e, z) = (5, 7, 6)$.

Simulated transcript: A simulator who doesn't know w picks $e = 7$ and $z = 6$ (both uniform), then computes $a = g^z \cdot h^{-e} = 2^6 \cdot 9^{-7} \equiv 9 \cdot 4^{-1} \equiv 9 \cdot 3 \equiv 5 \pmod{11}$ (since $4 \cdot 3 = 12 \equiv 1$). The simulated transcript is $(5, 7, 6)$, identical to the real one. The simulator produced a valid proof without knowing $w = 6$. This is HVZK in action: the transcript carries no information about the witness.

Pedersen Commitments and Σ-Protocols

Schnorr's protocol proves knowledge of a single discrete log. But in practice, we often need to prove knowledge of values hidden inside commitments. Chapter 6 introduced Pedersen commitments: $C = g^m h^r$ commits to message m with blinding factor r, where g, h are generators with unknown discrete log relation. Σ-protocols let us go further and *prove things* about committed values.

This is not a coincidence. Schnorr's protocol and Pedersen commitments are algebraically the same construction. In Schnorr, the prover commits to $a = g^r$ and later reveals $z = r + we$ (a linear combination of the randomness and the secret). In Pedersen, the committer computes $C = g^m h^r$ (a linear combination of two generators weighted by the message and randomness). Both rely on the same hardness assumption; both achieve the same hiding property.

Recall from Chapter 6: a Pedersen commitment $C = g^m h^r$ is perfectly hiding (reveals nothing about m) and computationally binding (opening to a different value requires solving discrete log). The additive homomorphism $C_1 \cdot C_2 = g^{m_1+m_2} h^{r_1+r_2}$ lets us compute on committed values.

What Chapter 6 couldn't address: how does a prover demonstrate they *know* the opening (m, r) without revealing it? This is precisely what Σ-protocols provide.

Proving Knowledge of Openings

Schnorr handles one exponent; Pedersen commitments involve two: $C = g^m h^r$. To prove knowledge of the opening (m, r), we need the two-dimensional generalization. The structure mirrors Schnorr exactly (commit, challenge, respond) but now with two secrets handled in parallel:

1. **Commitment.** Prover samples $d, s \leftarrow \mathbb{Z}_q$ and sends $a = g^d h^s$.
2. **Challenge.** Verifier sends random $e \leftarrow \mathbb{Z}_q$.
3. **Response.** Prover sends $z_1 = d + m \cdot e$ and $z_2 = s + r \cdot e$.
4. **Verification.** Check $g^{z_1} h^{z_2} = a \cdot C^e$.

This is just two Schnorr protocols glued together. One proves knowledge of the message part (m, committed via g^m), the other proves knowledge of the randomness part (r, committed via h^r). The same challenge e binds them, ensuring the prover cannot mix-and-match unrelated values.

The analysis parallels Schnorr's protocol:

Completeness.

$$g^{z_1}h^{z_2} = g^{d+me}h^{s+re} = g^d h^s \cdot (g^m h^r)^e = a \cdot C^e \checkmark$$

Special soundness. Two transcripts with the same a but different challenges e_1, e_2 yield:

$$g^{z_1^{(1)}-z_1^{(2)}} h^{z_2^{(1)}-z_2^{(2)}} = C^{e_1-e_2}$$

From which both m and r can be extracted.

HVZK. Simulator picks e, z_1, z_2 uniformly, sets $a = g^{z_1}h^{z_2} \cdot C^{-e}$.

The prover demonstrates knowledge of the commitment opening without revealing what that opening is.

Proving Relations on Committed Values

The homomorphic property enables proving statements about committed values without revealing them.

For addition, given commitments C_1, C_2, C_3, we can prove that the committed values satisfy $m_1 + m_2 = m_3$.

Consider the product $C_1 \cdot C_2 \cdot C_3^{-1}$. Expanding the Pedersen structure:

$$C_1 \cdot C_2 \cdot C_3^{-1} = g^{m_1}h^{r_1} \cdot g^{m_2}h^{r_2} \cdot g^{-m_3}h^{-r_3} = g^{m_1+m_2-m_3} \cdot h^{r_1+r_2-r_3}$$

If the relation $m_1 + m_2 = m_3$ holds, the g exponent vanishes:

$$C_1 \cdot C_2 \cdot C_3^{-1} = g^0 \cdot h^{r_1+r_2-r_3} = h^{r_1+r_2-r_3}$$

The combined commitment collapses to a pure power of h. To prove the relation holds, the prover demonstrates knowledge of this exponent $r_1 + r_2 - r_3$ (a single Schnorr proof with base h and public element $C_1 \cdot C_2 \cdot C_3^{-1}$).

Multiplication is harder. Pedersen commitments aren't multiplicatively homomorphic. Given $C_1 = g^{m_1}h^{r_1}, C_2 = g^{m_2}h^{r_2}, C_3 = g^{m_3}h^{r_3}$, how do we prove $m_1 \cdot m_2 = m_3$?

The trick is to change bases. Observe that:

$$g^{m_3} = g^{m_1 \cdot m_2} = (g^{m_1})^{m_2}$$

If $C_3 = g^{m_1 m_2} h^{r_3}$, then C_3 can also be viewed as:

$$C_3 = (g^{m_1})^{m_2} h^{r_3}$$

Now substitute $g^{m_1} = C_1 \cdot h^{-r_1}$:

$$C_3 = (C_1 \cdot h^{-r_1})^{m_2} h^{r_3} = C_1^{m_2} \cdot h^{r_3 - r_1 m_2}$$

This expresses C_3 as a "Pedersen commitment with base C_1" to the value m_2 with blinding factor $r_3 - r_1 m_2$.

The prover runs three parallel Σ-protocols:

1. Prove knowledge of (m_1, r_1) opening C_1 (standard Pedersen opening)
2. Prove knowledge of (m_2, r_2) opening C_2 (standard Pedersen opening)
3. Prove knowledge of $(m_2, r_3 - r_1 m_2)$ opening C_3 with respect to bases (C_1, h)

The third proof *links* to the second because the same m_2 appears. This linking requires careful protocol design, but the core technique is Σ-protocol composition with shared secrets.

Fiat-Shamir: From Interaction to Non-Interaction

Interactive proofs are impractical for many applications. A signature scheme cannot require real-time communication with every verifier. A blockchain proof must be verifiable by anyone, at any time, without the prover present.

The **Fiat-Shamir transform** removes interaction. The idea is elegant: replace the verifier's random challenge with a hash of the transcript.

In Schnorr's protocol:

1. Prover computes $a = g^r$
2. Instead of waiting for verifier's e, prover computes $e = H(a)$ (or $H(g, h, a)$ for domain separation)
3. Prover computes $z = r + we$
4. Proof is (a, z)

Verification:

1. Recompute $e = H(a)$
2. Check $g^z = a \cdot h^e$

The transform works because H is modeled as a **random oracle**: a function that returns uniformly random output for each new input. The prover cannot predict $H(a)$ before choosing a. Once a is fixed, the hash determines e deterministically. The prover faces a random challenge, just as in the interactive version.

In practice, H is a cryptographic hash function like SHA-256. The random oracle model is an idealization (hash functions aren't truly random functions) but the heuristic is empirically robust for well-designed protocols.

Schnorr signatures are the direct application. Given secret key w and public key $h = g^w$:

- **Sign message M:** Compute $a = g^r$, $e = H(h, a, M)$, $z = r + we$. Signature is (a, z).
- **Verify:** Check $g^z = a \cdot h^e$ where $e = H(h, a, M)$.

Schnorr patented his signature scheme in 1989 (U.S. Patent 4,995,082). NIST needed a standard and designed DSA, later ECDSA, specifically to work around the patent. The result was a signing equation $s = k^{-1}(H(m) + rx)$ that includes a modular inversion k^{-1}. This non-linearity is the algebraic cost of the workaround: you cannot simply add ECDSA signatures, because the inverses don't combine.

The patent expired in 2008, and Schnorr signatures finally entered widespread use as EdDSA (Ed25519), now standard in TLS, SSH, and cryptocurrency systems. Bitcoin launched in 2009, but ECDSA was already the entrenched standard, so Satoshi used it. Ethereum launched in 2015 with ECDSA as well: audited Schnorr implementations on secp256k1 simply did not exist yet, and Ethereum still uses ECDSA today. It took until the 2021 Taproot upgrade for Bitcoin to adopt Schnorr. The linearity of $z = r + we$ enables what ECDSA cannot:

- **Batch verification**: check many signatures faster than individually by taking random linear combinations (Schwartz-Zippel ensures invalid signatures can't cancel)
- **Native aggregation**: multiple signers can combine signatures into one. MuSig2 produces a single 64-byte signature for n parties that verifies against an aggregate public key
- **ZK-friendliness**: no modular inversions, so Schnorr verification is cheap inside arithmetic circuits

Composition: AND and OR

Σ-protocols compose cleanly, enabling proofs of complex statements from simple building blocks.

For AND composition, to prove "I know w_1 such that $h_1 = g^{w_1}$ AND w_2 such that $h_2 = g^{w_2}$":

1. Run both protocols in parallel with independent commitments

2. Use the same challenge e for both
3. Check both verification equations

If the prover knows both witnesses, they can respond to any challenge. If they lack either witness, they can't respond correctly.

OR composition is more subtle. To prove "I know w_1 OR w_2" (without revealing which):

1. For the witness you *don't* know, simulate a transcript (a_i, e_i, z_i) (using the honest-verifier simulator from the zero-knowledge property)
2. For the witness you *do* know, commit honestly to a_j
3. When you receive the verifier's challenge e, set $e_j = e - e_i$
4. Respond honestly to e_j using your witness

The verifier checks:

- Both verification equations hold
- $e_1 + e_2 = e$

As an example, suppose Alice knows the discrete log of $h_1 = g^{w_1}$ but not h_2. She wants to prove she knows at least one of them.

1. **Simulate the unknown:** Alice picks $e_2 = 7$ and $z_2 = 13$ at random, then computes $a_2 = g^{z_2} h_2^{-e_2} = g^{13} h_2^{-7}$. This is a valid-looking transcript for h_2.
2. **Commit honestly for the known:** Alice picks $r_1 = 5$ and computes $a_1 = g^{r_1} = g^5$. She sends (a_1, a_2) to the verifier.
3. **Split the challenge:** The verifier sends $e = 19$. Alice sets $e_1 = e - e_2 = 19 - 7 = 12$.
4. **Respond honestly:** Alice computes $z_1 = r_1 + w_1 \cdot e_1 = 5 + w_1 \cdot 12$ and sends $(e_1, z_1, e_2, z_2) = (12, z_1, 7, 13)$.

The verifier checks $g^{z_1} = a_1 \cdot h_1^{e_1}$ and $g^{z_2} = a_2 \cdot h_2^{e_2}$, plus $e_1 + e_2 = 19$. Both equations hold. The verifier cannot tell which transcript was simulated; the simulated (a_2, e_2, z_2) is statistically identical to an honest execution.

You can prove you know one of two secrets without revealing which. Ring signatures, anonymous credentials, and many privacy-preserving constructions build on this technique.

Key takeaways

1. **Three messages suffice** for zero-knowledge proofs of knowledge. Commit → Challenge → Response. The commitment must come before the challenge; reversing this order destroys soundness.

2. **Special soundness**: two accepting transcripts with different challenges enable witness extraction. This makes Σ-protocols *proofs of knowledge*, not merely proofs of existence.
3. **Zero-knowledge via simulation**: pick the challenge and response first, compute what the commitment must have been. If a transcript can be faked without the secret, the transcript carries no information about the secret.
4. **Schnorr is the archetype**. Every Σ-protocol in this chapter is a variation on $z = r + we$: Pedersen openings run two Schnorr proofs in parallel, relations on committed values reduce to Schnorr proofs after algebraic simplification.
5. **Fiat-Shamir** removes interaction by hashing the commitment to derive the challenge. This yields Schnorr signatures and non-interactive proofs.
6. **Composition** builds complex proofs from simple ones. AND runs protocols in parallel with a shared challenge. OR uses simulation for the unknown witness; the verifier cannot tell which branch is real.
7. **Minimal assumptions**: Σ-protocols require only the discrete logarithm assumption. No pairings, no trusted setup, no hash functions beyond Fiat-Shamir.

Chapter 17: The Zero-Knowledge Property

In 1982, Shafi Goldwasser and Silvio Micali submitted a paper to STOC proposing that a proof could convince a verifier of a statement's truth while revealing nothing beyond that single bit: true or false. The program committee rejected it. The concept seemed contradictory. A proof, by its nature, is a demonstration: it convinces by showing. How could showing suffice for conviction while simultaneously revealing nothing?

They persisted. The paper, expanded with Charles Rackoff, was published in 1985 as "The Knowledge Complexity of Interactive Proof Systems." It won the Gödel Prize in 1993. Goldwasser and Micali received the Turing Award in 2012. The reviewers' skepticism was not foolish; it reflected a genuine conceptual difficulty that the paper resolved.

Their resolution was a definition so clean it still underlies every modern proof system. A proof reveals nothing if everything the verifier sees could have been produced by a *simulator* who knows nothing about the secret. If real and simulated transcripts are indistinguishable, the real one carries no information about the witness. This immediately raises the question the rest of the chapter answers: if a fake transcript looks identical to a real one, what did the prover actually contribute?

The definition is deceptively simple. The consequences are not. Simulation is not a property that protocols possess by default. The sum-check protocol, central to this book, leaks witness data through its

round polynomials: no simulator can fake them without the witness, because the protocol was never designed to allow it. Understanding when simulation is possible, what makes it possible, and what flavors of indistinguishability suffice is the work of this chapter.

The Simulation Argument

We have already seen a simulator in action. Chapter 16 constructed one for the Schnorr protocol: to produce a valid transcript (a, e, z) without knowing the witness w, pick e and z first (both uniformly random), then compute $a = g^z h^{-e}$. The transcript satisfies the verification equation by construction, and its distribution matches a real transcript exactly.

What does this buy us? Recall that the *witness* w is the prover's secret (a private key, a satisfying assignment, the preimage of a hash), and the *transcript* is the full sequence of messages exchanged during the protocol. If someone who never touched w can produce transcripts identical to a real prover's, then the transcript itself cannot encode anything about w, even though the real prover used w to compute it. The computation depends on the witness; the distribution does not. The verifier, holding only the transcript, learns nothing. This reasoning generalizes beyond Schnorr. A proof system is zero-knowledge if a simulator (an efficient algorithm with no access to the witness) can produce transcripts indistinguishable from real protocol executions. This is the **simulation paradigm**. In short: a simulator is a machine that takes only the public statement (everything except the witness), generates challenges on behalf of the verifier, fabricates prover responses, and outputs a complete transcript whose distribution is indistinguishable from a real execution.

To make the argument precise: suppose the verifier could extract some information I about the witness from a real transcript. The simulator doesn't know the witness, so its transcript cannot possibly encode I. But the two transcripts are indistinguishable, so any extraction procedure that works on real transcripts must also work on simulated ones, yet simulated ones contain no witness information to extract. The assumption that I is extractable leads to a contradiction. Real transcripts don't leak I. The proof is convincing precisely because it could have been fabricated, and this indistinguishability is not a flaw to be patched; it is the definition of success.

The Graph Non-Isomorphism Protocol

We claimed the simulation paradigm is general. To build conviction, let's see it work in a protocol with entirely different structure: no group elements, no algebraic equations, just graphs and permutations. The Graph Non-Isomorphism protocol from Chapter 1 makes the mechanics of simulation visible without any algebraic machinery to hide behind.

Both parties see two graphs G_0 and G_1 (the public statement). The prover claims they are non-isomorphic: no permutation π of the vertex set satisfies $G_0 = \pi(G_1)$. Graph Isomorphism is in NP

(the permutation itself is the witness), but Graph Non-Isomorphism is not known to be in NP. There is no obvious short certificate for the *absence* of an isomorphism, and the verifier cannot efficiently check the claim on her own. This is precisely what makes GNI a natural candidate for interactive proofs.

The protocol works as follows. The verifier picks a secret bit $b \in \{0, 1\}$, applies a random permutation π to G_b, and sends $H = \pi(G_b)$ to the prover. The prover must identify which graph H came from. If the graphs are truly non-isomorphic, they have different structural fingerprints (spectrum, degree sequence, triangle counts), so an unbounded prover can determine b with certainty and sends back $b' = b$. The key observation: if the graphs *were* isomorphic, $\pi(G_0)$ and $\pi(G_1)$ would be identically distributed, and no prover could do better than guessing b correctly with probability $1/2$. Repeating k times drives the soundness error to 2^{-k}. Success at this task therefore proves non-isomorphism.

Now consider what the verifier actually sees after a successful execution:

- The challenge H that she generated herself
- The bit b' that matches her secret b

But b was her own random choice. H was her own computation. The prover's response $b' = b$ just echoes her own randomness back. The transcript (H, b') contains nothing the verifier didn't already know.

A simulator can exploit this. Given only the graphs G_0, G_1 (not the prover's ability to distinguish them), it plays both sides of the conversation:

1. Pick $b \leftarrow \{0, 1\}$ uniformly at random (playing the verifier)
2. Pick π uniformly from permutations of the vertex set (playing the verifier)
3. Compute $H = \pi(G_b)$ (playing the verifier)
4. Output the transcript (H, b) (playing the prover, using the b it already chose)

The simulator does not need to distinguish the graphs. It knows b because it generated b itself. A real cheating prover, facing a live verifier, would have to guess b from H alone (and could do no better than $1/2$). The simulator sidesteps this entirely by controlling both sides. The resulting distribution over (H, b) is identical to what an honest verifier would see in a real execution. The simulated and real distributions are not merely close; they are identical. This is **perfect zero-knowledge**: the statistical distance between real and simulated transcripts is exactly zero.

Simulation and Polynomial Commitments

The algebraic protocols that dominate the rest of this book share a structure that GNI lacks: a commit → challenge → respond sequence. A real prover commits to a polynomial $p(X)$, receives a verifier-chosen evaluation point z, and responds with $v = p(z)$. The simulator, just as in Schnorr, reverses this:

it picks z and v first, then constructs a commitment consistent with these choices. The commitment "could have been" to any polynomial that passes through (z, v), because a single evaluation does not determine a polynomial. One (e, z) pair in Schnorr is consistent with infinitely many secrets w; one evaluation (z, v) is consistent with infinitely many polynomials. The simulator exploits this ambiguity. The real prover is bound by her earlier commitment; the simulator is free to work backward from the challenge.

This is why KZG requires the verifier to choose z *after* the commitment, why FRI queries come after the oracle is fixed, and why Fiat-Shamir hashes the commitment before deriving challenges. The temporal ordering (commit → challenge → respond) is what makes the live proof convincing. The simulator's freedom from that ordering is what makes the transcript uninformative.

Formal Definition

Let $(\mathcal{P}, \mathcal{V})$ be an interactive proof system for a language $\mathcal{L}$ (recall from Chapter 1: a set of yes-instances for some decision problem). On input $x \in \mathcal{L}$, the prover $\mathcal{P}$ holds a witness w; the verifier $\mathcal{V}$ sees only x.

The verifier's view consists of:

1. The statement x
2. The verifier's random coins r
3. All messages received from the prover

We write $\text{View}_{\mathcal{V}}(\mathcal{P}(w) \leftrightarrow \mathcal{V})(x)$ for this random variable.

Definition (Zero-Knowledge). The proof system is **zero-knowledge** if there exists a probabilistic polynomial-time algorithm $\mathcal{S}$ (the simulator) such that for all $x \in \mathcal{L}$:

$$\text{View}_{\mathcal{V}}(\mathcal{P}(w) \leftrightarrow \mathcal{V})(x) \approx \mathcal{S}(x)$$

The symbol $\approx$ denotes indistinguishability; its precise meaning yields three flavors.

Three Flavors of Zero-Knowledge

Perfect zero-knowledge (PZK). The distributions are identical:

$$\text{View}_{\mathcal{V}} \equiv \mathcal{S}(x)$$

No adversary, even with unlimited computational power, can distinguish real from simulated transcripts. The two distributions have zero statistical distance.

This is the strongest notion. The Schnorr protocol (Chapter 16) achieves PZK against honest verifiers: the simulator's output (a, e, z) has exactly the same distribution as a real transcript.

Statistical zero-knowledge (SZK). The distributions are statistically close:

$$\Delta(\text{View}_{\mathcal{V}}, \mathcal{S}(x)) \leq \text{negl}(\lambda)$$

where the **statistical distance** (or total variation distance) between distributions P and Q is defined as:

$$\Delta(P, Q) = \frac{1}{2} \sum_{x} |P(x) - Q(x)| = \max_{S} |P(S) - Q(S)|$$

This is the maximum advantage any distinguisher (even computationally unbounded) can achieve. An unbounded adversary might distinguish the distributions, but only with probability $2^{-\Omega(\lambda)}$ (effectively never).

SZK allows for protocols where perfect simulation is impossible but the gap is cryptographically small. To see how this arises in practice, return to Schnorr. The simulator picks z uniformly from $\mathbb{Z}_q$ and achieves a perfect match. But suppose the implementation samples z uniformly from $\{0, \ldots, 2^{256}-1\}$ instead of $\{0, \ldots, q-1\}$ (a common shortcut when $q \approx 2^{256}$). The real transcript samples r from $\mathbb{Z}_q$, so $z = r + we$ is uniform over $\mathbb{Z}_q$; the simulated transcript has z uniform over a slightly larger range. The statistical distance is on the order of $(2^{256} - q)/2^{256}$, which is negligible when q is close to 2^{256}. No unbounded adversary can distinguish with non-negligible advantage, but the distributions are no longer identical. This is SZK, not PZK.

Computational zero-knowledge (CZK). No efficient algorithm can distinguish the distributions:

$$\text{View}_{\mathcal{V}} \overset{c}{\approx} \mathcal{S}(x)$$

The distributions might be statistically far apart, but every polynomial-time distinguisher's advantage is negligible. Security relies on computational hardness; an unbounded adversary could distinguish.

CZK is the weakest but most practical notion. Modern SNARKs typically achieve CZK. The simulator might use pseudorandom values where the real protocol uses true randomness; distinguishing requires breaking the underlying assumption.

Honest Verifiers and Malicious Verifiers

The definition above assumes the verifier follows the protocol honestly. What if she doesn't?

Honest-verifier zero-knowledge (HVZK). Zero-knowledge is guaranteed only when the verifier follows the protocol as specified, hence "honest." The simulator can hardcode this known strategy into its construction. For Schnorr, the honest verifier samples e uniformly and independently of a, and the simulator exploits exactly this: it picks (e, z) first, then derives $a = g^z h^{-e}$. The independence of e from a is what makes the reversal work. If the verifier instead chose $e = f(a)$ for some function f, the simulator would need to find a such that $f(a)$ equals the e it already committed to, which it generally cannot do.

Malicious-verifier zero-knowledge. The simulator must produce indistinguishable output against *any* efficient verifier strategy $\mathcal{V}^*$, including:

- Adversarial challenge selection
- Auxiliary information from other sources
- Arbitrary protocol deviations

To see what malicious verification looks like concretely, consider the Graph Non-Isomorphism protocol again. An honest verifier sends $H = \pi(G_b)$ for her secret b. But a malicious verifier could send some other graph H' (perhaps one she suspects is isomorphic to G_0 but isn't sure). The all-powerful prover will correctly identify whether H' matches G_0, G_1, or neither. The verifier learns something she couldn't efficiently compute herself!

The protocol is HVZK but not malicious-verifier ZK. The prover, dutifully answering whatever question is posed, inadvertently becomes an oracle for graph isomorphism.

Closing this gap requires additional machinery:

- *Coin-flipping protocols* force the verifier to commit to her randomness before seeing the prover's messages. The verifier's challenges become unpredictable even to her.
- *Trapdoor commitments* let the simulator "equivocate": commit to one value, then open to another after seeing the verifier's behavior.
- *The Fiat-Shamir transform* eliminates interaction entirely. With no verifier messages, there's no room for malicious behavior. The simulator controls the random oracle and programs it as needed.

Non-interactive proofs (after Fiat-Shamir) largely dissolve the HVZK/malicious distinction. The "verifier" merely checks a static proof string.

For malicious verifiers in interactive protocols, the simulator often needs a stronger technique: **rewinding**. Rather than constructing the transcript in one shot (as Schnorr's simulator does), it runs the verifier multiple times, replaying from an earlier state with fresh randomness until it finds a challenge it can handle. Rewinding is a proof technique, not a real capability: it shows that the transcript *could* have been generated without the witness, even though no real prover could rewind a live verifier.

This brings us back to the question posed in the introduction: if a simulator can produce valid transcripts without the witness, what did the prover actually contribute? The answer is not data but *compliance*: she demonstrates she can respond correctly to challenges she could not have predicted. That is soundness. Zero-knowledge is the other side of the same coin: the simulator's success shows that the static transcript, stripped of temporal ordering, contains no extractable information about the witness. The two properties coexist because they concern different things. Soundness is about the live process (commit, then challenge, then respond). Zero-knowledge is about the information content of the record. The simulator can fake the record precisely because it is free from the ordering that makes the process convincing.

The Limits of Zero-Knowledge

Perfect and statistical zero-knowledge seem strictly stronger than computational. Are they always preferable?

No. There are limits.

Theorem (Fortnow, Aiello-Håstad). Any language with a statistical zero-knowledge proof lies in AM$\cap$ coAM.

The class AM (Arthur-Merlin) consists of languages decidable by a two-move interactive proof in which the verifier's coins are public: the verifier sends a random string, the prover responds, and the verifier decides deterministically. Unlike IP, where the verifier's randomness is private, AM exposes it to the prover. The class coAM contains languages whose complements are in AM. Graph Non-Isomorphism, the protocol we studied earlier, is a natural example of a problem in AM $\cap$ coAM.

The intersection AM $\cap$ coAM is believed to be much smaller than NP. Under standard complexity-theoretic conjectures, it contains no NP-complete problems. The implication is stark: statistical zero-knowledge proofs for NP-complete problems likely do not exist.

The intuition is that statistical zero-knowledge is *too good* at hiding. If a simulator can reproduce the verifier's view without the witness, and no unbounded distinguisher can tell the difference, then the proof isn't leveraging the witness in any meaningful way. An all-powerful observer could use the simulator itself to decide membership: simulate the transcript, check if the distribution is close to what a real execution would produce, and conclude whether $x \in \mathcal{L}$. This effectively places both $\mathcal{L}$ and its complement in AM. For NP-hard problems, where the witness should be "hard to avoid using," this is too much to ask.

The way forward is to relax both soundness and zero-knowledge:

- **Computational soundness (arguments):** Security against cheating provers who are computationally bounded.

- **Computational zero-knowledge:** Security against distinguishers who are computationally bounded.

Modern SNARKs take both paths. They are *arguments* (computationally sound) with *computational zero-knowledge*. This combination enables practical ZK proofs for arbitrary computations, including NP-complete problems and beyond.

> **Witness Indistinguishability**
>
> Sometimes, full zero-knowledge is too expensive or impossible to achieve. A weaker but often sufficient property is **Witness Indistinguishability (WI)**. This guarantees that if there are multiple valid witnesses (e.g., two different private keys that both sign the same message, or two different paths through a maze), the verifier cannot tell which one the prover used.
>
> WI doesn't promise that the verifier learns *nothing*; it only promises they can't distinguish *which* witness was used. For many privacy applications (anonymous credentials, ring signatures), WI suffices and is easier to achieve than full ZK.

Zero-Knowledge in the Wild: Sum-Check

Let's ground this in the core protocol of the book. The sum-check protocol proves:

$$H = \sum_{b \in \{0,1\}^n} g(b)$$

In each round, the prover sends a univariate polynomial $g_i(X_i)$, the restriction of g to a partial evaluation. The verifier checks degree bounds and eventually evaluates g at a random point.

Is sum-check zero-knowledge? Not inherently. The univariate polynomials g_i reveal partial information about g. If g encodes secret witness data, this information leaks. For applications where g is derived from public inputs (verifiable computation on public data), this leakage is harmless. For private-witness applications, we need modifications.

Several masking techniques (developed in Chapter 18) add zero-knowledge to sum-check:

- Add random low-degree polynomials that cancel in the sum
- Commit to intermediate values instead of revealing them
- Use randomization to hide the structure of g

Zero-knowledge is a *system-level* property, not a per-protocol property. We can compose non-ZK building blocks (sum-check, FRI, polynomial commitments) into ZK systems by carefully controlling what the verifier sees.

> **Zero-knowledge vs. knowledge soundness**
>
> This chapter has focused on what the *verifier* learns. An orthogonal question is what the *prover* demonstrates. A proof system can be zero-knowledge without being a proof of knowledge (GNI proves membership in a language but extracts no witness), and a proof of knowledge without being zero-knowledge (the prover could send the witness in the clear). These are independent axes: zero-knowledge constrains the verifier's view, knowledge soundness (Chapter 16) constrains the prover's ability to cheat without knowing a witness. Practical SNARKs target both, but they are achieved by separate mechanisms: simulation for the former, extraction for the latter.

Auxiliary Input

If zero-knowledge is a system-level property achieved by composing building blocks, we need a definition that survives composition. The standard simulation definition assumes the verifier starts with only the public statement. But when a ZK proof runs as a subroutine in a larger protocol, the verifier may carry information from earlier stages: an IP address, previous proofs, partial knowledge of the secret from another source. A secure ZK protocol must ensure that even with this extra context, the proof leaks nothing *new*.

Definition (Auxiliary-Input ZK). A protocol is auxiliary-input zero-knowledge if for every efficient verifier $\mathcal{V}^*$ with auxiliary input z:

$$\mathsf{View}_{\mathcal{V}^*(z)}(\mathcal{P}(w) \leftrightarrow \mathcal{V}^*(z))(x) \approx \mathcal{S}(x, z)$$

The simulator receives the same auxiliary input z as the verifier. The key requirement: whatever the verifier knew beforehand, the proof adds nothing to it.

This definition handles composed protocols. Even if the verifier has side information about the statement or witness, the proof reveals nothing new. The simulator, given the same side information, produces indistinguishable transcripts.

Auxiliary-input ZK is necessary for security in complex systems where many proofs interleave.

Key takeaways

1. **Zero-knowledge** means existence of a simulator: an efficient algorithm that, without the witness, produces transcripts indistinguishable from real executions. If the transcript could have been fabricated, it carries no information about the witness.

2. **Three flavors**: Perfect (identical distributions), Statistical (negligible statistical distance), Computational (no efficient distinguisher). Modern SNARKs typically achieve computational ZK.
3. **HVZK vs. malicious-verifier ZK**: HVZK assumes the verifier follows the protocol; malicious-verifier ZK protects against arbitrary verifier strategies. Non-interactive proofs (post Fiat-Shamir) largely collapse this distinction.
4. **Simulation does not break soundness.** The simulator works offline, fabricating transcripts of true statements. A cheating prover faces a live verifier on false statements. Rewinding (the simulator's key technique) is a proof method, not a real capability.
5. **Limits of SZK**: Statistical zero-knowledge proofs exist only for languages in AM ∩ coAM, likely excluding NP-complete problems. Computational ZK, paired with computational soundness, sidesteps this barrier.
6. **Sum-check is not inherently ZK**: The round polynomials leak witness information. Masking techniques (Chapter 18) restore privacy. Zero-knowledge is a system-level property, not a per-protocol property.
7. **Auxiliary-input ZK** ensures security under composition: even when the verifier carries side information from other protocol stages, the proof leaks nothing new.

Chapter 18: Making Proofs Zero-Knowledge

A conventional proof convinces by exposing its internals. The verifier sees intermediate values, checks each step, and traces the chain of reasoning from hypothesis to conclusion. Conviction comes from transparency: every piece of the argument is laid bare for inspection.

Zero-knowledge proofs must convince without this transparency. The verifier still receives messages, checks relationships, and follows a protocol. But the values she sees are randomized so that they carry no information about the witness. She inspects a full transcript and becomes convinced the statement is true, yet the transcript could have come from any valid witness, or from no witness at all (a simulator). The challenge is preserving the structure that makes verification work while destroying the information that would make the witness recoverable.

This requires care.

Most proof systems were not designed with privacy in mind. The interactive proofs of the 1980s and 1990s were built to make verification cheap: a weak verifier checking claims from a powerful prover. The sum-check protocol, GKR, and the algebraic machinery underlying modern SNARKs all emerged from complexity theory, where the goal was efficient verification, not confidential computation. Privacy became necessary only later, as these tools migrated from theory to practice and applications

like blockchain transactions, private credentials, and anonymous voting demanded that proofs reveal nothing beyond validity. The result is a retrofit problem: taking elegant machinery built for transparency and making it opaque.

We saw zero-knowledge informally in Σ-protocols (Chapter 16), then formally in Chapter 17. We have also seen it applied in specific systems: the random scalars (r, s) in Groth16 (Chapter 12), the blinding polynomials $(b_1 X + b_2)Z_H(X)$ in PLONK (Chapter 13). Strip these additions and the proof systems still work, they are still sound and succinct, but they leak witness information. This chapter develops the general theory behind those additions. How do we take a working proof system and add the layer that makes it reveal nothing?

The chapter develops two general techniques, then shows how specific proof systems apply them.

Commit-and-prove works for any protocol: hide every witness-dependent value behind a commitment, then prove the required relations via Σ-protocols. This is general but expensive, with cost proportional to the number of multiplications.

Masking polynomials applies specifically to protocols where the prover sends polynomials (notably sum-check): add random noise that preserves validity while hiding the witness. This is efficient but requires algebraic structure.

Neither technique is used in isolation by production systems. Groth16 and PLONK each implement their own variants, tailored to their algebraic structure. After developing the general theory, we examine how these systems achieve zero-knowledge in practice.

The Leakage Problem

Let's be concrete about what leaks. Consider the sum-check protocol proving:

$$H = \sum_{b \in \{0,1\}^n} g(b)$$

When g encodes private witness values, the verifier should not learn g beyond what is necessary for verification. In a proper ZK protocol, the verifier would only learn $g(r)$ at a single random point r at the end (via a commitment opening), not the polynomial itself. But sum-check requires the prover to send intermediate polynomials.

In round i, the prover sends a univariate polynomial representing the partial sum with variable X_i free:

$$g_i(X_i) = \sum_{b_{i+1}, \ldots, b_n \in \{0,1\}} g(r_1, \ldots, r_{i-1}, X_i, b_{i+1}, \ldots, b_n)$$

This polynomial depends on g. Its coefficients encode information about the witness.

To see this concretely, suppose g encodes a computation with secret witness values (w_1, w_2, w_3):

$$g(X_1, X_2) = w_1 X_1 + w_2 X_2 + w_3 X_1 X_2$$

The verifier does not know this polynomial; they only know they are verifying a sum. The first round polynomial is:

$$g_1(X_1) = g(X_1, 0) + g(X_1, 1) = w_1 X_1 + (w_1 X_1 + w_2 + w_3 X_1) = (2w_1 + w_3)X_1 + w_2$$

The prover sends this polynomial to the verifier. The constant term is exactly w_2. The coefficient of X_1 is $2w_1 + w_3$. The verifier learns linear combinations of the secrets directly from the protocol message.

Consider what these witness values could represent. Suppose you are proving eligibility for a loan without revealing your finances. Your witness might encode: w_1 = your salary, w_2 = your social security number, w_3 = your total debt. The computation verifies that your debt-to-income ratio meets some threshold. From that single round polynomial, the verifier learns your SSN directly (the constant term) and a linear combination of your salary and debt. They did not need to learn any of this to verify your eligibility. The protocol leaked it anyway.

This isn't zero-knowledge. We need to hide these coefficients while still allowing verification.

Technique 1: Commit-and-Prove

The commit-and-prove approach is conceptually simple: never send a value in the clear. Always send a commitment, then prove the committed values satisfy the required relations.

The Paradigm

For any public-coin protocol that sends witness-dependent values (public-coin means the verifier's messages are random and visible to both parties, which is the case for sum-check, GKR, and all Fiat-Shamir-compiled protocols):

1. **Replace values with commitments.** Instead of sending v, send $C(v) = g^v h^r$ (a Pedersen commitment with random blinding r).
2. **Prove relations in zero-knowledge.** For each algebraic relation the original protocol checks (e.g., "this value equals that value," "this is the product of those two"), run a Σ-protocol on the committed values.

The verifier never sees actual values. They see commitments (opaque group elements that reveal nothing about the committed data). The Σ-protocols convince them the data satisfies the required structure.

Pedersen's Homomorphism as Leverage

Recall from Chapter 6 that Pedersen commitments ($C(v) = g^v h^r$) are perfectly hiding (the commitment reveals nothing about v, even to an unbounded adversary) and additively homomorphic ($C(a) \cdot C(b) = C(a+b)$). This means the verifier can check additive relations on committed values for free, without any interaction or Σ-protocol: given $C(a), C(b), C(c)$, verify $c = a + b$ by checking $C(c) = C(a) \cdot C(b)$.

Multiplication is more expensive. Checking $c = a \cdot b$ on committed values requires a Σ-protocol that proves the multiplicative relation without revealing the values. This takes three group elements and three field elements per multiplication gate.

Applying to Circuits

Since arithmetic circuits consist entirely of addition and multiplication gates, the cost of commit-and-prove is determined by the multiplication count M: the prover commits to every wire value, addition gates are verified for free via homomorphism, and each multiplication gate requires one Σ-protocol (~3 group elements). The proof contains $O(M)$ group elements (one Σ-protocol transcript per multiplication gate), and verification requires $O(M)$ group exponentiations, each costing $O(\lambda)$ field multiplications for security parameter λ.

This is not succinct. A circuit with a million multiplications produces a proof with millions of group elements. But it achieves *perfect* zero-knowledge: the simulator can produce indistinguishable transcripts by simulating each Σ-protocol independently.

Recovering Succinctness: Proof on a Proof

Commit-and-prove costs $O(M)$ per multiplication gate, so it is impractical for large circuits. But it does not need to be applied to the original circuit. The idea is to split the work into two layers:

1. **First layer (not zero-knowledge).** Run an efficient interactive proof, such as GKR (Chapter 7), on the original circuit C. GKR is sound and succinct: the verifier's work is polylogarithmic in $|C|$. The protocol produces a transcript T consisting of prover messages, verifier challenges, and a final evaluation claim. This transcript is not zero-knowledge; the prover's messages leak witness information.

2. **Second layer (zero-knowledge).** The GKR verifier is itself a computation: given T, check consistency and output accept or reject. Express this verification as a small circuit V_{GKR} of size $O(\text{polylog}(|C|))$. Now apply commit-and-prove to V_{GKR}: the prover commits to all transcript values (which include the witness-derived quantities), then proves via Σ-protocols that these commitments would make V_{GKR} accept.

This is the "proof on a proof": the first layer proves correctness (via GKR), the second layer proves that the first layer's transcript is valid without revealing it (via commit-and-prove on the small verifier circuit). The cost of the second layer depends on V_{GKR}'s multiplication count, which is polylogarithmic in $|C|$, not on $|C|$ itself.

The key detail is the structure of V_{GKR}. Recall from Chapter 7 that GKR verification consists mostly of sum-check consistency checks (pure additions, which Pedersen homomorphism handles for free). The only multiplications arise at layer boundaries, where the verifier checks an equation involving the product of two sub-circuit evaluations: one multiplication per layer. A circuit of depth $d = O(\log n)$ thus requires only $O(\log n)$ Σ-protocols in the second layer, not the $O(n)$ that direct commit-and-prove on the original circuit would require.

The verifier sees the public inputs and outputs (part of the statement), Pedersen commitments to all transcript values, and Σ-protocol proofs that the committed values satisfy GKR verification. The witness w is still encoded in the transcript coefficients c_j (the chain is $w \to$ gate values $\to$ layer MLEs $\to$ sum-check polynomials $\to$ c_j), but the commitments are perfectly hiding. The Σ-protocols prove only *structural* facts about these coefficients (that they satisfy certain arithmetic relations), never *semantic* facts (what they represent in the original computation). Every valid witness producing the same output y yields commitments with the same distribution, so the verifier cannot distinguish which w was used.

Technique 2: Masking Polynomials

Commit-and-prove hides values behind commitments and proves relations one at a time. This is general but expensive: the cost scales with the number of multiplications. For polynomial-based protocols like sum-check, a lighter approach exists: instead of hiding each value individually, randomize the polynomial itself so that the values the verifier sees carry no information about the witness.

The Core Idea

Whenever a protocol requires the prover to send a polynomial derived from the witness (as sum-check does with its round polynomials), the prover can mask it. Instead of sending $g(X)$ directly, the prover sends:

$$f(X) = g(X) + \rho \cdot p(X)$$

where $p(X)$ is a random polynomial (committed in advance) and ρ is a random scalar from the verifier.

Since p is random and ρ is chosen after the commitment, $\rho \cdot p(X)$ acts like a one-time pad: the verifier sees f but cannot extract g without knowing p.

The natural concern is soundness. The original protocol verified $\sum_b g(b) = H$; now the verifier sees $f = g + \rho p$ instead. The masked sum is:

$$\sum_b f(b) = \sum_b g(b) + \rho \cdot \sum_b p(b) = H + \rho \cdot P$$

where $P = \sum_b p(b)$ is sent alongside the commitment to p. The verifier checks $\sum_b f(b) = H + \rho P$. For a false claim $H' \neq H$, this requires $H + \rho P = H' + \rho P$, which implies $H = H'$. Masking is a *soundness-preserving transformation*: it changes the representation but not the truth value.

Constructing the Masking Polynomial

The masking polynomial $p(X)$ must have the same degree structure as g (otherwise $f = g + \rho p$ would fail the verifier's degree checks), known aggregate sum $P = \sum_b p(b)$ (so the verifier can adjust the check), and genuinely random coefficients (so the masking actually hides g).

Protocol flow:

1. Before the main protocol, the prover commits to a random masking polynomial p and sends its sum $P = \sum_b p(b)$.
2. The verifier sends a random ρ.
3. The prover runs sum-check on $f = g + \rho p$, sending masked round polynomials.
4. The verifier checks that round polynomials sum correctly to $H + \rho P$ (the adjusted claim).

The verifier sees f and knows $f = g + \rho p$ with ρ, but only has a commitment to p, not p itself. For any polynomial the verifier might guess for g, there exists a p consistent with the observed f. This is the polynomial one-time pad: the random masking makes all witness polynomials equally plausible. In the multivariate case, the prover commits to a masking polynomial $p(X_1, \ldots, X_n)$ with the same structure as g, and each sum-check round polynomial derived from $f = g + \rho p$ is masked.

Masking the Final Evaluation

The round polynomials are now hidden, but there is a remaining leak. At the end of sum-check, the prover must open $g(r_1, \ldots, r_n)$ at the random evaluation point (typically via a polynomial commitment). This final evaluation is a deterministic function of the witness and reveals information about it.

To handle this, the prover adds random terms that vanish on the Boolean hypercube but randomize evaluations outside it. Instead of committing to g directly, the prover commits to a randomized version:

$$\hat{g}(X_1, \ldots, X_n) = g(X_1, \ldots, X_n) + \sum_{i=1}^{n} c_i \cdot X_i(1 - X_i)$$

where $c_1, \ldots, c_n$ are random field elements chosen by the prover and never revealed. Since $X_i(1 - X_i) = 0$ when $X_i \in \{0, 1\}$, we have $\hat{g}(b) = g(b)$ on the Boolean hypercube: correctness is preserved, and the sum $\sum_b g(b)$ is unchanged. But at a random point $z \notin \{0, 1\}^n$ (where the verifier queries after sum-check), the evaluation becomes $\hat{g}(z) = g(z) + \sum_i c_i \cdot z_i(1 - z_i)$. The verifier sees $\hat{g}(z)$ via the commitment opening but does not know the c_i, so it cannot recover $g(z)$.

Worked example. Let $g(X) = 3X$ (a single-variable polynomial encoding witness data). Randomize with $c = 7$:

$$\hat{g}(X) = 3X + 7 \cdot X(1 - X) = 10X - 7X^2$$

On the hypercube: $\hat{g}(0) = 0 = g(0)$ and $\hat{g}(1) = 3 = g(1)$. At a random point $z = 0.5$: $g(0.5) = 1.5$ (would leak), but $\hat{g}(0.5) = 3.25$ (masked). Different c values produce different evaluations at z, hiding g.

For an n-variable polynomial, the n random scalars $c_1, \ldots, c_n$ provide enough entropy: the verifier learns one evaluation $\hat{g}(z) = g(z) + \sum_i c_i z_i(1 - z_i)$, which for uniform c_i is uniformly distributed over $\mathbb{F}$ regardless of g. A simulator who does not know g can produce identically distributed evaluations by choosing random c_i.

The Simulator

Chapter 17 defined zero-knowledge via simulation: a proof system is ZK if an efficient simulator, given only the public statement, can produce transcripts indistinguishable from real executions. Chapter 17 also observed that vanilla sum-check fails this test: the round polynomials are deterministic functions

of the witness, and no simulator can produce them without it. Masking repairs this. To close the loop, we construct the simulator explicitly and verify indistinguishability.

Real protocol:

1. Prover commits to random masking polynomial p
2. Verifier sends random ρ
3. Parties execute sum-check on $f = g + \rho p$
4. Prover opens $g(z)$ and $p(z)$ at random point z

Simulator $\mathcal{S}$ (no access to the witness, and therefore no access to g):

1. Commit to random polynomial p
2. Choose a random polynomial q of the same degree as g
3. Execute sum-check on $f' = q + \rho p$
4. Open $q(z)$ and $p(z)$

The simulator replaces g with a random q. The question is whether the verifier can tell the difference.

The verifier's view in both cases consists of: a commitment to p, sum-check round messages derived from f or f', the scalar ρ, and the final evaluations. The commitment to p is a random group element in both cases (Pedersen hiding). The round messages are derived from $f = g + \rho p$ (real) or $f' = q + \rho p$ (simulated). Since p is uniformly random and independent of g, and ρ is chosen after p is committed, the polynomial $\rho \cdot p$ is uniformly distributed among polynomials of its degree. Adding a uniform polynomial to any fixed g produces a uniform result, just as adding a uniform field element to any fixed value produces a uniform field element. The distribution of f depends only on the randomness in p and ρ, not on g. The same holds for f'. The two distributions are identical.

For the final evaluation: the verifier learns $g(z)$ and $p(z)$ (real) or $q(z)$ and $p(z)$ (simulated). Since q is uniformly random, $q(z)$ is uniform over $\mathbb{F}$. The masking of the final evaluation (via the $X_i(1 - X_i)$ terms from the previous section) ensures that $g(z)$ is also uniformly distributed from the verifier's perspective. Both views are identically distributed, completing the simulation argument.

Comparing the General Techniques

Aspect	Commit-and-Prove	Masking Polynomials
Generality	Works for any public-coin protocol	Specialized for polynomial protocols

Aspect	Commit-and-Prove	Masking Polynomials
Overhead	$O(M)$ Σ-protocols for M multiplications	$O(1)$ additional commitments
Succinctness	Requires "proof on a proof"	Naturally preserves succinctness
Post-quantum	No (relies on discrete log)	Yes (with hash-based PCS)
Complexity	Conceptually straightforward	Requires algebraic design

The cost difference reflects a difference in abstraction level. Commit-and-prove works on *scalars*: each field element gets its own commitment, and relations are proved one at a time. Masking polynomials works on *functions*: a single random polynomial masks all coefficients at once. Hiding n scalars requires n commitments; hiding an n-coefficient polynomial requires one random polynomial. The jump from scalar to function is what makes masking efficient for polynomial-based protocols.

Most production systems use masking for the main protocol body and commit-and-prove for auxiliary statements (range proofs, committed value equality, etc.).

A third approach, developed in the HyperNova paper (Kothapalli, Setty, Tzialla, 2023), sidesteps both techniques entirely. Instead of masking round polynomials or wrapping each check in a Σ-protocol, the prover replaces every sum-check message with a Pedersen commitment and then proves, via Nova folding, that the committed values satisfy the verifier's checks. The folding step acts as an algebraic one-time pad: the real witness is combined with a random satisfying instance, producing a folded witness that is uniformly distributed regardless of the original. The cost is roughly 3 KB of additional proof data and negligible prover overhead. This technique, called BlindFold, is what made production zkVMs (notably Jolt) genuinely zero-knowledge. Chapter 23 develops it in full after introducing the folding machinery it depends on.

Zero-Knowledge in Practice

The general techniques above provide the conceptual foundation, but production systems do not apply them directly. Groth16 and PLONK each exploit their own algebraic structure to achieve zero-knowledge more efficiently. The underlying principle is the same (randomize what the verifier sees while preserving what the verifier checks), but the mechanisms are system-specific.

Groth16

Recall from Chapter 12 that the prover constructs QAP polynomials $A(X), B(X), C(X)$ encoding the witness, evaluates them at the secret point τ from the structured reference string, and packages the results as three group elements (π_A, π_B, π_C). The polynomials never leave the prover; the verifier sees only these group elements. Without additional randomization, however, the proof elements are deterministic functions of the witness: same witness, same proof. An observer comparing two proofs could detect whether they use the same witness.

Groth16 addresses this the same way Pedersen commitments hide a message: by adding randomness in the exponent. The prover samples fresh scalars (r, s) and incorporates them into the proof elements, making (π_A, π_B, π_C) uniformly distributed while preserving the pairing verification equation.

The Blinding Mechanism Concretely, the prover samples $r, s \in \mathbb{F}$ and incorporates them:

$$\pi_A = g_1^{\alpha + A(\tau) + r\delta}$$
$$\pi_B = g_2^{\beta + B(\tau) + s\delta}$$

The $r\delta$ and $s\delta$ terms add randomness. But where do they go? They'd break the verification equation unless compensated. The construction of π_C absorbs them:

$$\pi_C = g_1^{\frac{\text{private terms}}{\delta} + \frac{H(\tau)Z_H(\tau)}{\delta} + s(\alpha + A(\tau) + r\delta) + r(\beta + B(\tau)) - rs\delta}$$

The terms $sA(\tau)$, $s\alpha$, $rB(\tau)$, $r\beta$, and $rs\delta$ in π_C exactly cancel the cross-terms that appear when expanding $e(\pi_A, \pi_B)$.

Why This Works The verification equation checks:

$$e(\pi_A, \pi_B) = e(g_1^{\alpha}, g_2^{\beta}) \cdot e(\mathsf{vk}_x, g_2^{\gamma}) \cdot e(\pi_C, g_2^{\delta})$$

Expanding $e(\pi_A, \pi_B)$ with blinding:

$$e(g_1^{\alpha + A(\tau) + r\delta}, g_2^{\beta + B(\tau) + s\delta})$$

The exponent becomes $(\alpha + A + r\delta)(\beta + B + s\delta)$, which expands to include cross-terms: $\alpha s\delta$, $As\delta$, $r\beta\delta$, $rB\delta$, $rs\delta^2$.

The π_C construction is designed so that when paired with g_2^δ, it produces exactly these cross-terms (plus the core QAP check). Everything cancels except the QAP identity $A(\tau)B(\tau) = C(\tau) + H(\tau)Z_H(\tau)$.

Different (r, s) produce different valid proofs for the same witness, making proofs for distinct witnesses indistinguishable from proofs for the same witness with different randomness. Note that the blinding depends on δ: the prover computes $g_1^{r\delta}$ as $(g_1^\delta)^r$ using the proving key, without knowing δ as a field element. The setup secret is required by the mechanism.

PLONK

PLONK's approach is closer to masking polynomials than to Groth16's element-level randomization. Recall from Chapter 13 that PLONK encodes constraints as polynomial identities that must hold on a multiplicative subgroup $H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$. The prover commits to witness polynomials $w(X)$ whose values on H encode the wire assignments. After Fiat-Shamir, the verifier queries these polynomials at a random point ζ outside H to check the constraints.

The separation between the constraint domain (H) and the query point ($\zeta \notin H$) is what PLONK exploits for zero-knowledge. Unlike Groth16, which randomizes proof elements, PLONK randomizes the polynomials themselves before committing: add a random multiple of the vanishing polynomial $Z_H(X) = X^n - 1$, which is zero on H. The committed polynomial agrees with the original where constraints are checked but is randomized where the verifier queries.

The Vanishing Polynomial Trick Concretely, to blind a witness polynomial $w(X)$, add a random low-degree polynomial times Z_H:

$$\tilde{w}(X) = w(X) + (b_1 X + b_2) \cdot Z_H(X)$$

where b_1, b_2 are random field elements.

On the constraint-check domain:

$$\tilde{w}(\omega^i) = w(\omega^i) + (b_1\omega^i + b_2) \cdot 0 = w(\omega^i)$$

Why a polynomial, not just a scalar? The verifier queries at a random point ζ, receiving $\tilde{w}(\zeta)$. A single scalar b would add the fixed value $b \cdot Z_H(\zeta)$, which might not provide enough entropy depending on what else the verifier learns. Using $(b_1 X + b_2)$ ensures sufficient randomness for simulation arguments.

Blinding the Accumulator PLONK's permutation argument uses an accumulator polynomial $Z(X)$ that tracks whether wire values are correctly copied. This polynomial also reveals structure.

The accumulator is checked at two points: ζ and $\zeta\omega$ (the "shifted" evaluation). To mask both, use three random scalars:

$$\tilde{Z}(X) = Z(X) + (c_1 X^2 + c_2 X + c_3) \cdot Z_H(X)$$

The boundary condition $Z(1) = 1$ and the recursive multiplicative relation are preserved on H. Outside H, both $\tilde{Z}(\zeta)$ and $\tilde{Z}(\zeta\omega)$ are randomized.

The same blinding applies to every polynomial PLONK commits to, including the quotient polynomial $t(X)$ (which is split into pieces for degree reasons, each blinded independently).

The Unifying Principle

Despite different mechanisms, Groth16 and PLONK follow the same pattern: find the *null space* of the verification procedure (transformations that don't affect the outcome) and inject randomness there. In Groth16, the null space is the set of (r, s) shifts that cancel in the pairing. In PLONK, it is the space of Z_H multiples that vanish on the constraint domain. This connects directly to the simulation paradigm from Chapter 17: the simulator can produce valid-looking transcripts because it can choose randomness within this null space.

Key takeaways

1. **Every ZK technique finds the null space of the verification procedure and injects randomness there.** Transformations that don't affect the verification outcome are the prover's freedom. Soundness constrains what the prover can do; zero-knowledge exploits what the prover is free to randomize.
2. **Commit-and-prove is general but expensive.** It works for any public-coin protocol by hiding values behind Pedersen commitments and proving relations via Σ-protocols. Cost scales with multiplication count ($O(M)$), but the "proof on a proof" trick recovers succinctness by applying commit-and-prove to the $O(\log n)$ verifier circuit instead of the original computation.
3. **Masking polynomials are efficient but specialized.** For polynomial-sending protocols like sum-check, adding $\rho \cdot p(X)$ (a committed random polynomial scaled by a verifier challenge) acts as a one-time pad. Succinctness is preserved naturally. Final evaluations require separate treatment via terms like $\sum_i c_i X_i(1 - X_i)$ that vanish on the Boolean hypercube.

4. **Production systems tailor the null space to their algebraic structure.**

    - *Groth16*: fresh scalars (r, s) shift the proof elements; π_C absorbs the cross-terms so the pairing equation is unchanged.
    - *PLONK*: random multiples of $Z_H(X)$ vanish on the constraint domain H but randomize evaluations at the query point $\zeta \notin H$.

5. **Production systems blend approaches.** Masking handles the core polynomial protocol. Commit-and-prove handles auxiliary statements (range proofs, committed value equality). BlindFold (Chapter 23) offers a third path via folding.

Chapter 19: Fast Sum-Check Proving

> *This chapter, along with Chapters 20 and 21, forms Part VI on prover optimization. These three chapters are optional on a first read: the rest of the book stands without them. They are essential for anyone designing or implementing a prover, and reward repeat reading once the foundations feel solid.*
>
> *Most chapters in this book can be read with pencil and paper. This one assumes you've already internalized the sum-check protocol (Chapter 3) and multilinear extensions (Chapter 4), not as definitions to look up, but as tools you can wield. If those still feel foreign, consider this chapter a preview of where the road leads, and return when the foundations feel solid.*

In 1992, the sum-check protocol solved the problem of succinct verification. Lund, Fortnow, Karloff, and Nisan had achieved something that sounds impossible: verifying a sum over 2^n terms while the verifier performs only $O(n)$ work. Exponential compression in verification time. The foundation of succinct proofs.

Then, for three decades, almost nobody used it.

Why? Because everyone thought the prover was too slow. The total work across all rounds sums to $O(d \cdot 2^n)$ (as Chapter 3 showed via the geometric series), but achieving this requires the prover to evaluate the partially-fixed polynomial efficiently at each round. Without a way to reuse work across rounds, each round's evaluations require going back to the original 2^n-entry table, inflating the cost to $O(n \cdot 2^n)$. For $n = 30$, that's over 30 billion operations per proof. Researchers chased other paths: PCPs, pairing-based SNARKs, trusted setups. Groth16 and PLONK took univariate polynomials, quotient-based constraints, FFT-driven arithmetic. Sum-check remained a theoretical marvel, admired in complexity circles but dismissed as impractical.

They were wrong.

It turned out that a simple algorithmic trick, available since the 90s but overlooked, made the prover linear time. With the right algorithms, sum-check proving runs in $O(2^n)$ time, linear in the number of terms. For sparse sums where only $T \ll 2^n$ terms are non-zero, prover time drops to $O(T)$. These are not approximations or heuristics; they're exact algorithms exploiting algebraic structure that was always present.

When this was rediscovered and popularized by Justin Thaler in the late 2010s, it triggered a revolution. The field realized it had been sitting on the "Holy Grail" of proof systems for three decades without noticing. This chapter explains the trick that woke up the giant, and then shows how it enables Spartan, the SNARK that proved sum-check alone suffices for practical zero-knowledge proofs. No univariate encodings. No pairing-based trusted setup. Just multilinear polynomials, sum-check, and a commitment scheme.

Why This Matters: The zkVM Motivation

These techniques find their most compelling application in *zkVMs* (zero-knowledge virtual machines): SNARKs that prove correct execution of arbitrary programs over an instruction set like RISC-V. A million CPU cycles at 50 constraints each yields 50 million constraints. At this scale, $O(n \log n)$ versus $O(n)$ proving is the difference between minutes and seconds. Even the constant factor matters. Fast sum-check proving is what makes zkVMs practical.

The Prover's Apparent Problem

Let's examine the naive prover cost more carefully.

The sum-check protocol proves:

$$H = \sum_{b \in \{0,1\}^n} g(b)$$

where $g : \mathbb{F}^n \to \mathbb{F}$ is an n-variate polynomial. The prover begins by sending the claimed sum H (this is V_0). Then in round i, the prover sends a univariate polynomial capturing the partial sum with X_i left as a formal variable:

$$s_i(X_i) = \sum_{(b_{i+1},\ldots,b_n) \in \{0,1\}^{n-i}} g(r_1, \ldots, r_{i-1}, X_i, b_{i+1}, \ldots, b_n)$$

The polynomial s_i is univariate in X_i, with degree equal to the degree of g in that variable. Call this degree d_i. A degree-d_i univariate is determined by d_i+1 evaluations, but the consistency check $s_i(0)+s_i(1) = V_{i-1}$ (where V_{i-1} is the claim from the previous round) lets the verifier derive one evaluation for free, so the prover sends only d_i values.

For simplicity, assume g has individual degree d in every variable (the common case in practice). Computing s_i requires evaluating it at $d+1$ points, and each evaluation sums over 2^{n-i} terms of the form $g(r_1, \ldots, r_{i-1}, t, b_{i+1}, \ldots, b_n)$.

Here is the problem. In round 1, no variables have been fixed to challenges yet, so each term in the sum for $s_1(t)$ has the form $g(t, b_2, \ldots, b_n)$ with all remaining coordinates Boolean. For $t \in \{0, 1\}$, these are values of g on the hypercube, which the prover computed before the protocol began. For $t > 1$ (the non-Boolean evaluation points needed to determine s_1), the prover must interpolate, but only in the first variable. Round 1 is manageable. But from round 2 onward, the first variables are fixed to non-Boolean challenges $r_1, \ldots, r_{i-1}$. The values $g(r_1, \ldots, r_{i-1}, t, b_{i+1}, \ldots, b_n)$ were never precomputed. Without a way to access them cheaply, the prover must recompute them from scratch each round by interpolating over the full 2^n Boolean evaluations. This costs $O(2^n)$ per round, and over n rounds the total is $O(n \cdot 2^n)$.

Notice, however, that round i only sums over 2^{n-i} terms. The work should shrink by half each round, and the geometric series gives:

$$\sum_{i=1}^{n} (d+1) \cdot 2^{n-i} = (d+1) \sum_{k=0}^{n-1} 2^k = (d+1)(2^n - 1) = O(d \cdot 2^n)$$

(using the geometric series identity $\sum_{k=0}^{n-1} r^k = \frac{r^n - 1}{r-1}$ with $r = 2$). The bottleneck is not the number of terms but *access*: can the prover obtain the 2^{n-i} partially-fixed values for round i without recomputing them from the original 2^n values each time? If not, each round costs $O(2^n)$ regardless of how few terms it sums, and the total remains $O(n \cdot 2^n)$.

The Halving Trick

The answer to the access problem is a single identity from Chapter 4: multilinear folding. After each challenge r_i, the prover can update a multilinear polynomial's table of Boolean evaluations in place, producing the restricted polynomial's table in half the space. No recomputation from scratch.

But folding applies to multilinear polynomials, and in the interesting sum-check instances g has degree > 1 in each variable, so g itself is not multilinear. The trick is that g does not need to be multilinear as long as it *decomposes* into multilinear factors. If $g = \tilde{a} \cdot \tilde{b}$ (or more generally a sum of products of MLEs), the prover folds each factor's table independently and recomputes g's values from the folded factors each round. This covers essentially all practical cases: GKR's layer reductions (Chapter 18) use products of MLEs with the equality polynomial, R1CS verification uses $\tilde{a} \cdot \tilde{b} \cdot \tilde{c}$, and Spartan (later in this chapter) reduces to the same form.

We develop the algorithm for the simplest case: $g(x) = \tilde{a}(x) \cdot \tilde{b}(x)$, a product of two multilinear polynomials over n variables.

Since multilinear polynomials have degree at most 1 in each variable, their product has degree at most 2. So $d = 2$, and each round the prover sends two field elements (say $s_i(0)$ and $s_i(2)$); the verifier recovers $s_i(1) = V_{i-1} - s_i(0)$ from the consistency check.

The Multilinear Folding Identity

Recall from Chapter 4 the streaming evaluation identity: for any multilinear polynomial $\tilde{a}(x_1, x_2, \ldots, x_n)$ and field element r_1,

$$\tilde{a}(r_1, x_2, \ldots, x_n) = (1 - r_1) \cdot \tilde{a}(0, x_2, \ldots, x_n) + r_1 \cdot \tilde{a}(1, x_2, \ldots, x_n)$$

This is linear interpolation: $\tilde{a}$ restricted to X_1 is a line through $(0, y_0)$ and $(1, y_1)$, given by $y_0 + (y_1 - y_0) \cdot X$. The identity evaluates that line at any $r_1 \in \mathbb{F}$. Chapter 4 used it with challenges in $[0, 1]$ to evaluate MLEs in $O(2^n)$ time. Here we also need non-Boolean points: setting $r_1 = 2$ gives $-y_0 + 2y_1$, extrapolating the line beyond its defining points using only stored Boolean evaluations.

This fact enables **folding**: after receiving challenge r_1, we can compute the restricted polynomial $\tilde{a}(r_1, x_2, \ldots, x_n)$ from the unrestricted polynomial $\tilde{a}$ in linear time.

The Algorithm

Initialization. Store all 2^n evaluations $\tilde{a}(b)$ and $\tilde{b}(b)$ for $b \in \{0,1\}^n$ in arrays $A[b]$ and $B[b]$.

Round 1. Compute three evaluations of $s_1(X_1) = \sum_{(b_2,\ldots,b_n)\in\{0,1\}^{n-1}} \tilde{a}(X_1, b_2, \ldots, b_n) \cdot \tilde{b}(X_1, b_2, \ldots, b_n)$:

- $s_1(0) = \sum_{(b_2,\ldots,b_n)\in\{0,1\}^{n-1}} A[(0, b_2, \ldots, b_n)] \cdot B[(0, b_2, \ldots, b_n)]$
- $s_1(1) = \sum_{(b_2,\ldots,b_n)\in\{0,1\}^{n-1}} A[(1, b_2, \ldots, b_n)] \cdot B[(1, b_2, \ldots, b_n)]$
- $s_1(2) = \sum_{(b_2,\ldots,b_n)\in\{0,1\}^{n-1}} A[(2, b_2, \ldots, b_n)] \cdot B[(2, b_2, \ldots, b_n)]$

For $s_1(0)$ and $s_1(1)$, we read directly from the stored arrays. For $s_1(2)$, apply the folding identity with $r_1 = 2$: $A[(2, b_2, \ldots, b_n)] = -A[(0, b_2, \ldots, b_n)] + 2 \cdot A[(1, b_2, \ldots, b_n)]$, and similarly for B.

Each evaluation sums over 2^{n-1} terms, so the three evaluations cost $3 \cdot 2^{n-1}$ operations total. The prover sends two; the verifier recovers the third from $s_1(0) + s_1(1) = H$.

Fold after round 1. Receive challenge r_1. Create a new array A' of size 2^{n-1}, indexed by $(b_2, \dots, b_n) \in \{0,1\}^{n-1}$:

$$A'[(b_2, \dots, b_n)] = (1 - r_1) \cdot A[(0, b_2, \dots, b_n)] + r_1 \cdot A[(1, b_2, \dots, b_n)] = \tilde{a}(r_1, b_2, \dots, b_n)$$

Discard the old array and rename $A' \to A$. The array now stores the restricted polynomial $\tilde{a}(r_1, x_2, \dots, x_n)$ evaluated on the $(n-1)$-dimensional hypercube. Similarly fold B.

Round i (general). After $i-1$ folds, the arrays A and B have size 2^{n-i+1}, storing $\tilde{a}(r_1, \dots, r_{i-1}, b_i, \dots, b_n)$ on the remaining Boolean hypercube. The array splits naturally into two halves of size 2^{n-i}: entries with $b_i = 0$ and entries with $b_i = 1$. Then:

- $s_i(0) = \sum_{(b_{i+1}, \dots, b_n)} A[(0, b_{i+1}, \dots)] \cdot B[(0, b_{i+1}, \dots)]$: the sum over the $b_i = 0$ half
- $s_i(1) = \sum_{(b_{i+1}, \dots, b_n)} A[(1, b_{i+1}, \dots)] \cdot B[(1, b_{i+1}, \dots)]$: the sum over the $b_i = 1$ half
- $s_i(2)$: apply the folding identity with $r = 2$ to get $A[(2, b_{i+1}, \dots)] = -A[(0, b_{i+1}, \dots)] + 2 \cdot A[(1, b_{i+1}, \dots)]$, then sum the products

Each evaluation sums over 2^{n-i} terms, costing $3 \cdot 2^{n-i}$ operations total. The prover sends two values, then folds A and B using challenge r_i, halving the arrays to size 2^{n-i}.

The arrays shrink by half each round: $2^n \to 2^{n-1} \to \cdots \to 2 \to 1$. By round n, the arrays are singletons and the protocol terminates.

Folding solves the access problem. After each challenge r_i, the prover updates the arrays in place via the folding identity, producing exactly the partially-fixed values needed for round $i+1$. No recomputation from scratch. Round i costs $O(2^{n-i})$ for evaluation and $O(2^{n-i})$ for folding, with a constant $c \leq 10$ field operations per entry for the product $\tilde{a} \cdot \tilde{b}$. The total is the geometric series from above:

$$T(n) = \sum_{i=1}^{n} c \cdot 2^{n-i} = c(2^n - 1) = O(2^n)$$

This is optimal: any prover must read all 2^n inputs at least once.

Worked Example: The Halving Trick with $n = 2$

Let's trace through a complete example. Take $n = 2$ variables and consider the sum-check claim:

$$H = \sum_{(b_1, b_2) \in \{0,1\}^2} \tilde{a}(b_1, b_2) \cdot \tilde{b}(b_1, b_2)$$

Suppose the tables are:

(b_1, b_2)	$A[b_1, b_2]$	$B[b_1, b_2]$	Product
$(0, 0)$	2	3	6
$(0, 1)$	5	1	5
$(1, 0)$	4	2	8
$(1, 1)$	3	4	12

The true sum is $H = 6 + 5 + 8 + 12 = 31$.

Round 1: Compute $s_1(X_1)$.

We need three evaluations to specify this degree-2 polynomial:

- $s_1(0) = A[0, 0] \cdot B[0, 0] + A[0, 1] \cdot B[0, 1] = 2 \cdot 3 + 5 \cdot 1 = 11$
- $s_1(1) = A[1, 0] \cdot B[1, 0] + A[1, 1] \cdot B[1, 1] = 4 \cdot 2 + 3 \cdot 4 = 20$
- $s_1(2)$: First interpolate A and B at $X_1 = 2$:
  - $A[2, 0] = -A[0, 0] + 2 \cdot A[1, 0] = -2 + 8 = 6$
  - $A[2, 1] = -A[0, 1] + 2 \cdot A[1, 1] = -5 + 6 = 1$
  - $B[2, 0] = -B[0, 0] + 2 \cdot B[1, 0] = -3 + 4 = 1$
  - $B[2, 1] = -B[0, 1] + 2 \cdot B[1, 1] = -1 + 8 = 7$
  - $s_1(2) = 6 \cdot 1 + 1 \cdot 7 = 13$

Verifier checks: $s_1(0) + s_1(1) = 11 + 20 = 31 = H$. ✓

Prover sends $(11, 20, 13)$. Verifier sends challenge $r_1 = 3$.

Fold after Round 1.

Update arrays using $A'[b_2] = (1 - r_1) \cdot A[0, b_2] + r_1 \cdot A[1, b_2]$:

- $A'[0] = (1 - 3) \cdot 2 + 3 \cdot 4 = -4 + 12 = 8$
- $A'[1] = (1 - 3) \cdot 5 + 3 \cdot 3 = -10 + 9 = -1$

Similarly for B:

- $B'[0] = (1 - 3) \cdot 3 + 3 \cdot 2 = -6 + 6 = 0$
- $B'[1] = (1 - 3) \cdot 1 + 3 \cdot 4 = -2 + 12 = 10$

Arrays now have size 2 (down from 4).

Round 2: Compute $s_2(X_2)$.

- $s_2(0) = A'[0] \cdot B'[0] = 8 \cdot 0 = 0$

- $s_2(1) = A'[1] \cdot B'[1] = (-1) \cdot 10 = -10$
- $s_2(2) = (-A'[0] + 2 \cdot A'[1]) \cdot (-B'[0] + 2 \cdot B'[1]) = (-8 - 2) \cdot (0 + 20) = (-10) \cdot 20 = -200$

Verifier checks: $s_2(0) + s_2(1) = 0 + (-10) = -10 \stackrel{?}{=} s_1(r_1) = s_1(3)$.

This is the core consistency check of sum-check. The prover committed to s_1 before knowing the challenge $r_1 = 3$. Now the verifier demands that s_2 (the next round's polynomial) sum to the value $s_1(r_1)$. If the prover lied about s_1, the fabricated polynomial almost certainly evaluates incorrectly at the random point r_1, and the check fails.

Compute $s_1(3)$ from the degree-2 polynomial through points $(0, 11), (1, 20), (2, 13)$:

Using Lagrange interpolation:

$s_1(X) = 11 \cdot \frac{(X-1)(X-2)}{(0-1)(0-2)} + 20 \cdot \frac{(X-0)(X-2)}{(1-0)(1-2)} + 13 \cdot \frac{(X-0)(X-1)}{(2-0)(2-1)} = 11 \cdot \frac{(X-1)(X-2)}{2} - 20 \cdot (X)(X-2) + 13 \cdot \frac{X(X-1)}{2}$

At $X = 3$: $s_1(3) = 11 \cdot \frac{2 \cdot 1}{2} - 20 \cdot 3 \cdot 1 + 13 \cdot \frac{3 \cdot 2}{2} = 11 - 60 + 39 = -10$. ✓

Total operations: Round 1 touched 4 entries; Round 2 touched 2 entries. Total: 6 operations, not $2 \cdot 4 = 8$ as naive analysis suggests. For larger n, the savings compound: $O(2^n)$ instead of $O(n \cdot 2^n)$.

Each round, the arrays halve in size. The total work across all rounds is the geometric series $N + N/2 + N/4 + \cdots = O(N)$. This is optimal: any prover must read all N inputs at least once.

Beyond Black-Box Arithmetic

The halving trick achieves $O(2^n)$ field operations, which is optimal. For a textbook, the story could end here. But in practice, sum-check provers over 256-bit fields remain slow even at optimal operation count, because each field multiplication carries a different concrete cost depending on the size of its operands. The next three sections (this one, high-degree products, and small-value proving) progressively reduce the concrete cost by exploiting structure that asymptotic analysis ignores. All three build on the same observation: not all field multiplications are equal.

Not all field multiplications are equal. Over a 256-bit prime field (BN254, BLS12-381), multiplying two arbitrary field elements requires multi-limb integer arithmetic plus modular reduction. But when one operand fits in a single 64-bit machine word, the cost drops dramatically. Three classes emerge:

- **big-big (bb)**: two arbitrary field elements. Roughly 8x the cost of sb.
- **small-big (sb)**: one machine-word integer, one field element.
- **small-small (ss)**: two machine-word integers. A single native multiplication, roughly 30x cheaper than bb.

A further optimization, *delayed reduction*, avoids redundant modular reductions when accumulating a linear combination $\sum c_i \cdot a_i$ with small coefficients c_i. Instead of reducing each product separately, the prover accumulates unreduced integer products and performs a single reduction at the end. This nearly halves the cost of sb-dominated loops, which is precisely the structure of the sum-check prover's inner loop.

Why does this matter for sum-check? In round 1, all evaluations lie on the Boolean hypercube. In a zkVM, these are witness values (register contents, memory entries), typically 32- or 64-bit integers stored in a 256-bit field. Round 1 uses only ss and sb operations. After the verifier sends challenge r_1, subsequent rounds involve full-width random elements and require bb multiplications.

Round 1 is not a small fraction of the work. By the geometric series, round 1 accounts for *half* the total operations (2^{n-1} out of $2^n - 1$). Rounds 1 and 2 together account for three-quarters. The most expensive rounds are precisely those where values are small. This observation, that the prover's bottleneck rounds coincide with the regime where cheap arithmetic applies, is the starting point for small-value proving.

High-Degree Products

The halving trick as presented handles $g = \tilde{a} \cdot \tilde{b}$, a product of two multilinear factors with degree $d = 2$. Each round, the prover evaluates the product at $d+1 = 3$ points, spending a constant number of multiplications per summand. The same idea generalizes to $g = \prod_{k=1}^{d} p_k$, a product of d multilinear factors: fold each factor independently, then multiply. The folding is unchanged; what changes is the cost of multiplying d factors together at $d + 1$ evaluation points.

Modern proof systems demand this generalization. In batch-evaluation arguments and lookup protocols, the sum-check polynomial is a product of d multilinear factors where d can be 16 or 32. The naive approach evaluates each factor at $d + 1$ points by extrapolation from the Boolean evaluations at 0 and 1, then multiplies pointwise. This costs $(d-1)(d+1) \approx d^2$ bb multiplications per summand. At $d = 32$, that is nearly 1000 bb multiplications per term per round, and the prover's cost balloons to $O(d^2 \cdot 2^n)$.

The question is whether the d^2 factor is intrinsic to the problem.

Divide-and-Conquer via Extrapolation

It is not. A recursive algorithm reduces the bb cost from $O(d^2)$ to $O(d \log d)$ per evaluation point.

We work entirely in *evaluation representation*: each polynomial is stored as its values at a fixed set of points. A linear polynomial (degree 1) is determined by two evaluations (at 0 and 1). A degree-d

product needs $d + 1$ evaluations. Multiplying two polynomials in evaluation form is just pointwise multiplication of their values at each point: one field multiplication per point.

Given evaluations of d linear polynomials $p_1, \ldots, p_d$ at the two points $\{0, 1\}$, the goal is to compute evaluations of their product $g = \prod_i p_i$ at $d + 1$ points.

1. Split the d polynomials into two halves of size $\lfloor d/2 \rfloor$ and $\lceil d/2 \rceil$.
2. Recursively compute the product of each half. Each half-product has degree $\sim d/2$ and is known at $\sim d/2 + 1$ points.
3. **Extrapolate** both half-products from their $\sim d/2+1$ known points to the full set of $d+1$ points, using Lagrange interpolation. The interpolation weights are small integers (derived from the evaluation-point coordinates $0, 1, 2, \ldots$), so each multiplication is sb. Cost: $O(d)$ sb multiplications per polynomial.
4. **Multiply pointwise**: at each of the $d + 1$ points, multiply the two half-product values. Both values are arbitrary field elements (results of prior recursion), so each multiplication is bb. Cost: $d + 1$ bb multiplications.

The only source of bb multiplications is step 4. Each level of recursion contributes $d + 1$ pointwise products, and the two recursive calls handle the subproblems. Writing $a(d)$ for the total bb count:

$$a(d) \leq d\lceil \log_2 d \rceil + d - 1$$

The extrapolation steps contribute $O(d^2)$ sb multiplications total, but since sb is far cheaper than bb, the wall-clock cost is dominated by the $O(d \log d)$ bb multiplications.

This extends to the multivariate case. In the sum-check prover's inner loop, the product involves d multilinear polynomials in v variables, evaluated on a grid of $(d + 1)^v$ points. Multivariate extrapolation reduces to repeated univariate extrapolation along each coordinate dimension. The bb cost becomes $O(d^v)$ for $v \geq 2$, improving by a factor of d over the naive $O(d^{v+1})$.

When used as a subroutine in the linear-time sum-check prover, the total bb cost across all rounds drops from $\Theta(d^2 \cdot 2^n)$ to $\Theta(d \log d \cdot 2^n)$. For $d = 32$, this represents roughly a 5x reduction in the dominant arithmetic cost.

Small-Value Round-Batching

There is a structural inefficiency hiding in the halving trick. In round 1, all evaluations lie on the Boolean hypercube: the values come from the witness table and fit in machine words. Round 1 uses only ss/sb operations. But the moment the prover binds $X_1 = r_1$ (a random 256-bit challenge), every subsequent value becomes a full-width field element. From round 2 onward, bb multiplications are unavoidable.

Round 1 accounts for half the total work. Round 2 accounts for a quarter. The most expensive rounds are exactly where values are small. Can we extend the cheap-arithmetic regime beyond a single round?

The idea is *delayed binding*: instead of binding $X_1 = r_1$ immediately, treat the first v variables as symbolic and precompute the v-variate polynomial:

$$q(X_1, \ldots, X_v) = \sum_{x' \in \{0,1\}^{n-v}} \prod_{k=1}^{d} p_k(X_1, \ldots, X_v, x')$$

Every summand has Boolean x' and small witness values, so the entire precomputation uses only ss multiplications. The polynomial q is stored as its evaluations on a $(d+1)^v$ grid. Once computed, the prover answers rounds 1 through v by evaluating q at the received challenges (which does require bb, but only $O((d+1)^v)$ work per round instead of $O(2^{n-i})$). After v rounds, the prover binds all v challenges at once and resumes the standard halving trick on arrays of size 2^{n-v}.

The optimal window size v balances the ss precomputation cost against the bb savings. For $d = 2$ over 256-bit fields, $v \approx 4$ or 5 rounds. The asymptotic complexity is unchanged, but the concrete runtime drops substantially because the largest rounds (which dominate the geometric series) now use the cheapest arithmetic.

Streaming provers

Round-batching generalizes beyond the small-value setting. For truly massive computations ($N = 2^{40}$ terms), even $O(N)$ memory becomes prohibitive: a terabyte of field elements. The halving trick is optimal in time but demands linear space.

A *streaming prover* applies round-batching iteratively, processing the input in sequential passes with sublinear memory. Instead of batching only the first v rounds, the streaming prover batches *every* group of rounds into windows. For a window of ω rounds, the prover scans the relevant terms in one pass, computes a ω-variate polynomial on a $(d+1)^\omega$ grid, answers ω rounds from it, then moves to the next window. Early windows are small (the input is large). Later windows grow larger (the remaining input shrinks exponentially). A final phase switches to the standard linear-time algorithm.

With a tunable parameter $k \geq 2$, the streaming prover achieves $O(N^{1/k})$ space and $O(d \log d \cdot N \cdot (k + \log\log N))$ time. For $k = 2$: two passes and $O(\sqrt{N})$ memory. This exploits the algebraic structure of sum-check directly, without recursive proof composition.

Sparse Sums

The halving trick solves the dense case: when all 2^n terms are present, we achieve optimal $O(2^n)$ proving time. But many applications involve **sparse** sums, where only $T \ll 2^n$ terms are non-zero, and here the halving trick falls short.

Consider a lookup table with $N = 2^{30}$ possible indices but only $T = 2^{20}$ actual lookups. The halving trick still touches all 2^{30} positions, folding arrays of zeros. We're wasting a factor of 1000 in both time and space.

Can the prover exploit sparsity?

Separable Product Structure

A clarification first: "sparse sum" means the *input data* is sparse (the table on the Boolean hypercube has mostly zeros). The multilinear extension of a sparse vector is typically dense over the continuous domain. Sparsity in the table is what we exploit. Doing so requires a specific factorization.

In lookup arguments and memory-checking protocols, the problem is constructed with a natural variable split: a prefix $p = (x_1, \ldots, x_{n/2})$ encodes an address or row index, and a suffix $s = (x_{n/2+1}, \ldots, x_n)$ encodes a value or column index. The constraints on addresses and values are independent by design, but a sparse selector connects them: most (address, value) pairs are unused, and only $T \ll 2^n$ entries are active. For instance, a memory with 2^{16} addresses and 2^{16} possible values has 2^{32} (address, value) pairs, but a program that performs $T = 10{,}000$ memory accesses touches only $10{,}000$ of them.

This gives the factorization:

$$g(p, s) = \tilde{a}(p, s) \cdot \tilde{f}(p) \cdot \tilde{h}(s)$$

where $\tilde{a}(p, s)$ is a sparse selector with only T non-zero entries, $\tilde{f}(p)$ depends only on prefix variables (dense, size $2^{n/2}$), and $\tilde{h}(s)$ depends only on suffix variables (dense, size $2^{n/2}$). The separability is what makes sparsity exploitable: to compute an aggregate like $\sum_s \tilde{a}(p, s) \cdot \tilde{h}(s)$, the prover touches only the T positions where $\tilde{a}$ is non-zero.

Two-Stage Proving

Given the separable product structure, we prove the sum in two stages: an outer sum-check over the prefix variables (addresses) and an inner sum-check over the suffix variables (values) to verify the

outer stage's evaluation claim. Each stage handles half the variables, building dense arrays of size $2^{n/2}$ by scanning only the T sparse entries.

Stage 1 (outer): Sum-check over prefix variables.

Define aggregated arrays P and F, each of size $2^{n/2}$, indexed by prefix bit-vectors $p \in \{0,1\}^{n/2}$:

$$P[p] = \sum_{s \in \{0,1\}^{n/2}} \tilde{a}(p, s) \cdot \tilde{h}(s)$$

$$F[p] = \tilde{f}(p)$$

The array P pre-sums all suffix contributions into a single value per prefix. This is the key move: the suffix variables are absorbed *before* the protocol starts, collapsing the original double sum $\sum_p \sum_s$ into a single sum over prefixes $\sum_p$. Computing P requires one pass over the T non-zero entries: for each non-zero (p, s) pair, add $\tilde{a}(p, s) \cdot \tilde{h}(s)$ to $P[p]$.

To see why this is correct, expand the original claim:

$$\sum_{p \in \{0,1\}^{n/2}} \sum_{s \in \{0,1\}^{n/2}} \tilde{a}(p, s) \cdot \tilde{f}(p) \cdot \tilde{h}(s) = \sum_{p \in \{0,1\}^{n/2}} \tilde{f}(p) \cdot \underbrace{\sum_{s \in \{0,1\}^{n/2}} \tilde{a}(p, s) \cdot \tilde{h}(s)}_{= P[p]}$$

So proving the original sum reduces to proving $\sum_p \tilde{P}(p) \cdot \tilde{F}(p)$, a sum-check with only $n/2$ variables. Here $\tilde{P}$ and $\tilde{F}$ are the multilinear extensions of arrays P and F.

Run the dense halving algorithm on these $2^{n/2}$-sized arrays. Time: $O(T)$ to build P from sparse entries, plus $O(2^{n/2})$ for the dense sum-check.

Stage 2 (inner): Sum-check over suffix variables.

Like any sum-check, Stage 1 ends with a final evaluation claim: "I claim $\tilde{P}(r_p) \cdot \tilde{F}(r_p) = v_1$." The verifier can check $\tilde{F}(r_p)$ via polynomial commitment. But $\tilde{P}(r_p)$ is itself defined as a sum over suffix variables:

$$\tilde{P}(r_p) = \sum_{s \in \{0,1\}^{n/2}} \tilde{a}(r_p, s) \cdot \tilde{h}(s)$$

This is where Stage 2 comes in: it runs sum-check over the suffix variables to verify this claim. Stage 1 summed out the prefixes; Stage 2 sums out the suffixes. Together they cover all n variables, but each stage operates on arrays of size $2^{n/2}$ instead of 2^n.

Define arrays H and Q, each of size $2^{n/2}$, indexed by suffix bit-vectors $s \in \{0,1\}^{n/2}$:

$$H[s] = \tilde{a}((r_1, \ldots, r_{n/2}), s)$$

$$Q[s] = \tilde{h}(s)$$

Here H is the sparse selector with its prefix fixed to the random challenges: it answers "what is the selector's value at address (r_p, s)?" The factor $\tilde{f}(r_p)$ is now a constant (computed once from the dense F array) that multiplies the entire Stage 2 sum.

Computing H requires the MLE interpolation identity: $\tilde{a}(r_p, s) = \sum_{p'} \tilde{a}(p', s) \cdot \widetilde{\mathsf{eq}}(p', r_p)$. For each sparse entry (p, s), we need the Lagrange coefficient $\widetilde{\mathsf{eq}}(p, r_p)$ to weight its contribution to $H[s]$.

(Recall from Chapter 4: $\widetilde{\mathsf{eq}}(\tau, x) = \prod_i (\tau_i x_i + (1 - \tau_i)(1 - x_i))$ is the multilinear Lagrange basis function, and $\sum_x \widetilde{\mathsf{eq}}(\tau, x) \cdot f(x) = \tilde{f}(\tau)$.)

A naive approach computes each $\widetilde{\mathsf{eq}}(p, r_p)$ independently in $O(n/2)$ field ops, giving $O(T \cdot n)$ total. But we can do better: precompute *all* $2^{n/2}$ values $\widetilde{\mathsf{eq}}(p, r_p)$ for every Boolean p in $O(2^{n/2})$ time using the product structure of $\widetilde{\mathsf{eq}}$. Then each sparse entry requires only a table lookup plus one multiplication. Total: $O(2^{n/2})$ for precomputation, $O(T)$ for the pass over sparse entries.

Run the dense halving algorithm on H and Q for the remaining $n/2$ rounds. Time: $O(2^{n/2})$ for precomputing $\widetilde{\mathsf{eq}}$ values, $O(T)$ to accumulate into H, plus $O(2^{n/2})$ for the dense sum-check.

The structure is two chained sum-checks:

1. **Stage 1** ($n/2$ rounds): proves the sum equals H, ends with evaluation claim about $\tilde{P}(r_p)$
2. **Stage 2** ($n/2$ rounds): proves that evaluation claim, ends with evaluation of $\tilde{a}(r_p, r_s)$ and $\tilde{h}(r_s)$

Together: $n/2 + n/2 = n$ rounds, matching the original n-variable sum-check. But prover work is only:

$$O(T + 2^{n/2})$$

Two passes over T sparse terms (one per stage), plus two $2^{n/2}$-sized dense sum-checks. With appropriate parameters, this can be much less than $O(2^n)$.

Worked Example: Sparse Sum with $N = 16$, $T = 3$

Consider a table of size $N = 16$ (so $n = 4$ variables), but only $T = 3$ entries are non-zero. We want to prove:

$$H = \sum_{(p,s) \in \{0,1\}^4} \tilde{a}(p, s) \cdot \tilde{f}(p) \cdot \tilde{h}(s)$$

where $p = (x_1, x_2)$ is the 2-bit prefix and $s = (x_3, x_4)$ is the 2-bit suffix.

Suppose the only non-zero entries are:

Index	Prefix p	Suffix s	$\tilde{a}(p, s)$	$\tilde{f}(p)$	$\tilde{h}(s)$	Product
5	$(0, 1)$	$(0, 1)$	3	2	4	24
9	$(1, 0)$	$(0, 1)$	5	1	4	20
14	$(1, 1)$	$(1, 0)$	2	3	7	42

True sum: $H = 24 + 20 + 42 = 86$.

A dense approach would store all 16 entries and fold arrays of size 16 → 8 → 4 → 2 → 1, touching all 16 positions even though 13 are zero. The sparse two-stage approach avoids this.

Stage 1: Build aggregated prefix array P.

Scan the 3 non-zero terms and accumulate:

$$P[p] = \sum_{s} \tilde{a}(p, s) \cdot \tilde{h}(s)$$

- Entry $(0, 1), (0, 1)$: Add $3 \cdot 4 = 12$ to $P[(0, 1)]$
- Entry $(1, 0), (0, 1)$: Add $5 \cdot 4 = 20$ to $P[(1, 0)]$
- Entry $(1, 1), (1, 0)$: Add $2 \cdot 7 = 14$ to $P[(1, 1)]$

Result: $P = [0, 12, 20, 14]$ (indexed by prefix $(0, 0), (0, 1), (1, 0), (1, 1)$).

Also store $F[p] = \tilde{f}(p)$:

$F = [\tilde{f}(0, 0), 2, 1, 3]$.

Run dense sum-check on $\tilde{P}(p) \cdot \tilde{F}(p)$ for 2 rounds.

This is a size-4 sum-check (not size-16). Suppose after rounds 1-2, we get challenges (r_1, r_2).

Stage 2: Verify Stage 1's evaluation claim.

Stage 1 ended with the claim "$\tilde{P}(r_1, r_2) \cdot \tilde{F}(r_1, r_2) = v_1$." The verifier can check $\tilde{F}(r_1, r_2)$ via polynomial commitment, but $\tilde{P}(r_1, r_2)$ is defined as a sum:

$$\tilde{P}(r_1, r_2) = \sum_{s \in \{0,1\}^2} \tilde{a}((r_1, r_2), s) \cdot \tilde{h}(s)$$

Stage 2 is a second sum-check to prove this. Define arrays indexed by suffix $s \in \{0, 1\}^2$:

$$H[s] = \tilde{a}((r_1, r_2), s)$$

$$Q[s] = \tilde{h}(s)$$

To build H, first precompute the Lagrange table for all 4 Boolean prefixes:

$\widetilde{\mathsf{eq}}((0,0),(r_1,r_2)) = (1-r_1)(1-r_2)$ $\widetilde{\mathsf{eq}}((0,1),(r_1,r_2)) = (1-r_1)\cdot r_2$ $\widetilde{\mathsf{eq}}((1,0),(r_1,r_2)) = r_1\cdot(1-r_2)$
$\widetilde{\mathsf{eq}}((1,1),(r_1,r_2)) = r_1 \cdot r_2$

This takes $O(4) = O(2^{n/2})$ field operations. Now scan the 3 sparse entries, looking up weights from the table:

- Entry $(p,s) = ((0,1),(0,1))$, $\tilde{a} = 3$: Add $3 \cdot \widetilde{\mathsf{eq}}((0,1),(r_1,r_2)) = 3(1-r_1)r_2$ to $H[(0,1)]$
- Entry $(p,s) = ((1,0),(0,1))$, $\tilde{a} = 5$: Add $5 \cdot \widetilde{\mathsf{eq}}((1,0),(r_1,r_2)) = 5r_1(1-r_2)$ to $H[(0,1)]$
- Entry $(p,s) = ((1,1),(1,0))$, $\tilde{a} = 2$: Add $2 \cdot \widetilde{\mathsf{eq}}((1,1),(r_1,r_2)) = 2r_1r_2$ to $H[(1,0)]$

Result: $H = [0,\ 3(1-r_1)r_2 + 5r_1(1-r_2),\ 2r_1r_2,\ 0]$.

Building Q: Just copy from the $\tilde{h}$ values: $Q = [\tilde{h}(0,0), 4, 7, \tilde{h}(1,1)]$.

Run dense sum-check on $\tilde{H}(s)\cdot\tilde{Q}(s)$ for 2 rounds to prove $\sum_s H[s]\cdot Q[s] = \tilde{P}(r_1,r_2)$.

Work analysis:

- Stage 1: $O(T)$ to build P + $O(2^{n/2})$ for dense sum-check = 3 + 4 = 7 operations
- Stage 2: $O(2^{n/2})$ to precompute $\widetilde{\mathsf{eq}}$ table + $O(T)$ to build H + $O(2^{n/2})$ for dense sum-check = 4 + 3 + 4 = 11 operations
- **Total:** $O(T + 2^{n/2})$ **= 18 operations** instead of $O(N) = 16$ for the dense approach

(In this tiny example, sparse isn't faster because $T = 3$ and $2^{n/2} = 4$ are similar to $N = 16$. The win comes at scale.)

For realistic parameters ($N = 2^{30}, T = 2^{20}$), the savings are dramatic: $O(2^{20} + 2^{15})$ instead of $O(2^{30})$, a 1000× speedup.

Generalization to c Chunks

Split into c chunks instead of 2. Each stage handles n/c variables:

- Time: $O(c \cdot T + c \cdot N^{1/c})$
- Space: $O(N^{1/c})$

Choosing $c \approx \log N / \log\log N$ yields prover time $O(T \cdot \text{polylog}(N))$ with polylogarithmic space. The prover runs in time proportional to the number of non-zero terms, with only logarithmic overhead.

Spartan: Sum-Check for R1CS

What's the simplest possible SNARK?

Not in terms of assumptions (transparent or trusted setup, pairing-based or hash-based). In terms of *conceptual machinery*. What's the minimum set of ideas needed to go from "here's a constraint system" to "here's a succinct proof"?

Spartan (Setty, 2020) provides a surprisingly clean answer: sum-check plus polynomial commitments. Nothing else. No univariate encodings, no FFTs over roots of unity, no quotient polynomials, no PCP constructions. Just the two building blocks we've already developed.

The R1CS Setup

An R1CS instance consists of sparse matrices $A, B, C \in \mathbb{F}^{m \times n}$ and a constraint: find a witness $z \in \mathbb{F}^n$ such that

$$Az \circ Bz = Cz$$

where $\circ$ denotes the Hadamard (entrywise) product. Each row of this equation is a constraint; the system has m constraints over n variables.

The Multilinear View

Interpret the witness z as evaluations of a multilinear polynomial $\tilde{z}$ over the Boolean hypercube $\{0,1\}^{\log n}$:

$$z_i = \tilde{z}(i) \quad \text{for } i \in \{0,1\}^{\log n}$$

Similarly, view the matrices A, B, C as bivariate functions: $A(i,j)$ is the entry at row i, column j. Their multilinear extensions $\tilde{A}, \tilde{B}, \tilde{C}$ are defined over $\{0,1\}^{\log m} \times \{0,1\}^{\log n}$.

The constraint $Az \circ Bz = Cz$ becomes: for every row index $x \in \{0,1\}^{\log m}$,

$$\left(\sum_{y \in \{0,1\}^{\log n}} \tilde{A}(x,y) \cdot \tilde{z}(y)\right) \cdot \left(\sum_{y \in \{0,1\}^{\log n}} \tilde{B}(x,y) \cdot \tilde{z}(y)\right) = \sum_{y \in \{0,1\}^{\log n}} \tilde{C}(x,y) \cdot \tilde{z}(y)$$

Define the error at row x:

$$g(x) = \left(\sum_y \tilde{A}(x,y)\tilde{z}(y)\right) \cdot \left(\sum_y \tilde{B}(x,y)\tilde{z}(y)\right) - \sum_y \tilde{C}(x,y)\tilde{z}(y)$$

The R1CS constraint is satisfied iff $g(x) = 0$ for all $x \in \{0,1\}^{\log m}$.

This multilinear view differs from the QAP approach in Chapter 12 (Groth16). There, R1CS matrices become univariate polynomials via Lagrange interpolation over roots of unity. The constraint $Az \circ Bz = Cz$ transforms into a polynomial divisibility condition: $A(X) \cdot B(X) - C(X) = H(X) \cdot Z_H(X)$, where Z_H is the vanishing polynomial over the evaluation domain. Proving satisfaction means exhibiting the quotient $H(X)$.

Spartan takes a different path. Instead of interpolating over roots of unity, it interprets vectors and matrices as multilinear extensions over the Boolean hypercube. Instead of checking divisibility by a vanishing polynomial, it checks that an error polynomial evaluates to zero on all Boolean inputs, via sum-check. No quotient polynomial, no FFT, no roots of unity. Just multilinear algebra and sum-check.

Both approaches reduce R1CS to polynomial claims. QAP reduces to divisibility; Spartan reduces to vanishing on the hypercube. The sum-check approach avoids the $O(n \log n)$ FFT costs and the trusted setup of pairing-based SNARKs, at the cost of larger proofs (logarithmic in the constraint count rather than constant).

The Zero-on-Hypercube Reduction

Spartan's R1CS encoding requires checking that g vanishes on the Boolean hypercube, i.e., $g(x) = 0$ for all $x \in \{0,1\}^{\log m}$. The technique that handles this reduces 2^n separate equality checks to a single sum-check, and works for any polynomial, not just R1CS errors.

A natural first attempt is to prove $\sum_{x \in \{0,1\}^n} g(x) = 0$ via sum-check. If g vanishes on the hypercube, this sum is indeed zero. But the converse fails. Suppose $g(0,0) = 5$ and $g(0,1) = -5$ with $g(1,0) = g(1,1) = 0$; then $\sum_x g(x) = 0$ despite g being non-zero at two points. Positive and negative values cancel. A bare sum cannot distinguish "all zeros" from "zeros that happen to add up."

The fix is to weight each term with a pseudorandom coefficient so that accidental cancellation becomes overwhelmingly unlikely. Recall from Chapter 4 the equality polynomial $\widetilde{\mathsf{eq}} : \mathbb{F}^n \times \mathbb{F}^n \to \mathbb{F}$:

$$\widetilde{\mathsf{eq}}(\tau, x) = \prod_{i=1}^{n} (\tau_i x_i + (1 - \tau_i)(1 - x_i))$$

On Boolean inputs, each factor equals 1 when $\tau_i = x_i$ and 0 when they differ, so the product is the indicator $\mathbf{1}[\tau = x]$. The formula extends smoothly to all field elements: this is the multilinear extension of the equality indicator over $\{0,1\}^n \times \{0,1\}^n$. By the MLE evaluation formula (Chapter 4), $\sum_x \widetilde{\mathsf{eq}}(\tau, x) \cdot f(x) = \tilde{f}(\tau)$ for any f with multilinear extension $\tilde{f}$.

The reduction works as follows. The verifier samples random $\tau \in \mathbb{F}^{\log m}$ and asks the prover to demon-

strate:

$$\sum_{x \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(\tau, x) \cdot g(x) = 0$$

This sum is a random linear combination of $\{g(x)\}_{x \in \{0,1\}^n}$, with coefficients determined by τ. If every $g(x) = 0$, the sum is trivially zero. If even one $g(x^*) \neq 0$, the sum is nonzero with high probability because the pseudorandom weights prevent cancellation. The equality polynomial turns "check 2^n values are all zero" into "check one random linear combination is zero."

We can bound the probability that a cheating prover passes this check. Define $Q(\tau) = \sum_x \widetilde{\mathsf{eq}}(\tau, x) \cdot g(x) = \tilde{g}(\tau)$. Let $\mathsf{ZERO}(g) := \forall x \in \{0,1\}^n, g(x) = 0$. Then:

$$\Pr_{\tau \leftarrow \mathbb{F}^n} [\tilde{g}(\tau) = 0 \mid \neg \mathsf{ZERO}(g)] \leq \frac{n}{|\mathbb{F}|}$$

Proof. If $\neg\mathsf{ZERO}(g)$, then $\tilde{g}$ is a nonzero multilinear polynomial (since $\tilde{g}(x) = g(x) \neq 0$ for some Boolean x). A nonzero multilinear polynomial has total degree at most n. By Schwartz-Zippel, a nonzero polynomial of degree d over $\mathbb{F}$ has at most $d \cdot |\mathbb{F}|^{n-1}$ roots in $\mathbb{F}^n$. Thus the probability of hitting a root is at most $n \cdot |\mathbb{F}|^{n-1}/|\mathbb{F}|^n = n/|\mathbb{F}|$. □

This reduces "check g vanishes on 2^n points" to "run sum-check on one random linear combination and verify it equals zero."

Spartan's outer sum-check

1. **Verifier sends** random $\tau \in \mathbb{F}^{\log m}$
2. **Prover claims** $\sum_{x \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(\tau, x) \cdot g(x) = 0$, where $g(x) = \left(\sum_y \tilde{A}(x,y)\tilde{z}(y)\right) \cdot \left(\sum_y \tilde{B}(x,y)\tilde{z}(y)\right) - \sum_y \tilde{C}(x,y)\tilde{z}(y)$
3. **Run sum-check** on this claim

At the end, the verifier holds a random point $r \in \mathbb{F}^{\log m}$ and needs to evaluate $g(r)$. This requires three matrix-vector products: $\sum_y \tilde{A}(r,y)\tilde{z}(y)$, $\sum_y \tilde{B}(r,y)\tilde{z}(y)$, and $\sum_y \tilde{C}(r,y)\tilde{z}(y)$.

Spartan's inner sum-checks

Each of these is itself a sum over the hypercube, requiring three more sum-checks. But now the sums are over y, and the polynomials have the form $\tilde{M}(r,y) \cdot \tilde{z}(y)$ for the fixed r from the outer sum-check.

After running these three inner sum-checks (which can be batched into one using random linear combinations), the verifier holds a random point $s \in \mathbb{F}^{\log n}$ and needs to check:

- $\tilde{A}(r,s)$, $\tilde{B}(r,s)$, $\tilde{C}(r,s)$: evaluations of the matrix MLEs
- $\tilde{z}(s)$: evaluation of the witness MLE

The matrix evaluations are handled by SPARK (below). The witness evaluation $\tilde{z}(s)$ is where polynomial commitments enter: the prover opens the committed $\tilde{z}$ at the random point s, and the verifier checks the opening proof.

This is the full reduction: R1CS satisfaction → zero-on-hypercube (outer sum-check) → matrix-vector products (inner sum-checks) → point evaluations (polynomial commitment openings).

Handling sparse matrices with SPARK

The inner sum-checks end with evaluation claims: the verifier needs $\tilde{A}(r,s)$, $\tilde{B}(r,s)$, $\tilde{C}(r,s)$ at the random points $(r,s) \in \mathbb{F}^{\log m} \times \mathbb{F}^{\log n}$ produced by the protocol. But the matrices A, B, C are $m \times n$, and a dense representation costs $O(mn)$ space. Committing to them naively would dominate the entire protocol.

R1CS matrices are sparse. A circuit with m constraints typically has only $O(m)$ non-zero entries total, not $O(mn)$. A sparse matrix with T non-zero entries can be stored as a list of (i,j,v) tuples at $O(T)$ cost. The question is how to evaluate the matrix MLEs at (r,s) from this sparse representation.

Applying the MLE evaluation formula to the bivariate function $M(i,j)$ gives:

$$\tilde{M}(r_x, r_y) = \sum_{(i,j) \in \{0,1\}^{\log m} \times \{0,1\}^{\log n}} M(i,j) \cdot \widetilde{\mathsf{eq}}(i, r_x) \cdot \widetilde{\mathsf{eq}}(j, r_y)$$

Since $M(i,j) = 0$ for most entries, this simplifies to a sum over only the T non-zero entries:

$$\tilde{M}(r_x, r_y) = \sum_{(i,j): M(i,j) \neq 0} M(i,j) \cdot \widetilde{\mathsf{eq}}(i, r_x) \cdot \widetilde{\mathsf{eq}}(j, r_y)$$

For each non-zero entry (i,j,v), we need $\widetilde{\mathsf{eq}}(i, r_x)$ and $\widetilde{\mathsf{eq}}(j, r_y)$. Computing $\widetilde{\mathsf{eq}}(i, r_x)$ directly from the formula $\prod_k (i_k \cdot (r_x)_k + (1 - i_k)(1 - (r_x)_k))$ costs $O(\log m)$. Over T entries, total cost: $O(T \log m)$.

SPARK reduces this to $O(T)$ by precomputing lookup tables.

1. **Precompute row weights.** Build a table $E_{\mathsf{row}}[i] = \widetilde{\mathsf{eq}}(i, r_x)$ for all $i \in \{0,1\}^{\log m}$. This costs $O(m)$ using the standard MLE evaluation algorithm (stream through bit-vectors, accumulate products).
2. **Precompute column weights.** Build a table $E_{\mathsf{col}}[j] = \widetilde{\mathsf{eq}}(j, r_y)$ for all $j \in \{0,1\}^{\log n}$. Cost: $O(n)$.

3. **Evaluate via lookups.** Initialize a running sum to zero. For each non-zero entry (i, j, v), look up $E_{\mathsf{row}}[i]$ and $E_{\mathsf{col}}[j]$, then add $v \cdot E_{\mathsf{row}}[i] \cdot E_{\mathsf{col}}[j]$ to the running sum. After processing all T entries, the sum equals $\tilde{M}(r_x, r_y)$. Cost: $O(T)$.

Total: $O(m + n + T)$, linear in the sparse representation size.

The remaining question is who checks the lookups. The prover claims to have read the correct $\widetilde{\mathsf{eq}}$ values from the precomputed tables, but the verifier does not have those tables. SPARK resolves this with a *memory-checking argument*: a protocol that verifies the prover's reads against the table contents by comparing random fingerprints of both. If any lookup is incorrect, the fingerprints mismatch with high probability. Chapter 21 develops this technique in full. The overhead is $O(\log T)$ in proof size and verification time, preserving SPARK's linear prover efficiency.

The full Spartan protocol

Putting it together:

1. **Commitment phase.** The prover commits to the witness $\tilde{z}$ using a multilinear polynomial commitment scheme. The matrices A, B, C are public (part of the circuit description), so no commitment is needed for them.

2. **Outer sum-check.** The verifier sends random $\tau \in \mathbb{F}^{\log m}$. The prover and verifier run sum-check on:

$$\sum_{x \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(\tau, x) \cdot g(x) = 0$$

   This reduces to evaluating $g(r)$ at a random point $r \in \mathbb{F}^{\log m}$.

3. **Inner sum-checks.** Evaluating $g(r)$ requires three matrix-vector products: $\sum_y \tilde{A}(r, y) \cdot \tilde{z}(y)$, $\sum_y \tilde{B}(r, y) \cdot \tilde{z}(y)$, and $\sum_y \tilde{C}(r, y) \cdot \tilde{z}(y)$. The verifier sends random $\rho_A, \rho_B, \rho_C \in \mathbb{F}$, and the parties run a single sum-check on the combined claim:

$$\sum_{y \in \{0,1\}^{\log n}} \left(\rho_A \tilde{A}(r, y) + \rho_B \tilde{B}(r, y) + \rho_C \tilde{C}(r, y)\right) \cdot \tilde{z}(y) = v$$

   where v is the prover's claimed value for the batched sum. At the end of sum-check, the verifier holds a random point $s \in \mathbb{F}^{\log n}$ and a claimed evaluation v_{final} of the polynomial at s.

4. **SPARK.** The prover provides claimed values $\tilde{A}(r, s)$, $\tilde{B}(r, s)$, $\tilde{C}(r, s)$ and proves they're consistent with the sparse matrix representation via memory-checking fingerprints.

5. **Witness opening.** The prover opens $\tilde{z}(s)$ using the polynomial commitment scheme. The verifier checks the opening proof and obtains the value $\tilde{z}(s)$.

6. **Final verification.** The verifier computes $\left(\rho_A \tilde{A}(r,s) + \rho_B \tilde{B}(r,s) + \rho_C \tilde{C}(r,s)\right) \cdot \tilde{z}(s)$ using the values from steps 4-5, and checks that it equals the final claimed value v_{final} from the inner sum-check. This is the "reduction endpoint": if the prover cheated anywhere in the sum-check, this equality fails with high probability.

Complexity

Component	Prover	Verifier	Communication	Technique
Outer sum-check	$O(m)$	$O(\log m)$	$O(\log m)$	Halving trick
Inner sum-checks	$O(n)$	$O(\log n)$	$O(\log n)$	Halving trick + batching
SPARK	$O(T)$	$O(\log T)$	$O(\log T)$	Precomputed $\widetilde{\text{eq}}$ tables + memory checking
Witness commitment	depends on PCS	depends on PCS	depends on PCS	Multilinear PCS (IPA, FRI, etc.)

Why each step achieves its complexity:

- **Outer sum-check** $O(m)$**:** The halving trick from earlier in this chapter. Instead of recomputing $2^{\log m} = m$ terms each round, fold the evaluation tables after each challenge. Total work across all $\log m$ rounds: $m + m/2 + m/4 + ... = O(m)$.
- **Inner sum-checks** $O(n)$**:** Same halving trick, but applied to three matrix-vector products at once. Batching with random coefficients ρ_A, ρ_B, ρ_C combines the three sums into one sum-check, avoiding a $3\times$ overhead.
- **SPARK** $O(T)$**:** Precompute $\widetilde{\text{eq}}(i, r_x)$ for all row indices and $\widetilde{\text{eq}}(j, r_y)$ for all column indices in $O(m+n)$ time. Then each of the T non-zero entries requires only two table lookups and one multiplication, with no logarithmic-cost $\widetilde{\text{eq}}$ computations per entry. Memory-checking fingerprints verify the lookups in $O(T)$ additional work.
- **Verifier** $O(\log m + \log n + \log T)$**:** The verifier never touches the full tables. In sum-check, it receives $O(d)$ evaluations per round and performs $O(1)$ field operations to check consistency. Over $\log m + \log n$ rounds, that's $O(\log m + \log n)$ work. SPARK verification adds $O(\log T)$ for the memory-checking fingerprint comparison.

With T non-zero matrix entries, total prover work is $O(m + n + T)$, linear in the instance size. No trusted setup is required when using IPA or FRI as the polynomial commitment.

Why This Matters

Step back and consider what we've built. Spartan proves R1CS satisfaction, the standard constraint system for zkSNARKs, using only sum-check and polynomial commitments. No univariate polynomial encodings (like PLONK's permutation argument). No pairing-based trusted setup (like Groth16). No PCP constructions (like early STARKs).

The architecture is minimal: multilinear polynomials, sum-check, commitment scheme. Three ideas, combined cleanly. This simplicity is the reason Spartan became the template for subsequent systems. Lasso added lookup arguments; Jolt extended further to prove virtual machine execution. Each built on the same foundation.

Notice the graph structure emerging. Spartan has two levels: an outer sum-check (over constraints) and inner sum-checks (over matrix-vector products). The outer sum-check ends with a claim; the inner sum-checks prove that claim. This is exactly the depth-two graph from the remark at the chapter's start. More complex protocols like Lasso (for lookups) and Jolt (for full RISC-V execution) extend this graph to dozens of nodes across multiple stages, but the pattern remains: sum-checks reducing claims to other sum-checks, bottoming out at committed polynomials.

When a construction is this simple, it becomes a building block for everything that follows.

The PCP Detour and Sum-Check's Return

Now that we've seen Spartan's architecture (sum-check plus commitments, nothing more), the historical question becomes pressing: why did the field spend two decades pursuing a different path?

The PCP path

In 1990, sum-check arrived. Two years later, the PCP theorem landed: every NP statement has a proof checkable by reading only a constant number of bits. This captured the field's imagination completely.

The PCP theorem seemed to obsolete sum-check. Why settle for logarithmic verification when you could have constant-query verification? Kilian showed how to compile PCPs into succinct arguments: commit to the PCP via Merkle tree, let the verifier query random locations, authenticate responses with hash paths. This became *the* template for succinct proofs.

Sum-check faded into the background, remembered as a stepping stone rather than a destination.

The redundant indirection

In hindsight, the PCP-based pipeline contained a redundancy. The PCP theorem transforms an interactive proof into a *static* proof string that the verifier queries non-adaptively. Interaction removed. But the proof string is enormous, so Kilian's construction has the prover commit to it via a Merkle tree and the verifier interactively requests query locations. Interaction reintroduced. Then Fiat-Shamir makes the protocol non-interactive. Interaction removed again.

The transformations: IP → PCP (remove interaction) → Kilian argument (add interaction back) → Fiat-Shamir (remove interaction again). Two removals of interaction. If Fiat-Shamir handles the final step anyway, why not apply it directly to the original interactive proof based on sum-check?

The return

Starting around 2018, the missing pieces fell into place: fast proving algorithms (the halving trick, sparse sums) and polynomial commitment schemes (KZG, FRI, IPA) that could handle multilinear polynomials directly. A wave of systems returned to sum-check:

- **Hyrax** (2018), **Libra** (2019): early sum-check-based SNARKs with linear-time provers
- **Spartan** (2020): sum-check for R1CS without trusted setup
- **HyperPlonk** (2023): sum-check meets Plonkish arithmetization
- **Lasso/Jolt** (2023-24): sum-check plus lookup arguments for zkVMs
- **Binius** (2024): sum-check over binary fields

The pattern: sum-check as the core interactive proof, polynomial commitments for cryptographic binding, Fiat-Shamir applied once.

What the PCP path got right

The architectural redundancy does not mean the PCP path was wasted. It produced STARKs, which remain among the most deployed proof systems. STARKs compile an IOP (the AIR + FRI pipeline from Chapter 15) using only hash functions, no elliptic curves. This gives them a property that sum-check-based systems struggle to match: post-quantum security out of the box.

Sum-check itself is information-theoretic and quantum-safe. But it produces evaluation claims that must be resolved by a polynomial commitment scheme, and the most mature multilinear PCS options (KZG, IPA, Dory) rely on discrete-log assumptions that Shor's algorithm breaks. Post-quantum alternatives exist: hash-based multilinear commitments and lattice-based schemes are active areas of research, but they remain less mature than the FRI-based commitments that STARKs use today.

The practical landscape reflects this. For applications where post-quantum security matters now (long-lived proofs, regulatory environments, sovereign infrastructure), STARKs offer a proven path. For applications where prover speed dominates and classical assumptions suffice, sum-check-based systems like Jolt and Binius achieve prover times closer to the witness computation itself. The two approaches are converging: Binius uses sum-check over binary fields with FRI-based commitments, combining both traditions. Chapter 20 develops the STARK-side optimization story in parallel with this chapter, showing how small-field techniques, NTT optimization, and FRI batching close the gap between STARK proving and witness computation from the other direction.

Key takeaways

1. **The halving trick achieves $O(N)$ prover time.** Fold evaluation tables after each challenge: $N \to N/2 \to N/4 \to \ldots$ via multilinear interpolation. Total work is the geometric series $N + N/2 + \cdots = O(N)$.

2. **Not all field multiplications are equal.** Over 256-bit fields, bb multiplications are roughly 8x more expensive than sb and 30x more expensive than ss. Delayed reduction amortizes modular reduction across linear combinations. These distinctions dominate wall-clock time despite being invisible in $O(\cdot)$ notation.

3. **High-degree products cost $O(d \log d)$, not $O(d^2)$.** A divide-and-conquer algorithm splits d factors in half, recurses, extrapolates via Lagrange (sb work), and multiplies pointwise (bb work). Only the pointwise step is expensive.

4. **Small-value round-batching exploits the geometric series.** The first v rounds dominate total work and operate on small witness values. Treating these variables as symbolic replaces bb multiplications with ss, reducing the concrete cost of the most expensive portion of the protocol.

5. **Streaming provers trade passes for memory.** Applying round-batching iteratively gives $O(N^{1/k})$ space for any $k \geq 2$, without recursive proof composition.

6. **Sparse sums exploit separable structure.** When the polynomial factors into a sparse selector and dense prefix/suffix components, two chained sum-checks over $n/2$ variables each achieve $O(T + 2^{n/2})$ cost instead of $O(2^n)$.

7. **Spartan reduces R1CS to sum-check.** The zero-on-hypercube reduction converts "g vanishes on $\{0,1\}^n$" into a single sum-check weighted by $\widetilde{\mathsf{eq}}(\tau, x)$, which acts as a random linear combination preventing cancellation. An outer sum-check ($O(m)$) plus batched inner sum-checks ($O(n)$) plus SPARK ($O(T)$) handle the full R1CS constraint system.

8. **Sum-check graphs structure complex protocols.** Each sum-check ends with evaluation claims. If the polynomial is committed, open it. If it is virtual, another sum-check proves the evaluation. The result is a DAG where depth determines sequential stages and width enables batching. Chapter 21 develops this perspective.

9. **The PCP path and the sum-check path are converging.** The IP → PCP → Kilian → Fiat-Shamir pipeline contains an architectural redundancy (interaction removed, reintroduced, removed again). Sum-check + Fiat-Shamir skips this. But the PCP lineage produced STARKs, which offer post-quantum security via hash-based commitments. Sum-check systems need a post-quantum PCS to match, and those remain less mature. Binius bridges both traditions: sum-check over binary fields with FRI-based commitments.

Chapter 20: Fast STARK Proving

> *This chapter is part of Part VI (Prover Optimization, Chapters 19-21), which is optional on a first read. The rest of the book does not depend on it. The material here is essential for anyone designing or implementing a STARK prover.*
>
> *Specific prerequisites: fluency with the STARK pipeline (Chapter 15), FRI (Chapter 10), and the small-field/small-value ideas from Chapter 19. This chapter parallels Chapter 19's treatment of sum-check prover optimization, now applied to the STARK side. Together they give a complete picture of how both proof traditions close the gap between witness computation and proof generation.*

A STARK prover does far more work than the computation it proves. Executing a million steps of a hash function takes microseconds. Generating a proof of that execution takes seconds. The prover overhead, the ratio of proof generation time to raw computation time, exceeded 1000× in early systems. Where does all that prover time go?

The answer is not one bottleneck but a shifting pipeline of them. For small traces, constraint evaluation and polynomial arithmetic consume most of the prover's cycles. For medium traces, the number-theoretic transform (NTT) takes over, since its $O(N \log N)$ cost eventually dominates linear-time constraint evaluation. For the largest traces, Merkle hashing for FRI commitments becomes the wall. Profiling data from production provers confirms this progression: NTT can account for up to 91% of prover runtime in workloads dominated by polynomial operations, while Merkle tree construction dominates at roughly 60% in hash-intensive recursive proving workloads. The prover engineer's task is to push each bottleneck down until the next one surfaces, then push that one down too.

The trajectory of improvement across the ecosystem has been dramatic. Early STARK provers (circa 2021) achieved roughly 10,000 Poseidon hashes per second. By 2024, provers built on small-field techniques and Circle STARKs over Mersenne31 exceeded 500,000 Poseidon2 hashes per second on com-

modity quad-core hardware, with some configurations surpassing 620,000 per second. That is a 50× improvement from algorithmic and field-choice optimizations alone, without GPU acceleration. Multiple independent teams converged on similar techniques: 31-bit prime fields, AIR-based constraint systems, batched FRI, LogUp bus arguments. Much of this convergence crystallized around **Plonky3** (Polygon), an open-source framework providing shared field arithmetic (BabyBear, Mersenne31), an AIR trait interface, and a modular FRI backend. SP1 (Succinct), OpenVM (Axiom/Scroll), and several other production provers build on Plonky3, while StarkWare's Stwo and RISC Zero's prover implement the same ideas independently. Understanding the shared principles behind these gains is the subject of this chapter.

The prover pipeline

The STARK prover executes a sequence of stages, each feeding into the next. Understanding where time goes requires tracing this pipeline end to end. The following variables recur throughout this chapter:

- T: trace length (number of rows/timesteps), always a power of two
- w: trace width (number of columns/registers)
- d: maximum constraint degree across all AIR transition polynomials
- ρ: blowup factor, the ratio $|D|/|H|$ between the LDE evaluation domain and the trace domain (typically 2, 4, or 8)
- λ: number of FRI query repetitions (security parameter)
- c_h: cost of one hash invocation measured in field multiplications

Stage 1: Trace generation. The prover runs the computation, filling the execution trace, a matrix with w columns (registers) and T rows (timesteps). For a hash function like Poseidon with 30 rounds and state width 12, the trace might have 12-24 columns and $30 \cdot B$ rows for B input blocks. This stage performs the same arithmetic the original computation would, plus bookkeeping for each intermediate state. Cost: $O(w \cdot T)$ field operations with a small constant per cell.

Stage 2: Constraint evaluation. The prover evaluates the AIR constraint polynomials at every row. If the maximum constraint degree is d over w registers, each row costs $O(d \cdot w)$ field operations. Total: $O(d \cdot w \cdot T)$.

Stage 3: Composition and quotient formation. The prover forms the composition polynomial by batching all constraint quotients with random Fiat-Shamir challenges (Chapter 15). The composition polynomial has degree roughly $d \cdot T$.

Stage 4: Low-degree extension (LDE). The prover evaluates trace polynomials and the composition polynomial on a domain D that is ρ times larger than H, where ρ is the blowup factor. The evaluation proceeds by inverse NTT (interpolation from H to coefficient form) followed by forward NTT (evaluation on D). The **number-theoretic transform** (NTT) is the finite-field analogue of the FFT: it converts between coefficient and evaluation representations of a polynomial over a domain of roots of unity, using the same butterfly algorithm that Chapter 5 developed for the discrete Fourier transform over $\mathbb{F}_p$. Each polynomial requires two NTTs costing $O(\rho T \log(\rho T))$. With w columns, the total is $O(w \cdot \rho T \log(\rho T))$.

Stage 5: Merkle commitment. The prover hashes every row of the LDE matrix into a Merkle tree. The ethSTARK specification (Ben-Sasson et al., 2021), which formalized the production STARK pipeline into a reference document, groups all field elements in a row of the trace LDE into a single leaf, so the tree has ρT leaves. Building it requires $O(\rho T)$ hash invocations.

Stage 6: FRI protocol. The prover executes FRI folding rounds, each halving the polynomial via an NTT and committing the result in a Merkle tree. The total across $\log_2(\rho T/d)$ rounds is a geometric series of NTTs and trees, dominated by the first round.

Stage 7: Query responses. The prover opens Merkle paths at queried positions. This is fast (logarithmic per query) and rarely a bottleneck.

The relative costs shift with scale. At $T = 2^{16}$, constraint evaluation can dominate. At $T = 2^{20}$, NTT takes over. At $T = 2^{24}$, Merkle hashing becomes comparable to NTT. Optimization must address each stage in sequence.

The following table summarizes the cost model. Here w is the trace width, T the trace length, d the maximum constraint degree, ρ the blowup factor, λ the number of FRI queries, and c_h the cost of one hash invocation measured in field multiplications.

Stage	Cost	Dominates when
Trace generation	$O(wT)$	Rarely (linear, small constant)
Constraint evaluation	$O(wT)$	Small T ($< 2^{18}$)
Composition	$O(dT \log(dT))$	High constraint degree d
LDE (NTT)	$O(w\rho T \log(\rho T))$	Medium T (2^{18} to 2^{24})
Merkle commitment	$O(w\rho T \cdot c_h)$	Large T with cheap field arithmetic
FRI (folding + trees)	$O(\rho T \log(\rho T) + \rho T \cdot c_h)$	Comparable to LDE + Merkle combined
Query responses	$O(\lambda \log(\rho T))$	Never

The ratio c_h is what determines the crossover between NTT-dominated and hash-dominated regimes. Over a 256-bit field, c_h is small relative to field multiplication cost, so NTT dominates. Over a 31-bit field like BabyBear, field multiplications become so cheap that c_h grows relatively large, shifting the bottleneck toward Merkle hashing. This ratio is the single most useful diagnostic for predicting where a given prover spends its time.

The cost table reveals two levers for optimization. The first is to reduce the *inputs* to the pipeline: the parameters w, d, and ρ that determine how much work each stage performs. This is the domain of AIR design. The second is to reduce the *cost per operation* within each stage: faster field arithmetic (small fields), better memory access (cache-friendly NTTs), fewer redundant commitments (FRI optimizations). The remainder of this chapter addresses these in order.

AIR design and the degree-blowup tradeoff

The encoding of a computation into an AIR often matters more than any algorithmic optimization applied afterward. Two designs for the same computation can differ in prover time by an order of magnitude, because they feed different values of d and w into the same pipeline. The cost table makes this concrete: doubling d doubles every row from "Composition" downward, while doubling w only adds to "LDE" and "Merkle commitment."

The central tension is between trace width and constraint degree. A wider trace (more columns) with low-degree constraints breaks complex expressions into simpler pieces by storing intermediate values. A narrower trace (fewer columns) requires higher-degree constraints to compress the same logic.

The reason width is cheap and degree is expensive comes from how they propagate through the pipeline. Adding a column costs one extra NTT of size ρT and widens each Merkle leaf; the cost grows linearly in w. Raising the constraint degree from d to $2d$ doubles the composition polynomial's degree, doubling the LDE domain, every NTT, every Merkle tree, and all FRI work. Degree is a *multiplicative* cost on the entire pipeline; width is an *additive* cost on one stage. The heuristic follows: add columns to reduce constraint degree until further splitting no longer lowers d to the next power of two. Production STARK frameworks formalize this by setting the blowup factor to the smallest power of two greater than or equal to the highest transition constraint degree, so any reduction in d that crosses a power-of-two boundary halves the downstream cost.

Consider a transition that computes $y = x^8$. With one column, the constraint is $P(\omega X) - P(X)^8 = 0$, degree 8. With three auxiliary columns storing x^2, x^4, x^8, the constraints become four degree-2 checks, where each squaring is $P_{i+1}(\omega X) - P_i(X)^2$. The constraint degree drops from 8 to 2, at the cost of widening the trace from 1 to 4 columns.

Why does constraint degree matter so much? The composition polynomial has degree roughly $d \cdot T$. FRI must prove a degree bound on this polynomial, so the LDE domain must be at least $d \cdot T \cdot \rho$ points. Every doubling of d doubles the NTT size, the Merkle tree size, all FRI operations. A degree-8 constraint over $T = 2^{20}$ rows produces a composition polynomial of degree $\approx 2^{23}$, requiring an LDE domain of 2^{25} at blowup $\rho = 4$. Reducing to degree 2 drops the LDE domain to 2^{23}, a 4× reduction in all subsequent stages. Most production systems keep constraint degree between 2 and 4.

The overarching recipe: keep $d \leq 4$ by trading additive cost in w for multiplicative savings through d. Two design patterns achieve this (periodic columns and trace widening), while a third pattern (interaction columns) addresses a different problem: extending AIR expressiveness to handle constraints that span distant rows.

Reducing degree: periodic columns

Many computations repeat structure at regular intervals. A hash function applies the same round constants in a cycle of length r. A CPU cycles through a fixed instruction decode pattern. Naively, these constants would occupy a committed trace column, adding one NTT and one column's worth of Merkle leaf data. Periodic columns avoid this cost entirely.

A **periodic column** encodes public constants that repeat on a fixed cycle. Suppose a hash function uses $r = 4$ round constants $[c_0, c_1, c_2, c_3]$ and the trace has $T = 16$ rows. The constants repeat: row 0 gets c_0, row 1 gets c_1, …, row 4 gets c_0 again, and so on. The key observation is that on the trace domain $H = \{1, \omega, \omega^2, \ldots, \omega^{15}\}$, the map $\omega^i \mapsto \omega^{4i}$ collapses all rows sharing the same round position to the same value (since $\omega^{4 \cdot 0} = \omega^{4 \cdot 4} = \omega^{4 \cdot 8} = \omega^{4 \cdot 12} = 1$). So the periodic column is really a polynomial of degree $r - 1 = 3$ in the "compressed" variable $X^{T/r} = X^4$, cycling automatically because roots of unity wrap around. Both prover and verifier can compute $c(X)$ from the r public constants without any commitment. The prover saves one NTT of size ρT and all associated Merkle leaf contributions.

The tradeoff is that $c(X)$ contributes degree $r - 1$ when it appears multiplicatively in a constraint. The design rule: use a periodic column when $r - 1 \leq d$, where d is the constraint degree already imposed by other terms. In that case the periodic column adds no degree overhead and saves an entire committed column. If $r - 1 > d$, the periodic column would raise the effective constraint degree, potentially crossing a power-of-two boundary and doubling all downstream costs. In that case, commit the constants as a regular trace column instead.

Poseidon2 illustrates the tension. The S-box degree is 5 (or 3 after decomposition into auxiliary columns), while the round-constant cycle has $r = 8$. Since $r - 1 = 7 > 3$, treating the round constants as periodic would push d from 3 to 7. Most implementations therefore either commit the round constants as ordinary columns or restructure the cycle into shorter sub-periods.

Reducing degree: wide versus tall traces

The principle above (add columns to reduce degree) applies to the overall trace architecture, not only to individual constraints like x^8. The total number of trace cells $w \cdot T$ is roughly fixed by the computation, so the question is how to partition that area: many columns with few rows, or few columns with many rows?

For hash functions, where the computation is regular and the state width is fixed, the trace width maps naturally to the state size. For virtual machines, the choice is less obvious. A zkVM instruction like `ADD R1, R2, R3` touches three registers, a program counter, various flags. Representing all of these as separate columns creates a wide trace (50-100 columns in practice) with degree-2 or degree-3 constraints. Alternatively, encoding multiple values per column via bit-packing creates a narrower trace with higher-degree constraints to extract individual fields.

Production systems overwhelmingly favor wide traces. When $w \cdot T$ is fixed, doubling w while halving T leaves the Merkle commitment cost ($w \cdot \rho \cdot T$) unchanged and saves roughly one butterfly stage per NTT ($\log(\rho T/2)$ vs $\log(\rho T)$). But the decisive reason is that wider traces enable lower constraint degree, and as established above, each halving of d that crosses a power-of-two boundary cuts all downstream costs in half.

Extending expressiveness: interaction columns and LogUp

Periodic columns and trace widening reduce the cost of constraints the AIR can already express. But there is a class of constraints that a pure AIR *cannot* express at all: relationships between non-adjacent rows. The AIR model from Chapter 15 sees only consecutive pairs (row i, row $i + 1$). Memory consistency, lookup arguments, and register-file reads all require matching values across distant rows. This is not a control-flow issue (a `JUMP` merely updates the program counter between adjacent rows) but a *data consistency* problem: values written at one timestep must be readable at another.

Consider a concrete example. A zkVM executes `LOAD R1, [addr]` at row 3,912, reading a value that was written by `STORE R1, [addr]` at row 47. The transition constraint at row 3,912 can verify that the instruction is well-formed (correct opcode, valid register index), but it sees only rows 3,912 and 3,913. It has no way to reach back to row 47 and check that the loaded value matches what was stored there.

The solution is to avoid checking distant-row consistency directly. Instead, the prover builds a *running summary* of all writes and all reads, then proves the two summaries match. If every read returned the value that was written, the summaries agree; if any read was faked, they disagree with overwhelming probability. The summary itself is stored as an auxiliary column that accumulates one entry per row, turning the global consistency check into a local transition constraint (each row updates the running total).

For this to be sound, the prover must not be able to choose summary values that hide an inconsistency. The protocol achieves this by making the summary depend on a random challenge β that the verifier provides (via Fiat-Shamir) *after* the prover has already committed to the main trace. The prover then extends the trace with **auxiliary columns** computed from β. Because β was unknown when the main trace was fixed, the prover cannot game the summary.

The most widely deployed version of this idea is **LogUp** (Chapter 14). If two multisets are equal, then for a random β the sums $\sum_i \frac{1}{a_i - \beta}$ and $\sum_j \frac{m_j}{t_j - \beta}$ must agree (where m_j counts how many times table entry t_j was looked up). This global sum-equality becomes a *local* transition constraint by introducing an accumulator column Z. Row i increments Z by $\frac{1}{a_i - \beta}$, so Z walks through the partial sums. A boundary constraint checks $Z_0 = 0$ and Z_T equals the expected table-side sum. If any lookup is invalid (some $a_i \notin t$), the sums disagree with probability $1 - T/|\mathbb{F}|$ over the random choice of β.

AIR constraints are polynomial equations; they can multiply but not divide. The prover cannot write $\frac{1}{a_i - \beta}$ directly in a constraint. Instead, the prover stores each reciprocal as a witness value in an auxiliary column h_i and adds the constraint $h_i \cdot (a_i - \beta) = 1$, which is degree 2. The verifier never computes the division; it just checks that the product equals 1. Each LogUp bus therefore adds 2 auxiliary columns: the accumulator Z and the reciprocal column h.

Each of these columns requires one additional NTT of size ρT and widens the Merkle leaf. For a zkVM with three buses (memory, instruction lookup, range checks), the auxiliary columns total roughly 6, increasing w by 6. Compared to the main trace width of 50-100, this is a 6-12% increase in per-column costs. Unlike periodic columns and trace widening, interaction columns are not an optimization. They are a requirement. A pure AIR without auxiliary columns cannot express memory consistency, lookup arguments, or any constraint relating non-adjacent rows. LogUp is what makes it possible to build zkVMs on top of the AIR model at all. The cost (a few extra columns per bus) is simply the price of expressiveness.

AIR design reduces the parameters feeding the pipeline. The next three sections reduce the cost *per operation* within the pipeline stages: field arithmetic (this section), the NTT (next section), and FRI (the section after).

Small fields and extension field lifting

Every cost in the pipeline table is measured in field multiplications. Making each multiplication cheaper is the most direct way to speed up the prover. The question is: how small can the field be before soundness breaks?

Chapter 19 showed that sum-check provers exploit small *witness values* within a large 256-bit field. STARK provers take a more radical approach: they work over a genuinely small field, where every element fits in a single machine word.

Chapter 19 introduced the cost hierarchy within a 256-bit field: bb (big-by-big) multiplications dominate because both operands span multiple machine words. Over BN254, a bb multiply splits each 254-bit element into four 64-bit limbs and performs multiple limb-by-limb products with carry propagation, typically 30-50 CPU cycles per field multiplication. STARK provers sidestep this hierarchy entirely by working over a prime small enough that every element fits in a single machine word, eliminating multi-limb arithmetic altogether.

Why 31 bits specifically? The constraint comes from hardware. Multiplying two k-bit values produces a $2k$-bit result. For the product to fit in a single 64-bit register without multi-word handling, we need $2k \leq 64$, giving $k \leq 32$. A 31-bit prime is the largest that satisfies this while leaving room for modular reduction. Two 31-bit field elements multiply via a single 32×32→64 hardware instruction, and the 62-bit result reduces modulo p in 3-4 cycles total. Compare this with the 30-50 cycles for a 256-bit bb multiply. (64-bit primes like Goldilocks, $p = 2^{64} - 2^{32} + 1$, also avoid multi-limb arithmetic since x86 `MUL` produces a 128-bit result in two registers, but the reduction is more expensive and vectorization is half as dense.)

Vectorization amplifies the advantage further. Modern CPUs process multiple field elements in parallel through SIMD (Single Instruction, Multiple Data) registers. A 512-bit vector register packs 16 elements of a 31-bit field (such as BabyBear or Mersenne31, introduced below) side by side, performing 16 independent multiplications in a single instruction. Over a 64-bit field like Goldilocks ($p = 2^{64} - 2^{32} + 1$, the previous generation of STARK-friendly primes), the same register holds only 8 elements. Over BN254, field multiplication cannot be vectorized at all because each multiply already consumes the full register width for its multi-limb computation. The combined speedup from native arithmetic and vectorization exceeds 10× per element compared to 256-bit fields, with some analyses reporting 40× improvement in end-to-end prover time.

The speedup comes at a cost to soundness. Every interactive protocol in this book (sum-check, FRI, DEEP-ALI) derives security from the verifier's random challenges being hard to predict. A cheating prover guessing a random challenge succeeds with probability $1/|\mathbb{F}|$. Over BN254, that probability is 2^{-254}, negligible. Over a 31-bit field, it is 2^{-31}, far from the 2^{-128} target. Shrinking the field made arithmetic cheaper but made each challenge weaker.

The solution is **extension field lifting**, which separates where the *data* lives from where the *randomness* lives. The trace, constraint evaluation, and NTT all operate over the base field $\mathbb{F}_p$ (cheap, 31-bit arithmetic). Verifier challenges, which must be unpredictable, live in an extension field $\mathbb{F}_{p^k}$ for $k = 4$ or $k = 8$. An element of $\mathbb{F}_{p^4}$ is a tuple $(a_0, a_1, a_2, a_3) \in \mathbb{F}_p^4$, representing $a_0 + a_1\alpha + a_2\alpha^2 + a_3\alpha^3$ modulo an irreducible quartic. Multiplication costs roughly 9 base field multiplications via Karatsuba

(compared to 16 for schoolbook expansion). The field size jumps to $p^4 \approx 2^{124}$, providing adequate soundness. Only the parts of the protocol that involve verifier randomness (FRI folding challenges, DEEP-ALI point z, LogUp challenge β) use extension arithmetic; the bulk of the prover's work never leaves the base field.

How much does extension arithmetic actually cost? The two largest stages provide the comparison. The NTT runs once per trace column, processing w polynomials in the base field. FRI folding processes only a single batched polynomial, but each operation uses an extension challenge and therefore costs roughly 9× a base multiplication. So the FRI work, measured in base field operations, is about $9\times$ the cost of a single column's NTT, while the total NTT work scales with all w columns. The extension overhead is a fraction roughly $9/w$ of the NTT cost. For a typical trace with $w = 40$, this is around 22%: a noticeable surcharge, but far from dominant.

Two 31-bit primes dominate modern STARK proving, chosen for different algebraic reasons:

BabyBear ($p = 2^{31} - 2^{27} + 1 = 15 \times 2^{27} + 1$). What matters is the multiplicative group order $p - 1 = 15 \times 2^{27}$. The factor 2^{27} means the field contains a subgroup of order 2^{27}, which serves as the NTT domain. This supports traces with up to $2^{27} \approx 134$ million rows. BabyBear is the field behind RISC Zero's zkVM.

Mersenne31 ($p = 2^{31} - 1$). This is a Mersenne prime, giving the cheapest possible modular reduction: since $2^{31} \equiv 1 \pmod{p}$, reducing a 62-bit product is just splitting it into a low 31-bit half and a high half, adding them, and doing one conditional subtract. No multi-limb arithmetic at all. The tradeoff: $p - 1 = 2(2^{30} - 1)$ has only one factor of 2, so the multiplicative group has no large power-of-two subgroup. Standard NTTs are impossible over this field. Circle STARKs (discussed in the FRI section below) resolve this by replacing the multiplicative group with the circle group $\{(x, y) : x^2 + y^2 = 1\}$, which has order $p + 1 = 2^{31}$. Stwo (StarkWare), Plonky3 (Polygon), and Airbender (ZKsync) all use M31 through this mechanism.

Worked example: counting base multiplications inside an extension multiply.

Let $p = 2013265921$ (BabyBear). A base field multiplication is one hardware operation: $1234567 \times 7654321 = 9449771988807$, which reduces to $9449771988807 - 4p = 1396708123$ with a single conditional subtract.

Now consider the same operation in the quadratic extension $\mathbb{F}_{p^2} = \mathbb{F}_p[\alpha]/(\alpha^2 + 1)$, which uses 2 coefficients per element (we use the quadratic case for clarity; the quartic extends the same idea). Let $a = a_0 + a_1\alpha$ and $b = b_0 + b_1\alpha$. The schoolbook expansion is:

$$ab = a_0b_0 + (a_0b_1 + a_1b_0)\alpha + a_1b_1\alpha^2 = (a_0b_0 - a_1b_1) + (a_0b_1 + a_1b_0)\alpha$$

where the last step uses $\alpha^2 = -1$. This requires 4 base multiplications: a_0b_0, a_1b_1, a_0b_1, a_1b_0.

Standard polynomial-multiplication tricks (Karatsuba's algorithm, which substitutes two of these multiplications with sums of the other products) reduce the count from 4 to 3 in the quadratic case. Applied recursively to the quartic extension, this drops the naive 16 base multiplications to roughly 9. That is the source of the 9 figure used in the overhead estimate above.

Concretely, with $a = 5 + 7\alpha$ and $b = 2 + 3\alpha$: the schoolbook computation gives $(5 \cdot 2 - 7 \cdot 3) + (5 \cdot 3 + 7 \cdot 2)\alpha = -11 + 29\alpha$. Each coefficient of the result requires 2 base multiplications and one addition, totaling 4 base multiplications for the full extension product.

NTT optimization

Small fields make each multiplication cheaper but do nothing to reduce the *number* of multiplications. If anything they raise the count slightly, since the parts of the protocol that touch extension elements pay roughly 9 base multiplies per extension multiply. The pipeline stage that consumes most of those multiplications is the low-degree extension (Stage 4): the prover takes each of the w trace polynomials defined on the trace domain H and re-evaluates it on the larger LDE domain D of size ρT. The algorithm that does this efficiently is the **number-theoretic transform** (NTT), which converts between coefficient and evaluation representations of a polynomial in $O(N \log N)$ field operations.

The arithmetic cost is fixed by the domain size: $O(\rho T \log(\rho T))$ multiplications per polynomial, $O(w\rho T \log(\rho T))$ total across all w trace columns. For a trace with $T = 2^{20}$ rows, $w = 40$ columns, and blowup $\rho = 4$, this comes to roughly $40 \times 4 \times 2^{20} \times 22 \approx 3.7 \times 10^9$ field multiplications in the NTT alone. Even at 3-4 cycles per BabyBear multiply, this is over a second on a single core. For medium-to-large traces, the NTT *is* the prover.

The algorithm is the same Cooley-Tukey butterfly as the FFT from Chapter 5; in the STARK context, "NTT" and "FFT" are interchangeable, with "NTT" emphasizing the finite-field setting. (Lattice cryptography also uses NTTs, but over a different domain: the reduction polynomial is $X^n + 1$ rather than $X^n - 1$, so the transform evaluates at primitive $2n$-th roots of unity and computes a *negacyclic* convolution. The STARK NTT uses n-th roots and computes a standard cyclic convolution, matching the vanishing polynomial $Z_H(X) = X^n - 1$ from Chapter 15.) Beyond the LDE, the prover also runs NTTs in each FRI folding round to extract even/odd parts of the polynomial, but those are smaller and form a geometric series dominated by the first round.

The asymptotic cost is not what makes NTTs hard to optimize. The problem is memory access. Modern CPUs have a small, fast on-chip memory called the **cache**, organized in layers (L1 at roughly 32-64 KB per core, accessed in 4 cycles; L2 at hundreds of KB; L3 in megabytes). Anything not in cache must be fetched from main RAM, which costs 100-300 cycles, fifty times slower than a hit. An algorithm

that touches data already in cache runs near peak compute throughput; an algorithm that constantly misses spends most of its time stalled, waiting for RAM.

An NTT of size N performs $O(N \log N)$ multiplications in $\log N$ "butterfly" stages. Each stage k pairs elements at distance $N/2^k$. The first stages access widely separated memory addresses (stride $N/2$), so each butterfly's two operands sit far apart in memory and almost certainly cause cache misses. The last stages access nearby elements, which are cache-friendly. For $N = 2^{20}$, the first stage's stride is 2^{19} elements ($\approx$ 2 MB for 32-bit fields), far exceeding any L1 or L2 cache. A naive implementation spends most of its time waiting for RAM rather than computing.

The four-step NTT

The **four-step NTT** rearranges the computation so that all but one stage operates on chunks small enough to fit entirely in L1 cache. Instead of one large NTT of size N, the prover treats the data as a $\sqrt{N} \times \sqrt{N}$ matrix and runs many small NTTs of size $\sqrt{N}$.

The procedure has four steps:

1. Perform $\sqrt{N}$ small NTTs of size $\sqrt{N}$, one along each row of the matrix
2. Multiply each element by a twiddle factor (a precomputed root of unity)
3. Transpose the matrix
4. Perform $\sqrt{N}$ small NTTs along each row again

Why this is faster: each small NTT reads and writes only $\sqrt{N}$ elements, which for realistic STARK sizes fits in L1 cache. For $N = 2^{24}$, $\sqrt{N} = 2^{12} = 4096$ elements, occupying $4096 \times 4 = 16$ KB over a 31-bit field. That fits in a 32 KB L1 cache with room to spare. The CPU loads the row once, runs the entire small NTT against fast on-chip memory, and writes the result back. Cache-miss penalties no longer dominate.

The naive NTT, by contrast, has $\log_2 N = 24$ butterfly stages, and the early stages have stride $N/2 = 2^{23}$ elements (≈ 32 MB), causing a cache miss on essentially every butterfly. The four-step layout incurs cache-line-friendly access in steps 1 and 4, and concentrates the unavoidable long-stride memory traffic into a single transposition (step 3) that can be done in cache-friendly blocked fashion.

GPU implementations extend this further. Research on GPU NTT optimization (Özcan, 2023) implements both "Merge" and "4-Step" NTT models for GPU architectures, where the memory hierarchy (global memory → shared memory → registers) creates an analogous cache structure. The four-step decomposition maps naturally to GPU thread blocks, with each block handling one row NTT in fast shared memory.

The blowup factor tradeoff

The four-step decomposition makes each NTT faster at fixed size N. The other way to make NTTs cheaper is to make N smaller. Recall that the NTT operates on the LDE domain D, whose size is $|D| = \rho T$ where $\rho = |D|/|H|$ is the blowup factor. Halving ρ halves N, which directly halves the work in every NTT, every Merkle tree, and every FRI round. The only obstacle is that ρ is not a free parameter: FRI's soundness depends on it.

The mechanism is straightforward. FRI proves that a committed function is close to a low-degree polynomial by spot-checking. A cheating prover who deviates from a low-degree polynomial must disagree with every degree-T polynomial on at least a $1 - 1/\rho$ fraction of the LDE domain (a Reed-Solomon distance bound from Chapter 10). Each random query catches such a deviation with probability at least $1 - 1/\rho$, so λ queries miss the deviation with probability at most $\rho^{-\lambda}$. Each query contributes $\log_2 \rho$ bits of security.

This relationship encodes a direct tradeoff. Going from $\rho = 16$ to $\rho = 2$ shrinks the LDE by 8× (massive prover speedup) but cuts the security per query from 4 bits to 1 bit, requiring 4× more queries to maintain the same target. Each extra query adds Merkle path openings to the proof, increasing proof size.

One subtlety affects the security accounting. The $\log_2 \rho$ figure depends on a strong assumption called the **proximity gap conjecture**. The conjecture concerns batched FRI, where the prover combines w committed polynomials into a single random linear combination $F = \sum \gamma_i P_i$ and runs FRI once on F instead of w times on each P_i. The natural worry is that even if some P_i is far from low-degree, the combination F might *accidentally* land close to low-degree (because the randomness γ_i canceled the deviations). The proximity gap conjecture asserts this cannot happen with non-negligible probability: the set of "bad" γ that move F across the soundness threshold is exponentially small. If true, batched FRI is as sound as running FRI on the worst individual P_i, giving $\log_2 \rho$ bits per query. The fully proven analysis (which makes no conjecture) loses a factor of 2, giving $\frac{1}{2} \log_2 \rho$ bits per query.

For years, deployed STARK systems used the conjectured numbers, halving query counts compared to the proven bound. In late 2025, counterexamples showed the conjecture does not hold in full generality: certain pathological codes admit "bad" γ probabilities that the conjecture's quantitative claim ruled out. The counterexamples do not break Reed-Solomon FRI in deployment, but they invalidate the strongest version of the conjecture and force a re-examination of soundness margins. Production systems are responding by either increasing ρ (widening the safety margin between conjectured and proven security) or switching to the proven analysis directly. The Ethereum Foundation's proving roadmap now targets 100 bits of *provable* security by mid-2026 and 128 bits provable by year-end, driving conservative parameter choices across the ecosystem.

For systems targeting 128-bit security, the parameter choices tighten:

Blowup ρ	Queries for 128 bits (conjectured)	Queries for 128 bits (proven)	LDE size ($T = 2^{20}$)
2	128	256	2^{21}
4	64	128	2^{22}
8	≈ 43	≈ 86	2^{23}
16	32	64	2^{24}

Smaller blowup saves prover time (smaller NTTs and Merkle trees) but increases proof size (more query openings). Most modern systems use $\rho = 2$ combined with grinding to compensate.

For a concrete sense of the tradeoff: moving from $\rho = 2$ to $\rho = 8$ on a typical trace ($T = 2^{20}$, $w = 40$) makes the LDE 4× larger and the NTT work roughly 4.4× more expensive, but cuts the FRI query count by about 3× (from 128 conjectured queries to 43), shrinking the proof from ≈ 2.9 MB to ≈ 1.1 MB at the same grinding. The blowup factor is the primary knob for trading prover speed against proof size.

FRI optimization

The NTT produces evaluations on the LDE domain. The Merkle tree commits them. But the proof is not yet succinct: the commitment alone proves nothing about degree. FRI (Chapter 10) is what makes the Merkle commitment into a polynomial commitment, by interactively testing that the committed function is close to a low-degree polynomial. The base FRI protocol is already efficient, but four optimizations, each addressing a different cost, combine to reduce FRI's contribution to prover time and proof size by an order of magnitude.

DEEP-ALI

The base STARK protocol (Chapter 15) commits to two things separately: the trace polynomials and the composition polynomial. Recall that the composition polynomial is the random linear combination of all constraint quotients, a single polynomial whose low-degree-ness certifies that every transition and boundary constraint is satisfied. Committing it requires a full Merkle tree of ρT leaves, just as expensive as the trace commitment. DEEP-ALI eliminates this second commitment entirely. As a bonus, it tightens FRI's per-query soundness.

The mechanism is a redirection. The composition polynomial is built algebraically from the trace polynomials and known constraint equations. For a Fibonacci-like recurrence $P(\omega X) - P(X) - P(\omega^{-1}X) = 0$, for example, the composition polynomial has the form

$$C(X) = \frac{P(\omega X) - P(X) - P(\omega^{-1}X)}{Z_H(X)}$$

where Z_H is the vanishing polynomial of the trace domain. The key point is that $C(X)$ is *determined* by $P(X)$ and the constraint, not an independent object. Once the verifier knows P's value at a point, it can compute C's value at that point with no help from the prover.

DEEP-ALI exploits this. Instead of asking the prover to commit C separately, the verifier picks a random point z outside the LDE domain D and asks the prover to evaluate the trace at z (and at ωz, to capture the "next row" needed by transition constraints). The prover sends the values $P_j(z)$ and $P_j(\omega z)$ for each trace column. The verifier plugs these into the constraint equations, divides by $Z_H(z)$, and obtains $C(z)$ on its own. No Merkle tree for C is needed; one reconstructed value at one point is all the protocol uses.

For this redirection to be sound, the verifier must check that the prover's claimed evaluation $P_j(z)$ actually agrees with the committed trace polynomial P_j. The trick is the **DEEP quotient**:

$$D_j(X) = \frac{P_j(X) - P_j(z)}{X - z}$$

If the claimed $P_j(z)$ is correct, this quotient is a polynomial of degree $\deg(P_j) - 1$. If the claim is wrong, the numerator does not vanish at z, so D_j has a pole there and is not a polynomial at all. The prover batches all DEEP quotients into a single polynomial via random linear combination and runs FRI on the result. If FRI accepts (i.e., the batched quotient is close to low-degree), then with overwhelming probability every $P_j(z)$ was honest, which means the verifier's reconstructed composition value is correct, which means all constraints hold.

To make the contrast concrete, consider the Fibonacci AIR with one trace column of length $T = 2^{20}$ and blowup $\rho = 4$ (LDE domain size 2^{22}).

Without DEEP-ALI, the prover's commitment phase is:

1. Build a Merkle tree over the 2^{22} trace evaluations on D (one tree).
2. Compute the composition polynomial $C(X)$ on D (one full NTT).
3. Build a Merkle tree over the 2^{22} composition evaluations on D (a second tree).
4. Run FRI on C (folding rounds and additional Merkle trees).

With DEEP-ALI, the same phase becomes:

1. Build a Merkle tree over the 2^{22} trace evaluations on D (one tree).

2. Receive a random point $z \notin D$ from the verifier.
3. Send $P(z)$ and $P(\omega z)$ (two field elements).
4. Form the DEEP quotient $D(X) = (P(X) - P(z))/(X - z)$ on D (a divide-by-linear pass, cheaper than a full NTT).
5. Run FRI on $D(X)$.

The composition polynomial never gets an NTT and never gets a Merkle tree. The prover's only added work is computing two trace evaluations at z and forming the DEEP quotient. The verifier's only added work is computing $C(z)$ from $P(z)$ and $P(\omega z)$ via the constraint formula, a constant-time operation. (For traces with multiple columns, all DEEP quotients combine into a single polynomial via batched FRI, the optimization covered in the next subsection.)

The first benefit is direct: the prover saves the entire composition-polynomial Merkle tree, ρT leaves of hashing, plus the corresponding NTT to evaluate the composition polynomial on D. For typical parameters this is a 30-50% reduction in commitment work.

The second benefit is subtler. Standard FRI has a per-query soundness gap: a cheating prover who deviates from low-degree only on a small subset of D might still pass FRI's spot-checks. By demanding answers at a point z chosen *outside* D, DEEP-ALI closes this gap. A cheater can fudge values inside D to look low-degree but cannot anticipate where z will land. The DEEP-FRI paper (Ben-Sasson et al., 2019) proves this raises per-query soundness from a constant below $1/8$ to arbitrarily close to 1.

The underlying heuristic generalizes: any polynomial that is algebraically determined by already-committed polynomials does not need to be committed. Evaluating it at a single random point suffices, since the verifier can reconstruct the value from the committed components. The composition polynomial is the obvious case, but the same principle reappears in sum-check-based systems (Chapter 21) under the name *virtual polynomials*. DEEP-ALI is the STARK-side instance of this idea.

In practice, DEEP-ALI is a strict improvement: it removes a Merkle tree, removes an NTT, and tightens soundness. There is no tradeoff against it. It is universal in production STARK provers.

Grinding

Each FRI query buys $\log_2 \rho$ bits of security (under the conjectured analysis) but adds proof bytes: a query requires opening Merkle paths across every committed layer, costing tens of KB per query at typical parameters. The query count therefore sets the proof size. At $\rho = 2$, achieving 128-bit security requires 128 queries, which produces a proof of several megabytes. The question grinding answers: can the prover *trade computation for proof bytes*, paying CPU time at proving to reduce the number of queries needed?

The mechanism is a hash puzzle. After the FRI commitment phase ends, the verifier's query positions are determined by hashing the transcript. The prover is required to find a 64-bit nonce such that hash-

ing (transcript ‖ nonce) yields a digest with g leading zero bits. Such a nonce exists with probability 2^{-g} per attempt, so finding one costs $\approx 2^g$ hash evaluations on average. The puzzle binds *every* committed value: a cheating prover who alters any Merkle root changes the hash input and must restart the search from scratch. Inverting a g-bit hash prefix costs $\approx 2^g$ work, so grinding contributes exactly g bits of security to the total budget.

To make the trade concrete, consider a target of 128-bit security at $\rho = 2$ and trace width $w = 40$ over BabyBear with $T = 2^{20}$ rows.

Without grinding: 128 queries are needed (1 bit per query). Each query opens Merkle paths of depth $\log_2(\rho T) = 21$ for each of the 40 columns at 32 bytes per hash, costing $\approx 40 \times 21 \times 32 = 26$ KB of authentication paths. Total proof contribution from queries: 128×26 KB ≈ 3.3 MB.

With $g = 20$ bits of grinding: only $128 - 20 = 108$ queries are needed. Proof size from queries drops to 108×26 KB ≈ 2.8 MB, a savings of roughly 520 KB. The grinding cost is $2^{20} \approx 10^6$ hash evaluations, which a modern CPU completes in under a millisecond. Trading sub-millisecond compute for half a megabyte of proof is an extraordinarily favorable trade.

The heuristic that emerges: grinding is essentially free up to the point where 2^g hash evaluations approach the prover's other costs. For modern provers running in seconds, g between 16 and 32 fits comfortably under "free" and shaves substantial proof bytes. Beyond $g = 32$, grinding starts taking measurable wall-clock time (4 billion hashes), and the marginal proof savings shrink. Production systems converge on this range: ethSTARK specifies 32 bits of grinding, RISC Zero uses 16, and typical BabyBear configurations land between 15 and 24.

Batched FRI

A STARK prover typically needs to prove low-degree-ness of *many* polynomials over the same domain: each of the w trace columns (or, with DEEP-ALI, each DEEP quotient). The naive approach runs an independent FRI instance for each one, which means independent folding rounds and independent Merkle trees per round. For $w = 50$ trace columns and $\log_2(\rho T) = 22$ folding rounds, this is $50 \times 22 = 1100$ Merkle trees built during FRI alone, dominating commitment costs. Batched FRI replaces all of these with a single FRI instance, paying for one set of folding rounds total.

The mechanism is random linear combination. The verifier provides challenges $\gamma_1, \ldots, \gamma_w$ via Fiat-Shamir, and the prover forms:

$$F(X) = \sum_{j=1}^{w} \gamma_j \cdot P_j(X)$$

A single FRI instance then proves F has degree less than T. The soundness argument is the same one that makes random linear combinations work throughout this book: if any P_j violates the degree bound, the combination F inherits that violation unless the verifier-chosen γ_j accidentally cancel it.

This is exactly the situation governed by the proximity gap conjecture from the blowup-factor section above. Under the conjectured analysis, the cancellation probability is negligible (Schwartz-Zippel bounds the linear case at $T/|\mathbb{F}|$ over a 124-bit extension field). Under the proven analysis, the bound is weaker by a constant factor, costing a few extra queries to compensate.

To make the savings concrete, take the same example as before: $w = 50$ trace columns (or DEEP quotients), trace length $T = 2^{20}$, blowup $\rho = 4$. The first FRI fold operates on a polynomial over the 2^{22}-element LDE domain, with subsequent folds halving each time, giving $\log_2(2^{22}) = 22$ folding rounds in total.

Without batching, each round builds 50 separate Merkle trees (one per polynomial). The first round alone hashes $50 \times 2^{22} = 2 \times 10^8$ leaves. Across 22 rounds the geometric series doubles this, totaling $\approx 4 \times 10^8$ hashes for FRI commitments.

With batching, each round builds one Merkle tree. The first round hashes $2^{22} \approx 4 \times 10^6$ leaves; the geometric series across rounds totals $\approx 8 \times 10^6$ hashes. That is a 50× reduction in FRI Merkle work, exactly the trace width.

The cost of batching is small: one extension multiplication per polynomial per LDE point during the random linear combination step, a single $O(w\rho T)$ pass. For typical parameters this is a few percent of the total prover time, far less than the FRI hashing it eliminates.

Combined with DEEP-ALI, the batched polynomial incorporates both the DEEP quotients and the trace polynomials in a single linear combination, so one FRI instance simultaneously handles degree verification, out-of-domain evaluation consistency, and composition polynomial correctness.

The heuristic: any time a prover needs to prove low-degree-ness of multiple polynomials over the same domain, batching wins. The savings scale linearly with the number of polynomials being batched, and the soundness loss is negligible. No production STARK system runs FRI without it.

Circle FRI

The small-field story has a gap. BabyBear ($p = 2^{31} - 2^{27} + 1$) supports NTTs natively because $p - 1$ has a large power-of-two factor (2^{27}). But Mersenne31 ($p = 2^{31} - 1$) has the cheapest arithmetic of any 31-bit prime, since reduction is a single addition plus a conditional subtract. Its multiplicative group order $p - 1 = 2(2^{30} - 1)$ has only one factor of 2, far too few for a power-of-two NTT domain. The cheapest field cannot use the standard algorithm.

Circle STARKs (Chapter 15) resolve this by replacing the multiplicative group with the **circle group** $\{(x, y) \in \mathbb{F}_p^2 : x^2 + y^2 = 1\}$, which has order $p + 1 = 2^{31}$, a perfect power of two. Circle FRI adapts the FRI folding protocol to this group structure.

Polynomials on the circle are not standard univariates but elements of a Riemann-Roch space, consisting of polynomials modulo the relation $x^2 + y^2 = 1$. This means y^2 terms reduce to $1 - x^2$, so every polynomial expression on the circle involves y at most linearly.

The first round of Circle FRI exploits the y-symmetry. For opposite points (x, y) and $(x, -y)$ on the circle, the folding decomposes a function F into even and odd parts:

$$f_0(x) = \frac{F(x,y) + F(x,-y)}{2}, \quad f_1(x) = \frac{F(x,y) - F(x,-y)}{2y}$$

Given a random challenge α, the folded function is $f_0 + \alpha \cdot f_1$, now depending only on x. This halves the domain.

Subsequent rounds use the **doubling map** $x \mapsto 2x^2 - 1$, which arises from the angle-doubling formula $\cos(2\theta) = 2\cos^2(\theta) - 1$. Opposite x-values (points at angles θ and $\pi - \theta$) map to the same doubled coordinate. The folding at each subsequent round is:

$$f_0(2x^2 - 1) = \frac{F(x) + F(-x)}{2}, \quad f_1(2x^2 - 1) = \frac{F(x) - F(-x)}{2x}$$

Each round halves the domain, just as standard FRI halves via the squaring map $x \mapsto x^2$. The total work across all rounds forms the same geometric series, giving $O(N)$ total field operations for the folding itself.

The takeaway is that Circle FRI delivers the algorithmic capabilities of standard FRI (low-degree testing via halving folds, $O(N \log N)$ NTTs, $O(N)$ folding work) over a field where standard FRI cannot run. The win is the ability to use Mersenne31 arithmetic, whose modular reduction is roughly 1.4× faster per multiplication than BabyBear. The Circle STARKs paper (Haböck, Levit, Papini, 2024) measures this speedup directly on real workloads. Combined with the 4× advantage of 31-bit over 64-bit arithmetic that motivated small fields in the first place, Circle STARKs over Mersenne31 represent the current frontier of STARK proving speed and are deployed in Stwo (StarkWare), Plonky3 (Polygon), and Airbender (ZKsync).

The design rule: if your prover's bottleneck is field arithmetic and you can build the rest of your stack (extension fields, hash function, recursion) over Mersenne31, Circle FRI is worth the additional machinery. If the bottleneck is elsewhere (constraint evaluation, Merkle hashing with an expensive hash function), the BabyBear/standard-FRI combination is simpler and gives most of the same benefit.

A worked example: proving Poseidon2 hashes

The chapter has introduced optimizations one at a time. To see how they compound, consider a single concrete task and trace what each optimization saves. The task: prove 1024 invocations of the Poseidon2 hash function. We will work through two configurations in parallel, a naive baseline and an optimized prover, and compare the cost at each stage.

The computation. Poseidon2 with state width 16 uses 8 full rounds and 14 partial rounds per permutation, for 22 rounds total. Each round applies a non-linear function called an **S-box** (substitution box, the standard term for the non-linear component of a hash or block cipher) to one or more state elements; in Poseidon2 the S-box is simply $x \mapsto x^5$. Full rounds apply it to all 16 state elements; partial rounds apply it to only one. After each S-box, an MDS matrix linearly mixes the state. The total trace length for 1024 hashes is $T = 1024 \times 22 = 22{,}528$ rows, rounded up to $T = 2^{15}$ for NTT compatibility.

Configuration A: naive baseline. Use BN254 (a 254-bit field), encode each S-box directly as a degree-5 constraint, commit the composition polynomial separately, run a fresh FRI instance per polynomial.

- Field arithmetic: each multiplication is 30-50 cycles (multi-limb 254-bit operations).
- Trace columns: $w = 16$ (just the state elements).
- Constraint degree: $d = 5$ (degree of $x \mapsto x^5$).
- LDE blowup: $\rho = 8$ (must satisfy $\rho \geq d$).
- LDE domain size: $\rho T = 2^{18}$.
- Per-column NTT: $\approx 2^{18} \times 18/2 \approx 2.4 \times 10^6$ multiplications. Across 16 trace columns plus a composition polynomial of degree-bound $\approx d \cdot T$, total NTT work is roughly 2^{19} multiplications times another factor of $\log$, giving $\approx 5 \times 10^7$ multiplications. At 30 cycles each on a 3 GHz core, that is ≈ 0.5 seconds for NTT alone.
- Commitment: two large Merkle trees (trace + composition), $\approx 2 \times 2^{18}$ hash invocations.
- FRI: 16 separate FRI instances (one per trace column) plus one for the composition polynomial.

Configuration B: optimized prover. Switch to Mersenne31 with Circle STARKs, decompose S-boxes into auxiliary columns to reduce degree, apply DEEP-ALI to skip the composition commitment, batch all FRI instances into one, add 20 bits of grinding.

- Field arithmetic: each multiplication is roughly 3 cycles (single 32-bit multiply + Mersenne reduction), with 8-wide AVX2 SIMD.
- Trace columns: $w = 24$ (16 state + ≈ 8 auxiliary $q = x^2$ columns averaged across full and partial rounds).
- Constraint degree: $d = 3$ (using $x^5 = x \cdot (x^2)^2$ via the auxiliary column, splitting into degree-2 and degree-3 checks).

- LDE blowup: $\rho = 2$ (the smallest practical value, compensated by grinding).
- LDE domain size: $\rho T = 2^{16}$.
- Per-column NTT: $\frac{1}{2} \cdot 2^{16} \cdot 16 \approx 5 \times 10^5$ M31 multiplications.
- Total NTT across 24 columns: $\approx 1.2 \times 10^7$ multiplications.
- With SIMD throughput of $\approx 8 \times 10^9$ multiplications per second on a single core, NTT wall time: ≈ 1.5 ms.
- Commitment: one Merkle tree on the trace, 2^{16} leaves of 24 field elements each, $\approx 10^5$ Poseidon2 hash invocations. No composition polynomial commitment (DEEP-ALI).
- FRI: one batched instance (all 24 DEEP quotients combined).
- Queries: 108 (128 bits target − 20 bits grinding) at $\rho = 2$.

Where the savings come from. Comparing the two configurations stage by stage:

Stage	Naive (Config A)	Optimized (Config B)	Source of savings
Field multiplication cost	30-50 cycles	≈ 3 cycles, 8-wide SIMD	Mersenne31 + small fields
Constraint degree	5	3	Auxiliary columns
Blowup factor	8	2	Allowed by lower d
LDE domain size	2^{18}	2^{16}	Lower ρ
NTT work	$\approx 5 \times 10^7$	$\approx 1.2 \times 10^7$	Smaller domain, more columns offset by lower $\log N$
NTT wall time	≈ 500 ms	≈ 1.5 ms	All of the above plus SIMD
Merkle trees committed	2 (trace + composition)	1 (trace only)	DEEP-ALI

Stage	Naive (Config A)	Optimized (Config B)	Source of savings
FRI instances	≈ 17	1	Batched FRI
Queries	128 (no grinding)	108	Grinding shifts 20 bits

The optimized prover finishes in a few milliseconds where the naive baseline would take hundreds of milliseconds, a roughly $300\times$ improvement. No single optimization in the table is worth more than a single-digit factor on its own; the orders-of-magnitude gap comes from the stack. This matches the trajectory described in the chapter introduction, where production STARK provers improved by 50× from algorithmic and field-choice optimizations alone, with commodity hardware delivering the rest.

Comparison with sum-check optimization

Chapters 19 and 20 solve the same problem from opposite directions. The techniques differ because the cost structures differ.

Sum-check provers run in $O(N)$ field operations. The bottleneck lies in the sum-check rounds themselves plus polynomial commitment openings. Optimization focuses on reducing cost per operation through small-value tricks, delayed binding, Karatsuba for high-degree products. Polynomial commitments (MSM for curve-based schemes) are a separate cost center, often the dominant one, addressed in Chapter 21.

STARK provers pay an $O(N \log N)$ NTT cost that sum-check avoids entirely, since multilinear polynomials need no Fourier transform. But their Merkle commitments are vastly cheaper than the MSMs that curve-based sum-check systems require. A Merkle commitment costs one hash per element; a KZG commitment costs one elliptic curve scalar multiplication per element, roughly 3000× more expensive per operation.

Both traditions exploit the principle of doing most work in cheap arithmetic. Sum-check provers work over 256-bit fields but exploit small witness values for fast ss/sb multiplications in early rounds (Chapter 19). STARK provers work over 31-bit base fields, lifting to 124-bit extension fields only for verifier challenges. The mechanism differs (small values within a large field vs. a genuinely small field with extensions) but the economics are identical: the prover's heaviest rounds coincide with the regime where the cheapest arithmetic applies.

A second shared principle is the avoidance of unnecessary commitments. DEEP-ALI (this chapter) eliminates the composition polynomial commitment by exploiting that the composition polynomial is algebraically determined by the trace. Sum-check systems take the same idea further with **virtual polynomials** (Chapter 21): any polynomial computable from already-committed polynomials can be evaluated at a verifier-chosen point without ever being committed. The principle generalizes: derived data should not be paid for twice. Different systems implement this differently (DEEP quotients, virtual polynomials, quotient-free PCS designs), but the underlying observation is the same.

Neither tradition dominates universally. STARKs pay $\log N$ overhead per element but get cheap commitments. Sum-check achieves linear time but faces expensive commitments. At small scales with structured computations (hashing), STARKs excel because their Merkle-based commitments scale linearly while curve-based MSMs grow superlinearly. At large scales with sparse constraints, sum-check's $O(T)$ sparse proving (Chapter 19) pulls ahead because the NTT processes the entire trace regardless of sparsity.

The convergence of the two traditions is already underway. Binius (Chapter 26) uses sum-check over binary tower fields with FRI-based commitments, combining sum-check's linear-time proving with hash-based post-quantum commitments. Systems like Plonky3 support both quotienting-based and sum-check-based frontends over the same small-field backend. Chapter 22 develops this comparison fully.

Key takeaways

1. **The STARK prover bottleneck shifts with scale.** Constraint evaluation dominates for small traces; the NTT dominates for medium traces ($T = 2^{18}$ to 2^{24}); Merkle hashing dominates at the largest scales. The crossover ratio is c_h, the cost of one hash relative to one field multiplication; the larger c_h, the earlier hashing takes over. Optimization must address whichever stage currently dominates.
2. **AIR design is the highest-leverage optimization.** Two encodings of the same computation can differ in prover time by an order of magnitude because they feed different w and d into the pipeline. Width w is an additive cost on a few stages; degree d is a multiplicative cost on the entire pipeline downstream of the composition polynomial. The design rule: add columns to reduce degree until d stops crossing power-of-two boundaries downward.
3. **Interaction columns are a requirement, not an optimization.** A pure AIR sees only adjacent rows and cannot express memory consistency, lookup arguments, or any constraint relating distant rows. LogUp uses verifier randomness to convert global multiset equalities into local

accumulator constraints, at the cost of a few extra columns per bus. Without this mechanism, AIR-based zkVMs would be impossible.

4. **Small fields exploit the hardware register hierarchy.** A 31-bit prime is the largest whose product fits in a single 64-bit register, eliminating multi-limb arithmetic. Combined with SIMD packing (16 elements per 512-bit vector), small fields deliver 10× per-element speedup over 256-bit fields. Extension fields supply the missing soundness for verifier challenges, at a cost of $\approx 9/w$ overhead relative to base field NTT work.

5. **The NTT optimizes by fitting into cache, not by algorithmic improvement.** The four-step decomposition restructures a size-N NTT into $\sqrt{N}$ small NTTs that fit in L1 cache, eliminating the cache misses that dominate naive implementations. The asymptotic stays $O(N \log N)$ but the constant factor improves dramatically because the CPU stops waiting for RAM.

6. **The blowup factor ρ trades prover speed against proof size.** Larger ρ means each FRI query catches a cheater with higher probability, so fewer queries are needed and proofs are smaller, but every NTT and Merkle tree grows proportionally. Most production systems use $\rho = 2$ with grinding to compensate.

7. **Many "optimizations" are really commitment avoidance.** DEEP-ALI eliminates the composition polynomial commitment by reconstructing it from an out-of-domain trace evaluation; the same principle reappears as virtual polynomials in sum-check systems (Chapter 21). The general rule: any polynomial algebraically determined by already-committed polynomials does not need its own commitment. Evaluating it at one verifier-chosen point suffices.

8. **Random linear combination is the universal batching technique.** Batched FRI replaces w separate FRI instances with one by combining all polynomials into a random linear combination, reducing FRI hashing by a factor of w. The soundness rests on the proximity gap conjecture, whose late-2025 partial refutation has driven the ecosystem toward larger blowup factors and provable-security parameter regimes.

9. **Grinding is essentially free up to $g \approx 32$.** Replacing FRI queries with proof-of-work shrinks proof size at sub-millisecond compute cost. Beyond 32 bits of grinding the wall-clock cost becomes noticeable, so production systems converge on g between 16 and 32.

10. **Circle STARKs unlock Mersenne31.** The circle group over M_{31} has order 2^{31}, enabling NTT-like algorithms (via the doubling map $x \mapsto 2x^2 - 1$) over the field with the cheapest arithmetic of any 31-bit prime. Production provers using this stack achieve over 500,000 Poseidon2 hashes per second on commodity hardware.

11. **No single optimization is worth more than a single-digit factor.** The 50-300× speedup achieved by modern STARK provers compared to early ones comes from compounding many small wins: small fields, extension lifting, AIR width tuning, four-step NTT, DEEP-ALI, batched

FRI, grinding, Circle STARKs. Each contributes individually; none replaces the others.

12. **STARKs and sum-check systems converge on the same principles via different mechanisms.** Both push the bulk of work into cheap arithmetic (small fields with extensions vs. small values within large fields). Both avoid unnecessary commitments (DEEP-ALI vs. virtual polynomials). The pipelines differ ($O(N \log N)$ NTT for STARKs vs. $O(N)$ sum-check; cheap Merkle commitments for STARKs vs. expensive MSMs for curve-based sum-check), but the design philosophies rhyme. Chapter 22 develops this comparison.

Chapter 21: Minimizing Commitment Costs

> *This chapter closes Part VI (Prover Optimization, Chapters 19-21), which is optional on a first read. The rest of the book does not depend on it. The material here is essential for anyone designing or implementing a fast prover.*
>
> *This chapter lives at the frontier. The techniques here, some from papers published in 2024 and 2025, represent the current edge of what's known about fast proving. We assume comfort with polynomial commitments (Chapter 9), sum-check (Chapter 3), and the memory checking ideas from Chapter 14. First-time readers may find themselves reaching for earlier chapters often; that's expected. The reward for persisting is a view of how the fastest SNARKs actually work.*

Profile any modern SNARK prover and the same pattern appears. The proving algorithm touches each constraint once. The information-theoretic protocol is near-optimal. Yet wall-clock time is dominated by something else entirely: polynomial commitments.

For elliptic curve-based systems, the bottleneck is multi-scalar multiplication (MSM): computing $\sum_i s_i \cdot G_i$ where each s_i is a scalar and each G_i is a curve point. A single curve exponentiation costs roughly 3,000 field multiplications. An MSM over N points costs about $N / \log N$ exponentiations. For a polynomial of degree 10^6, commitment alone requires $\approx 3 \times 10^8$ field operations, while the proving algorithm itself, after the linear-time sum-check techniques of Chapter 19, runs in only 10^7. The cryptography dwarfs the algebra. The two surrounding chapters develop the rest of the picture: Chapter 19 establishes why sum-check provers are now fast enough that commitments dominate, and Chapter 20 traces the STARK-side optimization story, where the bottleneck instead concentrates in NTT and hashing because FRI absorbs the commitment cost into the prover pipeline.

This chapter focuses on the elliptic curve setting, where sum-check-based minimization techniques apply most directly.

This observation crystallizes into a design principle: **commit to as little as possible**. Not zero (some commitment is necessary for succinctness) but the absolute minimum required for soundness.

This chapter develops the techniques that make minimization possible. Together with fast sum-check proving, they form the foundation of the fastest modern SNARKs.

The Two-Stage Paradigm

Every modern SNARK decomposes into two phases. First, the prover *commits* to the witness, to intermediate values, and to auxiliary polynomials that will help later proofs. Second, the prover runs an *interactive argument* that demonstrates those committed objects satisfy the required constraints.

Both phases cost time. And here's the trap: **more commitment means more proving**. Every committed object must later be shown well-formed. If you commit to a polynomial, you'll eventually need to prove something about it: its evaluations, its degree, its relationship to other polynomials. Each such proof compounds the cost.

The obvious extremes are both suboptimal. Commit nothing, and proofs cannot be succinct: the verifier must read the entire witness. Commit everything, and you drown in overhead: each intermediate value requires cryptographic operations and well-formedness proofs.

The art lies in the middle: commit to exactly what enables succinct verification. No more.

Untrusted Advice

Sometimes the sweet spot involves *enlarging the witness*: adding extra values that the prover must compute alongside the original ones. The witness is what gets committed, so adding a few helper values just makes the same witness polynomial slightly longer. The trade-off can be favorable: the extra values often let the constraint system avoid hard operations entirely.

Consider division. Proving "I correctly computed a/b" by directly encoding division as a constraint is expensive, since division is not a native operation in polynomial constraint systems. The constraint system speaks the language of multiplication and addition over a finite field, not Euclidean division.

The workaround is to enlarge the witness with the quotient q and remainder R, and then verify the multiplicative identity:

1. The prover adds q and R to the witness vector. They are committed as part of the same polynomial(s) that already hold a and b, with no separate commitment object.
2. The constraint system enforces $a = q \cdot b + R$ and $R < b$.

Every value lives inside the committed witness polynomial; the verifier never sees any of them in the clear. The constraint is checked the same way every other constraint is: as a polynomial identity opened at a random point via the PCS. The win is that this identity uses only multiplication and a

range check, both native, instead of requiring the constraint system to implement division. The prover paid for slightly more witness entries to avoid encoding a hard operation, and the verifier never had to learn what q and R actually are.

This pattern is called **untrusted advice**: the prover volunteers additional witness data that, *if* the constraints check out, accelerates the overall proof. The verifier does not trust the advice blindly; the constraints guarantee it is consistent with the original claim.

The trade-off is specific: we pay for a slightly longer witness polynomial (more entries to commit, so a slightly larger MSM) to save on constraint degree. The constraints that check the enlarged witness can be lower-degree than the constraints that would have encoded the hard operation directly. Since high-degree constraints are expensive to prove via sum-check, the exchange often favors a longer witness with simpler constraints.

The pattern generalizes. Any computation with an efficient verification shortcut benefits:

Square roots. To prove $y = \sqrt{x}$, the prover commits to y and proves $y^2 = x$ and $y \geq 0$. One multiplication plus a range check, rather than implementing the square root algorithm in constraints.

Sorting. To prove a list is sorted, the prover commits to the sorted output and proves: (1) it's a permutation of the input (via permutation argument), and (2) adjacent elements satisfy $a_i \leq a_{i+1}$. Linear comparisons rather than $O(n \log n)$ sorting constraints.

Inverses. To prove $y = x^{-1}$, commit to y and check $x \cdot y = 1$. Field inversion (expensive to express directly) becomes a single multiplication.

Exponentiation. To prove $y = g^x$, the prover commits to y and all intermediate values from the square-and-multiply algorithm: $r_0 = 1, r_1, r_2, \ldots, r_k = y$. Each step satisfies $r_{i+1} = r_i^2$ (if bit $x_i = 0$) or $r_{i+1} = r_i^2 \cdot g$ (if $x_i = 1$). Verifying k quadratic constraints is far cheaper than expressing the full exponentiation logic.

Whenever verifying a result costs less than computing it, the prover should compute and commit while the constraint system only checks. The prover bears the computational burden; the constraint system bears only the verification burden. This division of labor is the essence of succinct proofs, now applied within the proof system itself.

Batch Evaluation Arguments

Suppose the prover has committed to addresses $(y_1, \ldots, y_T)$ and claimed *read results* $(z_1, \ldots, z_T)$, the values the prover claims it received from each lookup. A public function $f : \{0,1\}^\ell \to \mathbb{F}$ is known to all. The prover wants to demonstrate:

$$z_1 = f(y_1), \quad z_2 = f(y_2), \quad \ldots, \quad z_T = f(y_T)$$

One approach: prove each evaluation separately. That's T independent proofs, linear in the number of evaluations. Can we do better?

Think of f as a memory array indexed by ℓ-bit addresses. Each (y_i, z_i) pair is a read operation, "I read value z_i from address y_i," and the prover claims all T reads are consistent with the memory f. (Later in this chapter we will see that this read-only setting is the simpler half of a more general *memory checking* problem, where the table itself can be updated over time.)

One approach uses lookup arguments (Chapter 14), proving that each (y_i, z_i) pair exists in the table $\{(x, f(x)) : x \in \{0,1\}^\ell\}$. But sum-check offers a more direct path that exploits the structure of the problem.

Three Flavors of Batching

Before diving into sum-check, let's map the batching landscape. The term "batching" appears throughout this book, but it means different things in different contexts.

Approach 1: Batching verification equations. The simplest form. Suppose you have T equations to check: $L_1 = R_1, \ldots, L_T = R_T$. Sample a random α and check the single combined equation $\sum_j \alpha^j L_j = \sum_j \alpha^j R_j$. By Schwartz-Zippel, if any original equation fails, the combined equation fails with high probability. This reduces T verification checks to one.

Chapter 2 uses this for Schnorr batch verification. Chapter 13 uses it to combine PLONK's constraint polynomials. Chapter 15 uses it to merge STARK quotients. The pattern is ubiquitous: random linear combination collapses many checks into one.

Approach 2: Batching PCS openings. Polynomial commitment schemes often support proving multiple evaluations cheaper than proving each separately. KZG's batch opening (Chapter 9) proves $f(z_1) = v_1, \ldots, f(z_k) = v_k$ with a single group element, using the quotient $\frac{f(X)-R(X)}{Z(X)}$ where R is the interpolant of the claimed evaluations and Z is the vanishing polynomial of the query points. This quotient exists as a polynomial iff every claimed evaluation is correct, so its commitment doubles as the batch proof. Proof size stays constant regardless of k. This batching is PCS-specific; other schemes have different mechanisms.

Approach 3: Batching via domain-level sum-check. This is what this section develops. Rather than batch the T *claims* directly, we restructure the problem as a sum over the *domain* of f. The key equation:

$$\tilde{z}(r') = \sum_{x \in \{0,1\}^\ell} \widetilde{ra}(x, r') \cdot \tilde{f}(x)$$

This sum nominally has 2^ℓ terms (one per address in the domain), but $\widetilde{ra}$ is sparse: out of $K \cdot T$ possible entries, only T are non-zero, since each access touches exactly one address. Sum-check exploits this sparsity in the *access matrix*, not in f itself (f can be perfectly dense). At the end of the protocol, the verifier needs a single evaluation $\tilde{f}(r)$ at a random point: one PCS opening, not T.

Comparing the three approaches The three approaches batch at different levels, and that is what determines what each one saves. Approaches 1 and 2 operate at the *claim level*: the prover must still open f at all T points $y_1, \ldots, y_T$. Approach 1 saves verifier work (one check instead of T) but does not reduce openings; Approach 2 compresses the proof but still requires the prover to compute all T evaluations internally. Approach 3 batches at the *domain level*: the T point evaluations collapse into a single random evaluation, and the prover opens $\tilde{f}$ at exactly one point.

Each approach therefore answers a different question.

Approach 1 (batch verification equations) answers "I have many unrelated checks; can the verifier handle them in one shot?" Use it whenever you have multiple equations to verify, even outside the PCS setting. The combiner is just transcript-level randomness, costing nothing beyond sampling one field element. The prover does the same work either way; only verifier work shrinks. This is what PLONK uses to combine constraint polynomials and what STARK quotient batching uses.

Approach 2 (PCS batch opening) answers "I have one committed polynomial; how do I send many opening proofs in one go?" Use it when f is already committed (typically via KZG) and you need to prove evaluations at multiple points. The win is purely in proof size: one group element instead of T. The prover still computes all T evaluations internally and does the corresponding MSM work; nothing about f's structure or the access pattern matters.

Approach 3 (sum-check over the domain) answers "I have many evaluations of the same polynomial with structured access; can the prover do less work overall?" Use it when (a) you are proving many evaluations $f(y_j)$ of the same f, and (b) the access pattern has structure the sum-check can exploit, in particular the one-hot or tensor-decomposable structure of the access matrix ra. This is structure in *how the polynomial is queried*, not structure in the polynomial itself. The decisive parameters are T (number of accesses) and $K = 2^\ell$ (domain size): when $T \ll K$, exploiting the access sparsity is what makes T accesses to a K-sized table feasible. Without that structure, Approach 3 has nothing to exploit and Approach 2 is simpler.

There is a deeper connection across all three. Evaluating an MLE at a random point r' *is* a random linear combination, weighted by the Lagrange basis $\widetilde{\mathsf{eq}}(r', \cdot)$ rather than powers of α. The sum-check formulation in Approach 3 is random linear combination in MLE clothing, but operating at the domain level unlocks optimizations that claim-level batching (Approaches 1 and 2) cannot reach.

The Sum-Check Approach

Now we develop Approach 3 in detail. Let $\tilde{f}$ be the multilinear extension of f. The access matrix $ra(x, j)$ from the previous section is the Boolean matrix with $ra(x, j) = 1$ iff $y_j = x$, so each column j is one-hot at the row corresponding to address y_j.

Example. Suppose f is defined on 2-bit addresses $\{00, 01, 10, 11\}$, and we have $T = 3$ accesses to addresses $y_1 = 01$, $y_2 = 11$, $y_3 = 01$. The access matrix is:

$$ra = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \quad \text{rows: } x \in \{00, 01, 10, 11\}, \quad \text{columns: } j \in \{1, 2, 3\}$$

Each column j encodes "which address did access j hit?" as a one-hot vector: column j equals the basis vector e_{y_j}. Here column 1 is e_{01} (since $y_1 = 01$), column 2 is e_{11} (since $y_2 = 11$), and column 3 is e_{01} again (since $y_3 = 01$).

For a single evaluation, we can write:

$$z_j = \sum_{x \in \{0,1\}^\ell} ra(x, j) \cdot f(x)$$

This looks like overkill. The one-hot structure of $ra(\cdot, j)$ zeroes out every term except the one at address y_j, so the sum trivially collapses to $f(y_j) = z_j$. Why bother?

The heuristic that turns this into a single check is the multilinear extension trick used throughout the book: lift a vector of values defined on the Boolean hypercube into a polynomial on the full field, then evaluate that polynomial at one random point off the hypercube. By Schwartz-Zippel, that one evaluation catches any error in the original vector with overwhelming probability.

Define the "error" at index j as the gap between the claimed output and what the lookup should return:

$$e_j = z_j - \sum_{x \in \{0,1\}^\ell} ra(x, j) \cdot f(x)$$

There are T such errors, one per access. All evaluations are correct iff $e_j = 0$ for every j. Checking T separate equalities defeats the purpose of batching, so we apply the trick. The vector $(e_1, \ldots, e_T)$ is defined on the hypercube $\{0, 1\}^{\log T}$. Its multilinear extension $\tilde{e}$ is a polynomial on $\mathbb{F}^{\log T}$, and $\tilde{e}$ is the

zero polynomial iff *every* $e_j = 0$. The verifier picks a random $r' \in \mathbb{F}^{\log T}$ and asks: is $\tilde{e}(r') = 0$? If all e_j vanish, the answer is yes for any r'; if any e_j is non-zero, Schwartz-Zippel says the answer is no with overwhelming probability. One evaluation, T checks collapsed.

Substituting the definition of e_j and using the linearity of the MLE construction, the check $\tilde{e}(r') = 0$ becomes:

$$\tilde{z}(r') = \sum_{x \in \{0,1\}^\ell} \widetilde{ra}(x, r') \cdot \tilde{f}(x)$$

If this single identity holds at the random r', all T original evaluations are correct with high probability. The T separate access claims have collapsed into one identity over the *entire domain* $\{0,1\}^\ell$, which sum-check is built to prove.

Sum-check proves this identity. The prover commits to $\widetilde{ra}$ and $\tilde{z}$, then runs sum-check to verify consistency with the public $\tilde{f}$.

The Sparsity Advantage

The sum nominally ranges over all 2^ℓ addresses, potentially enormous (imagine $\ell = 128$ for CPU word operations). The reason it stays tractable is the structure of the access matrix. A vector or matrix is **one-hot** if every column contains exactly one non-zero entry, and that entry equals 1. The access matrix ra is one-hot by construction: each access j touches exactly one address y_j, so column j has a 1 at row y_j and zeros everywhere else.

The consequence is dramatic. The matrix ra has dimensions $K \times T$ with $K = 2^\ell$, so naively it has $K \cdot T$ entries, but only T of them are non-zero. Any sum that appears to range over K positions actually touches only the T non-zero terms. This is why batch evaluation costs $O(T)$ rather than $O(KT)$: the one-hot structure makes the exponentially large table effectively linear-sized. When $K = 2^{128}$ (as in Jolt's instruction lookups), this is the difference between tractable and impossible.

One-hotness handles the access side (only T non-zero terms in $\widetilde{ra}$) but the sum still nominally folds the dense polynomial $\tilde{f}$ over the full 2^ℓ-element domain. Naive sum-check over this dense factor still costs $O(2^\ell)$. The prefix-suffix algorithm from Chapter 19 closes the remaining gap: by splitting the variables into halves and running two chained sum-checks, the dense work shrinks from $O(2^\ell)$ to $O(2^{\ell/c})$ for any constant c. Combined with the one-hot access, the prover runs in $O(T + 2^{\ell/c})$ total. Compared to proving each evaluation separately (which costs $\Omega(T)$ just to state the claims), the batch approach matches the lower bound while providing cryptographic guarantees.

Virtual Polynomials

Start with a toy case. Suppose the prover has committed to multilinear polynomials $\tilde{a}$ and $\tilde{b}$, and the protocol later refers to $\tilde{c}(x) = \tilde{a}(x) \cdot \tilde{b}(x)$. Should the prover separately commit to $\tilde{c}$?

No, because $\tilde{c}$ contains no information beyond what is already in $\tilde{a}$ and $\tilde{b}$. Whenever the verifier needs $\tilde{c}(r)$ at a random point r, the protocol can ask for $\tilde{a}(r)$ and $\tilde{b}(r)$ instead, then compute $\tilde{c}(r) = \tilde{a}(r) \cdot \tilde{b}(r)$ locally. The polynomial $\tilde{c}$ is **virtual**: it exists implicitly through the formula $\tilde{c} = \tilde{a} \cdot \tilde{b}$, never committed, never stored. The prover saves one MSM; the verifier loses nothing.

The general principle behind virtualization is that any polynomial algebraically determined by already-committed polynomials does not need its own commitment. Whenever the verifier needs an evaluation of the virtual polynomial at some point r, the protocol reduces that demand to evaluations of the source polynomials at r, and the verifier reconstructs the result from the formula. The savings cascade: if a virtual polynomial's sources are themselves virtual, the same trick applies recursively, and only the *root* polynomials in the dependency graph ever get committed.

This principle is what makes the access matrix tractable. In our batch evaluation, ra has $K = 2^\ell$ rows (one per possible address) and T columns (one per access). For a zkVM with 32-bit addresses, $K = 2^{32}$, so the matrix has billions of entries. Committing to it directly is impossible. The escape is to *not* commit ra as a single object: instead, decompose it into smaller pieces that can be committed and treat the full ra as virtual. The next subsection develops this decomposition.

Tensor Decomposition

The access matrix is the natural target for virtualization, but virtualization needs source polynomials to factor through. The trick is that addresses themselves are bit strings, and matching an ℓ-bit address means matching every bit. We can therefore factor the address-match into separate per-chunk matches, each over a much smaller space.

Concretely, an address $k \in \{0,1\}^\ell$ splits into d chunks of ℓ/d bits each:

$$k = (k_1, \ldots, k_d) \quad \text{where each } k_i \in \{0,1\}^{\ell/d}$$

For each chunk i, define a smaller access matrix ra_i where $ra_i(k_i, j) = 1$ iff the i-th chunk of access y_j equals k_i. Each ra_i has dimensions $K^{1/d} \times T$, exponentially smaller than the original $K \times T$.

The full access happens when *every* chunk matches, which is exactly the product:

$$ra(k, j) = \prod_{i=1}^{d} ra_i(k_i, j)$$

The original ra never gets committed. The prover commits only to the d small matrices $ra_1, \ldots, ra_d$, and the full ra exists virtually through this product formula.

Example. Return to our 2-bit addresses with accesses $y_1 = 01$, $y_2 = 11$, $y_3 = 01$. Split each address into $d = 2$ chunks of 1 bit each: $y_1 = (0, 1)$, $y_2 = (1, 1)$, $y_3 = (0, 1)$.

The chunk matrices are (columns: $j \in \{1, 2, 3\}$):

$$ra_1 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad \text{rows: first bit } k_1 \in \{0, 1\}$$

$$ra_2 = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix} \quad \text{rows: second bit } k_2 \in \{0, 1\}$$

In ra_1: row 0 has 1s in columns 1 and 3 because accesses $y_1 = 01$ and $y_3 = 01$ have first bit 0. Row 1 has a 1 in column 2 because $y_2 = 11$ has first bit 1.

In ra_2: row 1 has 1s in all columns because all three accesses $(01, 11, 01)$ have second bit 1.

To recover $ra(01, j = 1)$: check $ra_1(0, 1) \cdot ra_2(1, 1) = 1 \cdot 1 = 1$. Indeed, access 1 hit address 01. For $ra(10, j = 1)$: $ra_1(1, 1) \cdot ra_2(0, 1) = 0 \cdot 0 = 0$. Access 1 did not hit address 10.

Instead of one 4×3 matrix (12 entries), we store two 2×3 matrices (12 entries total, same here, but the savings grow with ℓ).

The commitment savings are dramatic. Instead of a $K \times T$ matrix, the prover commits to d matrices of size $K^{1/d} \times T$ each. For $K = 2^{128}$ and $d = 4$: from 2^{128} to 4×2^{32}.

The exponential has become polynomial.

Virtualizing Everything

Once you see virtualization, you see it everywhere. The product example above is the smallest case; in real systems the same principle applies to entire computation traces. A zkVM executing a million instructions touches several polynomials per instruction: opcode, operands, intermediate values, flags. Naive commitment requires millions of polynomials, each with its own MSM. Virtualization reduces this to perhaps a dozen root polynomials, with everything else derived. The difference is a 30-second proof versus a 3-second proof.

The read values need not exist. Recall the batch evaluation setup: $z = (z_1, \ldots, z_T)$ is the vector of *read results*, with z_j being the value returned when the prover read address y_j in step j. These feel like

primary data; in a zkVM they are exactly the values an instruction sees coming out of memory, and the rest of the computation depends on them. Surely they need to be committed?

They do not. The read results are completely determined by the access pattern ra (which addresses were read) and the table f (what each address contains). Concretely:

$$\tilde{z}(r') = \sum_{x \in \{0,1\}^\ell} \widetilde{ra}(x, r') \cdot \tilde{f}(x)$$

The right side *defines* $\tilde{z}$ implicitly from ra and f. The prover never commits to z. When the verifier needs $\tilde{z}(r')$, sum-check reduces this evaluation to evaluations of $\widetilde{ra}$ and $\tilde{f}$, both of which are already committed (the access matrix) or public (the table). The pattern is the same as $\tilde{c} = \tilde{a} \cdot \tilde{b}$ from earlier, just with a sum instead of a product as the defining formula.

GKR as virtualization. The GKR protocol (Chapter 7) builds an entire verification strategy from this idea. A layered arithmetic circuit computes layer by layer from input to output. The naive approach commits to every layer's values. GKR commits to almost nothing:

Let $\tilde{V}_k$ denote the multilinear extension of gate values at layer k. The layer reduction identity:

$$\tilde{V}_k(r) = \sum_{i,j \in \{0,1\}^s} \widetilde{\mathsf{mult}}_k(r, i, j) \cdot \tilde{V}_{k-1}(i) \cdot \tilde{V}_{k-1}(j) + \ldots$$

Each layer's values are virtual: defined via sum-check in terms of the previous layer. Iterate from output to input: only $\tilde{V}_0$ (the input layer) is ever committed. A circuit with 100 layers has 99 virtual layers that exist only as claims passed through sum-check reductions.

More examples. The pattern appears throughout modern SNARKs.

- *Constraint polynomials.* In Spartan (Chapter 19), the polynomial $\tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x)$ is never committed. Sum-check verifies it equals zero on the hypercube by evaluating at random points.
- *Grand products.* Permutation arguments express $Z(X)$ as a running product. Each $Z(i)$ is determined by $Z(i-1)$ and the current term. One starting value plus a recurrence defines everything.
- *Folding.* In Nova (Chapter 23), the accumulated instance is virtual. Each fold updates a claim about what could be verified (not data sitting in memory).
- *Write values from read values.* In read-write memory checking, the prover commits to read addresses, write addresses, and increments Δ. What about write values? They need not be committed: $\mathsf{wv}(j) = \mathsf{rv}(j) + \Delta(j)$. The write value at cycle j is the previous value at that address plus the change. Three committed objects define four.

The design principle that emerges from these examples is to ask not "what do I need to store?" but "what can I define implicitly?" Every polynomial expressible as a function of others is a candidate for virtualization. Every value recoverable from a sum-check reduction need never be committed. The fastest provers are the ones that commit least, because computation is cheap but cryptography is expensive.

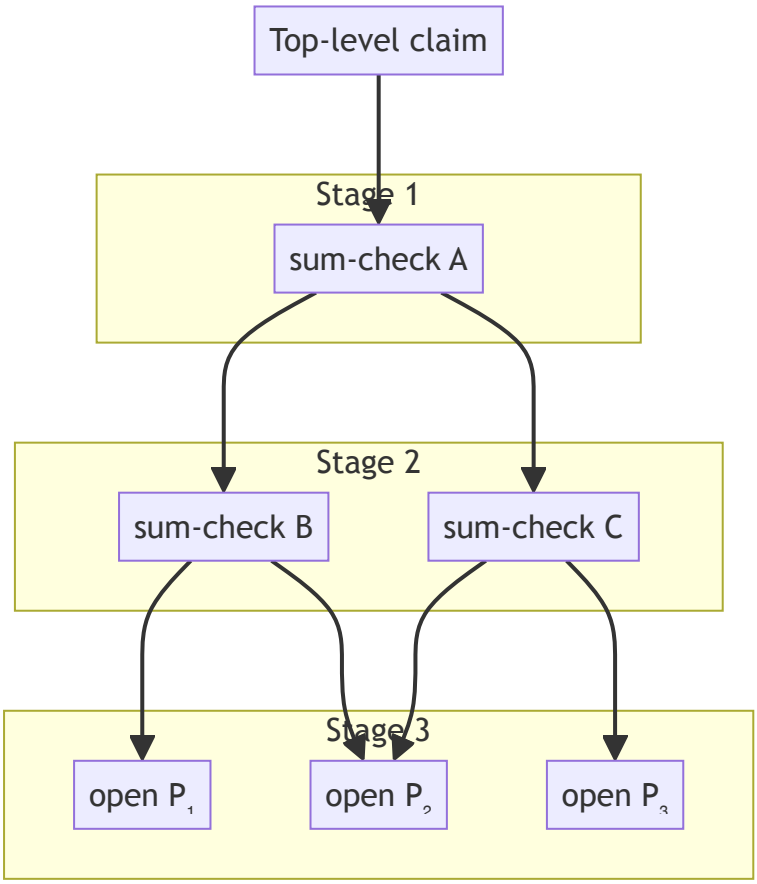
Sum-checks as a DAG

The design principle above applies to individual polynomials, but virtualization at scale creates a structural picture worth seeing in its own right. When a sum-check ends at a random point r and the polynomial it was reasoning about is virtual, the resulting evaluation claim has to be discharged by *another* sum-check. That second sum-check might itself end with a claim about another virtual polynomial, requiring a third, and so on. The dependencies form a directed acyclic graph (DAG): each sum-check is a node, the output claims it produces are outgoing edges, and the input claims it consumes are incoming edges. Committed polynomials are sources (no incoming edges from other sum-checks); the final opening proof is the sink.

The DAG induces a partial order, and that partial order determines the minimum number of *stages* the protocol must run in. Two sum-checks can share a stage only if neither depends on the other's output. The longest path in the DAG sets a lower bound on the number of stages: protocols with deep chains of virtualization unavoidably have many sequential rounds. Jolt, which proves RISC-V execution, runs roughly 40 sum-checks organized into 8 stages by this dependency structure.

Within each stage, independent sum-checks can be batched via random linear combination. Sample $\rho_1, \ldots, \rho_k$ from the verifier's transcript, form $g_{\text{batch}} = \sum_i \rho_i \cdot g_i$, and run one sum-check on the combined claim. This is the *horizontal* dimension of optimization: batching within a stage. Stages are the *vertical* dimension: sequential dependencies that cannot be avoided. The design recipe for a fast prover is to map the full DAG, minimize the number of stages (constrained by the longest path), and batch every independent sum-check within each stage.

A small example illustrates the structure:



Read top-to-bottom for execution order. Stage 1 runs one sum-check that ends with two residual claims, both about virtual polynomials. Stage 2 discharges those residual claims with two independent sum-checks (B and C), which collapse into a single batched sum-check via random linear combination. Stage 3 discharges the resulting claims with PCS openings on the three committed polynomials, which collapse into a single batched opening.

The vertical axis (stages) is bounded by dependencies: stage 2 cannot start until stage 1 has produced its residual claims, and stage 3 cannot start until stage 2 is done. The horizontal axis within each stage is free, so anything independent collapses via batching. A protocol designer cannot shrink the height (stages) without restructuring the protocol's data dependencies, but they can always shrink the width by batching anything independent.

Time-Varying Functions

So far virtualization has applied to *static* objects: a derived polynomial $\tilde{c} = \tilde{a} \cdot \tilde{b}$, an access matrix ra that factors into chunks, a vector of read results z determined by addresses and a fixed table. The

next test for the principle is a moving target: state that changes over time. This is the third instance of the virtualization theme, now applied to the trickier case where the table being read evolves between accesses.

Batch evaluation proves claims of the form $z_j = f(y_j)$ where f is fixed. Real computation does not work that way. Registers change. Memory gets written. The lookup tables from Chapter 14 assume static data, yet a CPU's registers are anything but static. When a zkVM executes ADD `R1, R2, R3`, it reads R1 and R2, computes the sum, writes to R3. The next instruction might read R3 and get the new value. The value at R3 depends on *when* you query it.

The general phenomenon is the **time-varying function problem**. A function f that gets updated at certain steps; a query $f(y_j)$ at time j returns the value f held at that moment. The claim "I correctly evaluated f" depends on the timing of the evaluation.

Setup and the Naive Cost

Formally, over T time steps the computation performs operations on a table with K entries. Each operation is either a *read* (query position k, receive value v) or a *write* (set position k to value v). The prover's job is to demonstrate that every read returns the value from the most recent write to that position.

The naive way to verify this is to commit to a $K \times T$ matrix where entry (k, j) records the value at position k after step j. For a zkVM with 32 registers and a million instructions, this is $32 \times 10^6 = 3.2 \times 10^7$ entries: expensive but conceivable. For RAM with 2^{32} addresses and a million instructions, this is 2^{52} entries, vastly beyond what any prover could commit. Direct commitment is impossible at zkVM scale.

This is exactly the situation virtualization was built for. The state table is enormous, but it is *determined* by the write history. We do not need to commit it; we need to commit only the data that uniquely determines it.

The Unified Principle

What lets us virtualize the state table is that read-only and time-varying tables turn out to share the same verification structure. Both answer the question "what value should this read return?" the same way: as a sum over positions, weighted by an access indicator, verified via sum-check. The only difference is whether the table itself is fixed or reconstructed from a write history. Throughout this subsection, K is the table size (number of positions), T is the number of operations, and we use the standard memory-checking notation: ra for *read addresses*, rv for *read values* (the same object as

z in the batch evaluation section), wa for *write addresses*, wv for *write values*. The parallel naming makes the read/write symmetry visible.

Recall the read-only case from the batch evaluation section: the value at position k is just a fixed $f(k)$, the verification equation is $rv_j = \sum_k ra(k,j) \cdot f(k)$, and the prover commits to the tensor-decomposed chunks of ra while leaving the read values rv virtual. The function f itself is public or preprocessed; nothing about f needs to be committed at all.

The read-write case has the same verification equation but with one critical change: f now depends on time. Define $f(k,j)$ as "what value is stored at position k just before time j?" Then:

$$rv_j = \sum_k ra(k,j) \cdot f(k,j)$$

The challenge is that $f(k,j)$ is now a $K \times T$ table, far too large to commit. The previous trick (tensor decomposition) does not save us: the time-dependence does not factor through chunking the way an address does. We need a different escape, and virtualization provides it. The state table is *determined* by the write history, so we can reconstruct it from writes rather than store it. Let $wa_{j'}$ denote the address written to at step j', and $\Delta_{j'}$ the value added to that address (zero if step j' is a read, non-zero if it is a write). Then:

$$f(k,j) = \text{initial}(k) + \sum_{j'<j} \mathbf{1}[wa_{j'} = k] \cdot \Delta_{j'}$$

Read this as a walk through history. For each past step $j' < j$, the indicator $\mathbf{1}[wa_{j'} = k]$ asks "did we write to address k at step j'?" If yes, include the increment $\Delta_{j'}$; if no, skip it. The sum picks out exactly the prior writes that targeted address k and adds them to the initial value.

The massive $K \times T$ state table dissolves into two sparse objects. The first is a T-vector of write addresses wa. Just like the read addresses ra, each entry of wa is an ℓ-bit position in the same K-sized table, so the same tensor decomposition applies: split each ℓ-bit address into d chunks of ℓ/d bits, encode wa as d smaller chunk matrices $wa_1, \ldots, wa_d$ of size $K^{1/d} \times T$ each, and treat the full wa as virtual through the product $wa(k,j) = \prod_i wa_i(k_i, j)$. Nothing about the read versus write distinction changes how chunking works; it depends only on addresses being bit strings. The second sparse object is a length-T increment vector Δ, which has no address structure to chunk and gets committed directly. The state table $f(k,j)$ itself is virtual.

The committed objects in each case:

Case	Committed	Virtual
Read-only	ra chunks	rv, table f (public)
Read-write	ra chunks, wa chunks, Δ	rv, state table $f(k, j)$

The read-write prover commits to a few extra objects (write addresses and the increment vector) but never commits the state table. This is what makes time-varying memory tractable.

Data	Changes?	Technique	Committed	Virtual
Instruction tables	No	Read-only	ra chunks	rv, table f
Bytecode	No	Read-only	ra chunks	rv, table f
Registers	Yes	Read-write	ra, wa chunks, Δ	rv, state $f(k, j)$
RAM	Yes	Read-write	ra, wa chunks, Δ	rv, state $f(k, j)$

Both techniques use the same sum-check structure. The difference is that read-only tables have $f(k)$ fixed (public or preprocessed), while read-write tables have $f(k, j)$ that must be virtualized from the write history.

Both paths lead to the same destination, where commitment cost is proportional to operations T (not table size K). A table with 2^{128} entries costs no more to access than one with 2^8.

Why This Matters for Real Systems

In a zkVM proving a million CPU cycles, memory operations dominate the execution trace. Every instruction reads registers, many access RAM, all fetch from bytecode. A RISC-V instruction like `lw t0, 0(sp)` involves: one bytecode fetch (read-only), one register read for `sp` (read-write), one memory read (read-write), one register write to `t0` (read-write). Four memory operations for one instruction.

If each memory operation required commitment proportional to memory size, proving would be impossible. A million instructions × four operations × 2^{32} addresses = 2^{54} commitments. The sun would burn out first.

The techniques above make it tractable. Registers, RAM, and bytecode all reduce to the same pattern: commit to addresses and values (or increments), virtualize everything else. The distinction be-

tween "read-only" and "read-write" is simply whether the table f is fixed or must be reconstructed from writes.

What emerges is a surprising economy. A zkVM with 2^{32} bytes of addressable RAM, 32 registers, and a megabyte of bytecode commits roughly the same amount per cycle regardless of these sizes. The commitment cost tracks operations, not capacity. Memory becomes (in a sense) free. You pay for what you use, not what you could use.

Circuit *wiring* (the copy-constraint problem from Chapter 13) is itself a memory access pattern. When the output of gate j feeds into gate k as an input, the circuit is "reading" a value that was "written" by gate j. Quotienting-based systems handle this through permutation arguments (grand products over accumulated ratios). In the memory-checking framework developed here, the same constraint reduces to a read-write access pattern over a table of wire values, verified via the same ra/wa machinery. Chapter 22 develops this parallel explicitly, showing that wiring constraints are where the two PIOP paradigms diverge most sharply in abstraction while converging in purpose.

The Padding Problem and Jagged Commitments

We've virtualized polynomials, memory states, and intermediate circuit layers. But a subtler waste remains: the boundaries between different-sized objects.

This problem emerged when zkVM teams tried to build *universal* recursion circuits. Recursion (Chapter 23) is the technique of proving that a verifier accepted another proof, expressed as a circuit and proven about. The dream of universal recursion is one such circuit that can verify any program's proof, regardless of what instructions that program used, so the same recursive infrastructure handles every workload. The reality was that different programs have different instruction mixes, and the verifier circuit seemed to depend on those mixes.

The Problem: Tables of Different Sizes

A zkVM's computation trace comprises multiple tables, one per CPU instruction type. The ADD table holds every addition executed; the MULT table every multiplication; the LOAD table every memory read. These tables have wildly different sizes depending on what the program actually does.

Consider two programs:

- Program A: heavy on arithmetic. 1 million ADDs, 500,000 MULTs, 10,000 LOADs.
- Program B: heavy on memory. 100,000 ADDs, 50,000 MULTs, 800,000 LOADs.

Same total operations, but completely different table shapes. If the verifier circuit depends on these shapes, we need a different circuit for every possible program behavior. That's not universal recursion

but combinatorial explosion.

Now we need to commit to all this data. What are our options?

Option 1: Commit to each table separately. Each table becomes its own polynomial commitment. The problem is that verifier cost scales linearly with the number of tables. In a real zkVM with dozens of instruction types and multiple columns per table, verification becomes expensive. Worse, in recursive proving, where we prove that a verifier accepted, each separate commitment adds complexity to the circuit we're proving.

Option 2: Pad everything to the same size. Put all tables in one big matrix, padding shorter tables with zeros until they match the longest. Now we commit once. The problem is that if the longest table has 2^{20} rows and the shortest has 2^{10}, we're committing to a million zeros for the short table. Across many tables, the wasted commitments dwarf the actual data.

Neither option is satisfactory. We want the efficiency of a single commitment without paying for empty space.

The Intuition: Stacking Books on a Shelf

Think of each table as a stack of books. The ADD table is a tall stack (many additions). The MULT table is shorter (fewer multiplications). The LOAD table is somewhere in between.

If we arrange them side by side, we get a jagged skyline: different heights and lots of empty space above the shorter stacks. Committing to the whole rectangular region wastes the empty space.

But what if we packed the books differently? Take all the books off the shelf and line them up end-to-end in a single row. The first million books come from ADD, the next 50,000 from MULT, then 200,000 from LOAD. No gaps and no wasted space. The total length equals exactly the number of actual books.

This is the jagged commitment idea, which is to *pack different-sized tables into one dense array*. We commit to the packed array (cheap and without wasted space) and separately tell the verifier where each table's data begins and ends.

A Concrete Example

Suppose we have three tiny tables:

	Table	Data	Height
A		$[a_0, a_1, a_2]$	3
B		$[b_0, b_1]$	2
C		$[c_0, c_1, c_2, c_3]$	4

If we arranged them as columns in a matrix, padding to height 4:

```
     A    B    C
0:  a₀   b₀   c₀
1:  a₁   b₁   c₁
2:  a₂    0   c₂
3:   0    0   c₃
```

We'd commit to 12 entries, but only 9 contain real data. The three zeros are waste.

Instead, pack them consecutively into a single array:

```
Index:  0   1   2   3   4   5   6   7   8
Value: a₀  a₁  a₂  b₀  b₁  c₀  c₁  c₂  c₃
```

Now we commit to exactly 9 values: the real data. We also record the cumulative heights: table A ends at index 3, table B ends at index 5, table C ends at index 9. Given these boundaries, we can recover which table any index belongs to, and its position within that table.

From Intuition to Protocol

Now formalize this. We have 2^k tables (columns), each with its own height h_y. Arranged as a matrix, this forms a *jagged function* $p(x, y)$ where x is the row (up to 2^n) and y identifies the table. The function satisfies $p(x, y) = 0$ whenever row $x \geq h_y$ (beyond that table's height).

The total non-zero entries number $M = \sum_y h_y$. This sum is the **trace area**, the only quantity that actually matters for proving.

The prover packs all non-zero entries into a single dense array q of length M, deterministically: table 0's entries first, then table 1's, and so on. The 2D table with variable-height columns becomes a 1D array that skips the padding zeros entirely. We will call this operation **flattening**, since the variable-height skyline of the original tables is collapsed into a single flat row.

The cumulative heights $t_y = \sum_{y'<y} h_{y'}$ track where each column starts in the flattened array. Given a dense index i, two functions recover the original coordinates:

- $\text{row}_t(i)$: the row within the padded table (offset from that column's start)
- $\text{col}_t(i)$: which column i belongs to (found by comparing i against cumulative heights)

For example, with heights $(16, 16, 256)$, the cumulative heights are $(0, 16, 32)$ (one entry per column, recording where each column *starts* in the dense array). The total trace area is $M = 16 + 16 + 256 = 288$, the position just past the last entry. Column 2 therefore occupies the range $[32, 288)$. Index $i = 40$ falls in column 2 (since $32 \leq 40 < 288$) at row $40 - 32 = 8$.

The prover commits to:

- q: the dense array of length M containing all actual values
- The cumulative heights $t_y = h_0 + h_1 + \cdots + h_{y-1}$, sent in the clear (just 2^k integers)

The jagged polynomial p is never committed. It exists only as a relationship between the dense q and the boundary information.

Making It Checkable

The verifier wants to query the original jagged polynomial and ask, "what is $\tilde{p}(z_r, z_c)$?" This asks for a weighted combination of entries from table z_c at rows weighted by z_r.

The key equation translates this into a sum over the dense array:

$$\tilde{p}(z_r, z_c) = \sum_{i \in \{0,1\}^m} q(i) \cdot \widetilde{\text{eq}}(\text{row}(i), z_r) \cdot \widetilde{\text{eq}}(\text{col}(i), z_c)$$

The two $\widetilde{\text{eq}}$ factors are selectors. The first, $\widetilde{\text{eq}}(\text{col}(i), z_c)$, picks out entries belonging to the requested table; the second, $\widetilde{\text{eq}}(\text{row}(i), z_r)$, picks out entries at the requested row. Their product enforces *double selection*: a term contributes $q(i)$ only when dense index i maps to both the correct row and the correct column.

This is a sum over M terms and exactly the sum-check form we've used throughout the chapter. The prover runs sum-check; at the end, the verifier needs $\tilde{q}(r)$ at a random point (handled by the underlying PCS) and the selector function evaluated at that point.

The selector function (despite involving $\text{row}_t(i)$ and $\text{col}_t(i)$) is efficiently computable, since it's a simple comparison of i against the cumulative heights. This comparison can be done by a small read-once branching program (essentially a specialized circuit that checks if an index falls within a specific range using very few operations). This means its multilinear extension evaluates in $O(m \cdot 2^k)$ field operations.

> **Remark (Batching selector evaluations).** During sum-check, the verifier must evaluate the selector function $\hat{f}_t$ at each round's challenge point. With m rounds, that's m evaluations at $O(2^k)$ each, totaling $O(m \cdot 2^k)$. A practical optimization: the prover claims all m evaluations upfront, and the verifier batches them via random linear combination. Sample random α, check $\sum_j \alpha^j \hat{f}_t(r_j) = \sum_j \alpha^j y_j$ where y_j are the claimed values. The left side collapses to a single $\hat{f}_t$ evaluation at a combined point. Cost drops from $O(m \cdot 2^k)$ to $O(m + 2^k)$.

The Payoff

The prover performs roughly $5M$ field multiplications, or five per actual trace element, regardless of how elements are distributed across tables. The constant 5 comes from the sum-check structure: the summand is a product of three multilinear factors (q and the two $\widetilde{\text{eq}}$ selectors), giving a degree-3 polynomial in each variable. The halving trick from Chapter 19, applied to a degree-d sum-check, costs roughly $(d+2) \cdot N$ field multiplications across all rounds (the d factor for folding each multilinear piece each round, the $+2$ for forming the round polynomial's evaluations). With $d = 3$ and $N = M$, that lands at $\approx 5M$. No padding, no wasted commitment, and a constant that does not depend on table count or table heights.

For the verifier, something useful happens. The verification circuit depends only on $m = \log_2(M)$ (the log of total trace area), not on the individual table heights h_y. Whether the trace has 100 tables of equal size or 100 tables of wildly varying sizes, the verifier does the same work.

This is the solution to the universal recursion problem from the beginning of this section. When proving proofs of proofs, the verifier circuit becomes the statement being proved. A circuit whose size depends on table configuration creates the combinatorial explosion we feared. But a circuit depending only on total trace area yields one universal recursion circuit.

One circuit to verify any program. The jagged boundaries dissolve into a single integer: total trace size.

The Deeper Point

Each virtualization earlier in this chapter replaced a *polynomial* with a formula: $\tilde{c} = \tilde{a} \cdot \tilde{b}$ avoided committing $\tilde{c}$; the tensor decomposition avoided committing the full access matrix ra; the write-history formula avoided committing the state table $f(k, j)$. In each case, the thing being virtualized was a *value* at each point.

Jagged commitments extend the same idea to *structure*. What gets virtualized is not a polynomial's values but its shape: the staircase of boundaries where each table ends. The prover never commits to the 2^{n+k}-sized jagged polynomial p with its zeros above each table's height. Instead it commits

to the dense M-sized array q and sends the cumulative heights $t_0, t_1, ...$ in the clear. The boundary information (which index belongs to which table, at which row) exists only through the formula that compares an index against the heights. The zeros that a padded approach would waste space on were never real data; they were artifacts of forcing variable-height tables into a rectangular grid. Flattening eliminates the grid, and the boundaries become metadata rather than committed content.

This is the chapter's recurring theme pushed to its furthest application: ask not what exists but what can be computed. Values, access patterns, state history, and now shape itself dissolve into formulas over committed roots.

Small-Value Preservation

We've focused on *what* to commit, but *how large* the committed values are matters too. Real witness values are usually small: 8-bit bytes, 32-bit words, 64-bit addresses. These fit in a single machine word even though the protocol's field has 256-bit elements. The dominant cost in curve-based commitment, computing g^x via double-and-add, scales as $O(\log |x|)$ group operations. If x is a 64-bit integer rather than a 256-bit field element, exponentiation takes 64 steps instead of 256, a 4× speedup. For an MSM over a million points, this translates to seconds of wall-clock time.

The optimization follows from keeping values small for as long as possible. Random challenges injected by the verifier are the main source of large field elements. Once a small witness value gets multiplied by a 256-bit challenge, the result is 256 bits and the cheapness is gone. A well-designed protocol postpones this inflation, arranging computations so that the bulk of the prover's work touches values that still fit in machine words. Jolt, Lasso, and related systems (Arasu et al., 2024) reported 4× prover speedups simply from tracking value sizes through the protocol, maintaining separate "small" and "large" categories for polynomials, and routing each to the appropriate arithmetic.

The impact compounds everywhere:

- MSM with 64-bit scalars: 4× faster than 256-bit
- Hashing small values has fewer field reductions
- FFT with small inputs gives smaller intermediate values and fewer overflows
- Sum-check products where inputs fit in 64 bits yield products that fit in 128 bits, so no modular reduction is needed

Modern sum-check-based systems track value sizes explicitly, and Jolt, Lasso, and related systems maintain separate "small" and "large" polynomial categories. Small polynomials get optimized 64-bit arithmetic. Large polynomials get full field operations. The boundary is tracked through the protocol.

The difference between a 10-second prover and a 2-second prover often lies in these details.

Key takeaways

1. **Commitment dominates prover cost in curve-based systems.** A single elliptic curve exponentiation costs $\approx 3{,}000$ field multiplications; an MSM over N points costs $\approx N/\log N$ exponentiations. After the linear-time sum-check techniques of Chapter 19, the prover spends more time committing than proving. Every optimization in this chapter reduces what the prover must commit.

2. **Enlarge the witness to simplify constraints (untrusted advice).** Adding helper values (quotients, square roots, intermediate exponentiation steps) to the witness polynomial makes the commitment slightly larger but lets the constraint system avoid expensive non-native operations. The prover computes; the constraints only verify.

3. **Batch many lookups into one evaluation via sum-check.** Proving T evaluations $z_j = f(y_j)$ reduces to a single sum-check instance over the domain of f, exploiting the one-hot sparsity of the access matrix ra. The access matrix factors via tensor decomposition (ℓ-bit addresses split into d chunks of ℓ/d bits), reducing commitment from $2^\ell \times T$ to $d \times 2^{\ell/d} \times T$. A table with 2^{128} entries costs no more to access than one with 2^8.

4. **Virtualization is the chapter's unifying principle.** Any polynomial algebraically determined by already-committed polynomials does not need its own commitment. This applies to derived polynomials ($\tilde{c} = \tilde{a} \cdot \tilde{b}$), read results (defined by access pattern and table), time-varying state (reconstructed from write history via $f(k, j) = \text{initial}(k) + \sum_{j' < j} \mathbf{1}[wa_{j'} = k] \cdot \Delta_{j'}$), and GKR layer values (each defined via sum-check in terms of the previous layer). The same principle appears on the STARK side as DEEP-ALI (Chapter 20).

5. **Virtualization creates a DAG of sum-checks.** Each virtual polynomial requires another sum-check to discharge the evaluation claim. The protocol's structure is a directed acyclic graph: committed polynomials are sources, the final opening proof is the sink. The longest path determines the minimum number of sequential stages; everything independent within a stage collapses via batching.

6. **Jagged commitments virtualize *structure*, not values.** Flattening variable-height tables into one dense array avoids committing to padding zeros. The verifier circuit depends only on total trace area M, not individual table heights, enabling one universal recursion circuit for all programs.

7. **Keep values small as long as possible.** MSM cost scales with scalar bit-width ($O(\log |x|)$ group operations per exponentiation). Witness values are typically 8-64 bits; random challenges are 256 bits. Postponing the inflation from multiplying small values by large challenges keeps the bulk of the prover's work in cheap machine-word arithmetic.

8. **Commitment cost tracks operations, not capacity.** A zkVM with 2^{32} addressable memory, dozens of instruction types, and millions of cycles commits roughly the same amount per cycle regardless of memory size or instruction mix. Memory is free; only actual computation costs.

Chapter 22: The Two Classes of PIOPs

Every modern SNARK, stripped to its essence, follows the same recipe: a Polynomial Interactive Oracle Proof (PIOP), compiled with a Polynomial Commitment Scheme (PCS), made non-interactive via Fiat-Shamir. The PIOP provides information-theoretic security: it would be sound even against unbounded provers if the verifier could magically check polynomial evaluations. The PCS adds cryptographic binding. Fiat-Shamir removes interaction.

But within this unifying framework, two distinct philosophies have emerged. They use different polynomial types, different domains, different proof strategies. They lead to systems with different performance profiles.

Understanding when to use which is not academic curiosity; it shapes every SNARK design decision.

The Divide

The two paradigms differ in their fundamental approach to constraint verification. At the deepest level, the split is *geometric*: where does your data live?

Quotienting-based PIOPs (Groth16, PLONK, STARKs) encode data as **univariate** polynomials of degree $< N$ and evaluate them on **roots of unity**, elements that cycle around the unit circle in the multiplicative group. Constraints become questions about divisibility: does the error polynomial vanish on this domain? The machinery is algebraic (division, remainder, quotient). PLONK and STARKs rely on the FFT to convert between evaluations and coefficients; Groth16 uses the same roots-of-unity domain for its QAP but performs its heavy work through MSMs in the exponent rather than FFTs.

Sum-check-based PIOPs (Spartan, HyperPlonk, Jolt) encode data as **multilinear** polynomials, n variables each of degree 1, and evaluate them on the **Boolean hypercube** $\{0,1\}^n$. Constraints become questions about sums: does the weighted average over all vertices equal zero? The machinery is probabilistic (randomization collapses exponentially many constraints into one) and the key algorithm is the halving trick, which scans data linearly.

The polynomial type and the domain are linked. Univariate polynomials need a structured evaluation domain with FFT-friendly symmetry (roots of unity provide this). Multilinear polynomials need the $\{0,1\}^n$ hypercube because that is where their Lagrange basis is defined and where the halving trick's fold-in-half structure applies. Choosing one determines the other.

For a decade, the circle dominated because its mathematical tools (pairings, FFTs) matured first. But the hypercube has risen recently because it fits better with how computers actually work: bits, arrays, and linear memory scans.

Both achieve the same goal: succinct verification of arbitrary computations. Both ultimately reduce to polynomial evaluation queries. But they arrive there by different paths, and those paths have consequences.

Historical Arc

The divide between paradigms has a history.

The PCP Era (1990s-2000s)

The theoretical foundations came from PCPs (Probabilistically Checkable Proofs). A PCP is a single, static proof string that the verifier queries at random positions, non-interactive by construction.

PCPs used univariate polynomials implicitly. The prover encoded the computation as polynomial evaluations; the verifier checked random positions. Soundness came from low-degree testing and divisibility arguments, the ancestors of quotienting.

Merkle trees provided commitment. Kilian showed how to make the proof succinct by hashing the full proof string, letting the verifier query random positions, and having the prover open those positions with Merkle paths.

The SNARK Era (2010s)

Groth16, PLONK, and their relatives refined the quotienting approach. KZG's constant-size proofs made verification fast (just a few pairings), and the trusted setup was an acceptable trade-off for many applications.

These systems dominated deployed ZK applications: Zcash, various rollups, privacy protocols. Quotienting became synonymous with "practical SNARKs."

The Sum-Check Renaissance (2020s)

Systems like Spartan, Lasso, and Jolt demonstrated that sum-check-based designs achieve the fastest prover times. The reason, crystallized in Chapter 19, is that interaction is a resource, and removing it twice (once in the PIOP, once via Fiat-Shamir) is wasteful.

GKR's layer-by-layer virtualization, combined with efficient multilinear PCS, enabled provers to approach linear time. Virtual polynomials slashed commitment costs.

The modern view is that quotienting and sum-check are both valid tools. Neither dominates universally. The choice depends on the application's specific constraints.

A Common Task: Proving $a \circ b = c$

To make the comparison concrete, consider the entrywise product constraint:

$$a_i \cdot b_i = c_i \quad \text{for all } i = 1, \ldots, N$$

where $N = 2^n$. The prover has committed to vectors $a, b, c \in \mathbb{F}^N$ and must prove this relationship holds at every coordinate.

This constraint captures half the logic of circuit satisfiability: verifying that gate outputs equal products of gate inputs. (The other half, wiring constraints that enforce copying, we'll address shortly.) Let's trace both paradigms through this single task.

The Quotienting Path

Setup

Choose an evaluation domain $H = \{\alpha_1, \ldots, \alpha_N\} \subset \mathbb{F}$ of size N. The standard choice: the N-th roots of unity, $H = \{1, \omega, \omega^2, \ldots, \omega^{N-1}\}$ where $\omega^N = 1$.

Define univariate polynomials by Lagrange interpolation:

- $\hat{a}(X)$ of degree $< N$: the unique polynomial satisfying $\hat{a}(\alpha_i) = a_i$
- $\hat{b}(X)$ and $\hat{c}(X)$ similarly

These are univariate low-degree extensions of the vectors, anchored at the roots of unity.

From pointwise constraints to divisibility

The constraint $a_i \cdot b_i = c_i$ for all i is equivalent to saying that $\hat{a}(\alpha) \cdot \hat{b}(\alpha) - \hat{c}(\alpha) = 0$ for all $\alpha \in H$.

By the Factor Theorem, a polynomial vanishes on all of H if and only if it's divisible by the vanishing polynomial:

$$Z_H(X) = \prod_{\alpha \in H} (X - \alpha)$$

So the constraint becomes: there exists a polynomial $Q(X)$ such that

$$\hat{a}(X) \cdot \hat{b}(X) - \hat{c}(X) = Q(X) \cdot Z_H(X)$$

The quotient Q is the witness to divisibility.

The Protocol

1. **Prover commits** to $\hat{a}, \hat{b}, \hat{c}$ using a univariate PCS (typically KZG)
2. **Prover computes** the quotient: $Q(X) = \frac{\hat{a}(X) \cdot \hat{b}(X) - \hat{c}(X)}{Z_H(X)}$
3. **Prover commits** to Q
4. **Verifier sends** random challenge $r \in \mathbb{F}$
5. **Prover provides** evaluations $\hat{a}(r), \hat{b}(r), \hat{c}(r), Q(r)$ with opening proofs
6. **Verifier checks**: $\hat{a}(r) \cdot \hat{b}(r) - \hat{c}(r) = Q(r) \cdot Z_H(r)$

Why Roots of Unity?

For arbitrary H, computing $Z_H(r)$ requires $O(N)$ operations: a factor of $(r - \alpha)$ for each element. But when H consists of N-th roots of unity:

$$Z_H(X) = X^N - 1$$

The verifier computes $Z_H(r) = r^N - 1$ in $O(\log N)$ time via repeated squaring. This simple structure, an accident of multiplicative group theory, makes quotienting practical. Chapter 13 develops this further: roots of unity also enable FFT-based polynomial arithmetic and the shift structure needed for accumulator checks.

Soundness

If the constraint fails at some $\alpha_i \in H$, then $\hat{a}(X) \cdot \hat{b}(X) - \hat{c}(X)$ is *not* divisible by $Z_H(X)$. Any claimed quotient Q will fail: the polynomial

$$\hat{a}(X) \cdot \hat{b}(X) - \hat{c}(X) - Q(X) \cdot Z_H(X)$$

is non-zero. By Schwartz-Zippel, a random r catches this with probability at least $1 - (2N-1)/|\mathbb{F}|$ (overwhelming for large fields).

Cost Analysis

The quotient polynomial has degree at most $2N-2-N = N-2$. Computing it requires polynomial division, typically done via FFT in $O(N \log N)$ time. Committing to Q costs additional PCS work.

The prover's dominant costs: FFT for quotient computation, MSM for commitment.

The hidden cost in univariate systems is the *memory access pattern*, not the $O(N \log N)$ time complexity. FFTs require "butterfly" operations that shuffle data across the entire memory space: element i interacts with element $i + N/2$, then $i + N/4$, and so on. These non-local accesses cause massive cache misses on modern CPUs. In contrast, sum-check's halving trick scans data linearly (adjacent pairs combine), which is cache-friendly and easy to parallelize across cores. For large N, the memory bottleneck often dominates the arithmetic.

The Sum-Check Path

Setup

The quotienting approach indexed vectors by roots of unity: a_i at ω^i. Sum-check indexes them by bit-strings instead: a_w for $w \in \{0,1\}^n$, where $N = 2^n$. For $N = 4$: positions $\omega^0, \omega^1, \omega^2, \omega^3$ become $00, 01, 10, 11$. Same data, different addressing scheme.

Define multilinear polynomials, the unique extensions that are linear in each variable:

- $\tilde{a}(x)$: satisfies $\tilde{a}(w) = a_w$ for all $w \in \{0,1\}^n$
- $\tilde{b}(x)$ and $\tilde{c}(x)$ similarly

Where quotienting uses Lagrange interpolation over roots of unity to get univariate polynomials of degree $N-1$, sum-check uses multilinear extension over the hypercube to get n-variate polynomials of degree 1 in each variable. Both encodings uniquely determine the original vector; they just live in different polynomial spaces.

From pointwise constraints to a random linear combination

The constraint $a_w \cdot b_w = c_w$ for all $w \in \{0,1\}^n$ means:

$$\tilde{a}(w) \cdot \tilde{b}(w) - \tilde{c}(w) = 0 \quad \text{for all } w \in \{0,1\}^n$$

Define $g(x) = \tilde{a}(x) \cdot \tilde{b}(x) - \tilde{c}(x)$. We want g to vanish on the hypercube.

Instead of proving divisibility (which would require a quotient polynomial), sum-check takes a *random linear combination*. Define:

$$q(r) = \sum_{w \in \{0,1\}^n} \widetilde{\mathsf{eq}}(r, w) \cdot g(w)$$

for verifier-chosen random $r \in \mathbb{F}^n$.

The polynomial $\widetilde{\mathsf{eq}}(r, x)$ is the multilinear extension of the equality predicate: it equals 1 when $x = r$ (on the hypercube) and 0 otherwise. But for general field elements, it acts as a random weighting function:

$$\widetilde{\mathsf{eq}}(r, w) = \prod_{i=1}^{n} (r_i \cdot w_i + (1 - r_i)(1 - w_i))$$

If any $g(w) \neq 0$, then q is a non-zero polynomial in r. By Schwartz-Zippel, $q(r) \neq 0$ with probability at least $1 - n/|\mathbb{F}|$.

The Protocol

1. **Prover commits** to $\tilde{a}, \tilde{b}, \tilde{c}$ using an MLE-based PCS
2. **Verifier sends** random $r \in \mathbb{F}^n$
3. **Prover and verifier run sum-check** on $\sum_w \widetilde{\mathsf{eq}}(r, w) \cdot g(w)$, claimed to equal 0
4. **Sum-check reduces** to evaluating $\widetilde{\mathsf{eq}}(r, z) \cdot g(z)$ at a random point $z \in \mathbb{F}^n$
5. **Prover provides** $\tilde{a}(z), \tilde{b}(z), \tilde{c}(z)$ with opening proofs
6. **Verifier computes** $\widetilde{\mathsf{eq}}(r, z)$ directly (just n multiplications) and checks that $\widetilde{\mathsf{eq}}(r, z) \cdot (\tilde{a}(z) \cdot \tilde{b}(z) - \tilde{c}(z))$ equals the claimed final value

Cost Analysis

Sum-check proving via the halving trick (Chapter 19) takes $O(N)$ time for dense polynomials. The prover provides three opening proofs, no quotient commitment needed.

The prover's dominant costs: sum-check field operations, PCS opening proofs.

The Comparison

Aspect	Quotienting	Sum-Check
Polynomial type	Univariate, degree $< N$	Multilinear, n variables
Domain	Roots of unity H	Boolean hypercube $\{0,1\}^n$
Constraint verification	Z_H divides error	Random linear combination
Extra commitment	Quotient $Q(X)$	None
Prover time	$O(N \log N)$ for FFT	$O(N)$ dense, $O(T)$ sparse
Interaction	1 round (after commitment)	n rounds (sum-check)
Sparsity handling	Quotient typically dense	Natural via prefix-suffix

The two paradigms embody different engineering mindsets, and an analogy helps sharpen the distinction. Quotienting is *signal processing*. It treats data like a sound wave, running a Fourier transform (FFT) to convert the signal into a frequency domain where errors stick out like a sour note. Divisibility by Z_H is the test: a clean signal has no energy at the forbidden frequencies. Sum-check is *statistics*. It treats data like a population, taking a random weighted average over the whole population and checking whether that average is zero. No frequency analysis required, just a linear scan.

The performance gap follows from this distinction. FFTs require butterfly operations that shuffle data across the entire memory space (Chapter 20's discussion of cache misses in the NTT), while sum-check's halving trick scans data linearly, which is cache-friendly and trivially parallelizable. Sparsity widens the gap further. Quotienting always pays $O(N \log N)$ for the FFT regardless of how many constraints are non-trivial, and the quotient polynomial $Q(X)$ must be committed even when most of the constraint evaluations are zero. Sum-check's cost drops to $O(T)$ for T non-zero terms, ignoring the zeros entirely (Chapter 19). At zkVM scale, where $T \ll N$, this difference is orders of magnitude. Prover speed is not the whole story, however. The PCS pairing and the "Choosing a Paradigm" sections below will show that quotienting recovers the advantage on proof size and verifier efficiency, dimensions where the tradeoff runs in the opposite direction.

Wiring Constraints: The Second Half

The $a \circ b = c$ constraint checks that gate computations are correct. But a circuit also has *wiring*: the output of gate j might feed into gates k and ℓ as inputs. We must verify that copied values match, that $a_k = c_j$ and $b_\ell = c_j$.

This is the "copy constraint" problem, and the two paradigms handle it differently.

Quotienting: Permutation Arguments

PLONK-style systems encode wiring as a permutation. Consider all wire values arranged in a single vector. The permutation σ maps each wire position to the position that should hold the same value.

The constraint: $a_{\sigma(i)} = a_i$ for all i.

PLONK verifies this through a grand product argument (Chapter 13). For each wire position, form the ratio:

$$\frac{a_i + \beta \cdot i + \gamma}{a_i + \beta \cdot \sigma(i) + \gamma}$$

If the permutation constraint is satisfied, multiplying all these ratios gives 1: a massive cancellation of numerators and denominators.

Proving this grand product requires an accumulator polynomial: $z_i = \prod_{j \leq i}(\text{ratio}_j)$. The prover commits to this accumulator and proves it satisfies the recurrence relation via… quotienting. An additional quotient polynomial for the accumulator constraint.

Sum-Check: Memory Checking

Sum-check systems take a different view: wiring is memory access.

Each wire value is "written" to a memory cell when it's computed. Each wire position that uses that value "reads" from the cell. The constraint: reads return the values that were written.

The verification reduces to sum-check over access patterns. For each read at position j, define an access indicator $ra(k, j) = 1$ if the read targets cell k, and 0 otherwise. The read value must satisfy:

$$rv_j = \sum_k ra(k, j) \cdot f(k)$$

where $f(k)$ is the value stored at cell k. This equation says: "the value I read equals the sum over all cells, but the indicator zeroes out everything except the cell I actually accessed."

For read-only tables (like bytecode or lookup tables), $f(k)$ is fixed. For read-write memory (like registers or RAM), $f(k)$ becomes $f(k, j)$: the value at cell k at time j, reconstructed from the history of writes. Chapter 21 shows how this state table can be virtualized: rather than committing to the full $K \times T$ matrix, commit only to write addresses and value increments, then compute the state implicitly via sum-check.

The access indicator matrix ra is sparse (each read touches exactly one cell) and decomposes via tensor structure, making commitment cost proportional to operations rather than memory size.

Wiring: The Comparison

Aspect	Permutation Argument	Memory Checking
Abstraction	Wires as permutation cycles	Wires as memory cells
Core mechanism	Grand product of ratios	Sum over access indicators
Extra commitment	Accumulator polynomial Z	Access matrices (tensor-decomposed)
Structured access	No special benefit	Exploits sparsity naturally
Read-write memory	Requires separate handling	Unified with wiring

The algebraic structure reflects this split. Permutation arguments use **products** (accumulators that multiply ratios), while memory checking uses **sums** (access counts weighted by values). In finite fields, sums are generally cheaper than products. Sums linearize naturally (the sum of two access patterns is the combined access pattern), while products require careful accumulator bookkeeping. This is why memory checking integrates more cleanly with sum-check's additive structure.

For circuits with random wiring, both approaches have similar cost. The permutation argument requires an accumulator commitment; memory checking requires access matrices. The difference emerges with structure: repeated reads from the same cell, locality in access patterns, or mixing read-only and read-write data all favor the memory checking view.

The PCS Connection

Each PIOP paradigm pairs with a matching polynomial commitment scheme, and the matching is not arbitrary. The reason is that every PIOP ends the same way: sum-check or quotient verification reduces the original claim to "evaluate this committed polynomial at a random point." The *shape* of that random point determines which PCS can serve it. A quotienting PIOP ends with a univariate evaluation query, "what is $f(r)$ for $r \in \mathbb{F}$?" A sum-check PIOP ends with a multilinear evaluation query, "what is $\tilde{f}(r_1, \ldots, r_n)$ for $r \in \mathbb{F}^n$?" The PCS must handle exactly the query type the PIOP produces.

Univariate PCS for quotienting. The query is a single field element r. KZG handles this with a single group element commitment and a constant-size opening proof (one pairing check), at the cost of a

trusted setup and pairing-friendly curves. FRI handles it with Merkle commitments and logarithmic-size proofs via folding, transparent and post-quantum but larger. Both operate over the same roots-of-unity domain that the PIOP already uses for FFT-based quotient computation.

Multilinear PCS for sum-check. The query is an n-dimensional point $r \in \mathbb{F}^n$. Bulletproofs/IPA handle this via recursive folding that halves the polynomial one variable at a time (logarithmic proofs, no trusted setup). Dory uses pairing-based inner products for efficient batch opening. Hyrax and Ligero use Merkle trees and linear codes. All commit to evaluation tables over $\{0,1\}^n$ and open at arbitrary points in $\mathbb{F}^n$, matching the query shape sum-check produces.

In principle, any PIOP can use any PCS of the matching polynomial type. In practice, the best systems co-optimize PIOP and PCS: the FFT that the quotienting PIOP uses for quotient computation is the same FFT that prepares the polynomial for KZG or FRI commitment, and the halving structure that sum-check uses for proving is the same halving structure that IPA uses for opening. The algorithm is shared; only its role changes.

Choosing a Paradigm

The comparisons above reveal a pattern. Quotienting and sum-check differ in what they optimize for, not only in mechanism.

Quotienting excels when structure is fixed and dense. The quotient polynomial costs $O(N)$ regardless of how many constraints actually matter. FFT runs in $O(N \log N)$ regardless of sparsity. The permutation argument handles any wiring pattern equally. This uniformity is a strength when constraints fill the domain densely and circuit topology is known at compile time. Small circuits with degree-2 or degree-3 constraints, existing infrastructure with optimized KZG and FFT libraries, applications where proof size matters more than prover time: these favor quotienting. Chapter 20 shows how small-field NTT optimization, DEEP-ALI, and batched FRI reduce the concrete cost of this $O(N \log N)$ path to the point where it competes with sum-check for structured workloads like hashing.

Sum-check excels when structure is dynamic and sparse. The prefix-suffix algorithm runs in $O(T)$ for T non-zero terms, ignoring the $N - T$ zeros entirely. Memory checking handles structured access patterns (locality, repeated reads) more efficiently than permutation arguments. Virtual polynomials let you skip commitment entirely for intermediate values. This adaptivity matters for large circuits with billions of gates, memory-intensive computation with lookup arguments and batch evaluation, and zkVMs where the constraint pattern depends on the program being executed.

The wiring story reinforces this. Permutation arguments treat all wire patterns uniformly: a random scramble costs the same as a structured dataflow. Memory checking adapts: tensor decomposition exploits address structure, virtualization skips commitment to state tables, and read-only versus read-write falls out of the same framework.

A useful heuristic: if you know exactly what your circuit looks like at compile time and it fits comfortably in memory, quotienting's simplicity wins. If your circuit's shape depends on runtime data, or if you're pushing toward billions of constraints, sum-check's adaptivity wins. The choice propagates further than this chapter covers: Chapter 23 shows that the recursion strategy (full verification vs. folding) also aligns with the paradigm, extending the compatibility chain from constraint system through PIOP, PCS, and into recursive composition.

Key takeaways

1. **One choice determines the rest.** Quotienting uses univariate polynomials over roots of unity and proves constraints via divisibility (Z_H divides the error). Sum-check uses multilinear polynomials over the Boolean hypercube and proves constraints via random linear combination. Polynomial type, domain, and constraint strategy are linked; choosing one determines the other two.
2. **Quotienting is signal processing; sum-check is statistics.** Quotienting runs an FFT to move data into a frequency domain where errors violate divisibility. Sum-check takes a random weighted average and checks whether it vanishes. The FFT shuffles data across memory (cache misses); the halving trick scans linearly (cache-friendly). This explains the prover-speed gap.
3. **Sparsity is where the paradigms diverge most in cost.** Quotienting pays $O(N \log N)$ for the FFT and commits a quotient polynomial $Q(X)$ regardless of how many constraints are non-trivial. Sum-check pays $O(T)$ for T non-zero terms, ignoring the rest. When $T \ll N$ (the zkVM regime), the difference is orders of magnitude in prover time.
4. **Proof size and verifier cost favor quotienting.** KZG-compiled quotienting gives constant-size proofs verified in a few pairings. Sum-check proofs grow logarithmically and require n rounds of verifier work. The prover-speed advantage of sum-check trades against this.
5. **Wiring constraints expose a deep abstraction gap.** Quotienting encodes copy constraints as permutations (grand product accumulators over ratios). Sum-check encodes them as memory access (sparse indicator matrices verified via the ra/wa machinery of Chapter 21). Same constraint, different algebraic worlds.
6. **The PCS must match the PIOP's query shape.** A quotienting PIOP ends with a univariate evaluation query ($f(r)$ for $r \in \mathbb{F}$); a sum-check PIOP ends with a multilinear one ($\tilde{f}(r_1, \ldots, r_n)$ for $r \in \mathbb{F}^n$). KZG and FRI serve the first; IPA, Dory, and Hyrax serve the second. The algorithms often coincide: the FFT that computes quotients is the same FFT that prepares KZG commitments; the halving that drives sum-check is the same halving that drives IPA opening.
7. **Both paradigms avoid unnecessary commitments, by different mechanisms.** Sum-check systems use virtual polynomials (Chapter 21): any polynomial computable from committed

ones is never committed. STARK-side quotienting uses DEEP-ALI (Chapter 20): the composition polynomial is reconstructed at a single out-of-domain point rather than committed. The principle is shared; the implementation diverges.

8. **Neither paradigm dominates; choose based on your bottleneck.** Fixed circuit, dense constraints, proof size matters: quotienting. Dynamic structure, sparse constraints, prover speed matters: sum-check. The two traditions are converging (Binius uses sum-check with FRI-based commitments; Plonky3 supports both frontends over the same small-field backend), but the choice still shapes every downstream design decision.

Chapter 23: Composition and Recursion

Could you build a proof system that runs forever? A proof that updates itself every second, attesting to the entire history of a computation, but never growing in size?

The only way to keep a proof from growing is to compress it at every step. That means each new proof must *verify the previous proof* and then replace it, absorbing all the history into a fixed-size certificate. The proof system must verify its own verification logic, "eating itself." For years, this remained a theoretical curiosity, filed under "proof-carrying data" and assumed impractical.

This chapter traces how the impossible became routine. We start with **composition**: wrapping one proof inside another to combine their strengths. We then reach **recursion**: proofs that verify themselves, enabling unbounded computation with constant-sized attestations. Finally, we arrive at **folding**: a recent revolution that makes recursion cheap by deferring verification entirely. The destination is IVC (incrementally verifiable computation), where proofs grow with time but stay constant-sized. Today's zkEVMs and app-chains are built on this foundation.

No single SNARK dominates all others. Fast provers tend to produce large proofs. Small proofs come from slower provers. Transparent systems avoid trusted setup but sacrifice verification speed. Post-quantum security demands hash-based constructions that bloat proof size. Every deployed system occupies a point in this multi-dimensional trade-off space.

But here's a thought: what if we could *combine* systems? Use a fast prover for the heavy computational lifting, then wrap its output in a small-proof system for efficient delivery to verifiers. Or chain proofs together, where each proof attests to the validity of the previous, enabling unlimited computation with constant verification.

These ideas, **composition** and **recursion**, transform SNARKs from isolated verification tools into composable building blocks. The result is proof systems that achieve properties no single construction could reach alone.

Composition: Proving a Proof Is Valid

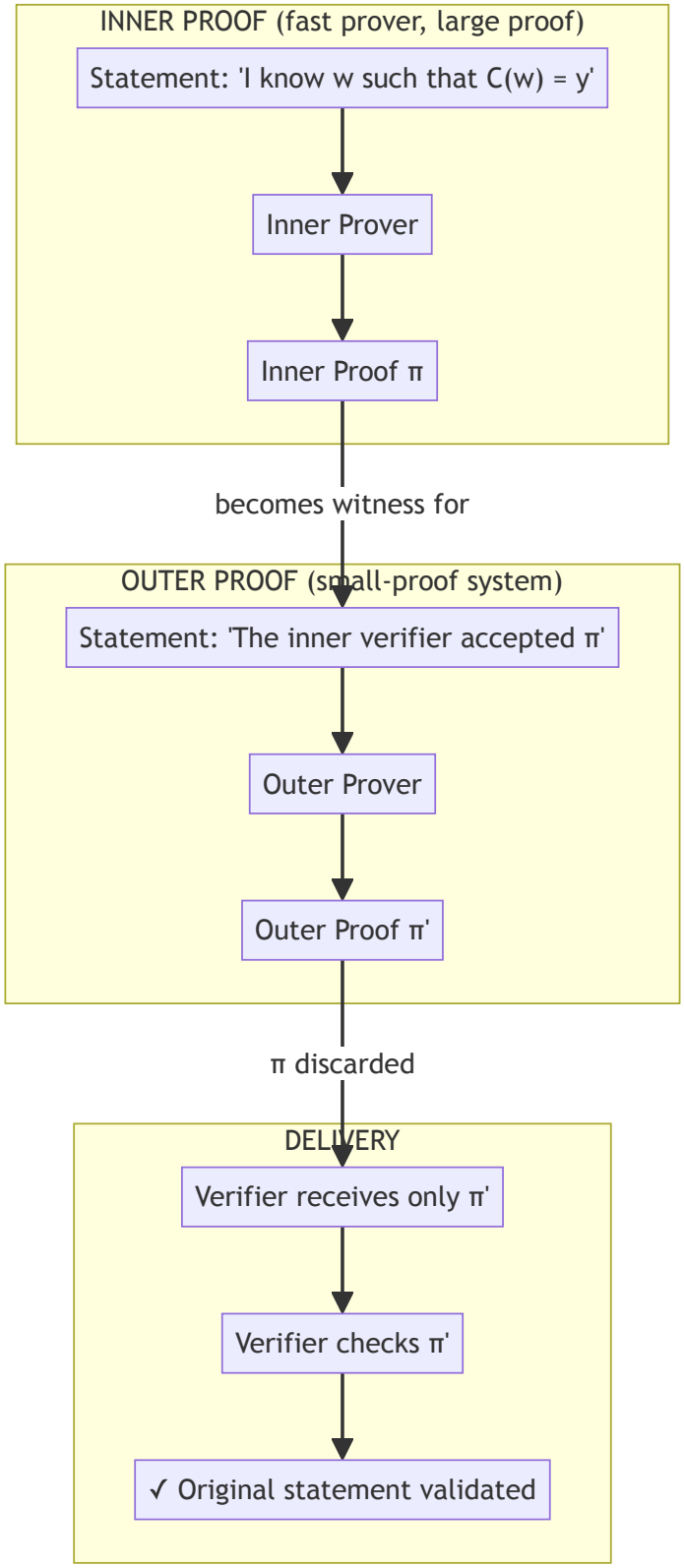
Composition means generating a *new proof* that an existing proof was valid. The distinction from verification is that the output is itself a proof, not a yes/no verdict. You have a proof π of some statement. Verifying π is a computation. You express that verification as a circuit, then prove the circuit was satisfied. The result is a second proof π' that attests to π's validity, potentially with different properties (smaller, faster to verify, based on different assumptions) than π itself.

Why do this? Different proof systems have different strengths. A STARK proves quickly but produces a 100KB proof. Groth16 produces a 128-byte proof but proves slowly. Composition lets you have both: prove the computation with a STARK, then compose the result into Groth16 for compact delivery. The formal treatment below shows why this combination inherits the fast prover of the first system and the small proof of the second, without either system's weakness dominating.

Inner and Outer

The names **inner** and **outer** describe the nesting:

- The **inner proof** is created first. It proves the statement you actually care about ("I executed this program correctly," "I know a secret satisfying this relation").
- The **outer proof** is the wrapper, created second. It proves "I ran the inner verifier and it accepted."


INNER PROOF (fast prover, large proof)
Statement: 'I know w such that C(w) = y'
Inner Prover
Inner Proof π
becomes witness for
OUTER PROOF (small-proof system)
Statement: 'The inner verifier accepted π'
Outer Prover
Outer Proof π'
π discarded
DELIVERY
Verifier receives only π'
Verifier checks π'
✓ Original statement validated

The verifier of the outer proof never sees the inner proof or the original witness. They see only π' and check that it's valid. If the outer system is zero-knowledge, nothing leaks about π or w.

Think of it like nested containers: the inner proof is a large box containing detailed evidence. The outer proof is a small envelope containing a signed attestation that someone trustworthy opened the box and verified its contents. Recipients need only check the signature on the envelope.

The Composition Construction

Composition works because verification is itself a computation, and any computation can be proven. To see why the composed system inherits the best of both components, we trace the construction step by step and analyze its cost.

Consider two SNARKs with complementary profiles. Let $|C| = N$ denote the original circuit size.

Inner SNARK $\mathcal{I}$ (fast prover, large proofs): prover time $O(N)$, proof size $O(\sqrt{N})$, verification time $O(\sqrt{N})$. Example: STARK-like systems.

Outer SNARK $\mathcal{O}$ (slow prover, tiny proofs): prover time $O(N \log N)$, proof size $O(1)$, verification time $O(1)$. Example: Groth16.

Step 1: Run the inner prover. The prover executes $\mathcal{I}$ on the original circuit C with witness w, producing proof π_I. Cost: $O(N)$.

Step 2: Arithmetize the inner verifier. The verification algorithm V_I of the inner SNARK is a computation: it reads the proof, performs checks, outputs accept or reject. Express this verification as a circuit C_{V_I} with public input x (the original statement), witness π_I, and output 1 iff V_I accepts. Since $\mathcal{I}$ has $O(\sqrt{N})$ verification time, $|C_{V_I}| = O(\sqrt{N})$, far smaller than C.

Step 3: Run the outer prover. The prover executes $\mathcal{O}$ on the verifier circuit C_{V_I}, using π_I as the witness. Cost: $O(\sqrt{N} \log \sqrt{N})$, since the outer prover is superlinear but operates on a circuit of size $\sqrt{N}$, not N.

Step 4: Deliver only the outer proof. The prover discards π_I and sends only π_O to the verifier. The inner proof was a means to an end; it never leaves the prover's machine.

Step 5: Verify. The end verifier runs V_O on π_O. Cost: $O(1)$ (inherited from the outer system). The verifier never sees π_I or w.

The cost analysis makes the payoff visible. Total prover time is $O(N) + O(\sqrt{N} \log N) \approx O(N)$, dominated by the fast inner prover. The slow outer prover contributes negligibly because it only processes the small verifier circuit. Proof size and verification time both inherit from $\mathcal{O}$: constant and fast.

For a concrete sense of scale: a million-gate circuit ($N = 10^6$) might take 5 seconds to prove with the inner STARK, producing a proof the verifier can check in ~ 1000 operations. The verifier circuit C_{V_I}

has ~ 1000 gates. Groth16 proves that 1000-gate circuit in about 1 second. Total: ~ 6 seconds. Proof size: ~ 100 bytes. Verification: 3 pairings. Without composition, running Groth16 directly on the full circuit would take minutes.

Soundness, witnesses, and delivery

The original witness w is consumed entirely in Step 1. The outer proof's witness is π_I (the inner proof), not w. The outer system proves "I possess a valid inner proof," not "I know the original witness." The soundness chain is:

$$\pi_O \text{ valid} \implies \pi_I \text{ valid} \implies w \text{ satisfies } C$$

The outer proof transitively guarantees the original statement without directly involving w. Only π_O is delivered; π_I is discarded. If the outer system is zero-knowledge, nothing leaks about π_I or w.

Field mismatches and verifier circuit blowup

The analysis above assumed the inner verifier circuit C_{V_I} is small and easy to express in the outer system. But what if the inner and outer systems speak different languages? STARKs operate over one field; Groth16 operates over another. Encoding foreign field arithmetic can blow up the verifier circuit by orders of magnitude. Trusted setup requirements, field mismatches, and post-quantum concerns all constrain which combinations actually work. The later sections on **The Verifier Circuit Problem** and **Curve Cycles** address these issues in detail.

Adding Zero-Knowledge

Here's a bonus. Suppose the inner SNARK lacks zero-knowledge: some STARK variants reveal execution traces that leak witness information. But the outer SNARK is fully ZK.

The composed system inherits zero-knowledge from the outer layer. The final proof π_O proves knowledge of a valid inner proof π_I without revealing π_I itself. Since π_I depends on the witness w, hiding π_I suffices to hide w.

The inner SNARK's lack of ZK is encapsulated and hidden by the outer layer.

Recursion: Composing with Yourself

If composing two different SNARKs is useful, what about composing a SNARK with *itself*?

Shrinking the verifier tower

Take a hypothetical SNARK $\mathcal{S}$ where verifying a proof for a circuit of size N costs $O(\sqrt{N})$ operations. (This is pedagogical; real SNARKs have $O(1)$ verification like Groth16, or $O(\text{polylog } N)$ like STARKs. The $\sqrt{N}$ gives clean math for illustration.)

Now trace what happens when we recurse:

Layer 0: Prove the original circuit C (size N). This produces proof π_0. Verifying π_0 costs $O(\sqrt{N})$ operations.

Layer 1: Wrap π_0 in another proof. The circuit being proved is now the *verifier* for π_0, which has size $O(\sqrt{N})$. This produces π_1. Verifying π_1 costs $O(\sqrt{\sqrt{N}}) = O(N^{1/4})$ operations.

Layer 2: Wrap π_1. The circuit is the verifier for π_1, size $O(N^{1/4})$. Verifying π_2 costs $O(N^{1/8})$ operations.

The pattern: each layer proves "the previous verifier accepted," and since verifiers are smaller than the circuits they verify, each layer's circuit shrinks.

After k layers:

$$\text{Verifier cost for } \pi_k = O(N^{1/2^k})$$

Verification cost reaches a constant after $O(\log \log N)$ layers, which is the **recursion threshold**. The derivation is short: we need $N^{1/2^k} \leq c$ for some constant c, giving $2^k \geq \log N / \log c$, so $k = O(\log_2 \log N)$. Each layer halves the exponent; doing this $\log \log N$ times reduces it to a constant.

We are not proving the original circuit C over and over. Each layer proves a *different* (smaller) circuit: the verifier of the previous layer. The shrinking comes from the fact that verification is cheaper than computation.

Proof of Proof of Proof...

From the prover's perspective, deep recursion means building a tower of proofs:

1. π_1: proves "I know witness w satisfying circuit C"
2. π_2: proves "I know a valid proof π_1"
3. π_3: proves "I know a valid proof π_2"
4. Continue until the verifier circuit is minimal

Each π_i is a proof *about* the previous proof. The final π_k can be verified in constant time regardless of the original computation's size.

The Strange Loop

A proof that proves a proof that proves a proof: the structure feels like it should be paradoxical. Gödel showed that sufficiently powerful formal systems can express statements about their own provability, and this self-reference produces incompleteness. "This statement is unprovable" is a sentence the system can formulate but cannot resolve.

Recursive SNARKs avoid the trap because they ask a different question. Gödel's self-reference asks "is this *provable*?", a meta-logical assertion the system cannot settle about itself. Recursive SNARKs ask "is this *verifiable*?", and verification is a concrete, bounded computation: read the proof, check some equations, output accept or reject. A proof system can prove statements about its own verifier for the same reason it can prove statements about any other circuit. The self-reference leads not to paradox but to compression.

The Extraction Caveat

Everything above assumed recursive SNARKs are sound. They are, in practice. But the standard way of *proving* soundness breaks down with recursion depth, and understanding why reveals a genuine gap between what we can prove and what we believe.

The problem in one sentence: proving a SNARK is secure requires running the attacker many times to extract a witness, and each layer of recursion multiplies the number of runs exponentially. At depth k, the security proof requires R^k runs of the attacker, where R is the extraction cost per layer. For $k = 100$ and $R = 100$, this is 10^{200} operations, far beyond anything meaningful. The security theorem degrades to vacuity even though no one can actually break the system.

To see where this exponential comes from, we need to trace how SNARK security proofs work. We cannot prove a cryptographic system is secure in an absolute sense (that would require proving $P \neq NP$ and more). Instead, we prove *relative* security: "if someone can break system X, they can also break problem Y." If we believe Y is hard, then X must be hard too.

A SNARK security proof constructs an algorithm (the "reducer") that treats any successful attacker as a black box. If the attacker can forge proofs, the reducer extracts a valid witness from those proofs. The witness encodes a solution to a hard problem like discrete log, because the commitment scheme was constructed so that knowing a valid witness implies knowing a discrete log. Since we believe discrete log is hard, forging proofs must also be hard.

The extraction step is where the cost enters. To extract a witness, the reducer uses **rewinding**: run the prover once, record its state, then rewind to an earlier point and run it again with a different random challenge. Two runs with different challenges on the same commitment overdetermine the witness.

Worked example (rewinding in a Σ-protocol). Consider a Σ-protocol (Chapter 16) where the prover sends commitment a, receives challenge e, and responds with z. The extractor recovers the witness as follows:

1. Run the prover. It sends a, you send challenge e_1, it responds with z_1.
2. Rewind to just after the prover sent a. (In a proof, we model the prover as a stateful algorithm we can checkpoint and restore.)
3. Send a *different* challenge e_2.
4. The prover responds with z_2.
5. From (e_1, z_1) and (e_2, z_2) with the same a, algebraically solve for the witness.

This works because Σ-protocols have **special soundness** (Chapter 16): the commitment a fixes enough structure that two different challenge-response pairs overdetermine the witness. In Schnorr's protocol, for instance, $z = k + e \cdot x$ where x is the secret. Two transcripts give $z_1 - z_2 = (e_1 - e_2)x$, so $x = (z_1 - z_2)/(e_1 - e_2)$. Not all interactive proofs have this property, but Σ-protocols are designed for it.

The Σ-protocol example needed only $R = 2$ rewinds (two transcripts with different challenges). More complex SNARKs may need more: a protocol with k rounds of interaction generally requires $R = O(d^k)$ rewinds, where d is the degree of the round polynomials. For a modern SNARK, R might be in the hundreds. Recursion compounds this cost. For a single-layer proof, extraction costs R prover runs. For a 2-layer recursive proof, you must:

1. Extract the inner proof π_I from the outer layer: R runs
2. For *each* of those R runs, extract the witness from π_I: R more runs each
3. Total: $R \times R = R^2$ runs

For depth k: R^k runs. At depth 100 with $R = 100$, that's $100^{100} = 10^{200}$ operations.

This breaks the security proof. Security theorems have the form: "if an attacker breaks the SNARK, our reducer solves discrete log."

But the reducer must be *efficient*. If the reducer takes 10^{200} operations to extract a witness, the theorem becomes: "if an attacker breaks the SNARK, discrete log can be solved in 10^{200} operations." This is useless. We already know discrete log can be brute-forced in $2^{256} \approx 10^{77}$ operations. The reduction no longer rules out attackers, and the provable security level drops accordingly: each additional layer of recursion multiplies the reducer's running time by R, weakening the security guarantee by $\log_2 R$ bits per layer.

To be clear: this degradation is in the *provable bound*, not in the system's actual resistance to attack. More rewinds doesn't make the system easier to break. It makes our proof technique too slow to demonstrate security. The reducer's inefficiency is a problem for the theorist writing the proof, not for the attacker trying to exploit the system.

In practice, the system might be perfectly secure. No one has found attacks that exploit the recursive structure, and the underlying hard problems (discrete log, collision resistance) remain hard. What breaks is not the system but the *proof technique*: standard reductions become too expensive to carry through, so the security theorem degrades even though the system itself does not weaken.

The practical heuristic for recursive SNARKs is therefore: security degrades *on paper* with recursion depth, but not *in reality*. A system with 100 layers of recursion has the same concrete security as one with 2 layers (no known attack exploits the depth), but its provable security guarantee is weaker because the reduction's running time grows as R^k. This parallels the random oracle model, where hash functions are used in ways that resist all known attacks but lack full theoretical justification. Practitioners accept the gap and ship; researchers work on tighter proof techniques (folding schemes, discussed next, partly sidestep this issue by avoiding deep recursion entirely).

The Curve Cycle Problem

For pairing-based SNARKs like Groth16, recursion faces a fundamental obstacle: field mismatch.

Two Fields, One Problem

Every pairing-based SNARK involves two distinct fields. To understand why, recall how elliptic curve cryptography works.

An elliptic curve E is defined over a **base field** $\mathbb{F}_q$. Points on the curve have coordinates (x, y) where $x, y \in \mathbb{F}_q$. When you add two points or compute $k \cdot P$ (scalar multiplication), you're doing arithmetic in $\mathbb{F}_q$: additions, multiplications, and inversions of these coordinates.

But the *scalars* k live in a different field. The curve's points form a group under addition, and this group has an **order** p: the number of elements in the group. For any point P, we have $p \cdot P = \mathcal{O}$ (the identity). Scalars are integers modulo p, giving us the **scalar field** $\mathbb{F}_p$.

A concrete example with BN254 (the curve Ethereum uses for precompiles):

- Base field: $\mathbb{F}_q$ where $q \approx 2^{254}$ (coordinates of curve points live here; all curve arithmetic, including point addition and pairings, is computed over $\mathbb{F}_q$)
- Scalar field: $\mathbb{F}_p$ where $p \approx 2^{254}$ (a completely different prime of similar size; scalars in $k \cdot G$, witness values, and circuit constraints live here)
- A point on the curve: (x, y) with $x, y \in \mathbb{F}_q$
- A Groth16 proof element: $\pi_A = s \cdot G$ where $s \in \mathbb{F}_p$ (scalar field) and the result π_A is a curve point with coordinates in $\mathbb{F}_q$ (base field)

Where each field appears in Groth16:

- *Scalar field* $\mathbb{F}_p$: Your circuit's witness values and all constraint equations. If you're proving "I know x such that $x^3+x+5=35$," then $x \in \mathbb{F}_p$. The reason constraints must live in $\mathbb{F}_p$ is that the commitment scheme requires it. In Groth16, committing to a witness value s means computing $s \cdot G$ (a scalar multiplication on the curve). For this to be a well-defined group operation, s must be an element of $\mathbb{F}_p$, because the curve group has order p and scalars are taken modulo p. The constraint system inherits this field because it constrains the same values the commitment scheme binds.
- *Base field* $\mathbb{F}_q$: The proof elements themselves. The proof $\pi = (\pi_A, \pi_B, \pi_C)$ consists of elliptic curve points, which have coordinates in $\mathbb{F}_q$. Verification requires point additions and pairings, all computed over $\mathbb{F}_q$.

In a single proof, the two fields coexist without friction. The prover uses both (scalars from $\mathbb{F}_p$ feed the constraint system and commitment scheme; curve arithmetic in $\mathbb{F}_q$ constructs the proof elements), but the *constraint system* lives entirely in $\mathbb{F}_p$. The verifier checks the proof using $\mathbb{F}_q$ operations (pairings, point arithmetic), but that happens outside any circuit. Recursion forces the two fields into the same circuit. When the inner proof $\pi = (\pi_A, \pi_B, \pi_C)$ becomes the witness of the outer circuit, the outer circuit must *verify* π, which means performing pairing checks and point arithmetic on the coordinates of π_A, π_B, π_C. Those coordinates are $\mathbb{F}_q$ elements. But the outer circuit's constraints are polynomial equations over $\mathbb{F}_p$ (the scalar field of whatever curve the outer system uses). The outer circuit must therefore manipulate $\mathbb{F}_q$ values using $\mathbb{F}_p$ arithmetic.

To do $\mathbb{F}_q$ arithmetic inside an $\mathbb{F}_p$ circuit, you must *emulate* it: decompose each $\mathbb{F}_q$ element into multiple $\mathbb{F}_p$-sized limbs and implement multiplication with schoolbook carry propagation, as Chapter 13 described for non-native arithmetic in PLONK. A single $\mathbb{F}_q$ multiplication can cost 50-100+ $\mathbb{F}_p$ constraints. The verifier circuit explodes from a few thousand native operations to hundreds of thousands of emulated ones.

Two terms clarify this distinction. When we say arithmetic is *native*, we mean it's cheap inside the circuit: one field operation becomes one constraint. A circuit over $\mathbb{F}_p$ can do $\mathbb{F}_p$ arithmetic natively. It must *emulate* $\mathbb{F}_q$ arithmetic, paying 100+ constraints per operation. The curve cycle trick ensures we're always doing native arithmetic by switching fields at every recursive step.

Cycles of Curves

For single composition, the fix is straightforward: choose an outer curve whose scalar field matches the inner curve's base field. If the inner verifier does $\mathbb{F}_q$ arithmetic, use an outer system over $\mathbb{F}_q$. One wrap, native arithmetic, done.

For deep recursion, this isn't enough. After wrapping once, you have a new proof whose verifier does arithmetic in some *other* field. To wrap again natively, you need yet another matching curve.

The solution is a **cycle of elliptic curves** (E_1, E_2):

- E_1 has scalar field $\mathbb{F}_p$ and base field $\mathbb{F}_q$
- E_2 has scalar field $\mathbb{F}_q$ and base field $\mathbb{F}_p$

The fields swap roles between curves. Recursion alternates:

- Proof π_1 on curve E_1: verifier performs $\mathbb{F}_q$ arithmetic
- Proof π_2 on curve E_2: verifier performs $\mathbb{F}_p$ arithmetic, can *natively* prove about π_1's verification
- Proof π_3 on curve E_1: can natively prove about π_2's verification
- And so on indefinitely

Each step's verifier circuit uses native field arithmetic. The alternation continues as long as needed, with no expensive cross-field emulation at any layer.

Practical Curve Cycles

Pasta curves (Pallas and Vesta): A true cycle, meaning the scalar field of each equals the base field of the other. This enables unbounded recursion: the prover alternates between the two curves at every step, and each step's verifier circuit uses native arithmetic because the next curve's scalar field matches the previous curve's base field. Neither curve is pairing-friendly, but both support efficient Pedersen commitments and inner-product arguments. Used in Halo 2 and related systems.

BN254 / Grumpkin: Also a true cycle (Grumpkin is obtained by swapping BN254's base and scalar fields), enabling the same unbounded alternating recursion as Pasta. The difference is asymmetry: BN254 is pairing-friendly while Grumpkin is not, so the cycle alternates between pairing-based proofs (on the BN254 side) and inner-product-based proofs (on the Grumpkin side). Since BN254 has Ethereum precompiles, proofs that land on the BN254 side can be verified cheaply on-chain. Aztec uses this cycle for their rollup architecture.

The choice between cycles depends on the deployment target. BN254/Grumpkin is the default when on-chain Ethereum verification matters, because BN254 has EVM precompiles and pairings enable constant-size final proofs (Groth16-style). The cost is a trusted setup on the BN254 side. Pasta is preferred when transparency matters (IPA on both sides, no trusted setup), as in Zcash's Orchard protocol via Halo 2, but lacks EVM support and produces larger proofs.

BLS12-377 / BW6-761: Both curves are pairing-friendly, giving pairings on *both* sides of the recursion (unlike BN254/Grumpkin where only one side has pairings). BW6-761's scalar field matches BLS12-377's base field, allowing native verification of BLS12-377 proofs. The pair is called a "half-cycle" because the match goes in one direction only (BW6-761 can natively verify BLS12-377 proofs, but not vice versa), so it supports efficient one-step composition rather than unbounded alternating recursion. Aleo uses this pair for their proof system.

A related curiosity: embedded curves. BabyJubjub is defined over BN254's scalar field $\mathbb{F}_p$, so BabyJubjub point operations can be expressed natively as BN254 circuit constraints. This enables in-circuit cryptography: EdDSA signatures, Pedersen hashes, and other EC-based primitives. BabyJubjub doesn't form a cycle with BN254 (its group order is much smaller than BN254's base field), so it cannot be used for recursion. The reader might wonder why Grumpkin doesn't replace BabyJubjub entirely, since both have their base field equal to BN254's scalar field. In principle it could: any curve with the right base field supports native in-circuit arithmetic. The reason BabyJubjub persists is practical. It is a twisted Edwards curve with complete addition formulas (no special cases for doubling or identity), making in-circuit point addition slightly cheaper than on Grumpkin (a short Weierstrass curve). It also predates Grumpkin and has years of deployed implementations and audited libraries. In a greenfield design you could use Grumpkin for both roles; in existing ecosystems the two curves coexist because they were optimized for different concerns.

Finding curve cycles is mathematically delicate. The size constraints (both fields must be large primes), security requirements (curves must resist known attacks), and efficiency demands (curves should have fast arithmetic) severely restrict the design space.

Incrementally Verifiable Computation (IVC)

Composition combines different proof systems; the recursion we've seen so far compresses proofs through towers of wrapping. But there is a different problem that recursion solves, one that isn't about shrinking proofs or mixing systems: proving computation that hasn't finished yet.

A blockchain processes transactions one by one. A verifiable delay function (VDF) computes a hash chain for hours, proving that real time elapsed. A zkVM executes a program instruction by instruction. In each case, the computation is *sequential*: step i depends on step $i-1$. You can't parallelize it. You can't wait until the end to start proving (the end might be days away, or never).

What you want is a proof that *grows* with the computation. After step 1, you have a proof of step 1. After step 1000, you have a proof of steps 1 through 1000. The proof at step 1000 shouldn't be 1000× larger than the proof at step 1. And creating the proof for step 1000 shouldn't require re-proving steps 1 through 999.

This is **incrementally verifiable computation**, or **IVC**: proofs that extend cheaply, verify in constant time, and accumulate the history of an unbounded sequential process. The term appears throughout the literature; systems like Nova, SuperNova, and ProtoStar are "IVC schemes."

The Setting

Consider a function $F : X \to X$ iterated T times:

$$y_T = F(F(\cdots F(x_0)\cdots)) = F^T(x_0)$$

For $T = 10^9$ iterations, directly proving this requires a circuit of size $O(T \cdot |F|)$: billions of gates. Even fast provers choke on circuits this large. And you'd have to wait until iteration 10^9 completes before generating any proof at all.

The Incremental Approach

Generate proofs *incrementally*, one step at a time:

- π_0: trivial (base case, no computation yet)
- π_1: proves "$y_1 = F(x_0)$ and I know a valid π_0"
- π_2: proves "$y_2 = F(y_1)$ and I know a valid π_1"
- π_i: proves "$y_i = F(y_{i-1})$ and I know a valid π_{i-1}"

Each π_i has constant size and proves the *entire* computation from x_0 to y_i. The proof for step i subsumes all previous proofs.

The Recursive Circuit

At step i, the prover runs a circuit that:

1. **Verifies π_{i-1}:** Checks that the previous proof is valid
2. **Computes $y_i = F(y_{i-1})$:** Performs one step of the function
3. **Produces π_i:** Outputs a new proof

The circuit size is $|V| + |F|$: the cost of verifying the previous proof plus the cost of one function evaluation. The overhead of recursion is $|V|$, the verifier circuit size.

For a SNARK with efficient verification, $|V|$ might be a few thousand gates. If $|F|$ is also a few thousand gates (a hash function, say), the overhead roughly doubles the per-step cost. For larger $|F|$, the overhead is proportionally smaller.

Where IVC Shines

Verifiable Delay Functions (VDFs). The canonical example: repeated squaring $x \mapsto x^2 \mod N$ for an RSA modulus N. Each squaring depends on the previous result; you can't compute x^{2^T} faster than T sequential multiplications (without knowing the factorization of N). After computing $y = F^T(x)$,

the prover produces a proof that y is correct, verifiable in time much less than T. IVC is natural here: the function is inherently sequential, and the proof accumulates with each step.

Succinct Blockchains. Each block contains a proof that:

1. This block's transactions are valid
2. The previous block's proof was valid

A new node syncing to the chain verifies a *single* proof (the most recent one) rather than replaying every transaction since genesis. Mina Protocol pioneered this approach.

Proof Aggregation. Multiple independent provers generate T separate proofs. An aggregator combines them into one proof via recursive composition. Batch verification becomes constant-time regardless of the number of original proofs.

Folding Schemes: Cheaper IVC

The IVC construction above requires the prover to *fully verify* the previous proof π_{i-1} at every step. This verification is itself a circuit (C_V), and it can be large. If the verifier circuit has $|V| = 10{,}000$ gates and the step function has $|F| = 1{,}000$ gates, the prover spends 90% of each step proving it verified correctly, only 10% proving it computed correctly. For tiny F (say, a single hash invocation), the ratio gets even worse. The verification overhead, not the computation itself, becomes the IVC bottleneck.

Folding schemes address this by replacing verification with something cheaper.

Folding instead of verifying

Instead of *fully verifying* π_{i-1} at step i, we **fold** the claim about step $i-1$ with the claim about step i. Folding combines two claims into one claim of the same structure, without verifying either. The accumulated claim grows no larger than each individual claim. Verification is deferred entirely: the prover folds at every step (a few group operations each) and produces a single real SNARK proof only at the very end. The cost of folding per step is drastically cheaper than the cost of in-circuit verification, which is what makes IVC practical for millions of steps.

Nova's Approach

Nova, the pioneering folding scheme (Kothapalli, Setty, Tzialla, 2021), introduced a modified constraint system: **relaxed R1CS**. Recall from Chapter 19's Spartan section that standard R1CS demands:

$$(A \cdot z) \circ (B \cdot z) = C \cdot z$$

where A, B, C are sparse matrices and $\circ$ is the entrywise (Hadamard) product. This is the same equation Spartan proves via sum-check. Nova adds two slack terms to make folding possible:

$$(A \cdot z) \circ (B \cdot z) = u \cdot (C \cdot z) + E$$

where u is a scalar and E is an "error vector." A standard R1CS instance has $u = 1$ and $E = 0$. Relaxed instances can have $u \neq 1$ and $E \neq 0$, but satisfying a relaxed instance still proves something about the underlying computation.

Why Relaxation Enables Folding

What makes relaxed R1CS useful is that instances can be *linearly combined*. Suppose we want to fold two instances by taking a random linear combination with challenge r:

$$z = z_1 + r \cdot z_2$$

This is the core of folding: two separate witnesses z_1 and z_2 become a single witness z. The folded witness has the same dimension as the originals; we're not concatenating, we're combining. Think of it geometrically: z_1 and z_2 are points in $\mathbb{F}^n$; the fold z is another point on the line through them, selected by the random challenge r.

What happens when we plug this combined witness into the constraint? Let's compute $(Az) \circ (Bz)$:

$$\begin{aligned}
&(A(z_1 + rz_2)) \circ (B(z_1 + rz_2)) \\
&= (Az_1 + r \cdot Az_2) \circ (Bz_1 + r \cdot Bz_2) \\
&= (Az_1 \circ Bz_1) + r \cdot (Az_1 \circ Bz_2 + Az_2 \circ Bz_1) + r^2 \cdot (Az_2 \circ Bz_2)
\end{aligned}$$

The Hadamard product distributes, but it creates **cross-terms**: the middle expression $Az_1 \circ Bz_2 + Az_2 \circ Bz_1$ mixes the two instances. This is the "interaction" between them.

For standard R1CS, these cross-terms would break everything: the equation $Az \circ Bz = Cz$ has no room for interaction terms that belong to neither instance. This is exactly why relaxation was introduced. The error vector E acts as a "trash can" that absorbs the cross-terms, keeping the equation valid despite the algebraic mess that linear combination creates. Define:

$$T = Az_1 \circ Bz_2 + Az_2 \circ Bz_1 - u_1 \cdot Cz_2 - u_2 \cdot Cz_1$$

This is the **cross-term vector**. To understand where each term comes from, recall that each relaxed R1CS instance has its own slack factor: instance 1 has (z_1, E_1, u_1) and instance 2 has (z_2, E_2, u_2). When we expand the *right side* of the relaxed constraint $u \cdot (Cz)$ using the folded values $u = u_1 + ru_2$ and $z = z_1 + rz_2$:

$$(u_1 + ru_2) \cdot C(z_1 + rz_2) = u_1 Cz_1 + r(u_1 Cz_2 + u_2 Cz_1) + r^2 \cdot u_2 Cz_2$$

The coefficient of r on the right side is $u_1 Cz_2 + u_2 Cz_1$. On the left side (the Hadamard product expansion above), the coefficient of r is $Az_1 \circ Bz_2 + Az_2 \circ Bz_1$. The cross-term T is exactly the difference between these: what the left side produces at r minus what the right side produces at r. This mismatch gets absorbed into the error vector.

Note that the r^2 coefficient works out automatically: the left side gives $Az_2 \circ Bz_2$ and the right side gives $u_2 Cz_2$, which is exactly instance 2's original constraint (up to E_2). The folded error $E = E_1 + rT + r^2 E_2$ absorbs the second instance's error at r^2.

The Folding Protocol

A relaxed R1CS instance consists of a witness vector z, an error vector E, and a slack scalar u. A fresh (non-folded) instance has $u = 1$ and $E = 0$; after folding, both accumulate non-trivial values.

Given two instances (z_1, E_1, u_1) and (z_2, E_2, u_2), the protocol folds them into one. But the verifier doesn't see the actual witness vectors, since that would defeat the point. Instead, the verifier works with **commitments**.

What the verifier holds: Commitments C_{z_1}, C_{z_2} to the witness vectors, commitments C_{E_1}, C_{E_2} to the error vectors, and the public scalars u_1, u_2. (Public inputs are also visible, but we omit them for clarity.)

The protocol:

1. **Prover computes the cross-term T:** The formula above requires knowing both witnesses, so only the prover can compute it.
2. **Prover commits to T:** Sends commitment C_T to the verifier. This is the only new cryptographic operation per fold.
3. **Verifier sends random challenge r.**
4. **Both compute the folded instance using commitments:**
   - $C_z = C_{z_1} + r \cdot C_{z_2}$ (the verifier computes this from the commitments)
   - $u = u_1 + r \cdot u_2$ (public scalars, both can compute)

- $C_E = C_{E_1} + r \cdot C_T + r^2 \cdot C_{E_2}$ (again from commitments)

The verifier never sees z_1, z_2, E_1, E_2, or T directly. They work entirely with commitments. Because commitments are additively homomorphic (Pedersen commitments satisfy $C(a)+C(b) = C(a+b)$), the folded commitment C_z is a valid commitment to the folded witness $z = z_1 + r \cdot z_2$, which only the prover knows.

Meanwhile, the prover computes the actual folded witness $z = z_1 + r \cdot z_2$ and the actual folded error $E = E_1 + r \cdot T + r^2 \cdot E_2$. The prover holds these for the next fold (or for the final SNARK).

The folded error vector absorbs the cross-terms at the r coefficient and the second instance's error at the r^2 coefficient. This is exactly what makes the constraint hold: the expansion of $(Az) \circ (Bz)$ produces terms at powers 1, r, and r^2, and the folded E and $u \cdot Cz$ absorb them all.

Expanding $(Az) \circ (Bz) - u \cdot Cz - E$ using the folded values shows why this is sound: all terms cancel if and only if both original instances satisfied their constraints. The random r acts as a Schwartz-Zippel check: a cheating prover who folds two *unsatisfied* instances would need the folded instance to satisfy the constraint, but this happens with negligible probability over random r.

Two claims have become one, without verifying either. The prover paid the cost of one commitment (to T) and some field operations. No expensive SNARK proving.

IVC with Folding

Now we connect the folding protocol to the IVC setting from earlier in the chapter. Recall the problem: prove $y_T = F^T(x_0)$ for large T without circuits that grow with T.

The folding protocol combines two relaxed R1CS instances into one. For IVC, we maintain a **running instance** that accumulates all previous steps and fold in each new step as it happens.

What gets folded: At each step, we have two things:

- The **running instance** $(C_{z_{acc}}, C_{E_{acc}}, u_{acc})$, where $C_{z_{acc}}$ is the Pedersen commitment to the accumulated witness, $C_{E_{acc}}$ is the commitment to the accumulated error vector, and u_{acc} is the accumulated scalar. Together these represent "all steps so far are correct."
- The **step instance** $(C_{z_i}, C_{E_i} = C_0, u_i = 1)$, a fresh claim that "step i was computed correctly." It is always a standard (non-relaxed) R1CS instance: $u_i = 1$ and $E_i = 0$.

The step instance is always fresh: $u_i = 1$ and $E_i = 0$ because it comes from a standard (non-relaxed) R1CS. Only the running instance accumulates non-trivial slack.

The IVC loop in detail:

Step 0 (Base case): Initialize the running instance to a trivial satisfiable state. No computation yet.

Step i (for $i = 1, 2, \ldots, T$):

1. **Compute:** Execute $y_i = F(y_{i-1})$
2. **Create the step instance:** Express "$y_i = F(y_{i-1})$" as an R1CS constraint. Build the witness vector z_i (the same object as z_1 or z_2 in the folding derivation above), encoding the input y_{i-1}, output y_i, and any intermediate values the step function produces. Commit to get C_{z_i}.
3. **Fold:** The two inputs to the folding protocol are the running instance $(C_{z_{acc}}, C_{E_{acc}}, u_{acc})$ from the previous iteration and the step instance $(C_{z_i}, C_0, 1)$ just created. This is exactly the two-instance fold from the derivation above, with the running instance playing the role of instance 1 and the step instance playing instance 2:
   - Prover computes the cross-term T_i (from the two witnesses z_{acc} and z_i) and commits to get C_{T_i}
   - Challenge r_i is derived via Fiat-Shamir from the transcript
   - Both parties compute the new running instance using the homomorphic update:
     - $C_{z_{acc}} \leftarrow C_{z_{acc}} + r_i \cdot C_{z_i}$ (fold the witnesses)
     - $u_{acc} \leftarrow u_{acc} + r_i \cdot 1$ (fold the scalars)
     - $C_{E_{acc}} \leftarrow C_{E_{acc}} + r_i \cdot C_{T_i} + r_i^2 \cdot C_0$ (absorb the cross-term)
   - The error update is where the cross-terms go. Recall from the Hadamard expansion that folding $z_{acc} + r_i \cdot z_i$ produces a degree-1 cross-term T_i in r_i. The $r_i \cdot C_{T_i}$ term absorbs it into the error commitment; the $r_i^2 \cdot C_0$ term accounts for the step instance's error (which is zero for a fresh instance, so this term vanishes in practice).
   - Prover updates the actual witnesses (which only the prover holds): $z_{acc} \leftarrow z_{acc} + r_i \cdot z_i$, $E_{acc} \leftarrow E_{acc} + r_i \cdot T_i$
4. **Repeat:** The new running instance becomes input to step $i + 1$

After T steps: The prover holds a final running instance $(C_{z_{acc}}, C_{E_{acc}}, u_{acc})$ with $u_{acc} = 1 + r_1 + r_1 r_2 + \cdots$ (accumulated from all the folds). This single instance encodes the claim "all T steps were computed correctly." The T individual steps, their witnesses, and their cross-terms have all been absorbed into the running instance's commitments and scalar. No trace of the intermediate states remains.

The final SNARK: The prover now produces one conventional SNARK proof demonstrating that the running instance is satisfiable, i.e., that there exists a witness z_{acc} and error vector E_{acc} such that:

$$(Az_{acc}) \circ (Bz_{acc}) = u_{acc} \cdot (Cz_{acc}) + E_{acc}$$

This is the only expensive cryptographic operation in the entire protocol. The preceding T folds

each cost only a few group operations (one commitment to T_i, one scalar multiplication per running-instance component). The full SNARK proof happens once, at the end, not at every step.

The verifier's job is correspondingly simple. It receives three things: the final running instance $(C_{z_{acc}}, C_{E_{acc}}, u_{acc})$, the SNARK proof for that instance, and the claimed output y_T. It verifies the SNARK (confirming the running instance is satisfiable) and checks that y_T matches the output encoded in the instance. If both pass, the verifier is convinced that $y_T = F^T(x_0)$. It never sees any of the T intermediate states, witnesses, or fold challenges. The entire computation history has been compressed into one relaxed R1CS instance and one proof.

Security Considerations for Folding

Folding schemes have a reputation in the zkVM community for being where security problems arise. This isn't accidental; the architecture creates several subtle attack surfaces.

Deferred verification. Traditional recursion verifies at each step: if something is wrong, you catch it immediately. Folding defers *all* verification to the final SNARK. Errors compound silently across thousands of folds before manifesting. Debugging becomes archaeology, trying to identify which of 10,000 folds went wrong.

The commitment to T must be binding. The cross-term T must be committed *before* the verifier sends challenge r. If the prover can open this commitment to different values after seeing r, soundness breaks completely: the prover can fold unsatisfied instances and make them appear satisfied. Nova uses Pedersen commitments (computationally binding under discrete log), so breaking the binding property would require solving discrete log. But implementation bugs in commitment handling have caused real vulnerabilities.

Accumulator state is prover-controlled. Between folding steps, the prover holds the running accumulated instance $(z_{acc}, E_{acc}, u_{acc})$. The final SNARK proves this accumulated instance is satisfiable, but doesn't directly verify it came from honest folding. A malicious prover who can inject a satisfiable-but-fake accumulated instance breaks the chain of trust. The "decider" circuit must carefully check that public inputs match the accumulator state.

Soundness error accumulates. Each fold uses a random challenge r to combine two instances. A cheating prover escapes detection only if the degree-d cross-term identity accidentally holds at that r, which Schwartz-Zippel bounds at probability $\leq d/|\mathbb{F}|$ per fold. Over T independent folds, a union bound gives total soundness error $\leq T \cdot d/|\mathbb{F}|$. For $T = 10^6$ folds over a 256-bit field with $d = 2$, this is $\approx 2^{-236}$, negligible. But for smaller fields or exotic parameters, verify the concrete security.

Implementation complexity. Folding has more moving parts than traditional recursion: cross-term computation, accumulator updates, commitment bookkeeping, the interaction between folding and

the final decider SNARK. Each is a potential bug location. Several folding implementations have had soundness bugs discovered post-audit. The abstraction is elegant, but the implementation details are unforgiving.

Post-quantum vulnerability. Every folding scheme discussed in this chapter (Nova, HyperNova, ProtoStar) uses Pedersen commitments for the cross-term T and the accumulated witness. Pedersen binding relies on discrete log, which Shor's algorithm breaks. A quantum attacker who can open commitments to different values after seeing the folding challenge r destroys soundness entirely, since they can fold unsatisfied instances and make them appear satisfied. The inner folding loop is therefore not post-quantum safe. The same vulnerability appears in composition: the STARK→Groth16 wrapper that production zkVMs use (the hybrid row in the compatibility table below) reintroduces pairing assumptions at the final step, making the on-chain proof quantum-vulnerable even though the inner STARK is hash-based. Lattice-based folding schemes (LatticeFold, Neo, SuperNeo) replace Pedersen with Module-SIS commitments to address this, though none are in production as of this writing. Hash-based full recursive verification (STARK proving a STARK verifier) avoids the problem entirely but at higher per-step cost.

None of this means folding is insecure against classical adversaries. It means the security argument is more delicate than "run a SNARK at each step." The efficiency gains are real, but so is the need for careful implementation, thorough auditing, and awareness of the quantum horizon.

Folding and the PIOP paradigms

Folding schemes operate at a different level of the proof-system stack than PIOPs or PCS:

1. **Constraint system**: R1CS, Plonkish, CCS, AIR
2. **PIOP paradigm**: How you prove constraints (quotienting or sum-check)
3. **Recursion strategy**: How you chain proofs (full verification, folding, accumulation)

Nova's folding operates at level 3. It takes R1CS instances and folds them algebraically, without committing to a specific PIOP for the final proof. Folding originated in the sum-check lineage (Nova came from the Spartan team, and relaxed R1CS fits naturally with multilinear machinery), but it is no longer confined to it. The "Beyond Nova" section below develops how folding has generalized to CCS, Plonkish, and pairing-based systems, and a compatibility table after that section maps the full landscape of which components pair naturally across all three levels.

Folding versus traditional recursion

Aspect	Traditional Recursion	Folding (Nova)
Per-step overhead	Full SNARK verification	Two group operations
Curves needed	Pairing-friendly or cycle	Any curve works
Final proof	Proves last recursive step	Proves folded instance
Prover bottleneck	Verification overhead	Actual computation F

For small F (hash function evaluations, state machine transitions), folding is an order of magnitude faster than traditional recursion. The per-step cost drops from thousands of gates to tens of operations.

Beyond Nova: HyperNova and ProtoStar

Nova achieves cheap IVC for R1CS. R1CS is fully general (any NP computation can be expressed, as Spartan demonstrated in Chapter 19), but its degree-2 restriction means that higher-degree operations must be decomposed into auxiliary variables. Encoding Poseidon's x^5 S-box, for example, requires intermediate squaring steps that inflate the constraint count. Custom gates (which handle common operations in fewer constraints) and higher-degree constraints (which pack more logic per gate) can reduce this overhead substantially. Lookups and structured access patterns are a separate concern: R1CS-based systems handle these efficiently through memory checking (Chapter 21's Lasso), not through the constraint system itself. The motivation for CCS is primarily constraint degree and gate flexibility, not lookup support. The question is whether folding can extend to these richer constraint representations without losing Nova's per-step efficiency.

The CCS Abstraction

The answer starts with a common language for constraints. **Customizable Constraint Systems (CCS)**, introduced in Chapter 8, unify R1CS, Plonkish, and AIR under one framework. As a reminder, the core equation is:

$$\sum_{j=1}^{q} c_j \cdot \bigcirc_{i \in S_j} (M_i \cdot z) = \mathbf{0}$$

Each term j takes a Hadamard product ($\bigcirc$) over the matrix-vector products $M_i \cdot z$ for matrices in multiset S_j, then scales by coefficient c_j. The multiset sizes determine constraint degree: R1CS uses $|S| = 2$ (degree 2), higher-degree gates use larger multisets, linear constraints use $|S| = 1$.

CCS matters for folding because it gives the scheme designer one interface to target. HyperNova folds CCS instances directly, so any constraint system expressible as CCS, which includes R1CS, Plonkish, and AIR, inherits folding automatically. You can fold circuits written in different constraint languages without converting to a common format first. The abstraction pays for itself when you want custom gates, higher-degree constraints, or mixed constraint types within a single IVC computation.

HyperNova: Folding CCS

HyperNova extends Nova's folding approach to CCS, but the generalization isn't straightforward. The degree problem that Nova sidestepped returns with a vengeance.

The degree problem. Recall Nova's cross-term: when folding $z = z_1 + r \cdot z_2$ into a degree-2 constraint, the expansion produces terms at r^0, r^1, and r^2. The error vector E absorbs the cross-term at r^1.

For a degree-d constraint, folding $z = z_1 + r \cdot z_2$ produces terms at powers $r^0, r^1, \ldots, r^d$. Each intermediate power $r^1, \ldots, r^{d-1}$ generates a cross-term that must be absorbed. Naive relaxation requires $d - 1$ error vectors, each requiring a commitment. The prover cost scales with degree.

HyperNova avoids this by observing that if one of the two instances is already *linear* (degree 1), then the cross-terms don't explode. Folding a linear instance with a degree-d instance produces at most degree d, with manageable cross-terms.

LCCCS (Linearized CCS). HyperNova converts an accumulated CCS instance into a different form before folding. Recall from above that a CCS constraint is a *vector* equation: all m entries of $\sum c_j \cdot \bigcirc_{i \in S_j}(M_i \cdot z)$ must equal zero. The "linearized" version collapses this to a *scalar* equation by taking a random linear combination of all m constraints. Given random $r \in \mathbb{F}^{\log m}$, weight each constraint by $\widetilde{\mathsf{eq}}(r, k)$ (the multilinear extension of the equality predicate from Chapter 4):

$$\sum_{k \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(r, k) \cdot \left(\sum_j c_j \cdot \bigcirc_{i \in S_j}(M_i \cdot z)_k \right) = 0$$

By Schwartz-Zippel, if any entry of the original vector is non-zero, this scalar equation fails with high probability over random r. This is the standard "batch to a single equation" trick.

The resulting scalar can be expressed in terms of multilinear extension evaluations: $\tilde{M}_i(r)$ is the MLE of $M_i \cdot z$ evaluated at r. The witness z now appears only through these evaluation claims, which sum-check can reduce to polynomial openings.

Why call this "linearized"? The term refers to how the *folding* works, not the constraint degree. When folding an LCCCS (which is a scalar evaluation claim) with a fresh CCS instance (a vector constraint), the interaction between them produces manageable cross-terms. The scalar form of LCCCS means folding doesn't multiply the number of error terms the way naive CCS folding would.

HyperNova therefore folds *asymmetrically*: the two instances being combined have different shapes, and this mismatch is what prevents the cross-term explosion that would occur if both were high-degree.

To see the contrast, recall that Nova folds two things of the *same* shape: relaxed R1CS instance + relaxed R1CS instance → relaxed R1CS instance. Both are degree-2, and the error vector absorbs the single degree-1 cross-term.

HyperNova folds two things of *different* shapes:

- **Running instance (LCCCS):** A scalar claim about polynomial evaluations
- **Fresh instance (CCS):** A vector constraint over m entries

You're not combining "vector + vector → vector." You're combining "scalar + vector → scalar." This asymmetry is what prevents cross-term explosion.

Sum-check is what bridges the two shapes. It takes a claim about a sum (the CCS vector constraint, batched into a scalar) and reduces it to an evaluation claim at a random point. After sum-check, both the running LCCCS and the fresh CCS have been reduced to evaluation claims at the *same* random point. These scalar claims can be linearly combined without degree blowup.

The loop:

1. **Running LCCCS:** A scalar claim "$\sum_j c_j \prod_{i \in S_j} \tilde{M}_i(r_{old}) = v_{old}$"
2. **Fresh CCS arrives:** A vector constraint that must hold at all m positions
3. **Sum-check:** Batch the CCS into a scalar claim at a new random point r_{new}, then combine with the LCCCS
4. **Result:** A new scalar claim at r_{new}, another LCCCS ready for the next fold

The sum-check rounds are the cost of generality: $O(\log m)$ rounds of interaction (or Fiat-Shamir hashing). But once sum-check finishes, combining the evaluation claims needs only one multi-scalar multiplication, the same per-fold cost as Nova regardless of constraint degree.

In Nova, the error vector E absorbs degree-2 cross-terms algebraically. In HyperNova, sum-check absorbs arbitrary-degree cross-terms interactively. Different mechanisms, same goal: constant prover cost per fold.

Additional benefits:

- *Multi-instance folding*: Fold k instances simultaneously by running sum-check over all k at once. The cost is $O(\log k)$ additional sum-check rounds. This enables efficient PCD (proof-carrying data), where proofs from multiple sources combine into one.
- *Zero-knowledge requires additional work*: Sum-check round polynomials leak witness information (their coefficients are linear combinations of witness entries). Neither Nova nor HyperNova

is zero-knowledge out of the box. The BlindFold technique (discussed later in this chapter) addresses this by committing to round polynomial coefficients and deferring their verification via folding.

ProtoStar: Accumulation for Special-Sound Protocols

ProtoStar takes a different generalization path. Rather than targeting a specific constraint system (as Nova targets R1CS and HyperNova targets CCS), it provides accumulation for any *special-sound interactive protocol*, regardless of how many messages the prover and verifier exchange. Sigma protocols (Chapter 16) are the simplest case: 3 messages, special soundness from two transcripts. But many proof components (sum-check rounds, polynomial evaluation arguments, lookup arguments) are also special-sound protocols with more rounds. ProtoStar accumulates them all under one framework.

Why special-soundness enables accumulation. A special-sound protocol has a key property: the verifier's check is a low-degree polynomial equation $V(x, \pi, r) = 0$, where x is the public input, π is the prover's messages, and r is the verifier's challenge. The degree d of V in r is typically small (often 1 or 2).

This algebraic structure is exactly what folding exploits. Given two protocol instances with the same structure, you can take a random linear combination:

$$V_{acc}(x, \pi, r) = V_1(x_1, \pi_1, r) + \beta \cdot V_2(x_2, \pi_2, r)$$

If both $V_1 = 0$ and $V_2 = 0$, then $V_{acc} = 0$ for any β. If either is non-zero, $V_{acc} = 0$ with probability at most $d/|\mathbb{F}|$ over random β. The accumulated check is equivalent to both original checks, with negligible soundness loss.

The cost difference. ProtoStar's per-fold cost is roughly 3 scalar multiplications, compared to 1 MSM for Nova/HyperNova (the comparison table below summarizes this). This reflects different trade-offs:

- Nova/HyperNova commit to the cross-term T (or run sum-check), requiring one multi-scalar multiplication per fold
- ProtoStar works directly with the protocol's algebraic structure, avoiding new commitments but requiring the prover to compute and send $d-1$ "error polynomials" that capture the cross-terms

For degree-2 checks (like most Σ-protocols), this means a few scalar multiplications instead of an MSM. The MSM dominates for large witnesses, so ProtoStar can be faster when the step function is small.

Lookup support. ProtoStar handles lookup arguments with overhead $O(d)$ in the lookup table size, compared to $O(d \log N)$ for HyperNova. The difference: HyperNova encodes lookups via sum-check over the table, adding $\log N$ rounds. ProtoStar accumulates the lookup protocol directly, paying only for the protocol's native degree. For applications with large tables (memory, range checks), this matters.

ProtoGalaxy. A refinement of ProtoStar that reduces the recursive verifier's work further. The key observation: when folding k instances, naive accumulation requires $O(k)$ verifier work. ProtoGalaxy uses a Lagrange-basis trick to compress this to $O(\log k)$ field operations plus a constant number of hash evaluations. For multi-instance aggregation (combining proofs from many sources), ProtoGalaxy approaches the minimal possible overhead.

Comparison

Feature	Nova	HyperNova	ProtoStar
Constraint system	R1CS only	Any CCS (R1CS, Plonk, AIR)	Any special-sound protocol
Constraint degree	2	Arbitrary	Arbitrary
Per-step prover cost	1 MSM	1 MSM	3 scalar muls
Lookup support	Via R1CS encoding	$O(d \log N)$	$O(d)$
Zero-knowledge	Requires blinding	Free from folding	Requires blinding
Multi-instance	Sequential only	Native support	Native support

When to Use What: A Practitioner's Guide

The progression from Nova to HyperNova to ProtoStar isn't a simple linear improvement. Each occupies a different point in the design space, and the "best" choice depends on your bottleneck.

The deciding factor is where the prover spends its time. Decompose total proving cost into two parts:

- **Step cost** $|F|$**:** Proving one iteration of your function (one hash, one VM instruction, one state transition)
- **Accumulation overhead** $|V|$**:** The cost of folding/recursing that step into the running proof

For traditional IVC (recursive SNARKs), $|V|$ is the verifier circuit size, typically thousands to tens of thousands of constraints. For folding, $|V|$ drops to a handful of group operations. The ratio $|F|/|V|$ determines whether folding helps.

Folding wins when $|F|$ is small:

- VDFs (repeated squaring): $|F| \approx$ a few hundred constraints per square
- Simple state machines: $|F| \approx$ hundreds to low thousands
- Hash chain proofs: $|F| \approx$ constraint count of one hash invocation

In these cases, traditional IVC spends most of its time proving the *verifier*, not the *computation*. Folding eliminates this overhead almost entirely.

Folding's advantage shrinks when $|F|$ is large:

- zkVM instruction execution: $|F| \approx 10,000$ to $100,000$ constraints per instruction
- Complex smart contract proofs: $|F|$ dominates regardless
- Batch proofs of many operations: amortization across the batch matters more than per-step overhead

When $|F| \gg |V|$, the prover spends 95%+ of time on the step function whether using folding or traditional IVC. Folding's 100× reduction in $|V|$ becomes a 5% improvement in total cost.

Engineering maturity matters beyond raw performance:

- *Folding schemes are newer.* Less battle-tested, fewer audits, more subtle security pitfalls.
- *AIR/STARK tooling is mature.* Well-understood compilation, debugging, and optimization paths.
- *Folding debugging is harder.* Errors compound across folds; traditional recursion catches bugs per-step.

Some production teams (Nexus, for example) explored folding and reverted to AIR-based approaches. Not because folding is inferior in theory, but because for their specific $|F|$ (complex zkVM execution), the engineering complexity didn't pay off.

The following table summarizes the decision:

Scenario	Recommended Approach
Small step function (< 1000 constraints), millions of steps	Folding (Nova/HyperNova)
Large step function (> 10000 constraints), complex logic	Traditional IVC or STARK

Scenario	Recommended Approach
Need multi-instance aggregation	HyperNova or ProtoStar
Custom gates, non-R1CS constraints	HyperNova (CCS) or ProtoStar
Maximum simplicity, proven tooling	STARK/AIR
Smallest possible final proof	Fold, then wrap in Groth16

CycleFold: Efficient Curve Switching

All folding schemes face the curve-cycle problem from earlier in this chapter: the folding verifier performs group operations, which are expensive to prove in-circuit over a different field. But folding has a unique advantage here that traditional recursion doesn't: the "verifier work" per step is tiny (a few scalar multiplications), not a full SNARK verification. CycleFold exploits this.

In Nova's IVC loop, the prover updates the running commitment:

$$C_{z_{acc}} \leftarrow C_{z_{acc}} + r \cdot C_{z_i}$$

This is a scalar multiplication on curve E_1. If our main circuit is over the scalar field $\mathbb{F}_p$ of E_1, we can't compute this operation natively. The curve points have coordinates in $\mathbb{F}_q$ (the base field), and $\mathbb{F}_q$ arithmetic inside an $\mathbb{F}_p$ circuit is expensive. This is the same field mismatch from the curve cycle section above, but now at the folding level rather than the full-verification level. CycleFold's solution relies on having a curve cycle (Pasta, BN254/Grumpkin) where a second curve E_2 has scalar field $\mathbb{F}_q$, making the expensive operation native on E_2.

Traditional recursion would embed the entire verifier (including pairings) in the circuit, paying hundreds of thousands of constraints. But Nova's "verifier" is just this one scalar multiplication. Can we handle it more cheaply?

The CycleFold idea. Instead of proving the scalar multiplication in the main circuit, *defer* it to a separate accumulator on the cycle curve E_2.

Recall the cycle: E_1 has scalar field $\mathbb{F}_p$ and base field $\mathbb{F}_q$; E_2 has scalar field $\mathbb{F}_q$ and base field $\mathbb{F}_p$. The scalar multiplication $r \cdot C$ on E_1 involves $\mathbb{F}_q$ arithmetic (the curve operations). But E_2 circuits are *natively* over $\mathbb{F}_q$. So:

1. **Main circuit (on E_1):** Proves "F was computed correctly" and that the folding challenges were derived correctly. It takes the *result* of the commitment update $C'_{z_{acc}}$ as a public input but does not check the scalar multiplication that produced it. The main circuit trusts this value for now.
2. **Auxiliary circuit (on E_2):** A tiny circuit that checks one claim: "given $C_{z_{acc}}$, r_i, and C_{z_i}, the output $C'_{z_{acc}} = C_{z_{acc}} + r_i \cdot C_{z_i}$ is correct." Because E_2's scalar field is $\mathbb{F}_q$, the curve arithmetic on E_1 points is native here. This circuit is roughly 10,000 constraints, compared to 100,000+ for emulating the same operation non-natively in the main circuit.
3. **Two parallel folding loops.** The main accumulator on E_1 folds each step's computation claim, just as in standard Nova. The auxiliary accumulator on E_2 folds each step's commitment-update claim. Both accumulators grow in parallel: one step of the IVC loop produces one fold on each curve.
4. **Two final SNARKs.** At the end, the prover produces one SNARK on E_1 (proving the accumulated computation is correct) and one SNARK on E_2 (proving all commitment updates were correct). The verifier checks both. Each SNARK operates over its native field, so neither requires emulation.

The two accumulators are coupled through shared public inputs: the commitment values that the main circuit assumes correct and the auxiliary circuit verifies. Soundness holds because a cheating prover who fakes a commitment update on the main side will fail the auxiliary accumulator's check at the end.

After T steps:

- Main accumulator: one final SNARK on E_1 proves "F was applied correctly T times"
- Auxiliary accumulator: one final SNARK on E_2 proves "all T commitment updates were computed correctly"

Both SNARKs are over their native fields. No cross-field emulation anywhere.

The cost breakdown:

- Per step: ~10,000 constraints on the cycle curve (the scalar multiplication circuit)
- Final proof: two SNARKs, one on each curve
- Total overhead: roughly $10,000 \cdot T$ constraints across all steps, versus $100,000 \cdot T$ without CycleFold

For long computations, this is a 10× reduction in the curve-cycle overhead.

CycleFold only works for folding, not traditional recursion. Traditional recursion embeds the *entire verifier* in circuit at each step, including pairings, hash evaluations, and complex checks that are all

entangled with the soundness argument. You cannot split these across two curves because the verifier's logic is monolithic. Folding's per-step verifier, by contrast, is just a few scalar multiplications on commitments, cleanly separable from the computation proof. This modularity is what lets CycleFold put the two concerns on different curves.

CycleFold applies to Nova, HyperNova, and ProtoStar, making all of them practical over curve cycles like Pasta (Pallas/Vesta) or BN254/Grumpkin.

The full compatibility landscape

With Nova, HyperNova, ProtoStar/ProtoGalaxy, and Mira all developed, we can now map the full landscape of which proof-system components pair naturally. Chapter 22 observed that "one choice determines the rest": picking univariate polynomials implies roots of unity, FFT, and KZG or FRI; picking multilinear polynomials implies the hypercube, the halving trick, and IPA or Dory. The same principle extends to the recursion layer, though the boundaries have softened as folding generalizes:

Lineage	Constraints	PIOP mechanism	PCS	Recursion
Sum-check native	R1CS	Sum-check (multilinear)	IPA, Dory, Hyrax	Nova (relaxed R1CS folding)
Sum-check + CCS	CCS (captures R1CS, Plonkish, AIR)	Sum-check (multilinear)	IPA, Dory, Hyrax	HyperNova (CCS folding via sum-check)
Quotienting native	Plonkish	Quotienting (univariate)	KZG	Mira (pairing-based folding); or curve cycles
STARK	AIR	Quotienting (univariate)	FRI	Full verification (recursion via hash-based PCS)
Hybrid	Mixed	STARK inner, Groth16 outer	FRI inner, KZG outer	Composition (STARK→Groth16 wrap)

ProtoStar and ProtoGalaxy do not appear in a single row because they are *protocol-agnostic*: they accumulate any special-sound interactive protocol, whether it comes from the sum-check or quotienting tradition. A ProtoStar-based system can accumulate sum-check rounds (landing in the sum-check

rows) or Plonkish quotient checks (landing in the quotienting row). They sit orthogonally to the lineage distinction, operating at the level of the interactive protocol's algebraic structure rather than the constraint system or PCS.

HyperNova also crosses boundaries: it uses sum-check as its PIOP mechanism but folds CCS instances, which can express Plonkish and AIR constraints. The constraint system and the PIOP mechanism are not always from the same lineage.

Within each pure lineage (sum-check native, quotienting native), each component exists because the previous one *requires* it. In the sum-check lineage, Nova's folding produces a *linear combination* of witness vectors ($z = z_1 + r \cdot z_2$). The folded witness is a multilinear polynomial's evaluation table, because R1CS witnesses *are* evaluation tables over the Boolean hypercube (Chapter 19). Sum-check is the natural PIOP for multilinear polynomials because it reduces claims about exponentially many hypercube evaluations to a single random evaluation (Chapter 3). The PCS must then open a multilinear polynomial at a random point in $\mathbb{F}^n$, which is exactly what IPA and Dory are built to do (Chapter 22). Each layer's choice is forced by the previous layer's output format.

In the quotienting lineage, the chain starts differently. Plonkish constraints are verified via quotient polynomials over roots of unity, which FFT computes. KZG commits to the same univariate polynomials the FFT produces, reusing the evaluation domain. Mira folds pairing-based arguments directly, staying within the univariate/KZG world. For STARK recursion, FRI replaces KZG; the verifier is hash-based and the recursion proceeds through full in-circuit verification rather than folding.

The hybrid row exists for a different reason: complementary strengths across stacks. STARKs (quotienting + FRI) give fast transparent proving; Groth16 (quotienting + KZG) gives tiny constant-size proofs. Composition bridges them by proving "the STARK verifier accepted" with Groth16, using hash-based transparency for the bulk and pairing-based compactness for on-chain delivery.

The heuristic for choosing a recursion strategy follows from which constraints matter most. For pure sum-check IVC with R1CS (simplest folding, smallest per-step overhead), Nova suffices. For richer constraint languages (custom gates, higher degree) with sum-check-based folding, HyperNova over CCS. For existing Plonkish/KZG infrastructure, Mira for pairing-based folding or ProtoGalaxy for general accumulation. For the smallest final proofs on-chain, fold with whichever machinery suits your stack and compose with Groth16 at the very end.

BlindFold: Folding for Zero-Knowledge

Most deployed "zkVMs" were not, for years, truly zero-knowledge. They were succinct (short proofs, fast verification) but the proofs leaked information about the prover's private inputs. The reason: the sum-check protocol at their core is not zero-knowledge. Each round polynomial g_j is a deterministic function of the witness, and its coefficients are linear combinations of the witness entries. After

enough rounds, a verifier accumulates enough constraints to recover the witness entirely. Chapter 18 showed this leakage concretely.

BlindFold (Section 7 of HyperNova, Kothapalli, Setty, Tzialla, 2023) resolves this in three moves:

Move 1: Hide the round polynomials. For a degree-d round polynomial $g_j(X) = c_0 + c_1 X + \cdots + c_d X^d$, the prover sends Pedersen commitments $C_k = c_k G + \rho_k H$ instead of the field elements c_k. The verifier sees only opaque group elements. This immediately hides the witness, but the verifier can no longer check anything (the consistency check $g_j(0) + g_j(1) = V_{j-1}$ requires knowing the actual coefficients).

Move 2: Encode the verifier as an R1CS. Every check the sum-check verifier would have performed becomes a constraint. The consistency check $2c_0 + c_1 + \cdots + c_d = V_{j-1}$ (verifying $g_j(0) + g_j(1)$ equals the previous claim) is one linear constraint. The evaluation check $c_0 + \gamma_j c_1 + \gamma_j^2 c_2 + \cdots + \gamma_j^d c_d = V_j$ (verifying $g_j(\gamma_j)$ at the Fiat-Shamir challenge) is another, with the powers γ_j^k baked into the R1CS matrices as constants since both parties derive them from the public transcript. For a sum-check with $R \approx 100$ rounds, this R1CS has roughly $2R \approx 200$ constraints, a tiny system.

Move 3: Fold with a random witness, then prove. Proving this R1CS with Spartan directly would reintroduce the problem (Spartan's own sum-check messages would leak the witness). BlindFold breaks the circularity by folding the real witness Z_1 with a uniformly random witness Z_2 before running Spartan. The folded witness $Z_f = Z_1 + r \cdot Z_2$ is uniformly distributed for any $r \neq 0$, because the map $Z_2 \mapsto Z_1 + r \cdot Z_2$ is an affine bijection on $\mathbb{F}^n$. This is the **algebraic one-time pad**: just as $c = m \oplus k$ is uniform for uniform key k regardless of message m, the folded witness is uniform regardless of the real witness. Spartan can now prove the folded R1CS in the clear without being zero-knowledge itself, because the data it operates on reveals nothing about Z_1.

The folding uses the same Nova protocol from earlier in this chapter (cross-term computation, commitment, Fiat-Shamir challenge, linear combination of witnesses and error vectors). The only difference from IVC folding is that the second instance is random rather than derived from a computation step. The final SNARK proves the folded instance is satisfiable; the verifier folds commitments homomorphically (Pedersen's additive structure) and checks the Spartan proof without ever seeing any witness value.

The cost is remarkably low. The blinded Phase 1 proof (Pedersen commitments replacing field elements) is often *shorter* than the original non-ZK proof. The Phase 2 BlindFold proof (Spartan over ≈ 256 constraints) adds roughly 3 KB. In multi-stage protocols, BlindFold encodes all stages' verifier checks into a single R1CS and folds once; the marginal cost of adding another sum-check stage is two more constraints. Earlier approaches (masking polynomials per Libra, Σ-protocol wrappers per Hyrax) pay their overhead per stage and scale with the number of sum-check invocations.

Choosing a strategy

The chapter has covered three recursion-related techniques, each at a different level of the proof system stack. The decision tree is:

1. **Composition** is for combining two different proof systems. Use it when you need a small final proof from a fast-but-large-proof inner system (the STARK→Groth16 pattern), or when wrapping a non-ZK system in a ZK outer layer. It is a one-time operation, not an iterative one.
2. **IVC (folding or traditional recursion)** is for unbounded sequential computation. Nova handles R1CS; HyperNova generalizes to CCS (capturing Plonkish and AIR); ProtoStar/ProtoGalaxy accumulate any special-sound protocol. The choice between them depends on the step function size $|F|$ relative to the verifier overhead $|V|$ and on which constraint system your application uses; the "When to Use What" guide in the Beyond Nova section develops this in detail. The compatibility table above maps which folding scheme pairs with which PCS and PIOP.
3. **Direct proving** (no recursion) is appropriate when the computation fits in a single circuit. Every recursive system has a minimum useful circuit size: if $|F| < |V|$, recursion adds overhead without benefit. Traditional recursion has a high threshold ($|V| \approx 10{,}000$-$50{,}000$ constraints for in-circuit verification). Folding lowers it dramatically (under 100 group operations per step), which is why folding made recursion practical for small step functions where it was previously absurd.

Key takeaways

1. **Composition combines complementary proof systems.** The outer prover handles only the inner verifier circuit, which is much smaller than the original computation. This is why Groth16 wrapping doesn't reintroduce the slow prover: Groth16 proves a circuit of size $O(\sqrt{N})$, not N.
2. **Recursion compresses through self-reference.** Each recursive layer proves a smaller circuit (the previous layer's verifier). After $O(\log \log N)$ layers, verification cost reaches a constant. IVC extends this to unbounded sequential computation, where each step's proof attests to the entire history.
3. **Field mismatch is the main obstacle to recursion.** Pairing-based verifiers do $\mathbb{F}_q$ arithmetic, but circuits constrain in $\mathbb{F}_p$. Emulation blows up circuit size by 100×. Curve cycles (Pasta, BN254/Grumpkin, BLS12-377/BW6-761) solve this by alternating between matched curve pairs where each step's verifier arithmetic is native.
4. **Deep recursion weakens security proofs but not security.** Extraction requires R^k rewinds for depth k, degrading provable security by $\log_2 R$ bits per layer. No known attack exploits the depth; the gap is between what we can prove and what we believe.

5. **Folding replaces per-step verification with accumulation.** Two relaxed R1CS claims fold into one via random linear combination, with the error vector E absorbing the cross-terms. Only the final accumulated claim requires a SNARK. Per-step cost drops from thousands of constraints to a handful of group operations, making IVC practical for small step functions where traditional recursion's overhead dominated.

6. **Folding has generalized beyond R1CS.** HyperNova folds CCS (which captures R1CS, Plonkish, and AIR) via asymmetric folding with sum-check. ProtoStar and ProtoGalaxy accumulate any special-sound protocol, sitting orthogonally to the sum-check/quotienting divide. Mira folds pairing-based arguments directly.

7. **The proof-system stack has natural compatibility lanes.** Choosing a constraint system, PIOP, PCS, and recursion strategy is not four independent decisions. Sum-check folding pairs with multilinear polynomials and IPA; quotienting pairs with univariate polynomials and KZG/FRI. HyperNova crosses the boundary by accepting quotienting-style constraints (via CCS) while using sum-check internally.

8. **Pedersen-based folding is not post-quantum safe.** Nova, HyperNova, and ProtoStar all use Pedersen commitments (discrete-log binding). The STARK→Groth16 wrapper reintroduces pairing assumptions at the final step. Lattice-based folding (LatticeFold, Neo, SuperNeo) and hash-based full STARK recursion are the emerging PQ-safe alternatives.

9. **BlindFold adds zero-knowledge via the algebraic one-time pad.** Commit to sum-check round polynomials, encode the verifier as a tiny R1CS, fold the real witness with a random one ($Z_f = Z_1 + r \cdot Z_2$ is uniform for uniform Z_2), and prove the folded instance with Spartan. Spartan need not be ZK because the data it sees is already masked. Cost: ≈ 3 KB.

10. **The decision tree has three levels.** Composition for combining complementary systems (one-time wrapping). Folding or traditional recursion for unbounded sequential computation (the $|F|/|V|$ ratio determines which). Direct proving when the computation fits in a single circuit. Every recursive system has a minimum useful circuit size; folding lowered this threshold by orders of magnitude.

Chapter 24: Choosing a SNARK

In 2016, Zcash launched with Groth16. The choice seemed obvious: smallest proofs, fastest verification, mature implementation. But Groth16 required a trusted setup ceremony. Six participants generated randomness, then destroyed their computers. The protocol was secure only if at least one participant was honest. If all six had colluded or been compromised, they could reconstruct the secret, mint unlimited currency, and no one would ever know.

Three years later, the Zcash team switched to Halo 2. No trusted setup. The proofs were larger. The proving was slower. But the existential risk evaporated.

This is the nature of SNARK selection: every choice trades one virtue for another. There is no universal optimum, no "best" system. There is only the right system for your constraints, your threat model, your willingness to accept which category of failure.

The preceding chapters developed a complete toolkit: sum-check protocols, polynomial commitments, arithmetization schemes, zero-knowledge techniques, composition and recursion. Each admits multiple instantiations. The combinations number in the dozens. Each combination produces a system with different properties: proof sizes ranging from 128 bytes to 100 kilobytes, proving times from milliseconds to hours, trust assumptions from ceremony-dependent to fully transparent.

This chapter provides a framework for navigating that landscape. Not a prescription (the field moves too fast for prescriptions) but a map of the territory and a compass for orientation.

The Five Axes of Trade-off

Every SNARK balances five properties. Improve one, and another suffers. The physics of cryptography permits no free lunch.

Proof Size

How many bytes cross the wire? For on-chain verification, proof size translates directly to gas costs (the blockchain section below gives concrete numbers). The spectrum spans three orders of magnitude:

- **Constant-size** (~100-300 bytes): Groth16, PLONK with KZG
- **Logarithmic** (~1-10 KB): Bulletproofs, Spartan
- **Polylogarithmic** (~10-100+ KB): STARKs, FRI-based systems

For on-chain verification, proof size is often the binding constraint. Everything else is negotiable.

Verification Time

How fast can the verifier check the proof?

On-chain, verification time translates directly to gas costs. A pairing operation costs roughly 45,000 gas. Groth16 needs 3 pairings. PLONK needs about 10. STARKs replace pairings with hashes, but require many of them.

The hierarchy:

- **Constant-time** (~3 pairings): Groth16
- **Logarithmic** (~10-20 pairings): PLONK, IPA-based systems
- **Polylogarithmic** (hash-dominated): STARKs

Groth16's 3-pairing verification is hard to beat. Everything else is playing catch-up. But pairings rely on discrete log, which Shor's algorithm breaks, so this advantage may not survive the quantum transition.

Prover Time

How fast can an honest prover generate a proof?

For small circuits, this barely matters. For zkVMs processing real programs, it's everything.

Consider a billion-constraint proof. At $O(n)$, with each field operation taking 10 nanoseconds, proving takes about 10 seconds. At $O(n \log n)$, with $\log n \approx 30$, the same proof takes 5 minutes. At $O(n^2)$, it takes 300 years.

The hierarchy:

- **Linear in constraint count**: Sum-check-based systems (Spartan, Lasso, Jolt)
- **Quasilinear** ($O(n \log n)$): PLONK, Groth16, FFT-dominated systems
- **Superlinear**: Some theoretical constructions (impractical at scale)

At billion-constraint scale, the $\log n$ factor (roughly 30) is the difference between a 10-second proof and a 5-minute proof. This is why zkVMs have increasingly moved toward sum-check-based architectures: when proving a million CPU instructions at 50 constraints each, linear time is a requirement, not a luxury.

The gap is wider than the asymptotics suggest. FFT-based provers (Groth16, PLONK) perform butterfly operations that jump across memory at strides of $N/2$, thrashing caches and stalling on RAM latency (Chapter 20 develops this in detail). Sum-check provers scan data linearly, keeping it streaming through the cache hierarchy. At billion-constraint scale, memory access patterns can dominate wall-clock time even more than the operation count, compounding sum-check's asymptotic advantage with a large constant-factor improvement.

Trust Assumptions

What must you trust for security?

The Zcash ceremony involved six participants on three continents. Each generated randomness, contributed to the parameters, then destroyed their machines. One participant used a Faraday cage. Another broadcast from an airplane. The paranoia was justified: if *all six* colluded or were compromised,

they could mint unlimited currency, and the counterfeits would be cryptographically indistinguishable from real coins.

This is the price of trusted setup.

The spectrum:

- **Circuit-specific trusted setup** (Groth16): Each circuit requires its own ceremony. Change the circuit, repeat the ritual.
- **Universal trusted setup** (PLONK, Marlin): One ceremony supports all circuits up to a size bound. The trust is amortized, not eliminated.
- **Transparent** (STARKs, Bulletproofs): No trusted setup. Security derives entirely from public-coin randomness and standard assumptions.

Transparency eliminates an entire category of catastrophic failure, at the cost of larger proofs, sometimes by two orders of magnitude.

Post-Quantum Security

Will the system survive Shor's algorithm?

Shor's algorithm solves discrete logarithm and factoring in polynomial time on a quantum computer. The day a cryptographically relevant quantum computer boots, every pairing-based SNARK becomes insecure. Groth16 proofs could be forged. KZG commitments could be opened to false values. The entire security model collapses.

The threatened systems:

- All pairing-based SNARKs (Groth16, KZG-based PLONK)
- All discrete-log commitments (Pedersen, Bulletproofs)

The resistant systems form a growing family:

- Hash-based constructions (STARKs with FRI, WHIR-based systems)
- Sum-check + hash-based PCS (Whirlaway combines SuperSpartan with WHIR, achieving both multilinear proving and post-quantum security with proofs smaller than FRI at the same security level)
- Sum-check + lattice-based PCS (Jolt with Akita, a Module-SIS commitment, since September 2026; lattice folding schemes such as LatticeFold and Neo remain research prototypes)

The sum-check tradition is no longer tied to discrete-log commitments. WHIR (EUROCRYPT 2025) provides a hash-based multilinear PCS with faster verification than FRI, enabling sum-check-based

provers to achieve post-quantum security without switching to the univariate/STARK paradigm. Lattice commitments offer a second exit. An Ajtai commitment $As \bmod q$ to a short vector s is linear in the way a Pedersen commitment is, so it keeps the property that made curve-based sum-check provers fast, namely that committing to small or zero values costs little. Both routes close the gap that previously forced sum-check systems to rely on IPA, KZG or Dory, all quantum-vulnerable.

When will quantum computers arrive? Estimates as of 2026 range from 5 to 20 years for cryptographically relevant machines, with the timeline compressing as investment accelerates. For a private transaction, the uncertainty is tolerable. For infrastructure meant to last decades (identity systems, legal records, financial settlements), the Ethereum Foundation's response is instructive: provable 128-bit security by end of 2026, with proof-size caps of 300 KiB that push the ecosystem toward hash-based schemes. The word *provable* carries weight. Between late 2025 and spring 2026 the up-to-capacity proximity gap conjectures behind FRI and WHIR parameters were disproved (Chapter 26), so a system advertising 100 conjectured bits may deliver closer to 80.

The System Landscape

Each major proof system occupies a different position in the trade-off space. None dominates all others. The choice depends on which constraints bind tightest.

Groth16: The Incumbent

Groth16 has the smallest proofs in the business: 128 bytes, three group elements. Verification requires three pairings. Implementations exist in every language, optimized for every platform, battle-tested across billions of dollars in transactions.

The cost is trust. Every circuit needs its own ceremony. Change one constraint, and the parameters are worthless. The ceremony participants must be trusted absolutely, or the "toxic waste" (the secret randomness) must never be reconstructed.

This combination (minimal proofs, maximal trust) made Groth16 the default for years. It remains dominant for on-chain verification where proof size is the binding constraint and the application can absorb a one-time ceremony.

PLONK: The Flexible Middle Ground

PLONK solved Groth16's upgrade problem. A single ceremony generates parameters that work for any circuit up to a size bound. Modify the circuit, keep the same parameters. The trust is amortized across an ecosystem rather than concentrated on a single application.

Proofs grow to 500-2000 bytes. Verification requires more pairings. But the flexibility is transformative: zkEVMs can upgrade their circuits without coordinating new ceremonies. Application developers can iterate without security theater.

Custom gates push PLONK further. Where Groth16 accepts only R1CS, PLONK's constraint system accommodates specialized operations. A hash function that requires 10,000 R1CS constraints might need only 100 Plonkish constraints with a custom gate.

Variants proliferated: UltraPLONK, TurboPLONK, HyperPLONK, each optimizing a different axis (proof size, custom gates, multilinear polynomials). PLONK became the platform on which much of the industry standardized for general-purpose proving.

STARKs: The Transparent Option

STARKs eliminate trust entirely. No ceremony. No toxic waste. No existential risk from compromised participants. Security rests on collision-resistant hashing, nothing more.

The price is size. STARK proofs run 50-100+ KB, sometimes larger. Verification is polylogarithmic rather than constant. For on-chain deployment, this can be prohibitive.

But STARKs offer compensations. Provers approach linear time (Chapter 20 develops how FRI folding and small-field techniques achieve this). Hash-based constructions are believed to be post-quantum secure, since the best known quantum attack (Grover's algorithm) provides only a quadratic speedup, manageable by doubling the hash output size. And there's a philosophical clarity: the proof stands alone, answerable only to mathematics.

StarkWare built a company on this trade-off. For rollups processing millions of transactions, the amortized proof cost per transaction becomes negligible. The prover speed matters; the verifier runs once.

Bulletproofs: The Pairing-Free Path

Bulletproofs occupy a specific niche: transparency without the STARK size explosion. Proofs grow logarithmically (typically 600-700 bytes for range proofs). No trusted setup. No pairings required.

The tradeoff is that verification takes linear time in the circuit size. For small circuits (range proofs, confidential transactions), this is acceptable. For large computations, it becomes prohibitive.

Monero adopted Bulletproofs for confidential amounts. The proofs are small enough to fit in transactions, transparent enough to satisfy decentralization purists, and specialized enough for the specific task of range proofs.

But Bulletproofs aren't post-quantum. They rely on discrete log hardness. The same quantum computer that breaks Groth16 breaks Bulletproofs.

Sum-check-based systems

Spartan, Lasso, Jolt, HyperPlonk, Binius, and the Whirlaway stack all belong to the sum-check tradition described in Chapters 19-21. Their shared characteristic is linear-time proving, which at billion-constraint scale is the difference between a 10-second proof and a 5-minute one.

Virtual polynomials minimize commitment costs (Chapter 21). Sparse sum-check handles irregular constraint structures naturally. The apparatus is optimized for general-purpose computation, which is why zkVMs have increasingly adopted sum-check architectures.

Sum-check systems produce larger proofs (logarithmic, not constant), have newer implementations (less battle-tested), and historically depended on discrete-log-based PCS (IPA, KZG) that made them quantum-vulnerable. This last limitation is dissolving from two directions. WHIR (EUROCRYPT 2025) provides a hash-based multilinear PCS with faster verification than FRI; Hachi (eprint 2026/156) provides a lattice-based multilinear PCS under Module-SIS with ≈ 55 KB proofs and 12.5× faster verification than prior lattice schemes. Its successor Akita (eprint 2026/1983) became Jolt's commitment in September 2026, yielding a post-quantum zkVM with proofs under 100 KB that proves faster than the curve-based Jolt it replaced. Whirlaway (SuperSpartan + WHIR) demonstrated that sum-check-based systems can achieve post-quantum security without switching to the univariate/STARK paradigm. The Ethereum Foundation's lean stack grew out of it. Its current zkVM, leanVM, works over a 192-bit binary tower field with WHIR and Ligerito commitments and standard hashes in place of Poseidon, targeting post-quantum on-chain verification with no reliance on proximity gap conjectures.

The converging zkVM landscape

The boundaries between the categories above are blurring in production zkVMs. The major systems as of 2026:

- **SP1** (Succinct): migrated from STARK-based (SP1 Turbo, FRI over BabyBear) to sum-check-based (SP1 Hypercube, multilinear polynomials over KoalaBear with a jagged PCS from Chapter 21 and Logup-GKR). On mainnet since February 2026, it proves 99.7% of Ethereum blocks in under 12 seconds on 16 RTX 5090 GPUs, at 100 provable bits with proofs of about 530 KiB.
- **OpenVM** (Axiom): version 2.0 replaced FRI with SWIRL, a multilinear stack built on WHIR and LogUp over BabyBear, reporting 100 provable bits with proofs of about 270 KiB. Its RV32IM extension is formally verified in Lean.

- **RISC Zero**: STARK-based with FRI over BabyBear, Groth16 wrapper for on-chain verification. Proves Ethereum blocks in under 45 seconds.
- **Jolt** (a16z): pure sum-check with Lasso lookups (Chapter 21) and Twist/Shout memory checking. With the lattice-based Akita commitment it proves over 2 million RV64 cycles per second on a laptop CPU with proofs under 100 KB, post-quantum under Module-SIS.
- **ZKsync Airbender**: STARK-based over Mersenne31 with a custom DEEP-ALI implementation.
- **Zisk** (Polygon spinoff): RISC-V 64 over Goldilocks with a 1.5 GHz execution engine, optimized for low-latency distributed proving. Reports 128 provable bits with proofs of about 270 KiB before its PLONK wrapper.
- **leanVM** (Ethereum Foundation): minimal zkVM that began as Whirlaway (SuperSpartan + WHIR over KoalaBear) and now runs over a binary tower field with standard hashes, targeting provable 128-bit post-quantum security.

Most of these share small or word-sized fields (BabyBear, KoalaBear, Mersenne31, Goldilocks), AIR or CCS constraints and Logup-style bus arguments for cross-table consistency. The exceptions are instructive. Jolt with Akita works over a 128-bit prime because its lattice commitment no longer needs a curve-sized field. leanVM moved to binary towers so that standard hashes such as SHA and BLAKE become cheap to prove. The convergence on shared primitives (Chapter 20) holds even as the field choices begin to diverge.

Application-Specific Guidance

Theory meets practice at the application boundary. The abstract trade-offs crystallize into concrete decisions.

Blockchain Verification (On-Chain)

The verifier runs on Ethereum, paying gas for every operation. Two costs dominate: calldata (bytes shipped to the chain) and computation (opcodes executed on-chain).

At current gas prices, a 128-byte Groth16 proof costs about 20,000 gas in calldata. Verification adds roughly 150,000 gas for the pairing checks. Total: under 200,000 gas. A simple ETH transfer costs 21,000 gas. The proof verification is economically viable.

A 50 KB STARK costs 800,000 gas in calldata alone. Verification adds another 300,000-500,000 gas. Total: over a million gas. For individual transactions, this is often prohibitive.

Composition (Chapter 23) bridges the gap. Generate a STARK proof (transparent, fast prover), then prove "the STARK verifier accepted" with Groth16 (small proof, cheap verification). The inner STARK provides transparency; the outer Groth16 provides on-chain efficiency. The trust assumption applies

only to the wrapper. The economics favor large computations: wrapping a million-constraint STARK in Groth16 adds $\approx 50{,}000$ constraints for the STARK verifier (5% overhead), while wrapping a thousand-constraint STARK adds 50× overhead.

zkRollups

Rollups amortize proof costs across thousands of transactions. A proof that costs 200,000 gas becomes 20 gas per transaction when it covers 10,000 transactions. The economics invert. Larger proofs become tolerable when they aggregate more computation.

StarkNet uses STARKs directly. The proofs are large (100+ KB), but the amortization across massive batches makes the per-transaction cost negligible. The transparency is a feature, not a compromise.

zkSync and Scroll use Groth16 wrappers around internal proving systems. The outer proof is tiny. The inner system can be whatever works best for their EVM implementation.

Prover efficiency matters most (the prover runs for every batch), while proof size matters less (it amortizes across all transactions in the batch).

zkVMs

Proving correct execution of arbitrary programs requires billions of constraints. The system landscape section above lists the major zkVMs; the choosing question is which architectural pattern fits your deployment.

The binding constraint is prover speed. A 10-second proof is a feature. A 10-minute proof is a bug. Virtual polynomials (Chapter 21) minimize commitment costs; lookup arguments (Chapter 14) replace expensive constraint checks with table lookups; small fields (Chapter 20) cut per-operation cost by 10×. Everything is oriented toward making the prover faster.

On-chain verification still demands small proofs, so zkVMs follow the same composition pattern described in the blockchain section above (STARK or sum-check inner proof, Groth16 wrapper for Ethereum). Eliminating this wrapper, via STARK verification precompiles on Ethereum or efficient hash-based on-chain verification via WHIR, is an active area of work.

Privacy-Preserving Applications

When zero-knowledge is the point rather than a bonus, implementation quality matters as much as theoretical properties.

Groth16 and PLONK produce ZK proofs with modest overhead. The masking techniques are well-understood. But implementation errors can leak information through timing side channels, error messages, or malformed proof handling.

STARKs require more care. The execution trace is exposed during proving, then masked. The masking must be done correctly. A bug here doesn't crash the system; it silently leaks witnesses. You might never know until the damage is done.

Tornado Cash used Groth16. Zcash used Groth16, then Halo 2. Aztec uses UltraPlonk and Honk (PLONK variants co-developed by the Aztec team). All chose mature implementations with extensive auditing, because privacy failures are catastrophic and silent.

Beyond the choice of proof system, privacy applications face a second decision that further constrains the options: *where the prover runs*. **Server-side proving** (zkRollups, zkVMs) runs provers on powerful infrastructure; the witness data reaches the server, which generates proofs and posts them on-chain. Privacy comes from the proof hiding witness details from the chain, not from the prover. **Client-side proving** (Aztec, Zcash) runs provers on user devices, so sensitive data never leaves the machine and only the proof and minimal public inputs reach the network.

Client-side proving constrains system choice dramatically. A browser or mobile device can't match datacenter hardware. Aztec's architecture is instructive: private functions execute locally, requiring proof systems efficient enough for consumer hardware. This rules out anything demanding server-grade resources for reasonable latency.

Post-quantum applications

The "Post-Quantum Security" axis above lists the resistant systems (STARKs, WHIR-based, lattice-based). For application guidance, the critical distinction is between integrity and privacy. For integrity-only applications (proving a computation was correct, no sensitive data in the witness), a dual-proof strategy works: generate both a classical proof (for efficiency today) and a post-quantum proof (for survival tomorrow), and migrate when quantum threatens. For applications involving *private data*, the dual strategy fails. A "harvest now, decrypt later" adversary records classical proofs today and breaks them with a future quantum computer, retroactively extracting the witness. Private data needs post-quantum security from day one.

The Trade-Off Triangle

Project managers know the Iron Triangle: Fast, Good, Cheap. Pick two. SNARKs have their own version: **Succinct, Transparent, Fast Proving**. The physics of cryptography enforces the same brutal constraint.

Three properties stand in tension: **proof size, prover time, and trust assumptions**.

System	Proof Size	Prover Time	Trust
Groth16	Minimal (128 B)	Quasilinear	Maximal (circuit-specific)
PLONK	Small (500 B)	Quasilinear	Moderate (universal)
STARKs	Large (50+ KB)	Linear	None

Pick any two vertices. The third suffers.

This is not a failure of engineering. It's a reflection of information-theoretic and complexity-theoretic constraints. Small proofs require structured commitments. Structured commitments require trusted setup or expensive verification. Fast provers require simple commitment schemes. Simple commitment schemes produce large proofs.

Every production system that appears to break this triangle does so through composition (Chapter 23). Halo 2 wraps a transparent IPA-based inner proof in a succinct accumulation scheme. RISC Zero and SP1 wrap transparent STARKs in Groth16. Folding-based systems defer all verification to a single final SNARK. In each case, the "escape" is architectural complexity: two or more proof systems cooperating, each contributing the vertex it handles best.

Implementation Realities

The best algorithm with a buggy implementation is worse than a mediocre algorithm implemented correctly.

Audit status

ZK bugs are silent: a soundness error lets attackers forge proofs, a witness leak exposes private data, and neither produces error messages. Zcash's Sprout had a soundness bug for years, discovered by a researcher rather than an attacker. Use audited implementations; multiple recent audits matter more than theoretical elegance.

Hardware acceleration

GPU proving is now standard for production zkVMs, with 10-100× speedups over CPU for NTT and MSM operations. SP1 Hypercube achieves real-time Ethereum proving on 16 GPUs. The choice of proof

system constrains which hardware optimizations are available: NTT-heavy systems (STARKs, PLONK) benefit most from GPU parallelism, while sum-check provers with linear memory access patterns also parallelize well across CPU cores via SIMD (Chapter 20).

Tooling

The choice of proof system often follows from the available tooling rather than the other way around. Circom targets Groth16 and PLONK circuits. Cairo is StarkWare's language for STARK-based programs. Noir (Aztec) compiles to multiple backends. At the library level, Arkworks provides modular Rust primitives for field arithmetic, curves, and SNARK components. Plonky3 (Polygon) is the shared proving toolkit beneath Pico, Miden and the first generation of OpenVM, with pluggable field backends (BabyBear, KoalaBear, Mersenne31, Goldilocks) and a modular AIR interface. On the multilinear side the reference implementations remain closer to research code. WHIR's own repository describes itself as an academic prototype, while Jolt ships its lattice commitment behind a feature flag in an alpha release. Mature tooling compounds over time; switching frameworks mid-project is expensive.

Quick Reference

System	Proof Size	Verify Time	Prove Time	Setup	Post-Quantum
Groth16	~128 B	3 pairings	$O(n \log n)$	Circuit-specific	No
PLONK+KZG	~500 B	~10 pairings	$O(n \log n)$	Universal	No
STARK (FRI)	~50-100 KB	$O(\log^2 n)$ hashes	$O(n)$	Transparent	Yes
Bulletproofs	~600 B + log	$O(n)$ exp	$O(n)$ exp	Transparent	No
Spartan/Jolt (curves)	~log KB	$O(\log n)$ exp	$O(n)$	Transparent	No
Sum-check + WHIR	~50-300 KB	$O(\log^2 n)$ hashes	$O(n)$	Transparent	Yes
Jolt + Akita (lattice)	~60-100 KB	$\tilde{O}(n^{1/K})$	$O(n)$	Transparent	Yes

The hash-based rows widen once security is stated in provable rather than conjectured bits. At 100 to 128 provable bits, production systems report proofs between 200 and 530 KiB.

Key takeaways

1. **Every application has a binding constraint; the system choice follows from it.** On-chain verification binds on proof size (Groth16/PLONK). zkVMs bind on prover speed (sum-check/STARKs). Privacy binds on implementation quality and client-side efficiency. Long-lived infrastructure binds on quantum resistance (hash-based systems only). Identify which constraint binds tightest; the rest is negotiable.
2. **The trade-off triangle is inescapable within a single system.** Small proofs + fast provers requires trusted setup. Small proofs + transparent requires slow verification. Fast provers + transparent requires large proofs. Composition (Chapter 23) breaks the triangle by combining systems, at the cost of architectural complexity.
3. **Sum-check systems are no longer quantum-vulnerable.** WHIR provides a hash-based multilinear PCS. Hachi and its successor Akita provide lattice-based ones. Akita already runs inside Jolt, which became post-quantum and faster at the same time. Both routes close the gap that previously forced sum-check provers onto discrete-log commitments. For private data, post-quantum security is needed from day one (harvest-now-decrypt-later attacks make deferred migration dangerous).
4. **The zkVM landscape is converging on shared primitives.** Small fields, AIR or CCS constraints, Logup bus arguments, and STARK→Groth16 composition appear across SP1, OpenVM, RISC Zero, Jolt, ZKsync Airbender and leanVM, even as their architectural choices diverge. Security is now reported in provable bits, measured with shared tools such as the Ethereum Foundation's soundcalc.
5. **Tooling and audit status constrain choices as much as theory.** ZK bugs are silent (Zcash's Sprout had a soundness bug for years), so multiple recent audits matter more than theoretical elegance. Mature tooling compounds; switching frameworks mid-project is expensive.

Chapter 25: MPC and ZK parallel paths

In 1982, Andrew Yao posed a puzzle that sounded like a parlor game. Two millionaires meet at a party. Each wants to know who is richer, but neither wants to reveal their actual wealth. Is there a protocol that determines who has more money without either party learning anything else?

The question seems impossible. To compare two numbers, someone must see both numbers. A trusted third party could collect the figures, announce the winner, burn the evidence. But what if there is no trusted party? What if the millionaires trust no one, not even each other?

The same tension appears wherever private data meets joint computation. Satellite operators want to check if their orbits will collide, but their trajectories are classified. Banks want to detect money laundering across institutions without opening their books to each other. Nuclear inspectors want to verify warhead counts without learning weapon designs. The underlying problem is always the same: the computation requires inputs that no single party should see.

Yao proved the comparison can be done. Not by clever social arrangements or legal contracts, but by cryptography alone. The protocol he constructed, now called *garbled circuits*, allows two parties to jointly compute any function on their private inputs while revealing nothing but the output. Neither party sees the other's input. The trusted third party dissolves into mathematics.

This was the birth of **Secure Multiparty Computation** (MPC). The field expanded rapidly. In 1988, Ben-Or, Goldwasser, and Wigderson showed that with an honest majority of participants, MPC could achieve *information-theoretic* security with no computational assumption required, just the mathematics of secret sharing. The same year, Chaum, Crépeau, and Damgård proved that with dishonest majorities, MPC remained possible under cryptographic assumptions. By the early 1990s, the core theoretical question was settled. Any function computable by a circuit could be computed securely by mutually distrustful parties.

Computation, it turns out, does not require a single trusted processor. It can be *distributed* across adversaries who share nothing but a communication channel and a willingness to follow a protocol. The output emerges from the collaboration, but the inputs remain private.

Why MPC belongs in this book

Throughout this book, we've focused on trust between prover and verifier. The verifier need not believe the prover is honest; the proof itself carries the evidence. But there's another trust relationship we've quietly assumed: the prover has access to the witness. What if the witness is too sensitive to give to any single party?

Consider a company that wants to prove its financial reserves exceed its liabilities without revealing the actual figures to the auditor, the proving service, or anyone else. The company holds the witness (the books), but generating a ZK proof requires computation. If the company lacks the infrastructure to prove locally, it faces a dilemma. Outsource the proving and expose the witness, or don't prove at all.

MPC offers an escape. The company secret-shares its witness among multiple proving servers. Each server sees only meaningless fragments. Together, they compute the proof without any single server

learning the books. The witness never exists in one place. Trust is distributed rather than concentrated.

This is one of several approaches to the "who runs the prover?" problem:

Prove locally. Keep the witness on your own hardware. No trust required, but you need sufficient compute. For lightweight proofs this works; for zkVM-scale computation it may not.

Distribute via MPC. The approach just described. Requires the servers not to collude (honest majority or computational assumptions). This chapter develops the techniques.

Hardware enclaves (TEEs). Run the prover inside a Trusted Execution Environment like Intel SGX or ARM TrustZone. The enclave attests that it ran the correct code on hidden inputs. Trust shifts from the server operator to the hardware manufacturer, not trustless but a different trust assumption.

(Chapter 27 discusses a fourth approach, computing on encrypted data via FHE, as part of the broader programmable cryptography landscape.)

MPC and ZK also connect at a deeper level. MPC techniques directly yield ZK constructions through the "MPC-in-the-head" paradigm, where the prover simulates an MPC protocol inside their own mind, commits to the simulated parties' views, then lets the verifier audit a subset. The parallel paths converge into a single construction.

The MPC problem

The intuition from Yao's millionaires is clear enough, but building protocols requires a precise target. What exactly does it mean to compute "securely"?

The formal setting has n parties holding private inputs $x_1, \ldots, x_n$. They want to learn $f(x_1, \ldots, x_n)$ for some agreed-upon function f, but nothing else. A trusted third party could collect everything, compute, then announce the result. MPC must achieve the same outcome without the trusted party. The question is what "nothing else" means, and against whom.

The answer uses the same simulation paradigm that defines zero-knowledge (Chapter 17). There, a proof is zero-knowledge if a simulator can produce a transcript indistinguishable from a real one without access to the witness. Here, an MPC protocol is secure if a simulator, given only the corrupt parties' inputs and the output, can produce a view indistinguishable from what those parties actually observed during the protocol. If such a simulator exists, the protocol leaks nothing beyond what the function itself reveals. The corrupt parties could have generated everything they saw on their own.

Two parameters shape what kind of security is achievable: the adversary's behavior and the number of corrupt parties.

Adversary models

A **semi-honest** (or passive) adversary follows the protocol faithfully but tries to extract information from the transcript. Think of a curious employee who logs every packet but never forges one. A **malicious** (or active) adversary can deviate arbitrarily by sending wrong values, aborting early, or colluding with others. Think of a compromised machine running modified software.

Most efficient protocols assume semi-honest adversaries. Malicious security is achievable at higher cost, as we'll see later in this chapter.

Collusion thresholds

How many parties can be corrupt before security breaks? Protocols specify a threshold t so that security holds as long as at most t of the n parties are corrupt. The dividing line is $t = n/2$.

With an **honest majority** ($t < n/2$), protocols can achieve information-theoretic security. No computational assumption, no cryptographic hardness. Even an unbounded adversary learns nothing. The mathematics of secret sharing suffices.

With a **dishonest majority** ($t < n$, potentially $t = n - 1$), information-theoretic security becomes impossible. If all but one party collude, they hold enough information to reconstruct any secret shared among the group. Cryptographic assumptions become necessary because the adversary *could* break the scheme given infinite time, but doing so requires solving hard problems.

With the adversary model and threshold specified, the problem is precise. The question that remains is how to actually build such a protocol.

Secret-sharing MPC

The most natural approach is to keep data distributed throughout the entire computation. The BGW protocol, named after Ben-Or, Goldwasser, and Wigderson, does exactly this. Secret-share each input, compute on the shares, reconstruct only the output. To understand how this works, we need to understand what secret sharing actually does.

Shamir's secret sharing

Shamir's scheme (Appendix A covers the full details, including reconstruction formulas and security properties) distributes a secret s among n parties with threshold t by constructing a random univariate polynomial of degree $t - 1$ that passes through the point $(0, s)$:

$$P(X) = s + a_1 X + a_2 X^2 + \cdots + a_{t-1} X^{t-1}$$

The coefficients $a_1, \ldots, a_{t-1}$ are chosen uniformly at random. The secret s is the constant term, recoverable as $P(0)$.

Each party i receives the share $s_i = P(i)$, the polynomial evaluated at their index. Any t parties can pool their shares and use Lagrange interpolation to recover the polynomial, hence the secret. But $t-1$ shares reveal nothing. A degree $t-1$ polynomial is determined by t points, so with only $t-1$ points, every possible secret is equally consistent with the observed shares.

Concrete example. Share the secret $s = 7$ among 3 parties with threshold $t = 2$. Choose a random linear polynomial passing through $(0, 7)$, say $P(X) = 7 + 3X$. The shares are:

- Party 1: $s_1 = P(1) = 10$
- Party 2: $s_2 = P(2) = 13$
- Party 3: $s_3 = P(3) = 16$

Any two parties can reconstruct. Parties 1 and 3, holding $(1, 10)$ and $(3, 16)$, interpolate to find the unique line through these points: $P(X) = 7+3X$, so $P(0) = 7$. But party 1 alone, holding only $(1, 10)$, knows nothing. Any line through $(1, 10)$ could have any y-intercept. The secret could be anything.

Setup

Each party i secret-shares their input x_i by constructing a random polynomial $P_i(X)$ with $P_i(0) = x_i$ then sending share $P_i(j)$ to party j. After this initial exchange, party j holds one share of every input: $P_1(j), P_2(j), \ldots, P_n(j)$. No single party can reconstruct any input, but the distributed shares encode everything needed to compute.

Linear operations

Shamir sharing is *linear*, which makes addition and scalar multiplication free. If parties hold shares of secrets a and b encoded by polynomials P_a and P_b, then adding the shares gives valid shares of $a + b$.

Party j holds $P_a(j)$ and $P_b(j)$. When they compute $P_a(j) + P_b(j)$, this equals $(P_a + P_b)(j)$, the evaluation of the sum polynomial at j. The sum polynomial $P_a + P_b$ has constant term $P_a(0) + P_b(0) = a + b$. So the parties now hold valid Shamir shares of $a + b$, without any communication.

The same holds for scalar multiplication. If party j holds share $P_a(j)$ and multiplies it by a public constant c, the result $c \cdot P_a(j)$ is the evaluation of the polynomial $c \cdot P_a$ at j. This polynomial has constant term $c \cdot a$. Each party scales locally; no messages needed.

What this means in practice is that two parties can add their secrets without ever revealing them. Return to the earlier example: we shared $a = 7$ via $P(X) = 7 + 3X$, giving shares $(10, 13, 16)$. Now a second party shares their private input $b = 5$ by constructing $Q(X) = 5 + 2X$ and distributing:

- Party 1: $q_1 = Q(1) = 7$
- Party 2: $q_2 = Q(2) = 9$
- Party 3: $q_3 = Q(3) = 11$

After this exchange, each party holds two shares: party 1 holds $(s_1 = 10, q_1 = 7)$, party 2 holds $(s_2 = 13, q_2 = 9)$, party 3 holds $(s_3 = 16, q_3 = 11)$. Nobody knows $a = 7$ or $b = 5$ except the original owners.

To compute shares of $a+b$, each party adds their shares locally: party 1 computes $10+7 = 17$, party 2 computes $13+9 = 22$, party 3 computes $16+11 = 27$. These are evaluations of $(P+Q)(X) = 12+5X$ at points $1, 2, 3$. Interpolating any two recovers $(P + Q)(0) = 12 = a + b$. The sum was computed without anyone learning the inputs.

Addition and scalar multiplication are free. The cost of MPC concentrates entirely on multiplication.

Multiplication

Multiplication breaks the easy pattern. The product of two shares is *not* a valid share of the product. Shamir sharing uses polynomials of degree $t - 1$. If parties locally multiply their shares $P_a(j) \cdot P_b(j)$, they get evaluations of the product polynomial $P_a \cdot P_b$, which has degree $2(t - 1)$. This polynomial does encode ab at zero, but the threshold has effectively doubled so that $2t-1$ parties are now needed to reconstruct, not t. Repeated multiplications would make the degree explode.

Donald Beaver's solution resolves this through preprocessed randomness. Before the computation begins, distribute shares of random *triples* (u, v, w) satisfying $w = u \cdot v$. Nobody knows u, v, or w individually, but everyone holds valid shares of all three.

To describe the protocol, we use bracket notation: $[a]$ means "the parties collectively hold Shamir shares of a," with each party holding one evaluation $P_a(j)$. To multiply $[a]$ by $[b]$ using a triple:

1. Parties compute $[\alpha] = [a] - [u]$ and $[\beta] = [b] - [v]$ locally (subtraction is linear, so each party j subtracts their shares)
2. Parties reconstruct α and β publicly by pooling shares (these values are masked by the random u and v, so they reveal nothing about a or b)
3. Parties compute $[ab] = [w] + \alpha \cdot [v] + \beta \cdot [u] + \alpha\beta$ locally (each party j uses their shares of w, v, u plus the now-public α, β)

The algebra works because $ab = (u + \alpha)(v + \beta) = w + \alpha v + \beta u + \alpha\beta$. Since α, β, and $\alpha\beta$ are now public scalars, party j can compute their share of ab locally as $w_j + \alpha \cdot v_j + \beta \cdot u_j + \alpha\beta$. This is a

linear combination of valid Shamir shares, so the result is itself a valid Shamir share of ab. No single party learns ab, but together the parties hold shares of a polynomial whose constant term is ab, ready to feed into subsequent gates.

Intermediate values are never reconstructed. Each triple enables exactly one multiplication because α and β are now public; reusing the same triple with different inputs would leak information. A fresh triple is needed for every multiplication gate, generated during a preprocessing phase before inputs are known.

Circuit evaluation

With these building blocks, any arithmetic circuit can be evaluated. Share the inputs, process gates in topological order so that addition gates require no communication while multiplication gates consume one Beaver triple each, then reconstruct only the final output.

The reconstruction step works like the earlier Shamir example, but now all parties contribute shares of the *same* value. Suppose the circuit's output wire carries the shared value $[y]$, with party j holding share $R(j)$ for some degree $t-1$ polynomial R with $R(0) = y$. Each party broadcasts their share. Given any t shares, Lagrange interpolation recovers $R(0) = y$. Before this moment, no party knew y; after it, everyone does. This is the only point in the entire protocol where a shared value becomes public.

The communication cost is $O(n^2)$ field elements per multiplication (each party sends one message to each other party). Round complexity equals the circuit's multiplicative depth, since multiplications at the same depth can proceed in parallel.

Garbled circuits

Secret-sharing MPC generalizes naturally to n parties, but requires rounds proportional to circuit depth. Each multiplication forces a round of communication. For deep circuits or high-latency networks, this cost compounds quickly. Yao's garbled circuits take a completely different approach, designed specifically for the **two-party** case. There are no thresholds, no secret sharing, no multiple rounds of interaction. Instead, one round of communication suffices regardless of circuit depth.

The setting is two parties, say Alice and Bob, each holding a private input. Neither trusts the other. They agree on a function f and want to learn $f(x_A, x_B)$ without revealing their inputs to each other. The protocol assigns asymmetric roles: Alice becomes the *garbler*, who encrypts the entire circuit before sending it, and Bob becomes the *evaluator*, who runs the encrypted circuit blindly.

The evaluator needs one label per input wire, but the two parties' inputs arrive through different channels. For the garbler's own input wires, the garbler knows their bits, so they simply send the corre-

sponding labels directly. For the evaluator's input wires, the garbler holds both labels but must not learn which bit the evaluator has. A primitive called *oblivious transfer* (developed later in this chapter) lets the evaluator receive the label matching their bit without the garbler learning which one was chosen. The evaluator learns nothing beyond the final output; the garbler learns nothing about the evaluator's input.

Labels as passwords

If the evaluator must compute on the garbler's circuit without learning what the wires carry, something must replace the raw bits. The idea is to use passwords. Each wire in the circuit carries not a 0 or 1, but a random cryptographic label. For each wire, the garbler creates two labels: one that "means 0" and one that "means 1." The evaluator receives exactly one label per wire, the one corresponding to the actual value, but cannot tell which meaning it carries.

This separation between *holding* a value and *knowing* a value is what makes garbled circuits work. The evaluator holds passwords that encode the computation, but a random 128-bit string looks the same whether it means 0 or 1.

Garbling a single gate

Each gate computes on passwords instead of bits by having the garbler precompute all possible outputs and encrypt them so only the correct one can be recovered.

Consider an AND gate with input wires L (left) and R (right) and output wire O. Suppose Alice (the garbler) holds the left input and Bob (the evaluator) holds the right input. Alice generates all six labels herself, two per wire, each a 128-bit string that doubles as a symmetric encryption key:

- Wire L: labels L_0 and L_1 (meaning "left input is 0" and "left input is 1")
- Wire R: labels R_0 and R_1
- Wire O: labels O_0 and O_1

Alice knows which label corresponds to which bit; the subscript in L_0 is her private bookkeeping. Bob will eventually receive exactly one label per wire: for wire L, Alice sends the label matching her own bit; for wire R, Bob obtains the label matching his bit via oblivious transfer (the primitive introduced above, detailed in its own section below). He ends up holding two labels (one per input wire) but has no way to tell which bit either one represents. He never learns the other label for either wire.

The plain truth table for AND is:

Left	Right	Output
0	0	0
0	1	0
1	0	0
1	1	1

Alice now uses *all* her labels to build the garbled table, covering every possible input combination. She can do this because she created all six labels. The table encodes what the correct output label would be for each pair of inputs, encrypted so that only someone holding the right pair can recover it:

Encrypted Entry
$\text{Enc}_{L_0,R_0}(O_0)$
$\text{Enc}_{L_0,R_1}(O_0)$
$\text{Enc}_{L_1,R_0}(O_0)$
$\text{Enc}_{L_1,R_1}(O_1)$

The encryption $\text{Enc}_{L_a,R_b}(O_c)$ is a symmetric-key operation (AES in practice) that uses both input labels as the key. Only someone who knows *both* L_a and R_b can decrypt the corresponding row.

This table has a flaw in its current form. If the rows stay in this order, the evaluator learns which row they decrypted and hence learns the input bits. The fix is to **randomly shuffle the rows**. After shuffling, the garbled table might look like:

Shuffled Encrypted Entry
$\text{Enc}_{L_1,R_1}(O_1)$
$\text{Enc}_{L_0,R_0}(O_0)$
$\text{Enc}_{L_1,R_0}(O_0)$
$\text{Enc}_{L_0,R_1}(O_0)$

Now Bob holds one label for each input wire. He tries to decrypt each of the four rows using his two labels as the key. Recall that each row was encrypted under a *specific pair* of labels via AES. AES decryption with the wrong key doesn't fail gracefully; it produces random-looking bytes. To tell valid

from garbage, each row includes a small authentication tag (a known padding pattern or checksum) alongside the output label. When Bob decrypts with the correct pair, the tag checks out and he recovers the output label. When he decrypts with the wrong pair, the tag is garbled and he knows to discard the result. Exactly one row matches his labels, so he recovers exactly one output label.

This doesn't leak which inputs were used. Bob knows *a* row succeeded, but the rows are shuffled and he doesn't know what bit his labels represent. The position of the successful row tells him nothing about Alice's input or his own in the context of the truth table.

Hash-indexed tables

Random shuffling forces the evaluator to try all four rows per gate. A more efficient approach uses the hash of the input labels as a row index:

Row Index	Encrypted Entry
$H(L_0, R_0)$	$\mathsf{Enc}_{L_0,R_0}(O_0)$
$H(L_0, R_1)$	$\mathsf{Enc}_{L_0,R_1}(O_0)$
$H(L_1, R_0)$	$\mathsf{Enc}_{L_1,R_0}(O_0)$
$H(L_1, R_1)$	$\mathsf{Enc}_{L_1,R_1}(O_1)$

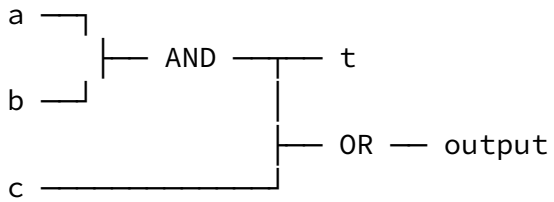
The evaluator, holding labels L_a and R_b, computes $H(L_a, R_b)$ and looks up that row directly. No trial decryptions needed. The hash reveals nothing about which row was accessed since the evaluator doesn't know the other labels to compute their hashes.

This structure scales better. Instead of trying all rows, the evaluator does one hash and one decryption per gate. For circuits with millions of gates, the difference matters.

Chaining gates together

A single gate is not a computation. With either table approach (shuffled or hash-indexed), the evaluator decrypts one entry per gate and obtains an output label. That output label becomes input to the next gate. Labels propagate through the circuit because the garbler ensures consistency: the output labels of one gate are the same labels used as inputs in the next. The evaluator, holding one label per wire, evaluates gate after gate, each time recovering exactly one output label to feed forward.

Example: A tiny circuit. Consider computing $(a \wedge b) \vee c$, which requires an AND gate followed by an OR gate.



The intermediate wire t connects AND's output to OR's input. The garbler:

1. Generates labels for wires a, b, c, t, and $output$ (two labels per wire)
2. Creates a garbled table for AND using a's and b's labels as input keys, encrypting t's labels as outputs
3. Creates a garbled table for OR using t's and c's labels as input keys, encrypting $output$'s labels as outputs
4. Sends both garbled tables to the evaluator

The consistency between gates requires no "enforcement" since the garbler controls construction. The labels t_0 and t_1 are created once, then used in two places: as the encrypted outputs of the AND table, and as the decryption keys indexed in the OR table. When the evaluator decrypts the AND gate and obtains (say) t_0, that exact string appears as an index in the OR table. The garbler wired them together at construction time.

The evaluator:

1. Receives labels for a, b, c (via oblivious transfer for their inputs, directly for the garbler's inputs)
2. Evaluates the AND gate, obtaining a label for t
3. Uses the t label plus the c label to evaluate the OR gate
4. Obtains a label for the output wire

At the final output, the garbler reveals the mapping: "If your output label is X, the result is 0; if it's Y, the result is 1." Only now does the evaluator learn the actual output bit. This isn't a security breach since the whole point is for both parties to learn $f(a, b)$. The protection is that intermediate wire mappings stay hidden, so the evaluator learns only the final answer, not the computation path that produced it.

A concrete walkthrough

Let's trace a complete example to see how Alice (garbler) and Bob (evaluator) actually interact. They want to compute $a \wedge b$ where Alice holds $a = 1$ and Bob holds $b = 0$.

Step 1: Alice creates the garbled circuit (offline, before any communication). Alice generates all labels for all wires, including Bob's input wire. She doesn't know Bob's input, so she creates labels for both possibilities:

- Wire a (Alice's input): $L_0 = \texttt{3a7f...}$, $L_1 = \texttt{9c2b...}$
- Wire b (Bob's input): $R_0 = \texttt{5e81...}$, $R_1 = \texttt{d4a3...}$
- Wire out: $O_0 = \texttt{72f9...}$, $O_1 = \texttt{1b6e...}$

She builds the garbled table, encrypting each output label under the pair of input labels that would produce it:

Input Labels	Output Label	Ciphertext
L_0, R_0	O_0	$\text{Enc}_{\texttt{3a7f...},\texttt{5e81...}}(\texttt{72f9...})$
L_0, R_1	O_0	$\text{Enc}_{\texttt{3a7f...},\texttt{d4a3...}}(\texttt{72f9...})$
L_1, R_0	O_0	$\text{Enc}_{\texttt{9c2b...},\texttt{5e81...}}(\texttt{72f9...})$
L_1, R_1	O_1	$\text{Enc}_{\texttt{9c2b...},\texttt{d4a3...}}(\texttt{1b6e...})$

She randomly shuffles the rows and sends the four ciphertexts to Bob. At this point Bob has the encrypted circuit but no labels. He cannot decrypt anything yet.

Step 2: Bob receives his input labels. Two things happen through different channels:

- *Alice's input*: Alice knows her bit is $a = 1$, so she sends $L_1 = \texttt{9c2b...}$ to Bob. She does not send L_0. Bob receives this label but has no way to tell it corresponds to the bit 1 rather than 0.
- *Bob's input*: Alice holds both R_0 and R_1 but must not learn Bob's bit. Bob knows he wants R_0 (his bit is 0) but cannot tell Alice which one. Via oblivious transfer, Bob receives $R_0 = \texttt{5e81...}$ without Alice learning he chose it, and without Bob learning R_1.

Bob now holds exactly one label per input wire: $\texttt{9c2b...}$ for wire a and $\texttt{5e81...}$ for wire b.

Step 3: Bob evaluates. Bob tries to decrypt each of the four shuffled ciphertexts using his two labels as the key. Each row was encrypted under a *specific pair* of labels. Bob's pair is $(\texttt{9c2b...}, \texttt{5e81...})$. Only the row that was encrypted under exactly this pair, the row corresponding to (L_1, R_0), decrypts successfully, yielding $O_0 = \texttt{72f9...}$. The other three rows, encrypted under different label pairs, produce garbage when Bob tries them. He cannot decrypt them because he doesn't hold L_0 or R_1.

Step 4: Output. Alice reveals the output mapping: "Label $\texttt{72f9...}$ means 0, label $\texttt{1b6e...}$ means 1." Bob sees he holds $\texttt{72f9...}$, so the result is $1 \wedge 0 = 0$.

What did Bob learn? Only the output. He never learned that $\texttt{9c2b...}$ "meant 1" or that Alice's input was 1. He never saw L_0, R_1, or O_1. What did Alice learn? Nothing about Bob's input, because oblivious transfer hid his choice. She knows the result (Bob can share it) but not which of Bob's bits produced it.

Complexity

The basic protocol requires four encryptions per gate (one per truth-table row). An optimization called **Free-XOR** eliminates the garbled table entirely for XOR gates by constraining all label pairs to differ by a global secret Δ; the evaluator simply XORs input labels to obtain the output label with no encryption needed. Since XOR is the most common gate in many circuits, this significantly reduces communication in practice.

Communication is $O(|C|)$, proportional to the circuit size. Computation uses only symmetric-key operations (AES). The protocol runs in constant rounds regardless of circuit depth: one round to send the garbled circuit, one for oblivious transfers.

Oblivious transfer

The garbled circuits walkthrough relied on a primitive we haven't yet built: a way for Bob to receive one of Alice's two labels without Alice learning which one he chose. This is **oblivious transfer** (OT). In its general form, a sender holds two messages m_0 and m_1, a receiver holds a choice bit b, and after the protocol the receiver learns m_b and nothing else while the sender learns nothing about b.

The requirement sounds contradictory. Several constructions make it possible.

Construction from commutative encryption

Imagine an encryption scheme where the order of encryption and decryption doesn't matter:

$$\mathsf{Dec}_b(\mathsf{Dec}_a(\mathsf{Enc}_b(\mathsf{Enc}_a(x)))) = x$$

Exponentiation in a finite group provides exactly this. Encrypt message g with key a by computing g^a. Decrypt by taking an a-th root. The order of encryption doesn't matter since $(g^a)^b = (g^b)^a = g^{ab}$, so either party can decrypt their layer without needing the other to go first.

The OT protocol. Alice has n messages $x_1, \ldots, x_n$. Bob wants x_i without Alice learning i.

1. Alice encrypts all messages with her key a and sends them in order: $\mathsf{Enc}_a(x_1), \ldots, \mathsf{Enc}_a(x_n)$
2. Bob knows he wants the i-th message, so he takes the i-th ciphertext from the list (he can't read it, but he knows its position). He encrypts it with his own key b and sends back $\mathsf{Enc}_b(\mathsf{Enc}_a(x_i))$
3. Alice decrypts with her key, obtaining $\mathsf{Enc}_b(x_i)$, and sends it to Bob
4. Bob decrypts with his key to recover x_i

Bob is protected because Alice sees only a doubly-encrypted blob. She doesn't know Bob's key b, so she can't decrypt it to see which message he chose.

Alice is protected because Bob receives only one singly-encrypted message ($\text{Enc}_b(x_i)$ in step 3). The other $n - 1$ messages remain encrypted under Alice's key, which Bob doesn't have.

Construction from Diffie-Hellman

The commutative encryption approach requires three rounds of communication between Alice and Bob. A construction based on Diffie-Hellman key exchange reduces this to two rounds by exploiting the fact that the receiver's choice bit can be hidden inside a group element.

Work in a group $\mathbb{G}$ of prime order q with generator g. The sender chooses random a and sends $A = g^a$. The receiver embeds their choice bit b into their response: if $b = 0$, choose random k and send $B = g^k$; if $b = 1$, send $B = A \cdot g^k = g^{a+k}$ for random k. Either way, the sender sees a random-looking group element B and cannot tell which case applies.

The sender computes two keys: $K_0 = B^a$ and $K_1 = (B \cdot A^{-1})^a$. Then the sender encrypts both messages, $c_0 = \text{Enc}_{K_0}(m_0)$ and $c_1 = \text{Enc}_{K_1}(m_1)$, and sends both ciphertexts.

The receiver can compute only one key. If $b = 0$, the receiver knows k and can compute $K_0 = A^k = g^{ak}$, which equals B^a since $B = g^k$. But $K_1 = (B/A)^a = g^{(k-a)a}$ requires knowing the discrete log of B/A, which the receiver doesn't have. The receiver decrypts c_0 and learns m_0. If $b = 1$, the situation reverses: the receiver can compute K_1 but not K_0.

The sender sees only B, a random group element that reveals nothing about whether the receiver chose $b = 0$ or $b = 1$.

Both constructions require public-key operations (exponentiations), which is fine for a handful of OTs but problematic when garbled circuits need one OT per input bit. **OT extension** (the IKNP protocol) solves this by using a small number of base OTs (typically 128) to bootstrap an unlimited number of extended OTs using only symmetric-key operations. The amortized cost drops to a few AES calls per OT, making garbled circuits practical even for million-bit inputs.

Mixing protocols

Real computations rarely fit neatly into one paradigm. A machine learning inference might need field arithmetic for the linear layers (where secret-sharing MPC excels) but comparisons for activation functions (where garbled circuits handle more efficiently). The most practical approach switches representations mid-computation, using each paradigm where it performs best.

Modern MPC frameworks formalize this by supporting three representations: arithmetic sharing for field operations, Boolean sharing for bitwise operations and comparisons, and Yao's garbled circuits for complex Boolean functions. Conversion protocols translate between them. Arithmetic-to-Boolean

(A2B) converts additive shares of a field element into XOR-shares of its bit representation. Boolean-to-Arithmetic (B2A) reverses the process, using oblivious transfer to handle the carry bits that arise when interpreting binary as an integer.

The design problem becomes partitioning a computation so that each segment uses its most efficient representation. Deep multiplicative chains favor arithmetic sharing. Complex comparisons favor Boolean or Yao representations. The optimal decomposition is often hand-tuned for applications where performance matters.

MPC-in-the-head

Everything so far has developed MPC as a tool for private computation among real parties. But this is a book about proof systems, and the MPC machinery we've built turns out to produce zero-knowledge proofs through an unexpected route, one that bypasses polynomial commitments, pairings, and algebraic IOPs entirely.

The transformation is called "MPC-in-the-head," and it rests on a symmetry between MPC security and zero-knowledge. In a real MPC protocol, multiple parties compute on secret-shared inputs with the guarantee that no coalition learns more than the output. MPC-in-the-head takes this guarantee and repurposes it: the prover secret-shares the witness among n *imaginary* parties, then simulates the MPC protocol that would compute $R(x, w)$ entirely inside their own mind, playing all n roles. Each simulated party accumulates a "view" consisting of the messages it sent and received, its random tape, and its share of the witness. The prover commits to all n views. What was privacy against colluding parties becomes zero-knowledge against the verifier.

Think of a one-person theater troupe performing a three-character scene. The prover writes out the full script: what Alice said to Bob, what Bob said to Charlie, what Charlie said to Alice. Then they seal each character's script in a separate envelope.

The verifier picks two envelopes at random and checks whether the scripts agree. Do the messages that party i claims to have sent match what party j claims to have received? Did both follow the protocol correctly? Does the output equal 1? If Alice's script says she sent "7" to Bob but Bob's script says he received "9," the inconsistency is caught. By checking different random pairs across repetitions, the verifier catches any forged execution with high probability.

Soundness holds because a cheating prover cannot forge consistency across all pairs of views. If the witness is invalid, the honest MPC would output 0. To fake acceptance, the prover must manufacture views where the protocol appears to output 1, but any inconsistency between a pair of views (mismatched messages, or a party that deviated from the protocol rules) gets caught when the verifier opens that pair. A cheating prover can make *some* pairs consistent, but not all. Each random challenge catches an inconsistent pair with constant probability; repetition amplifies.

Zero-knowledge follows directly from MPC privacy. The number of views the verifier opens must stay below the reconstruction threshold t of the underlying secret sharing scheme. In a t-threshold scheme, any $t - 1$ shares are consistent with every possible secret, so opening $t - 1$ views reveals nothing about the witness. The choice of t controls the tradeoff: higher thresholds allow opening more views (better soundness per repetition) while still preserving zero-knowledge. In the simplest case, 3-party additive sharing requires all 3 shares to reconstruct ($t = 3$), so opening 2 views is safe. Those 2 views suffice to check one pair for consistency, giving soundness error $1/3$ per round, driven down by repetition.

Instantiations

ZKBoo and ZKB++ use 3-party secret sharing. The verifier opens 2 of 3 parties, giving a soundness error of $1/3$ per repetition. These schemes excel at proving knowledge of hash preimages, where the circuit structure is fixed and well-optimized.

Ligero combines MPC-in-the-head with Reed-Solomon codes, achieving proof size $O(\sqrt{n})$ for circuits with n gates. This is sublinear, better than naive approaches though not as succinct as polynomial-based SNARKs.

Limbo and subsequent work push practical performance further, targeting real-world deployment for specific statement classes.

MPC-in-the-head shows that MPC techniques can build proof systems. But MPC also has direct applications of its own, and the most widely deployed is threshold cryptography.

Threshold cryptography

MPC computes arbitrary functions on distributed inputs, but some functions appear so frequently that they deserve specialized protocols. The most important special case is cryptographic key operations. A single signing key or decryption key creates a single point of failure, and the compromise of that one key invalidates all the security built on top of it. MPC provides a way to eliminate that single point by distributing the key itself.

Threshold cryptography applies this idea directly. Instead of a single party holding a signing or decryption key, n parties each hold a share. Any t of them can cooperate to sign or decrypt, but no coalition of fewer than t learns anything about the key. The secret never exists in one place.

Threshold key operations

A cryptocurrency exchange holding billions in assets cannot afford to store a signing key on a single machine. The traditional defense is multisig, where the blockchain verifies t-of-n separate signatures. But multisig reveals the signing structure on-chain and requires protocol-level support. Threshold signatures take a different approach: the n parties hold shares of a single signing key sk, and when t cooperate they produce a single signature indistinguishable from one generated by a solo signer. The blockchain sees nothing unusual. The distribution is invisible.

The reason Schnorr signatures lend themselves to this is linearity. A Schnorr signature has the form $s = k + e \cdot x$ where k is a nonce, e is the challenge hash, and x is the signing key. If parties hold Shamir shares k_i and x_i, they compute partial signatures $s_i = k_i + e \cdot x_i$. Lagrange interpolation reconstructs $s = k + e \cdot x$ exactly, the same reconstruction used throughout this chapter.

FROST builds a complete threshold Schnorr protocol around this observation. In the first phase, parties jointly generate shares of the nonce k using Feldman's verifiable secret sharing (Appendix A), so that each party contributes randomness without anyone learning the full nonce. In the second phase, each party computes their partial signature and the results combine via interpolation. Feldman's verifiability lets parties detect malformed shares during nonce generation, catching cheaters before they can disrupt signing.

FROST requires **synchronous coordination** during nonce generation: all participating signers must be online simultaneously to exchange commitments. If a signer drops offline, the protocol stalls. **ROAST** wraps FROST in an asynchronous coordinator that adaptively selects responsive signers, maintains concurrent sessions, and starts fresh with a different subset when someone times out. The first session to complete produces the signature. ROAST doesn't modify FROST's cryptography; it adds a session management layer that makes threshold signing practical across time zones and unreliable networks.

Threshold ECDSA is harder. ECDSA signatures involve a modular inversion step, $s = k^{-1}(z + r \cdot x)$, and inversion is not linear. Computing it on shared values requires a full MPC protocol for the inversion, adding rounds and computational overhead. Protocols like GG18 and GG20 solve this but at higher cost than FROST.

The same distribution principle applies beyond signing. Threshold decryption (used in e-voting systems like Helios and Belenios via threshold ElGamal) splits a decryption key so that encrypted ballots can only be opened after polls close and only if enough trustees cooperate. The pattern generalizes: any cryptographic operation that depends on a secret key can, in principle, be distributed so that the key never exists in one place.

Practical considerations

The protocols developed in this chapter are theoretically complete. Given enough time and bandwidth, any function can be computed securely. But deploying MPC in practice introduces constraints that the theory abstracts away.

Communication is the bottleneck

MPC and ZK have opposite performance profiles. A ZK prover performs heavy local computation (MSMs, FFTs, hashes) but sends a small proof. An MPC protocol does lightweight computation at each party but exchanges massive amounts of data between them. A ZK prover might spend 10 seconds computing and 10 milliseconds sending; an MPC protocol might spend 10 milliseconds computing and 10 seconds sending. You can run a ZK prover on a single powerful machine, but you can't run high-speed MPC over a slow network.

Within MPC, the binding constraint is usually either bandwidth or latency, and which one dominates determines the protocol choice. If bandwidth is cheap but latency is high (parties on different continents), garbled circuits win because they run in constant rounds despite sending more data. If bandwidth is limited but latency is low (parties in the same data center), secret-sharing MPC wins because each round sends less. The network, not the cryptography, is usually what makes MPC slow.

Preprocessing vs. online

MPC protocols are slow when inputs arrive because every operation pays a cryptographic cost. For latency-sensitive applications like sealed-bid auctions, where parties submit bids that must be processed immediately, this cost is unacceptable. The solution is to separate the computation into two phases. The preprocessing phase generates correlated randomness before the actual inputs are known. Beaver triples for multiplication, OT correlations for garbled circuits, random sharings for masking all fall into this category. The online phase consumes this preprocessed material to compute on the real inputs.

Because none of this preprocessed material depends on the actual inputs, it can be generated during idle time, spreading the heavy cryptographic work across hours or days. When inputs finally arrive, the online phase consumes the stockpiled randomness and runs fast, achieving sub-second latency despite the underlying complexity.

Where does the preprocessing come from? In production systems (cryptocurrency custody platforms, private computation services built on SPDZ), the parties typically generate it themselves via OT extensions or homomorphic encryption, paying the full cost upfront but requiring no trusted party. A simpler alternative is a trusted dealer who generates and distributes the correlated randomness, though

this reintroduces a single point of trust that MPC was designed to eliminate. Hybrid approaches using trusted execution environments as hardware-backed dealers are emerging in the wallet and custody space but remain less established.

Malicious security

Everything so far assumes semi-honest adversaries who follow the protocol faithfully but try to extract information from what they observe. Real deployments often face adversaries who can deviate arbitrarily, sending malformed messages or aborting at strategic moments. Adding security against such adversaries requires mechanisms to detect cheating.

For secret-sharing MPC, the main tool is authentication. The SPDZ protocol attaches a Message Authentication Code (MAC) to each shared value. When shares are combined or reconstructed, the MACs are verified. A cheating party who modifies a share will fail the MAC check with overwhelming probability. The SPDZ preprocessing includes authenticated Beaver triples so that the online phase can verify multiplications respect the triple structure. Recent work has brought the communication cost of malicious SPDZ close to the semi-honest baseline, narrowing a gap that was once a factor of two.

For garbled circuits, the problem is different. The semi-honest protocol assumes the garbler constructs the circuit correctly, but a malicious garbler could create a circuit that computes the wrong function, leaking information about the evaluator's input. Early solutions used **cut-and-choose**, where the garbler creates dozens of independent garbled circuits and the evaluator randomly selects some to verify and the rest to evaluate. This works but is expensive. Modern protocols use **authenticated garbling**, which achieves malicious security with a single garbled circuit by attaching authentication tags to each wire label, reducing the overhead substantially.

In practice, malicious security is standard for high-value operations. Cryptocurrency custody platforms (Fireblocks, Coinbase) use malicious-secure threshold signature protocols, since a compromised signing ceremony could mean direct financial loss. General-purpose malicious-secure MPC remains more expensive and is less common in production, though the cost gap continues to shrink.

Key takeaways

1. **MPC eliminates trusted third parties.** Any function computable by a circuit can be computed jointly by mutually distrustful parties, revealing only the output. Security is defined through simulation: whatever a corrupt coalition observes, it could have generated from its own inputs and the output alone.

2. **MPC solves the "who runs the prover?" problem.** When a witness is too sensitive for any single party, secret-sharing it among multiple proving servers lets them jointly compute a ZK proof without any server learning the witness.

3. **Two paradigms with different tradeoffs.** Secret-sharing MPC (BGW) handles n parties with free linear operations but rounds proportional to multiplicative depth. Garbled circuits achieve constant rounds for two parties but communicate proportional to circuit size. The network, not the cryptography, usually determines which wins.

4. **MPC-in-the-head bridges MPC and ZK.** Simulate an MPC protocol inside the prover's mind, commit to all party views, let the verifier audit a random subset. MPC privacy becomes zero-knowledge; MPC correctness becomes soundness. This yields proof systems (ZKBoo, Ligero) that bypass polynomial machinery entirely.

5. **Threshold cryptography distributes key operations.** Secret-share a signing or decryption key among n parties so that any t can operate but fewer than t learn nothing. FROST makes threshold Schnorr practical; ROAST adds asynchrony. This is the most widely deployed application of MPC.

6. **Malicious security is production-ready for high-value operations.** SPDZ authenticates shares with MACs; authenticated garbling verifies circuit construction. Cryptocurrency custody platforms use malicious-secure threshold signatures as standard, while general-purpose malicious MPC continues to close the cost gap with semi-honest protocols.

Chapter 26: Frontiers and Open Problems

In 1900, Lord Kelvin told the British Association for the Advancement of Science that physics was essentially complete. Only "two small clouds" remained on the horizon: the failure of the Michelson-Morley experiment to detect the luminiferous ether, and the inability of classical theory to predict the spectrum of blackbody radiation. Those two clouds became special relativity and quantum mechanics. Kelvin had mistaken a plateau for a summit.

In 2020, SNARKs felt similarly settled. Groth16 for minimal proofs, PLONK for universal setups. The trade-offs seemed fixed, the design space mapped. Then came lookups (2020), folding schemes (2021), and binary field techniques (2023). Each opened territory that the previous framework couldn't reach.

Small fields and Binius

Every proof system in this book operates over large prime fields, typically 254-bit or 256-bit elements. But most real-world data is small: booleans, bytes, 32-bit integers. Representing a single bit as a 256-bit field element wastes 255 bits of capacity. The waste is expensive. Field multiplications dominate prover time, and each multiplication operates on the full 256 bits even when the meaningful data is a single bit. For bit-level operations like hashing, AES, or bitwise logic, the overhead approaches 256×.

Binius attacks this problem directly by working over binary fields $\mathbb{F}_{2^k}$ where field elements are actual k-bit strings. A boolean is a 1-bit field element. A byte is an 8-bit field element. No padding, no waste.

The arithmetic of binary fields differs from prime fields. Addition is XOR (free in hardware). Multiplication uses polynomial arithmetic over $\mathbb{F}_2$. There are no "negative" elements; the field characteristic is 2. Binary fields lack the convenient structure of prime-order groups, but Binius recovers efficiency through protocol design that exploits the tower structure of binary extensions.

Tower architecture

Rather than a single large field, Binius organizes computation in a tower of nested extensions: $\mathbb{F}_2 \subset \mathbb{F}_{2^2} \subset \mathbb{F}_{2^4} \subset \mathbb{F}_{2^8} \subset \ldots$ up to $\mathbb{F}_{2^{128}}$ for cryptographic security. Each level doubles the extension degree, and every element of a smaller field is automatically an element of every larger field above it. A bit in $\mathbb{F}_2$ is a valid element of $\mathbb{F}_{2^{128}}$; it's just a very special one.

This nesting enables a natural optimization. Witness data lives in the smallest field that fits: bits stay in $\mathbb{F}_2$, bytes in $\mathbb{F}_{2^8}$, 32-bit integers in $\mathbb{F}_{2^{32}}$. Arithmetic happens at the appropriate level. Only when the verifier's random challenges enter does computation lift to the full tower height. The 256× overhead vanishes.

(This "tower" is unrelated to the "tower of proofs" in Chapter 23's recursion discussion. There, "tower" refers to recursive proof composition $\pi_1 \to \pi_2 \to \pi_3$. Here, it refers to nested field extensions.)

Protocol components

GKR-based multiplication (multilinear). Binary field multiplication is polynomial multiplication modulo an irreducible polynomial. Rather than encoding this as constraints, Binius uses the GKR protocol (Chapter 7) to verify multiplications via sum-check over multilinear extensions. The prover commits only to inputs and outputs; intermediate multiplication steps are checked interactively.

FRI over binary fields (univariate). For polynomial commitments, Binius adapts FRI to binary domains. The standard FRI folding doesn't directly transfer since the squaring map $x \mapsto x^2$ is not 2-to-1 on binary fields as it is over roots of unity. FRI-Binius instead uses *subspace vanishing polynomials* with an *additive NTT* to achieve the necessary folding structure, enabling commitment to polynomials over tiny fields like $\mathbb{F}_2$ with no embedding overhead.

Binius thus straddles both PIOP paradigms from Chapter 22: sum-check-based (multilinear) for the computation layer, FRI-based (univariate) for the commitment layer.

Where it stands

For bit-intensive computations, Binius achieves order-of-magnitude improvements:

Operation	Traditional (256-bit field)	Binius
SHA-256 hash	~25,000 constraints	~5,000 constraints
AES block	~10,000 constraints	~1,000 constraints
Bitwise AND	1 constraint + range check	1 native operation

Fewer constraints mean smaller polynomials, faster transforms, smaller proofs. But the path from theory to deployment has been instructive about the real tradeoffs.

Irreducible (the primary Binius implementation team) archived the original Binius codebase in September 2025 and replaced it with **Binius64**, a simplified design that operates natively over 64-bit words. The pivot reflected lessons from production experience: the original tower architecture was too general for practical use. Binius64 retains the core ideas (binary field towers, GKR multiplication, FRI-Binius commitments) but targets CPU-based client-side proving rather than competing as a general-purpose zkVM. Early benchmarks show Binius64 on multi-threaded CPUs outperforming SP1 and R0VM on GPUs by roughly 5× for hash-based signature aggregation.

The tradeoffs that motivated the pivot remain relevant for any binary-field system. Binius achieves faster proving at the cost of larger proofs and slower verification than prime-field FRI. Recursion is harder because verifying a binary-field proof inside another binary-field proof requires embedding the arithmetic, and the algebraic structure that makes Binius fast for computation makes it awkward for recursive self-verification. Zero knowledge itself was not yet implemented as of the Binius64 launch, listed as the top priority for subsequent releases.

The benefits are also workload-dependent. Binius shines for bit-intensive operations (hashing, AES, bitwise logic) but the advantage shrinks for 32/64-bit arithmetic, memory operations, or control flow.

The Binius64 team's focus on signature aggregation and client-side proving suggests binary fields may find their niche in specialized components rather than full VM execution, composed with prime-field provers via the techniques from Chapter 23.

The broader principle holds regardless of Binius's specific trajectory: matching the proof system's field to the computation's natural representation eliminates artificial overhead.

Field representation is only one axis of adaptation. Another looms larger: the cryptographic assumptions themselves.

Post-quantum SNARKs

Every system in Part IV of this book (Groth16, PLONK, KZG-based constructions) rests on the hardness of discrete logarithm or elliptic curve problems. These will break once cryptographically relevant quantum computers exist, because they all share **hidden periodic structure in abelian groups** that Shor's algorithm exploits via the quantum Fourier transform.

Timeline estimates have compressed sharply. Recent results reducing the qubit requirements for breaking elliptic curve cryptography (from millions to hundreds of thousands under newer architectures) have moved several expert assessments into the 5-10 year range. Google targets 2029 for full post-quantum migration; the Global Risk Institute rates a cryptographically relevant quantum computer as "quite possible" within 10 years. Even conservative government planning horizons have shortened to 15-25 years. For infrastructure with long lifespans (financial systems, identity, archival signatures) the question is no longer whether to prepare but how fast.

Paths forward

Hash-based systems. STARKs and FRI rely only on collision-resistant hashing. Hash functions resist Shor (no hidden periodic structure). The best known quantum attack on hashes is Grover's algorithm, which searches an unstructured space of N elements in $\sqrt{N}$ steps instead of N. This is a quadratic speedup, not an exponential one, so doubling the hash output (e.g., from 128-bit to 256-bit security) neutralizes it entirely. Beyond FRI, WHIR (EUROCRYPT 2025) provides a hash-based multilinear PCS with faster verification, giving sum-check-based provers (Chapter 22) a post-quantum commitment scheme without switching to the univariate/STARK paradigm. The Ethereum Foundation's lean stack (Chapter 24) began as Whirlaway, which combined WHIR with SuperSpartan over KoalaBear. Its successor leanVM moved to a 192-bit binary tower field with WHIR and Ligerito commitments, replacing Poseidon with standard hashes. Hash-based systems are the current practical choice for post-quantum proofs. Their proof sizes are larger than pairing-based alternatives, but they work today.

Lattice-based commitments. A lattice commitment to a vector s with small entries is $\mathrm{Com}(s) = As \bmod q$ for a public random matrix A over a module ring. Binding reduces to Module-SIS, the problem of finding a short nonzero vector in the kernel of A. Hiding, when the application needs it, reduces to Module-LWE. Neither problem carries the hidden abelian periodicity that the quantum Fourier transform extracts. The commitment is linear in the same way a Pedersen commitment is, so it supports the manipulations that sum-check protocols perform on committed polynomials. Linearity has a price in norm growth. Every linear combination enlarges the committed vector, so the protocol must periodically prove that norms stay below the bound at which Module-SIS remains hard.

The line of constructions moved from theory to deployment in four years. LaBRADOR (2022) gave compact lattice proofs for R1CS. Greyhound (2024) turned it into a polynomial commitment with proofs of about 53 KB at $N = 2^{30}$ and a $\sqrt{N}$ verifier. Hachi (eprint 2026/156) moved the sum-check from the cyclotomic ring to an extension field through ring switching, keeping 55 KB proofs while verifying 12.5× faster than Greyhound. Akita (eprint 2026/1983) lowered the verifier to $\tilde{O}(N^{1/K})$ for any fixed $K \geq 2$, with proofs of 61 to 72 KB and 128-byte commitments. In September 2026 Jolt replaced Dory with Akita. The resulting zkVM is post-quantum under Module-SIS at 128 bits, produces proofs under 100 KB and runs at more than two million RV64 cycles per second on a laptop CPU, roughly twice the curve-based version. The gain has a structural source. Akita's commitment cost scales with the nonzero entries of the committed vector, which preserves the small-value property of Chapter 21, while leaving elliptic curves lets Jolt work over a 128-bit field instead of a 256-bit one (Chapter 24 discusses the implications for SNARK selection).

Symmetric-key SNARKs. The MPC-in-the-head paradigm (Chapter 25) builds proofs entirely from hash-based commitments, with no algebraic assumptions at all. Ligero improved this with linear-time proving via interleaved Reed-Solomon codes, but constants remain large (10-100× slower than algebraic SNARKs). Security reduces to collision resistance of the hash function.

The proximity gap conjectures

The concrete security of FRI, STIR and WHIR rests on proximity gaps for Reed-Solomon codes. A proximity gap says that for an affine line of words $u_0 + z\,u_1$, either almost every point on the line is close to the code or almost none is (Chapter 10). Every deployed hash-based system derives its query count from how far that statement is assumed to hold. Up to the Johnson radius $1 - \sqrt{\rho}$ it is a theorem. Between the Johnson radius and capacity $1 - \rho$ it was a conjecture that deployed systems relied on anyway, because at rate $1/4$ it halves the number of queries.

Between November 2025 and April 2026 the conjecture failed in its strong form. Crites and Stewart (eprint 2025/2046) disproved the up-to-capacity versions of three conjectures at once, namely correlated agreement, the mutual correlated agreement used by WHIR and the list-decodability behind DEEP-FRI. Ben-Sasson, Carmon, Haböck, Kopparty and Saraf (eprint 2025/2055) improved

the provable bounds up to the Johnson radius while constructing Reed-Solomon codes over smooth subgroups of prime fields where the gap fails near capacity. Krachun, Kazanin and Haböck (eprint 2026/782) then exhibited an explicit affine line with $2^{\Omega(1/\eta)}$ bad points at distance $1 - \rho - \eta$.

No attack on a deployed system followed. What disappeared was the justification for the parameters, which the Ethereum Foundation estimates can turn an advertised 100 bits into roughly 80. The ecosystem responded by reporting only provable security. The Foundation's soundcalc tool now models the unique-decoding and Johnson regimes alone. Production zkVMs such as SP1 Hypercube, OpenVM 2.0 and ZisK publish provable figures between 100 and 128 bits with proofs of 270 to 530 KiB.

Open problems

Three interrelated challenges define this frontier. The first concerns lattices, which have caught up with hash-based commitments on prover speed but not on confidence. Their Module-SIS parameters carry far less cryptanalytic history than those of ML-KEM or ML-DSA. Verifying a lattice proof inside another proof, the step on which recursion depends, is still largely unexplored. The second is the gap between the Johnson radius and capacity for Reed-Solomon codes. Closing it with new theorems, or with codes whose proximity gaps are provable up to capacity, would recover the shorter proofs that the conjectures used to promise. The third is the transition itself. Hybrid systems secure against both classical and quantum adversaries should not pay twice the cost, yet most deployed chains still end in a Groth16 or PLONK wrapper that a quantum adversary could forge.

The post-quantum transition will reshape the SNARK landscape, but it operates on a timescale of years to decades. A different revolution is already underway.

zkVMs

Every proof system we've studied requires translating the computation into a constraint system: R1CS, AIR, PLONKish gates. This translation is a specialized craft. Experts hand-optimize circuits for months; a single bug invalidates the work. The barrier to entry is enormous.

zkVMs invert this relationship. Instead of adapting computations to proof systems, adapt proof systems to computations. Compile any program to a standard virtual machine (RISC-V, EVM, WASM) and prove correct execution. The zkVM handles memory, branching, loops, function calls. Write your logic in Rust. Compile to the target ISA. Prove execution. No circuit engineering required.

The current race

The zkVM landscape has stratified into distinct architectural approaches.

SP1 (Succinct). The most widely adopted zkVM, powering OP Succinct rollups, Polygon's Agglayer, and Celestia's bridge to Ethereum. Cross-table lookup architecture with a precompile system that accelerates common operations (signature verification, hashing) by 5-10× over raw RISC-V. SP1 Hypercube moved from univariate STARKs to multilinear polynomials committed through the jagged PCS (Chapter 21), which still rests on Reed-Solomon codes. It reached mainnet in February 2026 and proves 99.7% of Ethereum L1 blocks in under 12 seconds on 16 RTX 5090 GPUs. Succinct presents it as the first general-purpose hash-based zkVM whose parameters avoid the proximity gap conjectures, a claim that rests on working in the provable unique-decoding regime rather than on leaving codes behind (the post-quantum section above explains why that regime matters).

ZKsync Airbender. STARK-based over Mersenne31 with a custom DEEP-ALI implementation, open-sourced and live on mainnet. Claims the highest single-GPU throughput: 21.8 MHz (million cycles proven per second) on an H100, roughly 6× faster than competing zkVMs. Proves an average Ethereum block in ~17 seconds on a single H100 before recursion, ~35 seconds end-to-end. Designed as the proving backbone for the ZK Stack, with proving costs under $0.0001 per transfer.

RISC Zero. STARK-based with FRI commitments over BabyBear, targeting RISC-V. Uses continuations to split large computations into bounded segments (~10^6 cycles), proves each independently, then aggregates via recursion. Final proofs wrap in Groth16 for cheap on-chain verification. R0VM 2.0 (April 2025) reduced Ethereum block proving from 35 minutes to 44 seconds. The Boundless network provides a decentralized proof marketplace.

Note on continuations: Instead of proving the entire computation history at each step, continuations prove only the current segment plus a commitment to the previous segment's final state. This lets you pause and resume computation at arbitrary points, bounding peak prover memory regardless of total computation length.

Jolt (a16z). Built entirely on multilinear polynomials and the Lasso lookup argument (Chapter 21). Implements CPU instructions via lookups into structured tables rather than hand-crafted constraints. With the Twist and Shout memory-checking arguments integrated in 2025, the curve-based version reached over 1 million RISC-V cycles per second on a 32-core CPU with ~50KB proofs. In September 2026 the Dory commitment gave way to the lattice-based Akita, making Jolt the first post-quantum zkVM built on lattices. That version proves more than 2 million RV64 cycles per second on a laptop CPU and more than 10 million on a laptop GPU, with proofs under 100 KB. The lattice mode does not yet compose with Jolt's zero-knowledge mode, so it currently delivers succinctness without privacy. A streaming prover is under development for arbitrarily long executions in under 2GB RAM. Jolt does not yet support recursion or continuation, which limits direct comparison with SP1 and RISC Zero on long computations.

Zisk (Polygon spinoff). Spun out of Polygon's zkEVM team (led by co-founder Jordi Baylina) in June 2025, with all Polygon zkEVM IP transferred. Built on RISC-V 64, designed for low-latency distributed

proving with a 1.5 GHz zkVM execution engine, GPU-optimized code, and advanced aggregation circuits.

The convergence across these systems is notable: multilinear polynomials displacing univariate STARKs, real-time Ethereum proving as the benchmark target, precompiles for common cryptographic operations. Techniques developed for one system transfer rapidly to others.

Design patterns from production

Beyond the specific systems, several design patterns have emerged that generalize across implementations.

Physical CPUs distinguish registers (fast, few) from memory (slow, large). In ZK circuits this distinction vanishes because both register access and memory access are polynomial lookups with identical cost. Valida exploited this by eliminating general-purpose registers entirely in favor of a stack machine, reducing per-cycle constraint count. The deeper lesson is that zkVM architectures should not inherit assumptions from physical hardware that have no analogue in the proving system.

Long computations face a memory wall: proving 10^9 cycles requires holding intermediate state for 10^9 steps. Segment-based proving (pioneered by RISC Zero) splits execution into bounded segments of roughly 10^6 cycles, proves each independently, then aggregates via recursive composition. Peak prover memory stays bounded regardless of total computation length.

Memory consistency can be verified through Merkle trees (hashing inside the circuit, expensive) or through algebraic challenges that accumulate memory operations into fingerprint polynomials and verify consistency via Schwartz-Zippel. The challenge approach, formalized in Twist-and-Shout (Chapter 21) and used in SP1 and Jolt, relies only on field arithmetic and is 10× faster or more for memory-heavy workloads.

Finally, zkVMs expose *precompiles* for operations that appear frequently and have specialized efficient circuits (SHA256, Keccak, ECDSA, pairings). These run 10-100× faster than interpreted execution at the cost of additional engineering complexity per precompile. The ECDSA verification bottleneck has also driven adoption of EdDSA over "embedded" curves like BabyJubJub, whose base field matches the scalar field of the outer proving curve so that signature verification becomes native field arithmetic.

Open problems

The overhead gap is the defining challenge. Current zkVMs run 100-1000× slower than native computation; the near-term target is 10×. Where does this overhead come from, and which parts are compressible?

Part of it is inherent: every operation must produce a cryptographic trace, and that trace must be committed and checked. But much of the overhead is structural. Memory is one source. A 4GB address space means 2^{32} potential cells, far too many to commit individually. Virtual polynomial techniques (Chapter 21) help, but scaling to gigabytes of working memory remains open. Precompile selection is another. Current systems hand-pick which operations get dedicated circuits based on blockchain workloads. General-purpose proving may need different choices, and automating precompile discovery (profiling hot operations and generating specialized circuits) would change the economics of zkVM design. Sequentiality is a third source. Most zkVMs execute instructions one at a time, each depending on the previous state. Proving parallel programs efficiently, or even exploiting prover-side parallelism for sequential programs, remains largely unexplored.

These problems are connected. Memory overhead limits the computations you can prove. Precompile overhead limits the operations worth proving. Sequential execution limits the hardware you can exploit. Solving any one of them shifts the bottleneck to the others.

But speed means nothing if the proofs are wrong.

Formal verification

A soundness bug in a ZK system is unlike most software bugs. A crash announces itself; a soundness bug operates in silence. An attacker forges proofs, the verifier accepts, the system behaves as though everything is fine. By the time the compromise is discovered, the damage is done. High-profile vulnerabilities have been found in deployed systems: missing constraint checks that allowed invalid witnesses to pass, incorrect range assumptions that permitted overflow attacks, field confusion bugs where values were interpreted in the wrong field.

Several defenses are gaining traction. Verified compilers prove that compilation from a high-level circuit language to low-level constraints preserves semantics. Machine-checked soundness proofs (in Coq, Lean, Isabelle) establish that the protocol is sound by construction. OpenVM's RV32IM extension was formally verified in Lean by Nethermind Research in early 2026, and SP1 Hypercube's core RISC-V chips have been verified in Lean as well. Static analysis tools detect common vulnerability patterns before deployment: unconstrained variables, degree violations, missing range checks.

The persistent challenge is the gaps between verified components. You might verify the compiler but not the runtime, the protocol but not the implementation, the circuit but not the witness generator. Bugs hide at the boundaries. End-to-end verification, from source code to final proof, remains open. So does verification of optimized implementations: the fastest provers use hand-tuned assembly and GPU kernels that are inherently hard to reason about formally.

Formal verification addresses correctness. The remaining frontiers are systems-level problems: making proofs faster to generate, cheaper to verify at scale, and applicable to demanding workloads.

Deployment frontiers

Several bottlenecks sit between correct proof systems and practical deployment. They are less glamorous than new cryptographic constructions but increasingly determine what is actually buildable.

Hardware and memory. Prover computation (MSMs, NTTs, hashing) is massively parallel, making GPUs 10-100× faster than CPUs for these workloads. But the binding constraint is increasingly memory bandwidth rather than arithmetic throughput. Large circuits require gigabytes of data, and memory transfer between CPU and GPU often exceeds computation time. Proof systems designed around GPU memory hierarchy rather than adapted to it after the fact would look very different from what we have.

Witness generation. Academic benchmarks report "prover time" as the cryptographic work (commitments, sum-check, polynomial evaluations), but witness generation (computing all $O(n)$ intermediate values for an n-gate circuit) often takes longer. A paper might report "proving takes 10 seconds" while silently omitting that witness generation took 60 seconds. The two scale differently: proving parallelizes across GPUs while witness generation is often sequential and memory-bound. For zkVMs, the execution trace already exists; translating it into the format the prover needs is the expensive step.

Aggregation. A rollup processing millions of transactions generates millions of proofs. Verifying them individually costs $O(n)$ time. Recursive aggregation (Chapter 23) collapses n proofs into one but adds prover overhead. Proof compression (wrapping a STARK in a Groth16 proof) is already standard. The open targets are incremental aggregation (adding proof $n+1$ without recomputing the aggregate) and cross-system aggregation (combining proofs from different proof systems into a single attestation).

Privacy-preserving ML. The most demanding application on the horizon. Proof of inference for small neural networks (thousands of parameters) is tractable but carries 100×+ overhead. Proof of training at GPT scale (billions of parameters, trillions of operations) remains far out of reach. Non-linearities (ReLU, softmax) are expensive in arithmetic circuits; "ZK-friendly" model architectures with amenable activation functions could help but remain speculative. FHE offers a complementary path where the server computes on encrypted data without seeing the inputs (Chapter 27), with hybrid ZK+FHE approaches under active research.

Theoretical foundations

The engineering frontiers above rest on theoretical questions that remain open.

We lack tight lower bounds on proof size for a given soundness error. We have constructions, but no matching impossibility results. Perhaps dramatically better systems are possible; perhaps we're close to optimal. The answer determines whether to keep searching or focus on engineering.

Deep recursion may degrade knowledge soundness. Current security reductions lose tightness with each recursive layer. Whether this is inherent or an artifact of our proof techniques matters directly for the recursive composition that underpins modern zkVMs.

The assumptions underlying SNARKs (knowledge assumptions, generic group model) are stronger than standard cryptographic assumptions. Whether they hold is a matter of ongoing debate. Resolving this either validates the foundations or forces a rethinking of what we build on.

SNARK techniques also have implications beyond cryptography. Progress on proof compression connects to circuit lower bounds, algebraic computation, and the structure of NP. These are among the deepest problems in theoretical computer science.

The field is young enough that systems considered optimal five years ago have already been superseded. Some patterns are visible: post-quantum concerns driving hash-based systems, zkVMs becoming the default abstraction, multilinear polynomials displacing univariate encodings. But ZK proofs are part of a larger landscape that includes fully homomorphic encryption, program obfuscation, and the convergence of programmable cryptography. The next chapter steps back to see where ZK fits in that broader picture.

Key takeaways

1. **Binary fields eliminate representation overhead.** Binius and its successor Binius64 prove that matching the field to the data (bits as bits, bytes as bytes) removes the 256× penalty of encoding small values in large prime fields. The tower architecture enables this without sacrificing cryptographic security.

2. **Post-quantum migration is accelerating.** Hash-based systems (STARKs, WHIR) work today. Lattice-based commitments reached a production zkVM in September 2026, when Jolt adopted Akita and became faster than its curve-based predecessor. The fall of the up-to-capacity proximity gap conjectures pushed hash-based systems to report provable security only, at the cost of larger proofs. The Ethereum Foundation targets 128-bit provable security by end of 2026.

3. **zkVMs are converging on multilinear proofs.** SP1 Hypercube, OpenVM 2.0, Jolt and the Ethereum Foundation's leanVM have moved to multilinear polynomials and sum-check, while Airbender, ZisK and RISC Zero push STARK-based approaches to their limits. Real-time Ethereum block proving is now achieved by multiple teams.

4. **Formal verification is catching up to deployment.** Machine-checked proofs in Lean now cover core zkVM components (OpenVM, SP1 Hypercube). The persistent gap is end-to-end verification from source code to final proof, especially for optimized GPU implementations.

5. **The bottleneck is shifting from cryptography to systems engineering.** Witness generation, memory bandwidth, precompile selection, and proof aggregation increasingly determine real-world performance more than the choice of proof system.

6. **Tight lower bounds remain unknown.** We lack matching impossibility results for proof size, and deep recursion may degrade knowledge soundness in ways we cannot yet quantify. The theoretical foundations are solid but incomplete.

Chapter 27: ZK in the cryptographic landscape

In 1943, a resistance fighter in occupied France needs to send a message to London. She writes it in cipher, slips it into a dead letter drop, and waits. A courier retrieves it, carries it across the Channel, and a cryptographer at Bletchley Park decrypts it. The message travels safely because no one who intercepts it can read it.

For the next fifty years, this was cryptography's entire mission: move secrets from A to B without anyone in between learning them. Telegraph, radio, internet. The medium changed; the problem stayed the same. Encrypt, transmit, decrypt. A message sealed or opened, a secret stored or revealed.

Then computers stopped being message carriers and became *thinkers*. The question changed. It was no longer enough to ask "can I send a secret?" Now we needed to ask: "can I *use* a secret without exposing it?"

This is the dream of **programmable cryptography**: secure *computation* on secrets. The dream took many forms. "Can I prove I know a secret without revealing it?" led to zero-knowledge proofs. "Can we compute together while keeping our inputs private?" led to secure multiparty computation. "Can I encrypt data so someone else can compute on it?" led to fully homomorphic encryption. "Can I publish a program that reveals nothing about how it works?" led to program obfuscation.

These are different philosophies about who computes, who learns, and what trust means. For decades they developed in parallel, each with its own community, its own breakthroughs, its own brick walls.

This book taught you the path that arrived first: zero-knowledge proofs. ZK reached general practicality before the others. MPC is deployed for high-value operations like threshold signing (Chapter 25). FHE works for narrow applications and is improving rapidly. Program obfuscation remains theoretical. Understanding *why* ZK progressed fastest illuminates both the landscape and the road ahead.

Why ZK arrived first

The most important asymmetry is structural: the prover works in the clear. In ZK, the expensive cryptographic operations happen *after* the computation, not during it. The prover computes at native speed, then invests work in generating a proof. In fully homomorphic encryption (developed in the next section), every arithmetic operation carries cryptographic overhead because the data stays encrypted throughout. In program obfuscation, the program itself becomes the cryptographic object. This difference compounds across millions of operations.

ZK also benefited from mathematical serendipity. SNARKs exploit polynomial arithmetic over finite fields, exactly what elliptic curves, pairings, and FFTs handle efficiently. The tools developed for other purposes (error-correcting codes, number theory, algebraic geometry) turned out to fit the ZK problem well. FHE and obfuscation involve noise management and lattice arithmetic that fight against efficient computation rather than harmonizing with it.

The theory developed steadily over thirty years. The path from GMR (1985) to PCPs (1992) to IOPs (2016) to practical SNARKs (2016-2020) was long but each step built on the previous. The sum-check protocol from 1991 became the heart of modern systems. Polynomial commitments from 2010 enabled succinctness. The pieces accumulated until they clicked together.

Finally, blockchain created urgent demand. Scalability, privacy, trustless verification: billions of dollars flowed into ZK research. The ecosystem grew rapidly. FHE has applications but no comparable catalyst. Program obfuscation has no applications that couldn't wait until it works, a chicken-and-egg problem that starves it of engineering investment.

MPC also reached practicality, though with different trade-offs. Chapter 25 covers MPC in depth. This chapter focuses on the two dreams that remain partially unfulfilled: computing on encrypted data, and making programs incomprehensible.

Computing on ciphertexts

In 1978, Rivest, Adleman, and Dertouzos asked whether an encryption scheme could support computation on ciphertexts. They called it a "privacy homomorphism": encrypt data, compute on the ciphertexts, decrypt and get the correct result, all without the server ever seeing the plaintext. The question was whether this could work for *arbitrary* computations, not just a narrow class.

For thirty years, the answer was partial. RSA turned out to be multiplicatively homomorphic (the product of ciphertexts decrypts to the product of plaintexts) but couldn't do addition. Paillier (1999) achieved additive homomorphism but couldn't do multiplication. ElGamal was multiplicative too. Every scheme could do one operation or the other, never both. Since addition and multiplication

together are enough to compute any function, the gap between "partially homomorphic" and "fully homomorphic" was the gap between a curiosity and a revolution.

Craig Gentry's 2009 thesis closed that gap.

Learning with errors

Modern FHE rests on the **Learning With Errors (LWE)** problem, which admits two readings. Algebraically, LWE says that solving a system of linear equations becomes intractable when each equation carries a small random error. This is the view that matters for building encryption: the noise is what makes ciphertexts indistinguishable.

Geometrically, LWE is a lattice problem. A *lattice* is a regular grid of points in high-dimensional space (integer combinations of basis vectors), and recovering the secret from noisy equations amounts to finding a close lattice point through the noise. This is the view that matters for *analyzing* security: hardness reductions from worst-case lattice problems (finding shortest vectors, closest vectors) are what give us confidence that LWE resists both classical and quantum attacks. No quantum algorithm is known to solve these lattice problems efficiently.

LWE and its structured variants (Ring-LWE, Module-LWE) underlie the NIST post-quantum encryption standard ML-KEM. As Chapter 26 notes, recent constructions like Hachi are bringing lattice-based polynomial commitments into the ZK landscape as well.

LWE enables encryption by encoding a message bit as a large shift in a noisy inner product. The secret key is a vector $\vec{s}$. To encrypt a bit $m \in \{0, 1\}$, pick a random vector $\vec{a}$, pick small noise e, and compute $b = \langle \vec{a}, \vec{s} \rangle + e + m \cdot \lfloor q/2 \rfloor$. The ciphertext is $(\vec{a}, b)$. The bit m creates a large gap (adding $q/2$ or nothing); the noise obscures the exact value but not which half of the range we're in. Decryption subtracts the mask $\langle \vec{a}, \vec{s} \rangle$ and rounds. An attacker who doesn't know $\vec{s}$ faces the LWE problem.

The noise problem

The difficulty of FHE lies in how operations affect the error. A concrete example makes this vivid.

Say our modulus is $q = 1000$. We encode bit 0 as values near 0, bit 1 as values near 500 (that's $q/2$). Fresh ciphertexts have noise around ± 10. Decryption asks: "Is this value closer to 0 or to 500?"

Encrypt two bits, both equal to 1. Ciphertext c_1 decrypts to $500 + 7 = 507$ (the 7 is noise). Ciphertext c_2 decrypts to $500 - 4 = 496$. Both decrypt correctly since 507 and 496 are closer to 500 than to 0.

Addition is safe. The noises add: $(507 + 496) \mod 1000 = 3$. Noise is $7 + (-4) = 3$, still small. But multiplication is where trouble starts. Multiplying ciphertexts multiplies the noises: after one multiplication, noise ≈ 28; after two, ≈ 280; after three, ≈ 2800. The safety margin is only 250 (values

must stay closer to their target than to the alternative). After a few multiplications, noise overwhelms signal and decryption returns garbage.

This is the **noise budget**: every FHE scheme has a limit on how much computation can be performed before the ciphertext becomes useless. Addition is cheap (noise grows linearly). Multiplication is expensive (noise grows multiplicatively, becoming exponential in circuit depth).

Bootstrapping

The noise budget imposes a depth limit on computation. Gentry's breakthrough was **bootstrapping**: a way to reset the noise without ever decrypting in the clear.

Return to the example. The ciphertext encoding bit 1 has accumulated noise of 280. One more multiplication and decryption fails. The noise must come down while the message stays intact, and the plaintext must never be exposed.

The naive fix would be to decrypt the ciphertext and re-encrypt it fresh. But that exposes the plaintext, defeating the purpose. Bootstrapping achieves the same effect without ever leaving ciphertext space, by exploiting the fact that decryption is itself a computation (subtract the mask, round, output the bit). If we run this computation *homomorphically* on an encrypted copy of the secret key, the rounding step absorbs the old noise internally and the output emerges as a fresh ciphertext.

Concretely, publish an encryption of the secret key, Enc$(\vec{s})$, as a public parameter. Given the noisy ciphertext c, treat it as public data (it is already encrypted) and evaluate the decryption circuit homomorphically with Enc$(\vec{s})$ as the key input. Inside the homomorphic evaluation, the rounding step absorbs the old noise (780 rounds to 500, giving 1 correctly). Since the key was encrypted, the output is also encrypted: Enc(1) with fresh noise of perhaps 50 instead of the accumulated 280. The plaintext was never exposed.

This only works if the decryption circuit is shallow enough that running it homomorphically doesn't exhaust the noise budget it is trying to restore. Gentry's construction designs decryption to be "bootstrappable," but the cost is significant; early implementations took minutes per bootstrap. The payoff is that there is no longer a depth limit. Compute until noise grows dangerous, bootstrap to refresh, continue. Any computation becomes possible, one refresh at a time.

Current state

The fifteen years since Gentry's thesis have produced both real improvements and real deployments. FHE is no longer a research curiosity. Apple ships it on a billion phones (Live Caller ID Lookup, iOS 18) for private database queries. Microsoft uses it in Edge's Password Monitor for private set intersection

against breach databases. Google's Private Join and Compute handles encrypted advertising attribution. CryptoLab partners with Samsung for encrypted health analytics on Galaxy devices. These deployments share a pattern: the computations are shallow (one or two multiplication depths), the data is structured, and the workloads are embarrassingly parallel.

Three scheme families cover different workload shapes. **TFHE** optimizes for Boolean circuits through "programmable bootstrapping," where the bootstrap itself computes a function. On modern CPUs, gate evaluation takes roughly 10ms; on GPUs (Zama's TFHE-rs on H100 hardware), bootstrapping has dropped below 1ms. **BGV/BFV** optimize for integer arithmetic by batching thousands of values into a single ciphertext with SIMD-style parallelism. **CKKS** accepts approximate arithmetic on fixed-point real numbers, trading small errors for efficiency in workloads like ML inference where exact precision isn't needed.

For the shallow, parallel workloads that current deployments target, FHE overhead is manageable. For general computation the overhead remains 10^3 to 10^4 times native, depending on the workload and scheme. Early implementations were a million times slower; the trajectory is encouraging. Hardware acceleration through custom FPGAs and ASICs (programs like DARPA's DPRIVE) could deliver another 100-1000× improvement. But the overhead may have irreducible components: noise management and ciphertext expansion impose costs that shrink with better engineering but may never vanish entirely.

Libraries like Microsoft SEAL, OpenFHE, and Zama's Concrete have made FHE accessible to developers. NIST has begun a standardization process for FHE through its Multi-Party Threshold Schemes call, signaling institutional readiness for wider adoption.

Program obfuscation

Program obfuscation is the most ambitious dream of programmable cryptography. Where the rest of cryptography computes on secrets, obfuscation makes the *programs themselves* into secrets.

Virtual black-box obfuscation

The strongest notion is **virtual black-box (VBB) obfuscation**: transform a program's source code into a form that still runs correctly but reveals nothing about *how* it works. A password checker would still accept the correct password and reject all others, but someone reading the obfuscated code could not figure out what the secret password is.

Formally, an obfuscator $\mathcal{O}$ satisfies VBB if for any program P:

1. **Functionality**: $\mathcal{O}(P)(x) = P(x)$ for all inputs x

2. **Black-box security**: Anything efficiently computable from $\mathcal{O}(P)$ is also efficiently computable given only oracle access to P

Having the obfuscated code gives you no advantage over having a locked box that runs the program. The code is in front of you, but it's as opaque as a black box.

The impossibility result

In 2001, Barak, Goldreich, Impagliazzo, Rudich, Sahai, Vadhan, and Yang proved that **VBB obfuscation is impossible** in general. Some programs are inherently "unobfuscatable." The proof constructs a pair of programs P_0 and P_1 that have identical input-output behavior on almost all inputs but can be distinguished by examining their code. The construction exploits self-reference: program P_b behaves normally on most inputs, but if given *its own code* as input, it outputs b.

$$P_b(\mathcal{O}(P_b)) = b$$

Any obfuscation of P_b must output b when fed itself, revealing which program it came from. No amount of code transformation can hide this.

Indistinguishability obfuscation

A weaker notion survived. **Indistinguishability obfuscation (iO)** guarantees only that if programs P_0 and P_1 compute the *same function* (identical outputs on all inputs), then their obfuscations are computationally indistinguishable:

$$\mathcal{O}(P_0) \approx_c \mathcal{O}(P_1)$$

This seems weak. You're only hiding implementation details, not the function itself. The power comes from what you can hide inside equivalent programs.

Consider two programs that both output "Hello, World!":

```
Program A: print("Hello, World!")

Program B:
    secret_key = 0x7a3f...  # 256-bit key, embedded in the code
    if sha256(input) == target:
        return decrypt(secret_key, ciphertext)
    print("Hello, World!")
```

Program B has a secret key hidden inside it. On every normal input, it behaves identically to Program A. But if you find an input whose hash matches `target`, it decrypts and returns a hidden message. These programs compute the same function (assuming finding the hash preimage is computationally infeasible), so by iO their obfuscations are indistinguishable. The secret key is *in the code*, but no one can extract it. The obfuscated program is indistinguishable from an obfuscation of the trivial Program A, which contains no secrets at all.

With efficient iO, you could build almost any cryptographic primitive. The most striking is *witness encryption*: encrypt a message so that only someone who knows a solution to a puzzle can decrypt it. Not a specific person with a specific key, but *anyone* who can solve the puzzle.

$$\mathsf{WE.Enc}(\text{statement}, m) \to c \qquad \mathsf{WE.Dec}(c, \text{witness}) \to m$$

Witness encryption has a precise duality with zero-knowledge. A ZK proof says "I know a witness for statement x" without revealing it. Witness encryption says "only someone who knows a witness can read this" without specifying who. Both are parameterized by an NP statement. ZK proves *about* the statement; WE encrypts *to* it.

Beyond witness encryption, iO enables *functional encryption* (keys that compute $f(m)$ from encrypted m without learning m itself), *deniable encryption* (produce fake randomness that makes a ciphertext look like it encrypts a different message), and many other primitives. iO acts as a universal building block: given iO, you can construct almost any cryptographic tool. The constraint is not imagination but efficiency.

Construction and costs

In 2021, Jain, Lin, and Sahai constructed iO from well-founded assumptions (variants of LWE and related problems). The theoretical question was settled: iO exists. The construction uses branching programs as the computational model, encoding state transitions in matrix operations obscured by algebraic noise.

For years, all known constructions were exponentially slow: obfuscating a circuit of size n required operations scaling as $2^{O(n)}$, making anything beyond toy circuits infeasible. That began to change in 2025 with **Diamond iO** (eprint 2025/236), a lattice-based construction that replaces the costly recursive functional-encryption bootstrapping of earlier schemes with direct matrix operations. Diamond iO is simple enough to implement, and the Machina iO team produced the first end-to-end benchmarks: obfuscating the simplest possible circuit (depth 0, a single input bit, no multiplications) already takes about 16 minutes and produces a 6.3 GB program; at depth 10, obfuscation takes two hours and produces 20 GB. Evaluation of the obfuscated program takes minutes. These numbers are

far from practical for general use, but they are *finite* rather than cosmological. The same could be said of early FHE implementations circa 2011.

One way to sidestep the overhead is to obfuscate as little as possible. The Diamond iO team suggests obfuscating only a small constant-depth circuit whose job is to verify a ZK proof and decrypt a ciphertext, then letting FHE handle the actual application logic. The obfuscated piece stays tiny; iO contributes only what it uniquely provides (hiding the decryption key). The trajectory from "impossible to implement" to "first benchmarks" took four years. How far the next four take us is an open question.

A separate line of work has extended obfuscation to quantum programs. At FOCS 2025, Er-Cheng Tang received the Machtey Award for the first quantum state obfuscation scheme for unitary quantum programs, opening a direction that classical iO cannot address.

Convergence

The boundaries between ZK, MPC, FHE, and obfuscation are dissolving as researchers combine techniques.

The most natural combination is zkFHE. A server computes on encrypted data using FHE, but how does the client know the server computed correctly? The server generates a ZK proof of correct FHE evaluation. The client verifies without decrypting intermediate results, getting both privacy and verifiability in one protocol.

MPC and ZK compose similarly. Multiple parties compute together (Chapter 25) while ZK proves they followed the protocol honestly without revealing individual contributions. Threshold signatures, distributed key management, collaborative computation with verification: the primitives compose naturally.

Even the line between proving and computing is blurring. The folding and accumulation techniques from Chapter 23 let incrementally verifiable computation fold claims together, deferring expensive proof work. ZK handles verification without revelation. MPC enables joint computation. FHE supports outsourced computation on secrets. Each occupies a niche; together they cover territory no single approach could reach.

Trusted Execution Environments (TEEs) like Intel SGX and ARM TrustZone offer a non-cryptographic alternative: hardware isolation at near-native speed, but requiring trust in the hardware manufacturer. Side-channel attacks have repeatedly compromised their guarantees. The cryptographic approaches avoid this trust assumption at the cost of computational overhead.

The landscape at a glance

Approach	Who computes?	Who learns result?	Trust assumption	Status
ZK	Prover	Verifier	Soundness of proofs	Practical
MPC	All parties jointly	All parties	Threshold honesty	Practical (threshold signing, custody)
FHE	Untrusted server	Client only	Encryption security	Deployed for narrow workloads (~1000× general overhead)
iO	Anyone	Anyone	Obfuscation security	First implementations (far from practical)

Key takeaways

1. **ZK's structural advantage is that the prover works in the clear.** The cryptographic cost comes *after* computation, not during it. FHE pays per operation; iO pays per program gate. This asymmetry, combined with algebraic serendipity and blockchain funding, explains why ZK reached general practicality first.
2. **FHE is deployed for shallow, parallel workloads.** Apple, Microsoft, and Google ship FHE in production. Bootstrapping enables arbitrary-depth computation by homomorphically evaluating decryption. The overhead (~1000× for general computation, sub-millisecond per gate on GPU for TFHE) continues to shrink but may have irreducible components.
3. **iO moved from theory to first implementation.** VBB obfuscation is impossible (Barak et al. 2001), but iO exists (Jain-Lin-Sahai 2021). Diamond iO (2025) produced the first benchmarks. The programs are gigabytes and take hours to obfuscate, but the gap between "impossible"

and “merely impractical” is where progress begins.

4. **Trust models determine tool selection.** ZK: the prover sees data, the verifier learns only validity. MPC: parties jointly compute, no one sees others’ inputs. FHE: the server computes blindly, the client holds the decryption key. The choice depends on who you trust and what you’re hiding from whom.
5. **The primitives compose.** zkFHE gives encrypted computation with verifiable correctness. MPC + ZK proves honest protocol execution. iO + FHE lets you obfuscate a tiny verifier and outsource computation. No single approach covers the full landscape; together they do.

Appendix A: Cryptographic primitives

This appendix collects cryptographic building blocks used throughout the book but not central to the SNARK narrative. These primitives appear in trusted setups, commitment schemes, and protocol constructions.

Mathematical background

Finite fields

A **finite field** $\mathbb{F}_p$ (for prime p) is the set $\{0, 1, \dots, p-1\}$ with addition and multiplication modulo p. Every nonzero element has a multiplicative inverse.

- The multiplicative group $\mathbb{F}_p^*$ has order $p-1$
- **Fermat’s Little Theorem**: For $a \neq 0$, $a^{p-1} = 1$. Thus $a^{-1} = a^{p-2}$.
- **Primitive roots**: There exists $g \in \mathbb{F}_p^*$ such that $\{g^0, g^1, \dots, g^{p-2}\} = \mathbb{F}_p^*$

Extension fields $\mathbb{F}_{p^k}$ arise by adjoining roots of irreducible polynomials. Elements are degree-$(k-1)$ polynomials over $\mathbb{F}_p$, with multiplication modulo the irreducible polynomial. SNARK-friendly fields often have $p \approx 2^{254}$ for 128-bit security.

Roots of unity: If $n \mid (p-1)$, there exist n-th roots of unity ω satisfying $\omega^n = 1$. These enable FFT-based polynomial multiplication.

Elliptic curves

An **elliptic curve** over $\mathbb{F}_p$ is the set of points $(x, y) \in \mathbb{F}_p^2$ satisfying

$$y^2 = x^3 + ax + b$$

plus a "point at infinity" $\mathcal{O}$ serving as identity.

Points form an abelian group under a geometric addition rule. For distinct points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

The group order $|E(\mathbb{F}_p)|$ is approximately p (Hasse's theorem: $|p + 1 - |E|| \leq 2\sqrt{p}$).

Given P and $Q = kP$, finding k is the **discrete log problem**, believed hard for well-chosen curves. Computing kP for scalar k uses double-and-add, taking $O(\log k)$ group operations.

The Weierstrass form $y^2 = x^3 + ax + b$ is standard, but other forms offer advantages. **Montgomery curves** ($By^2 = x^3 + Ax^2 + x$) enable constant-time scalar multiplication via the Montgomery ladder. **Twisted Edwards curves** ($ax^2 + y^2 = 1 + dx^2y^2$) have unified addition formulas (the same formula works for doubling), making them efficient and resistant to side-channel attacks. BabyJubjub and Jubjub are twisted Edwards curves.

Bilinear pairings

A **pairing** is a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ between elliptic curve groups satisfying:

- **Bilinearity**: $e(aP, bQ) = e(P, Q)^{ab}$
- **Non-degeneracy**: If P and Q are generators, $e(P, Q)$ generates $\mathbb{G}_T$
- **Efficiency**: Computable in polynomial time

Pairings enable "multiplication in the exponent": given g^a and g^b, you can't compute g^{ab} directly, but $e(g^a, g^b) = e(g, g)^{ab}$ moves the product to a different group. KZG commitments use pairings to verify polynomial evaluations: the verifier checks $e([f(s)], [1]) = e([q(s)], [s - z]) \cdot e([f(z)], [1])$ without knowing s.

Not all curves support efficient pairings. BN254 and BLS12-381 are specifically designed for this purpose.

Discrete log assumptions

The security of elliptic curve cryptography rests on a hierarchy of assumptions:

- **Discrete Log Problem (DLP)**: Given P and $Q = kP$, find k.
- **Computational Diffie-Hellman (CDH)**: Given P, aP, and bP, compute abP.
- **Decisional Diffie-Hellman (DDH)**: Distinguish (P, aP, bP, abP) from (P, aP, bP, cP) for random c.

In pairing groups, DDH is easy (check via pairing), but CDH is still believed hard. This is the **gap Diffie-Hellman** setting that KZG exploits.

Secure random sampling

Many protocols require random field elements sampled uniformly from $\mathbb{F}_p$.

Modulo bias

A common implementation generates random bytes, interprets them as an integer, and takes the result modulo p.

```
x = random_bytes(32)  # 256 bits
r = int(x) mod p
```

This introduces bias. If $2^{256} \mod p \neq 0$, some residues are more likely than others. To sample from $\{0, 1, \ldots, 9\}$ using a random byte (0-255): values 0-5 appear with probability $26/256$ (26 preimages each) while values 6-9 appear with probability $25/256$ (25 preimages each). The bias is small but potentially exploitable over many samples.

Rejection sampling

Generate candidates and reject those outside an unbiased range.

```
repeat:
    x = random_bytes(32)
    if x < p * floor(2^256 / p):
        return x mod p
```

This ensures each residue has equal probability. Expected iterations: < 2 when p is close to a power of 2.

Hashing to field elements

When deriving field elements from structured data (Fiat-Shamir challenges, randomness beacons):

1. Hash the input: $h = H(\text{data})$

2. Interpret as integer and reduce modulo p
3. Or use a domain-specific "hash-to-field" function (RFC 9380)

The hash output should be larger than p (e.g., 512 bits for a 256-bit field) to minimize bias.

Nothing-up-my-sleeve (NUMS) constructions

Sometimes protocols require public constants that "couldn't have been chosen maliciously." If a constant c is needed (e.g., a generator, a hash input), how do we convince others it wasn't chosen to create a trapdoor?

The NUMS technique derives the constant from a public, unpredictable source: digits of π, e, or $\sqrt{2}$; hashes of fixed strings like $c = H(\text{"nothing up my sleeve"})$; or sequential integers ("Point number 1", "Point number 2", etc.).

In a Powers of Tau ceremony, the initial toxic waste τ should be derived via NUMS:

$$\tau_0 = H(\text{beacon hash} \| \text{round number})$$

Each participant then randomizes: $\tau_i = \tau_{i-1} \cdot r_i$ where r_i is their secret randomness.

Shamir's secret sharing

Distribute a secret s among n parties such that any t can reconstruct but $t-1$ learn nothing.

Construction

Work over a finite field $\mathbb{F}_p$ with $p > n$.

Sharing (by dealer):

1. Choose random polynomial $P(X) = s + a_1 X + a_2 X^2 + \cdots + a_{t-1} X^{t-1}$
2. The secret is $P(0) = s$
3. Give party i the share $s_i = P(i)$

Reconstruction (by any t parties):

1. Collect t shares: $(i_1, s_{i_1}), \ldots, (i_t, s_{i_t})$
2. Use Lagrange interpolation to find $P(0)$:

$$s = P(0) = \sum_{j=1}^{t} s_{i_j} \cdot \prod_{k \neq j} \frac{-i_k}{i_j - i_k}$$

Security

Any $t-1$ shares are consistent with every possible secret. The polynomial through $t-1$ points can have any value at 0. This is information-theoretic: even computationally unbounded adversaries learn nothing.

The threshold exhibits a sharp discontinuity. With $t-1$ shares, the entropy of the secret is $\log_2 p$ bits (maximum uncertainty). With t shares, the entropy drops to zero (the secret is uniquely determined). There is no intermediate state where information leaks gradually as shares accumulate.

Worked example

Secret $s = 10$, threshold $t = 2$, parties $n = 3$, field $\mathbb{F}_{17}$.

Polynomial: $P(X) = 10 + 5X$ (random coefficient $a_1 = 5$).

Shares:

- Party 1: $P(1) = 15$
- Party 2: $P(2) = 20 \equiv 3 \pmod{17}$
- Party 3: $P(3) = 25 \equiv 8 \pmod{17}$

Reconstruction from parties 1 and 3:

$$s = 15 \cdot \frac{-3}{1-3} + 8 \cdot \frac{-1}{3-1} = 15 \cdot \frac{-3}{-2} + 8 \cdot \frac{-1}{2}$$

In $\mathbb{F}_{17}$: $(-2)^{-1} = 8, 2^{-1} = 9$.

$$s = 15 \cdot (-3) \cdot 8 + 8 \cdot (-1) \cdot 9 = 15 \cdot 11 + 8 \cdot 8 = 165 + 64 \equiv 10 \pmod{17}$$

Feldman's verifiable secret sharing

Standard Shamir assumes an honest dealer. A malicious dealer could distribute inconsistent shares that don't reconstruct to any secret, or that reconstruct to different secrets for different groups. Feldman's VSS solves this by broadcasting commitments to the polynomial coefficients.

Setup: Group $\mathbb{G}$ of prime order q, generator g.

Sharing:

1. Dealer chooses $P(X) = s + a_1 X + \cdots + a_{t-1} X^{t-1}$
2. Dealer broadcasts commitments: $C_0 = g^s, C_1 = g^{a_1}, \ldots, C_{t-1} = g^{a_{t-1}}$

3. Dealer sends share $s_i = P(i)$ to party i

Verification: Party i checks:

$$g^{s_i} = \prod_{j=0}^{t-1} C_j^{i^j}$$

This holds because:

$$\prod_{j=0}^{t-1} C_j^{i^j} = \prod_{j=0}^{t-1} g^{a_j \cdot i^j} = g^{\sum_j a_j i^j} = g^{P(i)} = g^{s_i}$$

If verification fails, party i broadcasts a complaint. Honest parties can detect malicious dealers.

Feldman VSS reveals g^s (the "encrypted" secret). This may leak partial information (e.g., equality with other secrets). Pedersen VSS adds blinding for perfect hiding.

Hash functions in zero-knowledge

SNARKs use hash functions for Fiat-Shamir challenges, Merkle tree commitments (FRI, STARKs), and random oracle instantiation.

The circuit cost problem

Standard hashes (SHA-256, BLAKE3) are expensive in circuits. SHA-256 uses operations that CPUs handle efficiently (32-bit XOR, bit rotations, boolean operations), but these are catastrophic inside arithmetic circuits over prime fields.

A single XOR in an arithmetic circuit requires decomposing each input into bits (one constraint per bit to enforce booleanity: $b_i \cdot (1 - b_i) = 0$), then computing the XOR bit-by-bit as $a_i + b_i - 2 \cdot a_i \cdot b_i$. A 256-bit XOR that takes one CPU cycle becomes hundreds of constraints. SHA-256 costs roughly 25,000-30,000 constraints per invocation. A depth-20 Merkle tree (about 1 million leaves) requires 20 hashes, totaling 500,000-600,000 constraints just for hashing.

Algebraic hashes

Algebraically-friendly hashes use only native field operations: addition and multiplication. No bit operations at all.

Poseidon is the dominant choice. It uses a sponge construction with a permutation built from three layers per round:

1. **Add round constants**: Breaks symmetry. Cost: 0 constraints (additions are linear).

2. **S-box**: Apply x^α (typically x^5) for nonlinearity. Cost: 2 constraints per S-box.
3. **MDS matrix**: Multiply state by a maximum-distance-separable matrix for diffusion. Cost: 0 constraints (linear operations absorbed into next nonlinear step).

The **HADES** design uses *full rounds* (S-box on all state elements) at the beginning and end for statistical security, and *partial rounds* (S-box on only one element) in the middle for algebraic security. A typical configuration of 8 full rounds and 56 partial rounds totals ~160 constraints per hash, compared to ~25,000 for SHA-256.

Other algebraic hashes include **MiMC** (2016, simpler but higher multiplicative depth, largely superseded), **Rescue** (alternates S-box and inverse S-box), and **Poseidon2** (2023, same constraints as Poseidon but 3× faster witness generation).

Security considerations

Algebraic hashes have less cryptanalytic history than SHA-256. Poseidon has received sustained analysis (Grassi et al. 2019, subsequent Gröbner basis attacks), and current parameters include security margins. Conservative applications may use more rounds than the minimum recommended or fall back to SHA-256 for security-critical operations outside circuits.

Poseidon is *not* for general-purpose hashing. For files, passwords, or data at rest, use SHA-256 or BLAKE3. Poseidon is a specialized tool for proving hash computations inside ZK circuits.

Modular arithmetic implementation

SNARK provers spend most time in modular arithmetic. Implementation details matter enormously.

Montgomery multiplication

Standard modular multiplication computes $a \cdot b$, then divides by p and takes the remainder. Montgomery representation avoids the expensive division by storing $\bar{a} = a \cdot R \mod p$ where $R = 2^k$ for convenient k. The Montgomery product $\bar{c} = \bar{a} \cdot \bar{b} \cdot R^{-1} \mod p$ replaces division by p with division by R, which is a bit shift (essentially free in hardware). The conversion overhead is amortized over many operations.

SIMD and parallelism

Modern CPUs have vector instructions (AVX-256, AVX-512) that parallelize field arithmetic: four 64-bit multiplications simultaneously, or eight 32-bit multiplications simultaneously. GPU arithmetic parallelizes across thousands of threads. SNARK provers achieve 10-100× speedup from GPU acceleration.

Random beacons

Some applications require public randomness that cannot be predicted before a deadline, cannot be biased by any party, and is verifiable by all.

Blockchain-based beacons use the hash of a future block as randomness. The block hash is unpredictable until mined, but miners can withhold blocks to manipulate the beacon (at cost of block rewards).

VDF-based beacons use a Verifiable Delay Function that requires sequential time T to compute but is fast to verify. A beacon seeds a VDF; by the time the output is known, manipulation is impossible.

Multi-party beacons have multiple parties contribute randomness. If any one is honest, the result is unbiased. The simple protocol has each party commit to a random value, then all reveal; the beacon is the hash of all revealed values. The risk is that the last revealer sees the beacon before revealing; commit-then-reveal with timeouts mitigates this.

Elliptic curves in zero-knowledge

Not all elliptic curves work for SNARKs. Pairing-based systems (Groth16, KZG commitments) require curves with efficiently computable bilinear pairings. The choice of curve determines the scalar field, which in turn determines what field elements your circuit operates over.

BN254 (alt_bn128)

A Barreto-Naehrig curve with embedding degree 12 and the workhorse of practical SNARKs.

- **Scalar field**: $r \approx 2^{254}$ (254 bits)
- **Security**: Originally claimed ~128 bits, now estimated at ~100 bits due to advances in discrete log attacks on extension fields
- **Status**: Still widely used (Ethereum precompiles, most zkEVMs, Groth16 deployments)

BN254's scalar field prime:

$$r = 21888242871839275222246405745257275088548364400416034343698204186575808495617$$

Ethereum has native precompiles for BN254 operations (ecAdd, ecMul, ecPairing), making it the default for on-chain verification.

BLS12-381

A Barreto-Lynn-Scott curve with embedding degree 12. Designed to provide ~128-bit security even with improved attacks.

- **Scalar field**: $r \approx 2^{255}$ (255 bits)
- **Security**: Solid 128-bit security margin
- **Status**: Used in newer systems (Zcash Sapling, Ethereum 2.0 signatures, PLONK implementations)

BLS12-381 is larger than BN254 (larger field, more expensive operations) but future-proof against known attack improvements.

Embedded curves

Pairing curves have large coordinates. Computing BN254 point addition inside a BN254 circuit is expensive because the base field is ~254 bits, requiring big-integer arithmetic in constraints. The solution is to use a *different* curve whose base field matches the SNARK's scalar field.

BabyJubjub is a twisted Edwards curve defined over BN254's scalar field. Points on BabyJubjub have coordinates in $\mathbb{F}_r$ where r is BN254's scalar field order. BabyJubjub operations are native arithmetic in BN254 circuits, with point addition costing ~6 constraints instead of thousands. EdDSA signature verification becomes practical inside circuits.

Jubjub plays the same role for BLS12-381: a twisted Edwards curve over BLS12-381's scalar field.

The pattern: an "embedded" or "inner" curve lives over the outer curve's scalar field, enabling efficient in-circuit elliptic curve operations.

Curve cycles

For recursive SNARKs, you need to verify a proof *inside* a circuit. If both the proof system and the circuit use the same field, the verifier does arithmetic in the scalar field while the proof's group operations are over the base field.

A **curve cycle** pairs two curves where each curve's base field equals the other's scalar field. Pasta curves (Pallas and Vesta) form such a cycle, enabling efficient recursion in systems like Halo 2.

Curve	Base Field	Scalar Field
Pallas	$\mathbb{F}_p$	$\mathbb{F}_q$
Vesta	$\mathbb{F}_q$	$\mathbb{F}_p$

Prove over Pallas, verify in a Vesta circuit; prove over Vesta, verify in a Pallas circuit. The cycle enables indefinite recursion. The BN254/Grumpkin cycle matters for Ethereum developers: since BN254 is pre-compiled on Ethereum, systems like Aztec use this cycle to verify recursive proofs on-chain cheaply.

Group operations

Elliptic curve SNARKs rely on fast group operations.

Point addition (affine)

Given points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ on curve $y^2 = x^3 + ax + b$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

$$x_3 = \lambda^2 - x_1 - x_2$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

Affine coordinates require field inversion (expensive).

Projective coordinates

Represent (x, y) as $(X : Y : Z)$ where $x = X/Z$, $y = Y/Z$. Point addition and doubling use only multiplication, avoiding inversion until final conversion back to affine. **Jacobian coordinates** $(X : Y : Z)$ with $x = X/Z^2$, $y = Y/Z^3$ are optimized for repeated doubling.

Multi-scalar multiplication (MSM)

Compute $\sum_i s_i \cdot G_i$ for scalars s_i and points G_i.

Pippenger's algorithm groups scalars by their bit patterns, reducing work from $O(n \cdot \log |s|)$ to $O(n/\log n \cdot \log |s|)$.

MSM dominates KZG commitment time. Parallelization and GPU implementation are necessary for practical SNARKs.

Appendix B: Historical Timeline

The development of zero-knowledge proofs and succinct arguments spans four decades. This timeline traces the key theoretical breakthroughs and practical systems that shaped the field.

Theoretical foundations (1985-1992)

1985: GMR (Interactive Proofs and Zero-Knowledge) Goldwasser, Micali, and Rackoff introduce interactive proofs and define zero-knowledge. The paper "The Knowledge Complexity of Interactive Proof Systems" establishes the foundational concepts: completeness, soundness, and the simulation paradigm for zero-knowledge. A conceptual revolution: proving something is true without revealing why it's true.

1986: Fiat-Shamir Transform Fiat and Shamir show how to eliminate interaction by replacing verifier randomness with hash function outputs. The prover computes challenges as hashes of the transcript, producing a non-interactive proof. The random oracle model provides the security analysis.

1986-1987: GMW (Zero-Knowledge for All of NP) Goldreich, Micali, and Wigderson prove that every NP language has a zero-knowledge proof, assuming one-way functions exist. The graph 3-coloring construction is theoretical (impractical for real use) but establishes the surprising generality of zero-knowledge.

1990: LFKN (Algebraic Interactive Proofs) Lund, Fortnow, Karloff, and Nisan develop the sum-check protocol for proving claims about polynomial sums. This algebraic technique becomes the cornerstone of later efficient protocols. The paper shows #P $\subseteq$ IP.

1991: MIP = NEXP (Babai, Fortnow, Lund) Multi-prover interactive proofs, where the verifier interrogates two non-communicating provers, can verify nondeterministic exponential time computations. The result establishes the surprising power of multiple provers and connects to PCP theory.

1992: IP = PSPACE (Shamir) Shamir proves that interactive proofs can verify exactly the problems solvable in polynomial space. The result uses multilinear extensions and sum-check, establishing the power of interaction + randomness.

1992: The PCP Theorem (AS, ALMSS) Arora and Safra (AS) prove NP $\subseteq$ PCP[log n, polylog n]; Arora, Lund, Motwani, Sudan, and Szegedy (ALMSS) strengthen this to NP = PCP[log n, O(1)]. Every NP statement has a proof where the verifier reads only a constant number of bits. The theoretical foundation for succinct arguments.

1992: Kilian's Succinct Arguments Kilian shows how to compile PCPs using Merkle trees and collision-resistant hashing. The prover commits to the PCP, the verifier queries random bits, and the prover opens with authentication paths. This is the first succinct argument for NP, with proof size polylogarithmic in the computation.

The ZK winter (1992-2008)

For sixteen years, zero-knowledge proofs remained impractical. The PCP theorem promised succinct proofs, but the constructions had astronomical overhead ($O(n^{10})$ blowup in early versions). Researchers refined PCP constructions, developed new proof composition techniques, and explored connections to coding theory, but there were no implementations and no urgency.

Two developments changed that. In 2008, Goldwasser, Kalai, and Rothblum published GKR, showing that sum-check could verify arithmetic circuits with manageable overhead. In 2009, Bitcoin launched, creating a financial ecosystem with urgent demand for privacy, scalability, and trustless verification. The tool and the demand arrived at roughly the same time.

Path to practical systems (2008-2016)

2008: GKR (Efficient Verification of Arithmetic Circuits) Goldwasser, Kalai, and Rothblum develop a protocol for verifying layered arithmetic circuits using sum-check. The prover does polynomial work; the verifier does polylogarithmic work. Later refinements by Cormode, Mitzenmacher, and Thaler make it truly practical.

2010: Groth10 (First Practical Pairing-Based SNARK) Groth introduces succinct arguments using pairings, building on ideas from linear PCPs. The construction enables constant-size proofs verified with a constant number of pairings.

2010: Kate-Zaverucha-Goldberg (KZG) Commitments The KZG paper formalizes polynomial commitments using pairings. Commit to a polynomial with one group element; prove evaluations with one group element. This becomes the cryptographic engine for most practical SNARKs.

2013: Pinocchio Parno, Howell, Gentry, and Raykova build the first complete, implemented SNARK for general computation. C programs compile to circuits; circuits compile to proofs. Real-world verification becomes possible.

2014: Zcash Begins Development The Zerocoin team, building on Pinocchio, starts developing what becomes Zcash, the first major deployment of zkSNARKs for cryptocurrency privacy.

2016: Groth16 (The Speed King) Groth publishes an optimized SNARK with the smallest known proofs (3 group elements) and fastest verification (3 pairings). Despite requiring per-circuit trusted setup, Groth16 becomes the de facto standard for production systems.

2016: ZKBoo (MPC-in-the-Head) Giacomelli, Madsen, and Orlandi publish ZKBoo, the first practical implementation of "MPC-in-the-head." The prover simulates a multiparty computation internally, then lets the verifier audit random subsets. ZKBoo proves that zero-knowledge could be built entirely from symmetric primitives (hashes), offering a third path distinct from pairings (Groth16) and polynomial commitments (STARKs).

The scaling era (2017-2020)

2017: STARKs (Transparent Scalable Arguments) Ben-Sasson, Bentov, Horesh, and Riabzev introduce STARKs (Scalable Transparent ARguments of Knowledge). Based on FRI and hash functions, STARKs require no trusted setup and resist quantum attacks. Proofs are larger but prover time is quasi-linear.

2018: Bulletproofs (Logarithmic Range Proofs) Bünz, Bootle, Boneh, Poelstra, Wuille, and Maxwell develop Bulletproofs using inner-product arguments. Logarithmic proof size for range proofs without trusted setup. Adopted by Monero for confidential transactions.

2018: Zcash Sapling Upgrade Zcash launches Sapling with improved Groth16-based proofs. Proving time drops from ~40 seconds to ~7 seconds on mobile devices.

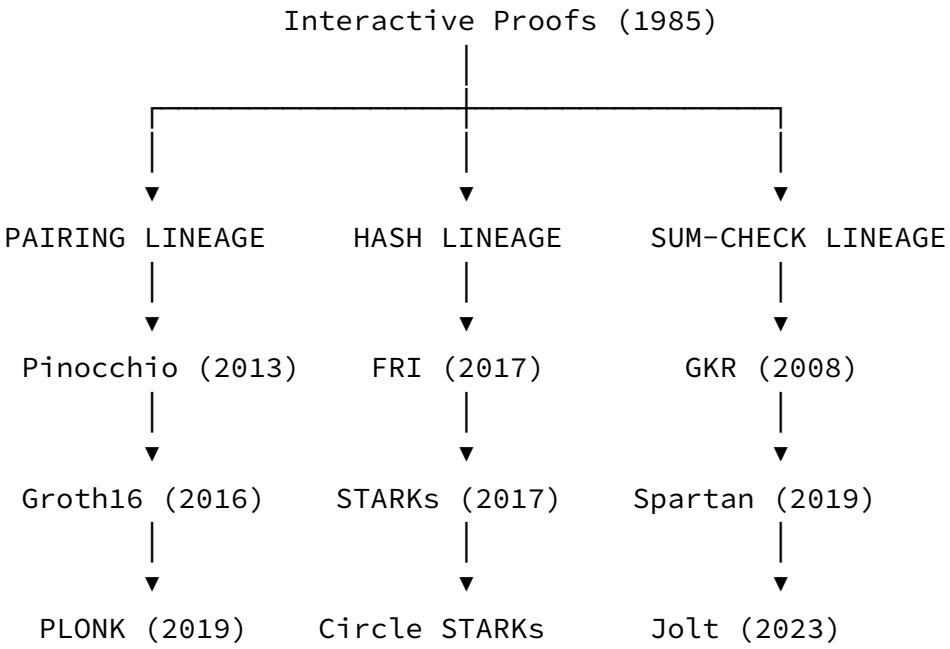
2019: PLONK (Universal Setup) Gabizon, Williamson, and Ciobotaru introduce PLONK (Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge). One trusted setup ceremony supports all circuits up to a size bound. The permutation argument elegantly handles copy constraints.

2019: Halo (Recursive Proofs Without Pairings) Bowe, Grigg, and Hopwood demonstrate recursion using inner-product arguments over elliptic curves, avoiding the pairing bottleneck. Proofs verify proofs verify proofs, with unlimited depth.

2019-2020: zk-Rollups Emerge Teams including Loopring, zkSync, and StarkWare deploy zk-rollups on Ethereum. Transaction data lives on-chain; execution validity is proven off-chain. Throughput increases 100-1000×.

Three lineages

By the end of this era, three distinct lineages of zero-knowledge proofs had emerged from a common ancestor:



Three lineages of zero-knowledge proofs, each with distinct cryptographic foundations: pairings, hashes, and sum-check.

The modern era (2020-present)

2020-2022: Lookup Arguments Mature Plookup (Gabizon, Williamson, and Maller, 2020), cq, and other lookup protocols become standard. Table-based constraint checking replaces expensive algebraic encoding for range checks, bitwise operations, and memory access.

2021-2022: Nova and Folding Schemes Kothapalli, Setty, and Tzialla introduce Nova, which replaces expensive recursive SNARK verification with cheap algebraic "folding." Per-step overhead drops from thousands of constraints to a handful of group operations.

2022: Plonky2 (PLONK + FRI) Polygon Zero combines PLONK's flexible arithmetization with FRI's transparent polynomial commitments over a small Goldilocks field. Fast recursion (under 300ms on a laptop) enables practical proofs of Ethereum execution.

2023: Lasso and Jolt Setty, Thaler, and colleagues develop Lasso (efficient lookups for sum-check-based systems) and Jolt (a RISC-V zkVM using these techniques). The sum-check renaissance: proving returns to its interactive-proof roots.

2023: zkEVMs Launch Multiple teams (Polygon, Scroll, zkSync Era, Linea) deploy zkEVMs that prove Ethereum Virtual Machine execution. Arbitrary smart contracts gain ZK privacy or scalability.

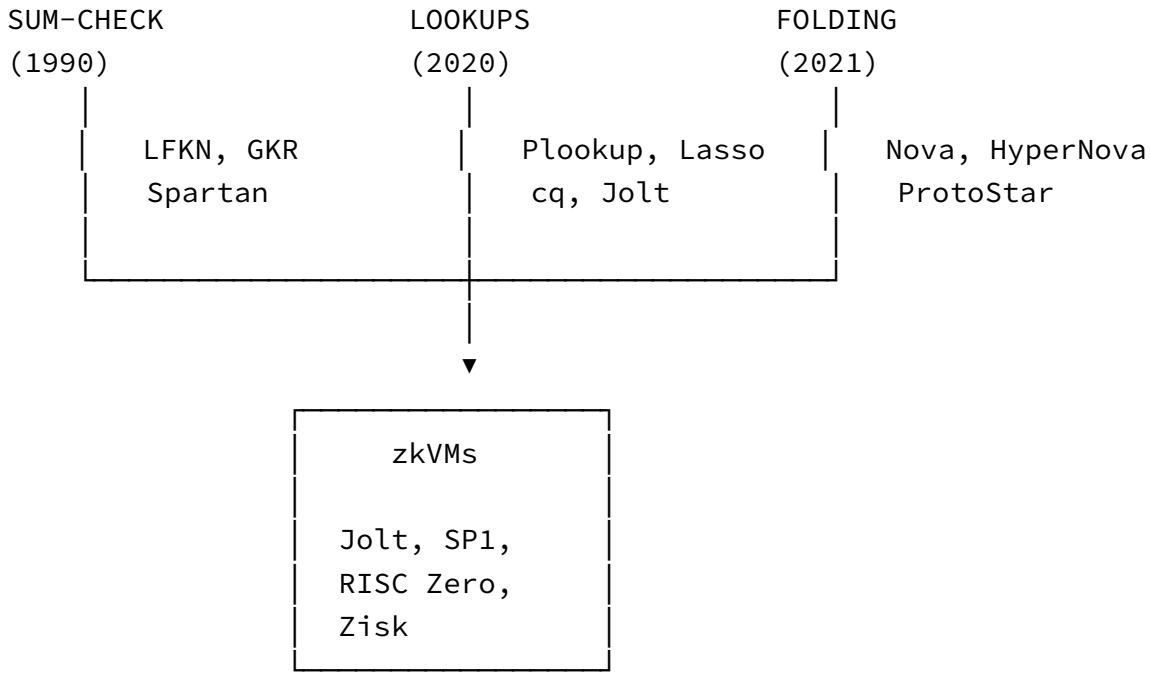
2023: SP1 and Competitive zkVMs Succinct Labs releases SP1, a RISC-V zkVM emphasizing developer experience. Competition intensifies: RISC Zero, Jolt, Valida, and others push proving speed and flexibility.

2024: Circle STARKs and Small Fields StarkWare and others explore STARKs over small fields (Mersenne primes, binary towers), trading field size for faster arithmetic. Proof sizes shrink; prover speeds increase.

2024-Present: Folding and IVC Proliferate Nova variants (SuperNova, HyperNova, ProtoStar) extend folding to handle complex constraint types. Incrementally verifiable computation becomes practical for long-running programs.

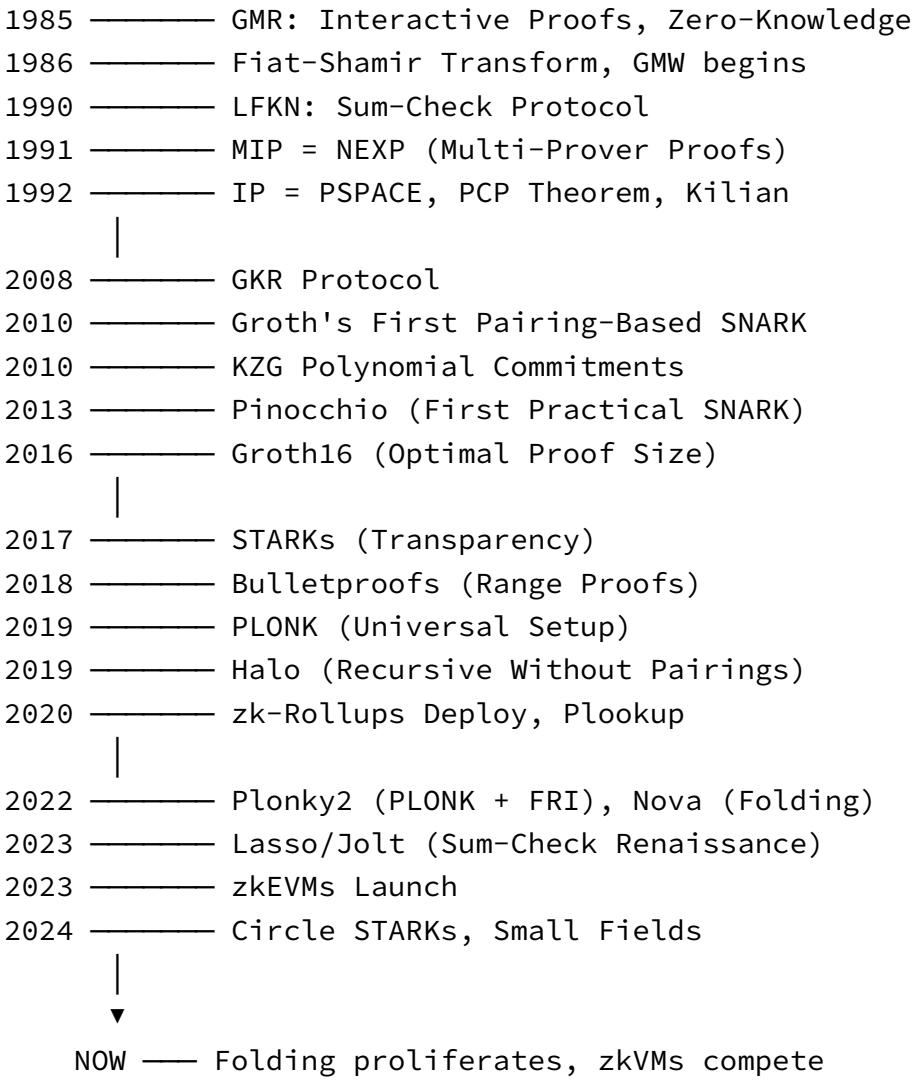
Convergence

Modern zkVMs are the confluence of three decades of distinct research streams:



Modern systems like Jolt and SP1 combine sum-check's linear proving, lookup arguments' efficient table access, and folding's cheap recursion. The zkVM is where three rivers meet.

Visual timeline



Key themes

Theory to practice (1985-2016): Early work established that zero-knowledge proofs exist for all of NP, but constructions were impractical. The path from GMR to Groth16 took 31 years.

The trusted setup debate (2016-2019): Groth16's efficiency came with per-circuit trusted setup. PLONK's universal setup and STARKs' transparency offered alternatives. The field fragmented into

camps, each valid for different applications.

The zkVM vision (2020-present): Rather than hand-crafting circuits for each application, prove correct execution of arbitrary programs. RISC-V emerges as the dominant target ISA.

The sum-check renaissance (2022-present): After years of PCP-inspired constructions, the field rediscovers sum-check's elegance. Linear-time proving, virtual polynomials, and folding schemes push efficiency toward theoretical limits.

Chapter 26 surveys the active frontiers in detail.

Appendix C: Field equations cheat sheet

A quick reference for the core equations in zero-knowledge proof systems.

Schwartz-Zippel lemma

For a non-zero polynomial $p(X_1, \dots, X_n)$ of total degree d over a field $\mathbb{F}$:

$$\Pr_{r \leftarrow \mathbb{F}^n}[p(r) = 0] \leq \frac{d}{|\mathbb{F}|}$$

Consequence: Random evaluation catches cheating with probability $\geq 1 - d/|\mathbb{F}|$.

Multilinear extensions

Lagrange basis polynomial

For $w \in \{0,1\}^n$:

$$L_w(X) = \prod_{i=1}^{n} (w_i \cdot X_i + (1 - w_i)(1 - X_i))$$

Property: $L_w(w) = 1$ and $L_w(b) = 0$ for $b \neq w$.

Multilinear extension formula

For $f : \{0,1\}^n \to \mathbb{F}$:

$$\tilde{f}(X) = \sum_{w \in \{0,1\}^n} f(w) \cdot L_w(X)$$

Equality polynomial

$$\widetilde{\mathsf{eq}}(X, Y) = \prod_{i=1}^{n} (X_i Y_i + (1 - X_i)(1 - Y_i))$$

Property: $\widetilde{\mathsf{eq}}(a, b) = 1$ if $a = b$, else 0 (on hypercube).

Sum-check protocol

Dimensions: Vector size $N = 2^n$; protocol runs n rounds.

The claim

Prove:

$$H = \sum_{b \in \{0,1\}^n} g(b_1, \dots, b_n)$$

Round i polynomial

Prover sends:

$$s_i(X_i) = \sum_{b_{i+1}, \dots, b_n \in \{0,1\}} g(r_1, \dots, r_{i-1}, X_i, b_{i+1}, \dots, b_n)$$

Verifier checks

- Round 1: $s_1(0) + s_1(1) = H$
- Round $i > 1$: $s_i(0) + s_i(1) = s_{i-1}(r_{i-1})$
- Final: query oracle at $(r_1, \dots, r_n)$ to check $s_n(r_n) = g(r_1, \dots, r_n)$

Soundness

$$\epsilon \leq \frac{n \cdot d}{|\mathbb{F}|}$$

where n is the number of variables and d is the maximum individual degree. (More precisely: $\sum_i d_i / |\mathbb{F}|$ where d_i is the degree in variable i.)

Vanishing polynomials

Over roots of unity

For domain $H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$ where $\omega^n = 1$:

$$Z_H(X) = X^n - 1$$

Property: $Z_H(\omega^i) = 0$ for all i, and $Z_H(r) \neq 0$ for $r \notin H$.

Over boolean hypercube

For proving a polynomial vanishes on $\{0,1\}^n$, use the univariate identity:

$$Z_{\{0,1\}}(X) = X(X-1)$$

applied variable by variable in multilinear settings.

R1CS and QAP

R1CS constraint

Dimensions: Matrices A, B, C are $m \times n$; witness z is $n \times 1$; result is $m \times 1$.

For witness vector $z = (1, x, w)$:

$$(A \cdot z) \circ (B \cdot z) = C \cdot z$$

where $\circ$ is entry-wise multiplication.

QAP polynomial identity

Define polynomials $A(X), B(X), C(X)$ by interpolating constraint matrices.

The constraint system is satisfied iff:

$$A(X) \cdot B(X) - C(X) = H(X) \cdot Z_H(X)$$

where $Z_H(X) = \prod_{\alpha \in H}(X - \alpha)$ is the vanishing polynomial.

KZG polynomial commitments

Dimensions: Polynomial degree $< D$; SRS size $D + 1$ elements.

Structured reference string (SRS)

Secret τ; public: $(g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^D})$

Commitment

For $f(X) = \sum_i c_i X^i$:

$$C = g^{f(\tau)} = \prod_i (g^{\tau^i})^{c_i}$$

Evaluation proof

To prove $f(z) = v$: *(Prover knows $f(X)$; Verifier knows C, z, v)*

1. Compute quotient: $w(X) = \frac{f(X) - v}{X - z}$
2. Proof: $\pi = g^{w(\tau)}$

Verification (pairing check)

$$e(\pi, g^{\tau} \cdot g^{-z}) = e(C \cdot g^{-v}, g)$$

Equivalently:

$$e(g^{w(\tau)}, g^{\tau - z}) = e(g^{f(\tau) - v}, g)$$

FRI folding

Split polynomial

For $f(X) = f_E(X^2) + X \cdot f_O(X^2)$:

- $f_E(Y)$: even coefficients
- $f_O(Y)$: odd coefficients

Folding with challenge α

$$f_1(Y) = f_E(Y) + \alpha \cdot f_O(Y)$$

Property: $\deg(f_1) < \deg(f)/2$

Consistency check

At query point x (where $-x$ is its conjugate on the same coset), verify:

$$f_1(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x}$$

This uses: $f_E(x^2) = \frac{f(x)+f(-x)}{2}$ and $f_O(x^2) = \frac{f(x)-f(-x)}{2x}$.

AIR (Algebraic Intermediate Representation)

Trace polynomials

For a trace matrix with w registers and T timesteps, interpolate each column over domain $H = \{1, \omega, \dots, \omega^{T-1}\}$:

$$P_j(\omega^i) = \text{trace}[i][j]$$

Transition constraints

For a constraint "register 0 at next step equals f of current registers":

$$P_0(\omega X) = f(P_0(X), P_1(X), \dots)$$

The shift ωX accesses "next row" values. Define constraint polynomial:

$$C(X) = P_0(\omega X) - f(P_0(X), P_1(X), \dots)$$

Quotient check

Valid trace iff $C(X)$ vanishes on transition domain $H' = \{1, \omega, \dots, \omega^{T-2}\}$:

$$Q(X) = \frac{C(X)}{Z_{H'}(X)}$$

is a polynomial (not rational function).

Boundary constraints

Pin inputs/outputs. For $P_j(\omega^k) = v$:

$$\frac{P_j(X) - v}{X - \omega^k}$$

must be a polynomial.

PLONK

Gate equation

$$Q_L(X) \cdot a(X) + Q_R(X) \cdot b(X) + Q_O(X) \cdot c(X) + Q_M(X) \cdot a(X) \cdot b(X) + Q_C(X) = 0$$

on domain $H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$.

Permutation grand product

Accumulator $Z(X)$ satisfies:

$$Z(1) = 1$$

$$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(a_i + \beta\omega^i + \gamma)(b_i + \beta k_1\omega^i + \gamma)(c_i + \beta k_2\omega^i + \gamma)}{(a_i + \beta\sigma_a(\omega^i) + \gamma)(b_i + \beta\sigma_b(\omega^i) + \gamma)(c_i + \beta\sigma_c(\omega^i) + \gamma)}$$

Property: The product telescopes, so $Z(\omega^n) = Z(1) = 1$ iff all copy constraints hold.

Quotient check

All constraints satisfied iff there exists $t(X)$ with:

$$\text{(gate)} + \alpha \cdot \text{(permutation)} = t(X) \cdot Z_H(X)$$

Groth16

Public input combination

Given public inputs $(z_0, z_1, \ldots, z_\ell)$ where $z_0 = 1$:

$$\mathsf{vk}_x = \sum_{j=0}^{\ell} z_j \cdot (\mathsf{vk}_{IC})_j$$

where $(\mathsf{vk}_{IC})_j = g_1^{\frac{\beta A_j(\tau) + \alpha B_j(\tau) + C_j(\tau)}{\gamma}}$ are verification key elements.

Verification equation

Given proof $(\pi_A, \pi_B, \pi_C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$:

$$e(\pi_A, \pi_B) \stackrel{?}{=} e(g_1^\alpha, g_2^\beta) \cdot e(\mathsf{vk}_x, g_2^\gamma) \cdot e(\pi_C, g_2^\delta)$$

Verification cost: One MSM (size ℓ) + 3-4 pairings, independent of circuit size.

Proof size

3 group elements: 128 bytes over BN254 (32 + 64 + 32 for $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_1$).

Lookup arguments

Plookup identity

For lookups $f = \{f_1, \ldots, f_n\}$ and table $t = \{t_1, \ldots, t_d\}$, let $s = \text{sort}(f \cup t)$.

$$\prod_i (\gamma + f_i) \cdot \prod_i (\gamma(1+\beta) + t_i + \beta t_{i+1}) \cdot (1+\beta)^n$$

$$= \prod_i (\gamma(1+\beta) + s_i + \beta s_{i+1})$$

Property: Equality holds iff $f \subseteq t$.

LogUp identity

For lookups $f = \{f_1, \ldots, f_n\}$ into table $t = \{t_1, \ldots, t_d\}$ with multiplicities m_j:

$$\sum_{i=1}^{n} \frac{1}{\gamma + f_i} = \sum_{j=1}^{d} \frac{m_j}{\gamma + t_j}$$

Property: Equality holds iff each $f_i \in t$ and m_j counts occurrences correctly.

Soundness: By Schwartz-Zippel, equality holds with probability $\geq 1 - (n+d)/|\mathbb{F}|$ over random γ.

Advantage: No sorting required; additive structure enables multi-table batching.

GKR protocol

Dimensions: Layer i has S_i gates; layer $i+1$ (inputs) has S_{i+1} gates; $k = \log_2 S_i$.

Layer reduction

For layered circuit with values $V^{(i)}$ at layer i:

$$\tilde{V}^{(i)}(r) = \sum_{p,q \in \{0,1\}^k} \widetilde{\mathsf{add}}^{(i)}(r,p,q) \cdot (\tilde{V}^{(i+1)}(p) + \tilde{V}^{(i+1)}(q))$$
$$+ \widetilde{\mathsf{mult}}^{(i)}(r,p,q) \cdot \tilde{V}^{(i+1)}(p) \cdot \tilde{V}^{(i+1)}(q)$$

Sum-check reduction

A claim about $\tilde{V}^{(i)}(r)$ reduces via sum-check to claims about $\tilde{V}^{(i+1)}(p^*)$ and $\tilde{V}^{(i+1)}(q^*)$ for random p^*, q^*.

Soundness: Compound over d layers, each with $O(\log n)$ sum-check rounds.

Inner product argument (IPA)

The claim

Prove $\langle \vec{a}, \vec{b} \rangle = c$ for committed $\vec{a}$.

Folding step

Given challenge α:

$$\vec{a}' = \alpha \cdot \vec{a}_L + \alpha^{-1} \cdot \vec{a}_R$$
$$\vec{b}' = \alpha^{-1} \cdot \vec{b}_L + \alpha \cdot \vec{b}_R$$

Property: $\langle \vec{a}', \vec{b}' \rangle = \langle \vec{a}, \vec{b} \rangle + \alpha^2 L + \alpha^{-2} R$

where $L = \langle \vec{a}_L, \vec{b}_R \rangle$ and $R = \langle \vec{a}_R, \vec{b}_L \rangle$.

Proof size

$O(\log n)$ group elements after $\log n$ rounds.

Nova folding

Relaxed R1CS

Standard R1CS: $(A \cdot z) \circ (B \cdot z) = C \cdot z$

Relaxed R1CS with scalar u and error E:

$$(A \cdot z) \circ (B \cdot z) = u \cdot (C \cdot z) + E$$

A satisfying instance has $u = 1$ and $E = 0$.

Folding two instances

Given instances (u_1, E_1, z_1) and (u_2, E_2, z_2), with challenge r:

$$u = u_1 + r \cdot u_2$$

$$E = E_1 + r \cdot T + r^2 \cdot E_2$$

$$z = z_1 + r \cdot z_2$$

where T is the "cross-term" computed by the prover.

Property: If both inputs satisfy relaxed R1CS, so does the folded instance.

Fiat-Shamir transform

Challenge derivation

$$r_i = H(\text{transcript prefix including all previous messages})$$

Security requirement

The hash must include:

- The public statement x
- All previous commitments $C_1, \ldots, C_{i-1}$
- All previous challenges $r_1, \ldots, r_{i-1}$

Complexity summary

System	Proof Size	Verification	Prover	Setup
Groth16	$O(1)$	$O(1)$	$O(n \log n)$	Per-circuit
PLONK+KZG	$O(1)$	$O(1)$	$O(n \log n)$	Universal
STARK/FRI	$O(\log^2 n)$	$O(\log^2 n)$	$O(n \log n)$	Transparent
Bulletproofs	$O(\log n)$	$O(n)$	$O(n \log n)$	Transparent
Sum-check IP	$O(\log n)$	$O(\log n)$	$O(n)$	None

Field sizes (common choices)

Field	Size	Security	Use Case
BN254 scalar	$\approx 2^{254}$	~100 bits	Ethereum, Groth16, PLONK
BLS12-381 scalar	$\approx 2^{255}$	~128 bits	Zcash, many SNARKs
Goldilocks	$2^{64} - 2^{32} + 1$	~100 bits*	Plonky2, fast arithmetic
Baby Bear	$2^{31} - 2^{27} + 1$	~100 bits*	RISC Zero
KoalaBear	$2^{31} - 2^{24} + 1$	~100 bits*	SP1 Hypercube, Plonky3
Mersenne-31	$2^{31} - 1$	~100 bits*	Circle STARKs, Airbender
Jolt prime	$2^{128} - 2^{32} + 22537$	~128 bits	Jolt with Akita (lattice PCS)
Binary towers	$\mathbb{F}_{2^k}$, k up to 192	~128 bits	Binius64, leanVM

*Small fields require extension fields for cryptographic security; base field security refers to the overall system design.

Quick reference

Proving a sum over hypercube: Sum-check protocol

Encoding data as polynomial: Multilinear extension (hypercube) or Lagrange interpolation (roots of unity)

Binding prover to polynomial: KZG (trusted setup, constant size), FRI (transparent, $\log^2$ size), IPA (no pairings, log size)

Checking polynomial identity on a domain: Quotient by $Z_H(X) = X^n - 1$ for roots of unity

Checking table membership: Lookup argument (Plookup with sorting, LogUp without)

Verifying circuit layer-by-layer: GKR protocol with sum-check at each layer

Incremental computation: Nova folding (amortize SNARK cost across steps)

Eliminating interaction: Fiat-Shamir with complete transcript hashing

Appendix D: Advanced polynomial commitment schemes

This appendix covers polynomial commitment schemes that achieve specialized trade-offs beyond the KZG and IPA schemes of Chapter 9.

Hyrax

Chapter 9's IPA scheme has linear verification time: the verifier must compute the folded generators, doing $O(N)$ work for a polynomial with N coefficients. **Hyrax** (Wahby et al., 2018) reduces verification to $O(\sqrt{N})$ by exploiting the tensor structure of multilinear polynomials. Polynomial evaluation can be written as a vector-matrix-vector product, and this matrix structure enables a commitment scheme where the prover commits to rows separately.

A multilinear polynomial $\tilde{f}$ over n variables has $N = 2^n$ coefficients. The naive approach stores these as a flat vector $(f_0, f_1, \ldots, f_{N-1})$ and commits with a single Pedersen commitment using N generators. Evaluation then requires $O(N)$ work.

Hyrax reshapes the flat vector into a $\sqrt{N} \times \sqrt{N}$ matrix M:

$$\underbrace{(f_0, f_1, \ldots, f_{N-1})}_{\text{flat vector}} \quad \longrightarrow \quad \underbrace{M = \begin{pmatrix} f_0 & f_1 & \cdots & f_{\sqrt{N}-1} \\ f_{\sqrt{N}} & f_{\sqrt{N}+1} & \cdots & f_{2\sqrt{N}-1} \\ \vdots & \vdots & \ddots & \vdots \end{pmatrix}}_{\text{matrix form}}$$

The entry $M[a][b]$ stores the coefficient $f_{a \cdot \sqrt{N}+b}$, which corresponds to the evaluation at the Boolean point whose binary representation concatenates a and b.

Polynomial evaluation then decomposes into a vector-matrix-vector product, and the prover can commit to rows separately, reducing verification from $O(N)$ to $O(\sqrt{N})$.

Tensor structure of multilinear evaluation

Recall from Chapter 5 that multilinear evaluation uses the equality polynomial:

$$\tilde{f}(r_1, \ldots, r_n) = \sum_{b \in \{0,1\}^n} f_b \cdot \mathsf{eq}(b, r)$$

where $\mathsf{eq}(b, r) = \prod_{i=1}^{n}(b_i r_i + (1 - b_i)(1 - r_i))$.

The key observation: eq factors across the split. If we partition the evaluation point $r = (r_L, r_R)$ where $r_L = (r_1, \ldots, r_{n/2})$ and $r_R = (r_{n/2+1}, \ldots, r_n)$, then:

$$\mathsf{eq}((a, b), r) = \mathsf{eq}(a, r_L) \cdot \mathsf{eq}(b, r_R)$$

Define the Lagrange coefficient vectors:

$$L[a] = \mathsf{eq}(a, r_L) \quad \text{for } a \in \{0,1\}^{n/2}$$

$$R[b] = \mathsf{eq}(b, r_R) \quad \text{for } b \in \{0,1\}^{n/2}$$

Then evaluation becomes a bilinear form. Starting from the MLE definition:

$$\tilde{f}(r) = \sum_{b \in \{0,1\}^n} f_b \cdot \mathsf{eq}(b, r)$$

Split each index $b = (a, c)$ where a indexes rows and c indexes columns:

$$= \sum_{a \in \{0,1\}^{n/2}} \sum_{c \in \{0,1\}^{n/2}} M[a][c] \cdot \mathsf{eq}((a, c), r)$$

Factor the equality polynomial:

$$= \sum_a \sum_c M[a][c] \cdot \mathsf{eq}(a, r_L) \cdot \mathsf{eq}(c, r_R)$$

Substitute the Lagrange vectors $L[a] = \mathsf{eq}(a, r_L)$ and $R[c] = \mathsf{eq}(c, r_R)$:

$$= \sum_{a,c} M[a][c] \cdot L[a] \cdot R[c] = \vec{L}^T M \vec{R}$$

This is a rank-2 tensor contraction: two vectors contracting with a matrix. The factorization of eq separates "row selection" ($\vec{L}$) from "column selection" ($\vec{R}$), which is what makes the matrix reshaping useful. A flat vector evaluation $\sum_i f_i \cdot \chi_i(r)$ requires touching all N terms, but $\vec{L}^T M \vec{R}$ can be computed in two steps: first $\vec{u} = M^T \vec{L}$ (a length-$\sqrt{N}$ vector), then $\langle \vec{u}, \vec{R} \rangle$ (a single dot product). Each step involves only $\sqrt{N}$ operations.

The Hyrax commitment scheme

Public parameters Random generators $\vec{G} = (G_0, \ldots, G_{\sqrt{N}-1}) \in \mathbb{G}^{\sqrt{N}}$ and $H \in \mathbb{G}$ for blinding.

Commitment Instead of committing to all N coefficients at once (which would require N generators), commit to each row separately:

$$C_a = \langle M[a], \vec{G} \rangle + r_a \cdot H = \sum_{b=0}^{\sqrt{N}-1} M[a][b] \cdot G_b + r_a \cdot H$$

where r_a is a blinding factor for row a. The full commitment is the tuple of row commitments:

$$\text{Com}(M) = (C_0, C_1, ..., C_{\sqrt{N}-1})$$

This requires $\sqrt{N}$ group elements, not one. The trade-off: larger commitment size for cheaper verification.

The opening protocol

To prove $\tilde{f}(r) = v$ where $r = (r_L, r_R)$:

Step 1: Both parties compute Lagrange vectors From the evaluation point $r = (r_L, r_R)$, both prover and verifier compute: - $\vec{L}$: row Lagrange coefficients from r_L - $\vec{R}$: column Lagrange coefficients from r_R

Step 2: Prover computes the projection vector The prover computes $\vec{u} = M^T \vec{L}$, the weighted column sums:

$$u_b = \sum_{a=0}^{\sqrt{N}-1} L[a] \cdot M[a][b]$$

Each u_b is the L-weighted sum of column b.

Step 3: Verifier computes combined commitment (MSM #1) The verifier combines the original row commitments (from the commitment phase) using $\vec{L}$:

$$C' = \sum_{a=0}^{\sqrt{N}-1} L[a] \cdot C_a$$

This is computed by the verifier during opening, not as part of the initial commitment. The verifier doesn't have access to the matrix M, but they don't need it. By Pedersen's homomorphism, a linear combination of commitments *is* a commitment to the linear combination of the underlying vectors:

$$C' = \sum_a L[a] \cdot C_a = \sum_a L[a] \cdot \langle M[a], \vec{G} \rangle = \langle \sum_a L[a] \cdot M[a], \vec{G} \rangle$$

The inner sum $\sum_a L[a] \cdot M[a]$ is exactly the projection vector $(u_0, u_1, \ldots, u_{\sqrt{N}-1})$, where each u_b is defined in Step 2. So C' would equal $\langle \vec{u}, \vec{G} \rangle$ if the prover computed $\vec{u}$ correctly. The verifier doesn't know $\vec{u}$ yet, but they have computed what a commitment to the *correct* $\vec{u}$ should be.

Step 4: Verify consistency (MSM #2) The prover sends $\vec{u} = (u_0, \ldots, u_{\sqrt{N}-1})$. The verifier computes a commitment to the claimed $\vec{u}$:

$$C'' = \sum_{b=0}^{\sqrt{N}-1} u_b \cdot G_b = \langle \vec{u}, \vec{G} \rangle$$

Check: $C' \stackrel{?}{=} C''$

If $C' = C''$, the prover's $\vec{u}$ is consistent with the committed matrix. The verifier derived C' from the row commitments (which bind the prover to M), so equality means the prover computed $\vec{u} = M^T \vec{L}$ correctly.

Step 5: Verify the dot product Check: $\langle \vec{u}, \vec{R} \rangle \stackrel{?}{=} v$

Why this proves evaluation The tensor contraction gives:

$$\tilde{f}(r) = \sum_{a,b} M[a][b] \cdot L[a] \cdot R[b] = \sum_b \left(\sum_a L[a] \cdot M[a][b] \right) \cdot R[b] = \sum_b u_b \cdot R[b] = \langle \vec{u}, \vec{R} \rangle$$

So the dot product check verifies that the claimed value v equals the polynomial evaluation.

Zero-knowledge variant The prover doesn't send $\vec{u}$ directly (which would leak information about M). Instead, both checks are combined into a ZK dot product protocol that proves consistency without revealing $\vec{u}$.

Zero-knowledge dot product protocol

Hyrax uses a Schnorr-style protocol for proving $\langle \vec{a}, \vec{u} \rangle = v$ where $\vec{u}$ is committed (with blinding) and $\vec{a}$ is public.

Setup Prover holds $\vec{u}$ with Pedersen commitment $C = \langle \vec{u}, \vec{G} \rangle + s \cdot H$ and blinding factor s.

Protocol

1. Prover picks random masking vector $\vec{d} \in \mathbb{F}^{\sqrt{N}}$ and blinding $s_d \in \mathbb{F}$
2. Prover sends commitment $D = \langle \vec{d}, \vec{G} \rangle + s_d \cdot H$ and masked dot product $e = \langle \vec{a}, \vec{d} \rangle$
3. Verifier sends random challenge c
4. Prover responds with $\vec{z} = \vec{d} + c \cdot \vec{u}$ and $s_z = s_d + c \cdot s$
5. Verifier checks:
    - $\langle \vec{z}, \vec{G} \rangle + s_z \cdot H = D + c \cdot C$ (commitment consistency)
    - $\langle \vec{a}, \vec{z} \rangle = e + c \cdot v$ (dot product relation)

The first check ensures $\vec{z}$ opens the linear combination $D + c \cdot C$. The second check verifies that $\langle \vec{a}, \vec{d} + c \cdot \vec{u} \rangle = \langle \vec{a}, \vec{d} \rangle + c \cdot \langle \vec{a}, \vec{u} \rangle$, which holds only if $\langle \vec{a}, \vec{u} \rangle = v$.

Communication cost: $O(\sqrt{N})$ field elements (the response vector $\vec{z}$).

Worked example

Let's trace through Hyrax for $n = 4$ variables, so $N = 16$ evaluations arranged as a 4×4 matrix.

Setup Polynomial evaluations on $\{0,1\}^4$ arranged as matrix M (row index = first 2 bits, column index = last 2 bits):

$$M = \begin{pmatrix} 3 & 1 & 4 & 1 \\ 5 & 9 & 2 & 6 \\ 5 & 3 & 5 & 8 \\ 9 & 7 & 9 & 3 \end{pmatrix}$$

Generators: $\vec{G} = (G_0, G_1, G_2, G_3)$ and blinding generator H. Evaluation point: $r = (0.5, 0.5, 0.5, 0.5)$ (working over reals for clarity).

Step 1: Commitment phase Prover commits to each row (omitting blinding for clarity):

$$C_0 = 3G_0 + 1G_1 + 4G_2 + 1G_3$$

$$C_1 = 5G_0 + 9G_1 + 2G_2 + 6G_3$$

$$C_2 = 5G_0 + 3G_1 + 5G_2 + 8G_3$$

$$C_3 = 9G_0 + 7G_1 + 9G_2 + 3G_3$$

The commitment is (C_0, C_1, C_2, C_3): four group elements.

Step 2: Compute Lagrange vectors Split $r = (r_L, r_R)$ where $r_L = (0.5, 0.5)$ and $r_R = (0.5, 0.5)$.

$$L[00] = (1 - 0.5)(1 - 0.5) = 0.25, \quad L[01] = (1 - 0.5)(0.5) = 0.25$$
$$L[10] = (0.5)(1 - 0.5) = 0.25, \quad L[11] = (0.5)(0.5) = 0.25$$

Similarly $\vec{R} = (0.25, 0.25, 0.25, 0.25)$. Both prover and verifier compute these from the evaluation point.

Step 3: Compute projection vector The prover computes $\vec{u} = M^T \vec{L}$. Each u_b is the L-weighted sum of column b:

$$u_0 = 0.25(3 + 5 + 5 + 9) = 5.5$$
$$u_1 = 0.25(1 + 9 + 3 + 7) = 5$$
$$u_2 = 0.25(4 + 2 + 5 + 9) = 5$$
$$u_3 = 0.25(1 + 6 + 8 + 3) = 4.5$$

So $\vec{u} = (5.5, 5, 5, 4.5)$. The prover sends $\vec{u}$ (in the non-ZK variant).

Step 4: Two MSMs MSM #1: Combine row commitments with $\vec{L}$:

$$C' = 0.25 \cdot C_0 + 0.25 \cdot C_1 + 0.25 \cdot C_2 + 0.25 \cdot C_3$$

Expanding: $C' = 0.25[(3+5+5+9)G_0 + (1+9+3+7)G_1 + (4+2+5+9)G_2 + (1+6+8+3)G_3]$ $= 5.5G_0 + 5G_1 + 5G_2 + 4.5G_3$

MSM #2: Commit to $\vec{u}$ using generators:

$$C'' = 5.5G_0 + 5G_1 + 5G_2 + 4.5G_3$$

Check: $C' = C''$ ✓ (The projection vector is consistent with the committed matrix.)

Step 5: Dot product check

$$v = \langle \vec{u}, \vec{R} \rangle = 5.5(0.25) + 5(0.25) + 5(0.25) + 4.5(0.25) = 5$$

Check: $\langle \vec{u}, \vec{R} \rangle = v = 5 \checkmark$

Verification cost The verifier performed two MSMs of size 4 (not 16), plus field arithmetic for the dot product. Total: $O(\sqrt{N})$ group operations.

Using Bulletproofs for logarithmic proof size

The basic Hyrax protocol has $O(\sqrt{N})$ communication because the prover sends $\vec{z}$ (length $\sqrt{N}$) in the Schnorr-style dot product proof. This can be reduced to $O(\log N)$ by replacing Schnorr with Bulletproofs' inner product argument.

Bulletproofs (Bünz et al., 2018) proves $\langle \vec{a}, \vec{b} \rangle = c$ with $O(\log n)$ proof size but $O(n)$ verifier time (linear in vector length). When applied to Hyrax's dot product step (vectors of length $\sqrt{N}$):

- Proof size: $O(\sqrt{N} + \log \sqrt{N}) = O(\sqrt{N})$ (row commitments dominate)
- Verifier time: $O(\sqrt{N})$ (MSM for C' plus Bulletproofs verification on length-$\sqrt{N}$ vectors)

The ι parameter

The Hyrax paper introduces a generalization parameter $\iota \geq 2$ that controls a communication vs. computation trade-off. Instead of a square matrix, arrange the coefficients as $N^{1/\iota} \times N^{(\iota-1)/\iota}$:

- $\iota = 2$ **(square-root)**: $\sqrt{N} \times \sqrt{N}$ matrix, $O(\sqrt{N})$ commitment, $O(\sqrt{N})$ verification
- $\iota = 3$: $N^{1/3} \times N^{2/3}$ matrix, $O(N^{1/3})$ commitment, $O(N^{2/3})$ verification
- **General**: $O(N^{1/\iota})$ commitment size, $O(N^{(\iota-1)/\iota})$ verification time

Higher ι reduces commitment size (fewer row commitments) at the cost of higher verification time (longer dot product vectors). Since the commitment is sent once but may be opened many times, the square-root case ($\iota = 2$) typically offers the best balance.

Properties and trade-offs

Property	Hyrax (square-root, $\iota = 2$)
Trusted setup	**None (Transparent)**
Commitment size	$O(\sqrt{N})$ group elements
Proof size	$O(\log N)$ with Bulletproofs
Verification time	$O(\sqrt{N})$ group operations
Prover time	$O(N)$ for commitment, $O(\sqrt{N})$ per opening
Assumption	Discrete log only
Quantum-safe	No

Comparison with IPA:

	IPA	Hyrax
Commitment size	$O(1)$	$O(\sqrt{N})$
Verification time	$O(N)$	$O(\sqrt{N})$
Proof size	$O(\log N)$	$O(\log N)$

Both IPA and Hyrax (with Bulletproofs) achieve logarithmic proof size, but Hyrax trades larger commitments for faster verification. This trade-off is worthwhile when: - The same polynomial is opened at multiple points (amortizes commitment cost) - Verification speed matters more than proof/commitment size - You want transparency without paying IPA's linear verification cost

Connection to Dory

Hyrax's square-root verification is an improvement over IPA's linear verification, but can we do better? **Dory** answers yes by combining Hyrax's matrix structure with pairings.

The key observation: Hyrax's verifier bottleneck is the MSM $C' = \sum_a L[a] \cdot C_a$. This is $O(\sqrt{N})$ group operations. Dory eliminates this by:

1. **Tier 2 commitment:** Instead of storing row commitments directly, Dory combines them into a single $\mathbb{G}_T$ element using pairings
2. **Lazy verification:** The verifier never computes C' explicitly; instead, they track commitments in $\mathbb{G}_T$ and verify everything with a single final pairing check

Where Hyrax achieves $O(\sqrt{N})$ verification, Dory achieves $O(\log N)$. The cost is more complex cryptographic machinery (pairings, two-tier structure, SXDH assumption instead of plain discrete log).

Dory

Hyrax reduces IPA's $O(N)$ verification to $O(\sqrt{N})$ by exploiting tensor structure. **Dory** (Lee, 2021) pushes further to $O(\log N)$ by combining Hyrax's matrix arrangement with pairings.

In IPA, the verifier recalculates the folded generators at each step, doing $O(n)$ work. Dory's verifier instead accumulates commitments in $\mathbb{G}_T$ and defers all verification to a single final pairing check. The algebraic structure of pairings makes this possible: the verifier "absorbs" folding challenges into target group elements without touching the original generators directly.

Two-tier commitment structure

Dory commits to polynomials using AFGHO commitments (Abe et al.'s structure-preserving commitments) combined with Pedersen commitments.

Public parameters (SRS): Generated transparently by sampling random group elements (the notation $\xleftarrow{\$}$ means "sampled uniformly at random from"): - $\Gamma_1 \xleftarrow{\$} \mathbb{G}_1^{\sqrt{N}}$: commitment key for row commitments - $\Gamma_2 \xleftarrow{\$} \mathbb{G}_2^{\sqrt{N}}$: commitment key for final commitment - $H_1 \xleftarrow{\$} \mathbb{G}_1$, $H_2 \xleftarrow{\$} \mathbb{G}_2$: blinding generators (for hiding/zero-knowledge) - $H_T = e(H_1, H_2)$: derived blinding generator in $\mathbb{G}_T$

All parameters are **public**. The prover's secrets are the blinding factors $r_i, r_{\text{fin}} \in \mathbb{F}$.

Tier 1: Row Commitments ($\mathbb{G}_1$)

Treat the polynomial coefficients as a $\sqrt{N} \times \sqrt{N}$ matrix M. For each row i, compute a Pedersen commitment:

$$R_i = \langle M[i], \Gamma_1 \rangle + r_i \cdot H_1 = \sum_{j=0}^{\sqrt{N}-1} M[i][j] \cdot \Gamma_1[j] + r_i \cdot H_1$$

where $r_i \in \mathbb{F}$ is a **secret** blinding factor. This produces $\sqrt{N}$ elements in $\mathbb{G}_1$.

Tier 2: Final Commitment ($\mathbb{G}_T$)

Combine row commitments via pairing with generators $\Gamma_2 \in \mathbb{G}_2^{\sqrt{N}}$:

$$C = \langle \vec{R}, \Gamma_2 \rangle_T + r_{\text{fin}} \cdot H_T = \sum_{i=0}^{\sqrt{N}-1} e(R_i, \Gamma_2[i]) + r_{\text{fin}} \cdot e(H_1, H_2)$$

where r_{fin} is a final blinding factor. This produces one $\mathbb{G}_T$ element (the commitment).

Why two tiers?

Tier	Purpose
Tier 1 (rows)	Enables streaming: process row-by-row with $O(\sqrt{N})$ memory
	Row commitments serve as "hints" for efficient batch opening
Tier 2 ($\mathbb{G}_T$)	Provides succinctness: one element regardless of polynomial size
	Binding under SXDH assumption in Type III pairings

The AFGHO commitment is hiding because $r_{\text{fin}} \cdot e(H_1, H_2)$ is uniformly random in $\mathbb{G}_T$. Both tiers are additively homomorphic, which the evaluation protocol relies on.

From coefficients to matrix form

Dory uses the same tensor decomposition as Hyrax. The evaluation point $r = (r_1, \ldots, r_n)$ splits into row coordinates $r_L = (r_1, \ldots, r_{n/2})$ and column coordinates $r_R = (r_{n/2+1}, \ldots, r_n)$. Each half determines a vector of Lagrange coefficients $\vec{\ell}$ and $\vec{\rho}$ via the equality polynomial (see the Hyrax derivation above). The evaluation becomes a bilinear form:

$$f(r) = \vec{\ell}^T M \vec{\rho}$$

Dory uses ℓ for row (left) and ρ for column (right) coefficients, distinct from the evaluation point r.

The opening protocol (Dory-Innerproduct)

The key reduction: Polynomial evaluation becomes an inner product. Define two vectors:

- $\vec{v}_1 = M \cdot \vec{\rho}$, the matrix times the column Lagrange vector. Each entry $(v_1)_j = \langle M[j], \vec{\rho} \rangle$ is row j evaluated at the column coordinates.

- $\vec{v}_2 = \vec{\ell}$, the row Lagrange vector.

Then $\langle \vec{v}_1, \vec{v}_2 \rangle = \vec{\ell}^T M \vec{\rho} = f(r)$. The inner product of these two vectors *is* the polynomial evaluation.

Goal: Prove $\langle \vec{v}_1, \vec{v}_2 \rangle = v$ for committed vectors, which proves $f(r) = v$ for the polynomial.

Dory proves membership in the language:

$$\mathcal{L}_{n,\Gamma_1,\Gamma_2,H_1,H_2} = \{(C, D_1, D_2) : \exists(\vec{v}_1, \vec{v}_2, r_C, r_{D_1}, r_{D_2}) \text{ s.t.}$$
$$D_1 = \langle \vec{v}_1, \Gamma_2 \rangle + r_{D_1} H_T, \quad D_2 = \langle \Gamma_1, \vec{v}_2 \rangle + r_{D_2} H_T, \quad C = \langle \vec{v}_1, \vec{v}_2 \rangle + r_C H_T\}$$

In words: D_1 commits to $\vec{v}_1$ (using Γ_2), D_2 commits to $\vec{v}_2$ (using Γ_1), and C commits to their inner product. The protocol proves these three commitments are consistent, that the same vectors appear in all three.

How verification works

The prover knows $\vec{\ell}$, $\vec{\rho}$, and M. The verifier can compute $\vec{\ell}$ and $\vec{\rho}$ from the evaluation point but doesn't know M. The verifier never needs M directly. Instead:

Step 1: The verifier has the commitment C (which encodes M cryptographically) and the claimed evaluation v.

Step 2: The prover sends a VMV message $(C_{\text{vmv}}, D_2, E_1)$ where:

- $C_{\text{vmv}} = e(\langle \vec{R}, \vec{v}_1 \rangle, H_2)$
- $D_2 = e(\langle \Gamma_1, \vec{v}_1 \rangle, H_2)$
- $E_1 = \langle \vec{R}, \vec{\ell} \rangle$ (row commitments combined with row Lagrange coefficients)

Recall $\vec{v}_1 = M \cdot \vec{\rho}$ from earlier. This is the non-hiding variant; the row commitments $\vec{R}$ already contain blinding from tier 1.

Step 3: First verification check. The verifier checks:

$$e(E_1, H_2) \stackrel{?}{=} D_2$$

Why this works: By Pedersen linearity:

$$E_1 = \langle \vec{R}, \vec{\ell} \rangle = \sum_i \ell_i \cdot R_i = \sum_i \ell_i \cdot \langle M[i], \Gamma_1 \rangle = \langle \vec{\ell}^T M, \Gamma_1 \rangle$$

Note that $\vec{\ell}^T M$ is a row vector, while $\vec{v}_1 = M \cdot \vec{\rho}$ is a column vector. However, both represent "partial evaluations" of the matrix. The key point: E_1 is determined by the row commitments and Lagrange coefficients. The check $e(E_1, H_2) = D_2$ verifies that the prover's D_2 is consistent with the row commitments $\vec{R}$. This binds the prover's intermediate computation to the committed polynomial.

Step 4: The verifier computes $E_2 = H_2 \cdot v$ (not from the prover).

The verifier computes this themselves from the claimed evaluation v. This is how the claimed value enters the protocol: it's bound to the blinding generator H_2. If the prover lied about $v = f(r)$, then E_2 won't match the prover's internal computation, and the final check will fail.

Step 5: Initialize verifier state.

- $C \leftarrow C_{\text{vmv}}$ (from VMV message)
- $D_1 \leftarrow$ the polynomial commitment (the tier-2 commitment the verifier already has)
- $D_2 \leftarrow$ from VMV message
- E_1, E_2 as computed above

What remains to prove: The prover must demonstrate that $\langle \vec{v}_2, \vec{v}_1 \rangle = v$. That is, the intermediate vector $\vec{v}_1$ (committed implicitly via the consistency check) inner-producted with $\vec{v}_2 = \vec{\ell}$ yields the claimed evaluation. This is where Dory-Reduce takes over.

The folding protocol

Each round halves the problem size. Given vectors of length $2m$, the round uses two challenges (β, then α) and two prover messages:

First message (before any challenge):

- $D_{1L} = \langle \vec{v}_{1L}, \Gamma'_2 \rangle$, $D_{1R} = \langle \vec{v}_{1R}, \Gamma'_2 \rangle$ (cross-pairings of $\vec{v}_1$ halves with generator halves)
- $D_{2L} = \langle \Gamma'_1, \vec{v}_{2L} \rangle$, $D_{2R} = \langle \Gamma'_1, \vec{v}_{2R} \rangle$ (cross-pairings of $\vec{v}_2$ halves with generator halves)

Verifier sends first challenge $\beta \xleftarrow{\$} \mathbb{F}$

Prover updates vectors:

- $\vec{v}_1 \leftarrow \vec{v}_1 + \beta \cdot \Gamma_1$
- $\vec{v}_2 \leftarrow \vec{v}_2 + \beta^{-1} \cdot \Gamma_2$

Second message (computed with β-modified vectors):

- $C_+ = \langle \vec{v}_{1L}, \vec{v}_{2R} \rangle$, $C_- = \langle \vec{v}_{1R}, \vec{v}_{2L} \rangle$ (cross inner products of modified vectors)

Verifier sends second challenge $\alpha \xleftarrow{\$} \mathbb{F}$

Prover folds vectors:

- $\vec{v}_1' = \alpha\vec{v}_{1L} + \vec{v}_{1R}$
- $\vec{v}_2' = \alpha^{-1}\vec{v}_{2L} + \vec{v}_{2R}$

Verifier updates accumulators (no pairing checks, just $\mathbb{G}_T$ arithmetic):

- $C' = C + \chi_k + \beta D_2 + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-$
- $D_1' = \alpha D_{1L} + D_{1R}$
- $D_2' = \alpha^{-1} D_{2L} + D_{2R}$

where $\chi_k = e(\Gamma_1[0..2^k], \Gamma_2[0..2^k])$ is a **precomputed SRS value** (the pairing of generator prefixes at round k).

Recurse with vectors of length m.

After $\log(\sqrt{N})$ rounds, vectors have length 1.

Final pairing check: After all rounds:

$$e(E_1' + d \cdot \Gamma_{1,0}, E_2' + d^{-1} \cdot \Gamma_{2,0}) = C' + \chi_0 + d \cdot D_2' + d^{-1} \cdot D_1'$$

where primes denote folded values, and d is a final challenge.

The invariant: Throughout folding, (C, D_1, D_2) satisfy:

- $C = \langle \vec{v}_1, \vec{v}_2 \rangle$ (inner product commitment)
- $D_1 = \langle \vec{v}_1, \Gamma_2 \rangle$, $D_2 = \langle \Gamma_1, \vec{v}_2 \rangle$ (commitments to each vector)

The verifier does no per-round pairing checks, only accumulator updates. Soundness comes from the final check verifying this invariant for the length-1 vectors.

Why binding works

The prover provides row commitments $\vec{R}$ alongside the tier-2 commitment. Why can't the prover cheat by providing fake rows?

1. **Tier 2 constrains Tier 1:** The tier-2 commitment $C = \langle \vec{R}, \Gamma_2 \rangle_T + r_{\text{fin}} H_T$ is a deterministic function of the row commitments. Changing any R_i changes C.
2. **Tier 1 constrains the data:** Each $R_i = \langle M[i], \Gamma_1 \rangle + r_i H_1$ is a Pedersen commitment. Under discrete log hardness, the prover cannot find two different row vectors that produce the same R_i.
3. **No trapdoor:** The SRS generators are sampled randomly. Without their discrete logs, the prover is computationally bound to the original coefficients.

If the Dory proof verifies, then with overwhelming probability (under SXDH), the prover knew valid openings for all original commitments.

Properties and trade-offs

Property	Dory
Trusted setup	**None (Transparent)**
Commitment size	$O(1)$ (one $\mathbb{G}_T$ element)
Proof size	$O(\log N)$ group elements
Verification time	$O(\log N)$ (the key improvement!)
Prover time	$O(N)$ for commitment, $O(\sqrt{N})$ per opening
Assumption	SXDH (on Type III pairings)
Quantum-safe	No (uses pairings)

Dory uses pairings (like KZG) but achieves transparency (like IPA). It gets logarithmic verification (better than IPA's linear) at the cost of more complex pairing machinery. This makes Dory particularly attractive for systems with many polynomial openings that can be batched (like Jolt's zkVM), where the amortized cost per opening becomes very small.

Implementations like Jolt store row commitments $\vec{R} \in \mathbb{G}_1^{\sqrt{N}}$ as "opening hints." This increases proof size by $O(\sqrt{N})$ per polynomial but enables efficient batch opening without recomputing expensive MSMs. For Jolt's ~26 committed polynomials with $N = 2^{20}$, this means ~26 KB of hints instead of ~800 bytes, but saves massive computation during batch verification.

Batching multiple polynomials exploits Pedersen's homomorphism. When batching k polynomials with random linear combination coefficient γ, we combine corresponding rows across all polynomials:

$$R_j^{(\text{joint})} = \sum_{i=1}^{k} \gamma^i \cdot R_j^{(i)}$$

Row j of $f_{\text{joint}} = \sum_i \gamma^i f_i$ has coefficients $M_{\text{joint}}[j] = \sum_i \gamma^i M_i[j]$. By linearity of Pedersen commitments, $\langle M_{\text{joint}}[j], \Gamma_1 \rangle = \sum_i \gamma^i R_j^{(i)} = R_j^{(\text{joint})}$. The joint row commitments feed directly into Dory-Reduce, avoiding $k \cdot \sqrt{N}$ expensive MSM recomputations.

Why Dory achieves logarithmic verification

Why does Dory achieve logarithmic verification while IPA requires linear time? IPA's linear cost comes from computing folded generators. Dory sidesteps this entirely: the verifier works with commitments in $\mathbb{G}_T$, updating accumulators each round without touching generators. The algebraic structure of pairings ($e(aG_1, bG_2) = e(G_1, G_2)^{ab}$) lets the verifier "absorb" folding challenges into commitments. The precomputed χ_k values handle the generator contributions.

HyperKZG and Zeromorph: KZG for multilinear polynomials

Hyrax and Dory build new commitment schemes from scratch. A different strategy reuses the existing univariate KZG infrastructure (trusted setup, pairing verification, batching) and adds a reduction layer that converts multilinear evaluation claims into univariate ones. HyperKZG (Setty, 2023) and Zeromorph (Kohrita and Towa, 2023) both take this approach, with different trade-offs.

The shared problem

A multilinear polynomial $f(x_1, \ldots, x_n)$ over n variables has $N = 2^n$ coefficients (its evaluations on the Boolean hypercube). The prover commits to f by interpreting these N values as coefficients of a univariate polynomial $\hat{f}(X) = \sum_{i=0}^{N-1} f(i) \cdot X^i$ and computing a standard KZG commitment $[\hat{f}]_1 = \hat{f}(\tau) \cdot G_1$ using a powers-of-tau SRS.

Commitment is straightforward. The difficulty is *opening*: given a multilinear evaluation point $r = (r_1, \ldots, r_n)$, proving that $f(r_1, \ldots, r_n) = v$ using only the univariate commitment $[\hat{f}]_1$. The multilinear evaluation $f(r)$ is not the same as the univariate evaluation $\hat{f}(r)$ at some single point. A reduction is needed.

HyperKZG

HyperKZG (an adaptation of Gemini by Setty) reduces the multilinear evaluation to n univariate claims via a protocol resembling sum-check. The prover sends n auxiliary univariate polynomials, one per variable, that represent the intermediate "partial bindings" as each x_i is fixed to r_i. The verifier checks consistency between consecutive polynomials using KZG openings.

The protocol proceeds in n rounds. In round i, the prover has a polynomial f_i of degree $2^{n-i} - 1$ (starting from $f_0 = \hat{f}$). The prover splits f_i into even and odd coefficients, commits to the odd-coefficient polynomial, and the verifier sends challenge r_i. The prover computes $f_{i+1}(X) = f_i^{\text{even}}(X) + r_i \cdot f_i^{\text{odd}}(X)$, halving the degree. After n rounds, f_n is a constant equal to $f(r_1, \ldots, r_n)$.

Verification requires $O(\log N)$ group operations and 3 pairings (via batching). Proof size is $O(\log N)$ group elements plus $O(\log N)$ field elements. The prover runs in $O(N)$ field operations.

Zeromorph

Zeromorph takes a more algebraic route. It uses the identity that for any multilinear f and evaluation point $r = (r_1, \ldots, r_n)$:

$$f(X_1, \ldots, X_n) - f(r_1, \ldots, r_n) = \sum_{i=1}^{n} (X_i - r_i) \cdot q_i(X_1, \ldots, X_n)$$

where each q_i is a quotient polynomial. This is the multilinear analogue of the univariate fact that $(X - r)$ divides $f(X) - f(r)$. Zeromorph maps this identity to the univariate setting: the prover commits to univariate encodings of the q_i and proves the divisibility relation holds under the encoding.

The result is a proof with $n + 3$ group elements (slightly smaller than HyperKZG) and verification with $O(\log N)$ group operations plus 3 pairings. Zeromorph also achieves the zero-knowledge property more cheaply: adding ZK costs only $n + 5$ extra group operations, compared to an extra $O(N)$-size MSM in naive approaches.

Comparison and practical relevance

Property	HyperKZG	Zeromorph	Dory	Hyrax
Setup	Trusted (SRS)	Trusted (SRS)	Transparent	Transparent
Commitment size	$O(1)$	$O(1)$	$O(1)$	$O(\sqrt{N})$
Proof size	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\sqrt{N})$
Verification	$O(\log N)$ + 3 pairings	$O(\log N)$ + 3 pairings	$O(\log N)$ pairings	$O(\sqrt{N})$
Prover	$O(N)$	$O(N)$	$O(N)$	$O(N)$
ZK overhead	Moderate	Low ($n + 5$ ops)	Low	Low

HyperKZG and Zeromorph occupy the same niche: constant-size commitments with logarithmic proofs, leveraging existing KZG infrastructure. Their main advantage over Dory and Hyrax is compatibility with the powers-of-tau ceremonies already deployed for Groth16 and PLONK. Systems

that already have a trusted setup (Ethereum's KZG ceremony, for example) can adopt HyperKZG or Zeromorph with no additional setup cost.

In practice, Jolt uses HyperKZG as its default PCS (with Zeromorph as an alternative). Nova's implementation also supports HyperKZG. For systems that require transparency, Dory or hash-based schemes (Basefold, FRI) are preferred.

Recent work (Mercury, Samaritan) pushes further toward $O(1)$ proof size while maintaining $O(N)$ prover time, representing the current frontier of KZG-based multilinear commitments.